Orange Pi 3B
用户手册

目录

1. Orange Pi 3B 的基本特性 10
1.1. 什么是 Orange Pi 3B 10
1.2. Orange Pi 3B 的用途 10
1.3. Orange Pi 3B 的硬件规格 11
1.4. Orange Pi 3B 的顶层视图和底层视图 13
1.4.1. Orange Pi 3B（v1.1.1）的顶层视图和底层视图 13
1.4.2. Orange Pi 3B（v2.1）的顶层视图和底层视图 14
1.5. Orange Pi 3B 的接口详情图 15
1.5.1. Orange Pi 3B（v1.1.1）的接口详情图 15
1.5.2. Orange Pi 3B（v2.1）的接口详情图 17
2. 开发板使用介绍 19
2.1. 准备需要的配件 19
2.2. 下载开发板的镜像和相关的资料 22
2.3. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法 24
2.3.1. 使用 balenaEtcher 烧录 Linux 镜像的方法 24
2.3.2. 使用 RKDevTool 烧录 Linux 镜像到 TF 卡中的方法 27
2.3.3. 使用 Win32Diskimager 烧录 Linux 镜像的方法 35
2.4. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法 37
2.5. 烧录 Linux 镜像到 eMMC 中的方法 41
2.5.1. 使用 RKDevTool 烧录 Linux 镜像到 eMMC 中的方法 41
2.5.2. 使用 dd 命令烧录 Linux 镜像到 eMMC 中的方法 49
2.6. 烧写 Linux 镜像到 SPIFlash+NVMe SSD 中的方法 51
2.6.1. 使用 RKDevTool 烧录的方法 51
2.6.2. 使用 dd 命令烧录的方法 60
2.6.3. 使用 balenaEtcher 软件烧录的方法 65
2.7. 烧录 Android 镜像到 TF 卡中的方法 80
2.7.1. 通过 USB2.0 烧录口将 Android 镜像烧录到 TF 卡中的方法 80

2.7.2. 使用 SDDiskTool 工具将 Android 镜像烧到 TF 卡的方法 87
2.8. 烧录 Android 镜像到 eMMC 中的方法 90
2.8.1. 通过 USB2.0 烧录口将 Android 镜像烧录到 eMMC 中的方法 90
2.8.2. 通过 TF 卡将 Android11 镜像烧录到 eMMC 中的方法 97
2.9. 烧录 Android 镜像到 SPIFlash+NVMe SSD 中的方法 102
2.10. 烧录 Orange Pi OS（OH）镜像到 TF 卡中的方法 107
2.11. 烧录 Orange Pi OS（OH）镜像到 eMMC 中的方法 114
2.12. 使用 RKDevTool 清空 SPIFlash 的方法 121
2.13. 启动香橙派开发板 128
2.14. 调试串口的使用方法 129
2.14.1. 调试串口的连接说明 129
2.14.2. Ubuntu 平台调试串口的使用方法 130
2.14.3. Windows 平台调试串口的使用方法 133
2.15. 使用开发板 40pin 接口中的 5v 引脚供电说明 137
3. Ubuntu/Debian Server 和 Xfce 桌面系统使用说明 138
3.1. 已支持的 Linux 镜像类型和内核版本 138
3.2. Linux 系统适配情况 138
3.2.1. Linux5.10 系统适配情况 138
3.2.2. Linux6.6 系统适配情况 139
3.3. 本手册 Linux 命令格式说明 140
3.4. Linux 系统登录说明 142
3.4.1. Linux 系统默认登录账号和密码 142
3.4.2. 设置 Linux 系统终端自动登录的方法 142
3.4.3. Linux 桌面版系统自动登录说明 143
3.4.4. Linux 桌面版系统 root 用户自动登录的设置方法 144
3.4.5. Linux 桌面版系统禁用桌面的方法 145
3.5. 板载 LED 灯测试说明 145
3.6. 网络连接测试 146
3.6.1. 以太网口测试 146
3.6.2. WIFI 连接测试 148

3.6.3. 设置静态 IP 地址的方法 155
3.6.4. 通过 create_ap 创建 WIFI 热点的方法 163
3.7. SSH 远程登录开发板 170
3.7.1. Ubuntu 下 SSH 远程登录开发板 170
3.7.2. Windows 下 SSH 远程登录开发板 171
3.8. 上传文件到开发板 Linux 系统中的方法 172
3.8.1. 在 Ubuntu PC 中上传文件到开发板 Linux 系统中的方法 172
3.8.2. 在 Windows PC 中上传文件到开发板 Linux 系统中的方法 176
3.9. HDMI 测试 180
3.9.1. HDMI 显示测试 180
3.9.2. HDMI 转 VGA 显示测试 181
3.9.3. HDMI 分辨率设置方法 182
3.10. 蓝牙使用方法 185
3.10.1. 桌面版镜像的测试方法 185
3.11. USB 接口测试 188
3.11.1. 连接 USB 鼠标或键盘测试 188
3.11.2. 连接 USB 存储设备测试 189
3.11.3. USB 无线网卡测试 189
3.11.4. USB 摄像头测试 193
3.12. 音频测试 196
3.12.1. 在桌面系统中测试音频方法 196
3.12.2. 使用命令播放音频的方法 198
3.12.3. 使用命令测试录音的方法 200
3.13. 温度传感器 200
3.14. 40 Pin 接口引脚说明 200
3.15. 安装 wiringOP 的方法 202
3.16. 40pin 接口 GPIO、I2C、UART、SPI 和 PWM 测试 204
3.16.1. 40pin GPIO 口测试 204
3.16.2. 40pin GPIO 口上下拉电阻的设置方法 206
3.16.3. 40pin SPI 测试 207
3.16.4. 40pin I2C 测试 210

3.16.5. 40pin 的 UART 测试 213
3.16.6. 使用/sys/class/pwm 测试 PWM 的方法 216
3.17. wiringOP 硬件 PWM 的使用方法 220
3.17.1. 使用 wiringOP 的 gpio 命令设置 PWM 的方法 220
3.17.2. PWM 测试程序的使用方法 224
3.18. wiringOP-Python 的安装使用方法 226
3.18.1. wiringOP-Python 的安装方法 226
3.18.2. 40pin GPIO 口测试 228
3.18.3. 40pin SPI 测试 230
3.18.4. 40pin I2C 测试 233
3.18.5. 40pin 的 UART 测试 236
3.19. 硬件看门狗测试 240
3.20. 查看 RK3566 芯片的序列号 240
3.21. 下载安装 arm64 版本 balenaEtcher 的方法 240
3.22. 使用 NPU 的方法 242
3.22.1. 准备工具 242
3.22.2. 在 Ubuntu PC 端安装 RKNN-Toolkit2 243
3.22.3. 使用 RKNN-Toolkit2 进行模型转换和模型推理 244
3.22.4. 调用 C 接口部署 RKNN 模型到开发板上运行 249
3.23. 宝塔 Linux 面板的安装方法 252
3.24. 设置中文环境以及安装中文输入法 255
3.24.1. Debian 系统的安装方法 255
3.24.2. Ubuntu 20.04 系统的安装方法 262
3.24.3. Ubuntu 22.04 系统的安装方法 266
3.25. 远程登录 Linux 系统桌面的方法 272
3.25.1. 使用 NoMachine 远程登录 272
3.25.2. 使用 VNC 远程登录 276
3.26. Linux 系统支持的部分编程语言测试 278
3.26.1. Debian Bullseye 系统 278
3.26.2. Debian Bookworm 系统 280
3.26.3. Ubuntu Focal 系统 281

3.26.4. Ubuntu Jammy 系统 283
3.27. QT 的安装方法 285
3.28. ROS 安装方法 293
3.28.1. Ubuntu20.04 安装 ROS 1 Noetic 的方法 293
3.28.2. Ubuntu20.04 安装 ROS 2 Galactic 的方法 297
3.28.3. Ubuntu22.04 安装 ROS 2 Humble 的方法 300
3.29. 安装内核头文件的方法 302
3.30. 树莓派 5 寸屏幕的使用 304
3.30.1. 树莓派 5 寸屏幕的组装方法 304
3.30.2. 打开树莓派 5 寸屏幕配置的方法 306
3.30.3. 服务器版镜像旋转显示方向的方法 309
3.30.4. 桌面版镜像旋转显示和触摸方向的方法 310
3.31. eDP 屏幕的使用方法 312
3.31.1. eDP 屏幕的组装方法 312
3.31.2. 打开 eDP 屏幕配置的方法 313
3.32. 开关机 logo 使用说明 316
3.33. ZFS 文件系统的使用方法 317
3.33.1. 安装 ZFS 的方法 317
3.33.2. 创建 ZFS 池的方法 319
3.33.3. 测试 ZFS 的数据去重功能 320
3.33.4. 测试 ZFS 的数据压缩功能 321
3.34. RTC 的使用方法 321
3.35. Linux6.6 系统 GPU 的测试方法 323
3.36. 关机和重启开发板的方法 325
4. Linux SDK——orangepi-build 使用说明 327
4.1. 编译系统需求 327
4.1.1. 使用 x64 的 Ubuntu22.04 电脑编译 327
4.2. 获取 Linux sdk 的源码 329
4.2.1. 从 github 下载 orangepi-build 329
4.2.2. 下载交叉编译工具链 330

4.2.3. orangepi-build 完整目录结构说明 332
4.3. 编译 u-boot 333
4.4. 编译 Linux 内核 336
4.5. 编译 rootfs 341
4.6. 编译 Linux 镜像 344
5. Orange Pi OS Arch 系统的使用说明 349
5.1. Orange Pi OS Arch 系统功能适配情况 349
5.2. Orange Pi OS Arch 系统用户向导使用说明 350
5.3. 设置 DT overlays 的方法 355
5.4. 树莓派 5 寸屏幕的使用 357
5.4.1. 树莓派 5 寸屏幕的组装方法 357
5.4.2. 打开树莓派 5 寸屏幕配置的方法 357
5.5. eDP 屏幕的使用方法 358
5.5.1. eDP 屏幕的组装方法 358
5.5.2. 打开 eDP 屏幕配置的方法 358
5.6. 安装软件的方法 359
6. Orange Pi OS OH 系统的使用说明 360
6.1. Orange Pi OS OH 系统功能适配情况 360
6.2. 千兆网口的使用方法 361
6.3. WIFI 的使用方法 363
7. Android 11 系统的使用说明 365
7.1. 已支持的 Android 版本 365
7.2. Android 功能适配情况 365
7.3. WIFI 的连接测试方法 366
7.4. Wi-Fi hotspot 的使用方法 368
7.5. 蓝牙的测试方法 370
7.6. 树莓派 5 寸屏幕的使用方法 374

7.7. eDP 屏幕的使用方法 375
7.8. 40pin 接口 GPIO、UART、SPI 和 PWM 测试 377
7.8.1. 40pin GPIO 口测试 377
7.8.2. 40pin 的 UART 测试 380
7.8.3. 40pin 的 SPI 测试 383
7.8.4. 40pin 的 PWM 测试 386
7.9. ADB 的使用方法 388
7.9.1. USB OTG 模式切换的方法 388
7.9.2. 使用数据线连接 adb 调试 392
7.9.3. 使用网络连接 adb 调试 393
8. OpenWRT 系统的使用说明 395
8.1. OpenWRT 版本 395
8.2. OpenWRT 适配情况 395
8.3. 第一次启动扩容 rootfs 395
8.4. 登录系统的方法 396
8.4.1. 通过串口登录 396
8.4.2. 通过 SSH 登录系统 396
8.4.3. 登录 LuCI 管理界面 397
8.4.4. 通过 LuCI 管理界面登录终端 398
8.4.5. 使用 IP 地址+端口号方式登录终端 400
8.5. 通过命令行修改 LAN 口 IP 地址的方法 401
8.6. 修改 root 密码的方法 402
8.6.1. 通过命令行修改 402
8.6.2. 通过 LuCI 管理界面修改 403
8.7. USB 接口测试 405
8.7.1. 在命令行下挂载 USB 存储设备 405
8.7.2. 在 LuCI 管理界面挂载 USB 存储设备 405
8.8. USB 无线网卡测试 408
8.8.1. 使用 USB 无线网卡创建 WIFI 热点的方法 409
8.8.2. 使用 USB 无线网卡连接 WIFI 热点的方法 413
8.9. 通过命令行安装软件包 416

8.9.1. 在终端通过 opkg 安装 416
8.10. OpenWRT 管理界面安装软件包 416
8.10.1. 查看系统可用软件包列表 417
8.10.2. 安装软件包示例 418
8.10.3. 移除软件包示例 420
8.11. 使用 Samba 网络共享 422
8.12. zerotier 使用说明 425
9. Android11 源码的编译方法 430
9.1. 下载 Android 11 的源码 430
9.2. 编译 Android 11 的源码 431
10. OpenWRT 源码的编译方法 433
10.1. 下载 OpenWRT 源码 433
10.2. 编译 OpenWRT 源码 433
11. 附录 435
11.1. 用户手册更新历史 435
11.2. 镜像更新历史 435

1. Orange Pi 3B 的基本特性

1.1. 什么是 Orange Pi 3B

Orange Pi 3B 采用了瑞芯微 RK3566 四核 64 位 Cortex-A55 处理器，采用的 22nm 工艺，主频最高可达 1.8GHz，集成 ARM Mali-G52 GPU，内嵌高性能 2D 图像加速模块，内置 0.8Tops 算力的 AI 加速器 NPU，可选 2GB、4GB 或者 8GB 内存，具有高达 4K 显示处理能力。

Orange Pi 3B 引出了相当丰富的接口，包括 HDMI 输出、M.2 PCIe2.0x1、千兆网口、USB2.0、USB3.0 接口和 40pin 扩展排针等。可广泛适用于高端平板、边缘计算、人工智能、云计算、AR/VR、智能安防、智能家居等领域，覆盖 AIoT 各个行业。

Orange Pi 3B 支持 Android11、Ubuntu22.04、Ubuntu20.04、Debian11、Debian12、开源鸿蒙 4.0 Beta1、Orange Pi OS（Arch）、基于开源鸿蒙的 Orange Pi OS（OH）等操作系统。

1.2. Orange Pi 3B 的用途

我们可以用它实现：

- 一台 Linux 桌面计算机
- 一台 Linux 网络服务器
- Android 平板
- Android 游戏机等

当然还有其他更多的功能，依托强大的生态系统以及各式各样的扩展配件，Orange Pi 可以帮助用户轻松实现从创意到原型再到批量生产的交付，是创客、梦想家、业余爱好者的理想创意平台。

1.3. Orange Pi 3B 的硬件规格

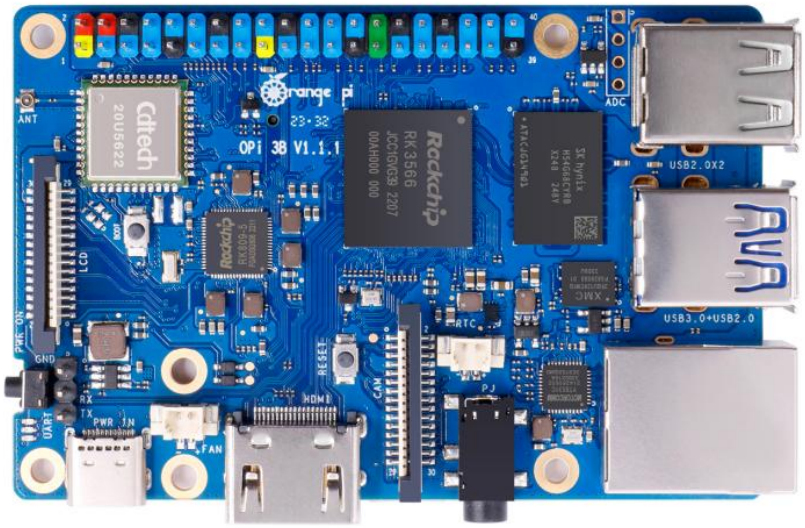
硬件规格参数	
主控芯片	Rockchip RK3566
CPU	四核 64 位 Cortex-A55 处理器，22nm 先进工艺，主频最高 1.8GHz
GPU	• ARM Mali G52 2EE 图形处理器 • 支持 OpenGL ES 1.1/2.0/3.2，OpenCL 2.0，Vulkan 1.1 • 内嵌高性能 2D 加速硬件
NPU	• 集成 RKNN NPU AI 加速器，0.8Tops@INT8 性能 • 支持 Caffe/TensorFlow/TFLite/ONNX/PyTorch/Keras/Darknet 架构模型一键转换
VPU	• 4K@60fps H.265/H.264/VP9 视频解码 • 1080P@60fps H.265/H.264 视频编码
PMU	Rockchip RK809-5
内存	2GB/4GB/8GB （LPDDR4/4x）
存储	• 支持 eMMC 模块:16GB/32GB/64GB/128GB/256GB • SPI Flash: 16MB/32MB • M.2 M-KEY 插槽（可选）：SATA3 或者 PCIe2.0 NVME SSD • TF 卡槽
Wi-Fi+蓝牙	Wi-Fi 5+BT 5.0，BLE
以太网收发器	10/100/1000Mbps 以太网（板载 PHY 芯片：YT8531C）
显示	• 1x HDMI TX 2.0，最大支持 4K@60FPS • 1xMIPI DSI 2 Lane • eDP1.3
摄像头	1xMIPI CSI 2 Lane 摄像头接口
USB	• 1xUSB 2.0 支持 Device 或 HOST 模式 • 1xUSB 3.0 HOST

	• 2xUSB 2.0 HOST
音频	3.5mm 耳机孔音频输入/输出
按键	1x MaskROM 键，1xRESET 键，1 xPOWER 键
FAN	2Pin 1.25mm 规格的 5V 风扇接口
RTC	2Pin 1.25mm 规格的备用电池接口
40Pin	40Pin 功能扩展接口，支持以下接口类型: GPIO、UART、I2C、SPI、PWM
电源	Type-C 5V3A
支持的操作系统	Android11、Ubuntu22.04、Ubuntu20.04、Debian11、Debian12、开源鸿蒙 4.0 Beta1、Orange Pi OS（Arch）、基于开源鸿蒙的 Orange Pi OS（OH）等操作系统。
外观规格介绍	
PCB 尺寸	89mm x 56mm x 17mm
重量	52g
rangePi™是深圳市迅龙软件有限公司的注册商标	

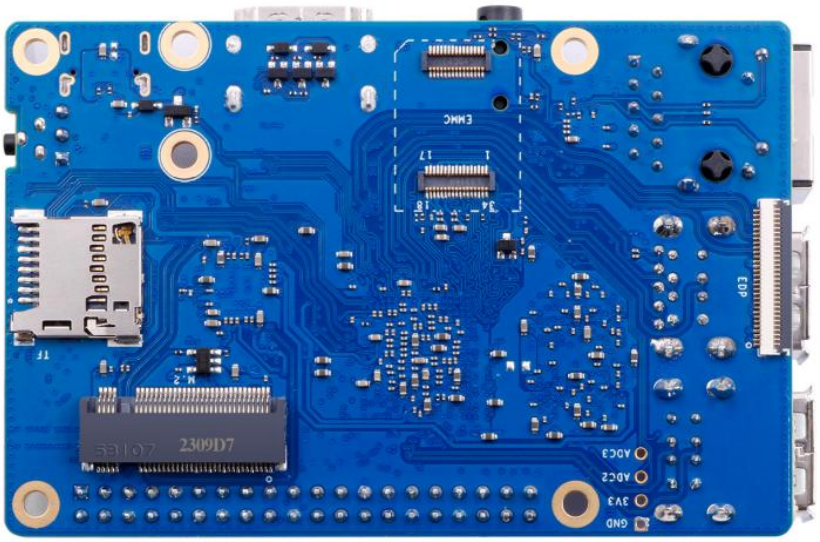
1. 4. Orange Pi 3B 的顶层视图和底层视图

1. 4. 1. Orange Pi 3B（v1.1.1）的顶层视图和底层视图

顶层视图：

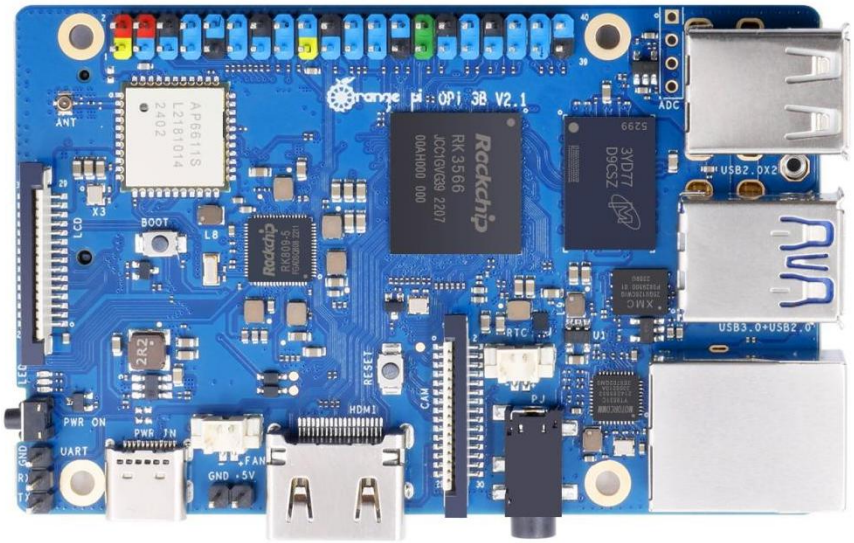


底层视图：

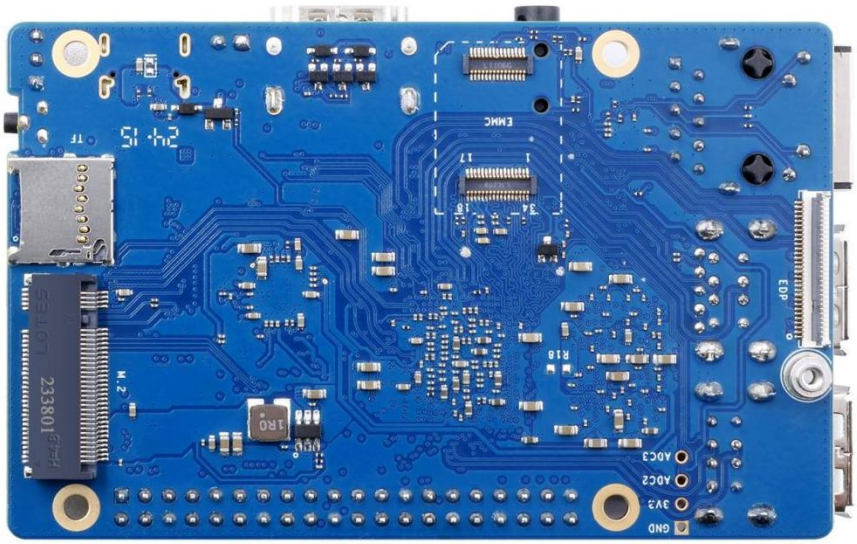


1.4.2. Orange Pi 3B（v2.1）的顶层视图和底层视图

顶层视图：

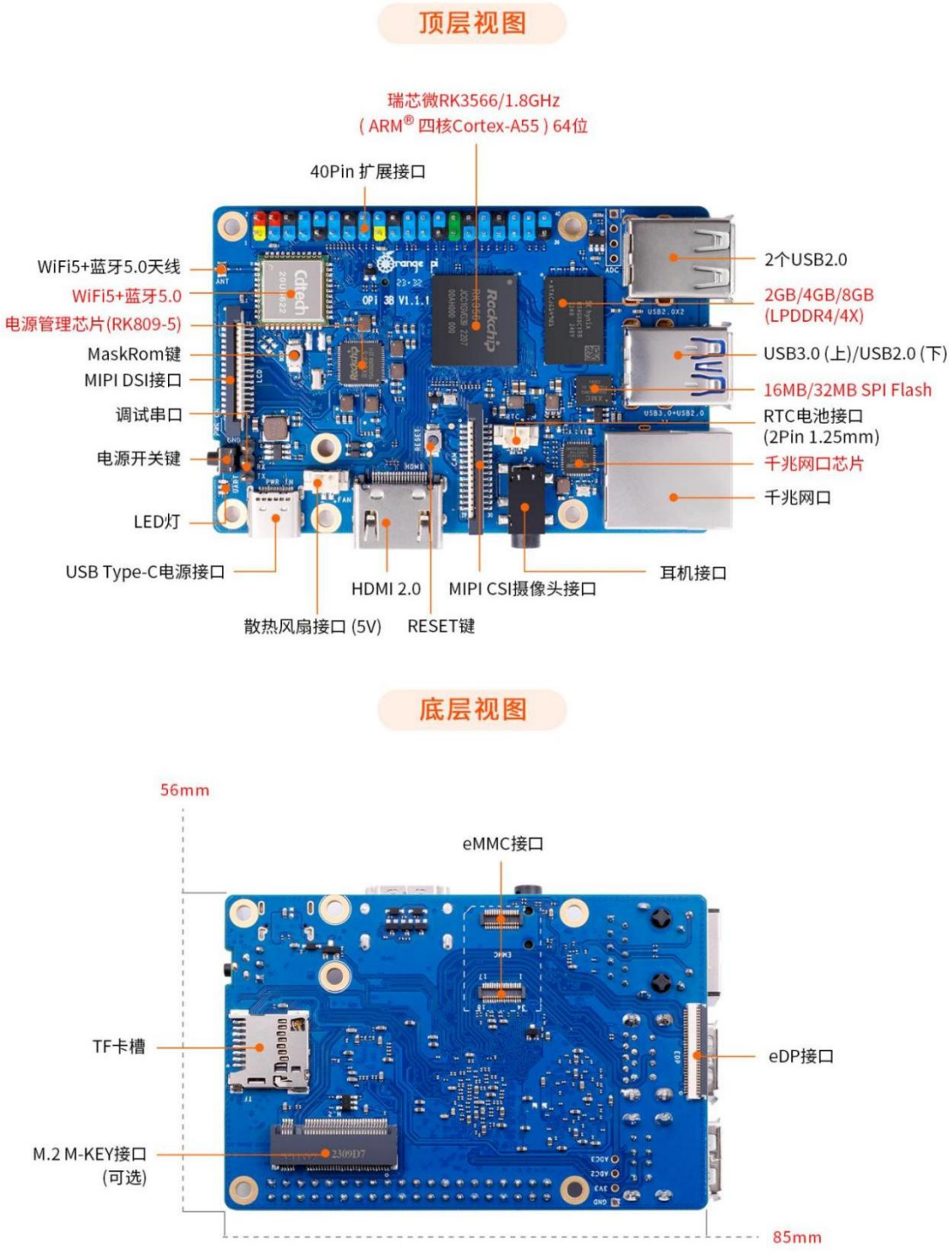


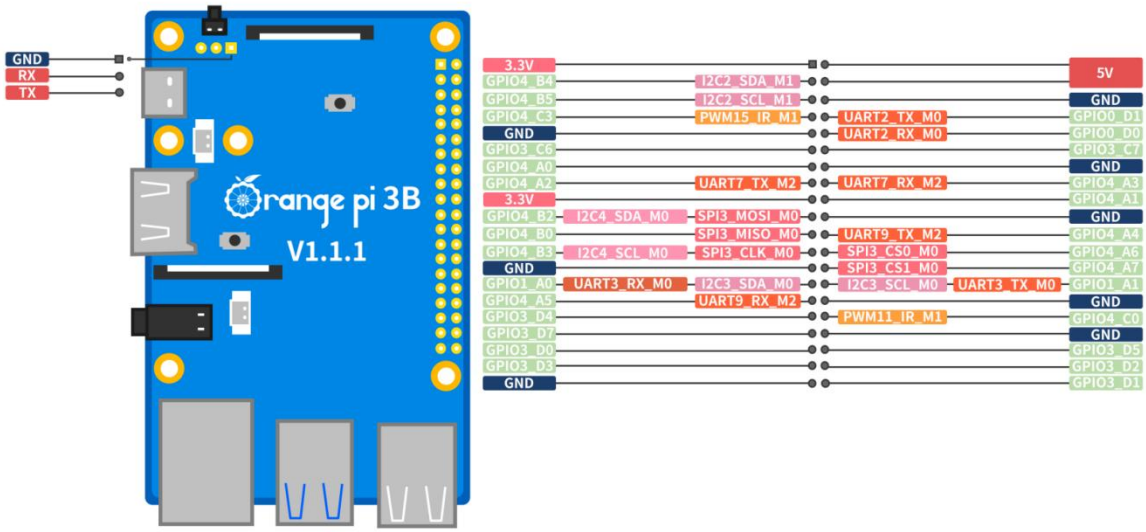
底层视图：



1.5. Orange Pi 3B 的接口详情图

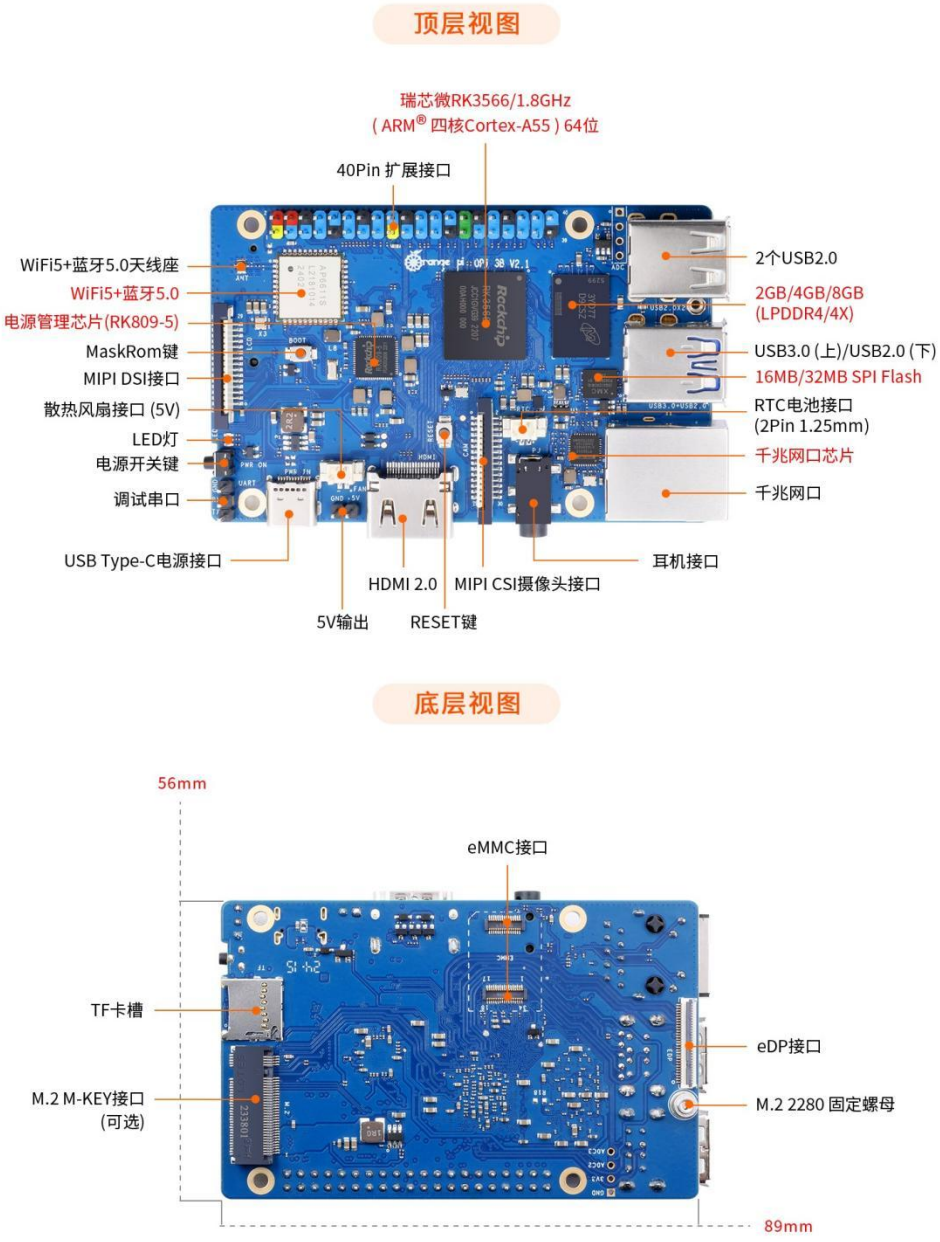
1.5.1. Orange Pi 3B（v1.1.1）的接口详情图

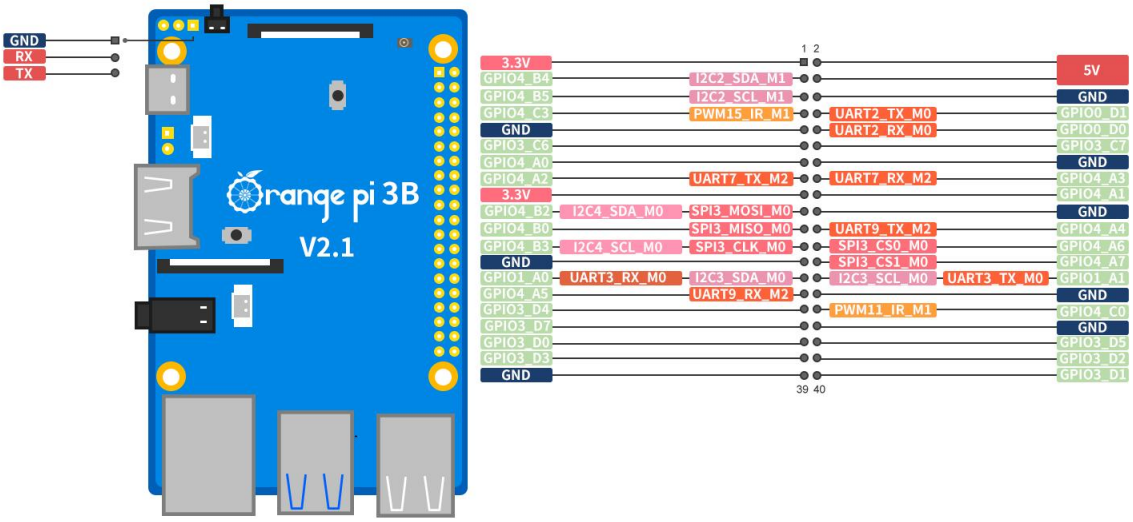




四个定位孔的直径都是 2.7mm，M.2 PICE设备固定孔直径为 2.9mm。

1.5.2. Orange Pi 3B（v2.1）的接口详情图





四个定位孔的直径都是 2.7mm。

2. 开发板使用介绍

2.1. 准备需要的配件

1) TF 卡，最小 16GB 容量（推荐 32GB 或以上）的 **class10** 级或以上的高速闪迪卡

2) TF 卡读卡器，用于将镜像烧录到 TF 卡中

3) HDMI 接口的显示器

4) HDMI 转 HDMI 连接线，用于将开发板连接到 HDMI 显示器或者电视进行显示

注意，如果想接 4K显示器，请确保HDMI线支持 4K视频输出。

5) 电源适配器，Orange Pi 3B 建议使用 5V/3A 的 Type-C 电源供电

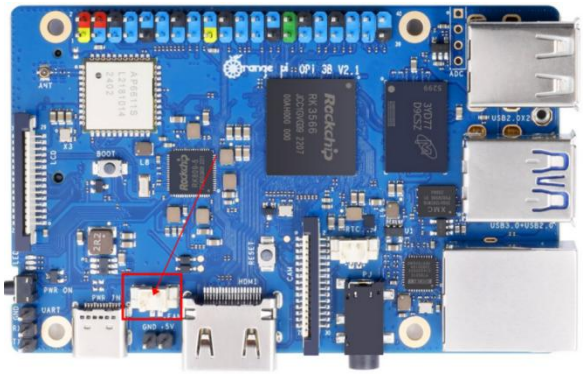
6) USB接口的鼠标和键盘，只要是标准USB接口的鼠标和键盘都可以，鼠标和键盘可以用来控制Orange Pi开发板

7) USB摄像头

8) PCIe NVMe SSD。Orange Pi 3B v1.1.1 开发板支持M.2 2230 和M.2 2242 规格的SSD，M.2 2280 也是可以支持的，只是没法用螺丝固定。Orange Pi 3B v2.1 开发板支持固定M.2 2280 的SSD。

9) 5V的散热风扇。如下图所示，开发板上有用于接散热风扇的接口，接口规格为**2pin 1.25mm间距**

开发板上的风扇可以通过PWM来调节转速和开关。



10) 百兆或者千兆网线，用于将开发板连接到因特网

11) USB2.0 公对公数据线，用于烧录镜像到eMMC、NVMe SSD等功能

12) **3.3V** 的 USB 转 TTL 模块和杜邦线，使用串口调试功能时，需要 USB 转 TTL 模块和杜邦线来连接开发板和电脑

13) 安装有 Ubuntu 和 Windows 操作系统的个人电脑

1	Ubuntu22.04 PC	可选，用于编译 Linux 源码
2	Windows PC	用于烧录 Android 和 Linux 镜像

2.2. 下载开发板的镜像和相关的资料

1) 中文版资料的下载网址为：

http://www.orangepi.cn/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-3B.html

2) 英文版资料的下载网址为：

http://www.orangepi.org/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-3B.html

3) 资料主要包含

a. **Android 源码**：保存在百度云盘和谷歌网盘上
b. **Linux 源码**：保存在 Github 上
c. **用户手册和原理图**：保存在百度云盘和谷歌网盘上
d. **官方工具**：主要包括开发板使用过程中需要用到的软件
e. **Android 镜像**：保存在百度云盘和谷歌网盘上
f. **Ubuntu 镜像**：保存在百度云盘和谷歌网盘上
g. **Debian 镜像**：保存在百度云盘和谷歌网盘上
h. **Orange Pi OS Arch 镜像**：保存在百度云盘和谷歌网盘上
i. **Orange Pi OS OH 镜像**：保存在百度云盘和谷歌网盘上
j. **OpenWRT 镜像**：保存在百度云盘和谷歌网盘上

2.3. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法

注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian、Ubuntu、OpenWRT或者OPi OS Arch这样的Linux发行版镜像。

2.3.1. 使用 balenaEtcher 烧录 Linux 镜像的方法

1) 首先准备一张 16GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

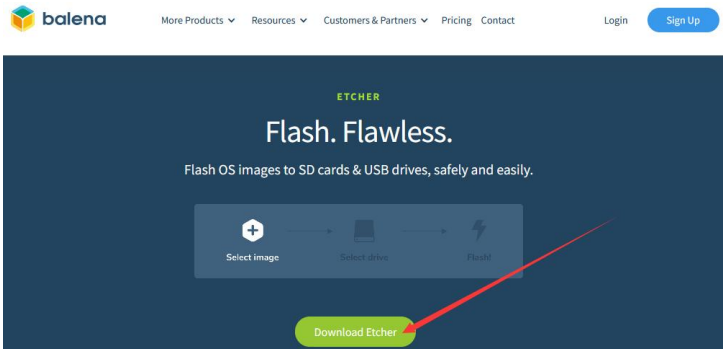
2) 然后使用读卡器把 TF 卡插入电脑

3) 从 **Orange Pi 的资料下载页面**下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 2GB 以上。

4) 然后下载 Linux 镜像的烧录软件——**balenaEtcher**，下载地址为：

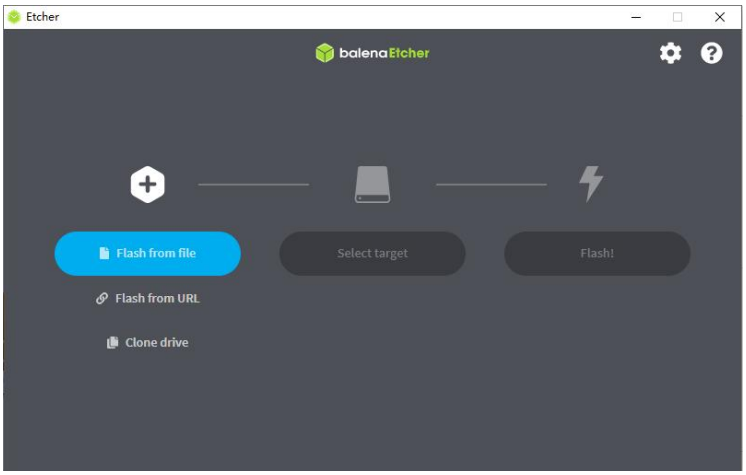
https://www.balena.io/etcher/

5) 进入 balenaEtcher 下载页面后，点击绿色的下载按钮会跳到软件下载的地方

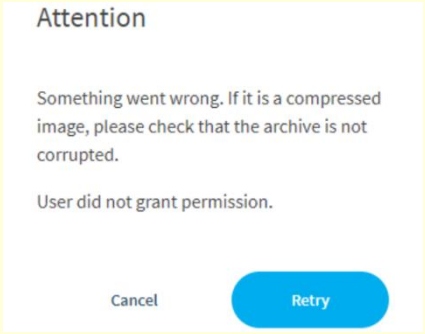


6) 然后可以选择下载 balenaEtcher 的 Portable 版本的软件，Portable 版本无需安装，双击打开就可以使用

7) 如果下载的是需要安装版本的 balenaEtcher，请先安装再使用。如果下载的 Portable 版本 balenaEtcher，直接双击打开即可，打开后的 balenaEtcher 界面如下图所示：



打开 balenaEtcher 时如果提示下面的错误：



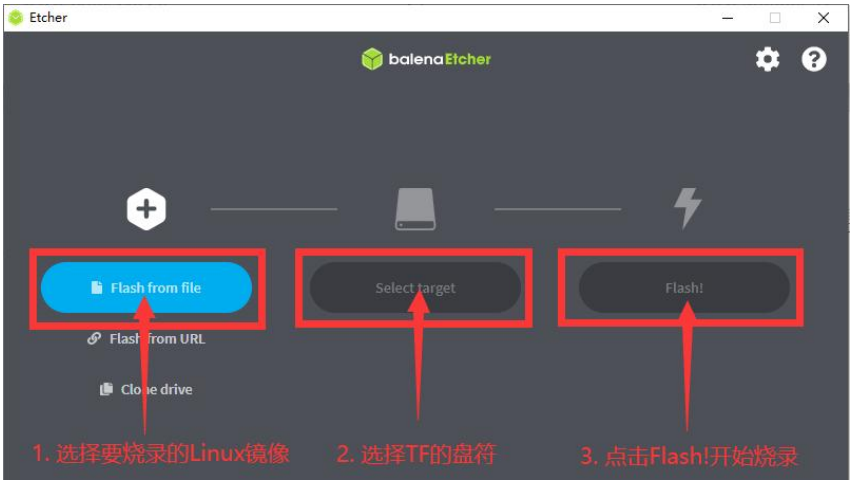
请选择 balenaEtcher 后点击右键，然后选择以管理员身份运行。

8) 使用 balenaEtcher 烧录 Linux 镜像的具体步骤如下所示

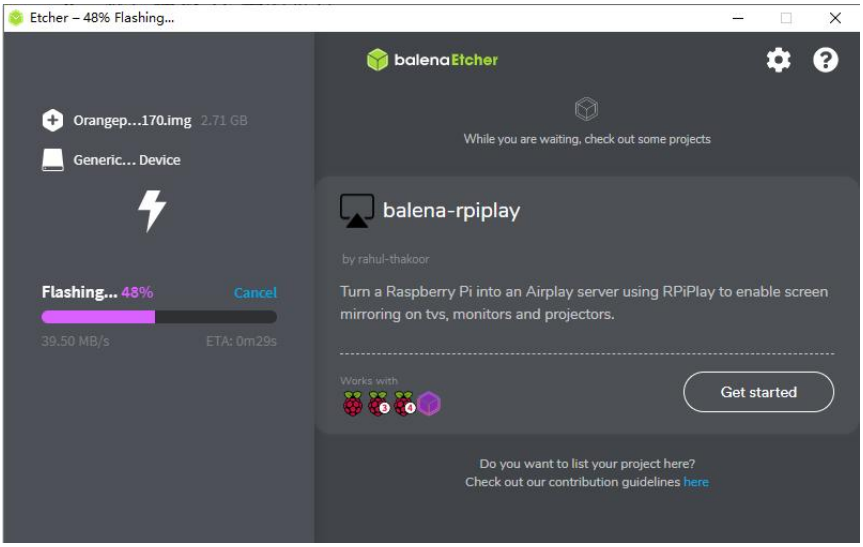
a. 首先选择要烧录的 Linux 镜像文件的路径

b. 然后选择 TF 卡的盘符

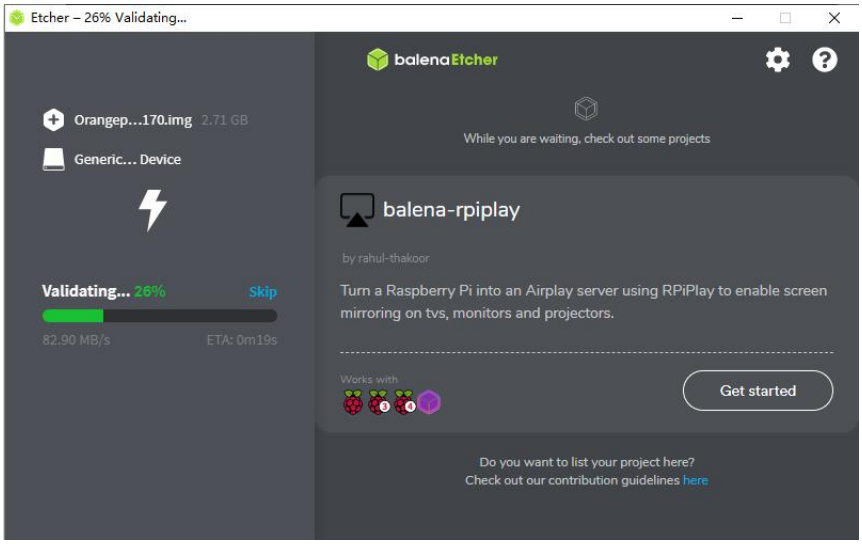
c. 最后点击 Flash 就会开始烧录 Linux 镜像到 TF 卡中



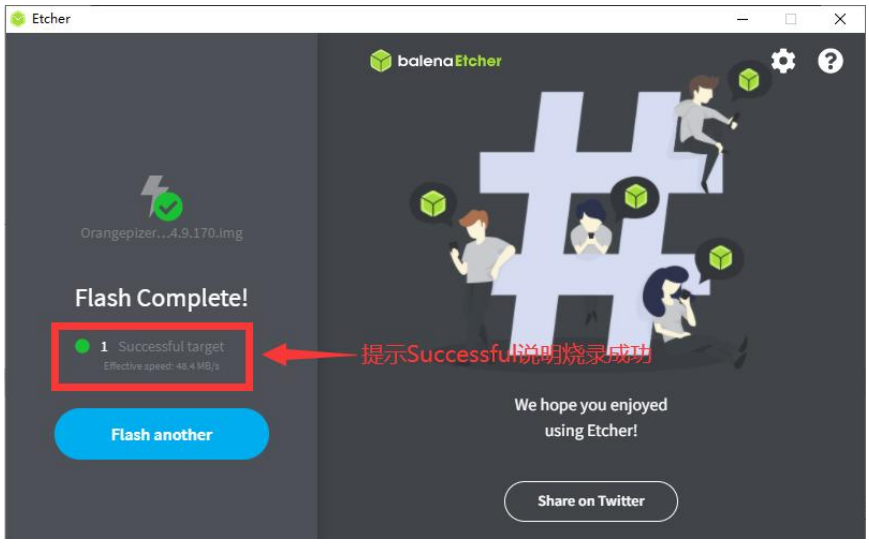
9) balenaEtcher 烧录 Linux 镜像的过程显示的界面如下图所示，另外进度条显示紫色表示正在烧录 Linux 镜像到 TF 卡中



10) Linux 镜像烧录完后，balenaEtcher 默认还会对烧录到 TF 卡中的镜像进行校验，确保烧录过程没有出问题。如下图所示，显示绿色的进度条就表示镜像已经烧录完成，balenaEtcher 正在对烧录完成的镜像进行校验



11) 成功烧录完成后 balenaEtcher 的显示界面如下图所示，如果显示绿色的指示图标说明镜像烧录成功，此时就可以退出 balenaEtcher，然后拔出 TF 卡插入到开发板的 TF 卡槽中使用了



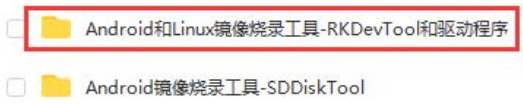
2.3.2. 使用 RKDevTool 烧录 Linux 镜像到 TF 卡中的方法

1) 首先需要准备一根品质良好的 USB2.0 公对公数据线

2) 还需要准备一张 16GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

3) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和 **MiniLoader** 以及烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面**首先选择**官方工具**，然后进入下面的文件夹中



b. 然后下载下面的所有文件

注意，“MiniLoader-烧录Linux镜像才需要用到的东西”文件夹下文简称为MiniLoader文件夹。

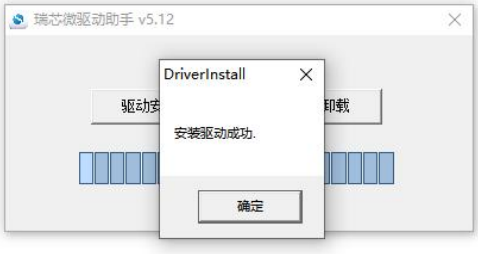
4) 然后从 **Orange Pi 的资料下载页面**下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 2GB 以上

5) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

6) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

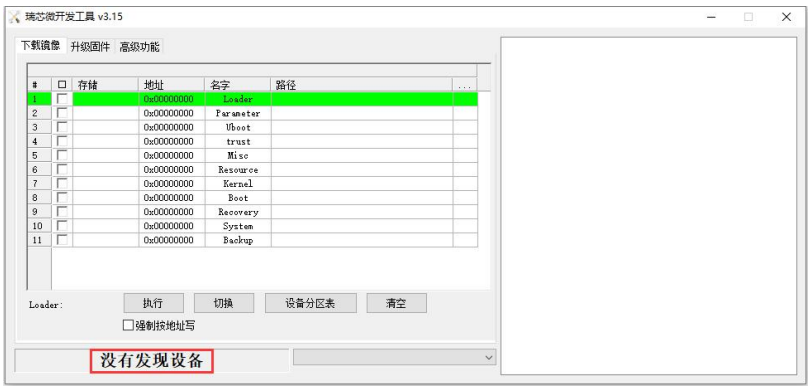
a. 点击“**驱动安装**”按钮

b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



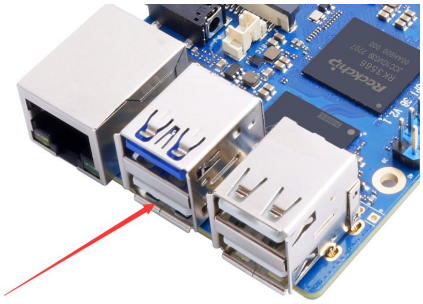
7) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

8) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



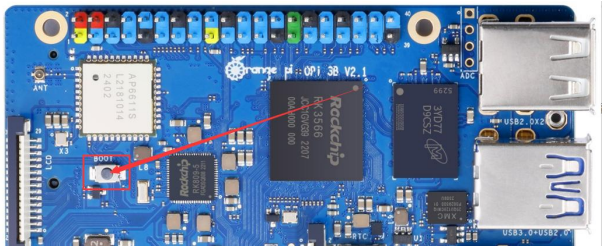
9) 然后开始烧录 Linux 镜像到 TF 卡中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录口的位置如下图所示

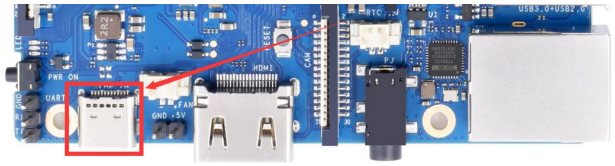


b. 确保 TF 卡插槽没有插入 TF 卡

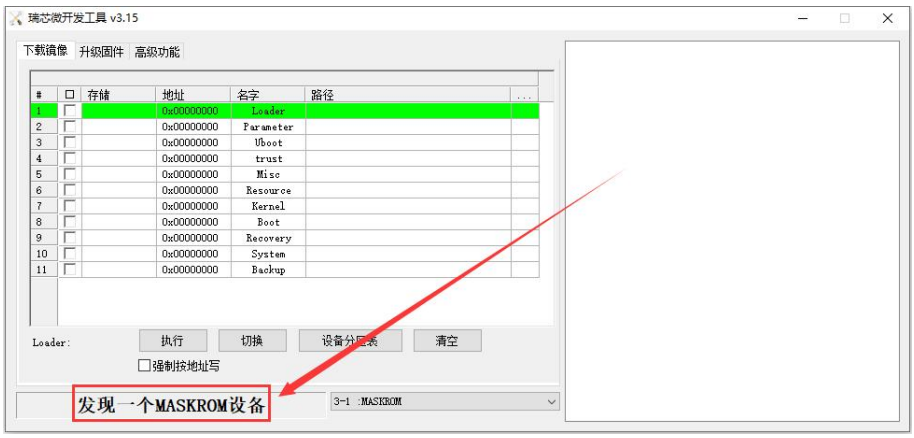
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



d. 然后给开发板接上 Type-C 接口的电源，并上电，然后就可以松开 MaskROM 按键了

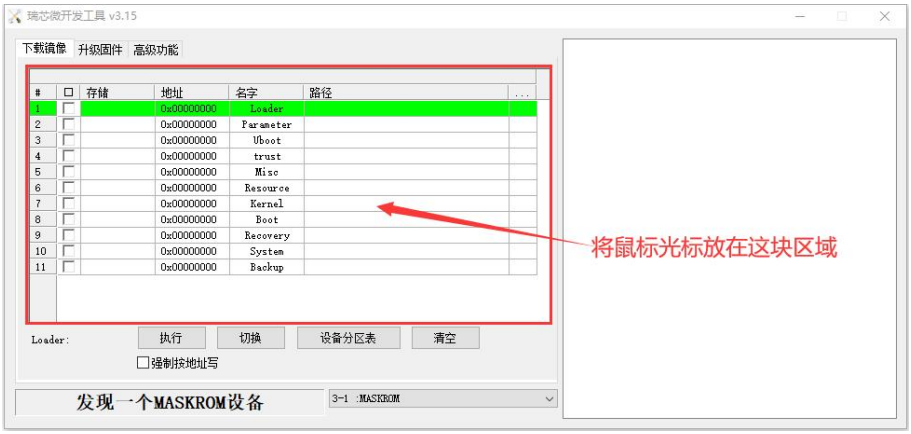


e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”

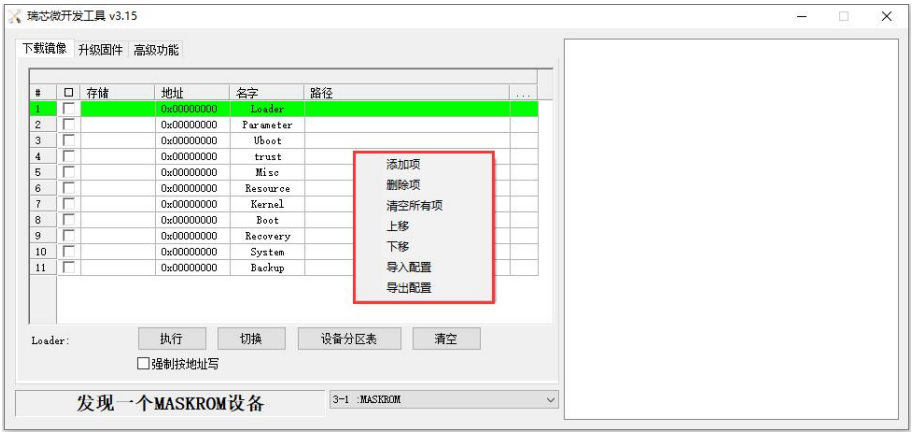


f. 此时请在 TF 卡插槽中插入 TF 卡

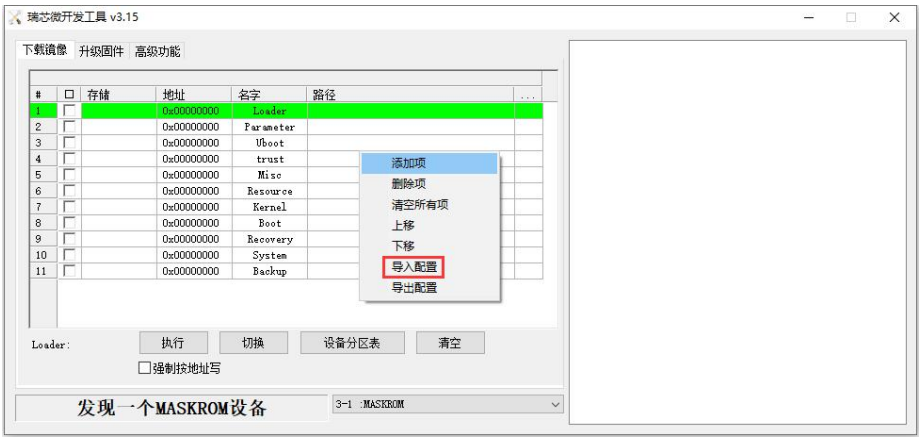
g. 然后将鼠标光标放在下面的这片区域中



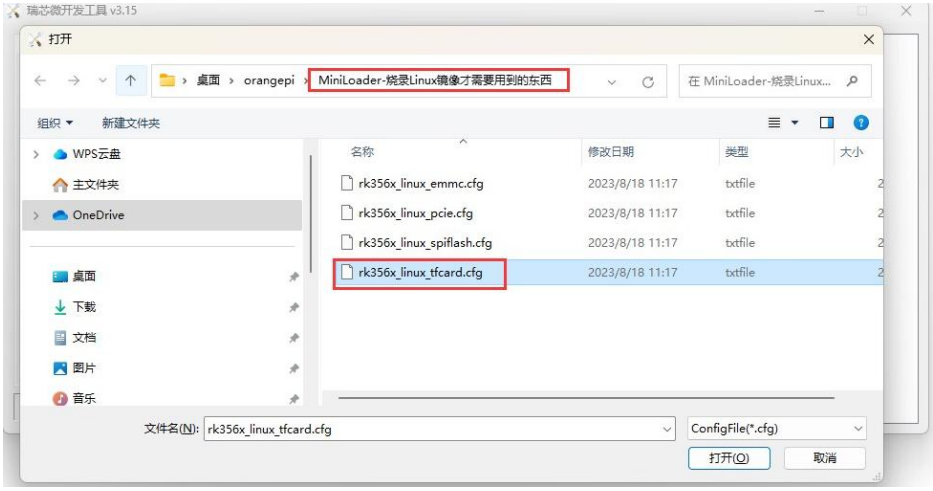
h. 然后点击鼠标右键会弹出下图所示的选择界面



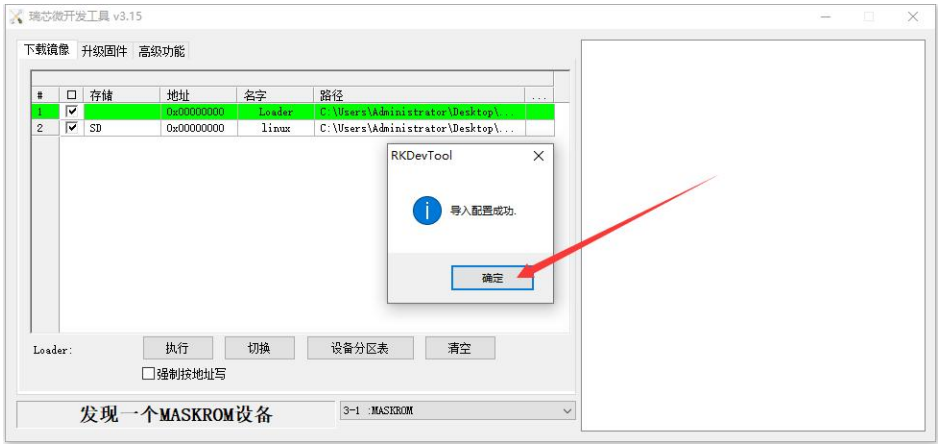
i. 然后选择**导入配置**选项



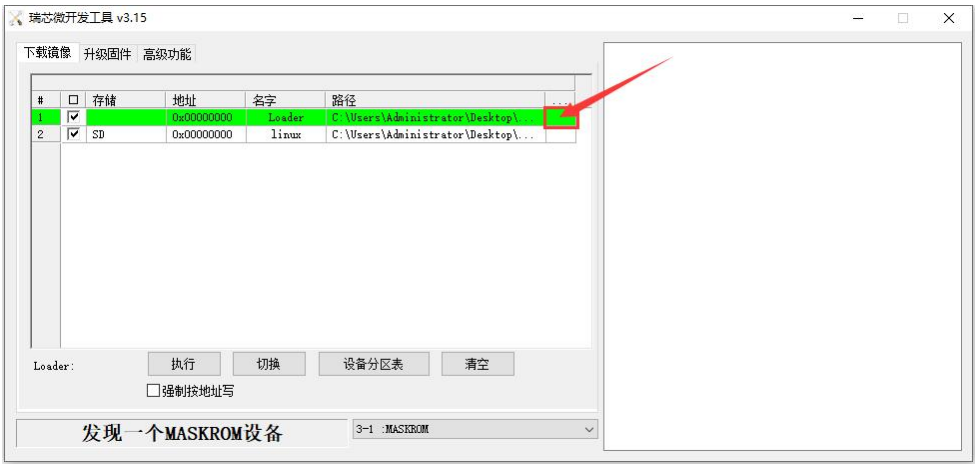
j. 然后选择前面下载的 **MiniLoader** 文件夹中的 **rk356x_linux_tfcard.cfg** 配置文件，再点击**打开**



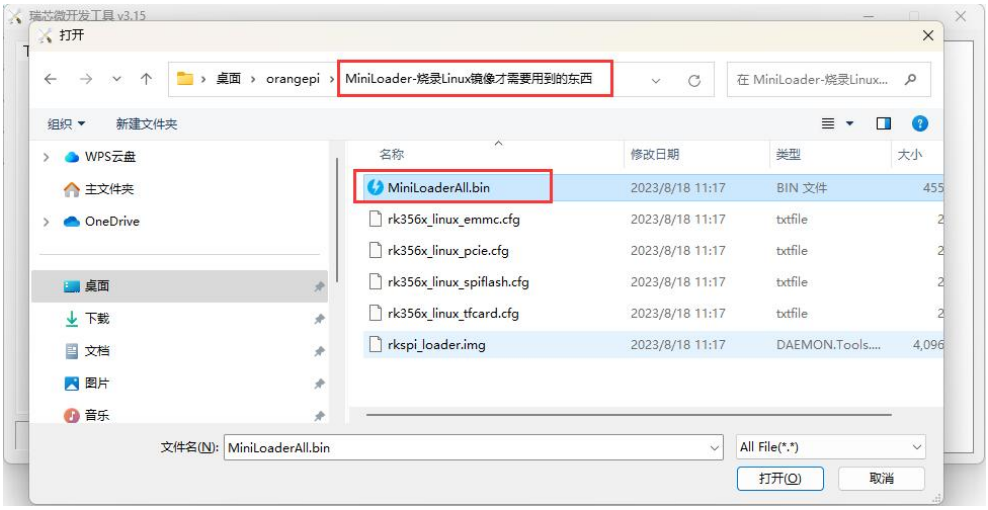
k. 然后点击**确定**



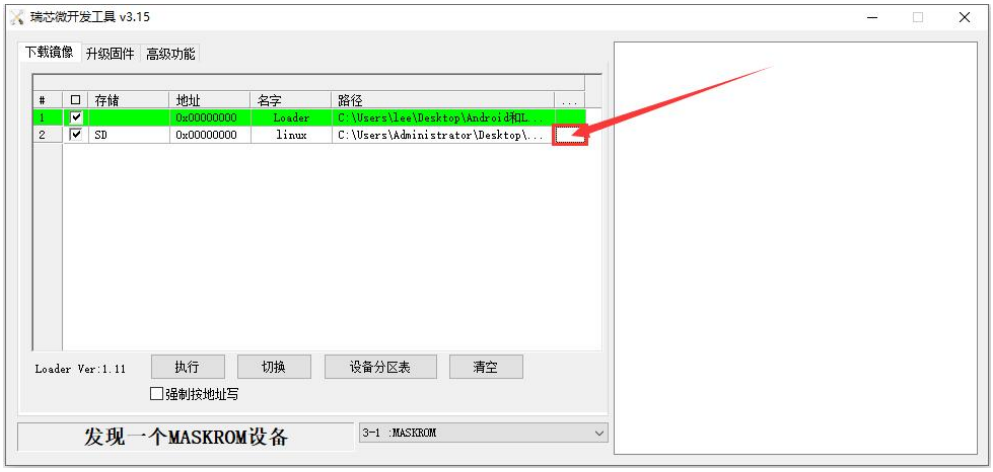
l. 然后点击下图所示的位置



m. 再选择前面下载的 **MiniLoader** 文件夹中 **MiniLoaderAll.bin**，再点击**打开**



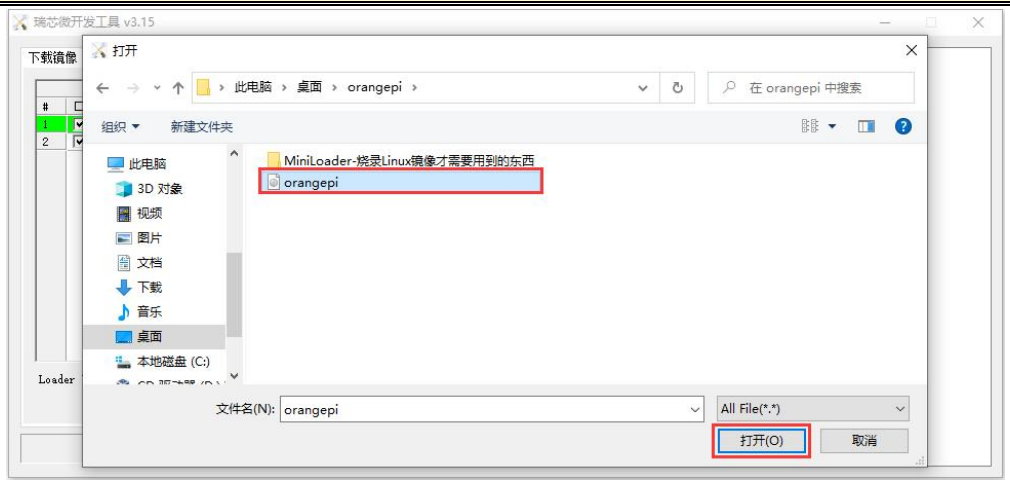
n. 然后点击下图所示的位置



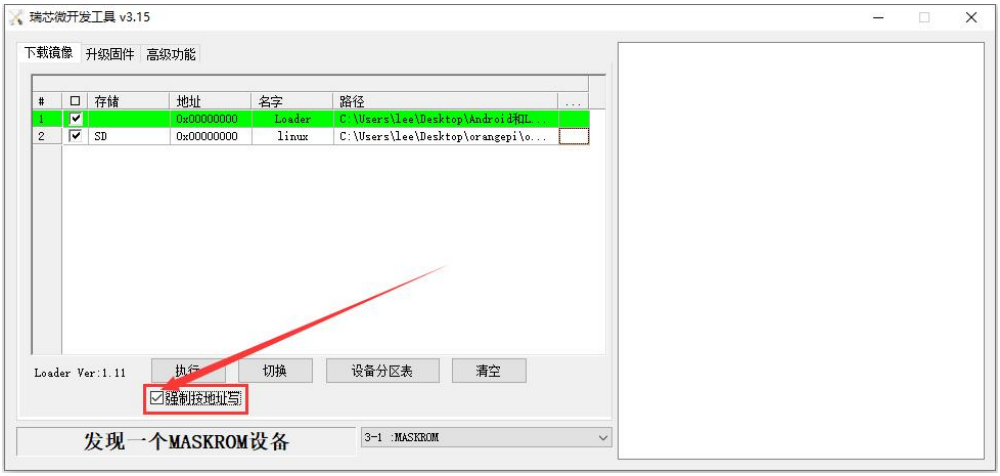
o. 然后选择想要烧录的 Linux 镜像的路径，再点击**打开**

在烧录镜像前，建议将要烧录的Linux镜像重命名为orangepi.img或者其它比较

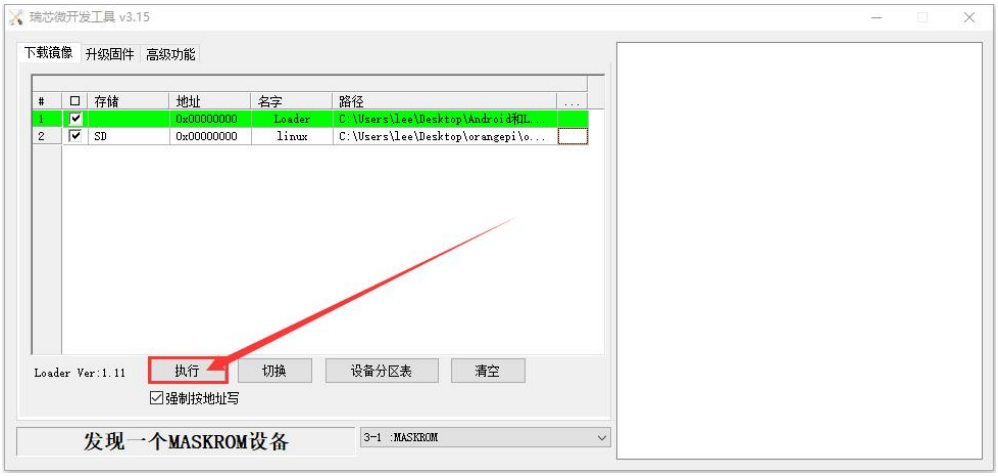
短的名字，这样在烧录镜像的时候就能看到烧录进度的百分比数值。



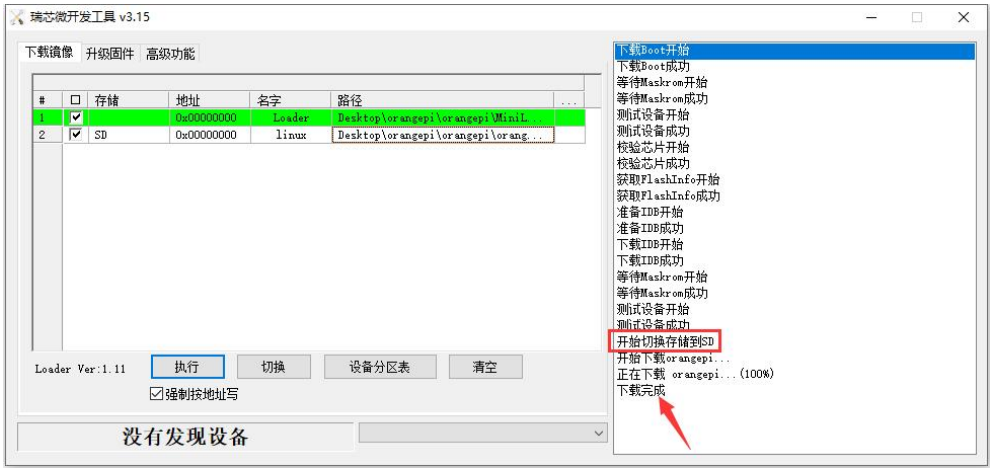
p. 然后请勾选上**强制按地址写**选项



q. 再点击执行按钮就会开始烧录 Linux 镜像到开发板的 tf 卡中



r. Linux 镜像烧录完后的显示 log 如下图所示



s. 烧录完 Linux 镜像到 tf 卡中后，Linux 系统会自动启动。

2.3.3. 使用 Win32Diskimager 烧录 Linux 镜像的方法

1) 首先准备一张 16GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

2) 然后使用读卡器把 TF 卡插入电脑

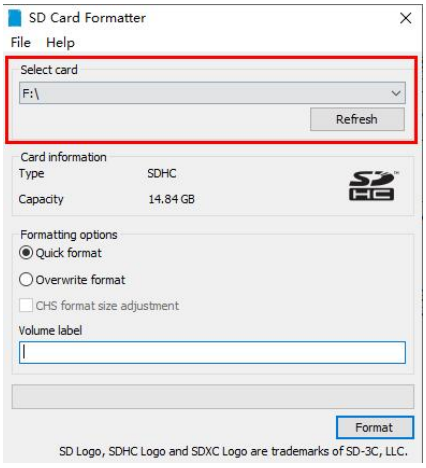
3) 接着格式化 TF 卡

a. 可以使用 **SD Card Formatter** 这个软件格式化 TF 卡，其下载地址为

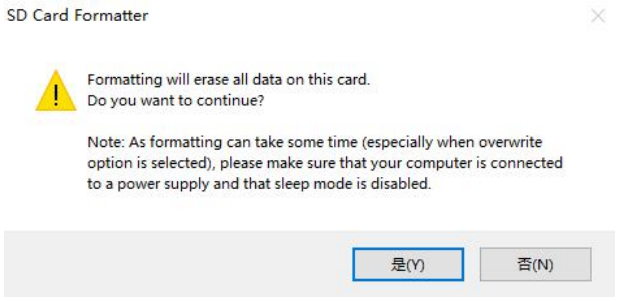
https://www.sdcard.org/downloads/formatter/eula_windows/SDCardFormatterv5_WinEN.zip

b. 下载完后直接解压安装即可，然后打开软件

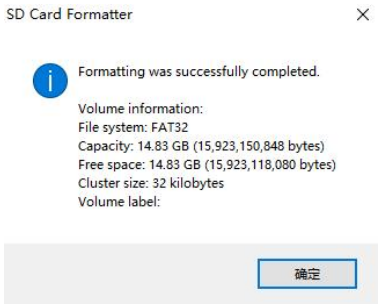
c. 如果电脑只插入了 TF 卡，则“**Select card**”一栏中会显示 TF 卡的盘符，如果电脑插入了多个 USB 存储设备，可以通过下拉框选择 TF 卡对应的盘符



d. 然后点击“**Format**”，格式化前会弹出一个警告框，选择“**是(Y)**”后就会开始格式化



e. 格式化完 TF 卡后会弹出下图所示的信息，点击确定即可



4) 从**Orange Pi的资料下载页面**下载想要烧录的Linux操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 2GB以上

5) 使用 **Win32Diskimager** 烧录 Linux 镜像到 TF 卡

a. Win32Diskimager 的下载页面为

http://sourceforge.net/projects/win32diskimager/files/Archive/

b. 下载完后直接安装即可，Win32Diskimager 界面如下所示

a) 首先选择镜像文件的路径

b) 然后确认下 TF 卡的盘符和“**设备**”一栏中显示的一致

c) 最后点击“**写入**”即可开始烧录

c. 镜像写入完成后，点击“**退出**”按钮退出即可，然后就可以拔出 TF 卡插到开发板中启动

2.4. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法

注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian、Ubuntu、OpenWRT或者OPi OS Arch这样的Linux发行版镜像，Ubuntu PC指的是安装了Ubuntu系统的个人电脑。

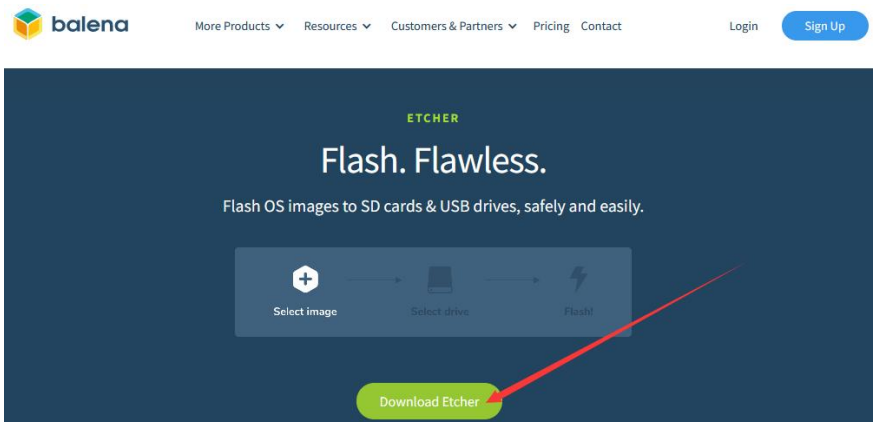
1) 首先准备一张 16GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡

2) 然后使用读卡器把 TF 卡插入电脑

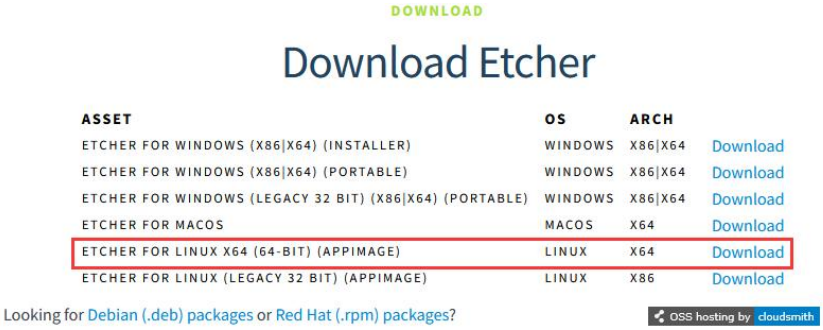
3) 下载 balenaEtcher 软件，下载地址为

https://www.balena.io/etcher/

4) 进入 balenaEtcher 下载页面后，点击绿色的下载按钮会跳到软件下载的地方



5) 然后选择下载 Linux 版本的软件即可



6) 从 **Orange Pi 的资料下载页面**下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 2GB 以上

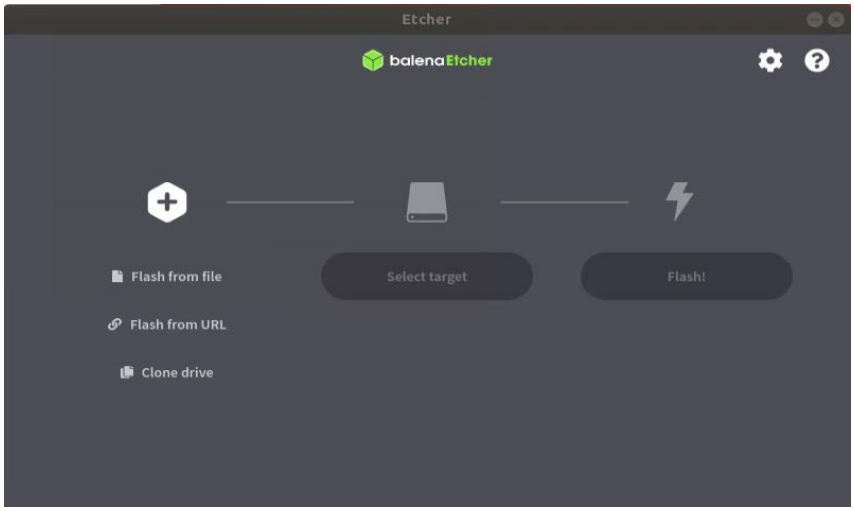
7z 结尾的压缩包的解压命令如下所示

```
test@test:~$ 7z x Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.7z
test@test:~$ ls Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.*
Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.7z
Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.sha     #校验和文件
Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.img     #镜像文件
```

7) 解压镜像后可以先用 **sha256sum -c *.sha** 命令计算下校验和是否正确，如果提示**成功**说明下载的镜像没有错，可以放心的烧录到 TF 卡，如果提示**校验和不匹配**说明下载的镜像有问题，请尝试重新下载

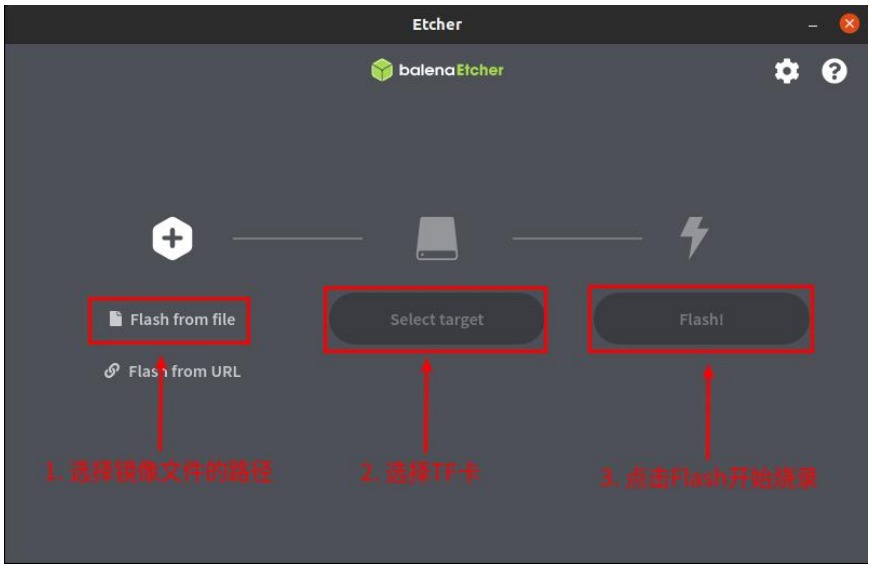
```
test@test:~$ sha256sum -c *.sha
Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.img: OK
```

8) 然后在 Ubuntu PC 的图形界面双击 **balenaEtcher-1.5.109-x64.AppImage** 即可打开 balenaEtcher（**无需安装**），balenaEtcher 打开后的界面显示如下图所示

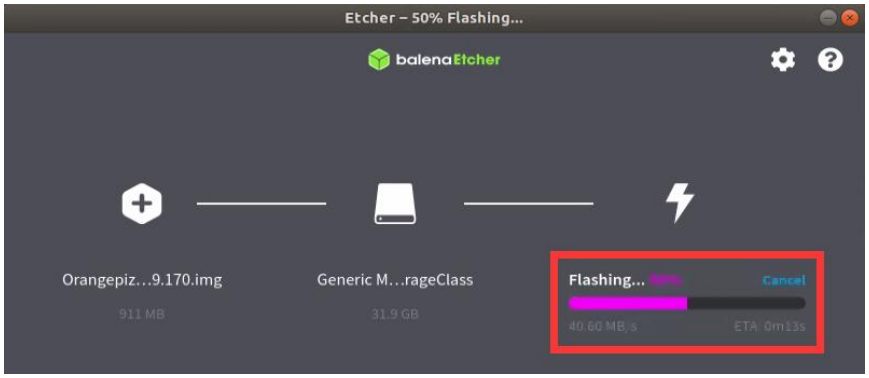


9) 使用 balenaEtcher 烧录 Linux 镜像的具体步骤如下所示

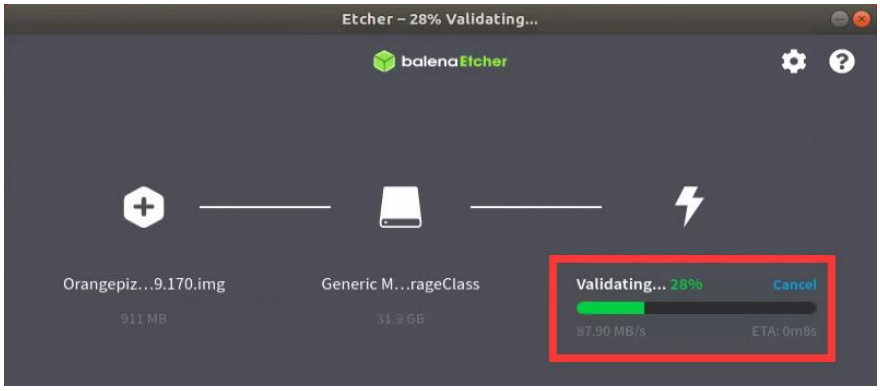
a. 首先选择要烧录的 Linux 镜像文件的路径
b. 然后选择 TF 卡的盘符
c. 最后点击 Flash 就会开始烧录 Linux 镜像到 TF 卡中



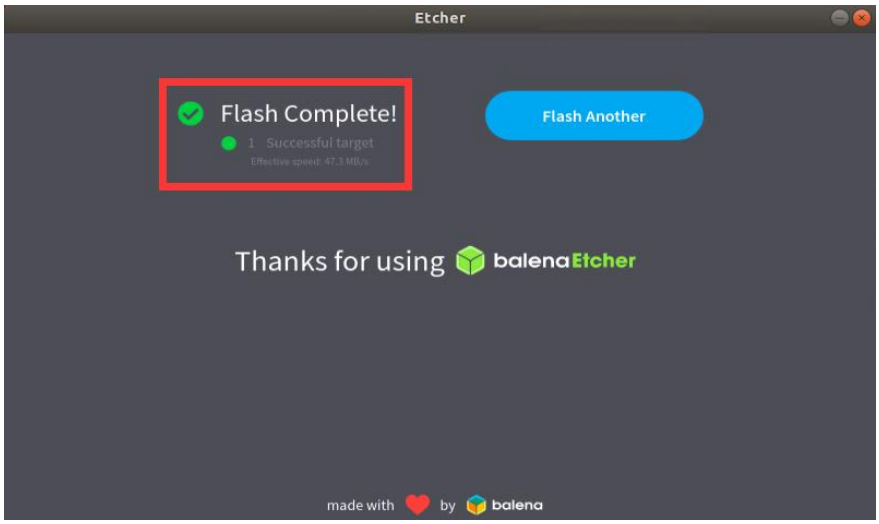
10) balenaEtcher 烧录 Linux 镜像的过程显示的界面如下图所示，另外进度条显示紫色表示正在烧录 Linux 镜像到 TF 卡中



11) Linux 镜像烧录完后，balenaEtcher 默认还会对烧录到 TF 卡中的镜像进行校验，确保烧录过程没有出问题。如下图所示，显示绿色的进度条就表示镜像已经烧录完成，balenaEtcher 正在对烧录完成的镜像进行校验



12) 成功烧录完成后 balenaEtcher 的显示界面如下图所示，如果显示绿色的指示图标说明镜像烧录成功，此时就可以退出 balenaEtcher，然后拔出 TF 卡插入到开发板的 TF 卡槽中使用了



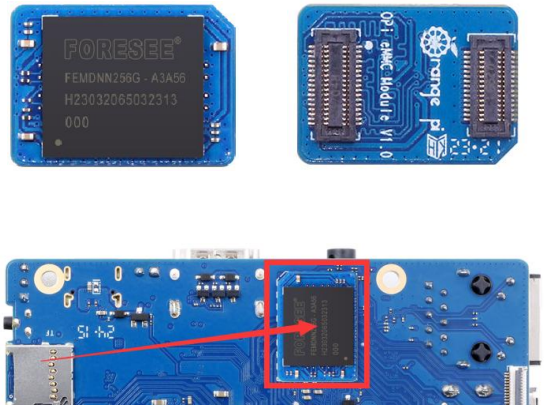
2.5. 烧录 Linux 镜像到 eMMC 中的方法

2.5.1. 使用 RKDevTool 烧录 Linux 镜像到 eMMC 中的方法

注意，下面所有的操作都是在 Windows 电脑中进行的。

注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian、Ubuntu、OpenWRT或者OPi OS Arch这样的Linux发行版镜像。

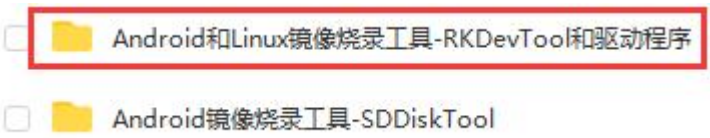
1) 开发板预留了 eMMC 模块的扩展接口，烧录系统到 eMMC 前，首先需要购买一个与开发板 eMMC 接口相匹配的 eMMC 模块。然后将 eMMC 模块安装到开发板上。eMMC 模块和插入开发板的方法如下所示：



2) 还需要准备一根品质良好的 USB2.0 公对公数据线

3) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和 **MiniLoader** 以及烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面**首先选择官方工具，然后进入下面的文件夹中



b. 然后下载下面的所有文件

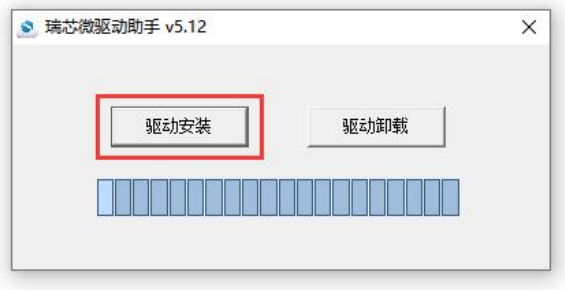
注意，“MiniLoader-烧录Linux镜像才需要用到的东西”文件夹下文简称为MiniLoader文件夹。

4) 然后从 **Orange Pi 的资料下载页面**下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 2GB 以上

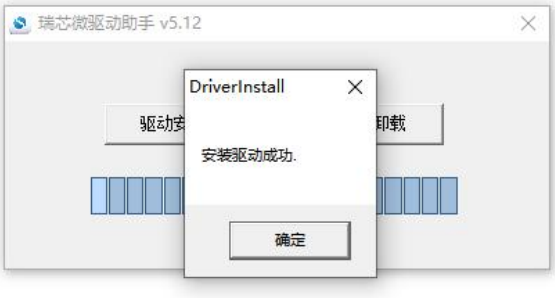
5) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

6) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

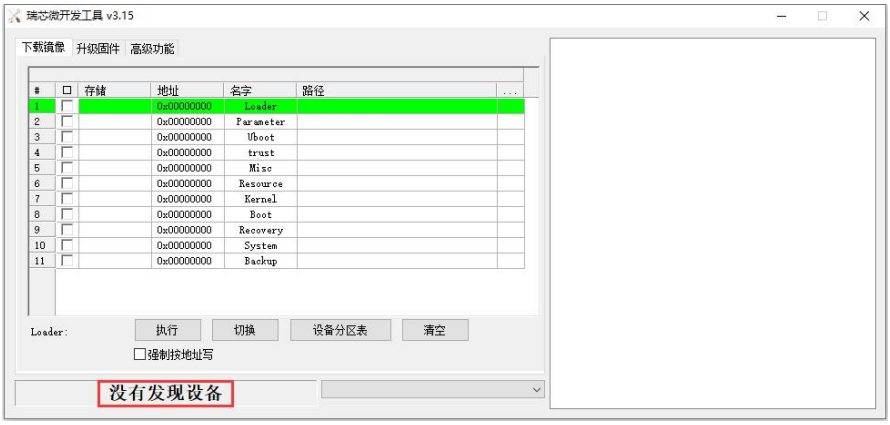


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



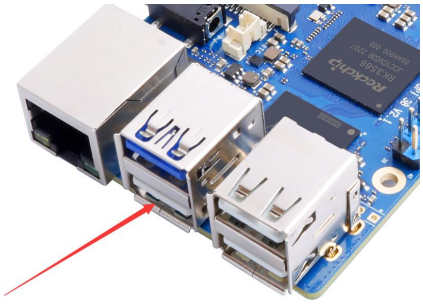
7) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

8) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



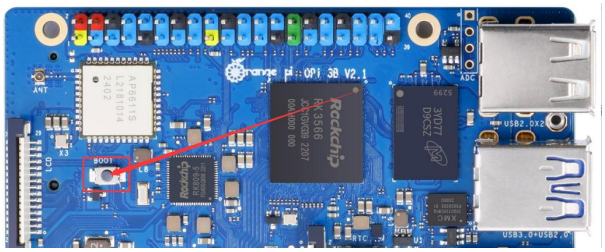
9) 然后开始烧录 Linux 镜像到 eMMC 中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录口的位置如下图所示

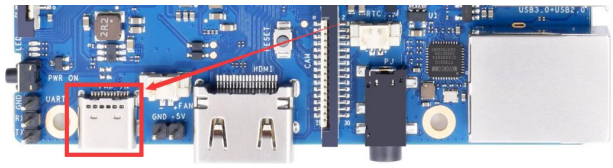


b. 确保开发板没有插入 TF 卡，没有连接电源

c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：

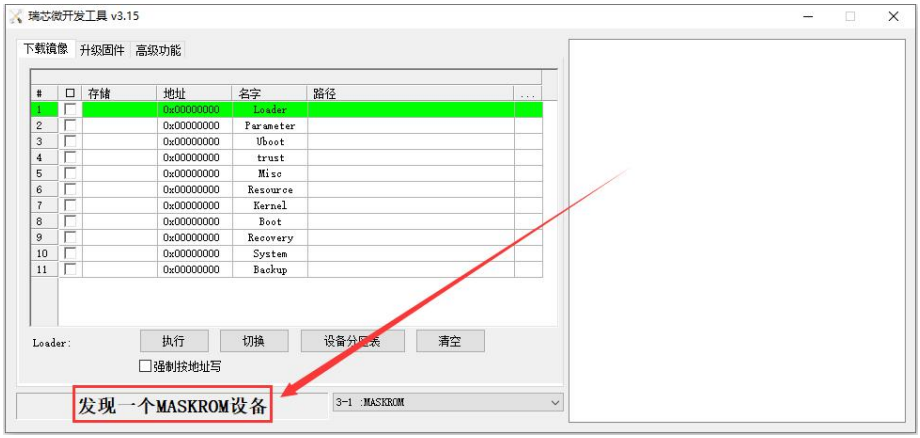


d. 然后给开发板接上 Type-C 接口的电源，并上电，然后就可以松开 MaskROM 按键了

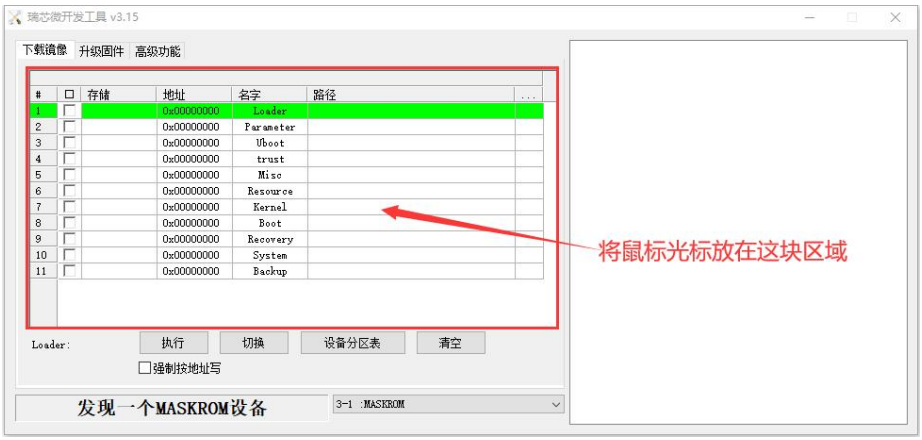


e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的

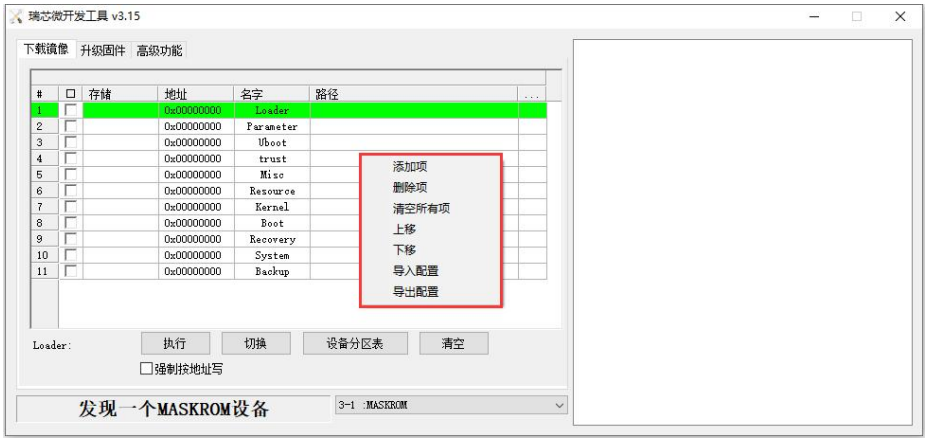
界面上会提示“**发现一个 MASKROM 设备**”



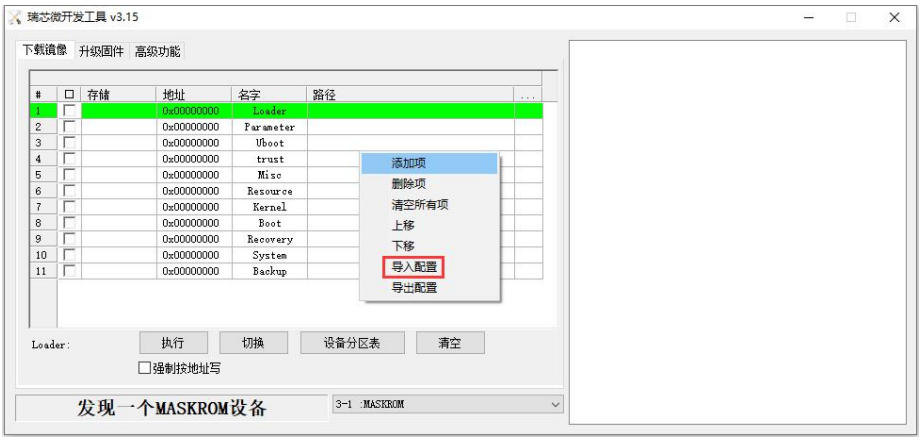
f. 然后将鼠标光标放在下面的这片区域中



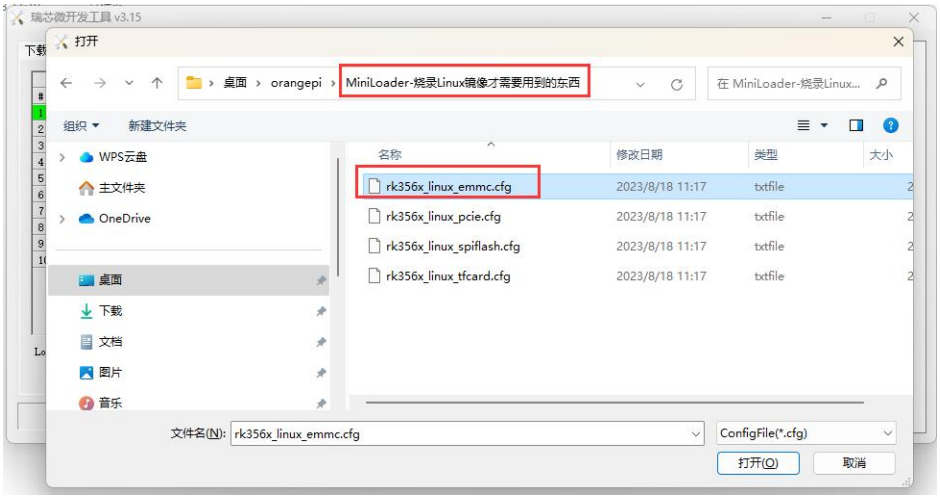
g. 然后点击鼠标右键会弹出下图所示的选择界面



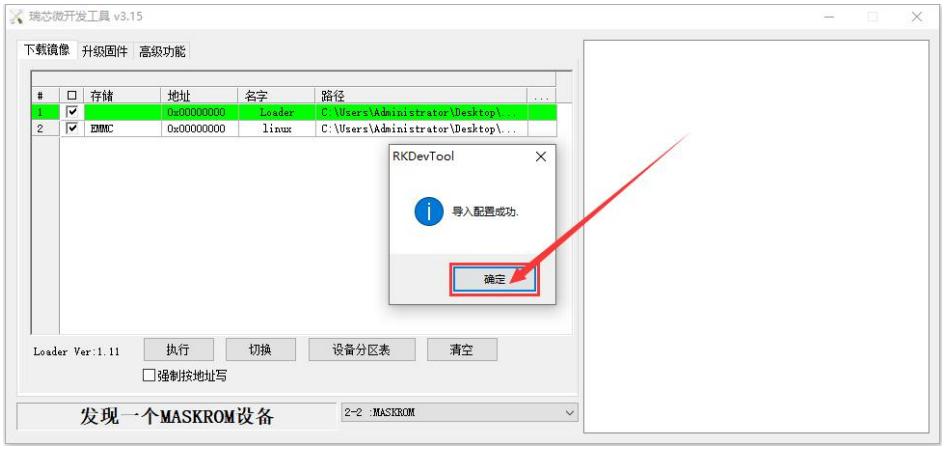
h. 然后选择**导入配置**选项



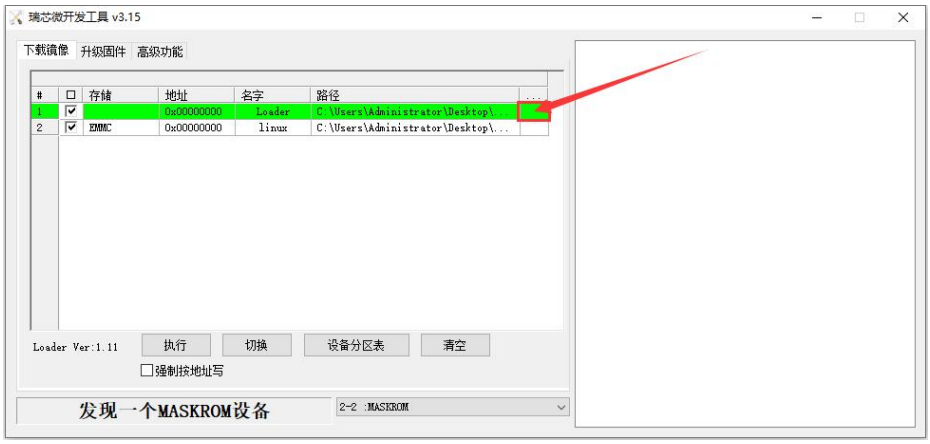
i. 然后选择前面下载的 **MiniLoader** 文件夹中的 **rk356x_linux_emmc.cfg** 配置文件，再点击**打开**



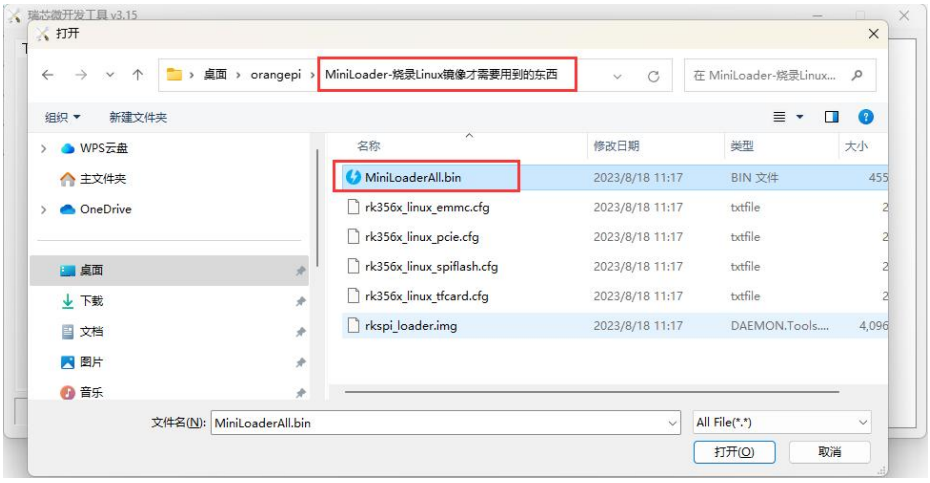
j. 然后点击**确定**



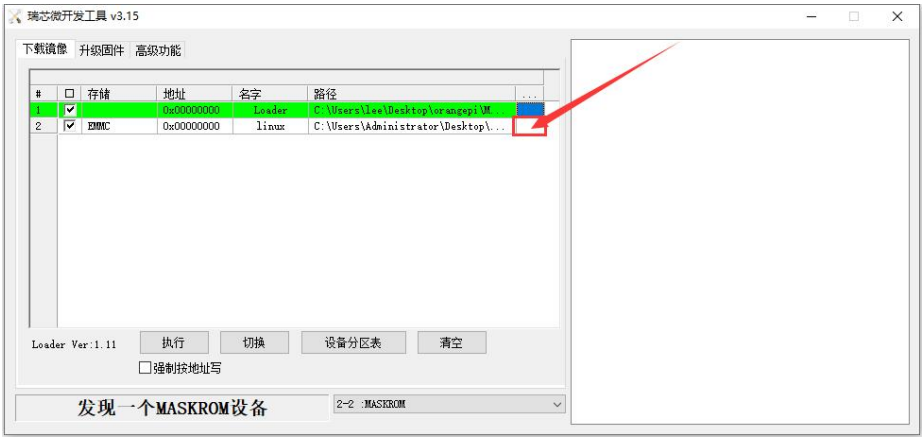
k. 然后点击下图所示的位置



l. 再选择前面下载的 **MiniLoader** 文件夹中 **MiniLoaderAll.bin**，再点击**打开**

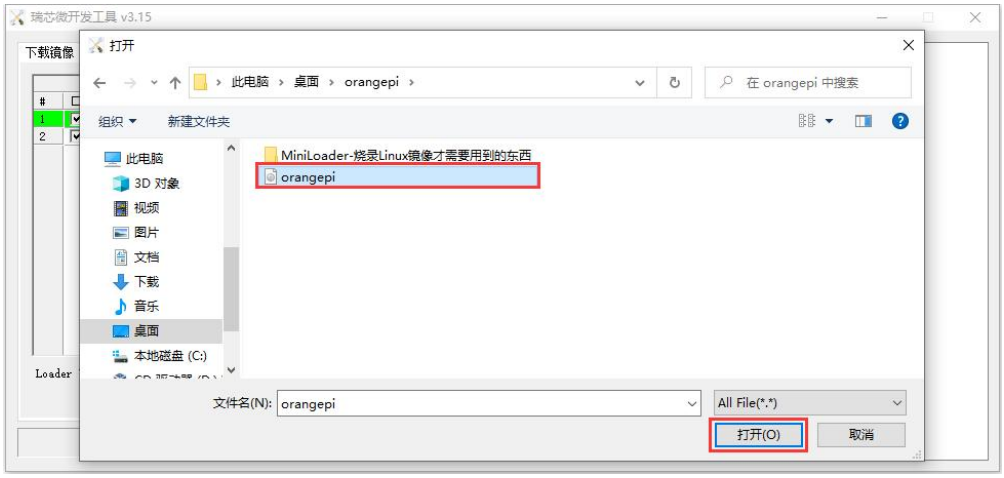


m. 然后点击下图所示的位置

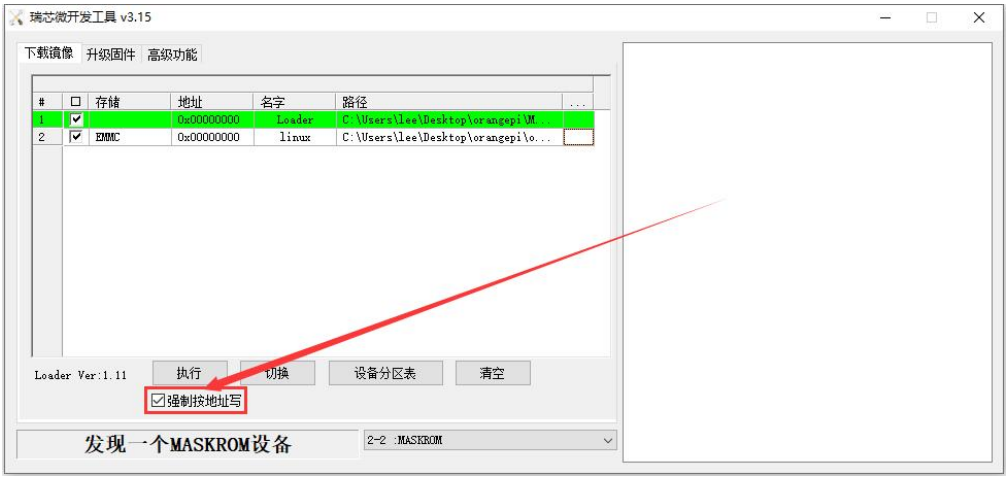


n. 然后选择想要烧录的 Linux 镜像的路径，再点击**打开**

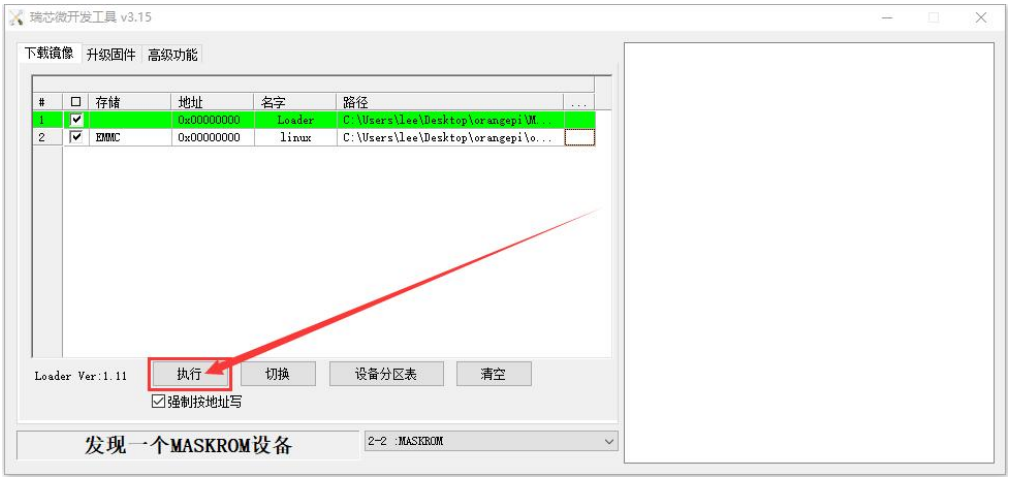
在烧录镜像前，建议将要烧录的Linux镜像重命名为orangepi.img或者其它比较短的名字，这样在烧录镜像的时候就能看到烧录进度的百分比数值。



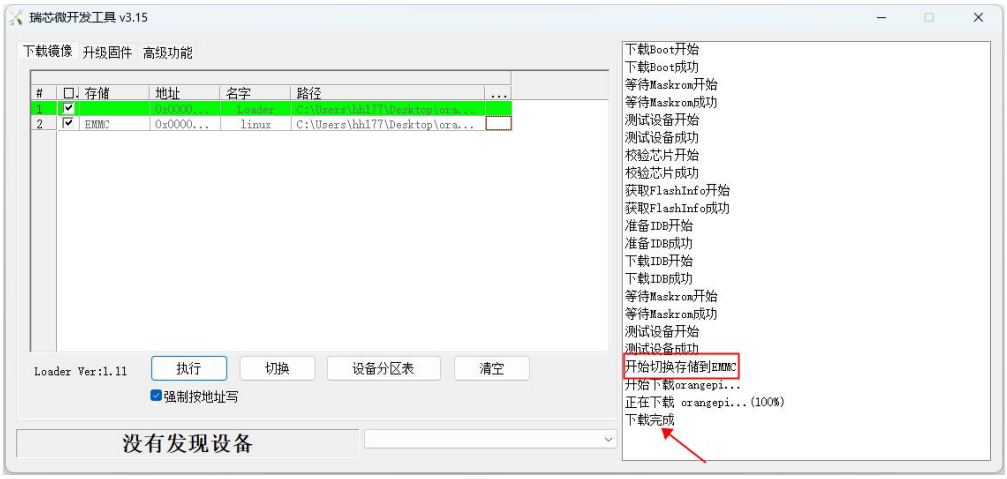
o. 然后请勾选上**强制按地址写**选项



p. 再点击执行按钮就会开始烧录 Linux 镜像到开发板的 eMMC 中



q. Linux 镜像烧录完后的显示 log 如下图所示



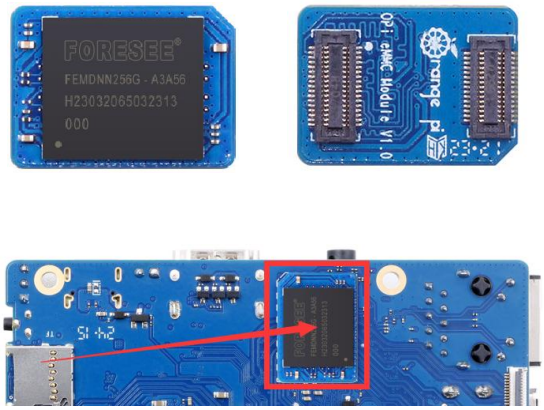
r. 烧录完 Linux 镜像到 eMMC 中后，Linux 系统会自动启动。

注意，烧录镜像到eMMC中后如果测试发现启动不了，请清空下SPIFlash后再启动试下。清空SPIFlash的方法请参考使用RKDevTool清空SPIFlash的方法小节的内容。

2.5.2. 使用 dd 命令烧录 Linux 镜像到 eMMC 中的方法

注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian、Ubuntu、OpenWRT或者OPi OS Arch这样的Linux发行版镜像。

1) 开发板预留了 eMMC 模块的扩展接口，烧录系统到 eMMC 前，首先需要购买一个与开发板 eMMC 接口相匹配的 eMMC 模块。然后将 eMMC 模块安装到开发板上。eMMC 模块和插入开发板的方法如下所示：



2) 使用 dd 命令烧录 Linux 镜像到 eMMC 中需要借助 TF 卡来完成，所以首先需要将 Linux 镜像烧录到 TF 卡上，然后使用 TF 卡启动开发板进入 Linux 系统。烧录 Linux

镜像到 TF 卡的方法请见**基于 Windows PC 将 Linux 镜像烧录到 TF 卡的方法**和**基于 Ubuntu PC 将 Linux 镜像烧录到 TF 卡的方法**两小节的说明。

3) 使用 TF 卡启动 Linux 系统后，我们首先将解压后的 Linux 镜像文件（从官网下载的 Debian、Ubuntu 镜像、OpenWRT 或者 OPi OS（Arch）镜像）上传到 TF 卡中。上传 Linux 镜像文件到开发板中的方法请参考**上传文件到开发板 Linux 系统中的方法**小节的说明。

4) 上传完镜像到开发板的 Linux 系统中后，我们再在开发板 Linux 系统的命令行中进入镜像文件的存放路径，比如，我将开发板的 Linux 镜像存放在 **/home/orangepi/Desktop** 目录下了，然后进入**/home/orangepi/Desktop** 目录就能看到上传的镜像文件了。

```
orangepi@orangepi:~$ cd /home/orangepi/Desktop
orangepi@orangepi:~/Desktop$ ls
Orangepi3b_x.x.x_debian_bullseye_desktop_xfce_linux5.10.160.img
```

怎么进入开发板Linux系统的命令行？

1. **使用串口登录终端的方法请参考调试串口的使用方法一小节的说明。**
2. **使用ssh远程登录Linux系统请参考SSH远程登录开发板一小节的说明。**
3. **如果接了HDMI、LCD等显示屏幕，可以在桌面中打开一个命令行终端。**

5) 接下来，我们先使用下面的命令确认下 eMMC 的设备节点

```
orangepi@orangepi:~/Desktop$ ls /dev/mmcblk*boot0 | cut -c1-12
/dev/mmcblk0
```

6) 然后我们可以使用 dd 命令清空下 eMMC，注意 **of=**参数后面请填入上面命令输出的结果

```
orangepi@orangepi:~/Desktop$ sudo dd bs=1M if=/dev/zero of=/dev/mmcblk0 count=1000 status=progress
orangepi@orangepi:~/Desktop$ sudo sync
```

7) 然后就可以使用 dd 命令烧录开发板的 Linux 镜像到 eMMC 中

a. 下面的命令中 **if=** 参数后面是要填写 Linux 镜像存放的完整路径+Linux 镜像的名字（比如**/home/orangepi/Desktop/Linux 镜像的名字**）。因为上面我们已经进入 Linux 镜像的路径下了，所以只需要填写 Linux 镜像的名字的即可。

b. 下面命令中的 Linux 镜像名请不要照抄，要替换为实际的镜像名（因为镜像的版本号可能会更新）。

```
sudo dd bs=1M if=Orangepi3b_x.x.x_debian_bullseye_desktop_xfce_linux5.10.160.img of=/dev/mmcblk0 status=progress


sudo sync
```

注意，如果上传的是 .7z或者.xz 结尾Linux镜像压缩文件，使用dd命令烧录前请记得先解压。

dd命令的所有参数的详细说明和更多用法可以在Linux系统中执行man dd命令来查看。

8) 当成功烧录开发板的 Linux 镜像到 eMMC 后，此时就可以使用 **poweroff** 命令关机了。然后请拔出 TF 卡，再短按电源按键开机，此时就会启动 eMMC 中的 Linux 系统了。

注意，烧录镜像到eMMC中后如果测试发现启动不了，请清空下SPIFlash后再启动试下。清空SPIFlash的方法请参考使用RKDevTool清空SPIFlash的方法小节的内容。

2.6. 烧写 Linux 镜像到 SPIFlash+NVMe SSD 中的方法

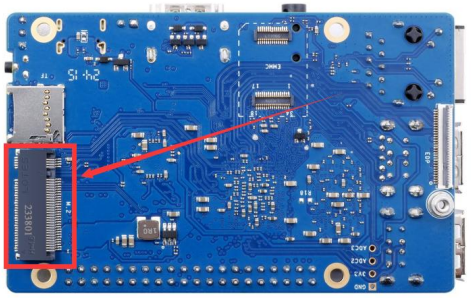
注意，这里说的Linux镜像具体指的是从Orange Pi资料下载页面下载的Debian、Ubuntu、OpenWRT或者OPi OS Arch这样的Linux发行版镜像。

注意，下面所有的操作都是在 Windows 电脑中进行的。

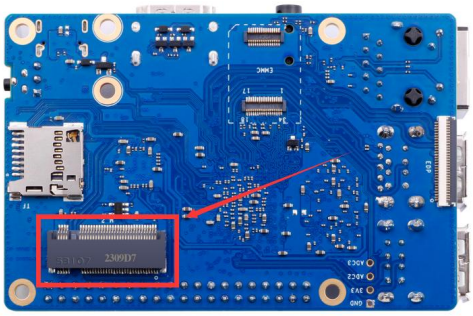
2.6.1. 使用 RKDevTool 烧录的方法

1) 首先需要准备一个 NVMe SSD 固态硬盘，开发板 M.2 插槽支持的 PCIe 为 PCIe2.0x1，理论最高速度为 500MB/s。PCIe3.0 和 PCIe4.0 的 NVMe SSD 也是可以用的，只是速度最高只有 PCIe2.0x1 的速度。

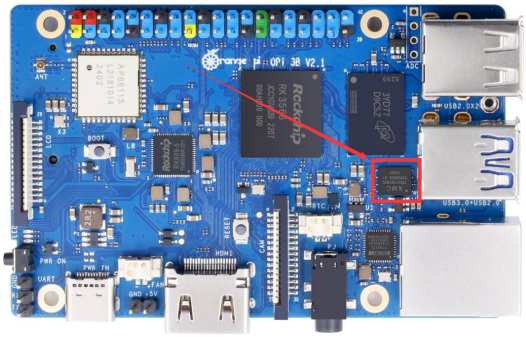
2) 然后把 NVMe SSD 插入开发板的 M.2 PCIe 接口，并固定好，Orange Pi 3B 有两个硬件版本，v2.1 版本 M.2 PCIe 接口位置如下图所示：



v1.1.1 版本 M.2 PCIe 接口位置如下图所示：



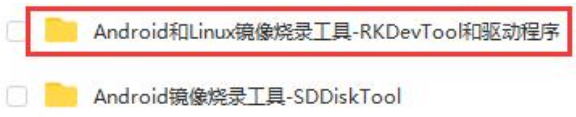
3) SPI Flash 在开发板上的位置如下图所示，开始烧录前无需其他设置



4) 然后需要准备一根品质良好的 USB2.0 公对公数据线

5) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和 **MiniLoader** 以及烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面**首先选择**官方工具**，然后进入下面的文件夹中



b. 然后下载下面的所有文件

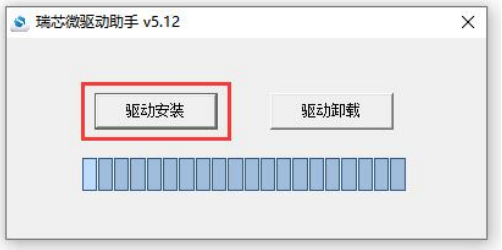
注意，“MiniLoader-烧录Linux镜像才需要用到的东西”文件夹下文简称为MiniLoader文件夹。

6) 然后从 **Orange Pi 的资料下载页面**下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“**.img**”结尾的文件就是操作系统的镜像文件，大小一般都在 2GB 以上

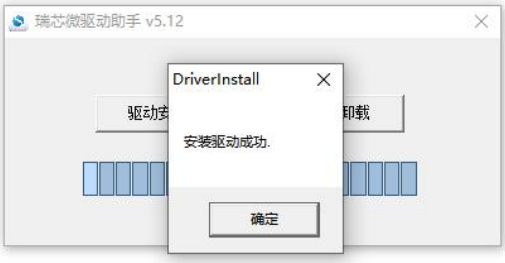
7) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

8) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

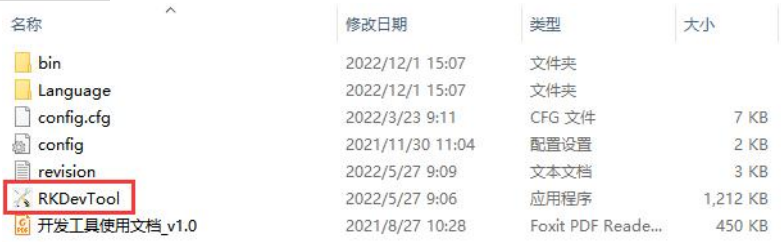
a. 点击“**驱动安装**”按钮



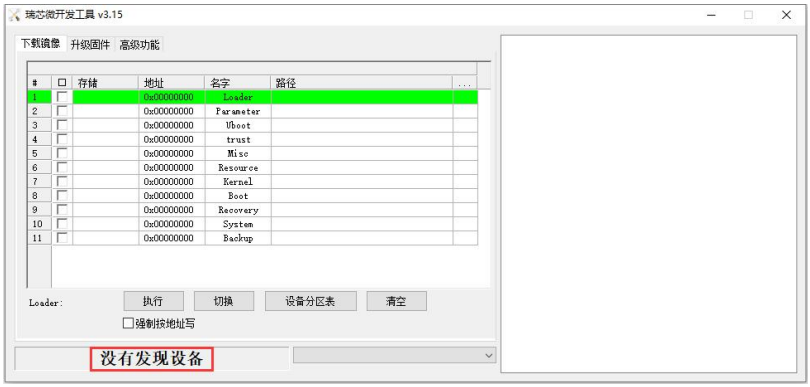
b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



9) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

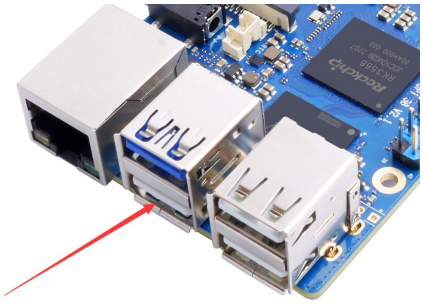


10) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



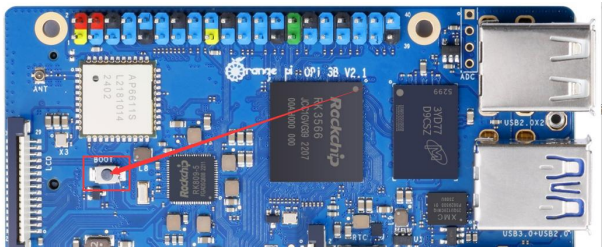
11) 然后开始烧录 Linux 镜像到 SSD 中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录接口的位置如下图所示

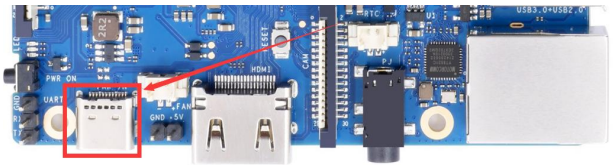


b. 确保开发板没有连接电源和插入 TF 卡以及 eMMC

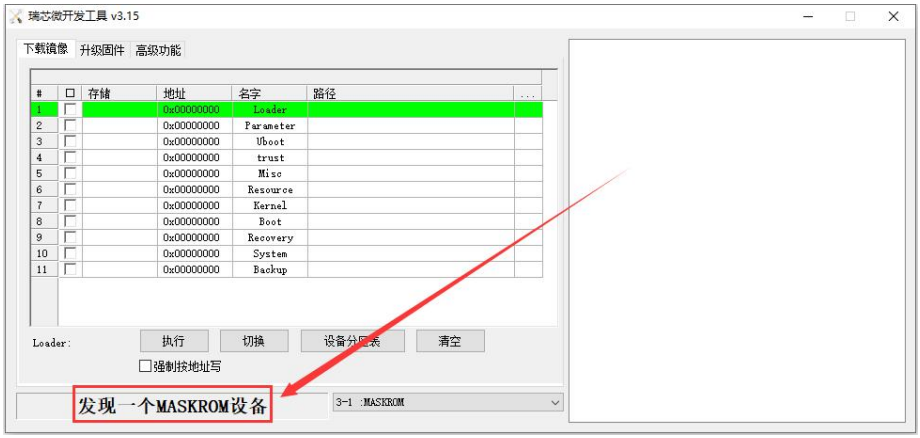
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



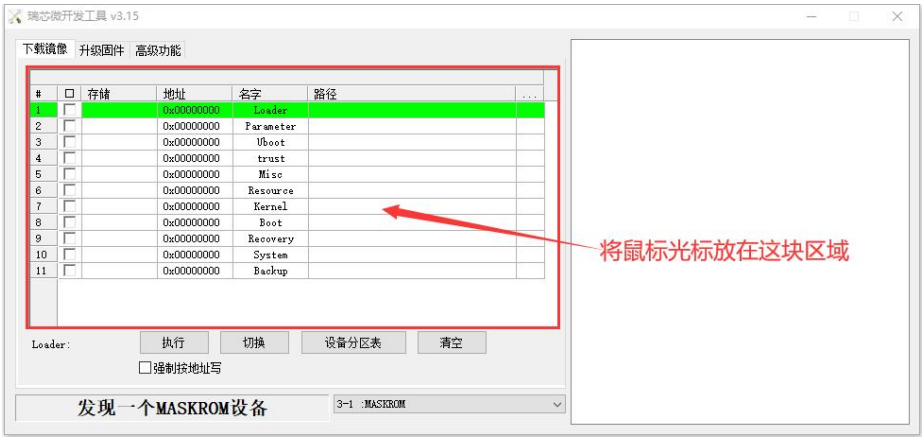
d. 然后给开发板接上 Type-C 接口的电源，并上电，然后就可以松开 MaskROM 按键了



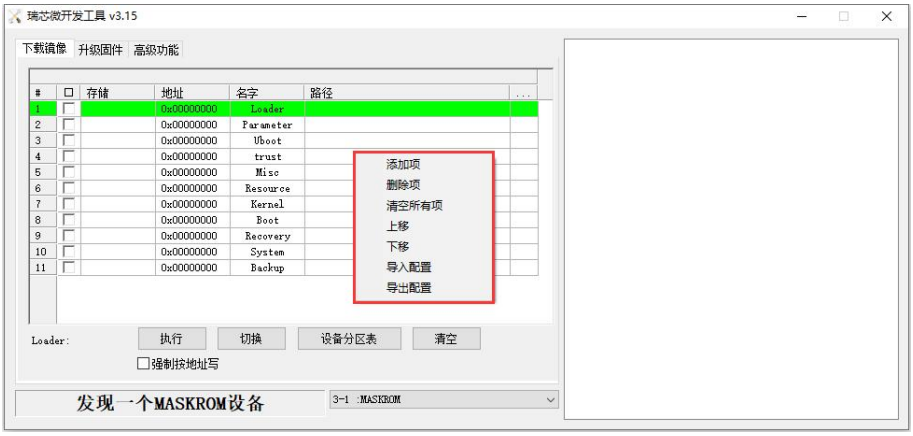
e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”



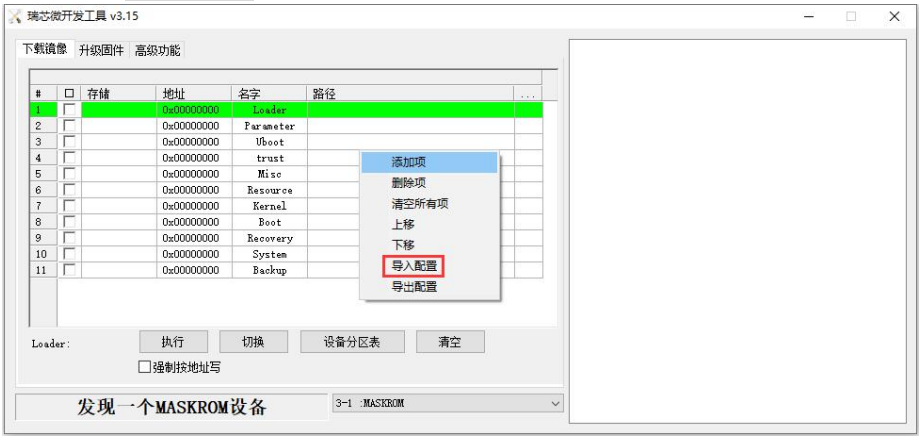
f. 然后将鼠标光标放在下面的这片区域中



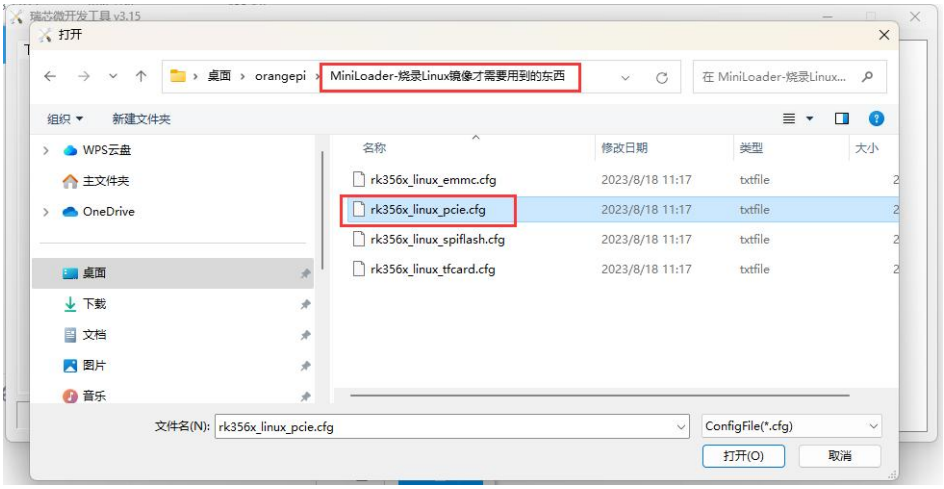
g. 然后点击鼠标右键会弹出下图所示的选择界面



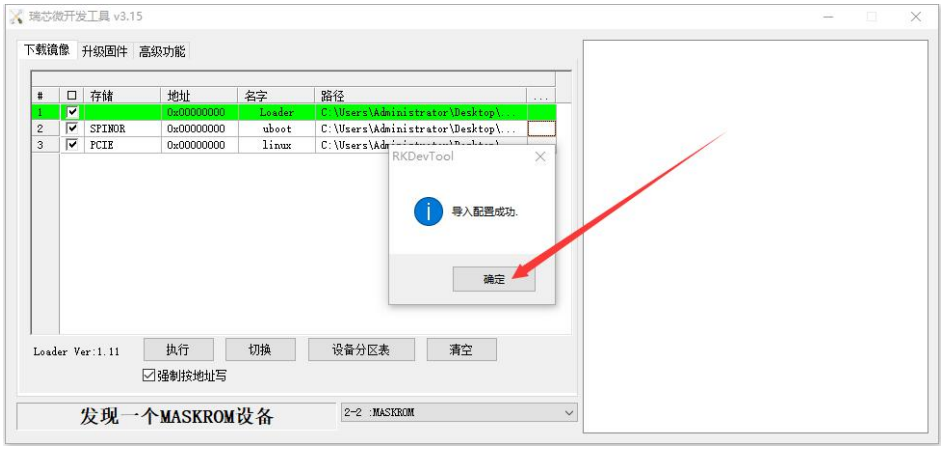
h. 然后选择**导入配置**选项



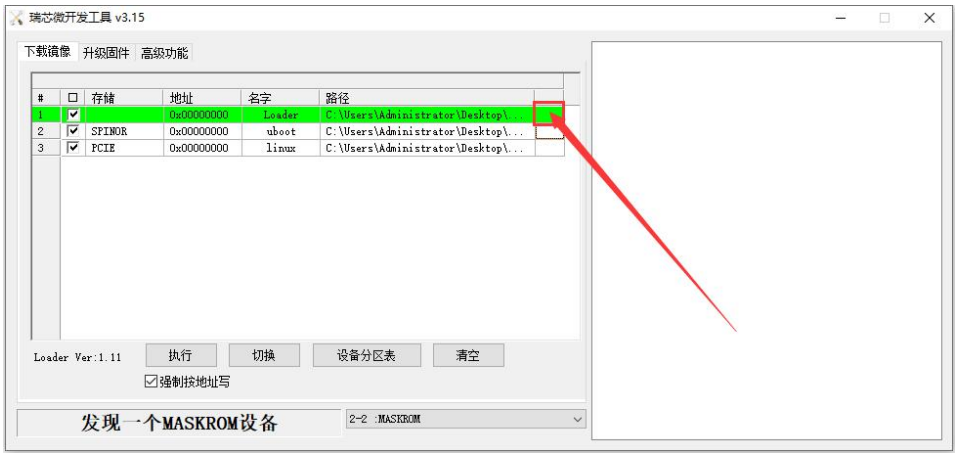
i. 然后进入前面下载的 **MiniLoader** 文件夹中，然后选择 **rk356x_linux_pcie.cfg** 配置文件，再点击**打开**



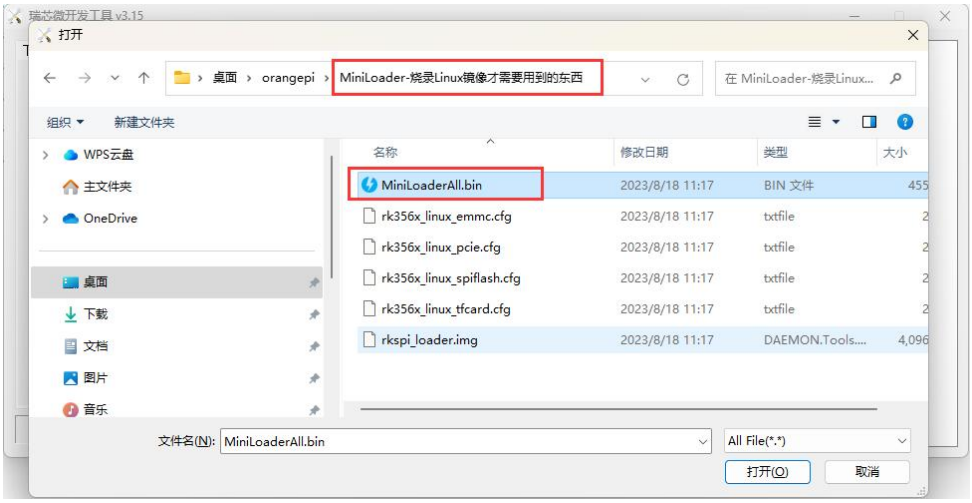
j. 然后点击**确定**



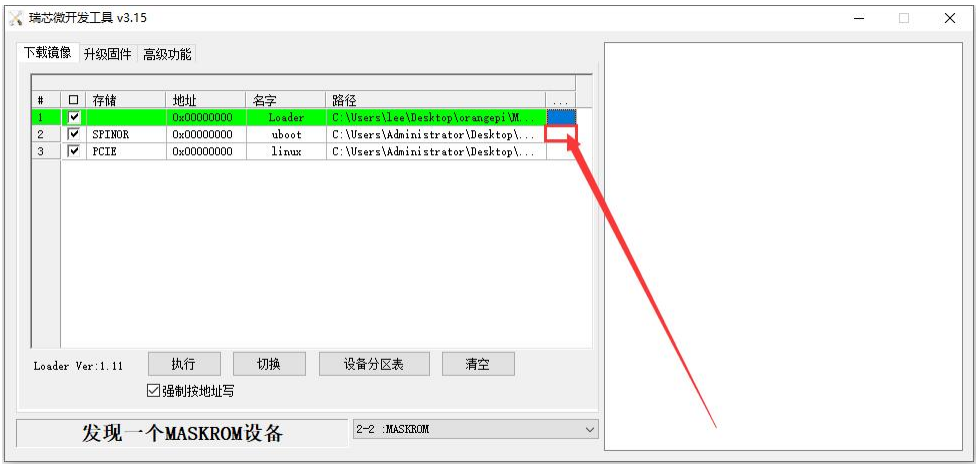
k. 然后点击下图所示的位置



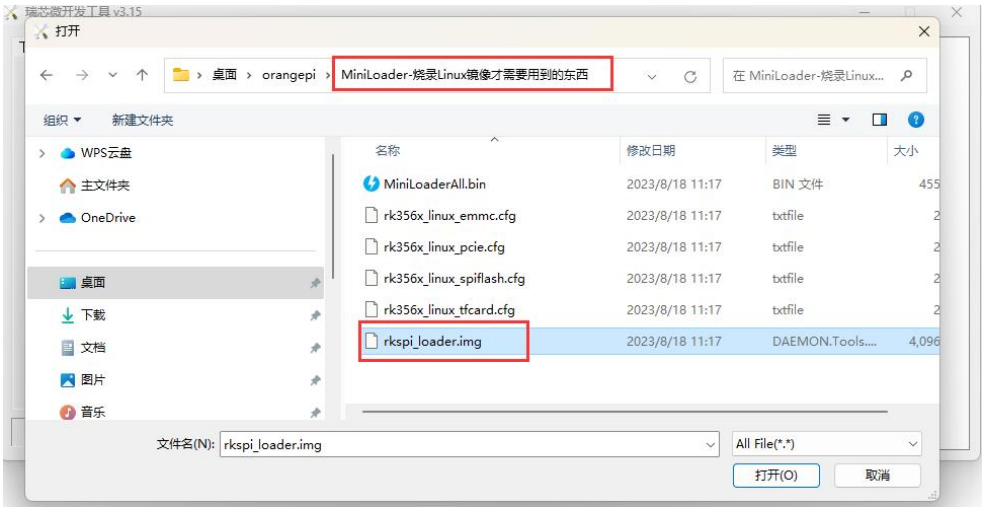
l. 再选择前面下载的 **MiniLoader** 文件夹中 **MiniLoaderAll.bin**，再点击**打开**



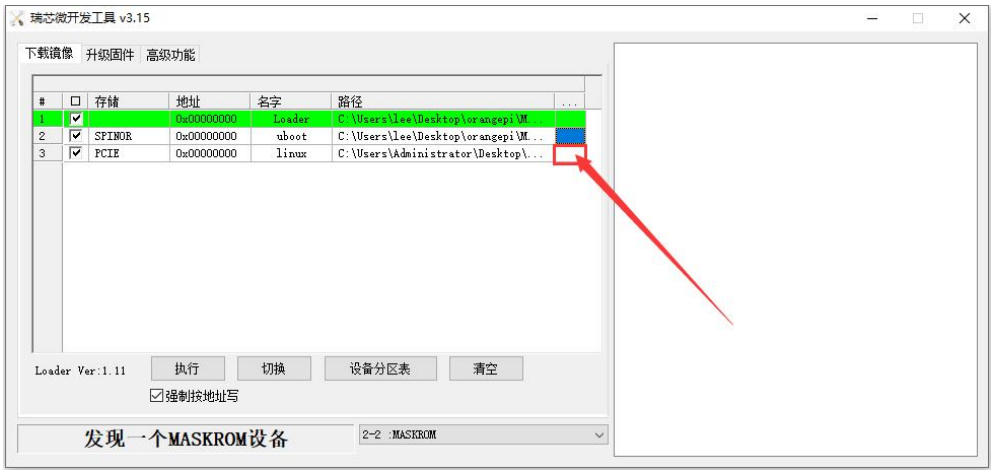
m. 然后点击下图所示的位置



n. 然后进入前面下载的 **MiniLoader** 文件夹中，再选择 **rkspi_loader.img**，再点击**打开**

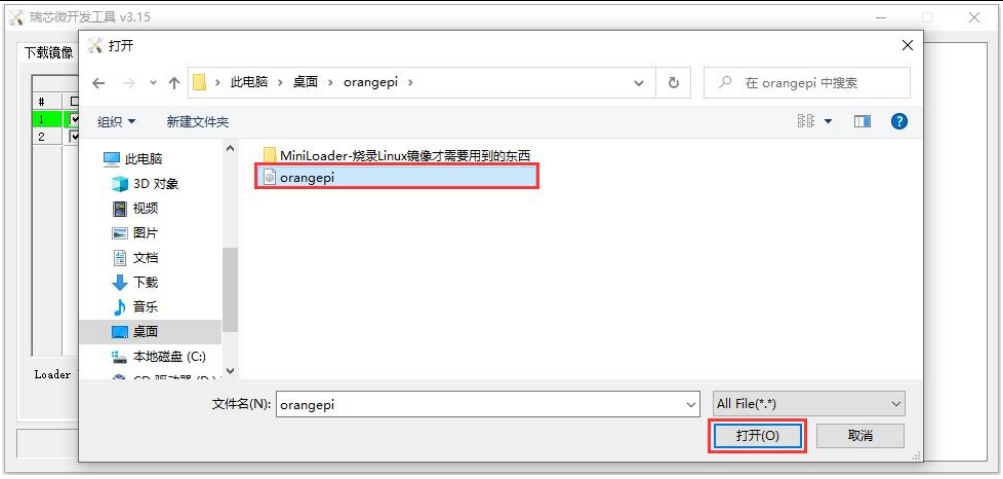


o. 然后点击下图所示的位置

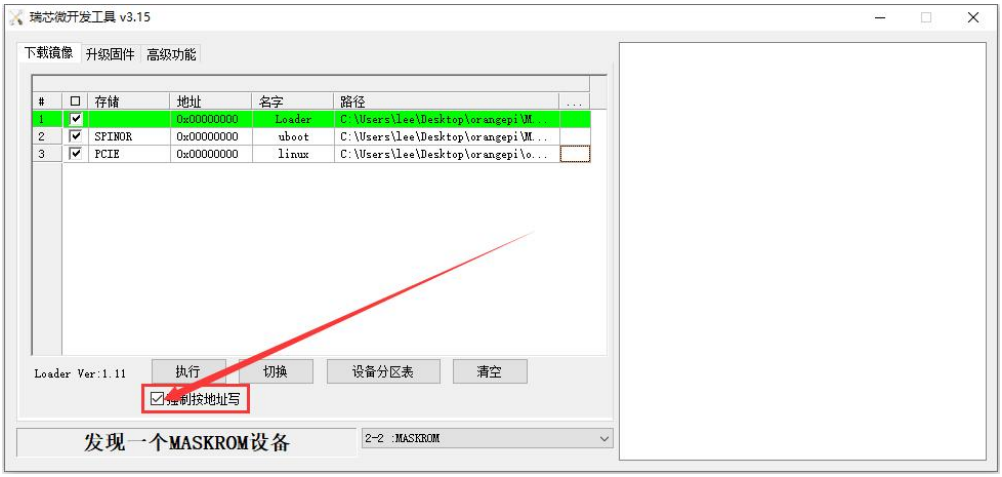


p. 然后选择想要烧录的 Linux 镜像的路径，再点击**打开**

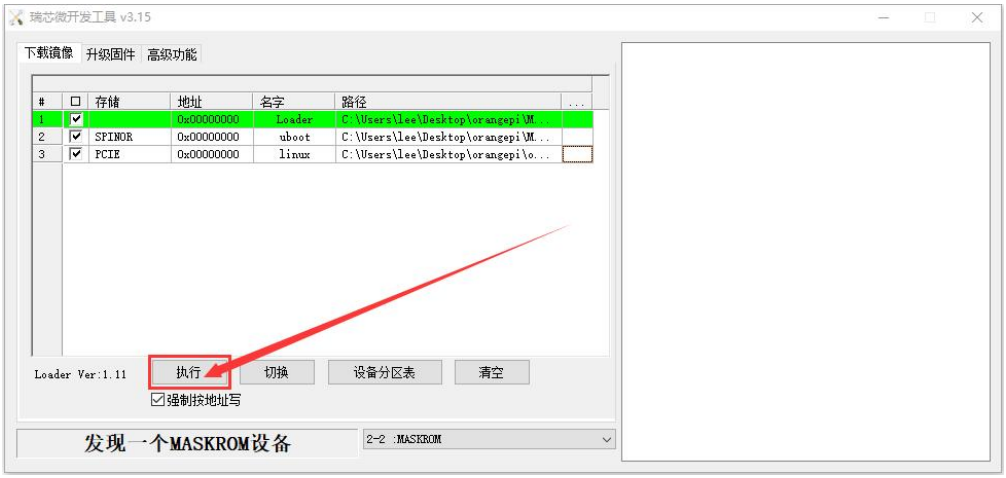
在烧录镜像前，建议将要烧录的Linux镜像重命名为orangepi.img或者其它比较短的名字，这样在烧录镜像的时候就能看到烧录进度的百分比数值。



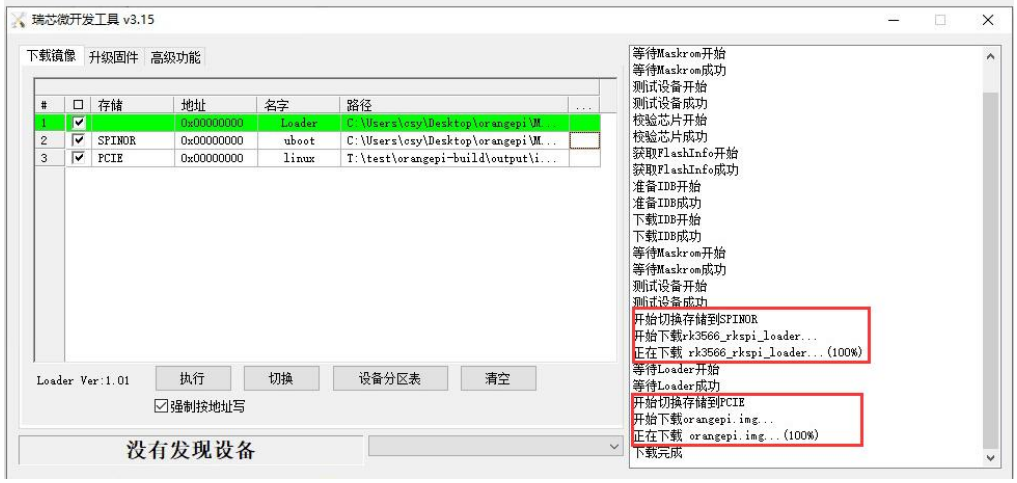
q. 然后请勾选上**强制按地址写**选项



r. 再点击执行按钮就会开始烧录 Linux 镜像到 SSD 中



s. Linux 镜像烧录完后的显示 log 如下图所示



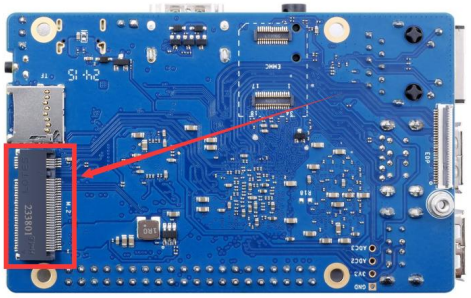
如果烧录有问题，请先清空下 SPIFlash 然后再烧录试下。清空 SPIFlash 的方法请参考下使用 RKDevTool 清空 SPIFlash 的方法一小节的说明。

t. 镜像烧录完后会自动启动 SPIFlash+PCIe SSD 中的 Linux 系统，如果没有正常启动，请重新上电试下。

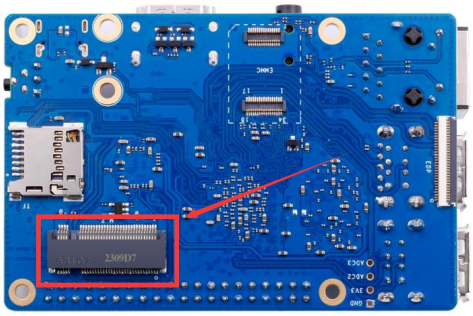
2.6.2. 使用 dd 命令烧录的方法

1) 首先需要准备一个 NVMe SSD 固态硬盘，开发板 M.2 插槽支持的 PCIe 为 PCIe2.0x1，理论最高速度为 500MB/s。PCIe3.0 和 PCIe4.0 的 NVMe SSD 也是可以用的，只是速度最高只有 PCIe2.0x1 的速度。

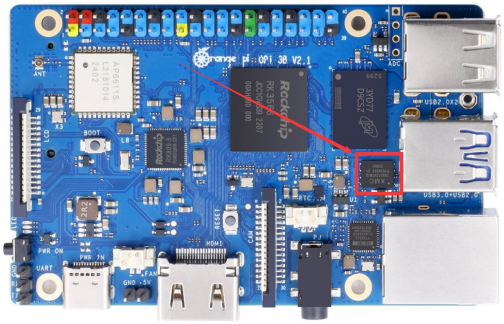
2) 然后把 NVMe SSD 插入开发板的 M.2 PCIe 接口，并固定好，Orange Pi 3B 有两个硬件版本，v2.1 版本 M.2 PCIe 接口位置如下图所示：



v1.1.1 版本 M.2 PCIe 接口位置如下图所示：



3) SPI Flash 在开发板上的位置如下图所示，开始烧录前无需其他设置



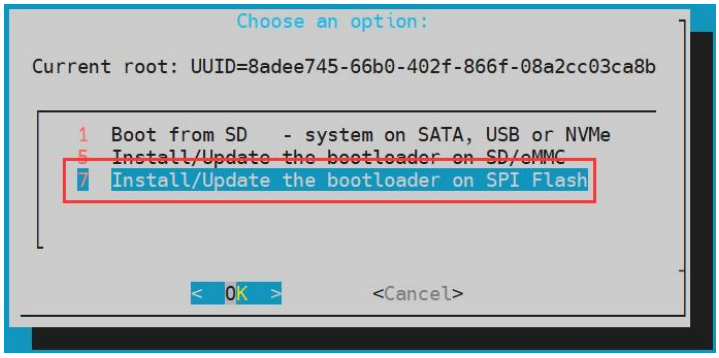
4) 烧录 Linux 镜像到 SPIFlash+NVMe SSD 中需要借助 TF 卡来完成，所以首先需要将 Linux 镜像烧录到 TF 卡上，然后使用 TF 卡启动开发板进入 Linux 系统。烧录 Linux 镜像到 TF 卡的方法请见**基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法**和**基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法**两小节的说明。

5) 使用 TF 卡启动 Linux 系统后，我们首先烧录 u-boot 镜像到 SPI Flash 中

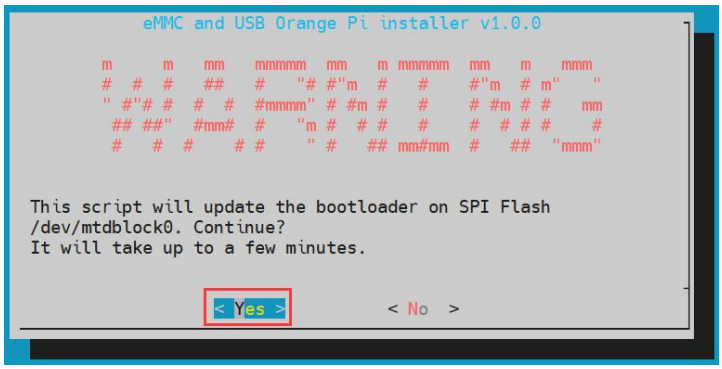
a. 先运行下 **nand-sata-install**，**普通用户记得加 sudo 权限**

```
orangepi@orangepi:~$ sudo nand-sata-install
```

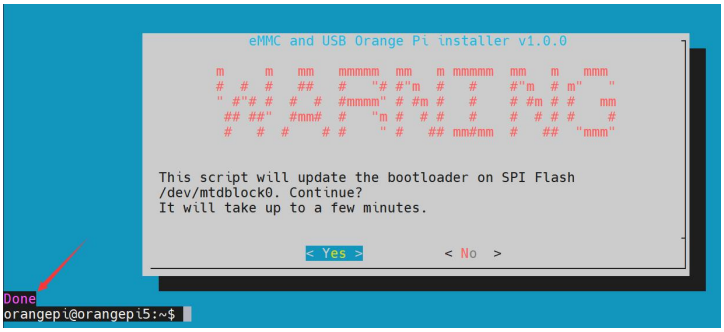
b. 然后选择 **7 Install/Update ther bootloader on SPI Flash**



c. 然后选择**<Yes>**



d. 然后请耐心等待烧录完成，烧录完成后显示如下所示（左下角会显示一个**Done**）：



6) 然后将 Linux 镜像文件（从官网下载的 Debian 或者 Ubuntu 镜像）上传到 TF 卡中。上传 Linux 镜像文件到开发板中的方法请参考**上传文件到开发板 Linux 系统中的方法**小节的说明。

7) 上传完镜像到开发板的 Linux 系统中后，我们再在开发板 Linux 系统的命令行中进入镜像文件的存放路径，比如，我将开发板的 Linux 镜像存放在**/home/orangepi/Desktop** 目录下了，然后进入**/home/orangepi/Desktop** 目录就能看到上传的镜像文件了。

```
orangepi@orangepi:~$ cd /home/orangepi/Desktop
```

```
orangepi@orangepi:~/Desktop$ ls
Orangepi3b_x.x.x_debian_bullseye_desktop_xfce_linux5.10.160.img
```

怎么进入开发板Linux系统的命令行？

1. **使用串口登录终端的方法请参考调试串口的使用方法一小节的说明。**
2. **使用ssh远程登录Linux系统请参考SSH远程登录开发板一小节的说明。**
3. **如果接了HDMI、LCD等显示屏幕，可以在桌面中打开一个命令行终端。**

8) 接下来，我们先确认下NVMe SSD已经被开发板的Linux正常识别了。如果NVMe SSD正常识别了的话，使用 **sudo fdisk -l** 命令就能看到 **nvme** 相关的信息

```
orangepi@orangepi:~/Desktop$ sudo fdisk -l | grep "nvme0n1"
Disk /dev/nvme0n1: 1.86 TiB, 2048408248320 bytes, 4000797360 sectors
```

使用 **lspci** 命令可以看到一个 NVMe 相关的 PCI 设备

```
orangepi@orangepi:~/Desktop$ lspci
00:00.0 PCI bridge: Fuzhou Rockchip Electronics Co., Ltd Device 3566 (rev 01)
01:00.0 Non-Volatile memory controller: Realtek Semiconductor Co., Ltd. Device 5765 (rev 01)
```

9) 然后我们可以使用 dd 命令清空下 NVMe SSD（可选）

```
orangepi@orangepi3b:~/Desktop$ sudo dd bs=1M if=/dev/zero of=/dev/nvme0n1 count=2000 status=progress
orangepi@orangepi3b:~/Desktop$ sudo sync
```

10) 然后就可以使用 dd 命令烧录开发板的 Linux 镜像到 NVMe SSD 中

a. 下面的命令中 **if=** 参数后面是要填写 Linux 镜像存放的完整路径+Linux 镜像的名字（比如**/home/orangepi/Desktop/Linux 镜像的名字**）。因为上面我们已经进入 Linux 镜像的路径下了，所以只需要填写 Linux 镜像的名字的即可。

b. 下面命令中的 Linux 镜像名请不要照抄，要替换为实际的镜像名（因为镜像的版本号可能会更新）。

```
sudo dd bs=1M if=Orangepi3b_x.x.x_debian_bullseye_desktop_xfce_linux5.10.160.img of=/dev/nvme0n1 status=progress


sudo sync
```

注意，如果上传的是 .7z或者.xz 或者.gz结尾Linux镜像压缩文件，使用dd命令烧录前请记得先解压。

dd命令的所有参数的详细说明和更多用法可以在Linux系统中执行man dd命令来查看。

11) 当成功烧录开发板的 Linux 镜像到 NVMe SSD 后，此时就可以使用 **poweroff** 命令关机了。然后请拔出 TF 卡，再短按电源按键开机，此时就会启动 SPIFlash+NVMe SSD 中的 Linux 系统了。

12) 启动 NVMe SSD 中的系统后，使用 **df -h** 命令可以看到实际的硬盘容量

a. 128GB NVMe SSD

```
orangepi@orangepi:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            3.8G  8.0K  3.8G   1% /dev
tmpfs           769M  1.4M  768M   1% /run
/dev/nvme0n1p2  118G  5.8G  111G   5% /
tmpfs           3.8G     0  3.8G   0% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
tmpfs           3.8G   16K  3.8G   1% /tmp
/dev/nvme0n1p1  256M   90M  166M  36% /boot
/dev/zram1      194M  9.9M  170M   6% /var/log
tmpfs           769M   60K  769M   1% /run/user/1000
tmpfs           769M   48K  769M   1% /run/user/0
```

b. 2TB NVMe SSD

```
orangepi@orangepi:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            3.8G  8.0K  3.8G   1% /dev
tmpfs           769M  1.4M  768M   1% /run
/dev/nvme0n1p2  1.9T  4.1G  1.8T   1% /
tmpfs           3.8G     0  3.8G   0% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
/dev/zram2      3.7G   76K  3.5G   1% /tmp
/dev/nvme0n1p1  256M   90M  166M  36% /boot
/dev/zram1      194M   15M  165M   9% /var/log
tmpfs           769M   60K  769M   1% /run/user/1000
tmpfs           769M   48K  769M   1% /run/user/0
```

13) 当 TF 卡和 NVMe SSD 中烧录了一模一样的系统，**如果 TF 卡和 NVMe SSD 都插在了开发板中，此时上电启动开发板，u-boot 会优先启动 TF 卡中的系统**。但由于 TF 卡和 NVMe SSD 中的系统一模一样，所以两个存储设备中的/**boot** 分区和 **rootfs** 分区的 UUID 也一样，这样会导致 TF 卡启动的时候有可能会加载了 NVMe SSD 中的分区。运行下面的脚本可以解决这个问题。

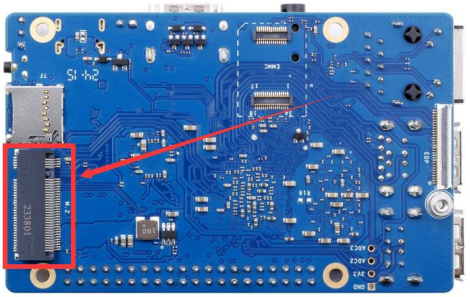
```
orangepi@orangepi:~$ sudo fix_mmc_ssd.sh
```

一模一样的系统是指镜像名一字不差的意思。就算都是Debian11 系统，版本不同也是不同的。

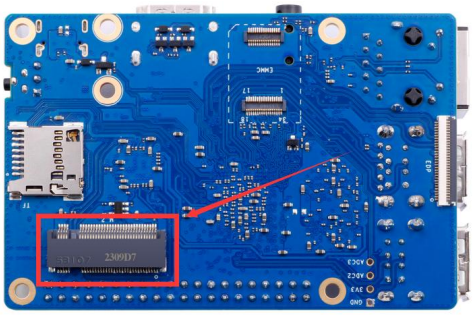
2.6.3. 使用 balenaEtcher 软件烧录的方法

1) 首先需要准备一个 NVMe SSD 固态硬盘，开发板 M.2 插槽支持的 PCIe 为 PCIe2.0x1，理论最高速度为 500MB/s。PCIe3.0 和 PCIe4.0 的 NVMe SSD 也是可以用的，只是速度最高只有 PCIe2.0x1 的速度。

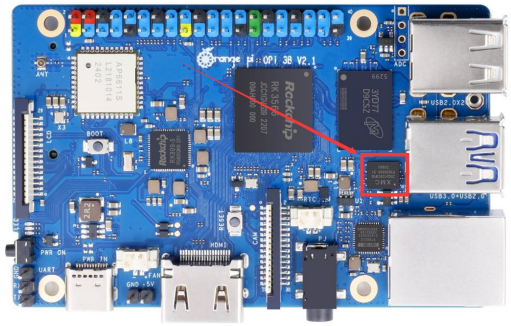
2) 然后把 NVMe SSD 插入开发板的 M.2 PCIe 接口，并固定好，Orange Pi 3B 有两个硬件版本，v2.1 版本 M.2 PCIe 接口位置如下图所示：



v1.1.1 版本 M.2 PCIe 接口位置如下图所示：



3) SPI Flash 在开发板上的位置如下图所示，开始烧录前无需其他设置



4) 烧录 Linux 镜像到 SPIFlash+NVMe SSD 中需要借助 TF 卡来完成，所以首先需要将 Linux 镜像烧录到 TF 卡上，然后使用 TF 卡启动开发板进入 Linux 系统。烧录 Linux 镜像到 TF 卡的方法请见**基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法**和**基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法**两小节的说明。

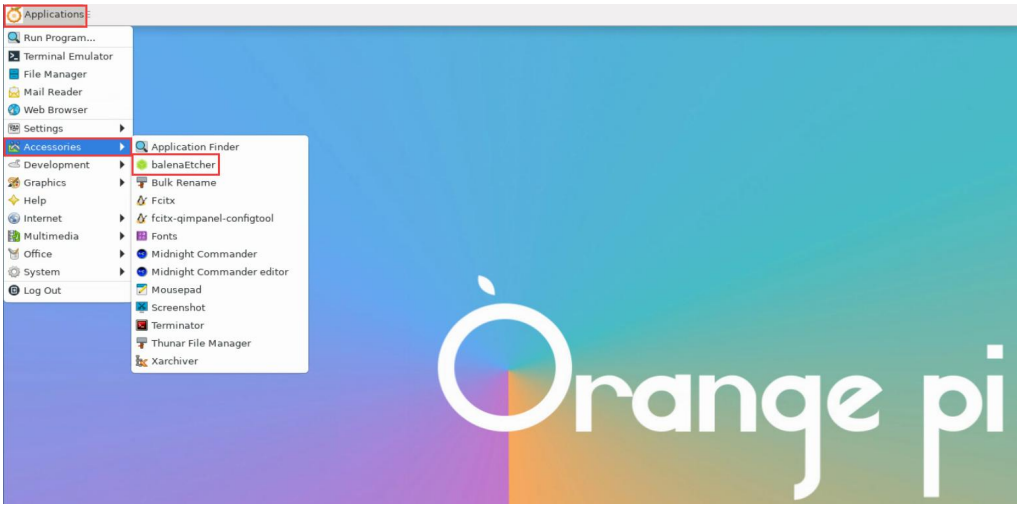
5) 启动进入 TF 卡中的 Linux 系统后，请先确认下 NVMe SSD 已经被开发板的 Linux 正常识别了。如果 NVMe SSD 正常识别了的话，使用 **sudo fdisk -l** 命令就能看到 **nvme** 相关的信息

```
orangepi@orangepi:~/Desktop$ sudo fdisk -l | grep "nvme0n1"
Disk /dev/nvme0n1: 1.86 TiB, 2048408248320 bytes, 4000797360 sectors
```

使用 **lspci** 命令可以看到一个 NVMe 相关的 PCI 设备

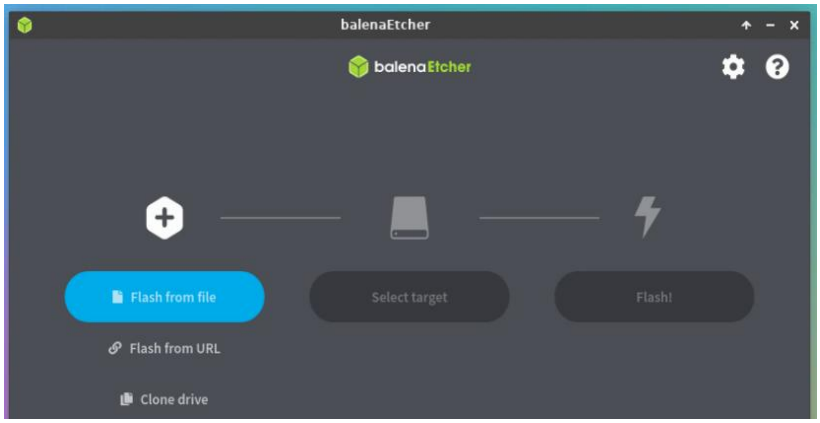
```
orangepi@orangepi:~/Desktop$ lspci
00:00.0 PCI bridge: Fuzhou Rockchip Electronics Co., Ltd Device 3566 (rev 01)
01:00.0 Non-Volatile memory controller: Realtek Semiconductor Co., Ltd. Device 5765 (rev 01)
```

6) Linux 镜像中已经预装了 balenaEtcher，打开方法如下所示：



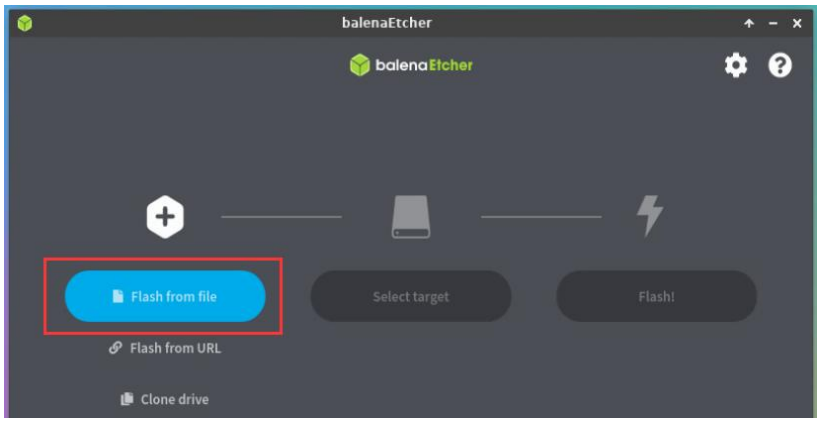
如果没有预装，balenaEtcher arm64 版本的下载安装方法请参考下载安装arm64版本balenaEtcher的方法小节的说明。

7) balenaEtcher 打开后的界面如下所示：

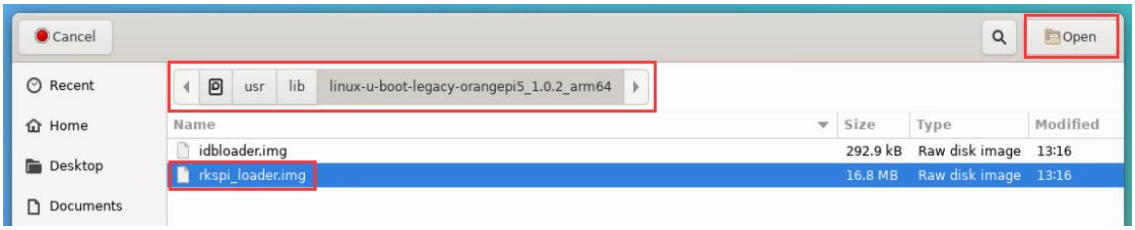


8) 使用 balenaEtcher 烧录 u-boot 到开发板 SPI Flash 中的方法如下所示：

a. 首先点击 **Flash from file**



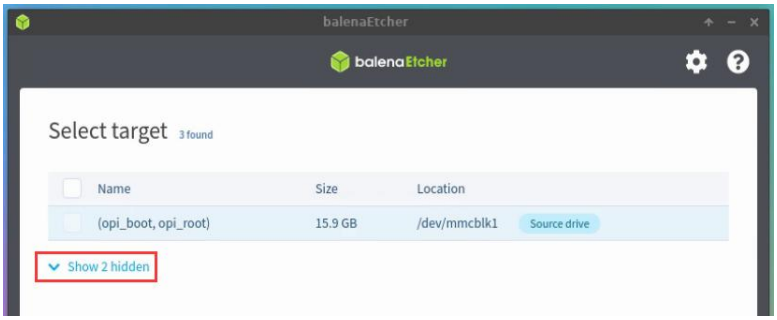
b. 然后进入**/usr/lib/linux-u-boot-legacy-orangepi3b_1.x.x_arm64** 目录，再选择**rkspi_loader.img**，再点击 **Open** 打开



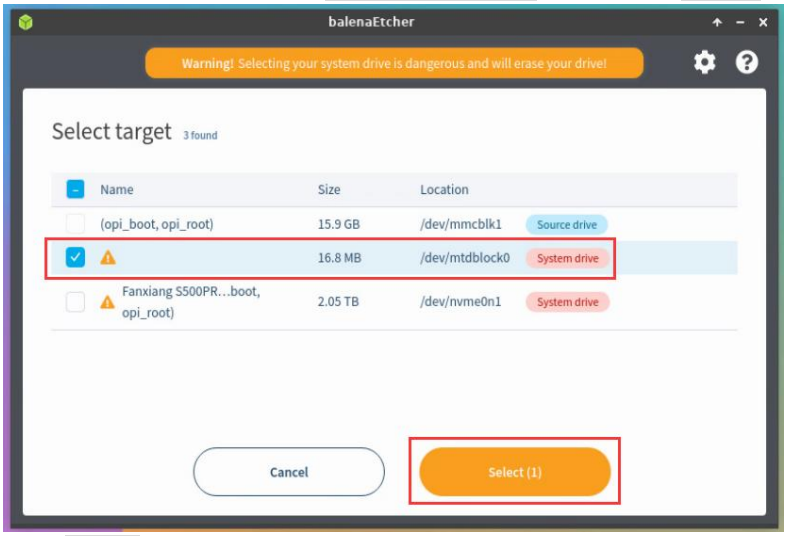
c. 打开 **rkspi_loader.img** 后的界面如下所示：

d. 然后点击 **Select target**

e. 然后点击 **Show 2 hidden** 打开更多存储设备的选项

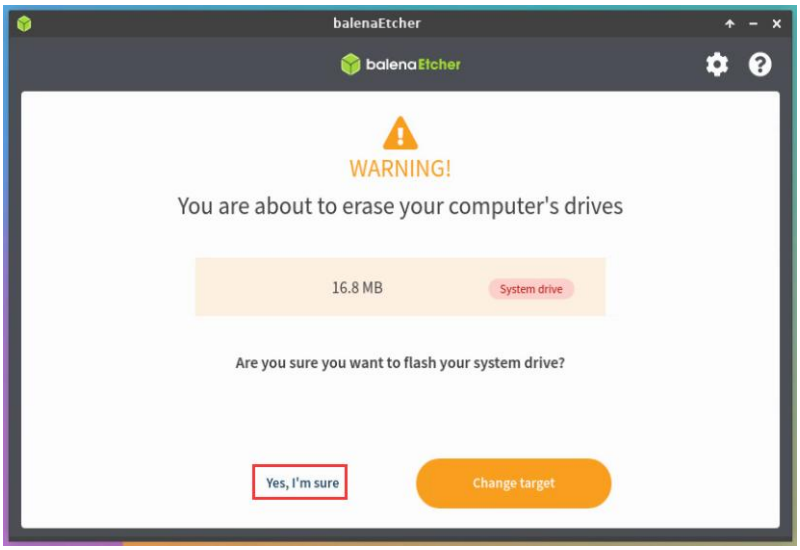


f. 然后选择 SPI Flash 的设备名**/dev/mtdblock0**,然后点击 **Select**

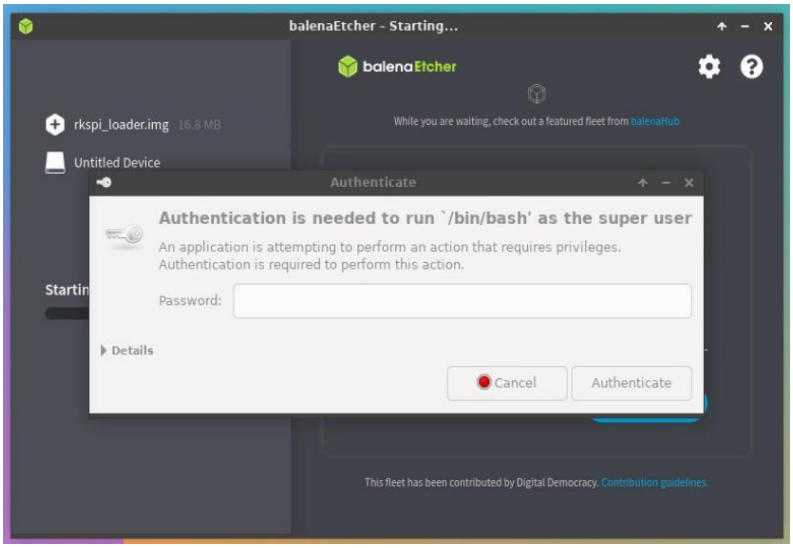


g. 然后点击 **Flash**

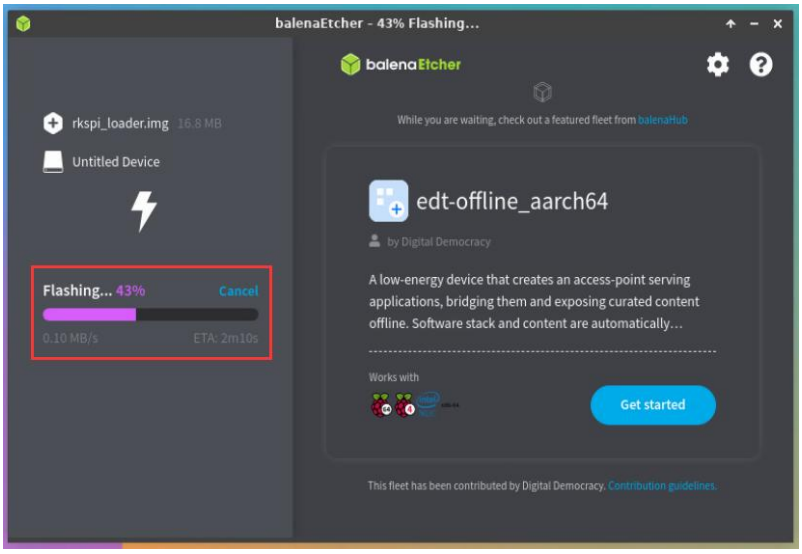
h. 然后点击 **Yes, I'm sure**



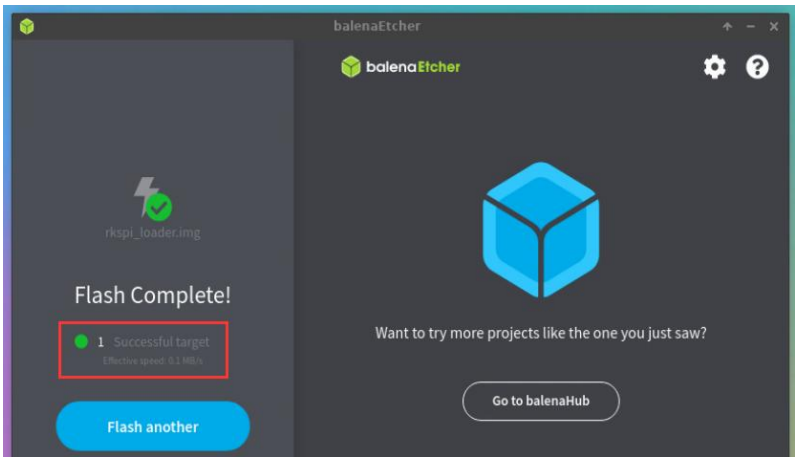
i. 然后输入开发板 Linux 系统的密码 **orangepi**，就会开始烧录 u-boot 镜像到 SPI Flash 中



j. 烧录过程的显示如下所示：

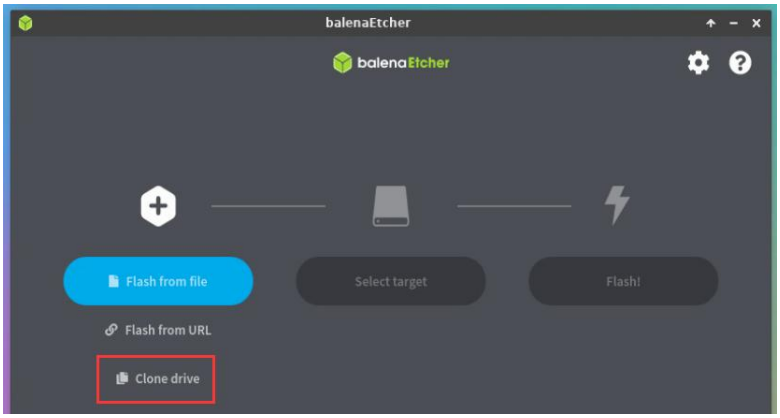


k. 烧录完成后的显示如下所示：

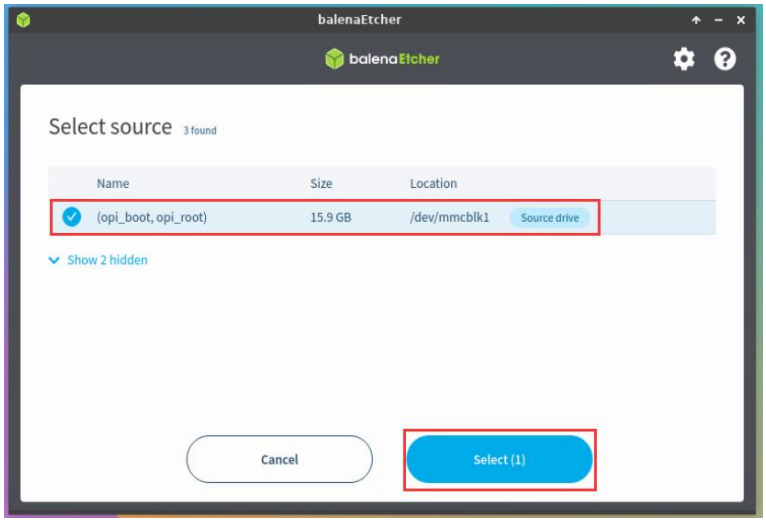


9) 烧录 TF 卡中的 Linux 系统到 NVMe SSD 中的方法（此方法相当于将 TF 卡中的系统克隆到 NVMe SSD）

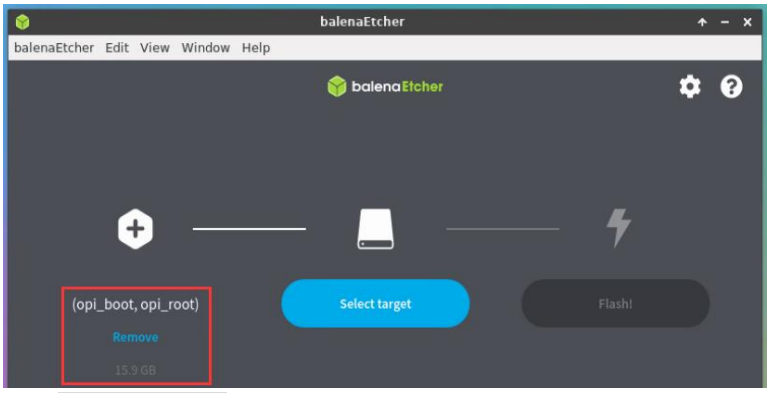
a. 首先点击 **Clone drive**



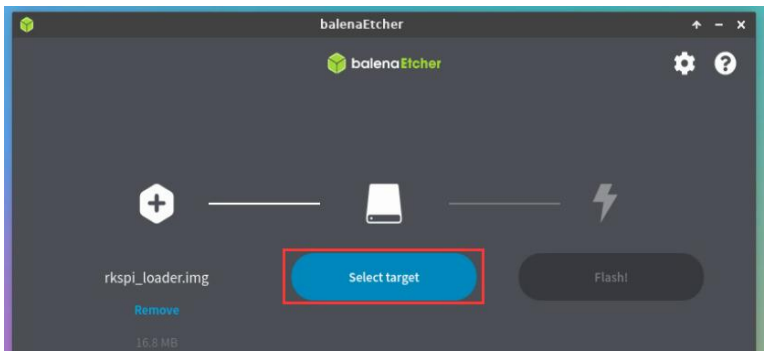
b. 然后选择 TF 卡的设备名**/dev/mmcblk1**



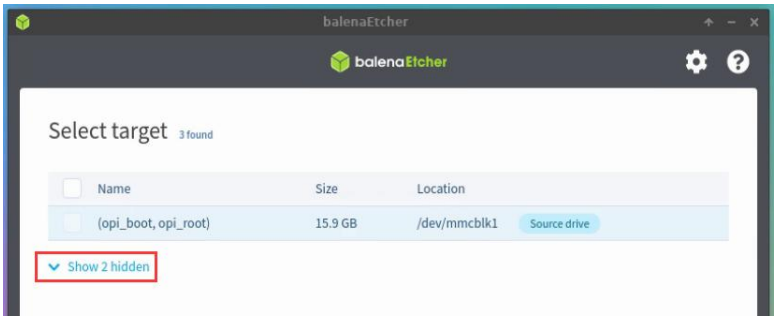
c. 打开 TF 卡后的界面如下所示：



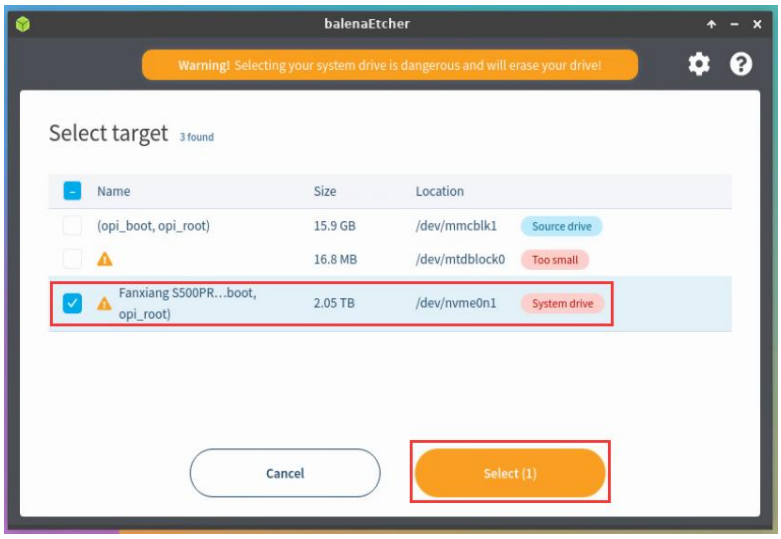
d. 然后点击 **Select target**



e. 然后点击 **Show 2 hidden** 打开更多存储设备的选项

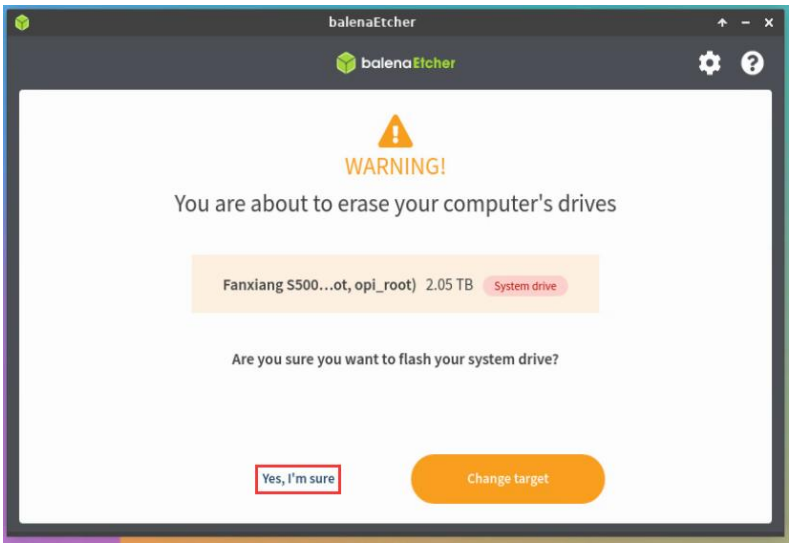


f. 然后选择 NVMe SSD 的设备名**/dev/nvme0n1**,然后点击 **Select**

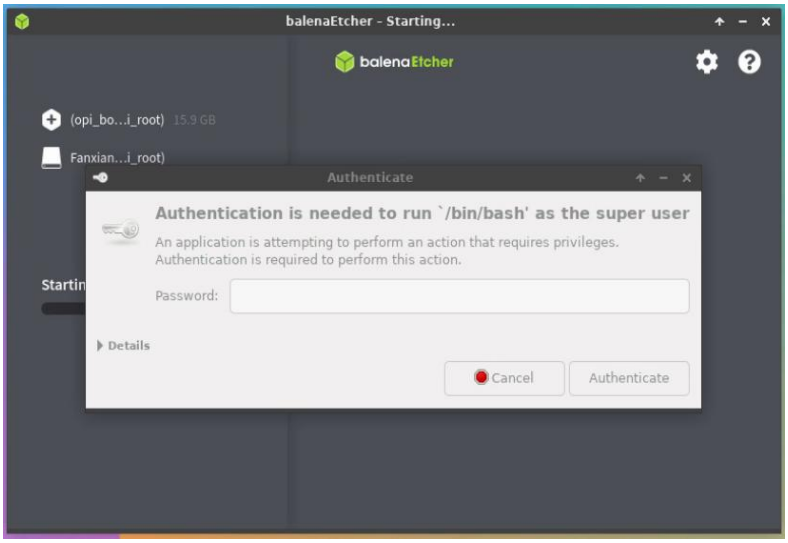


g. 然后点击 **Flash**

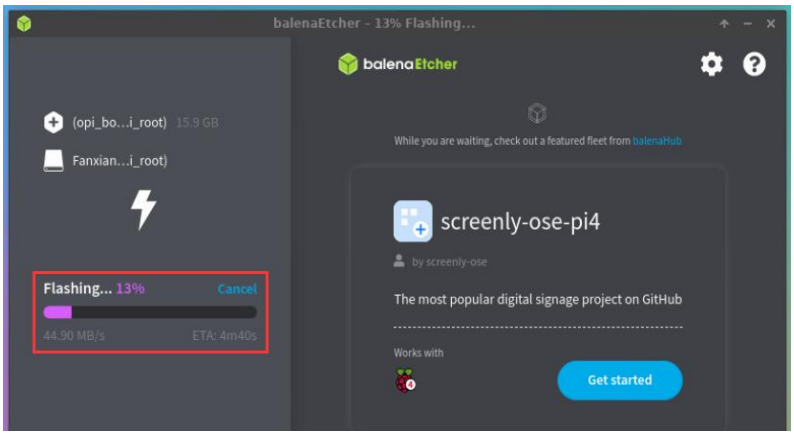
h. 然后点击 **Yes, I'm sure**

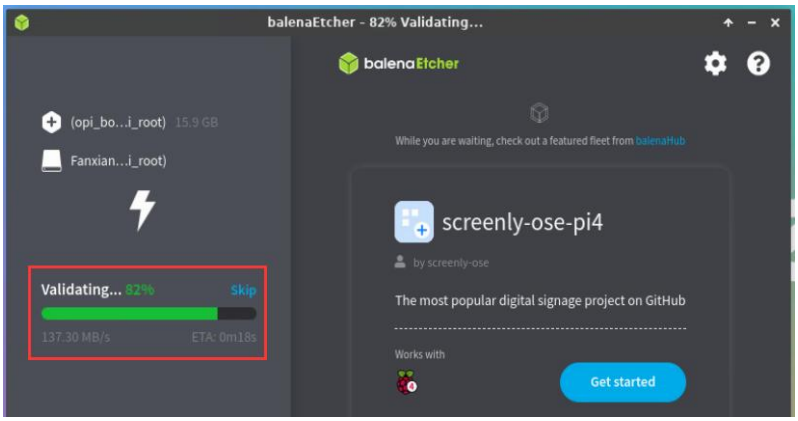


i. 然后输入开发板 Linux 系统的密码 orangepi，就会开始烧录 Linux 镜像到 SSD 中

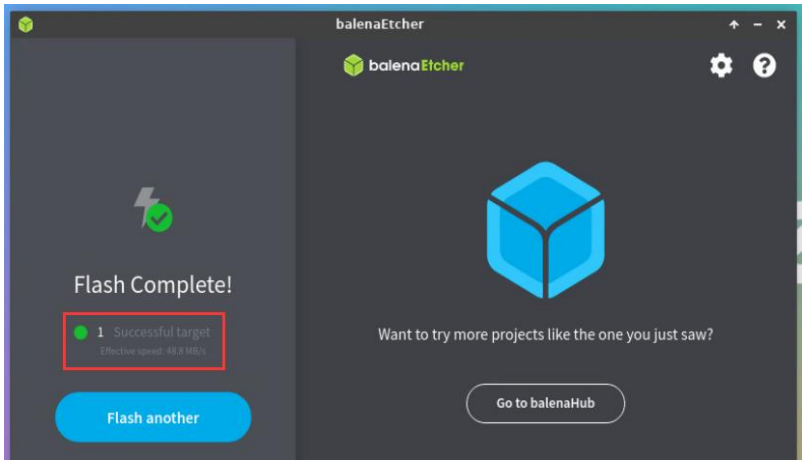


j. 烧录过程的显示如下所示：





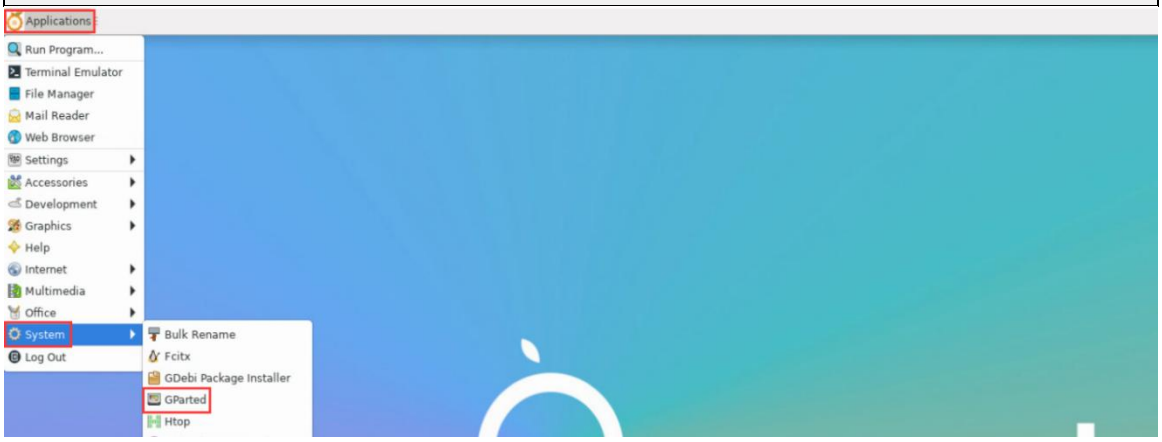
k. 烧录完成后的显示如下所示：



l. 然后需要扩容下 NVMe SSD 中 rootfs 分区的容量，步骤如下所示：

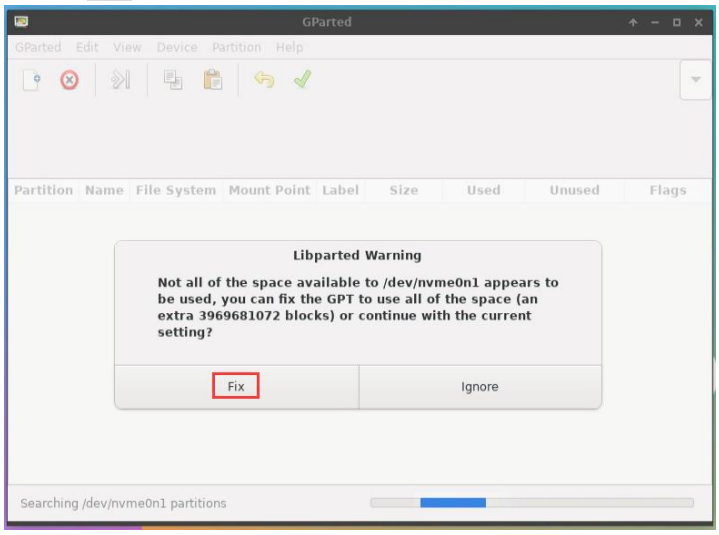
a) 首先打开 **GParted**，如果系统没有预装 Gparted，请使用 apt 命令安装下

```
orangepi@orangepi:~$ sudo apt-get install -y gparted
```

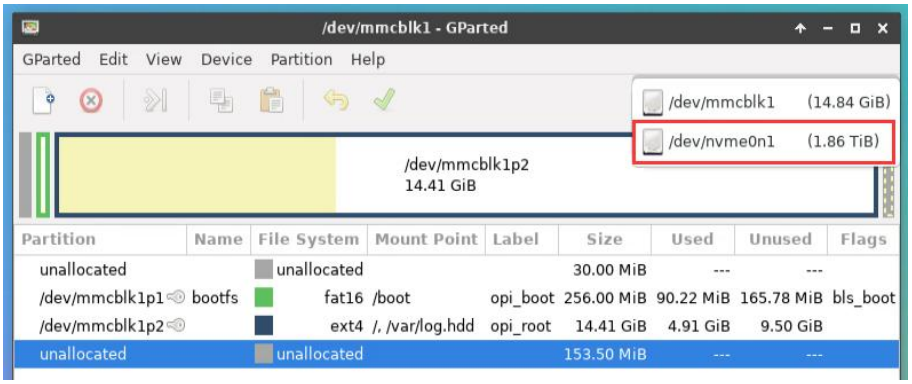


b) 然后输入 Linux 系统的密码 orangepi，再点击 **Authenticate**

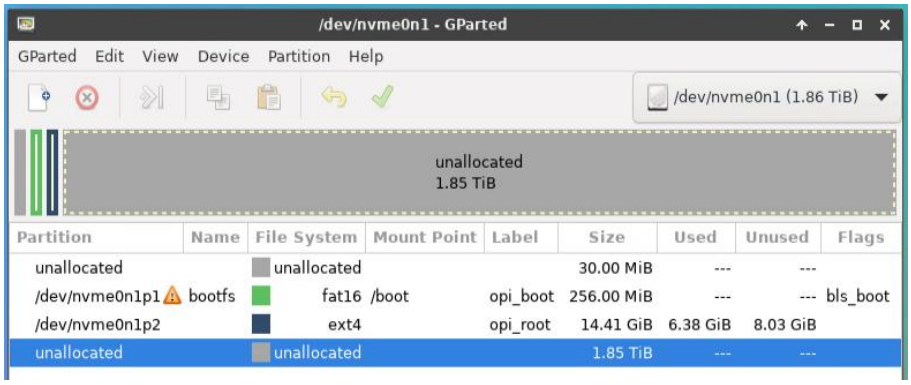
c) 然后点击 **Fix**



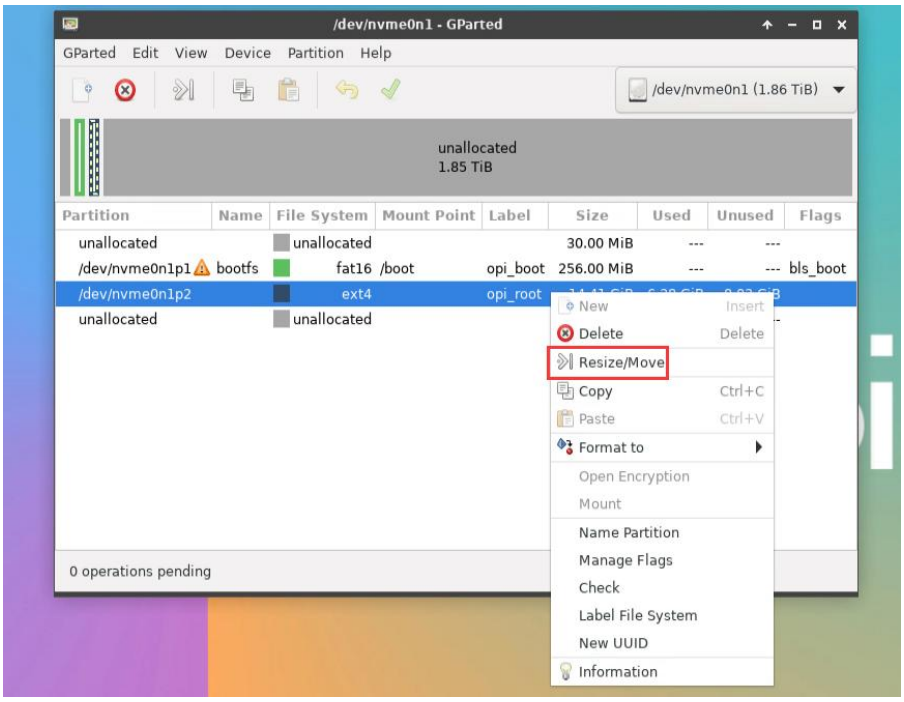
d) 然后选择 NVMe SSD



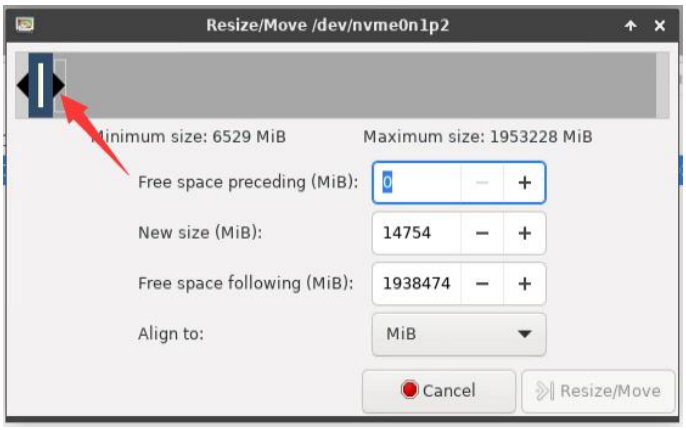
e) 选择 NVMe SSD 后的显示界面如下所所示：



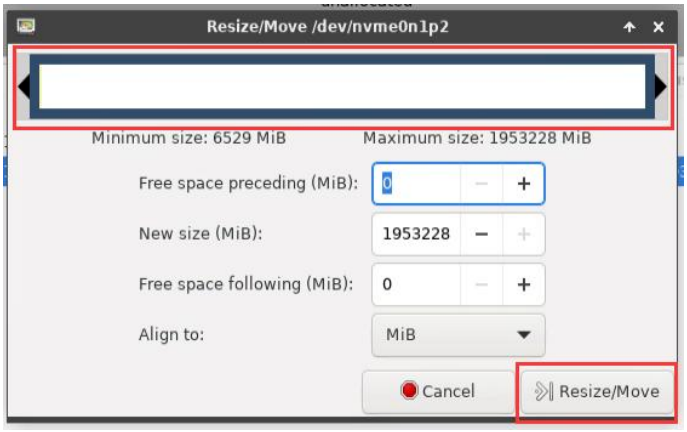
f) 然后选择**/dev/nvme0n1p2** 分区，再击点击右键，再选择 **Resize/Move**



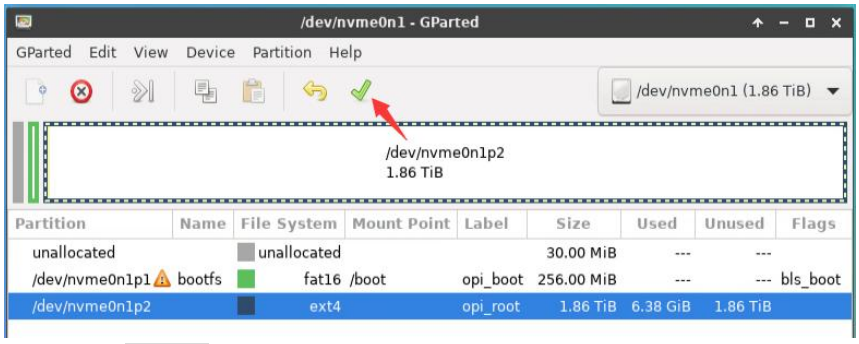
g) 然后在下图所示的位置将容量拖动到最大



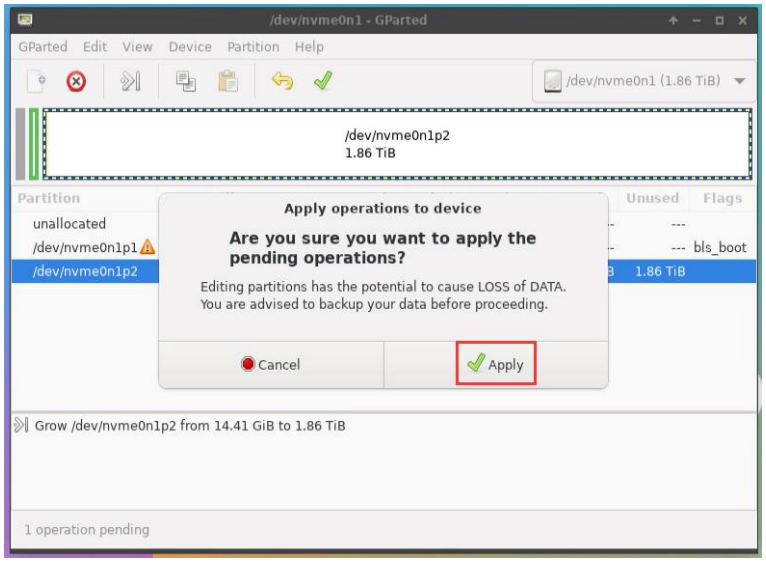
h) 然后点击 **Resize/Move**



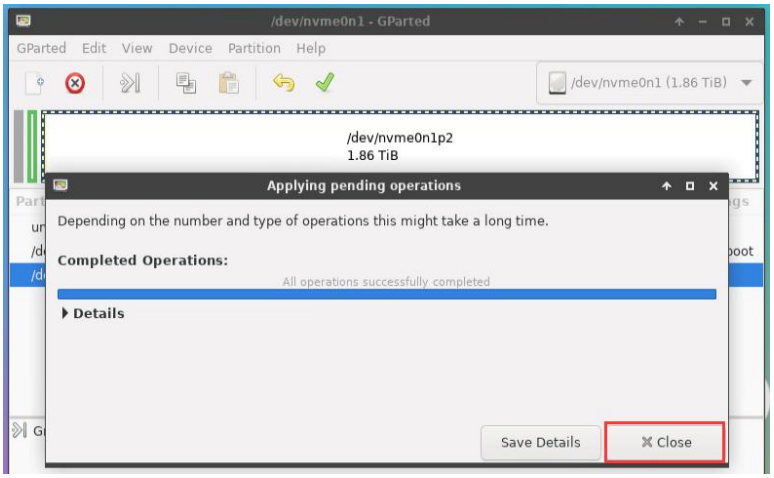
i) 然后点击下图位置绿色的√



j) 再点击 **Apply**



k) 然后点击 **Close** 关闭即可



m. 此时就可以使用 **sudo poweroff** 命令关机了。然后请拔出 TF 卡，再短按电源按键开机，此时就会启动 SPIFlash+NVMe SSD 中的 Linux 系统了。

10) 第 **9)** 步是克隆 TF 卡中的系统到 NMVe SSD 中，我们还可以直接烧录 Linux 镜像文件到 NVMe SSD 中，这里大概说下步骤：

a. 上传 Linux 镜像文件到开发板的 Linux 系统中

b. 然后使用 balenaEtcher 烧录即可

c. **使用这种方法烧录镜像后无需手动扩容，第一次启动会自动扩容。**

2.7. 烧录 Android 镜像到 TF 卡中的方法

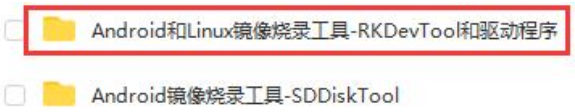
2.7.1. 通过 USB2.0 烧录口将 Android 镜像烧录到 TF 卡中的方法

1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 class10 或以上，建议使用闪迪等品牌的 TF 卡

2) 还需要准备一根品质良好的 USB2.0 公对公数据线

3) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面**首先选择**官方工具**，然后进入下面的文件夹中



b. 然后下载下面的所有文件

4) 然后从 **Orange Pi 的资料下载页面**下载 Android 的镜像。

a. 打开安卓镜像的下载链接后可以看到下面两种类型的安卓镜像，请选择 **TF 卡和 eMMC 启动镜像**文件夹中的镜像进行下载

b. 进入 **TF 卡和 eMMC 启动镜像**文件夹后可以看到下面两个镜像，它们的区别是：

a) 第一个镜像专门用于 HDMI 显示的，支持 4K 显示，如果不使用 LCD 屏幕，请下载不带 lcd 的镜像

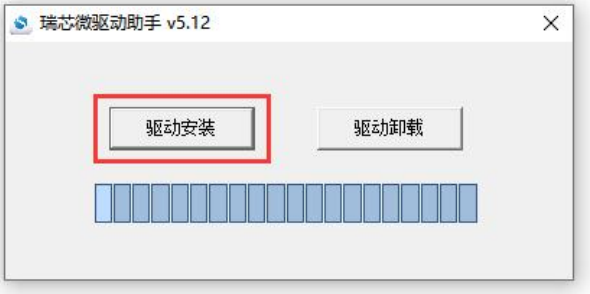
b) 如果要使用 lcd 屏幕，请选择带 lcd 的镜像

5) 然后使用解压软件解压下载的 Android 镜像的压缩包，解压后的文件中，以“.img”结尾的文件就是 Android 镜像文件，大小在 1GB 以上

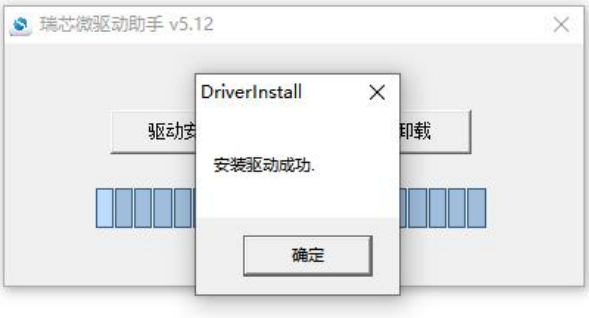
6) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

7) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

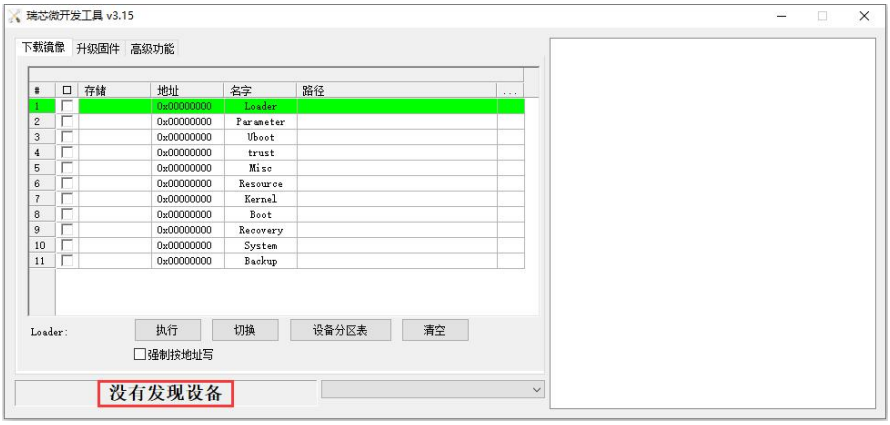


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



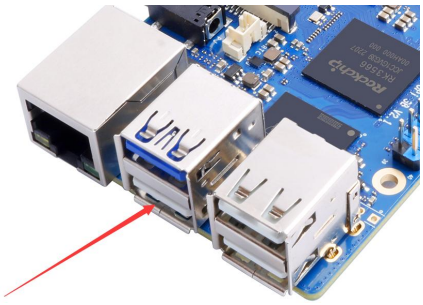
8) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

9) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



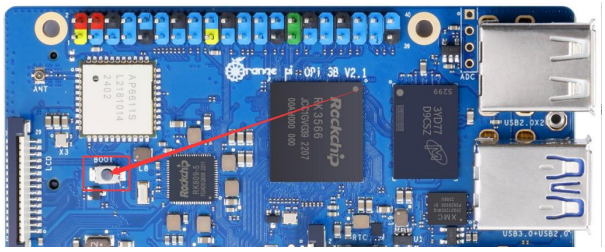
10) 然后开始烧录 Android 镜像到 TF 卡中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录接口的位置如下图所示

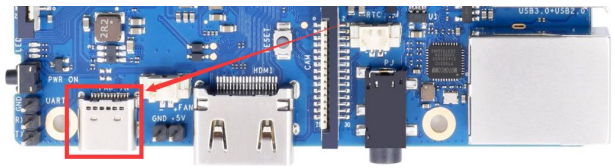


b. 然后确保开发板没有插入 TF 卡，没有连接电源

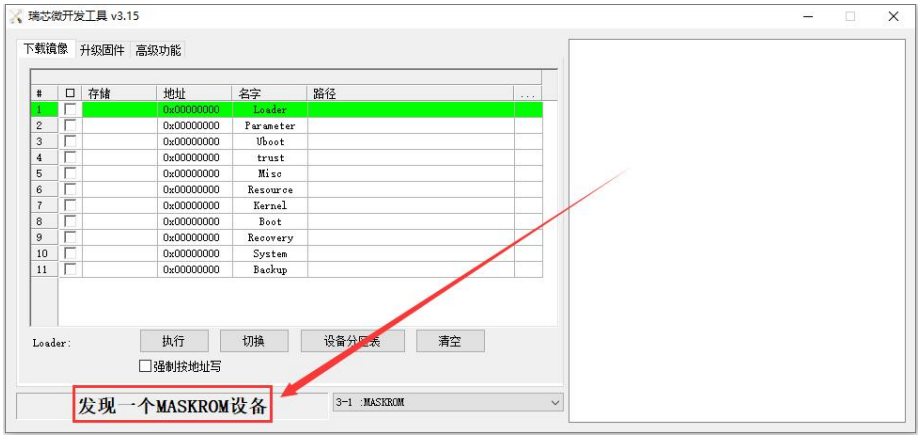
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



d. 然后给开发板接上 Type-C 接口的电源，并上电

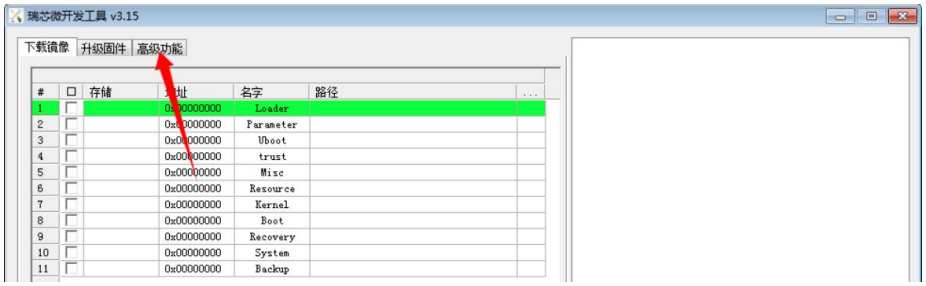


e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”

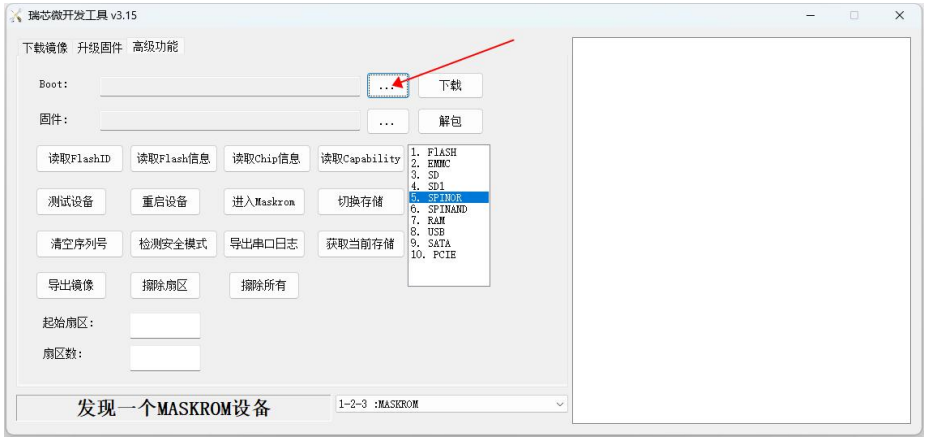


f. 然后再插入 TF 卡到开发板

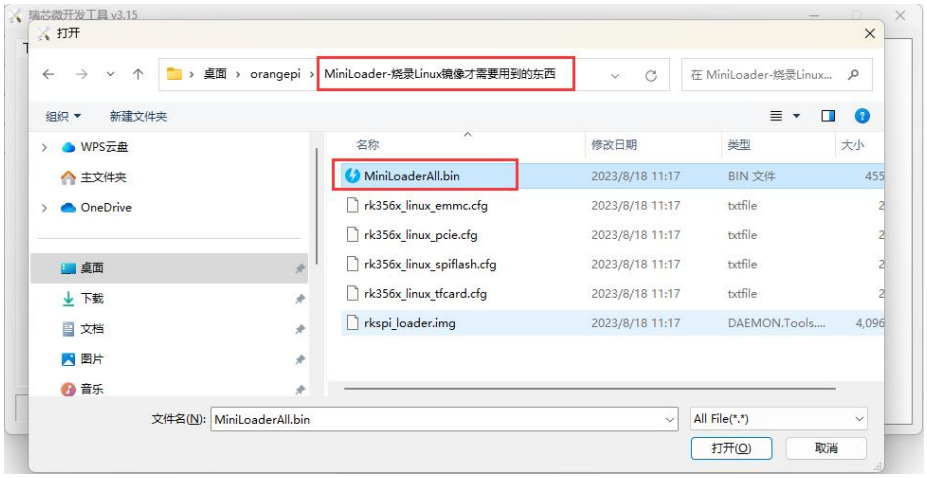
g. 然后请选择**高级功能**



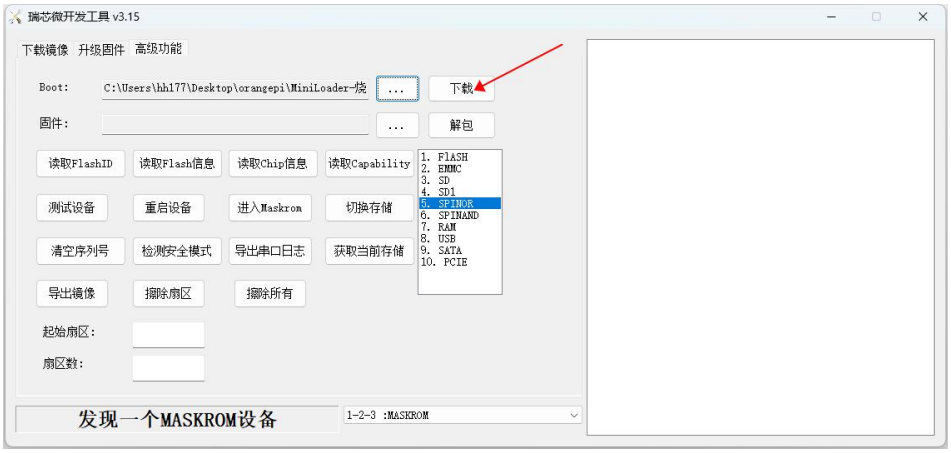
h. 然后点击下图所示的位置



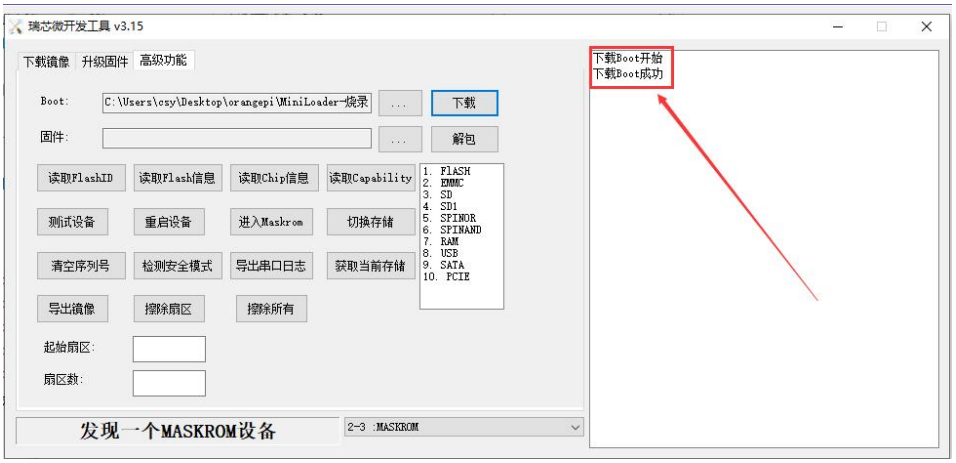
i. 再选择前面下载的 **MiniLoader** 文件夹中的 **MiniLoaderAll.bin**，再点击打开



j. 然后点击**下载**

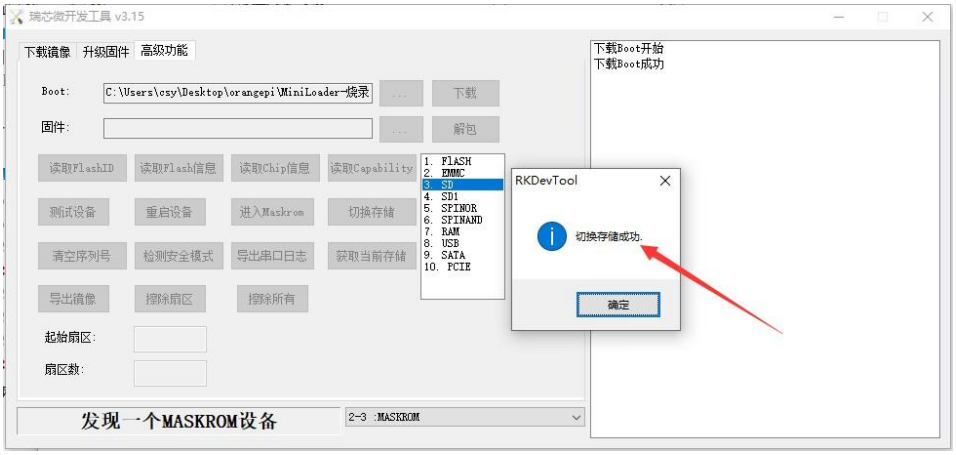


k. 下载完 **MiniLoaderAll.bin** 后的显示如下图所示



l. 然后选择存储设备为 **SD**，再点击**切换存储**

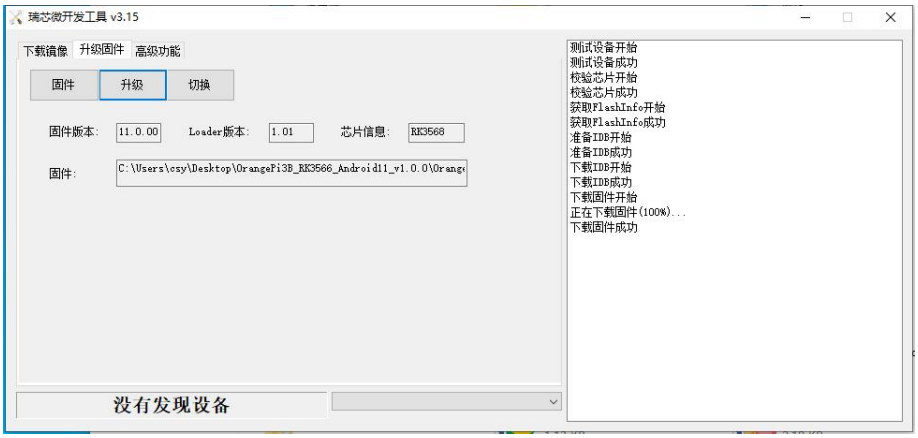
m. 切换成功的显示如下图所示



n. 然后点击烧录工具的“**升级固件**”一栏

o. 接着点击“**固件**”按钮选择需要烧录的 Android 镜像的路径

p. 最后点击“**升级**”按钮就会开始烧录，烧录过程中的 log 如下图所示。烧录完成后 Android 系统会自动启动。



2.7.2. 使用 SDDiskTool 工具将 Android 镜像烧到 TF 卡的方法

此方法不推荐使用，有些 TF 卡容易出现卡在开机 LOGO 无法启动的问题

1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 或以上，建议使用闪迪等品牌的 TF 卡

2) 然后使用读卡器把 TF 卡插入电脑

3) 然后从 **Orange Pi 的资料下载页面**下载 SDDiskTool 烧写工具，**请确保 SDDiskTool 工具的版本为最新的 v1.72**

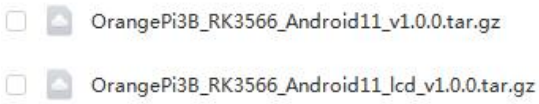
4) 然后从 **Orange Pi 的资料下载页面**下载 Android11 的镜像

a. 打开安卓镜像的下载链接后可以看到下面两种类型的安卓镜像，请选择 **TF 卡和 eMMC 启动镜像**文件夹中的镜像进行下载

b. 进入 **TF 卡和 eMMC 启动镜像**文件夹后可以看到下面两个镜像，它们的区别是：

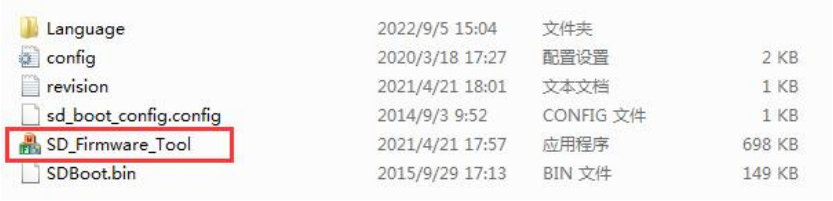
a) 不带 lcd 的镜像是专门用于 HDMI 显示的，支持 4K 显示，如果不使用 LCD 屏幕，请下载不带 lcd 的镜像

b) 如果要使用 LCD 屏幕，请选择带 lcd 的镜像

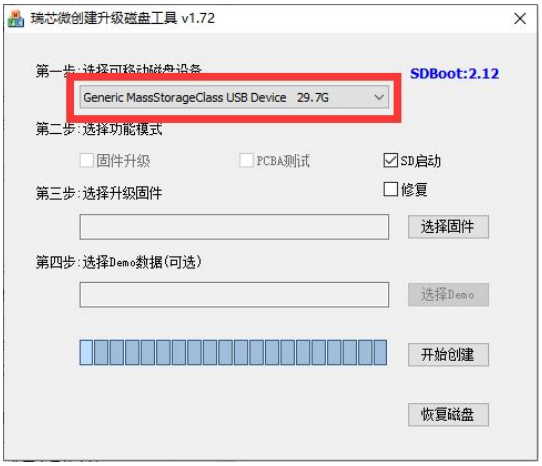


5) 然后使用解压软件解压下载的 Android 镜像的压缩包，解压后的文件中，以“.img”结尾的文件就是 Android 镜像文件，大小在 1GB 以上

6) 然后使用解压软件解压 **SDDiskTool_v1.72.zip**，此软件无需安装，在解压后的文件夹中找到 **SD_Firmware_Tool.exe** 打开即可



7) 打开 **SDDiskTool** 后，如果 TF 卡识别正常，会在“**选择可移动磁盘设备**”一栏中显示插入的磁盘设备，**请务必确认显示的磁盘设备和你想烧录的 TF 卡的盘符是一致的**，如果没有显示可以尝试拔插下 TF 卡



8) 确认完盘符后，可以先格式化下 TF 卡，点击 **SDDiskTool** 中的**恢复磁盘**按钮即可，也可使用前面提到的 **SD Card Formatter** 进行 TF 卡的格式化

9) 然后开始将 Android 镜像写入 TF 卡

a. 首先在“**选择功能模式**”中勾选“**SD 启动**”

b. 然后在“**选择升级固件**”一栏中选择 Android 镜像的路径

c. 最后点击“**开始创建**”按钮就会开始烧录 Android 镜像到 TF 卡中

10) 烧录完后即可退出 SDDiskTool 软件，然后就可以把 TF 卡从电脑中拔出来插到开发板中启动了

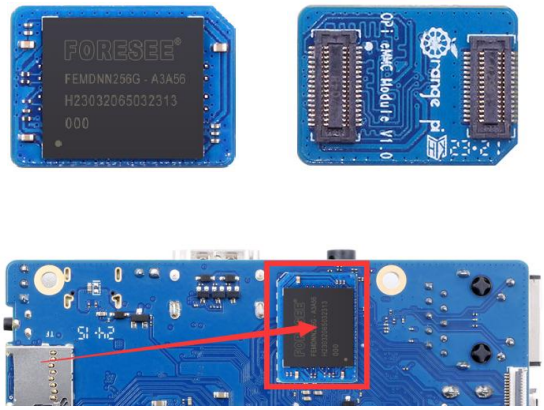
2.8. 烧录 Android 镜像到 eMMC 中的方法

注意，烧录镜像到eMMC中后如果测试发现启动不了，请清空下SPIFlash后再启动试下。清空SPIFlash的方法请参考使用RKDevTool清空SPIFlash的方法小节的内容。

2.8.1. 通过 USB2.0 烧录口将 Android 镜像烧录到 eMMC 中的方法

注意，下面所有的操作都是在 Windows 电脑中进行的。

1) 开发板预留了 eMMC 的扩展接口，烧录系统到 eMMC 前，首先需要购买一个与开发板 eMMC 接口相匹配的 eMMC 模块。然后将 eMMC 模块安装到开发板上。eMMC 模块和插入开发板的方法如下所示：



2) 还需要准备一根品质良好的 USB2.0 公对公数据线

3) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和烧录工具 **RKDevTool_Release_v3.15.zip**

4) 然后从 **Orange Pi 的资料下载页面**下载 Android 的镜像。

a. 打开安卓镜像的下载链接后可以看到下面两种类型的安卓镜像，请选择 **TF 卡和 eMMC 启动镜像**文件夹中的镜像进行下载

b. 进入 **TF 卡和 eMMC 启动镜像**文件夹后可以看到下面两个镜像，它们的区别是：

a) 第一个镜像专门用于 HDMI 显示的，支持 4K 显示，如果不使用 LCD 屏幕，请下载不带 lcd 的镜像

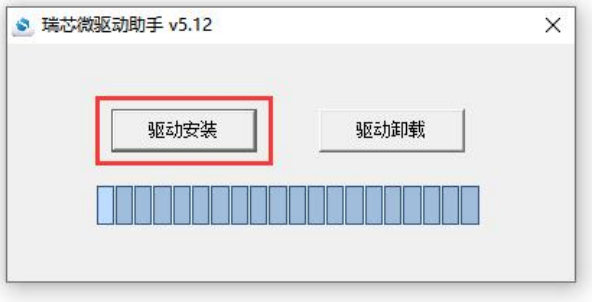
b) 如果要使用 lcd 屏幕，请选择带 lcd 的镜像

5) 然后使用解压软件解压下载的 Android 镜像的压缩包，解压后的文件中，以“.img”结尾的文件就是 Android 镜像文件，大小在 1GB 以上

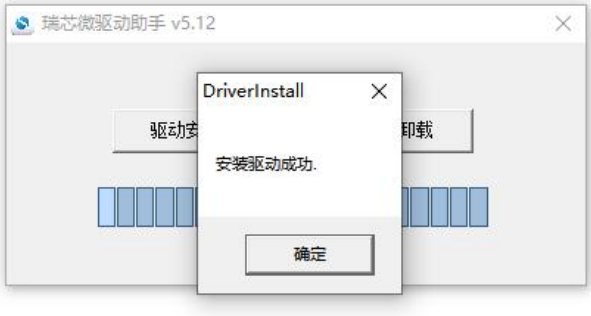
6) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

7) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

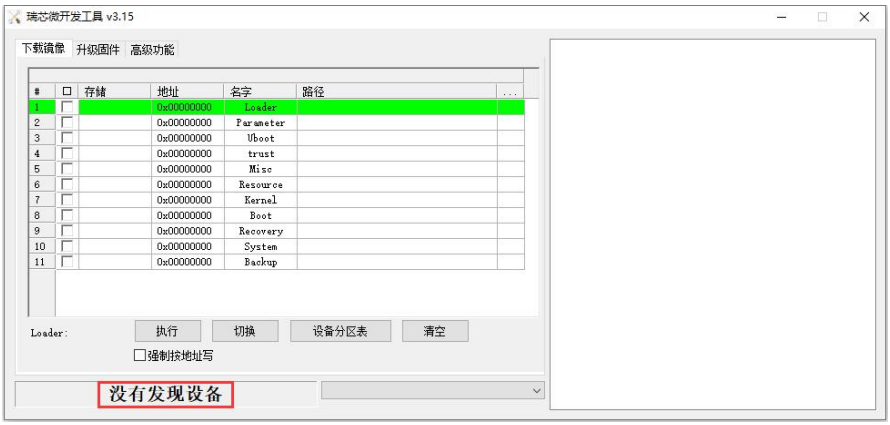


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



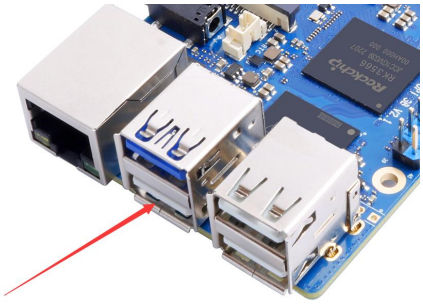
8) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

9) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



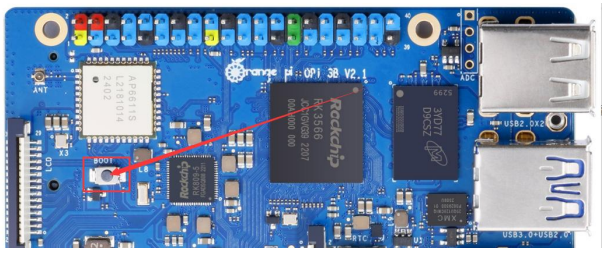
10) 然后开始烧录 Android 镜像到 eMMC 中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录接口的位置如下图所示

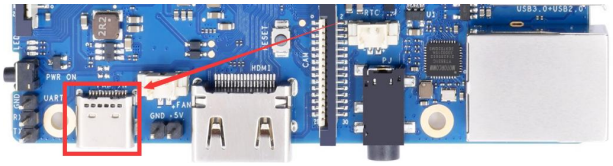


b. 然后确保开发板没有插入 TF 卡，没有连接电源

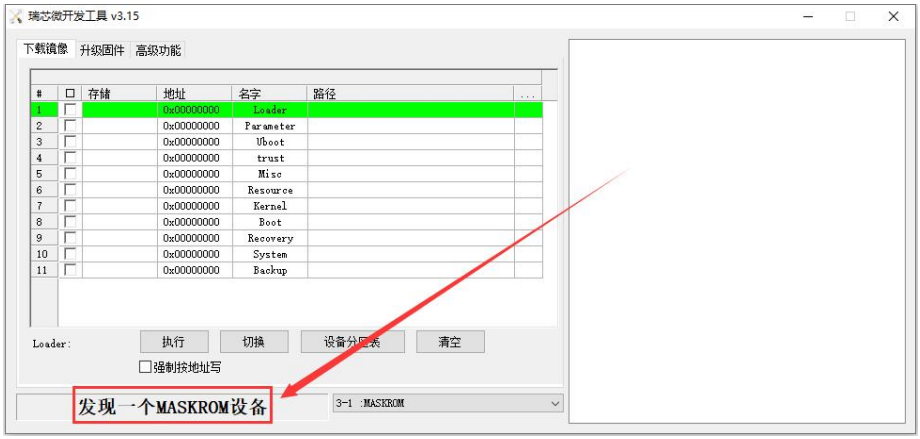
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



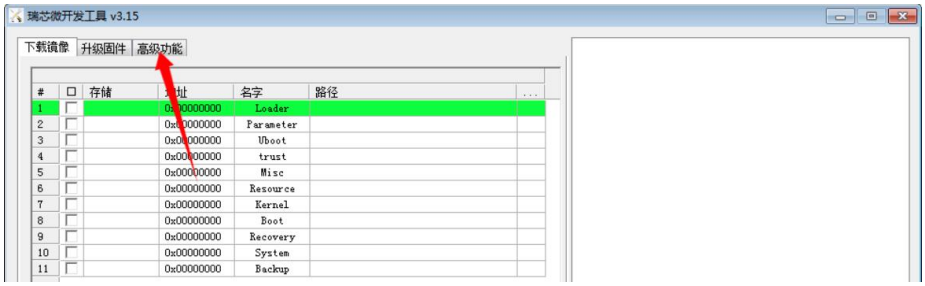
d. 然后给开发板接上 Type-C 接口的电源，并上电



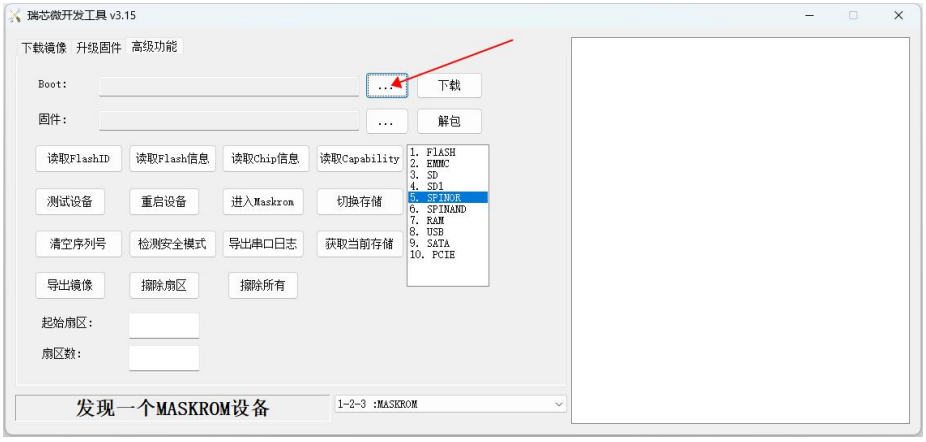
e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”



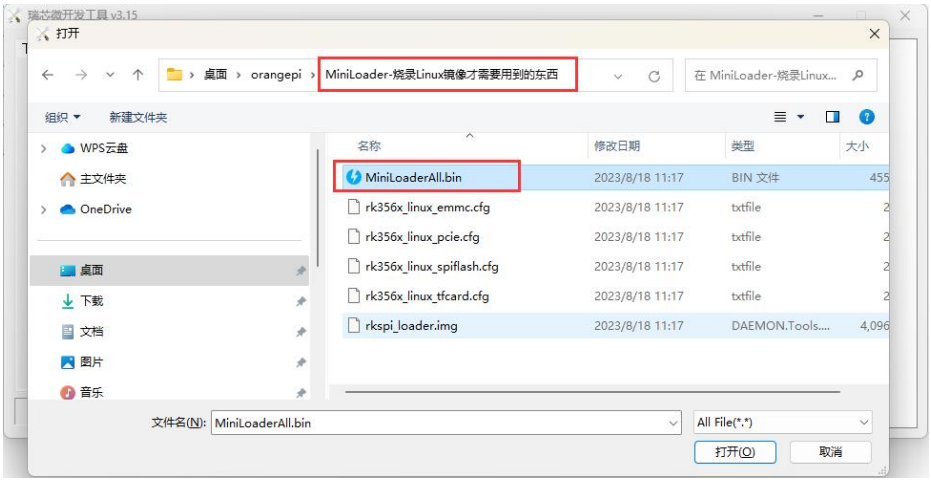
f. 然后请选择**高级功能**



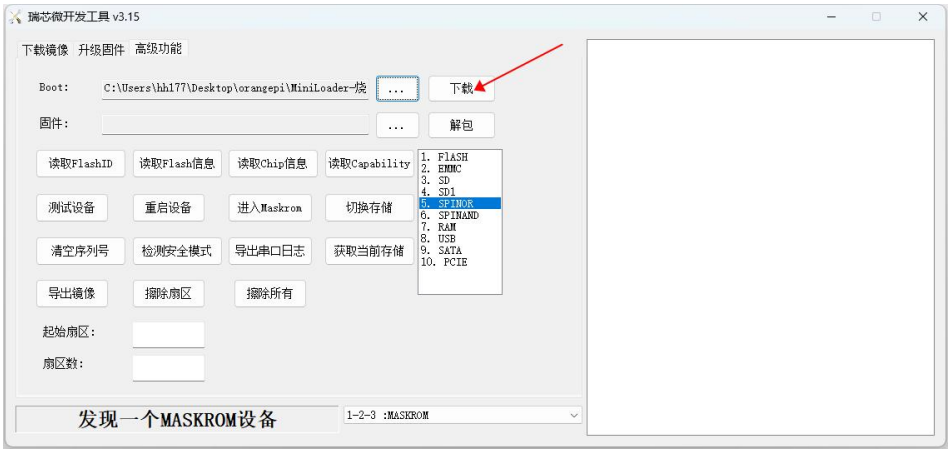
g. 然后点击下图所示的位置



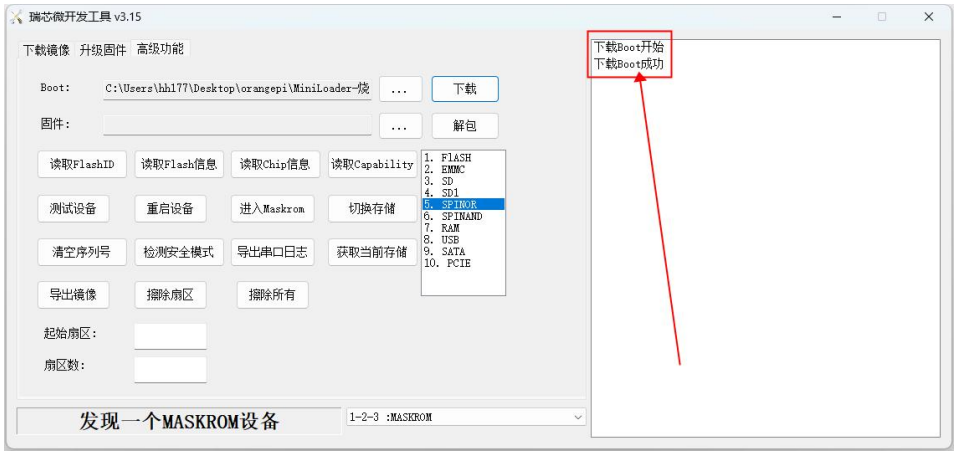
h. 再选择前面下载的 **MiniLoader** 文件夹中的 **MiniLoaderAll.bin**，再点击打开



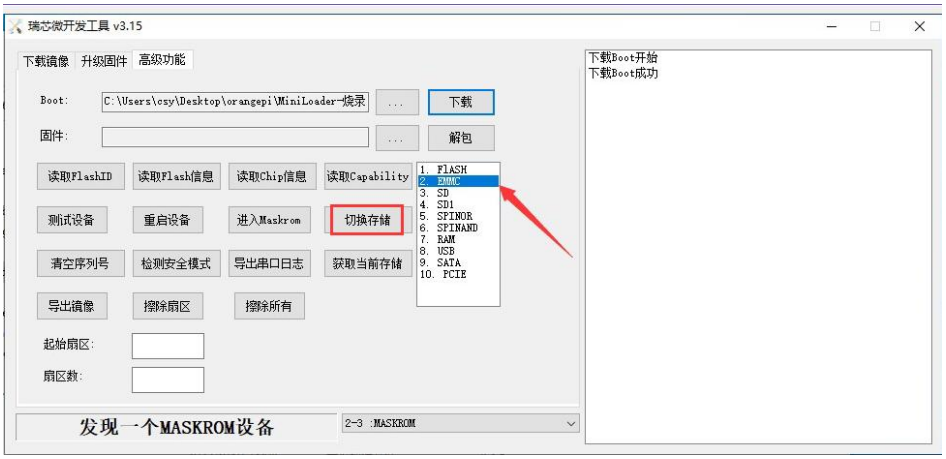
i. 然后点击**下载**



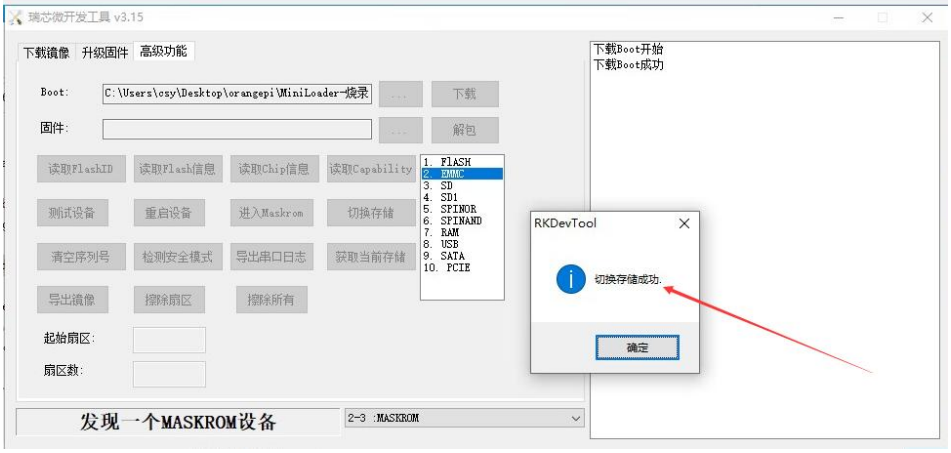
j. 下载完 **MiniLoaderAll.bin** 后的显示如下图所示



k. 然后选择存储设备为 **EMMC，** 再点击**切换存储**



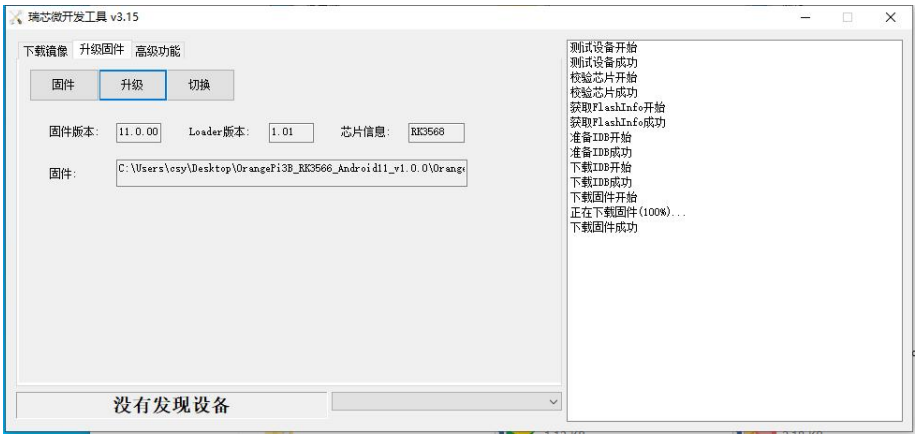
l. 切换成功的显示如下图所示



m. 然后点击烧录工具的“**升级固件**”一栏

n. 接着点击“**固件**”按钮选择需要烧录的 Android 镜像的路径

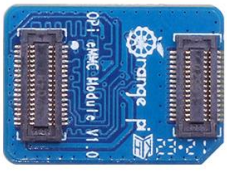
o. 最后点击“**升级**”按钮就会开始烧录，烧录过程中的 log 如下图所示。烧录完成后 Android 系统会自动启动。

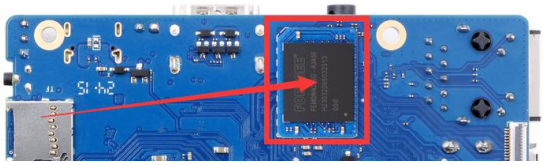


2.8.2. 通过 TF 卡将 Android11 镜像烧录到 eMMC 中的方法

注意，下面所有的操作都是在 Windows 电脑中进行的。

1) 开发板预留了 eMMC 的扩展接口，烧录系统到 eMMC 前，首先需要购买一个与开发板 eMMC 接口相匹配的 eMMC 模块。然后将 eMMC 模块安装到开发板上。eMMC 模块和插入开发板的方法如下所示：





2) 还需要准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 或以上，建议使用闪迪等品牌的 TF 卡

3) 然后使用读卡器把 TF 卡插入电脑

4) 然后从 **Orange Pi 的资料下载页面** 下载 SDDiskTool 烧写工具，**请确保 SDDiskTool 工具的版本为最新的 v1.72**

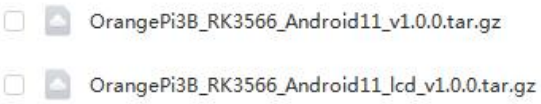
5) 然后从 **Orange Pi 的资料下载页面** 下载 Android 的镜像

a. 打开安卓镜像的下载链接后可以看到下面两种类型的安卓镜像，请选择 **TF 卡和 eMMC 启动镜像** 文件夹中的镜像进行下载

b. 进入 **TF 卡和 eMMC 启动镜像** 文件夹后可以看到下面两个镜像，它们的区别是：

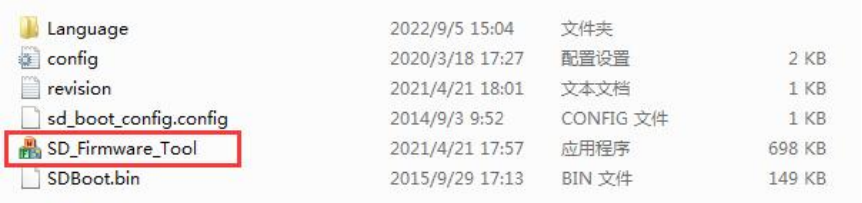
a) 第一个镜像专门用于 HDMI 显示的，支持 4K 显示，如果不使用 LCD 屏幕，请下载不带 lcd 的镜像

b) 如果要使用 lcd 屏幕，请选择带 lcd 的镜像



6) 然后使用解压软件解压下载的 Android 镜像的压缩包，解压后的文件中，以“.img”结尾的文件就是 Android 镜像文件，大小在 1GB 以上

7) 然后使用解压软件解压 **SDDiskTool_v1.72.zip**，此软件无需安装，在解压后的文件夹中找到 **SD_Firmware_Tool.exe** 打开即可

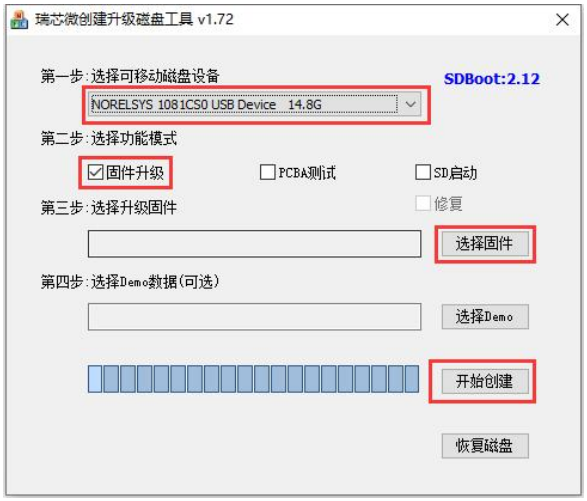


8) 打开 **SDDiskTool** 后，如果 TF 卡识别正常，会在“**选择可移动磁盘设备**”一栏中显示插入的磁盘设备，**请务必确认显示的磁盘设备和你想烧录的 TF 卡的盘符是一致的**，如果没有显示可以尝试拔插下 TF 卡

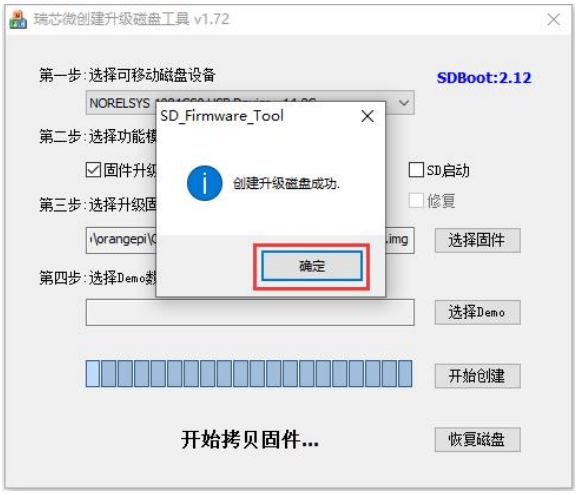
9) 确认完盘符后，可以先格式化下 TF 卡，点击 **SDDiskTool** 中的**恢复磁盘**按钮即可，也可使用前面提到的 **SD Card Formatter** 进行 TF 卡的格式化

10) 然后开始将 Android 镜像写入 TF 卡

a. 首先在**“选择可移动磁盘设备”**下面确认显示的盘符为 TF 卡对应的盘符
b. 然后在“**选择功能模式**”中选择**“固件升级”**
c. 然后在“**选择升级固件**”一栏中选择 Android 固件的路径
d. 最后点击“**开始创建**”按钮就会开始烧录

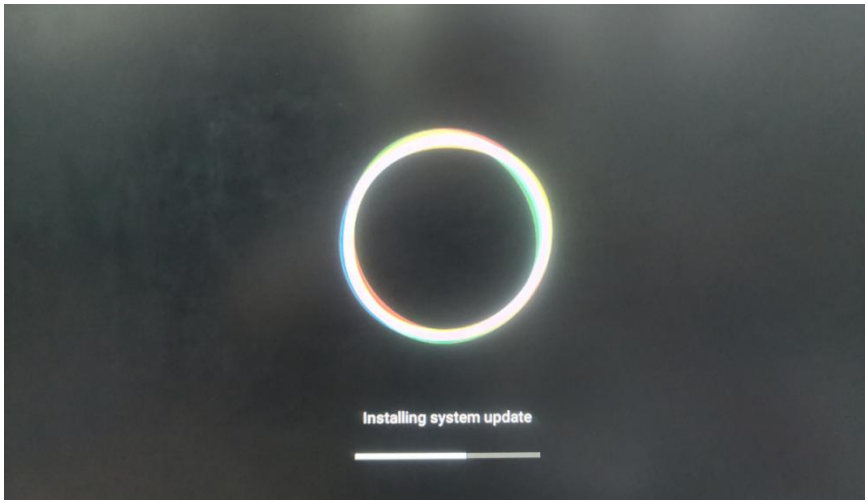


11) 烧录完成后的显示如下图所示，然后就可以退出 SDDiskTool

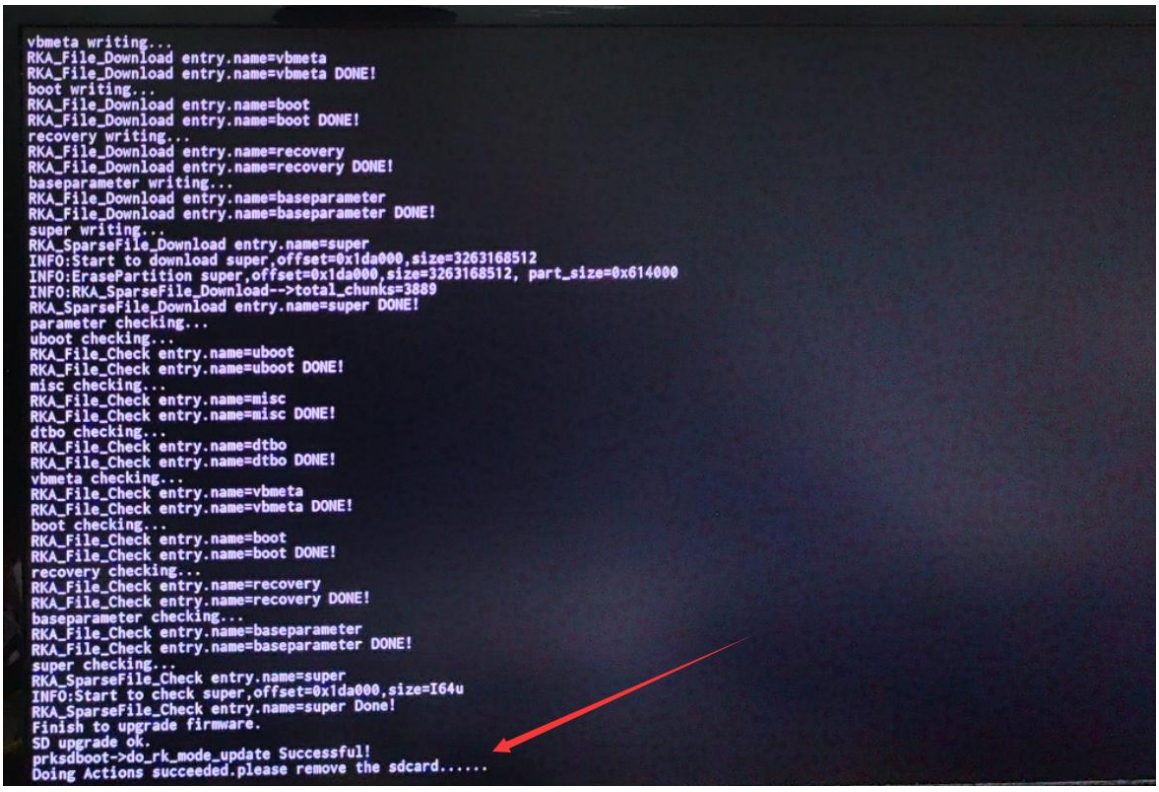


12) 然后把 TF 卡从电脑中拔出来插到开发板中，开发板上电启动后就会自动开始将 TF 卡中的 Android 镜像烧录到开发板的 eMMC 中

13) 如果开发板连接了 HDMI 显示器，还可以从 HDMI 显示器中看到烧录 Android 镜像到 eMMC 中的进度条



14) 当 HDMI 显示器显示如下信息时，说明烧录 Android 镜像到 eMMC 中已完成，此时就可以拔出 TF 卡，然后 eMMC 中的安卓系统就会开始启动。

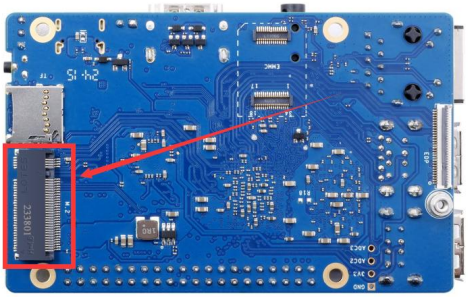


2.9. 烧录 Android 镜像到 SPIFlash+NVMe SSD 中的方法

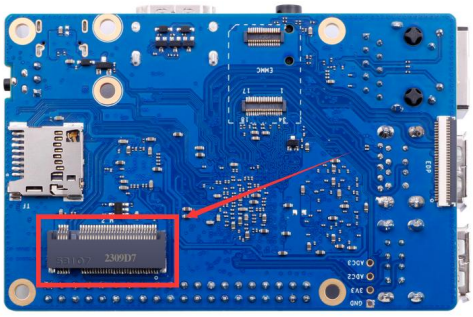
注意，下面所有的操作都是在 Windows 电脑中进行的。

1) 首先需要准备一个 NVMe SSD 固态硬盘

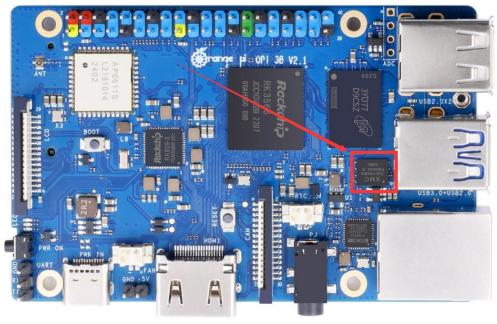
2) 然后把 NVMe SSD 插入开发板的 M.2 PCIe 接口，并固定好，Orange Pi 3B 有两个硬件版本，v2.1 版本 M.2 PCIe 接口位置如下图所示：



v1.1.1 版本 M.2 PCIe 接口位置如下图所示：



3) SPI Flash 在开发板上的位置如下图所示，开始烧录前无需其他设置



4) 还需要准备一根品质良好的 USB2.0 公对公数据线

5) 然后从 **Orange Pi 的资料下载页面** 下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和烧录工具 **RKDevTool_Release_v3.15.zip**

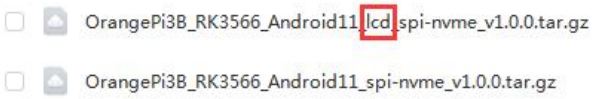
6) 然后下载 Android11 的镜像

a. 打开安卓镜像的下载链接后可以看到下面两种类型的安卓镜像，请选择 **SPIFlash-NVME SSD 启动镜像** 文件夹中的镜像进行下载

b. 进入 **SPIFlash-NVME SSD 启动镜像** 文件夹后可以看到下面两个镜像，它们的区别是：

a) 不带 lcd 的镜像是专门用于 HDMI 显示的，支持 4K 显示，如果不使用 LCD 屏幕，请下载不带 lcd 的镜像

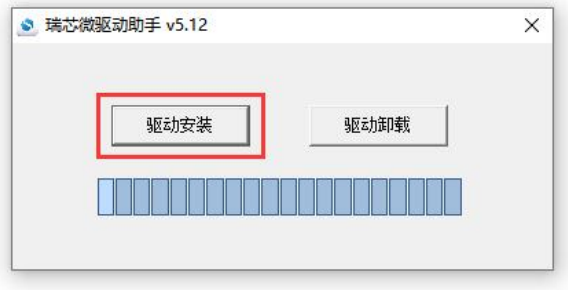
b) 如果要使用 LCD 屏幕，请选择带 lcd 的镜像



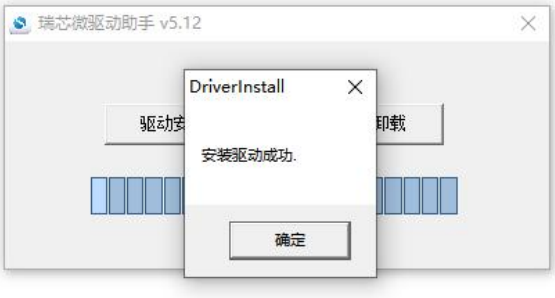
7) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

8) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

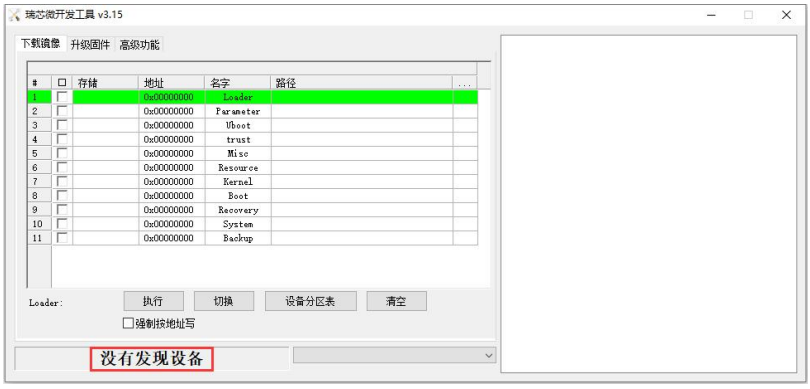


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



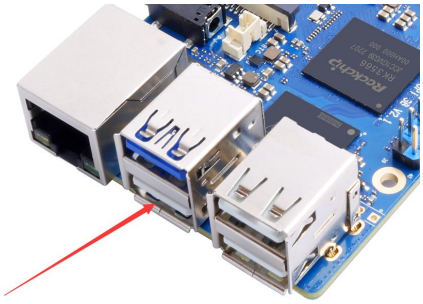
9) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

10) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



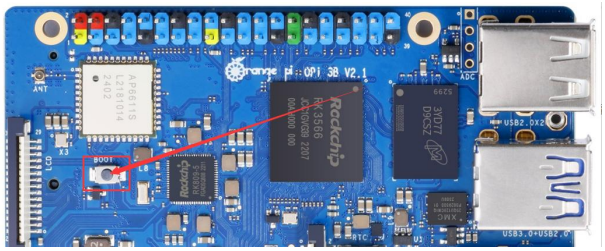
11) 然后开始烧录 Android 镜像到 SPIFlash+NVMe SSD 中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录口的位置如下图所示

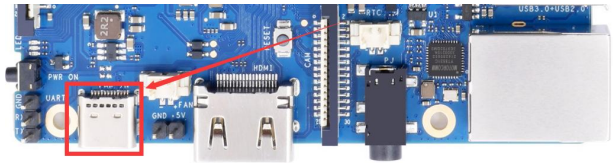


b. 确保开发板没有插入 TF 卡，没有连接电源

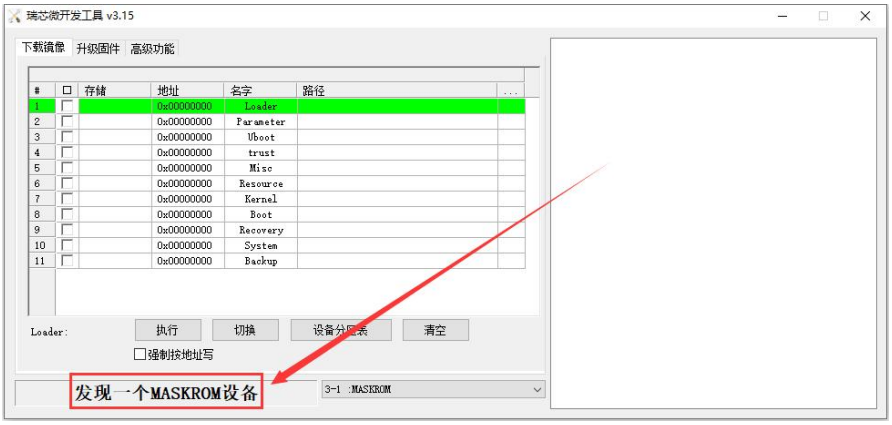
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



d. 然后给开发板接上 Type-C 接口的电源，并上电，然后就可以松开 MaskROM 按键了



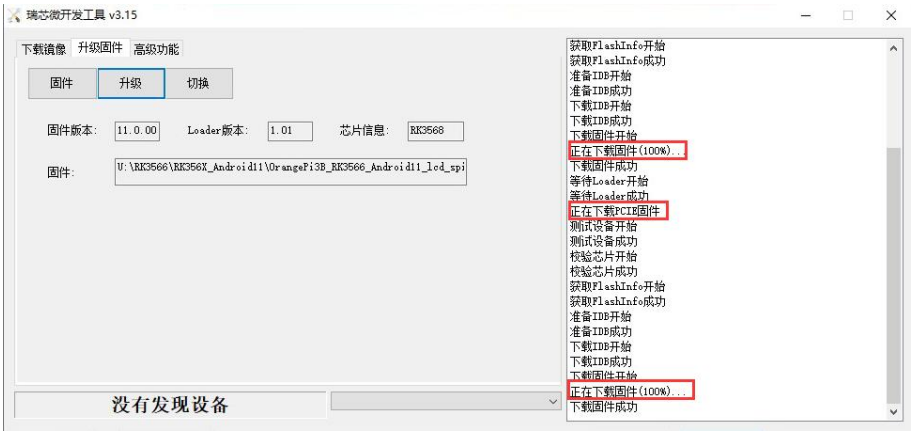
e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”



f. 然后点击烧录工具的“**升级固件**”一栏

g. 接着点击“**固件**”按钮选择需要烧录的 Android 镜像

h. 最后点击“**升级**”按钮就会开始烧录，烧录的过程如下图所示，可以看到，首先会烧录固件到 SPIFlash 中，然后烧录固件到 PCIE 中。烧录完成后 Android 系统会自动启动。



2.10. 烧录 Orange Pi OS（OH）镜像到 TF 卡中的方法

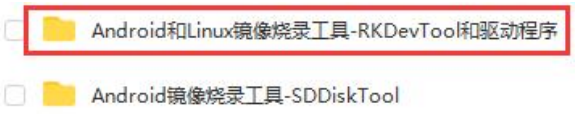
注意，下面所有的操作都是在 Windows 电脑中进行的。

1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 class10 或以上，建议使用闪迪等品牌的 TF 卡

2) 还需要准备一根品质良好的 USB2.0 公对公数据线

3) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面**首先选择**官方工具**，然后进入下面的文件夹中



b. 然后下载下面的所有文件

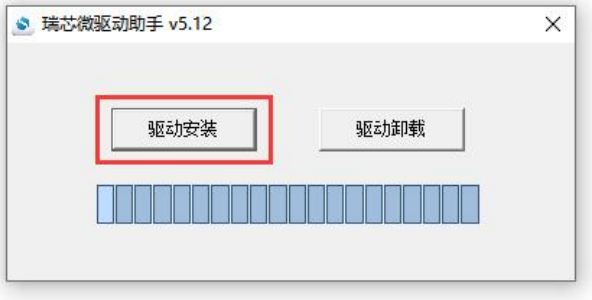
4) 然后从 **Orange Pi 的资料下载页面**下载 OPi OS（OH）的镜像

5) 然后使用解压软件解压下载的 OPi OS（OH）镜像的压缩包

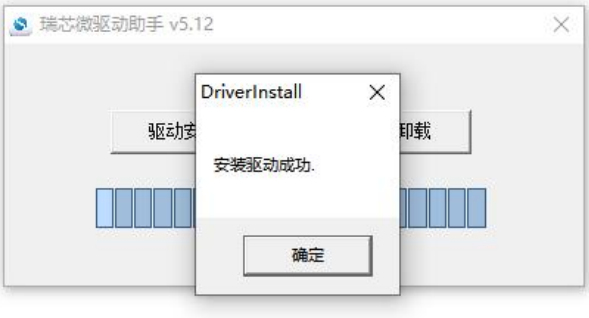
6) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

7) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

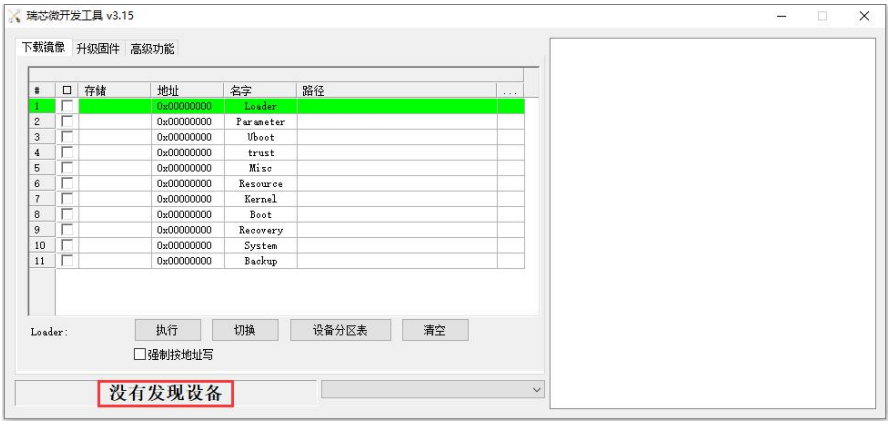


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



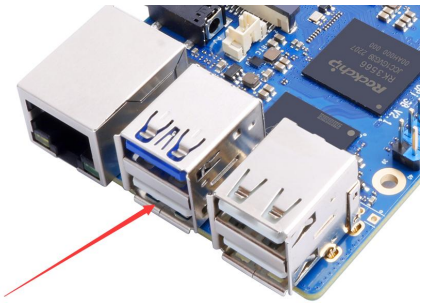
8) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

9) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



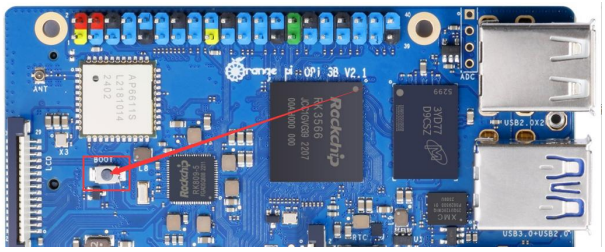
10) 然后开始烧录 OPi OS（OH）镜像到 TF 卡中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录接口的位置如下图所示

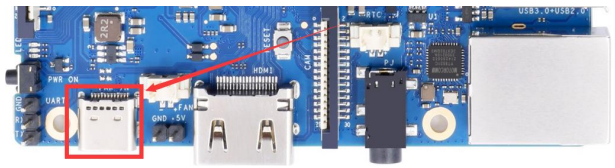


b. **然后确保开发板没有插入 TF 卡，没有连接电源**

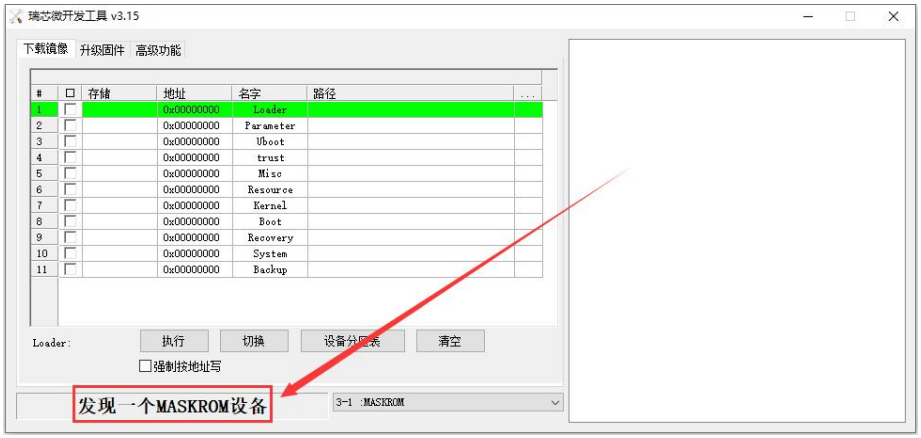
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



d. 然后给开发板接上 Type-C 接口的电源，并上电

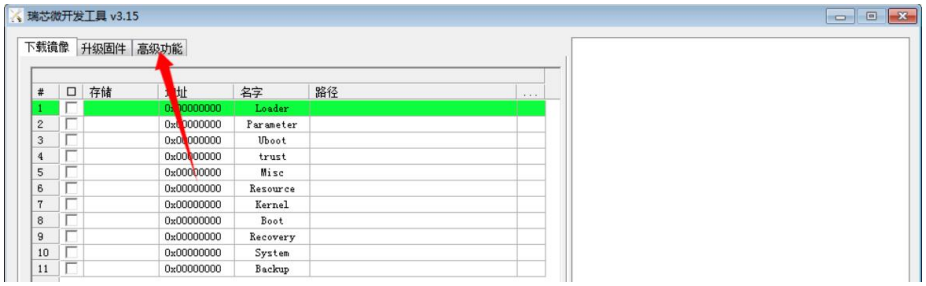


e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”

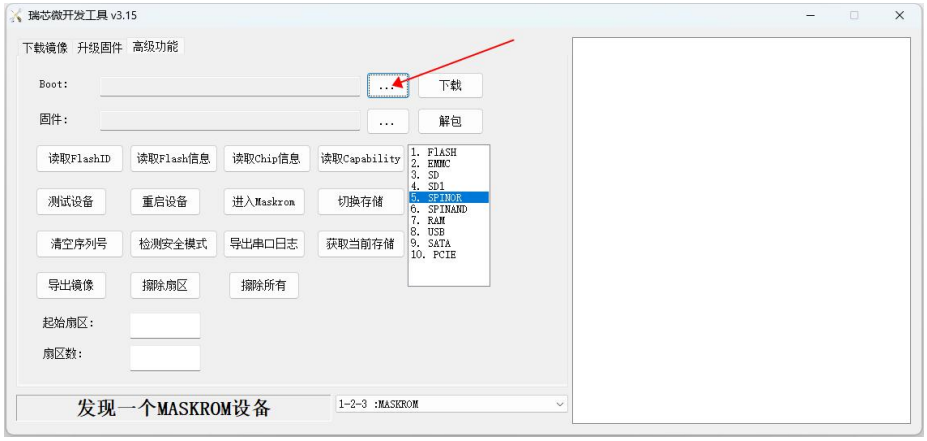


f. 然后再插入 TF 卡到开发板

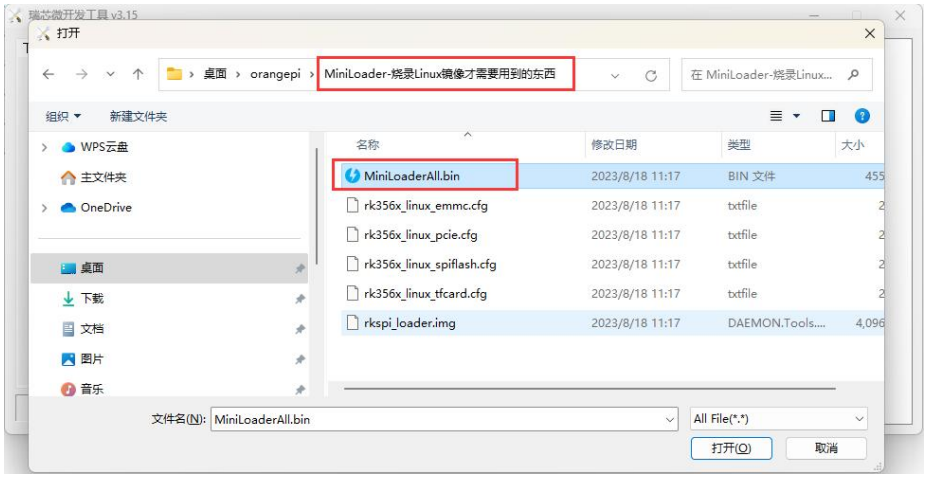
g. 然后请选择**高级功能**



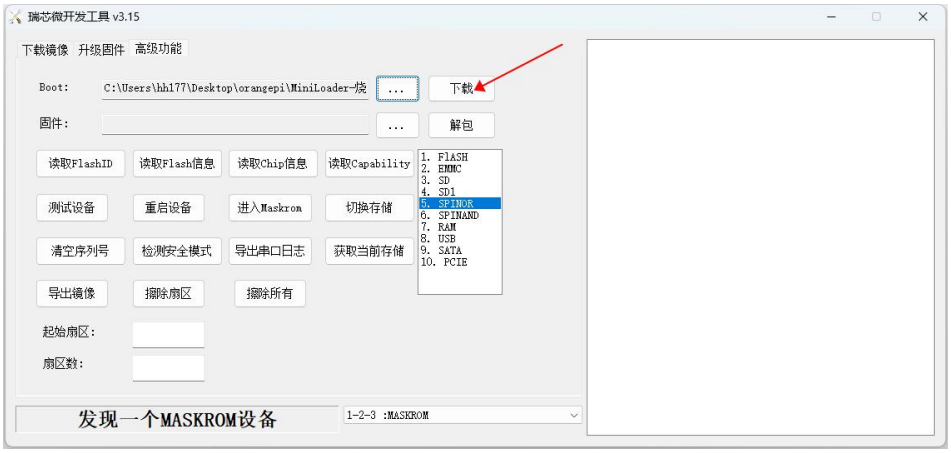
h. 然后点击下图所示的位置



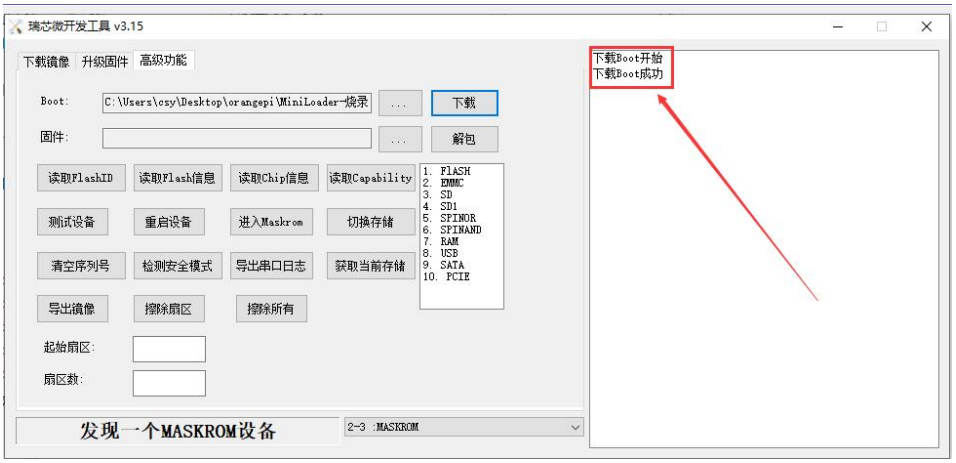
i. 再选择前面下载的 **MiniLoader** 文件夹中的 **MiniLoaderAll.bin**，再点击打开



j. 然后点击**下载**

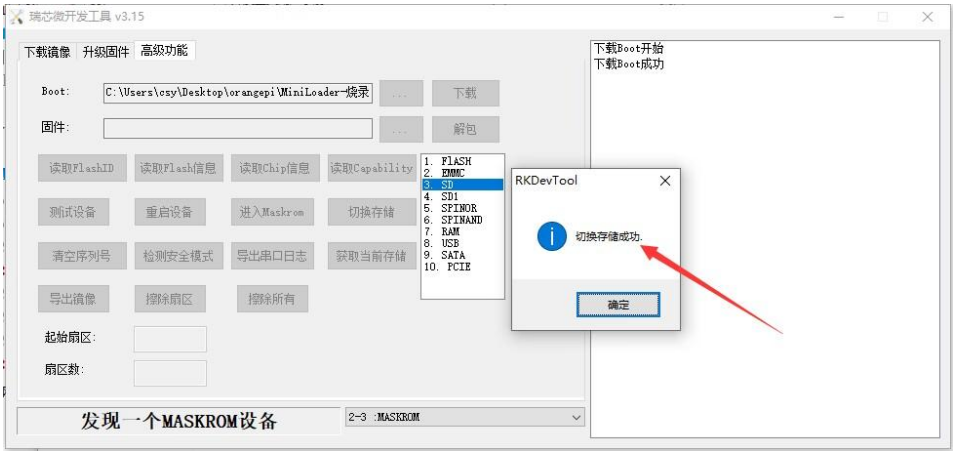


k. 下载完 **MiniLoaderAll.bin** 后的显示如下图所示



l. 然后选择存储设备为 **SD**，再点击**切换存储**

m. 切换成功的显示如下图所示



n. 然后点击烧录工具的“**升级固件**”一栏

o. 接着点击“**固件**”按钮选择需要烧录的 OPi OS（OH）镜像的路径

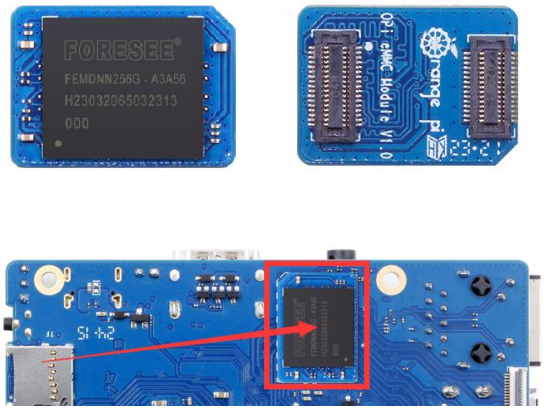
p. 最后点击“**升级**”按钮就会开始烧录，烧录过程中的 log 如下图所示。烧录完成后 OPi OS（OH）系统会自动启动。

2.11. 烧录 Orange Pi OS（OH）镜像到 eMMC 中的方法

注意，烧录镜像到eMMC中后如果测试发现启动不了，请清空下SPIFlash后再启动试下。清空SPIFlash的方法请参考使用RKDevTool清空SPIFlash的方法小节的内容。

注意，下面所有的操作都是在 Windows 电脑中进行的。

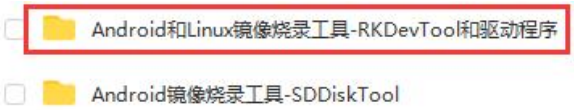
1) 开发板预留了 eMMC 的扩展接口，烧录系统到 eMMC 前，首先需要购买一个与开发板 eMMC 接口相匹配的 eMMC 模块。然后将 eMMC 模块安装到开发板上。eMMC 模块和插入开发板的方法如下所示：



2) 还需要准备一根品质良好的 USB2.0 公对公数据线

3) 然后从 **Orange Pi 的资料下载页面** 下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面** 首先选择**官方工具**，然后进入下面的文件夹中



b. 然后下载下面的所有文件

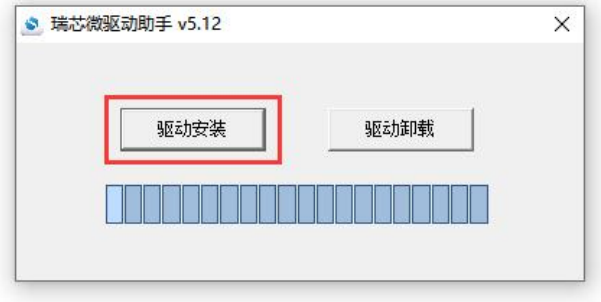
4) 然后从 **Orange Pi 的资料下载页面** 下载 OPi OS（OH）的镜像

5) 然后使用解压软件解压下载的 OPi OS（OH）镜像的压缩包

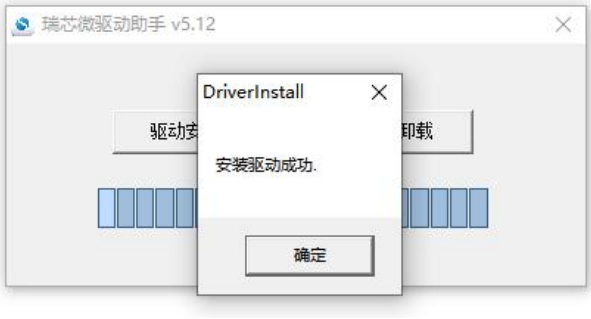
6) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

7) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

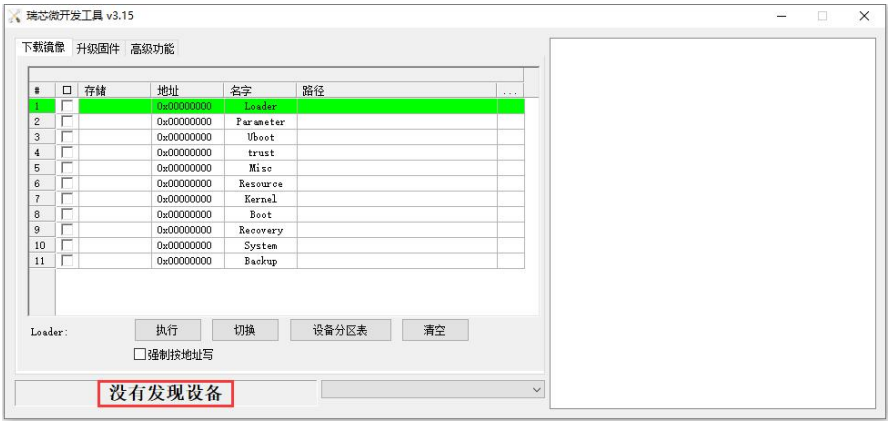


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



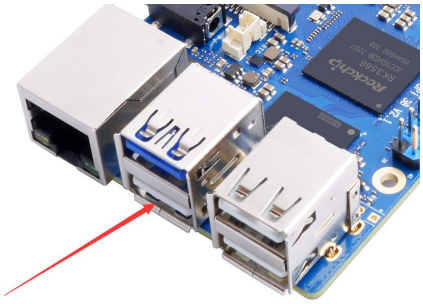
8) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

9) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



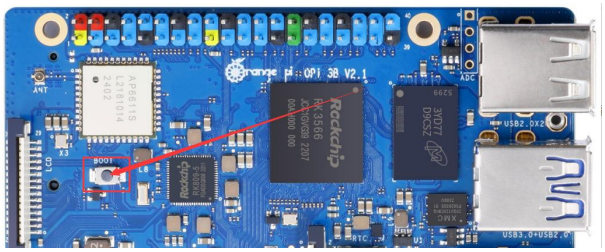
10) 然后开始烧录 OPi OS（OH）镜像到 eMMC 中

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录接口的位置如下图所示

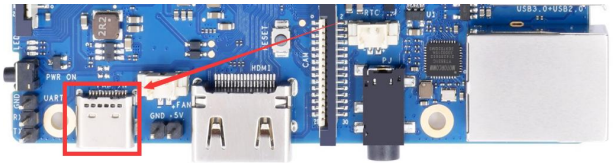


b. 然后确保开发板没有插入 TF 卡，没有连接电源

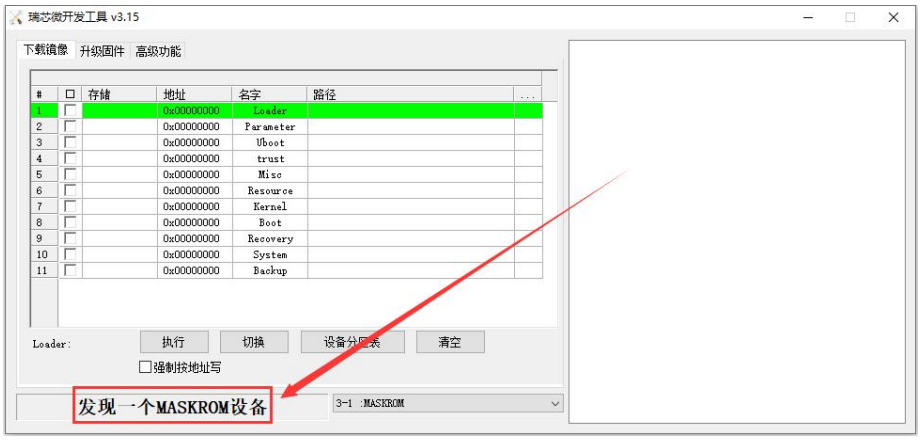
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



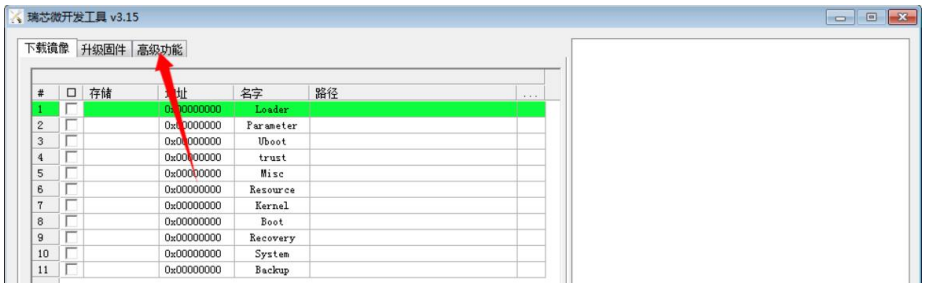
d. 然后给开发板接上 Type-C 接口的电源，并上电



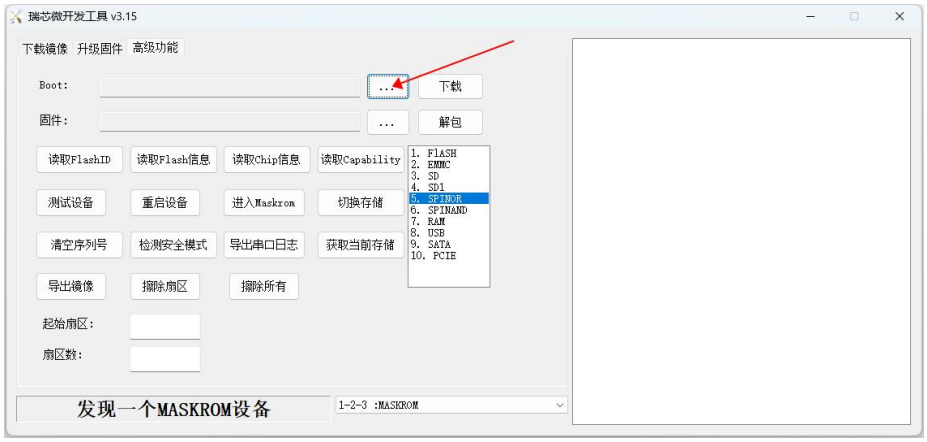
e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”



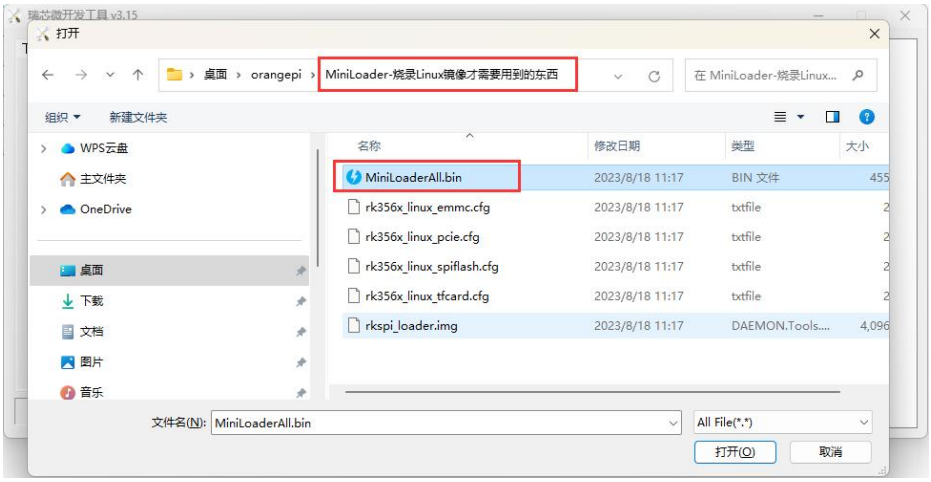
f. 然后请选择**高级功能**



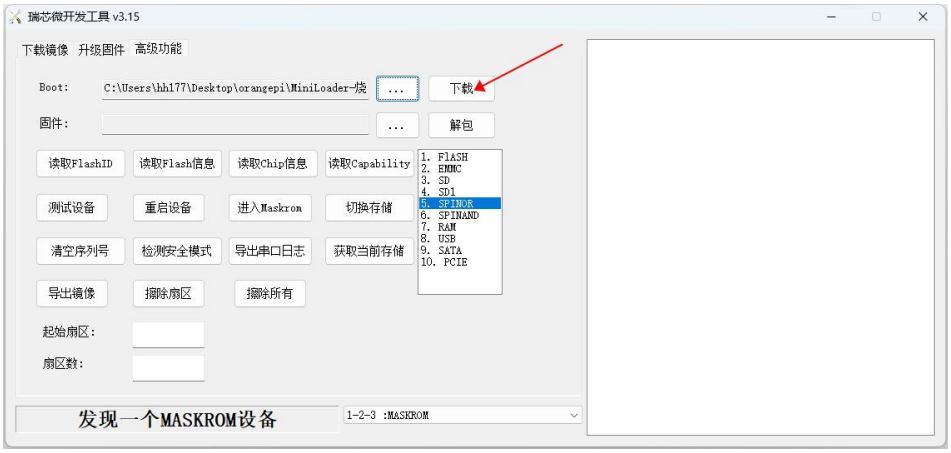
g. 然后点击下图所示的位置



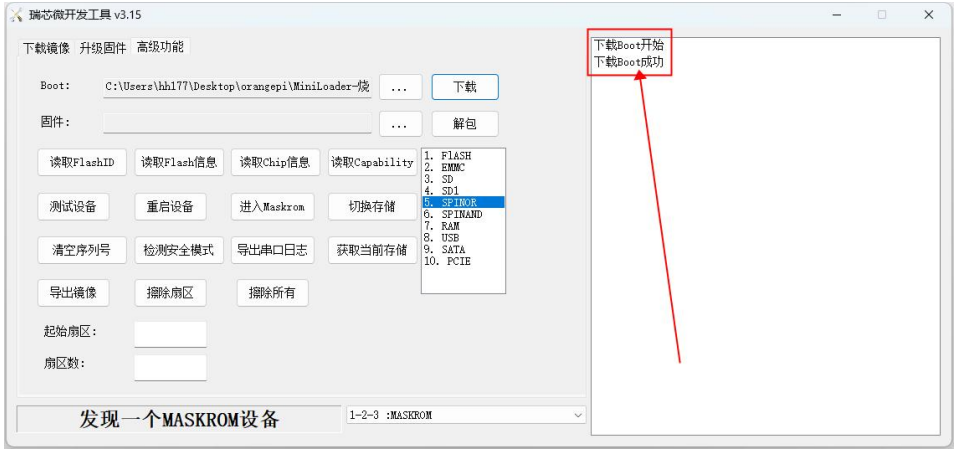
h. 再选择前面下载的 **MiniLoader** 文件夹中的 **MiniLoaderAll.bin**，再点击打开



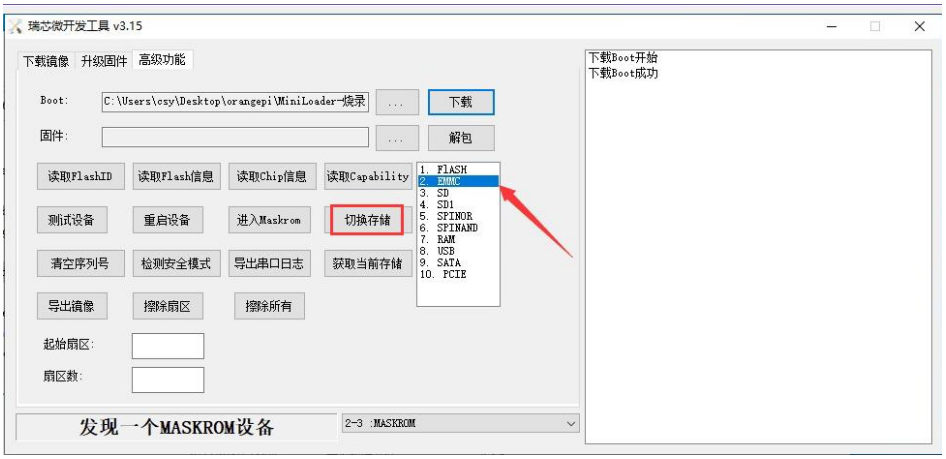
i. 然后点击**下载**



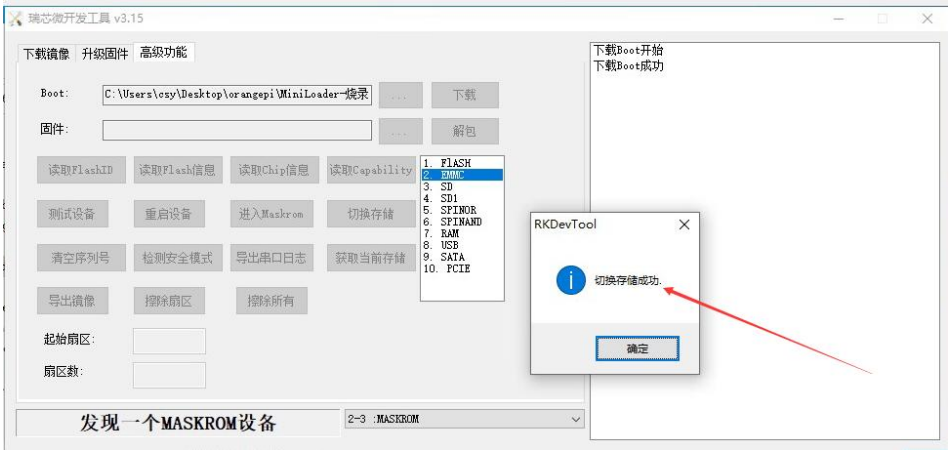
j. 下载完 **MiniLoaderAll.bin** 后的显示如下图所示



k. 然后选择存储设备为 **EMMC，** 再点击**切换存储**



l. 切换成功的显示如下图所示



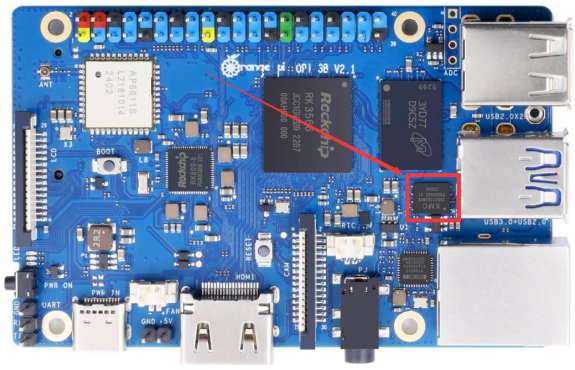
m. 然后点击烧录工具的“**升级固件**”一栏

n. 接着点击“**固件**”按钮选择需要烧录的 OPi OS（OH）镜像的路径

o. 最后点击“**升级**”按钮就会开始烧录，烧录过程中的 log 如下图所示。烧录完成后 OPi OS（OH）系统会自动启动。

2.12. 使用 RKDevTool 清空 SPIFlash 的方法

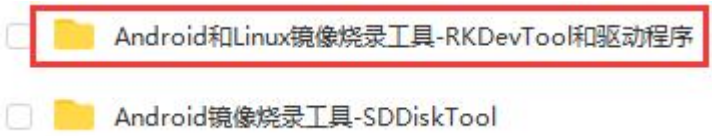
1) SPI Flash 在开发板上的位置如下图所示



2) 首先需要准备一根品质良好的 USB2.0 公对公数据线

3) 然后从 **Orange Pi 的资料下载页面**下载瑞芯微驱动 **DriverAssitant_v5.12.zip** 和 **MiniLoader** 以及烧录工具 **RKDevTool_Release_v3.15.zip**

a. 在 **Orange Pi 的资料下载页面**首先选择**官方工具**，然后进入下面的文件夹中



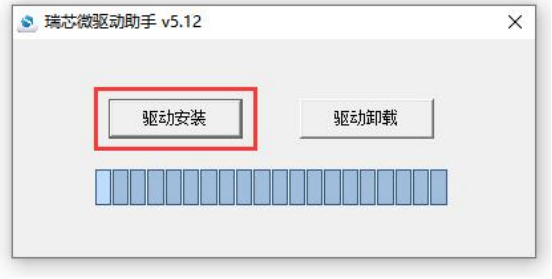
b. 然后下载下面的所有文件

注意，“MiniLoader-烧录Linux镜像才需要用到的东西”文件夹下文简称为MiniLoader文件夹。

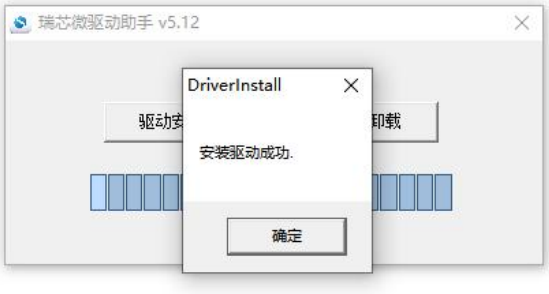
4) 然后用解压软件解压 **DriverAssitant_v5.12.zip**，再在解压后的文件夹中找到 **DriverInstall.exe** 可执行文件并打开即可

5) 打开 **DriverInstall.exe** 后安装瑞芯微驱动的步骤如下所示

a. 点击“**驱动安装**”按钮

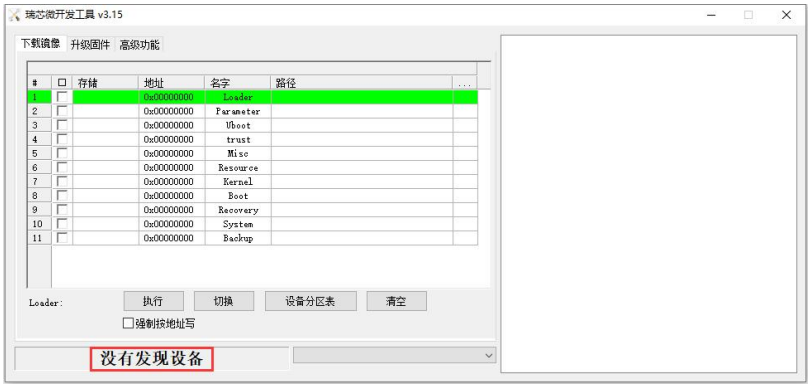


b. 等待一段时间后，会弹出窗口提示“**安装驱动成功**”，然后点击**“确定”**按钮即可



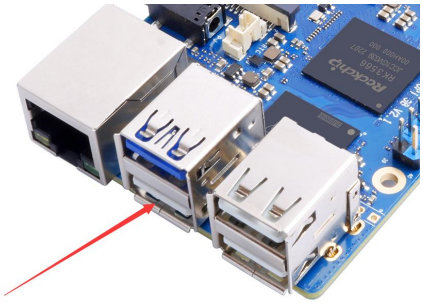
6) 然后解压 **RKDevTool_Release_v3.15.zip**，此软件无需安装，在解压后的文件夹中找到 **RKDevTool** 打开即可

7) 打开 **RKDevTool** 烧录工具后，因为电脑此时还没有通过 USB2.0 公对公数据线连接上开发板，所以左下角会提示“**没有发现设备**”



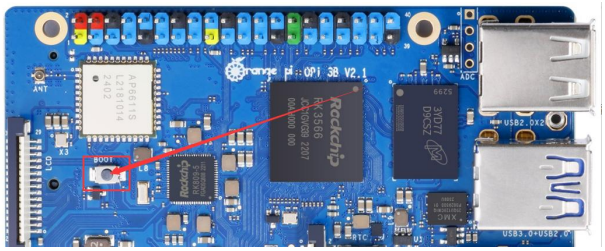
8) 然后就可以开始清空 SPI FLASH 中的内容

a. 首先通过 USB2.0 公对公数据线连接好开发板与 Windows 电脑，开发板 USB2.0 烧录口的位置如下图所示

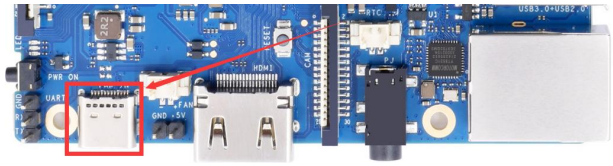


b. 确保开发板没有插入 TF 卡，没有连接电源

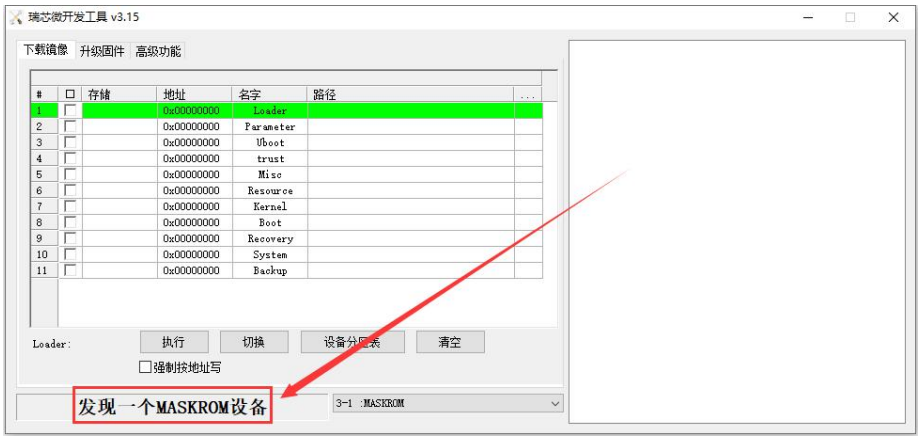
c. 然后按住开发板的 MaskROM 按键不放，MaskROM 按键在开发板的位置如下图所示：



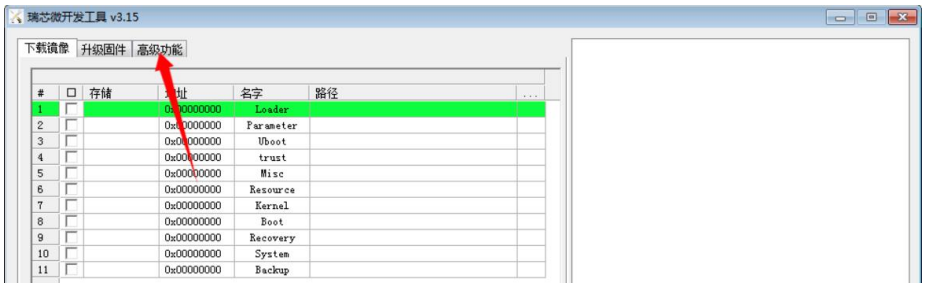
d. 然后给开发板接上 Type-C 接口的电源，并上电，然后就可以松开 MaskROM 按键了



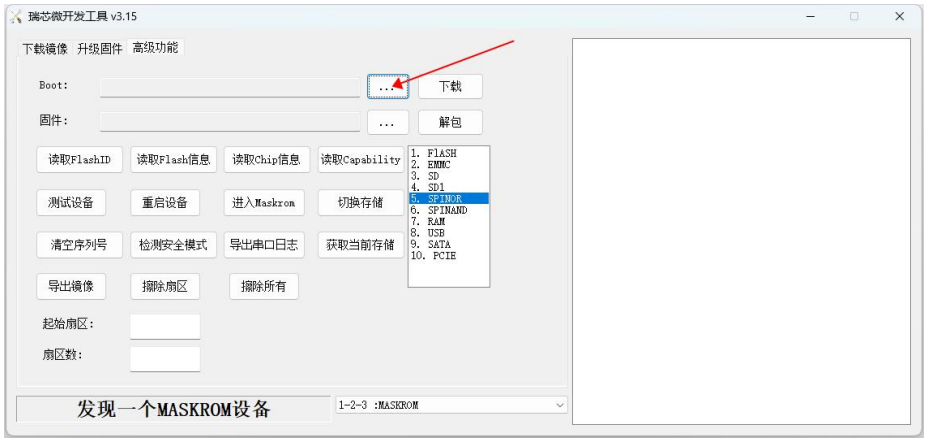
e. 如果前面的步骤顺利，此时开发板会进入 **MASKROM** 模式，在烧录工具的界面上会提示“**发现一个 MASKROM 设备**”



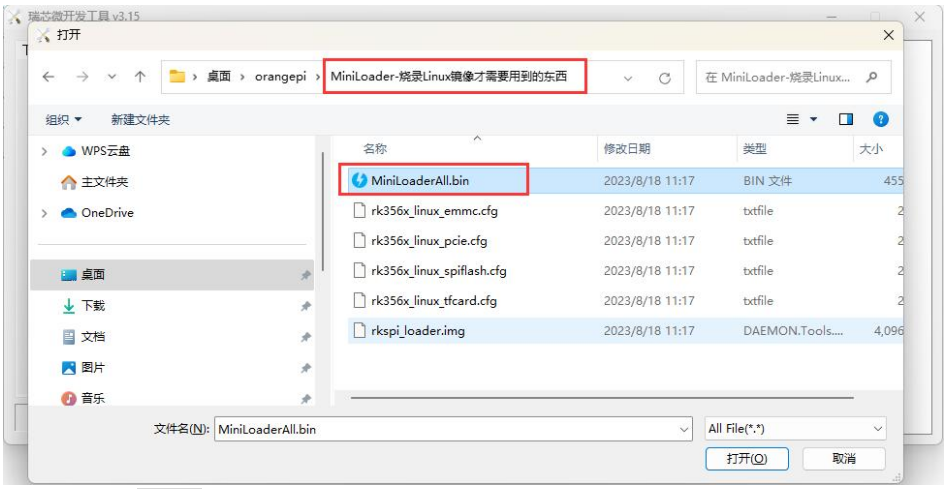
f. 然后请选择**高级功能**



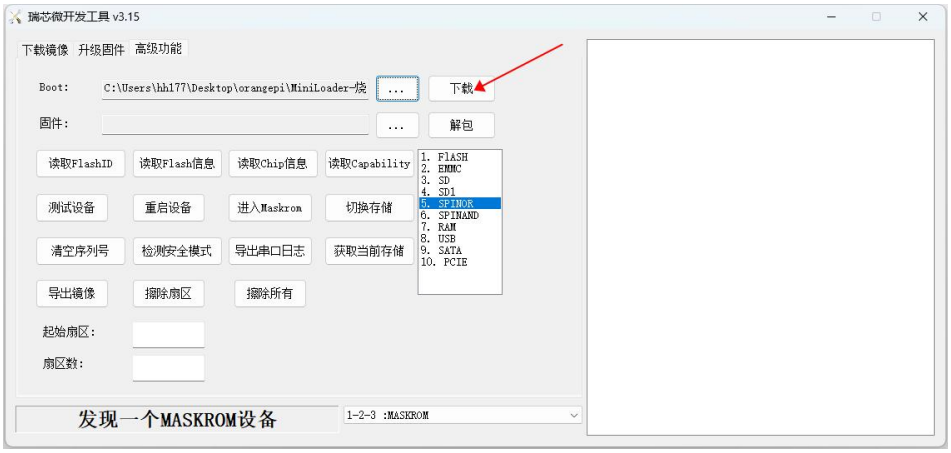
g. 然后点击下图所示的位置



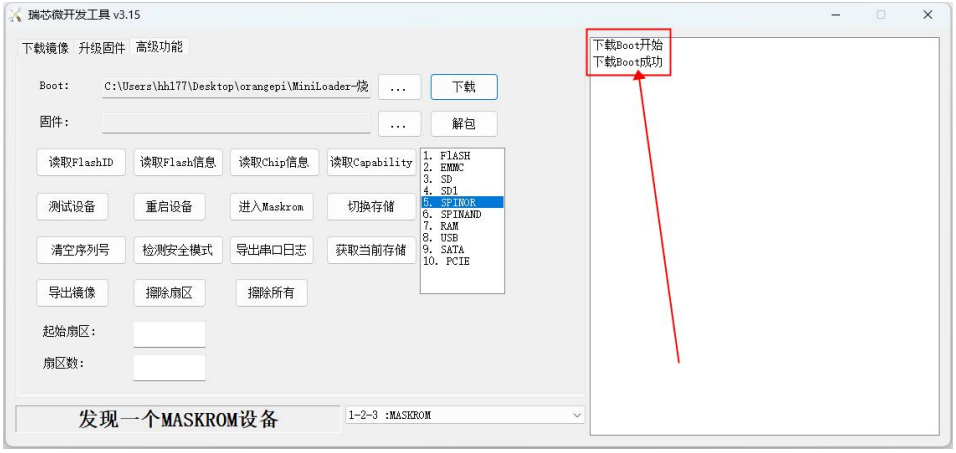
h. 再选择前面下载的 **MiniLoader** 文件夹中的 **MiniLoaderAll.bin**，再点击打开



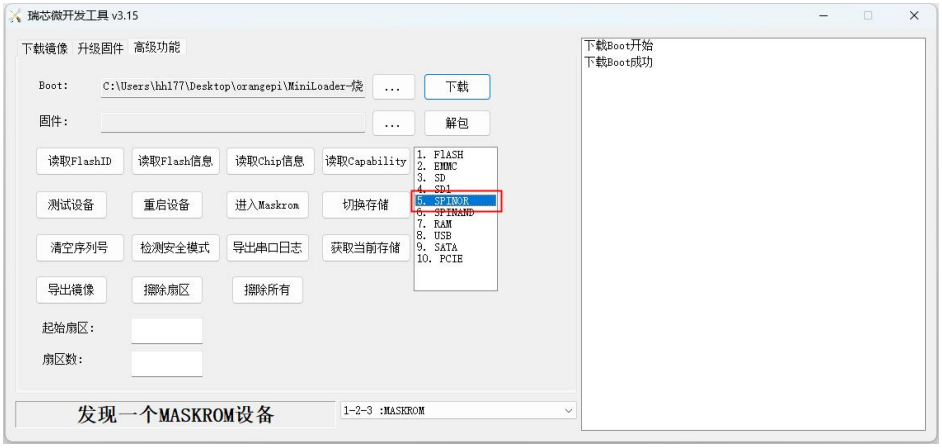
i. 然后点击**下载**



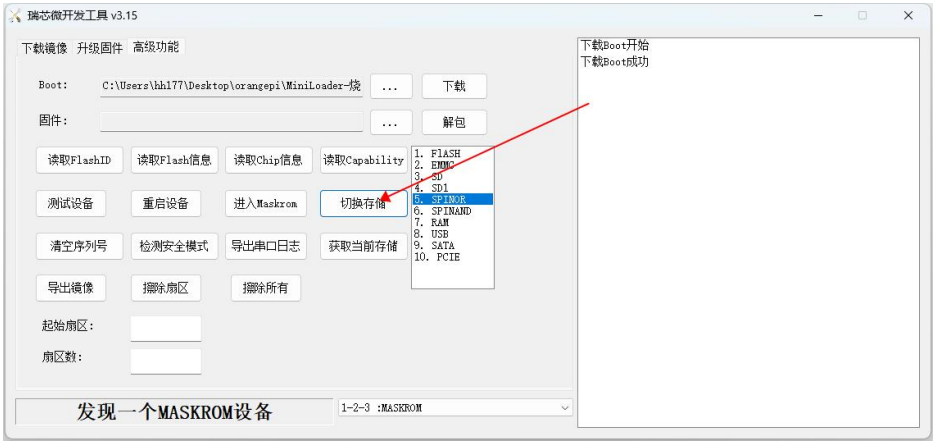
j. 下载完 **MiniLoaderAll.bin** 后的显示如下图所示



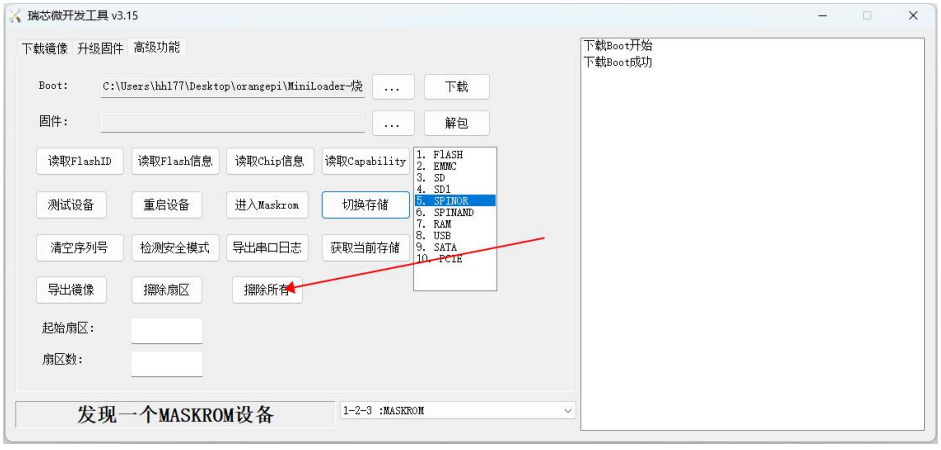
k. 然后选择存储设备为 **SPINOR**



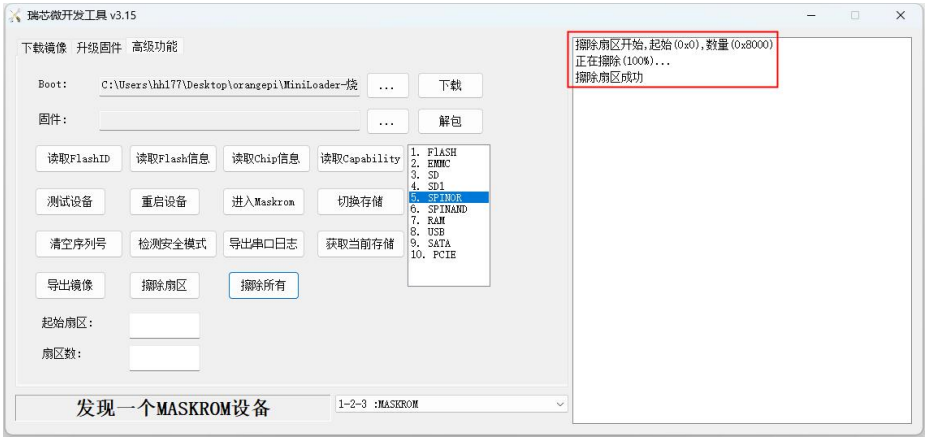
l. 然后点击**切换存储**



m. 然后点击**擦除所有**就会开始擦除 SPIFlash 了



n. 擦除 SPIFlash 后的显示 log 如下图所示



2.13. 启动香橙派开发板

1) 将烧录好镜像的 TF 卡插入香橙派开发板的 TF 卡插槽中。如果 SPIFlash+NVMe SSD 已经烧录好镜像了，那么就不需要插入 TF 卡了，只需确保 NVMe SSD 正常插入到开发板中即可。

2) 开发板有 HDMI 接口，可以通过 HDMI 转 HDMI 连接线把开发板连接到电视或者 HDMI 显示器。如果有购买 LCD 屏幕，也可以使用 LCD 屏幕来显示开发板的系统界面。

3) 接上 USB 鼠标和键盘，用于控制香橙派开发板。

4) 开发板有以太网口，可以插入网线用来上网。

5) 连接一个 5V/3A 的 USB Type-C 接口的**高品质**的电源适配器。

切记不要插入电压输出大于 5V 的电源适配器，会烧坏开发板。

系统上电启动过程中很多不稳定的现象基本都是供电有问题导致的，所以一个靠谱的电源适配器很重要。如果启动过程中发现有不断重启的现象，请更换下电源或者 Type-C 数据线再试下。

Type-C 电源接口是不支持 PD 协商的。

另外请不要接到电脑的 USB 接口来给开发板供电。

6) 然后打开电源适配器的开关，如果一切正常，此时 HDMI 显示器或者 LCD 屏幕

就能看到系统的启动画面了。

7) 如果想通过调试串口查看系统的输出信息，请使用串口线将开发板连接到电脑，串口的连接方法请参看调试串口的使用方法一节。

2.14. 调试串口的使用方法

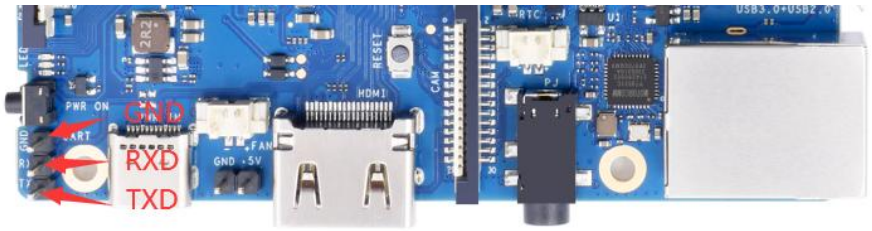
2.14.1. 调试串口的连接说明

1) 首先需要准备一个 **3.3V** 的 USB 转 TTL 模块，然后将 USB 转 TTL 模块的 USB 接口一端插入到电脑的 USB 接口中。

为了更好的兼容性，推荐使用 CH340 USB 转 TTL 模块，请不要使用 CP2102、PL2303 类型的 USB 转 TTL 模块。

购买 USB 转 TTL 模块前，请确认模块支持 1500000 速率的波特率。

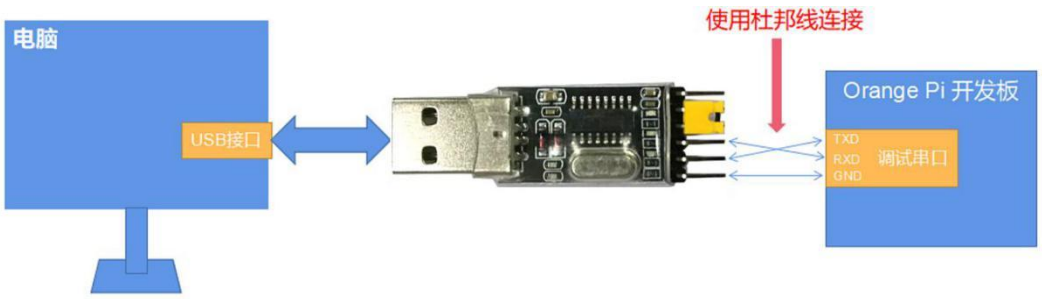
2) 开发板的调试串口 GND、RXD 和 TXD 引脚的对应关系如下图所示



3) USB 转 TTL 模块 GND、TXD 和 RXD 引脚需要通过杜邦线连接到开发板的调试串口上

a. USB 转 TTL 模块的 GND 接到开发板的 GND 上
b. USB 转 TTL 模块的 **RX 接到开发板的 TX 上**
c. USB 转 TTL 模块的 **TX 接到开发板的 RX 上**

4) USB 转 TTL 模块连接电脑和 Orange Pi 开发板的示意图如下所示



USB转TTL模块连接电脑和 Orange Pi 开发板的示意图

串口的 TX 和 RX 是需要交叉连接的，如果不想仔细区分 TX 和 RX 的顺序，可以把串口的 TX 和 RX 先随便接上，如果测试没有输出再交换下 TX 和 RX 的顺序，这样就总有一种顺序是对的

2.14.2. Ubuntu 平台调试串口的使用方法

Linux 下可以使用的串口调试软件有很多，如 putty、minicom 等，下面演示 putty 的使用方法。

1) 首先将 USB 转 TTL 模块插入 Ubuntu 电脑的 USB 接口，如果 USB 转 TTL 模块连接识别正常，在 Ubuntu PC 的**/dev** 下就可以看到对应的设备节点名，记住这个节点名，后面设置串口软件时会用到

```
test@test:~$ ls /dev/ttyUSB*
/dev/ttyUSB0
```

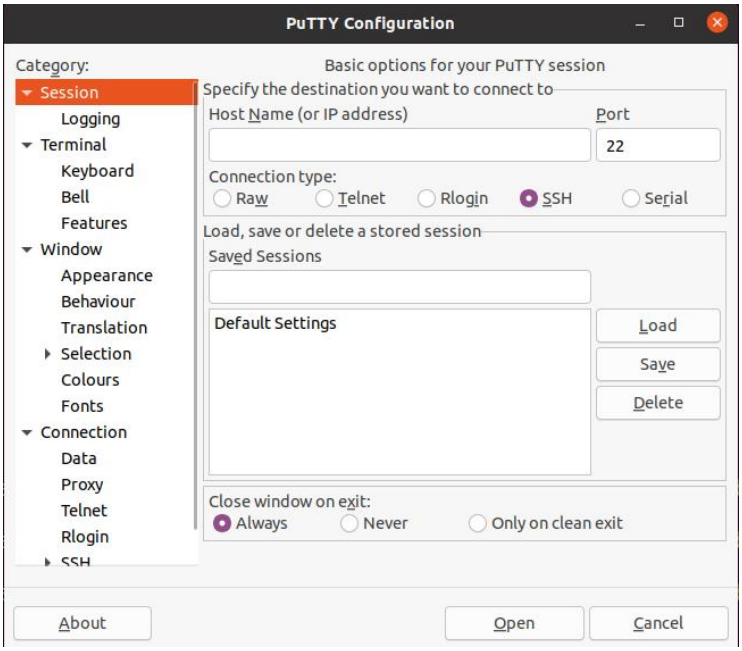
2) 然后使用下面的命令在 Ubuntu PC 上安装下 putty

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y putty
```

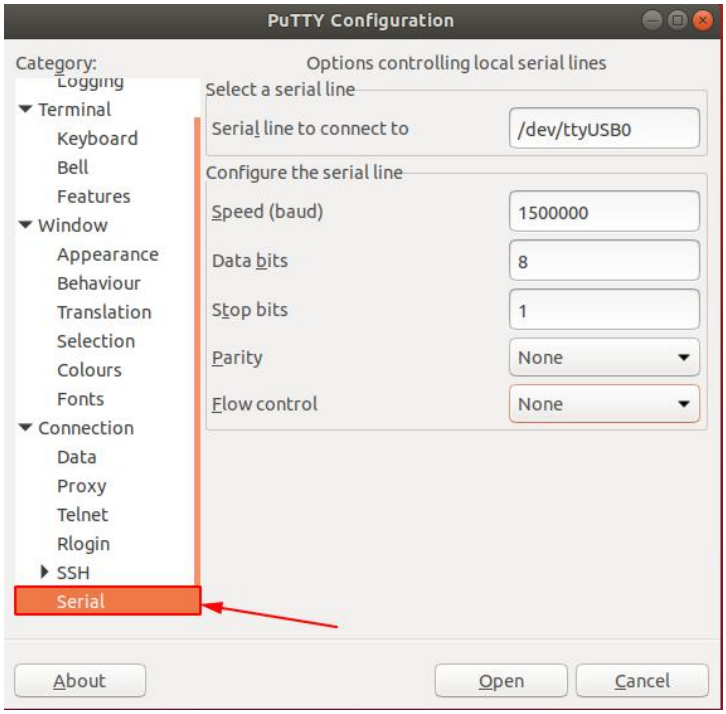
3) 然后运行 putty，**记得加 sudo 权限**

```
test@test:~$ sudo putty
```

4) 执行 putty 命令后会弹出下面的界面



5) 首先选择串口的设置界面

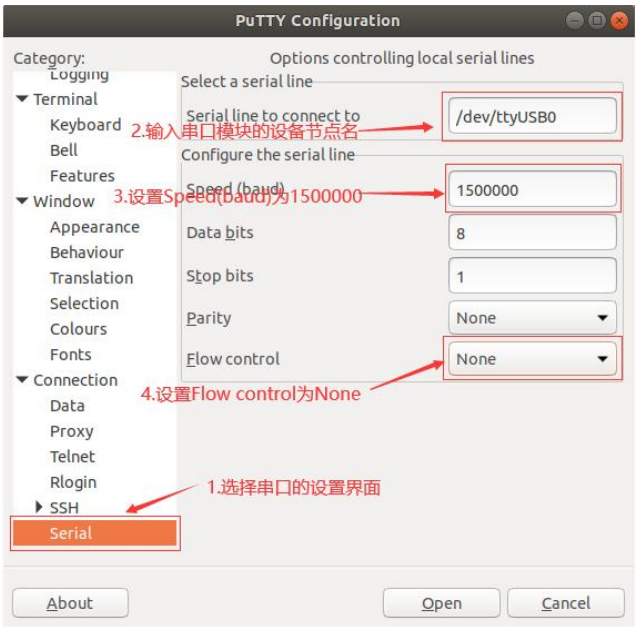


6) 然后设置串口的参数

a. 设置 Serial line to connect to 为/dev/ttyUSB0（修改为对应的节点名，一般为/dev/ttyUSB0）

b. 设置 Speed(baud)为 **1500000**（串口的波特率）

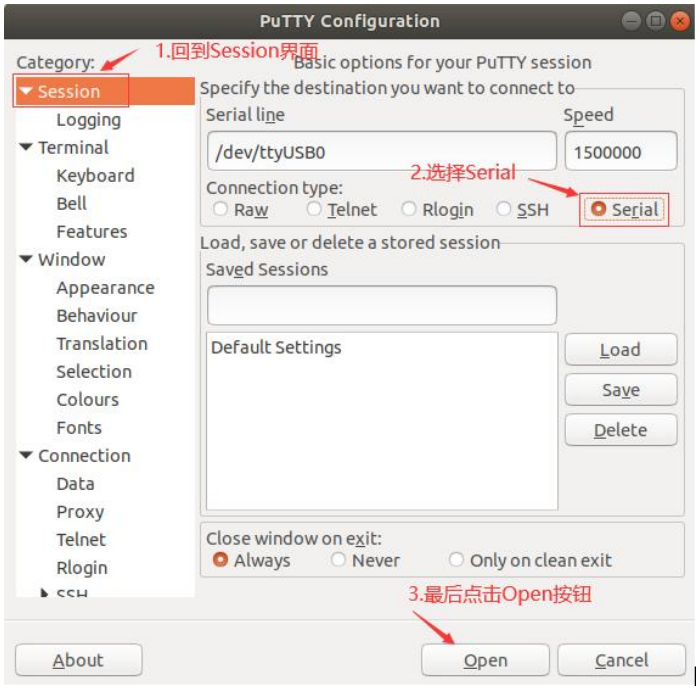
c. 设置 Flow control 为 None



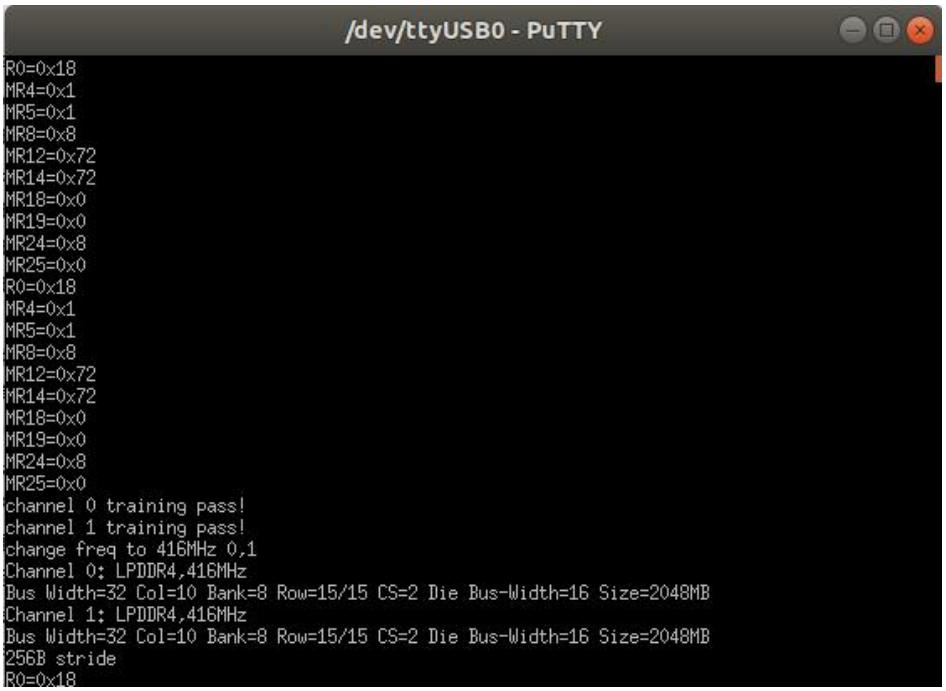
7) 在串口的设置界面设置完后，再回到 Session 界面

a. 首先选择 Connection type 为 Serial

b. 然后点击 Open 按钮连接串口



8) 启动开发板后，就能从打开的串口终端中看到系统输出的 Log 信息了



2.14.3. Windows 平台调试串口的使用方法

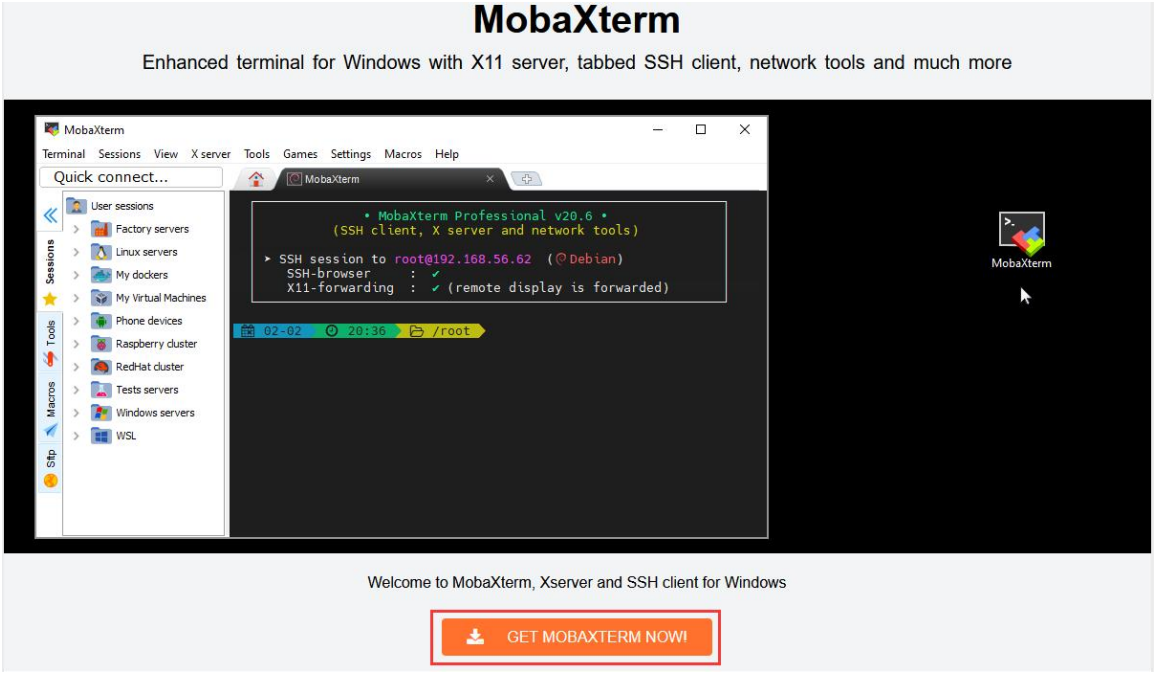
Windows 下可以使用的串口调试软件有很多，如 SecureCRT、MobaXterm 等，下面演示 MobaXterm 的使用方法，这款软件有免费版本，无需购买序列号即可使用。

1) 下载 MobaXterm

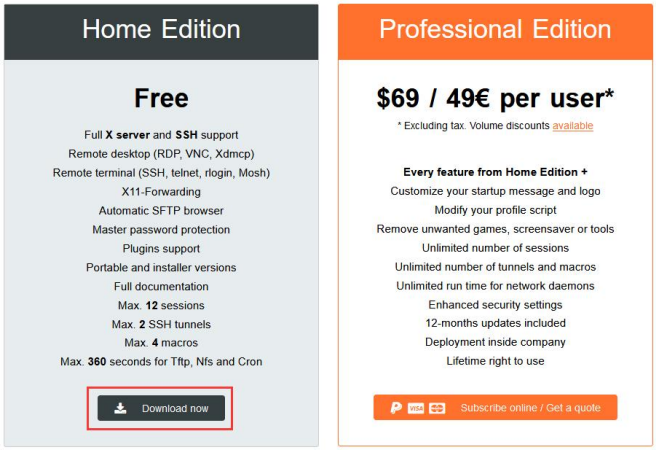
a. 下载 MobaXterm 网址如下

https://mobaxterm.mobatek.net

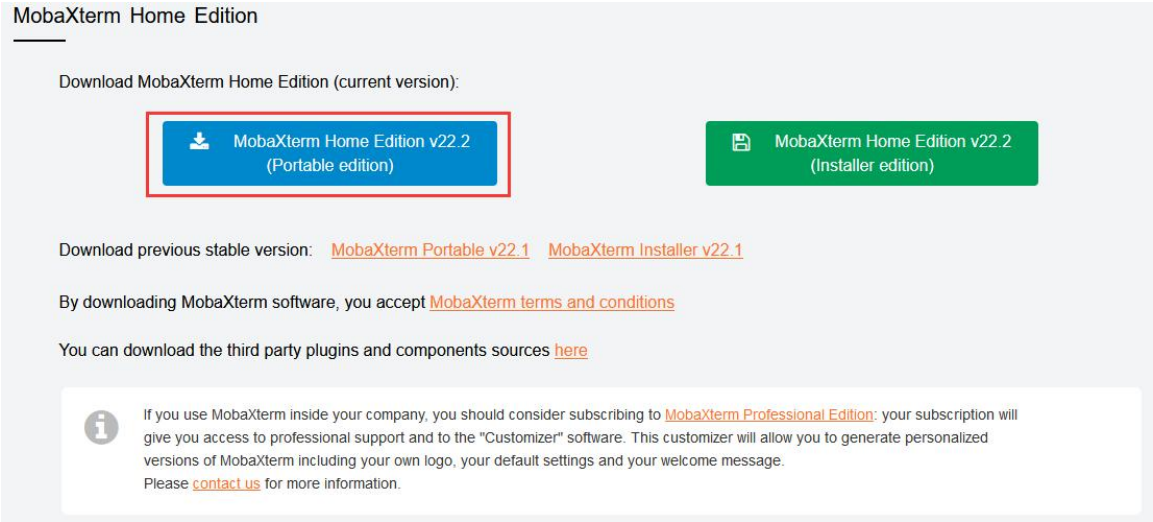
b. 进入 MobaXterm 下载网页后点击 **GET XOBATERM NOW!**



c. 然后选择下载 Home 版本



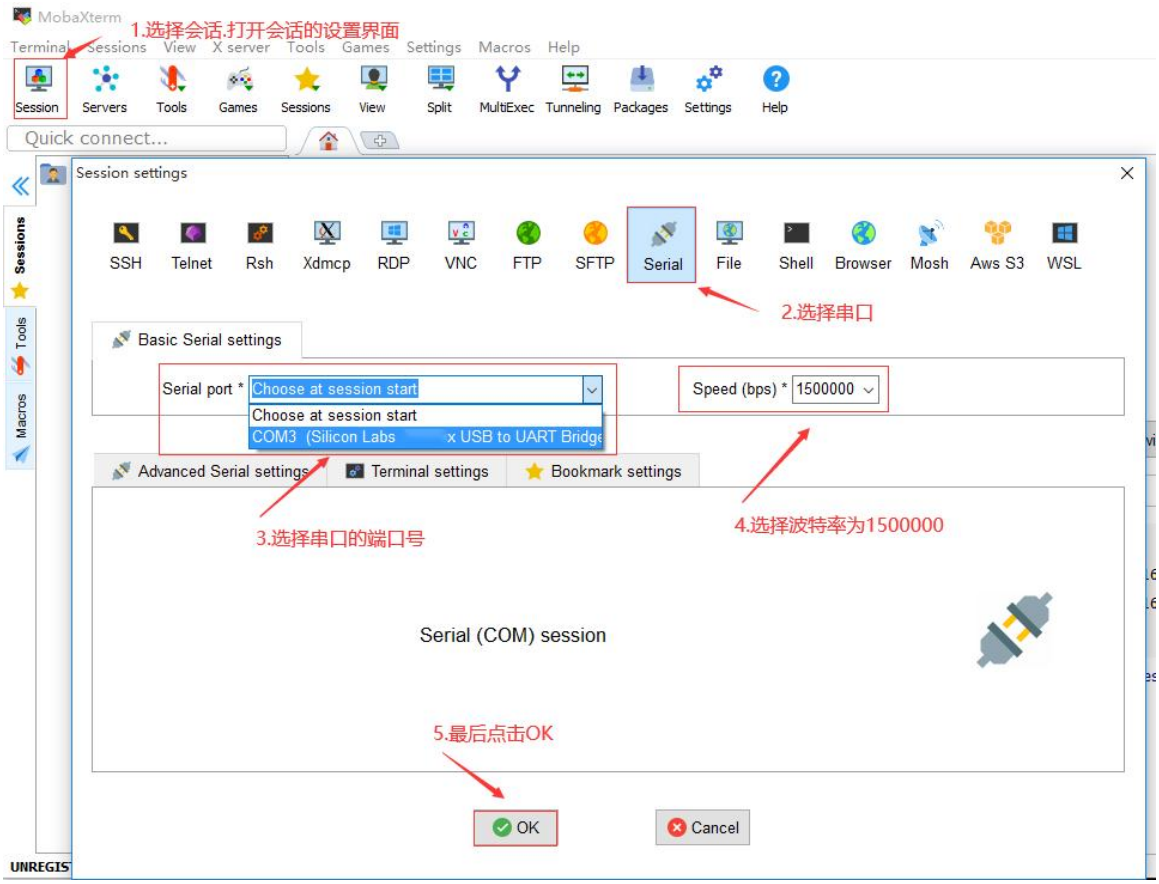
d. 然后选择 Portable 便携式版本，下载完后无需安装，直接打开就可以使用



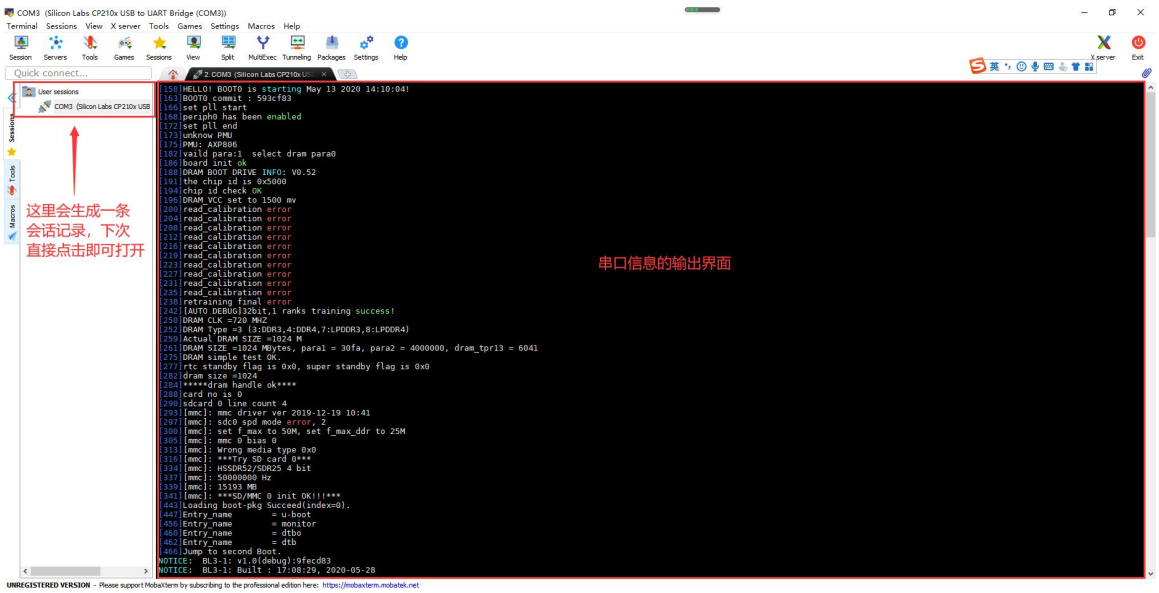
2) 下载完后使用解压缩软件解压下载的压缩包，即可得到 MobaXterm 的可执软件，然后双击打开

3) 打开软件后，设置串口连接的步骤如下

a. 打开会话的设置界面
b. 选择串口类型
c. 选择串口的端口号（根据实际的情况选择对应的端口号），如果看不到端口号，请使用 **360 驱动大师**扫描安装 USB 转 TTL 串口芯片的驱动
d. 选择串口的波特率为 **1500000**
e. 最后点击“**OK**”按钮完成设置



4) 点击“**OK**”按钮后会进入下面的界面，此时启动开发板就能看到串口的输出信息了



2.15. 使用开发板 40pin 接口中的 5v 引脚供电说明

我们推荐的开发板的供电方式是使用 5V/3A 的 Type C 接口的电源线插到开发板的 Type-C 电源接口来供电的。如果需要使用 40pin 接口中的 5V 引脚来给开发板供电，请确保使用的电源线和电源适配器能满足开发板的供电需求。如果有使用不稳定的情况，请换回 Type-C 电源供电。

1) 首先需要准备一根下图所示的电源线

上图所示的电源线在淘宝可以买到，请自行搜索购买。

2) 使用 40pin 接口中的 5V 引脚来给开发板供电，电源线的接法如下所示

a. 上图所示的电源线 USB A 口需要插到 5V/3A 的电源适配器接头上（**请不要插到电脑的 USB 接口来供电**）

b. 红色的杜邦线需要插到开发板 40pin 的 5V 引脚上

c. 黑色的杜邦线需要插到 40pin 接口的 GND 引脚上

d. 40pin 接口 5V 引脚和 GND 引脚在开发板中的位置如下图所示，**切记不要接反了**

3. Ubuntu/Debian Server 和 Xfce 桌面系统使用说明

本章内容是基于 Linux 服务器版本的镜像和 xfce 桌面版本镜像编写的。

3.1. 已支持的 Linux 镜像类型和内核版本

Linux 镜像类型	内核版本	服务器版	桌面版
Debian 11 - Bullseye	Linux5.10	支持	支持
Debian12 - Bookworm	Linux5.10	支持	支持
Ubuntu 20.04 - Focal	Linux5.10	支持	支持
Ubuntu 22.04 - Jammy	Linux5.10	支持	支持
Debian 11 - Bullseye	Linux6.6	支持	支持
Debian12 - Bookworm	Linux6.6	支持	支持
Ubuntu 22.04 - Jammy	Linux6.6	支持	支持

3.2. Linux 系统适配情况

3.2.1. Linux5.10 系统适配情况

功能	Debian11	Debian12	Ubuntu20.04	Ubuntu22.04
USB2.0x3	OK	OK	OK	OK
USB3.0x1	OK	OK	OK	OK
M.2 NVMe SSD 启动	OK	OK	OK	OK
WIFI	OK	OK	OK	OK
蓝牙	OK	OK	OK	OK
GPIO（40pin）	OK	OK	OK	OK
UART（40pin）	OK	OK	OK	OK
SPI（40pin）	OK	OK	OK	OK
I2C（40pin）	OK	OK	OK	OK
PWM（40pin）	OK	OK	OK	OK
PWM 风扇接口	OK	OK	OK	OK

3pin 调试串口	OK	OK	OK	OK
EMMC	OK	OK	OK	OK
TF 卡启动	OK	OK	OK	OK
HDMI 视频	OK	OK	OK	OK
HDMI 音频	OK	OK	OK	OK
OV5647 摄像头	内核驱动 OK，3A 未调			
LCD	OK	OK	OK	OK
eDP 显示	OK	OK	OK	OK
千兆网口	OK	OK	OK	OK
网口状态灯	OK	OK	OK	OK
耳机播放	OK	OK	OK	OK
耳机录音	OK	OK	OK	OK
LED 灯	OK	OK	OK	OK
RTC	OK	OK	OK	OK
GPU	OK	OK	OK	OK
NPU	OK	OK	OK	OK
VPU	OK	OK	OK	OK
看门狗测试	OK	OK	OK	OK
Chromium 硬解视频	OK	OK	OK	OK

3.2.2. Linux6.6 系统适配情况

功能	Debian11	Debian12	Ubuntu22.04
USB2.0x3	OK	OK	OK
USB3.0x1	OK	OK	OK
M.2 NVMe SSD 启动	OK	OK	OK
WIFI	OK	OK	OK
蓝牙	OK	OK	OK
GPIO（40pin）	OK	OK	OK
UART（40pin）	OK	OK	OK
SPI（40pin）	OK	OK	OK
I2C（40pin）	OK	OK	OK
PWM（40pin）	OK	OK	OK
PWM 风扇接口	OK	OK	OK

3pin 调试串口	OK	OK	OK
EMMC	OK	OK	OK
TF 卡启动	OK	OK	OK
HDMI 视频	OK	OK	OK
HDMI 音频	OK	OK	OK
OV5647 摄像头	NO	NO	NO
LCD	NO	NO	NO
eDP 显示	NO	NO	NO
千兆网口	OK	OK	OK
网口状态灯	OK	OK	OK
耳机播放	NO	NO	NO
耳机录音	NO	NO	NO
LED 灯	OK	OK	OK
RTC	OK	OK	OK
GPU	OK	OK	OK
NPU	NO	NO	NO
VPU	NO	NO	NO
看门狗测试	NO	NO	NO

3.3. 本手册 Linux 命令格式说明

1) 本手册中所有需要在 Linux 系统中输入的命令都会使用下面的方框框起来

```
```

如下所示，黄色方框里内容表示需要特别注意的内容，这里面的命令除外。

```
```

2) 命令前面的提示符类型说明

a. 命令前面提示符指的是下面方框内红色部分的内容，这部分内容不是 Linux 命令的一部分，所以在 Linux 系统中输入命令时，请不要把红色字体部分的内容也输入进去。

```
orangepi@orangepi:~$ sudo apt update
root@orangepi:~# vim /boot/boot.cmd
```

```
test@test:~$ ssh root@192.168.1.xxx
root@test:~# ls
```

b. **root@orangepi:~$** 提示符表示这个命令是在**开发板的 Linux 系统**中输入的， 提示符最后的 **$** 表示系统当前用户为普通用户，当执行特权命令时，需要加上 **sudo**

c. **root@orangepi:~#** 提示符表示这个命令是在**开发板的 Linux 系统**中输入的， 提示符最后的 **#** 表示系统当前用户为 root 用户，可以执行任何想要执行的命令

d. **test@test:~$** 提示符表示这个命令是在 Ubuntu PC 或者 Ubuntu 虚拟机中输入的，而不是开发板的 Linux 系统中。提示符最后的 **$** 表示系统当前用户为普通用户，当执行特权命令时，需要加上 **sudo**

e. **root@test:~#** 提示符表示这个命令是在 Ubuntu PC 或者 Ubuntu 虚拟机中输入的，而不是开发板的 Linux 系统中。提示符最后的 **#** 表示系统当前用户为 root 用户，可以执行任何想要执行的命令

3) 哪些是需要输入的命令？

a. 如下所示，**黑色加粗部分**是需要输入的命令，命令下面的是输出的内容（有些命令有输出，有些可能没有输出），这部分内容是不需要输入的

```
root@orangepi:~# cat /boot/orangepiEnv.txt
verbosity=7
bootlogo=false
console=serial
```

b. 如下所示，有些命令一行写不下会放到下一行，只要黑色加粗的部分就都是需要输入的命令。当这些命令输入到一行的时候，每行最后的“\”是需要去掉的，这个不是命令的一部分。另外命令的不同部分都是有空格的，请别漏了

```
orangepi@orangepi:~$ echo  \
"deb [arch=$(dpkg --print-architecture)  \
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  \
https://download.docker.com/linux/debian  \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3.4. Linux 系统登录说明

3.4.1. Linux 系统默认登录账号和密码

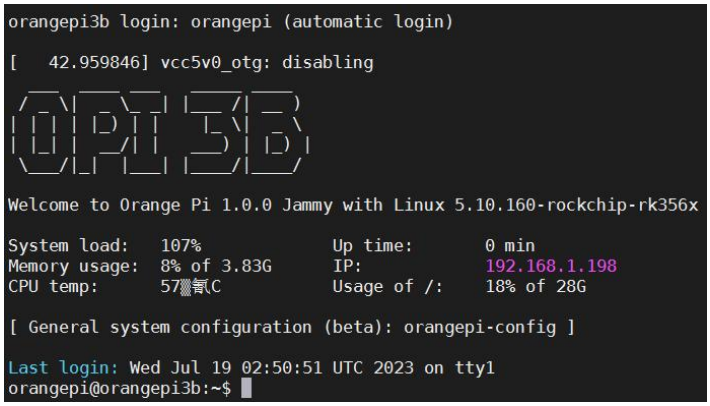
账号	密码
root	**orangepi**
orangepi	**orangepi**

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。

当输入密码提示错误，或者 ssh 连接有问题，请注意，只要使用的是 Orange Pi 提供的 Linux 镜像，就请不要怀疑上面的密码不对，而是要找其它的原因。

3.4.2. 设置 Linux 系统终端自动登录的方法

1) Linux 系统默认就是自动登录终端的，默认登录的用户名是 **orangepi**



2) 使用下面的命令可以设置 root 用户自动登录终端

```
orangepi@orangepi:~$ sudo auto_login_cli.sh root
```

3) 使用下面的命令可以禁止自动登录终端

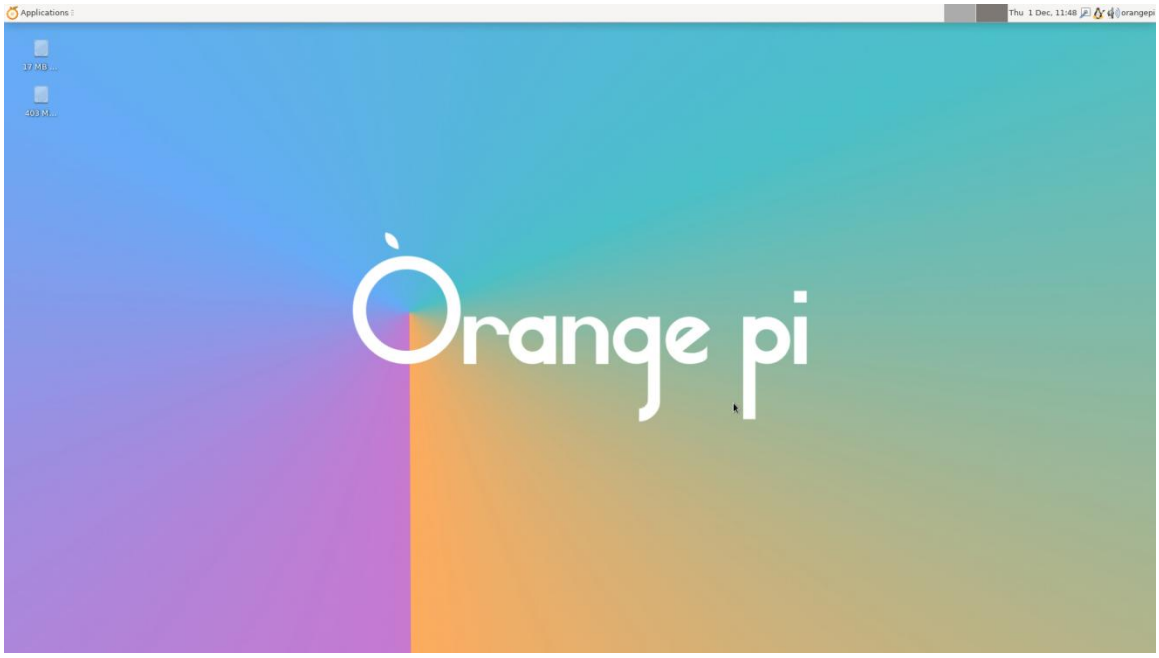
```
orangepi@orangepi:~$ sudo auto_login_cli.sh -d
```

4) 使用下面的命令可以再次设置 orangepi 用户自动登录终端

```
orangepi@orangepi:~$ sudo auto_login_cli.sh orangepi
```

3. 4. 3. Linux 桌面版系统自动登录说明

1) 桌面版系统启动后会自动登录进入桌面，无需输入密码



2) 运行下面的命令可以禁止桌面版系统自动登录桌面

```
orangepi@orangepi:~$ sudo disable_desktop_autologin.sh
```

3) 然后重启系统就会出现登录对话框，此时需要输入密码才能进入系统

3. 4. 4. Linux 桌面版系统 root 用户自动登录的设置方法

1) 执行下面的命令可以设置桌面版系统使用 root 用户自动登录

```
orangepi@orangepi:~$ sudo desktop_login.sh root
```

2) 然后重启系统，就会自动使用 root 用户登录桌面了

注意，如果使用 root 用户登录桌面系统，是无法使用右上角的 pulseaudio 来管理音频设备的。

另外请注意这并不是一个 bug，因为 pulseaudio 本来就不允许在 root 用户下运行。

3) 执行下面的命令可以再次设置桌面版系统使用 orangepi 用户自动登录

```
orangepi@orangepi:~$ sudo desktop_login.sh orangepi
```

3.4.5. Linux 桌面版系统禁用桌面的方法

1) 首先在命令行中输入下面的命令，**请记得加 sudo 权限**

```
orangepi@orangepi:~$ sudo systemctl disable lightdm.service
```

2) 然后重启 Linux 系统就会发现不会显示桌面了

```
orangepi@orangepi:~$ sudo reboot
```

3) 重新打开桌面的步骤如下所示：

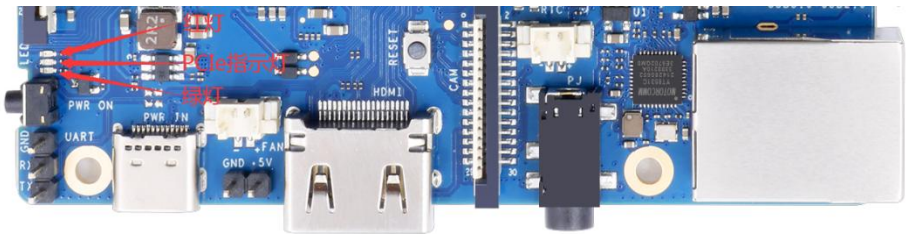
a. 首先在命令行中输入下面的命令，**请记得加 sudo 权限**

```
orangepi@orangepi:~$ sudo systemctl start lightdm.service
orangepi@orangepi:~$ sudo systemctl enable lightdm.service
```

b. 命令执行完成后就会显示桌面了

3.5. 板载 LED 灯测试说明

1) 开发板上有三个 LED 灯，一个绿灯，一个红灯，一个 PCIe 指示灯，所在位置如下图所示：



2) 只要开发板打开了电源，红色的 LED 灯就会常亮，这是由硬件控制的，软件无法关闭。

3) 绿色的 LED 灯在内核启动后会一直闪烁，这是由软件控制的。

4) PCIe 指示灯在 PCIe 接口有数据传输时会闪烁。

5) 设置绿灯亮灭和闪烁的方法如下所示

注意，下面的操作请在 root 用户下进行。

a. 首先进入绿灯的设置目录

```
root@orangepi:~# cd /sys/class/leds/status_led
```

b. 设置绿灯停止闪烁的命令如下

```
root@orangepi:/sys/class/leds/status_led# echo none > trigger
```

c. 设置绿灯常亮的命令如下

```
root@orangepi:/sys/class/leds/status_led# echo default-on > trigger
```

d. 设置绿灯闪烁的命令如下

```
root@orangepi:/sys/class/leds/status_led# echo heartbeat > trigger
```

3.6. 网络连接测试

3.6.1. 以太网口测试

1) 首先将网线的一端插入开发板的以太网接口，网线的另一端接入路由器，并确保网络是畅通的

2) 系统启动后会通过 **DHCP** 自动给以太网卡分配 IP 地址，不需要其他任何配置

3) 在开发板的 Linux 系统中查看 IP 地址的命令如下所示

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 4a:fe:2b:3d:17:1c brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.150/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
       valid_lft 43150sec preferred_lft 43150sec
    inet6 fe80::9a04:3703:faed:23be/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

当使用 ifconfig 查看 IP 地址时，如果提示下面的信息，是因为没有加 sudo 导致的，正确的命令是：sudo ifconfig

```
orangepi@orangepi:~$ ifconfig
```

```
Command 'ifconfig' is available in the following places
  * /sbin/ifconfig
  * /usr/sbin/ifconfig
The command could not be located because '/sbin:/usr/sbin' is not included in the PATH environment variable.
This is most likely caused by the lack of administrative privileges associated with your user account.
ifconfig: command not found
```

开发板启动后查看 IP 地址有三种方法：

1. 接 HDMI 显示器，然后登录系统使用 ip addr show eth0 命令查看 IP 地址

2. 在调试串口终端输入 ip addr show eth0 命令来查看 IP 地址

3. 如果没有调试串口，也没有 HDMI 显示器，还可以通过路由器的管理界面来查看开发板网口的 IP 地址。不过这种方法经常有人会无法正常看到开发板的 IP 地址。如果看不到，调试方法如下所示：

A）首先检查 Linux 系统是否已经正常启动，如果开发板的绿灯在闪烁了，一般是正常启动了，如果只亮红灯，说明系统都没正常启动；

B）检查网线有没有插紧，或者换根网线试下；

C）换个路由器试下（路由器的问题有遇到过很多，比如路由器无法正常分配 IP 地址，或者已正常分配 IP 地址但在路由器中看不到）；

D）如果没有路由器可换就只能连接 HDMI 显示器或者使用调试串口来查看 IP 地址。

另外需要注意的是开发板 DHCP 自动分配 IP 地址是不需要任何设置的。

4) 测试网络连通性的命令如下，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行

```
orangepi@orangepi:~$ ping www.baidu.com -I eth0
PING www.a.shifen.com (14.215.177.38) from 192.168.1.12 eth0: 56(84) bytes of data.
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=1 ttl=56 time=6.74 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=2 ttl=56 time=6.80 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=3 ttl=56 time=6.26 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=4 ttl=56 time=7.27 ms
^C
--- www.a.shifen.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
```

```
rtt min/avg/max/mdev = 6.260/6.770/7.275/0.373 ms
```

3.6.2. WIFI 连接测试

请不要通过修改/etc/network/interfaces 配置文件的方式来连接 WIFI，通过这种方式连接 WIFI 网络使用会有问题。

3.6.2.1. 服务器版镜像通过命令连接 WIFI

当开发板没有连接以太网，没有连接 HDMI 显示器，只连接了串口时，推荐使用此小节演示的命令来连接 WIFI 网络。因为 nmtui 在某些串口软件（如 minicom）中只能显示字符，无法正常显示图形界面。当然，如果开发板连接了以太网或者 HDMI 显示屏，也可以使用此小节演示的命令来连接 WIFI 网络的。

1) 先登录 Linux 系统，有下面三种方式
   a. 如果开发板连接了网线，可以通过 **ssh 远程登录 Linux 系统**
   a. 如果开发板连接好了调试串口，可以使用串口终端登录 Linux 系统
   b. 如果连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 Linux 系统

2) 首先使用 **nmcli dev wifi** 命令扫描周围的 WIFI 热点

```
orangepi@orangepi:~$ nmcli dev wifi
```

```
root@orangepi:~# nmcli dev wifi
IN-USE  BSSID              SSID                MODE   CHAN  RATE        SIGNAL  BARS  SECURITY
        28:6C:07:6E:87:2E  [redacted]_orangepi  Infra  9     260 Mbit/s  97      ▂▄▆▇  WPA1 WPA2
        D8:D8:66:A5:BD:D1  [redacted]           Infra  10    270 Mbit/s  90      ▂▄▆▇  WPA1 WPA2
        A0:40:A0:A1:72:20  [redacted]           Infra  4     405 Mbit/s  82      ▂▄▆▇  WPA2
        28:6C:07:6E:87:2F  [redacted]_orangepi_5G Infra  149   540 Mbit/s  80      ▂▄▆▇  WPA1 WPA2
        CA:50:E9:89:E2:44  [redacted]           Infra  1     130 Mbit/s  79      ▂▄▆_  WPA1 WPA2
        A0:40:A0:A1:72:31  [redacted]           Infra  100   405 Mbit/s  67      ▂▄▆_  WPA2
        D4:EE:07:08:A9:E0  [redacted]           Infra  4     130 Mbit/s  55      ▂▄__  WPA1 WPA2
        88:C3:97:49:25:13  [redacted]           Infra  6     130 Mbit/s  52      ▂▄__  WPA1 WPA2
        00:BD:82:51:53:C2  [redacted]           Infra  12    130 Mbit/s  49      ▂▄__  WPA1 WPA2
        C0:61:18:FA:49:37  [redacted]           Infra  149   270 Mbit/s  47      ▂▄__  WPA1 WPA2
        04:79:70:8D:0C:B8  [redacted]           Infra  153   270 Mbit/s  47      ▂▄__  WPA2
        04:79:70:FD:0C:B8  [redacted]           Infra  153   270 Mbit/s  47      ▂▄__  WPA2
        9C:A6:15:DD:E6:0C  [redacted]           Infra  10    270 Mbit/s  45      ▂▄__  WPA1 WPA2
        B4:0F:3B:45:D1:F5  [redacted]           Infra  48    270 Mbit/s  45      ▂▄__  WPA1 WPA2
        E8:CC:18:4F:7B:44  [redacted]           Infra  157   135 Mbit/s  45      ▂▄__  WPA1 WPA2
        B0:95:8E:D8:2F:ED  [redacted]           Infra  11    405 Mbit/s  39      ▂▄__  WPA1 WPA2
        C0:61:18:FA:49:36  [redacted]           Infra  11    270 Mbit/s  24      ▂___  WPA1 WPA2
root@orangepi:~#
```

3) 然后使用 **nmcli** 命令连接扫描到的 WIFI 热点，其中：

a. **wifi_name** 需要换成想连接的 WIFI 热点的名字
b. **wifi_passwd** 需要换成想连接的 WIFI 热点的密码

```
orangepi@orangepi:~$ sudo nmcli dev wifi connect wifi_name password wifi_passwd
Device 'wlan0' successfully activated with 'cf937f88-ca1e-4411-bb50-61f402eef293'.
```

4) 通过 **ip addr show wlan0** 命令可以查看 wifi 的 IP 地址

```
orangepi@orangepi:~$ ip addr show wlan0
11: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 23:8c:d6:ae:76:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
       valid_lft 259192sec preferred_lft 259192sec
    inet6 240e:3b7:3240:c3a0:c401:a445:5002:ccdd/64 scope global dynamic noprefixroute
       valid_lft 259192sec preferred_lft 172792sec
    inet6 fe80::42f1:6019:a80e:4c31/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

5) 使用 **ping** 命令可以测试 wifi 网络的连通性，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行

```
orangepi@orangepi:~$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

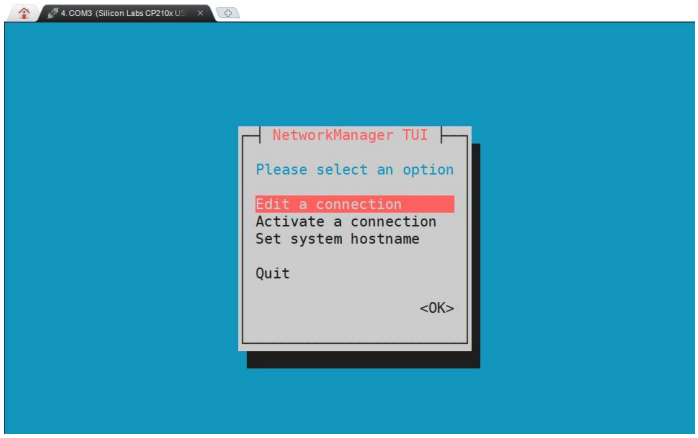
3.6.2.2. 服务器版镜像通过图形化方式连接 WIFI

1) 先登录 Linux 系统，有下面三种方式
   a. 如果开发板连接了网线，可以通过 **ssh 远程登录 Linux 系统**
   b. 如果开发板连接好了调试串口，可以使用串口终端登录 Linux 系统（串口软件请使用 MobaXterm，使用 minicom 无法显示图形界面）
   c. 如果连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 Linux 系统

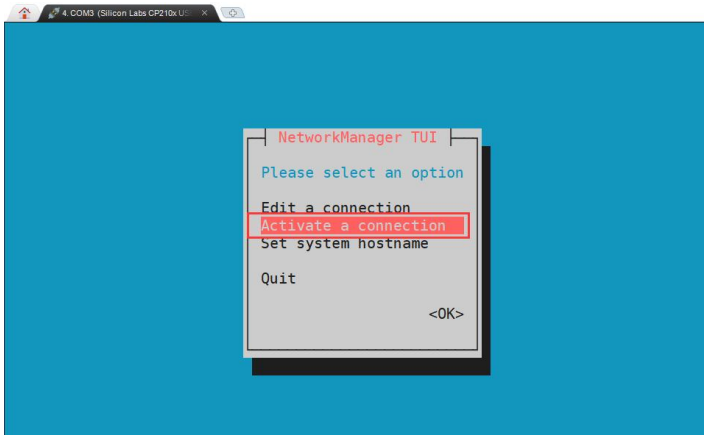
2) 然后在命令行中输入 nmtui 命令打开 wifi 连接的界面

```
orangepi@orangepi:~$ sudo nmtui
```

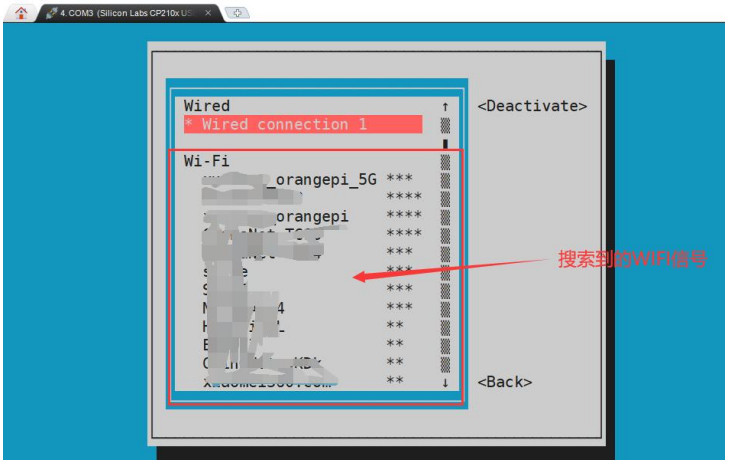
3) 输入 nmtui 命令打开的界面如下所示



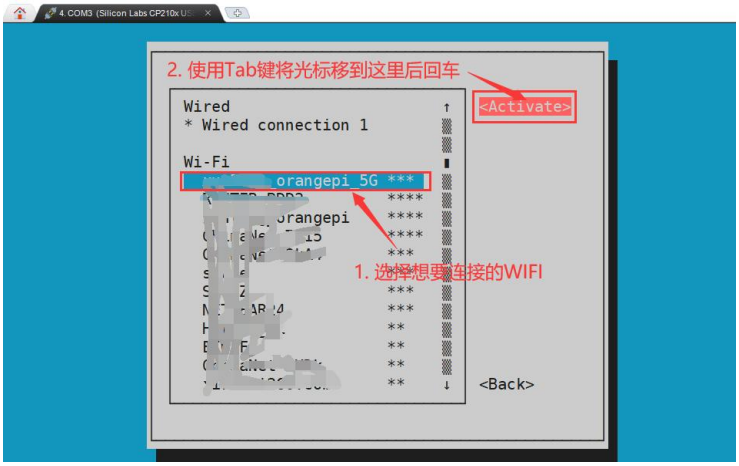
4) 选择 **Activate a connect** 后回车



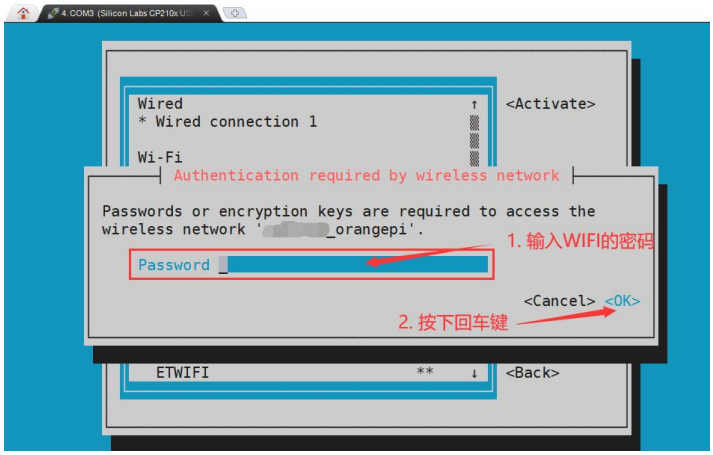
5) 然后就能看到所有搜索到的 WIFI 热点



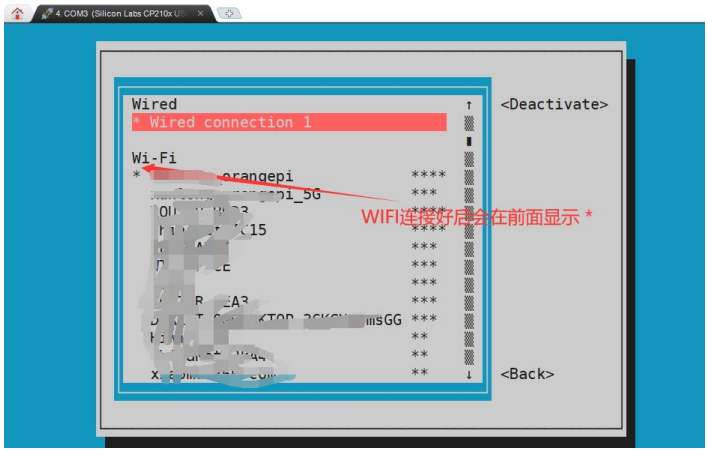
6) 选择想要连接的 WIFI 热点后再使用 Tab 键将光标定位到 **Activate** 后回车



7) 然后会弹出输入密码的对话框，在 **Password** 内输入对应的密码然后回车就会开始连接 WIFI



8) WIFI 连接成功后会在已连接的 WIFI 名称前显示一个“*”



9) 通过 **ip addr show wlan0** 命令可以查看 wifi 的 IP 地址

```
orangepi@orangepi:~$ ip addr show wlan0
11: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 24:8c:d3:aa:76:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.11/24 brd 192.168.1.255 scope global dynamic noprefixroute wlan0
       valid_lft 259069sec preferred_lft 259069sec
    inet6 240e:3b7:3240:c4a0:c401:a445:5002:ccdd/64 scope global dynamic noprefixroute
       valid_lft 259071sec preferred_lft 172671sec
    inet6 fe80::42f1:6019:a80e:4c31/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

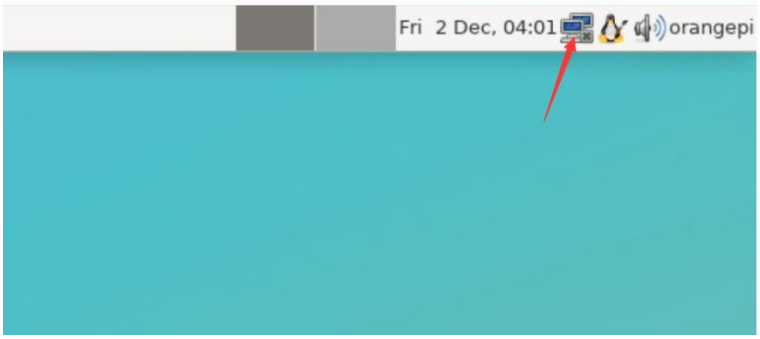
10) 使用 **ping** 命令可以测试 wifi 网络的连通性，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行

```
orangepi@orangepi:~$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
```

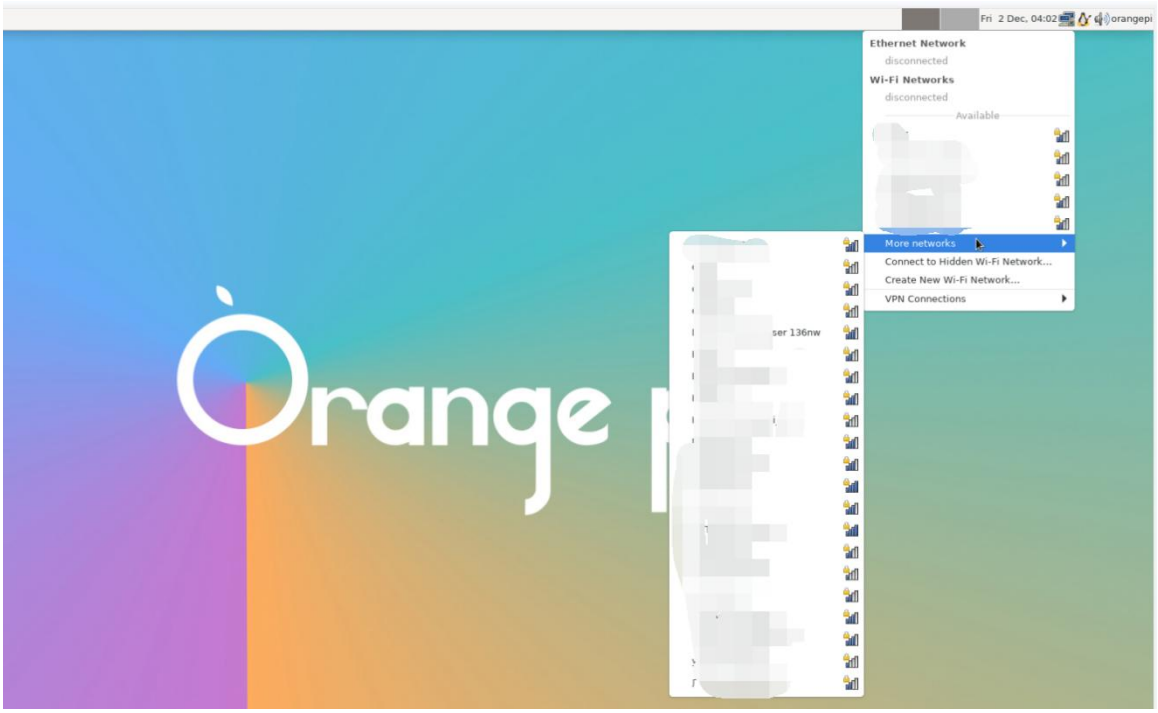
```
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

3.6.2.3. 桌面版镜像的测试方法

1) 点击桌面右上角的网络配置图标（测试 WIFI 时请不要连接网线）

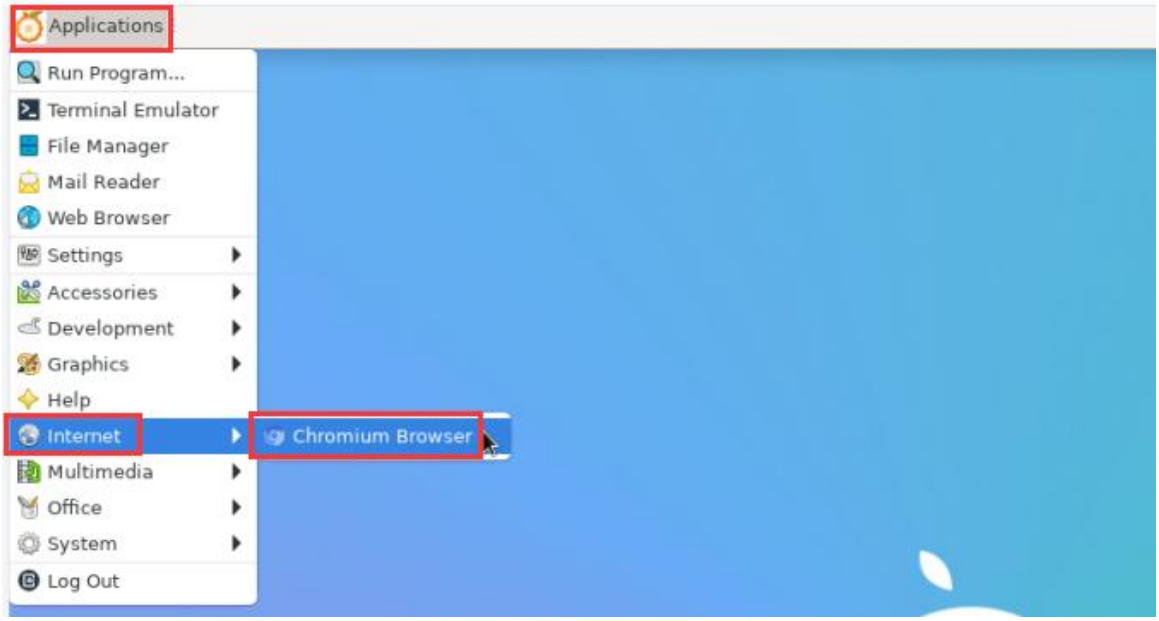


2) 在弹出的下拉框中点击 **More networks** 可以看到所有扫描到的 WIFI 热点，然后选择想要连接的 WIFI 热点



3) 然后输入 WIFI 热点的密码，再点击 **Connect** 就会开始连接 WIFI

4) 连接好 WIFI 后，可以打开浏览器查看是否能上网，浏览器的入口如下图所示



5) 打开浏览器后如果能打开其他网页说明 WIFI 连接正常

3.6.3. 设置静态 IP 地址的方法

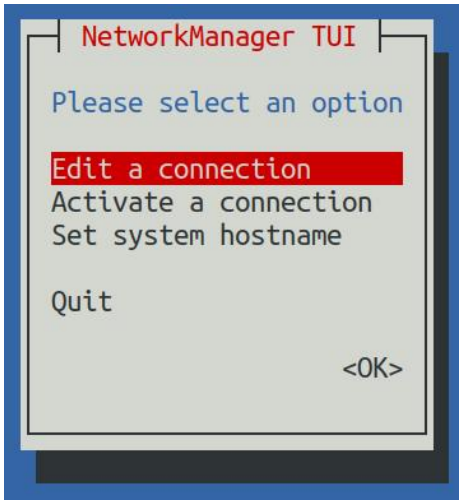
请不要通过修改/etc/network/interfaces 配置文件的方式来设置静态 IP 地址。

3.6.3.1. 使用 nmtui 命令来设置静态 IP 地址

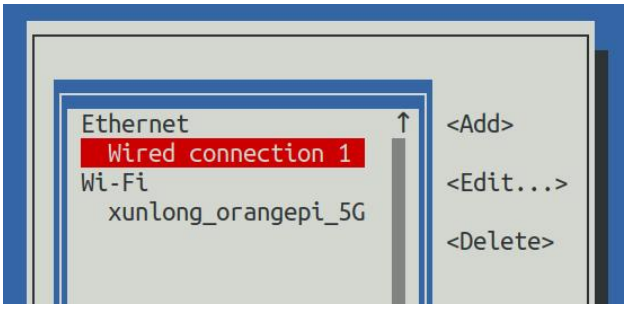
1) 首先运行 **nmtui** 命令

```
orangepi@orangepi:~$ sudo nmtui
```

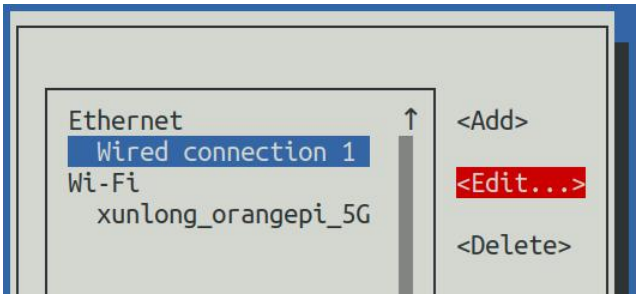
2) 然后选择 **Edit a connection** 并按下回车键



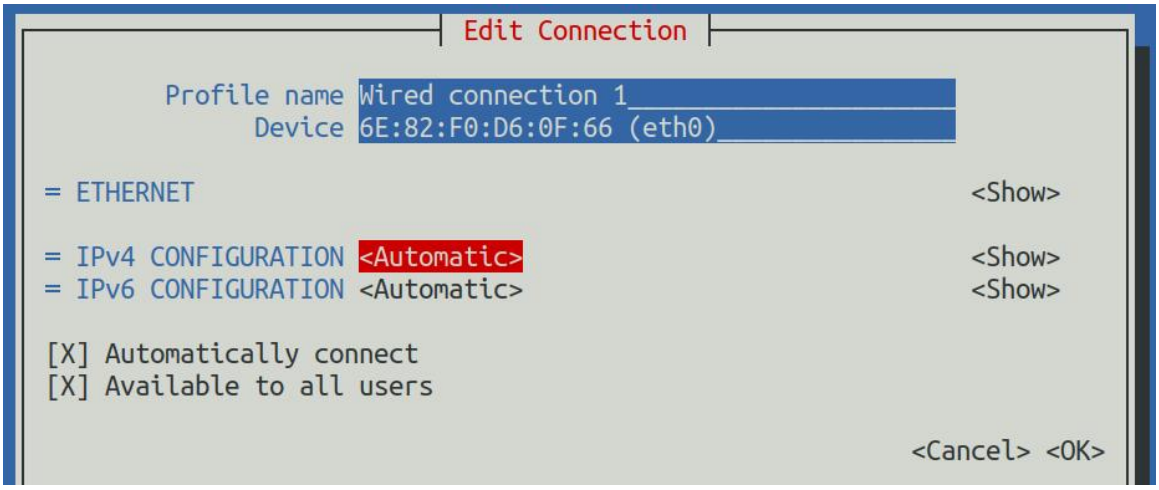
3) 然后选择需要设置静态 IP 地址的网络接口，比如设置 **Ethernet** 接口的静态 IP 地址选择 **Wired connection 1** 就可以了



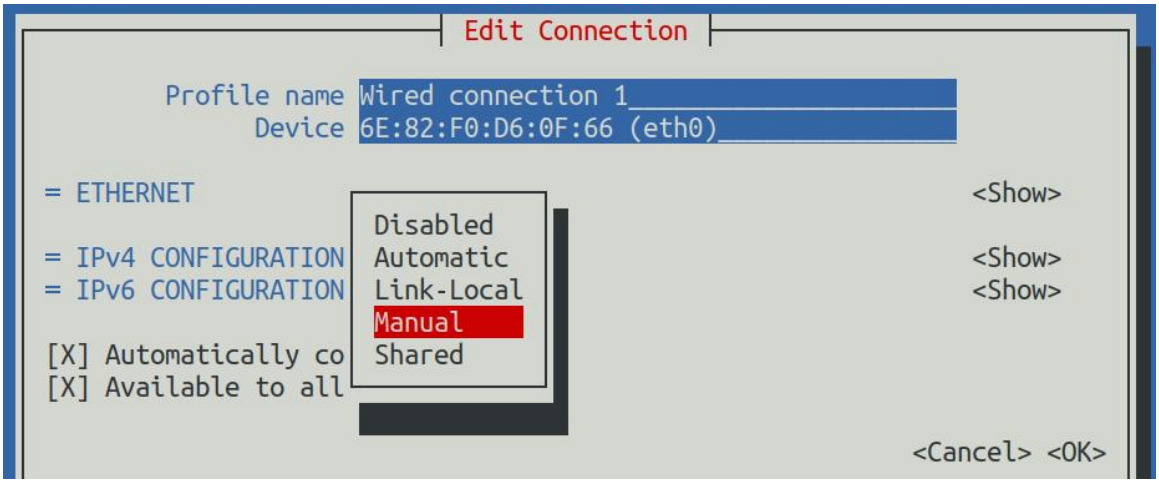
4) 然后通过 **Tab** 键选择 **Edit** 并按下回车键



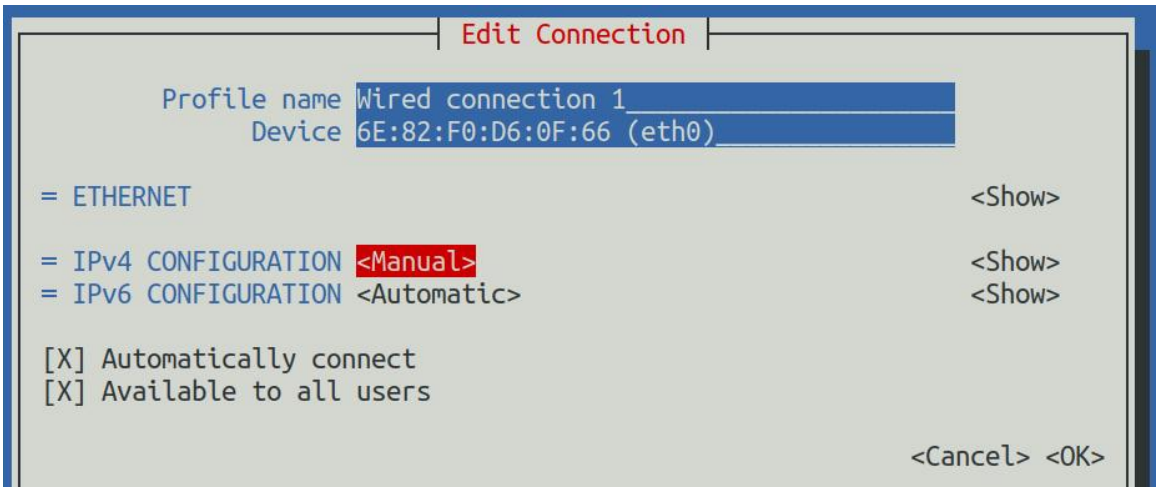
5) 然后通过 Tab 键将光标移动到下图所示的**<Automatic>**位置进行 IPv4 的配置



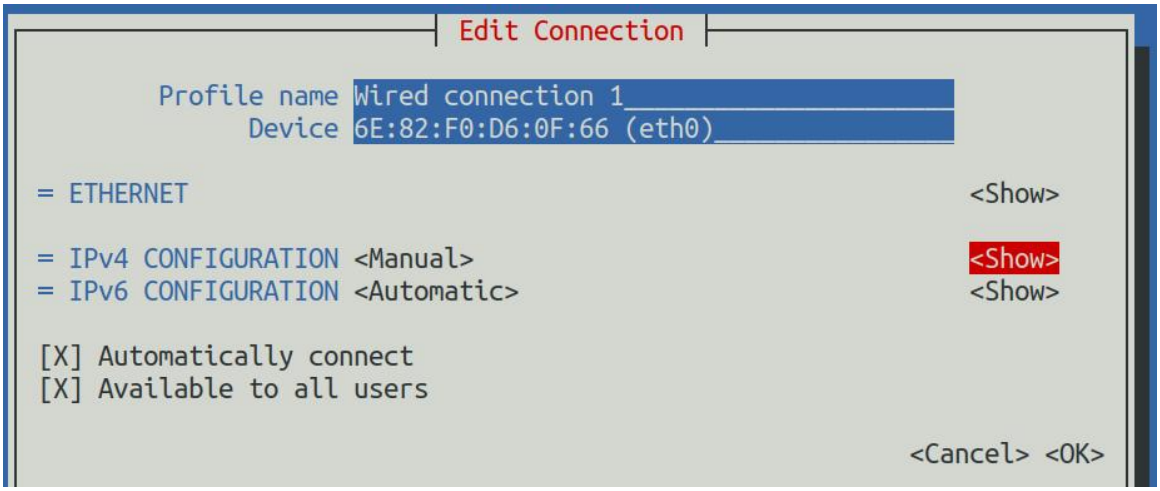
6) 然后回车，通过上下方向键选择 **Manual**，然后回车确定



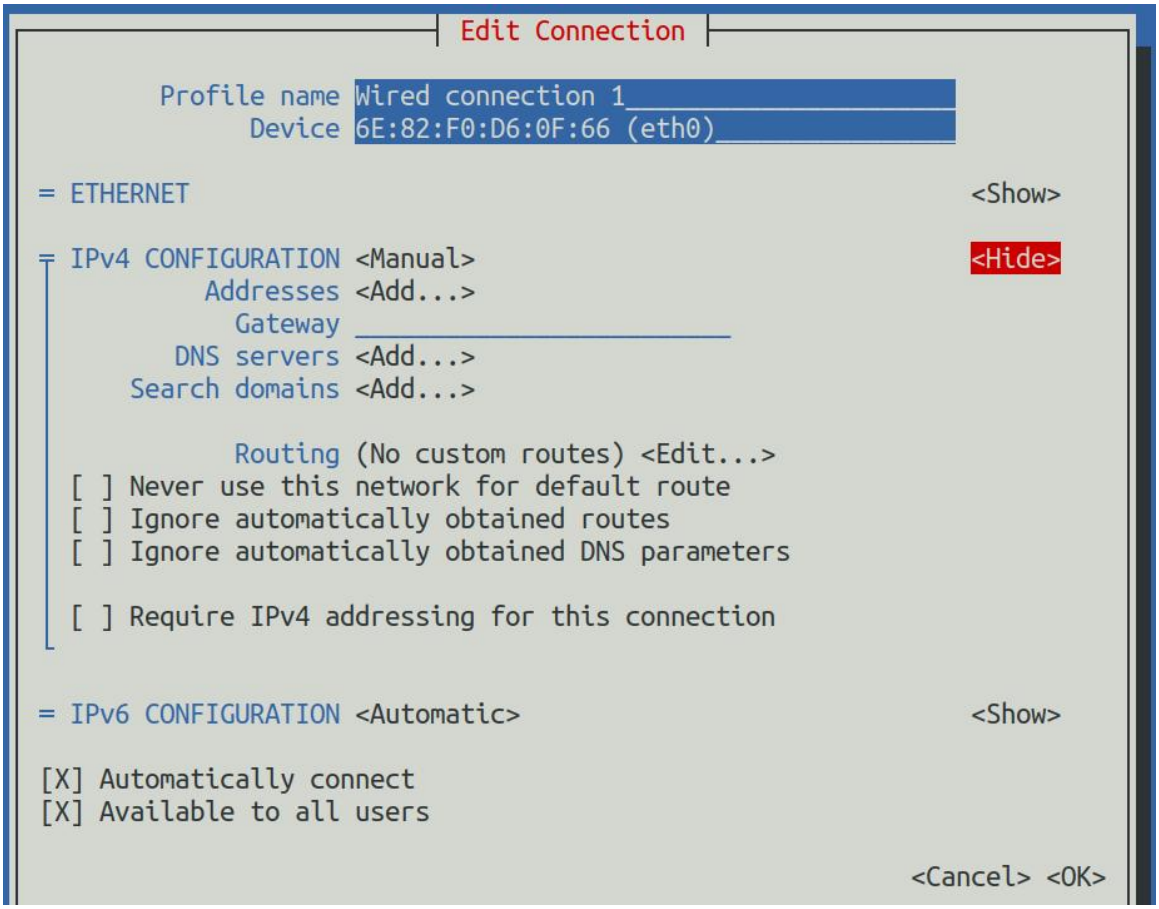
7) 选择完后的显示如下图所示



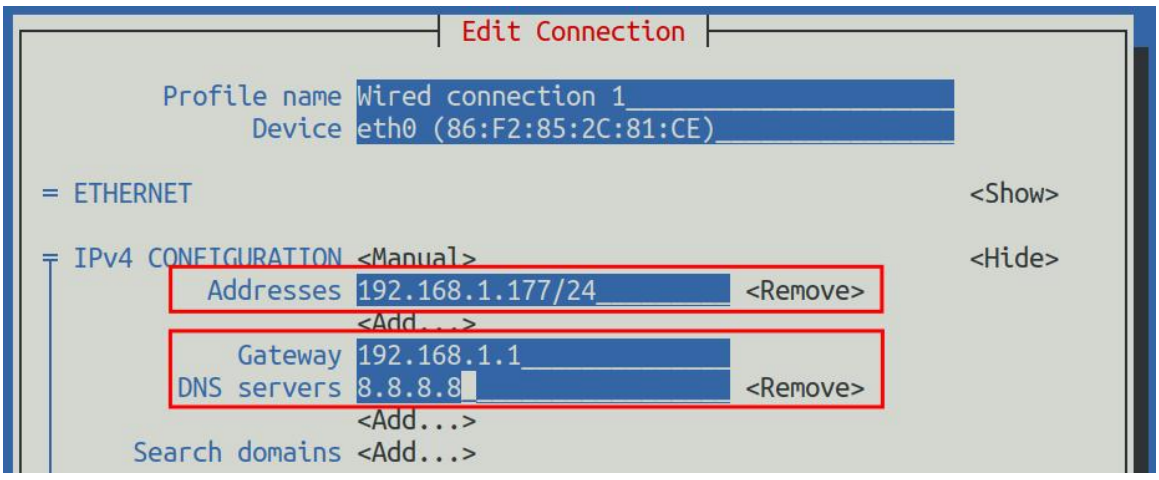
8) 然后通过 Tab 键将光标移动到**<Show>**



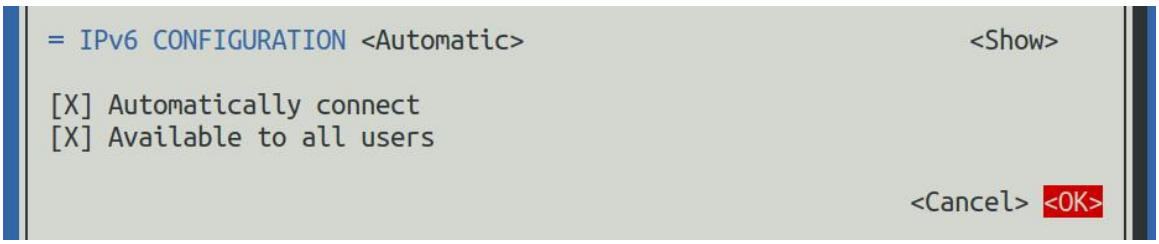
9) 然后回车，回车后会弹出下面的设置界面



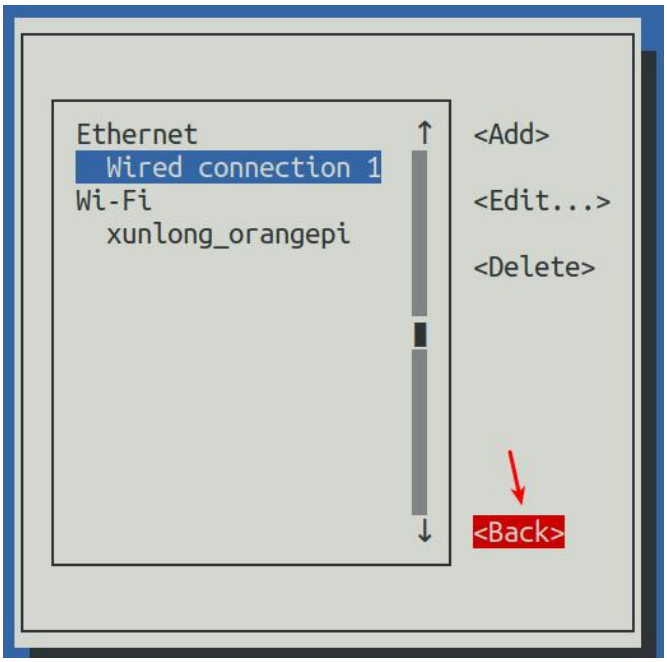
10) 然后就可以在下图所示的位置设置 IP 地址(Addresses)、网关(Gateway)和 DNS 服务器的地址（里面还有很多其他设置选项，请自行探索），**请根据自己的具体需求来设置，下图中设置的值只是一个示例**



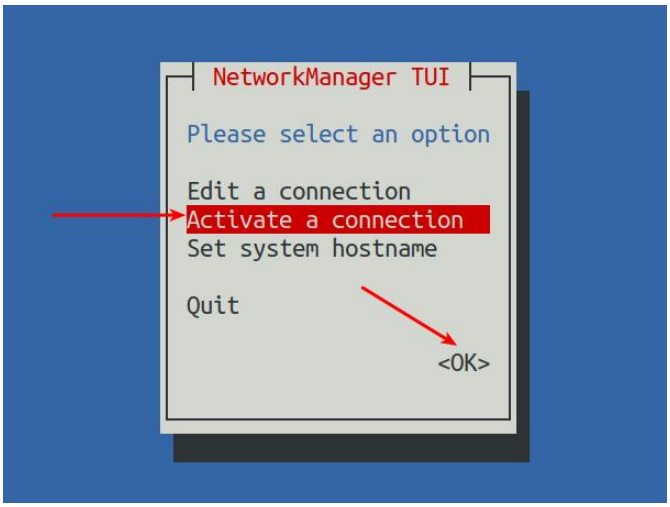
11) 设置完后将光标移动到右下角的**<OK>**，然后回车确认



12) 然后点击**<Back>**回退到上一级选择界面

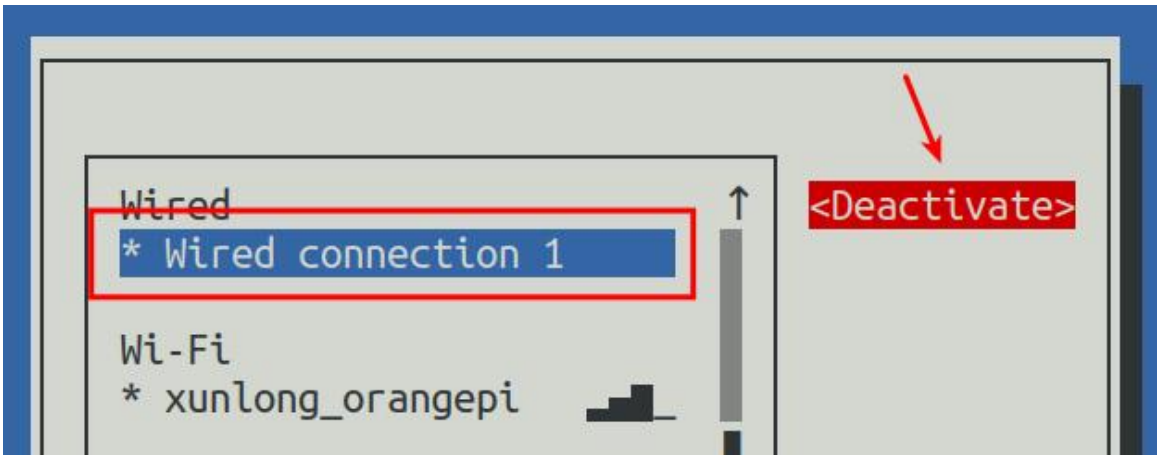


13) 然后选择 **Activate a connection**，再将光标移动到**<OK>**，最后点击回车

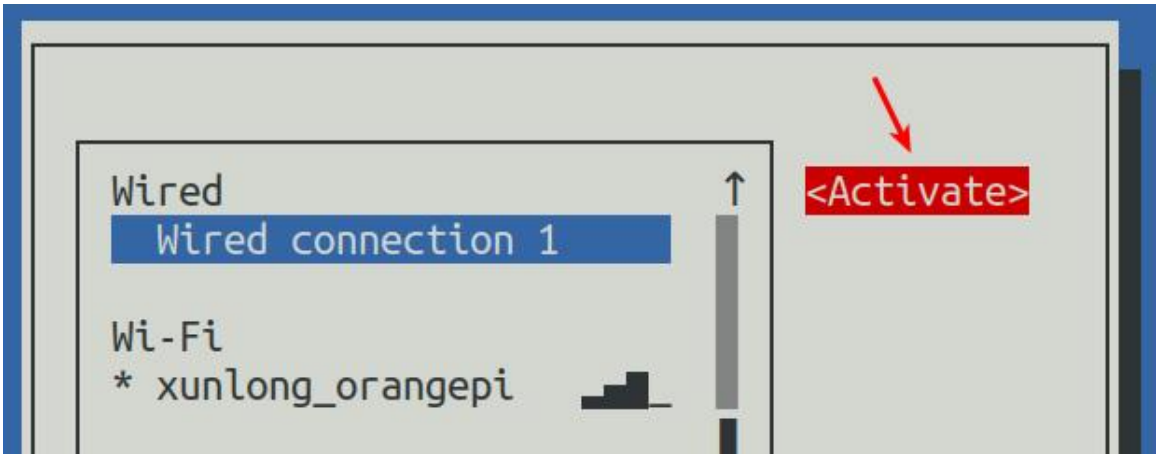


14) 然后选择需要设置的网络接口，比如 **Wired connection 1**，然后将光标移动到

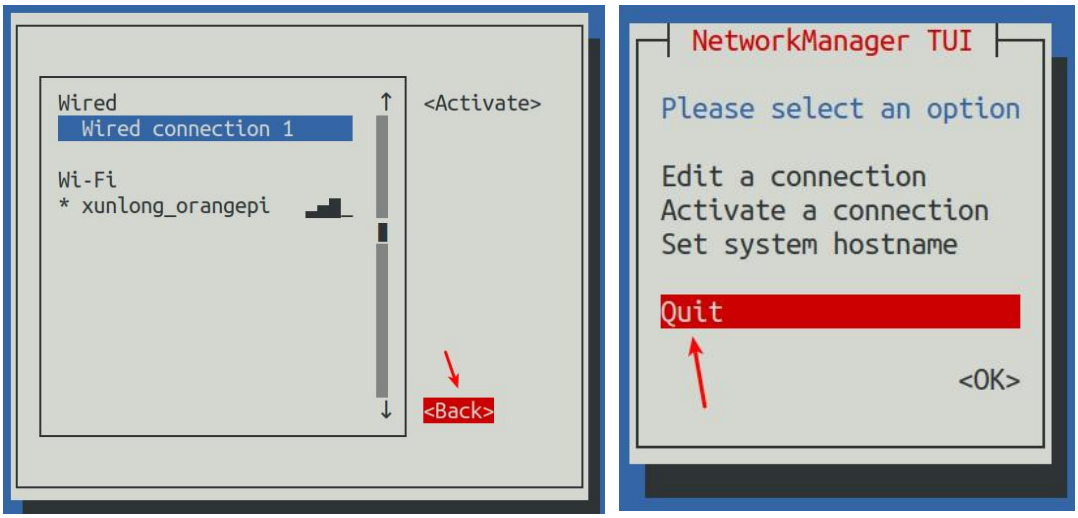
<Deactivate>，再按下回车键禁用 **Wired connection 1**



15) 然后请不要移动光标，再按下回车键重新使能 **Wired connection 1**，这样前面设置的静态 IP 地址就会生效了



16) 然后通过**<Back>**和 **Quit** 按钮就可以退出 nmtui



17) 然后通过**ip addr show eth0**就能看到网口的IP地址已经变成前面设置的静态IP地址了

```
orangepi@orangepi:~$ ip addr show eth0
3: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
UP group default qlen 1000
    link/ether 5e:ac:14:a5:92:b3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.177/24 brd 192.168.1.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 241e:3b8:3240:c3a0:e269:8305:dc08:135e/64 scope global dynamic
noprefixroute
       valid_lft 259149sec preferred_lft 172749sec
    inet6 fe80::957d:bbbe:4928:3604/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

18) 然后就可以测试网络的连通性来检查IP地址是否配置OK了，**ping**命令可以通过**Ctrl+C**快捷键来中断运行

```
orangepi@orangepi:~$ ping 192.168.1.47 -I eth0
PING 192.168.1.47 (192.168.1.47) from 192.168.1.188 eth0: 56(84) bytes of data.
64 bytes from 192.168.1.47: icmp_seq=1 ttl=64 time=0.233 ms
64 bytes from 192.168.1.47: icmp_seq=2 ttl=64 time=0.263 ms
64 bytes from 192.168.1.47: icmp_seq=3 ttl=64 time=0.273 ms
64 bytes from 192.168.1.47: icmp_seq=4 ttl=64 time=0.269 ms
64 bytes from 192.168.1.47: icmp_seq=5 ttl=64 time=0.275 ms
^C
```

```
--- 192.168.1.47 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4042ms
rtt min/avg/max/mdev = 0.233/0.262/0.275/0.015 ms
```

3.6.3.2. 使用 nmcli 命令来设置静态 IP 地址

1) 如果要设置网口的静态 IP 地址，请先将网线插入开发板，**如果需要设置 WIFI 的静态 IP 地址，请先连接好 WIFI**，然后再开始设置静态 IP 地址

2) 然后通过 **nmcli con show** 命令可以查看网络设备的名字，如下所示

a. **orangepi** 为 WIFI 网络接口的名字（名字不一定相同）

b. **Wired connection 1** 为以太网接口的名字

```
orangepi@orangepi:~$ nmcli con show
NAME                UUID                                  TYPE      DEVICE
orangepi            cfc4f922-ae48-46f1-84e1-2f19e9ec5e2a  wifi      wlan0
Wired connection 1  9db058b7-7701-37b8-9411-efc2ae8bfa30  ethernet  eth0
```

3) 然后输入下面的命令，其中

a. **"Wired connection 1"** 表示设置以太网口的静态 IP 地址，如果需要设置 WIFI 的静态 IP 地址，请修改为 WIFI 网络接口对应的名字（通过 **nmcli con show** 命令可以获取到）

b. **ipv4.addresses** 后面是要设置的静态 IP 地址，可以修改为自己想要设置的值

c. **ipv4.gateway** 表示网关的地址

```
orangepi@orangepi:~$ sudo nmcli con mod "Wired connection 1"  \
ipv4.addresses "192.168.1.110"  \
ipv4.gateway "192.168.1.1"  \
ipv4.dns "8.8.8.8"  \
ipv4.method "manual"
```

4) 然后重启 Linux 系统

```
orangepi@orangepi:~$ sudo reboot
```

5) 然后重新进入 Linux 系统使用 **ip addr show eth0** 命令就可以看到 IP 地址已经设置为想要的值了

```
orangepi@orangepi:~$ ip addr show eth0
```

```
3: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
UP group default qlen 1000
    link/ether 5e:ae:14:a5:91:b3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.110/32 brd 192.168.1.110 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 240e:3b7:3240:c3a0:97de:1d01:b290:fe3a/64 scope global dynamic
noprefixroute
       valid_lft 259183sec preferred_lft 172783sec
    inet6 fe80::3312:861a:a589:d3c/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

3.6.4. 通过 create_ap 创建 WIFI 热点的方法

create_ap是一个帮助快速创建Linux上的WIFI热点的脚本，并且支持bridge和NAT模式，能够自动结合hostapd, dnsmasq和iptables完成WIFI热点的设置，避免了用户进行复杂的配置，github地址如下：

https://github.com/oblique/create_ap

如果使用的是最新的镜像，那么就已经预装了create_ap脚本，可以通过create_ap命令来创建WIFI热点，create_ap的基本命令格式如下所示：

create_ap [options] <wifi-interface> [<interface-with-internet>]
[<access-point-name> [<passphrase>]]

*** options：可以通过该参数指定加密方式、WIFI热点的频段、频宽模式、网络共享方式等，具体可以通过create_ap -h获取到有哪些option**
*** wifi-interface：无线网卡的名称**
*** interface-with-internet：可以联网的网卡名称，一般是eth0**
*** access-point-name：热点名称**
*** passphrase：热点的密码**

3.6.4.1. create_ap 以 NAT 模式创建 WIFI 热点的方法

1）输入下面的命令以 NAT 模式创建名称为 **orangepi**、密码为 **orangepi** 的 WIFI 热点

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m nat wlan0 eth0 orangepi orangepi
```

2）如果有下面的信息输出，说明 WIFI 热点创建成功

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m nat wlan0 eth0 orangepi orangepi
Config dir: /tmp/create_ap.wlan0.conf.Ji9Coeqo
PID: 5526
Network Manager found, set wlan0 as unmanaged device... DONE
Sharing Internet using method: nat
hostapd command-line interface: hostapd_cli -p
/tmp/create_ap.wlan0.conf.Ji9Coeqo/hostapd_ctrl
wlan0: interface state UNINITIALIZED->ENABLED
wlan0: AP-ENABLED
```

3）此时拿出手机，在搜索到的 WIFI 列表中就能找到开发板创建的名为 **orangepi** 的 WIFI 热点，然后可以点击 **orangepi** 连接热点，密码就是上面设置的 **orangepi**

4）连接成功后的显示如下图所示

5）在 NAT 模式下，连接到开发板热点的无线设备是向开发板的 DHCP 服务请求 IP 地址的，所以会有两个不同的网段，如这里开发板的 IP 是 192.168.1.X

```
orangepi@orangepi:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.150  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::938f:8776:5783:afa2  prefixlen 64  scopeid 0x20<link>
        ether 4a:a0:c8:25:42:82  txqueuelen 1000  (Ethernet)
        RX packets 25370  bytes 2709590 (2.7 MB)
        RX errors 0  dropped 50  overruns 0  frame 0
        TX packets 3798  bytes 1519493 (1.5 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 83
```

而开发板的 DHCP 服务默认会给接入热点的设备分配 **192.168.12.0/24** 的 IP 地址，这时点击已经连接的 WIFI 热点 **orangepi**，然后就可以看到手机的 IP 地址是 **192.168.12.X**

6）如果想要为接入的设备指定不同的网段，可以通过-g 参数指定，如通过-g 参数指定接入点 AP 的网段为 192.168.2.1

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m nat wlan0 eth0 orangepi orangepi -g 192.168.2.1
```

此时通过手机连接到热点后，点击已经连接的 WIFI 热点 **orangepi**，然后可以看

到手机的 IP 地址是 **192.168.2.X**

7）在不指定**--freq-band** 参数的情况下，默认创建的热点是 2.4G 频段的，如果想要创建 5G 频段的热点可以通过**--freq-band 5** 参数指定，具体命令如下

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m nat wlan0 eth0 orangepi orangepi --freq-band 5
```

8）如果需要隐藏 SSID，可以指定**--hidden** 参数，具体命令如下

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m nat wlan0 eth0 orangepi orangepi --hidden
```

此时手机是搜索不到 WIFI 热点的，需要手动指定 WIFI 热点名称，并输入密码来连接 WIFI 热点

3.6.4.2. create_ap 以 bridge 模式创建 WIFI 热点的方法

1）输入下面的命令以 bridge 模式创建名称为 **orangepi**、密码为 **orangepi** 的 WIFI 热点

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m bridge wlan0 eth0 orangepi orangepi
```

2）如果有下面的信息输出，说明 WIFI 热点创建成功

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m bridge wlan0 eth0 orangepi orangepi
[sudo] password for orangepi:
Config dir: /tmp/create_ap.wlan0.conf.hXrfLdof
PID: 8372
Network Manager found, set wlan0 as unmanaged device... DONE
Sharing Internet using method: bridge
Create a bridge interface... br0 created.
hostapd command-line interface: hostapd_cli -p
/tmp/create_ap.wlan0.conf.hXrfLdof/hostapd_ctrl
wlan0: interface state UNINITIALIZED->ENABLED
```

3）此时拿出手机，在搜索到的 WIFI 列表中就能找到开发板创建的名为 **orangepi** 的 WIFI 热点，然后可以点击 **orangepi** 连接热点，密码就是上面设置的 **orangepi**

4）连接成功后的显示如下图所示

5）在 bridge 模式下，连接到开发板热点的无线设备也是向主路由（开发板连接的路由器）的 DHCP 服务请求 IP 地址的，如这里开发板的 IP 是 **192.168.1.X**

```
orangepi@orangepi:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.150  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::938f:8776:5783:afa2  prefixlen 64  scopeid 0x20<link>
        ether 4a:a0:c8:25:42:82  txqueuelen 1000  (Ethernet)
        RX packets 25370  bytes 2709590 (2.7 MB)
        RX errors 0  dropped 50  overruns 0  frame 0
        TX packets 3798  bytes 1519493 (1.5 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 83
```

而接入 WIFI 热点的设备的 IP 也是由主路由分配的，所以连接 WIFI 热点的手机和开发板处于相同的网段，这时点击已经连接的 WIFI 热点 **orangepi**，然后就可以看到手机的 IP 地址也是 **192.168.1.X**。

6）在不指定**--freq-band** 参数的情况下，默认创建的热点是 2.4G 频段的，如果想要创建 5G 频段的热点可以通过**--freq-band 5** 参数指定，具体命令如下

注意，下面的命令中，linux5.10 的 Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m bridge wlan0 eth0 orangepi orangepi --freq-band 5
```

7）如果需要隐藏 SSID，可以指定**--hidden** 参数，具体命令如下

注意，下面的命令中，Debian12 需要修改 eth0 为 end1，linux6.6 的 Debian12 需要修改 eth0 为 end0。

```
orangepi@orangepi:~$ sudo create_ap --no-virt -m bridge wlan0 eth0 orangepi orangepi --hidden
```

此时手机是搜索不到 WIFI 热点的，需要手动指定 WIFI 热点名称，并输入密码来连接 WIFI 热点

3.7. SSH 远程登录开发板

Linux 系统默认都开启了 ssh 远程登录，并且允许 root 用户登录系统。ssh 登录前首先需要确保以太网或者 wifi 网络已连接，然后使用 ip addr 命令或者通过查看路由器的方式获取开发板的 IP 地址。

3.7.1. Ubuntu 下 SSH 远程登录开发板

1) 获取开发板的 IP 地址

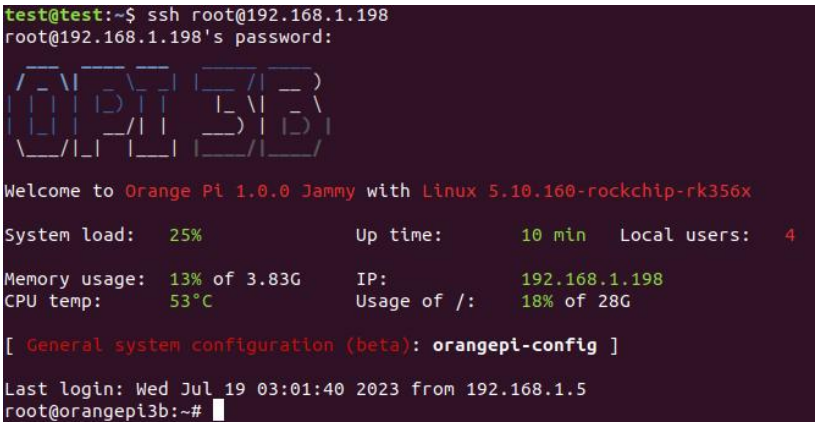
2) 然后就可以通过 ssh 命令远程登录 Linux 系统

```
test@test:~$ ssh root@192.168.1.xxx          (需要替换为开发板的 IP 地址)
root@192.168.1.xx's password:          （在这里输入密码，默认密码为 orangepi）
```

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。

如果提示拒绝连接，只要使用的是 Orange Pi 提供的镜像，就请不要怀疑 orangepi 这个密码是不是不对，而是要找其他原因。

3) 成功登录系统后的显示如下图所示



如果 ssh 无法正常登陆 Linux 系统，首先请检查下开发板的 IP 地址是否能 ping 通，如果 ping 通没问题的话，可以通过串口或者 HDMI 显示器登录 Linux 系统然后在开发板上输入下面的命令后再尝试是否能连接：

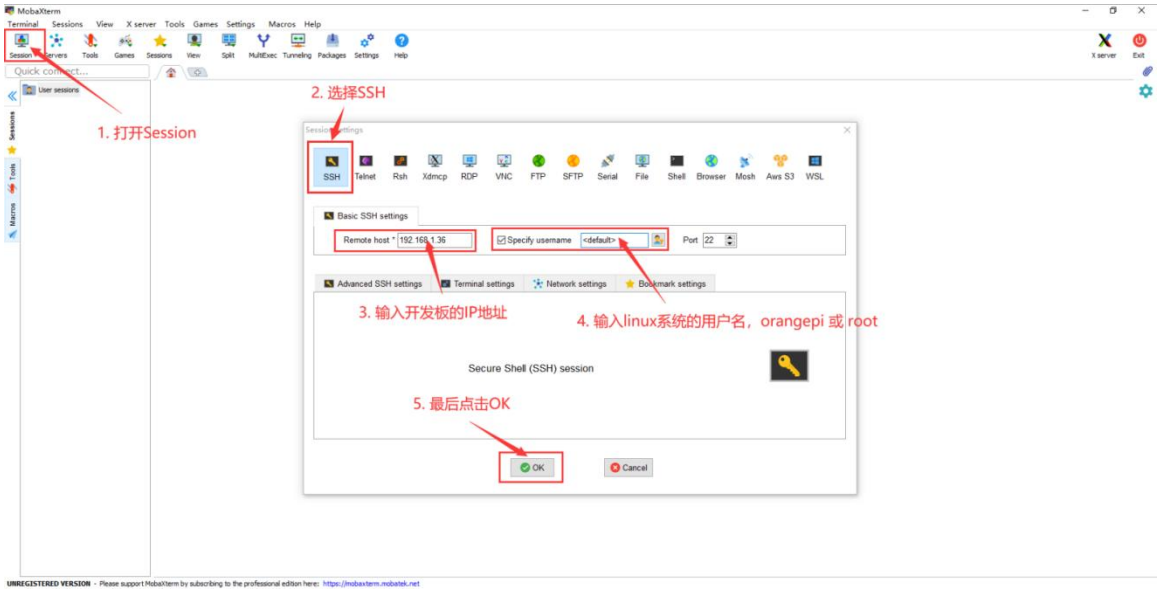
root@orangepi:~# **reset_ssh.sh**

如果还不行，请重烧系统试下。

3.7.2. Windows 下 SSH 远程登录开发板

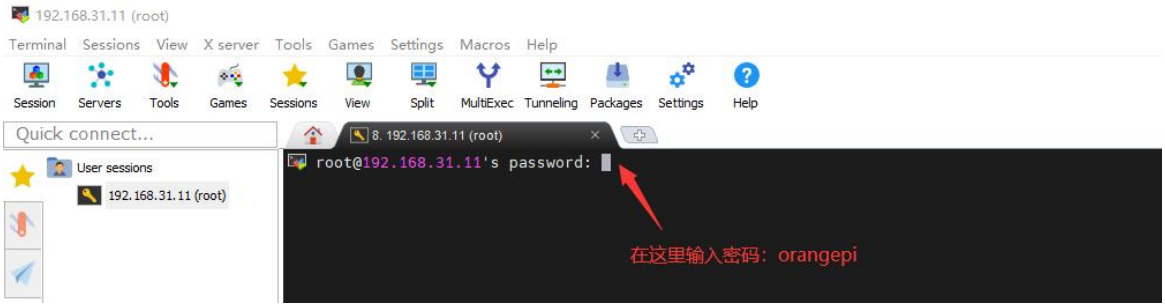
1) 首先获取开发板的 IP 地址

2) 在 windows 下可以使用 MobaXterm 远程登录开发板，首先新建一个 ssh 会话
   a. 打开 **Session**
   b. 然后在 **Session Setting** 中选择 **SSH**
   c. 然后在 **Remote host** 中输入开发板的 IP 地址
   d. 然后在 **Specify username** 中输入 Linux 系统的用户名 **root** 或 **orangepi**
   e. 最后点击 **OK** 即可

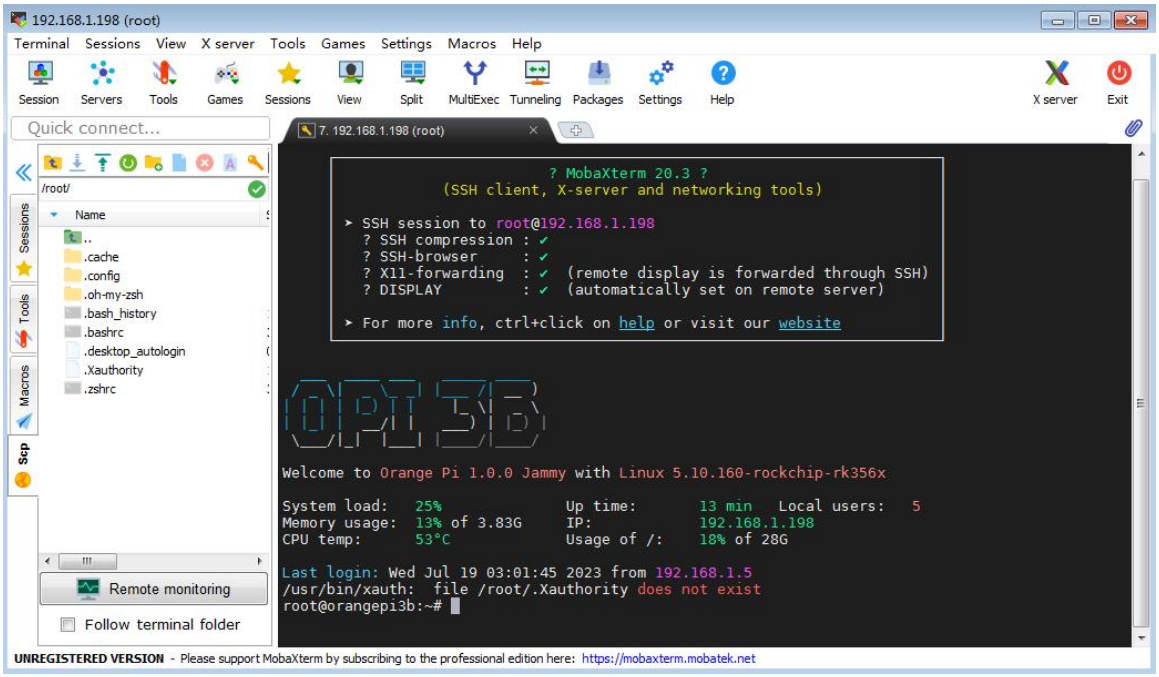


3) 然后会提示输入密码，默认 root 和 orangepi 用户的密码都为 orangepi

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。



4) 成功登录系统后的显示如下图所示



3.8. 上传文件到开发板 Linux 系统中的方法

3.8.1. 在 Ubuntu PC 中上传文件到开发板 Linux 系统中的方法

3.8.1.1. 使用 scp 命令上传文件的方法

1) 使用 scp 命令可以在 Ubuntu PC 中上传文件到开发板的 Linux 系统中，具体命令如下所示

a. **file_path：**需要替换为要上传文件的路径

b. **orangepi：**为开发板 Linux 系统的用户名，也可以替换成其它的，比如 root

c. **192.168.xx.xx:** 为开发板的 IP 地址，请根据实际情况进行修改

d. **/home/orangepi:** 开发板 Linux 系统中的路径，也可以修改为其它的路径

```
test@test:~$ scp    file_path    orangepi@192.168.xx.xx:/home/orangepi/
```

2) 如果要上传文件夹，需要加上-r 参数

```
test@test:~$ scp    -r    dir_path    orangepi@192.168.xx.xx:/home/orangepi/
```

3) scp 还有更多的用法，请使用下面的命令查看 man 手册

```
test@test:~$ man scp
```

3.8.1.2. 使用 filezilla 上传文件的方法

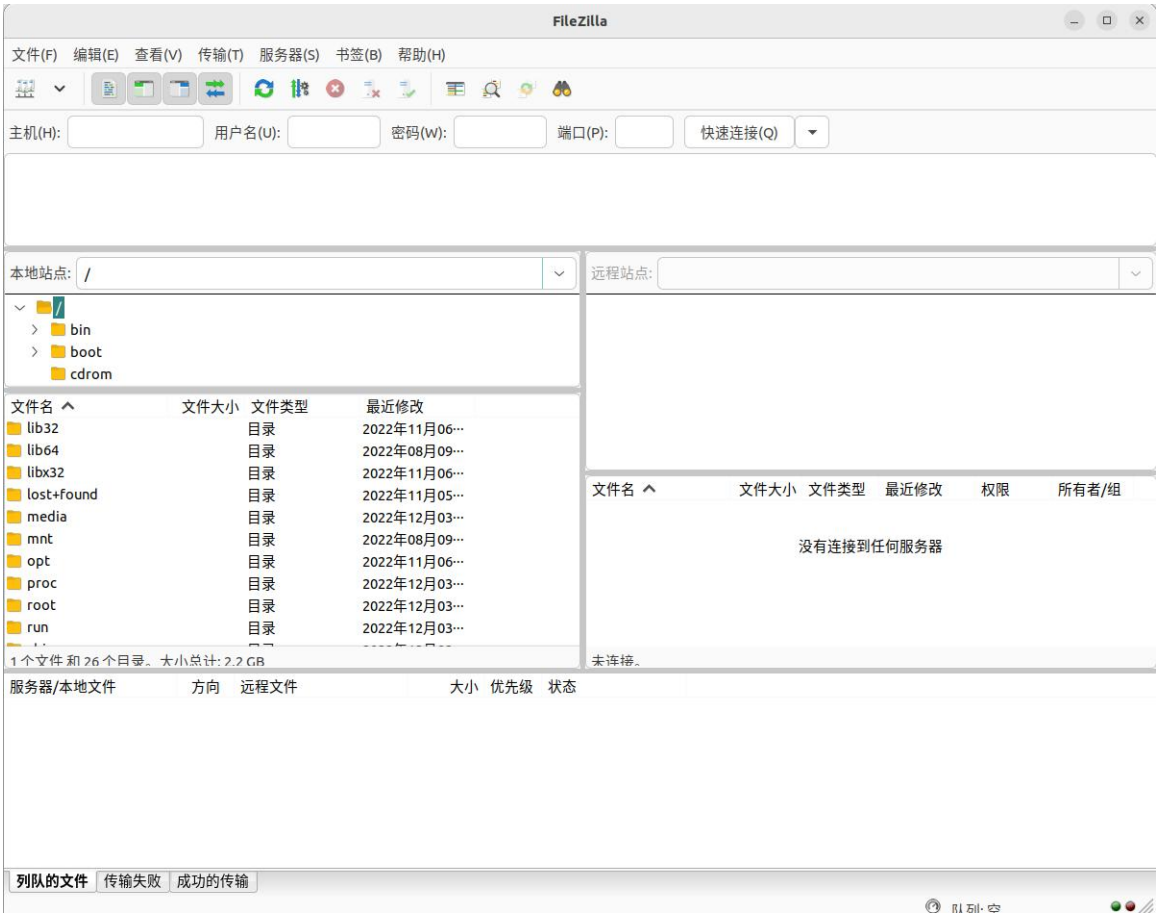
1) 首先在 Ubuntu PC 中安装 filezilla

```
test@test:~$ sudo apt install -y filezilla
```

2) 然后使用下面的命令打开 filezilla

```
test@test:~$ filezilla
```

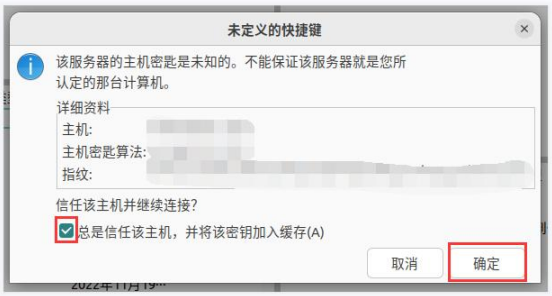
3) filezilla 打开后的界面如下所示，此时右边远程站点下面显示的是空的



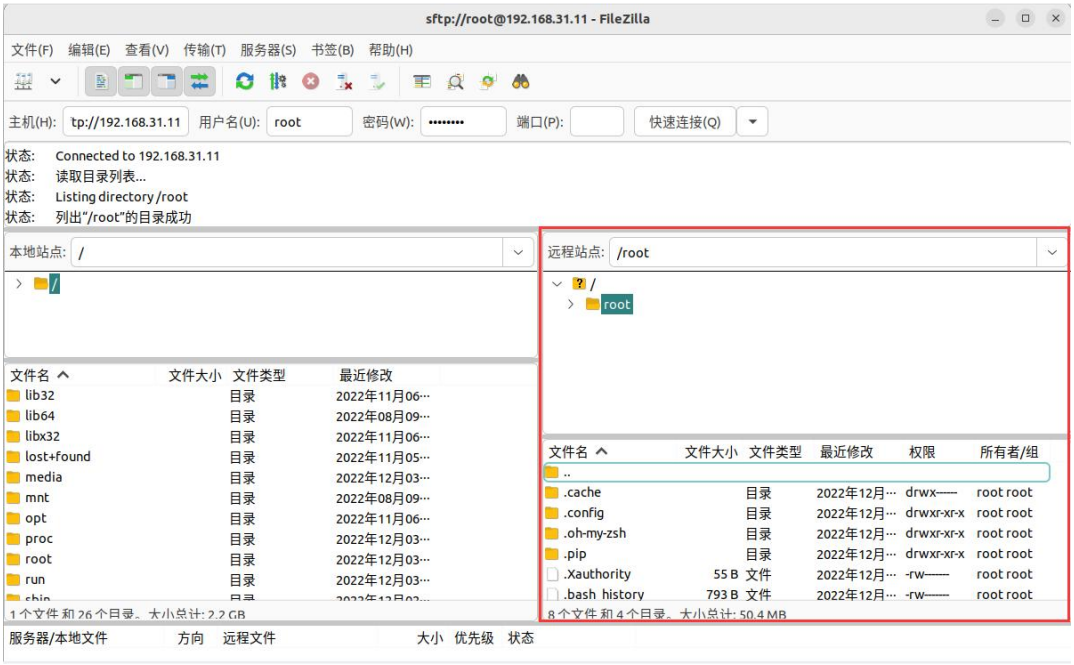
4) 连接开发板的方法如下图所示

5) 然后选择**保存密码**，再点击**确定**

6) 然后选择**总是信任该主机**，再点击**确定**

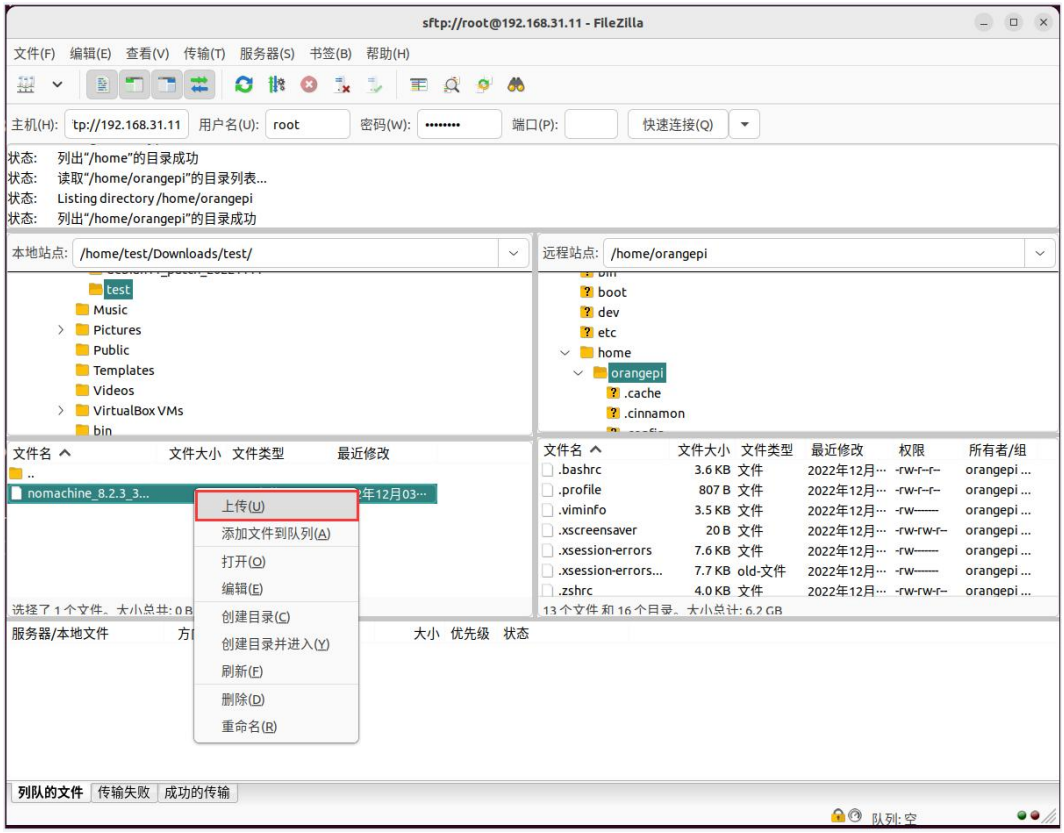


7) 连接成功后在 filezilla 软件的右边就可以看到开发板 Linux 文件系统的目录结构了



8) 然后在 filezilla 软件的右边选择要上传到开发板中的路径，再在 filezilla 软件的左

边选中 Ubuntu PC 中要上传的文件，再点击鼠标右键，再点击上传选项就会开始上传文件到开发板中了。



9) 上传完成后就可以去开发板 Linux 系统中的对应路径中查看上传的文件了

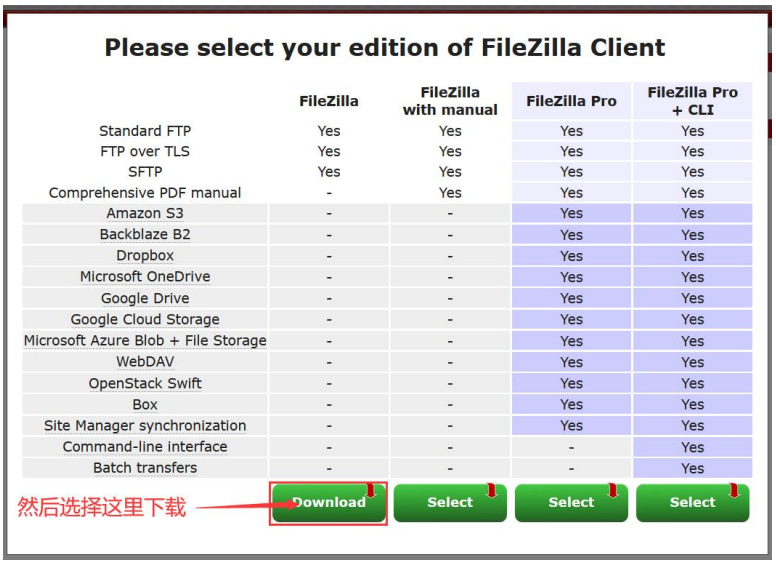
10) 上传文件夹的方法和上传文件的方法是一样的，这里就不再赘述了

3.8.2. 在 Windows PC 中上传文件到开发板 Linux 系统中的方法

3.8.2.1. 使用 filezilla 上传文件的方法

1) 首先下载 filezilla 软件 Windows 版本的安装文件，下载链接如下所示

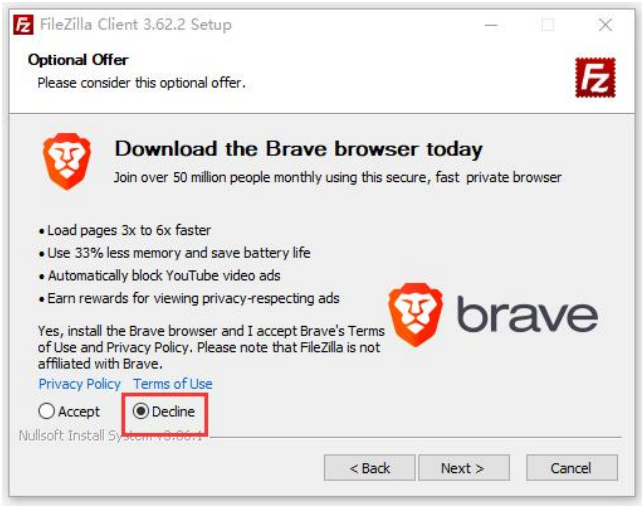
https://filezilla-project.org/download.php?type=client



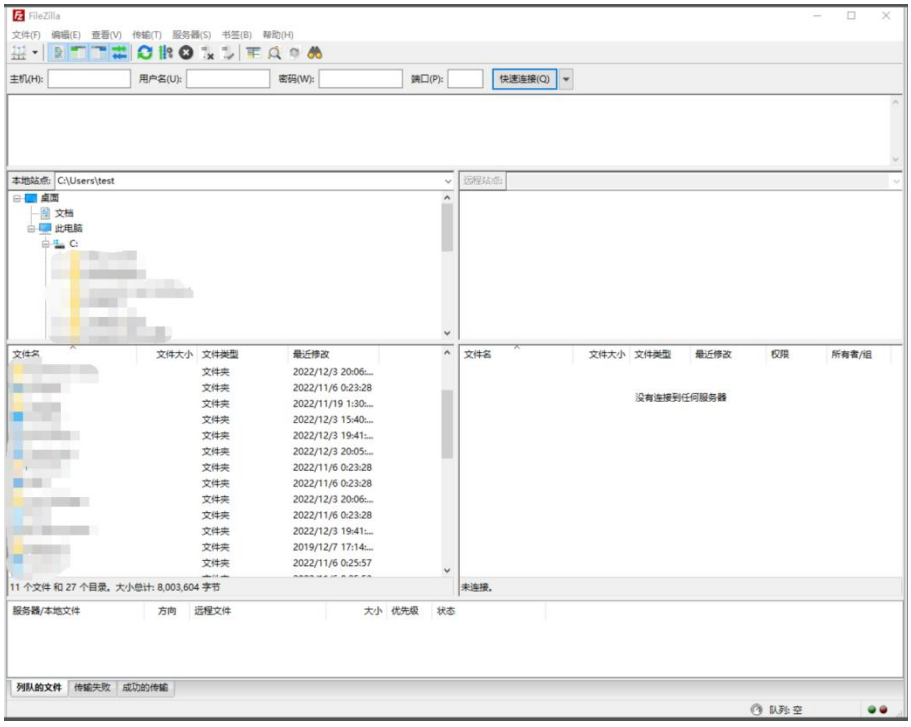
2) 下载的安装包如下所示，然后双击直接安装即可

FileZilla_Server_1.5.1_win64-setup.exe

安装过程中，下面的安装界面请选择 **Decline**，然后再选择 **Next>**

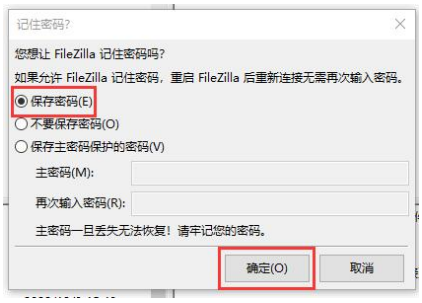


3) filezilla 打开后的界面如下所示，此时右边远程站点下面显示的是空的

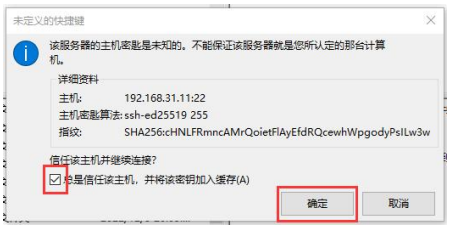


4) 连接开发板的方法如下图所示：

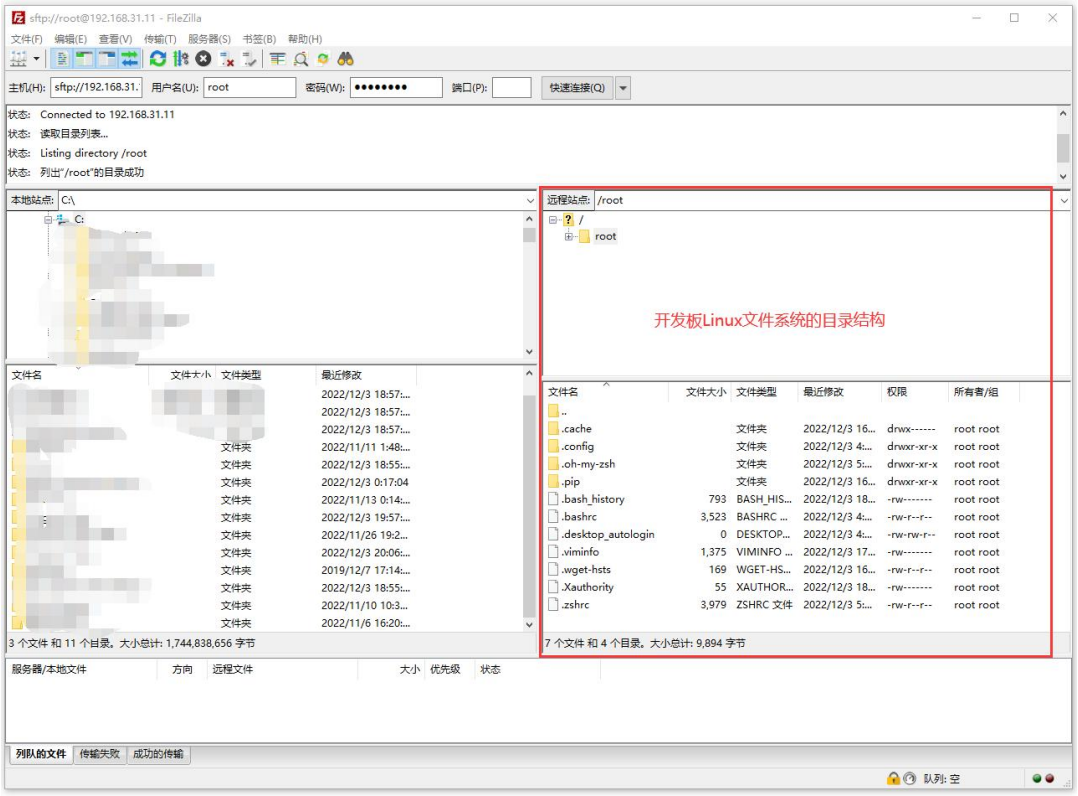
5) 然后选择**保存密码**，再点击**确定**



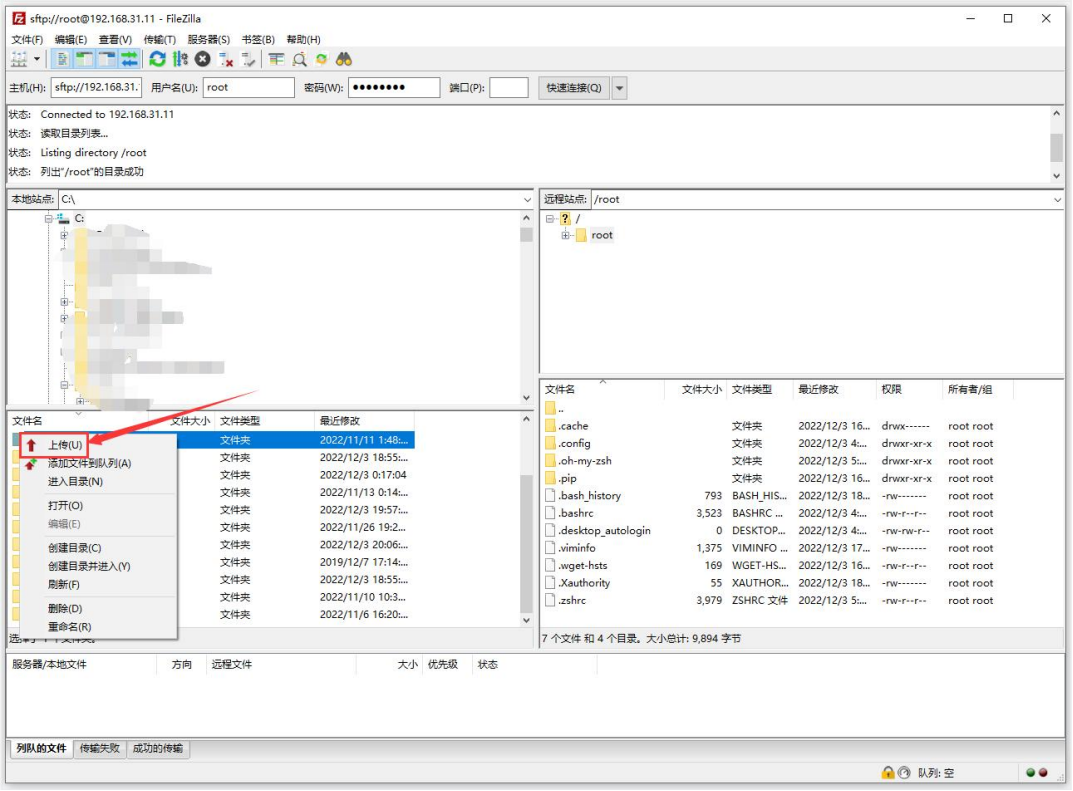
6) 然后选择**总是信任该主机**，再点击**确定**



7) 连接成功后在 filezilla 软件的右边就可以看到开发板 Linux 文件系统的目录结构了



8) 然后在 filezilla 软件的右边选择要上传到开发板中的路径，再在 filezilla 软件的左边选中 Windows PC 中要上传的文件，再点击鼠标右键，再点击上传选项就会开始上传文件到开发板中了



9) 上传完成后就可以去开发板 Linux 系统中的对应路径中查看上传的文件了

10) 上传文件夹的方法和上传文件的方法是一样的，这里就不再赘述了

3.9. HDMI 测试

3.9.1. HDMI 显示测试

1) 使用 HDMI 转 HDMI 线连接 Orange Pi 开发板和 HDMI 显示器

2) 启动 Linux 系统后如果 HDMI 显示器有图像输出说明 HDMI 接口使用正常

注意，很多笔记本电脑虽然带有 HDMI 接口，但是笔记本的 HDMI 接口一般只有输出功能，并没有 HDMI in 的功能，也就是说并不能将其他设备的 HDMI 输出显示到笔记本的屏幕上。

当想把开发板的 HDMI 接到笔记本电脑 HDMI 接口时，请先确认清楚您的笔记本是支持 HDMI in 的功能。

当 HDMI 没有显示的时候，请先检查下 HDMI 线有没有插紧，确认接线没问题后，可以换一个不同的屏幕试下有没有显示。

3.9.2. HDMI 转 VGA 显示测试

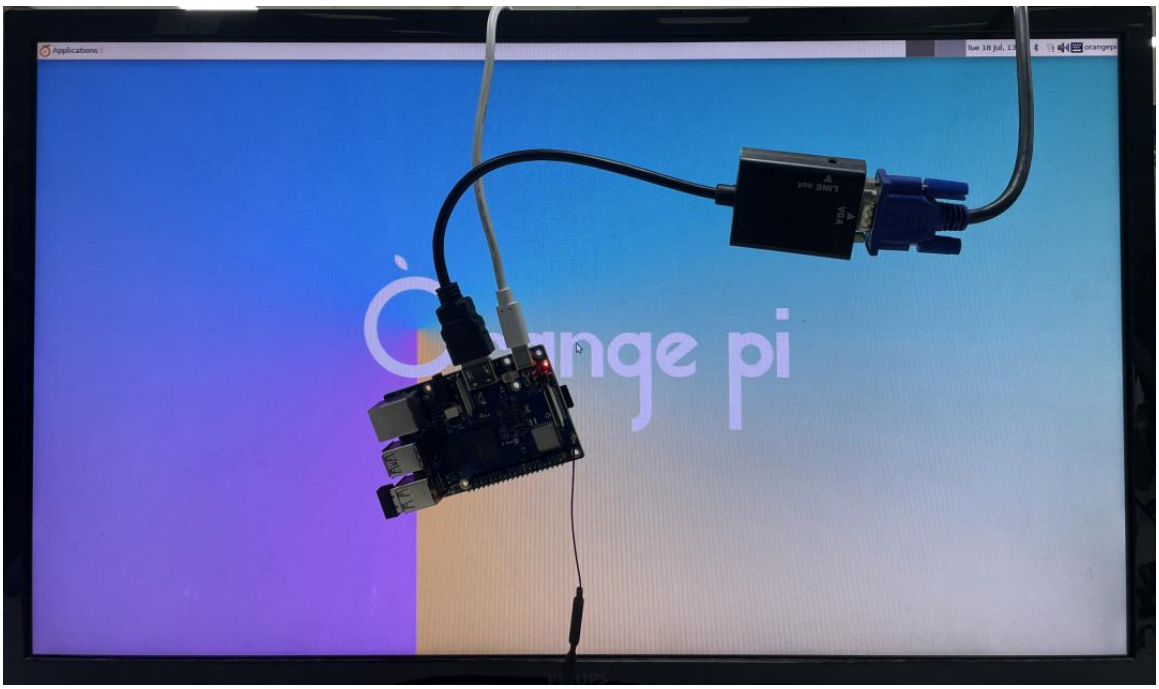
1) 首先需要准备下面的配件

a. HDMI 转 VGA 转换器

b. 一根 VGA 线

c. 一个支持 VGA 接口的显示器或者电视

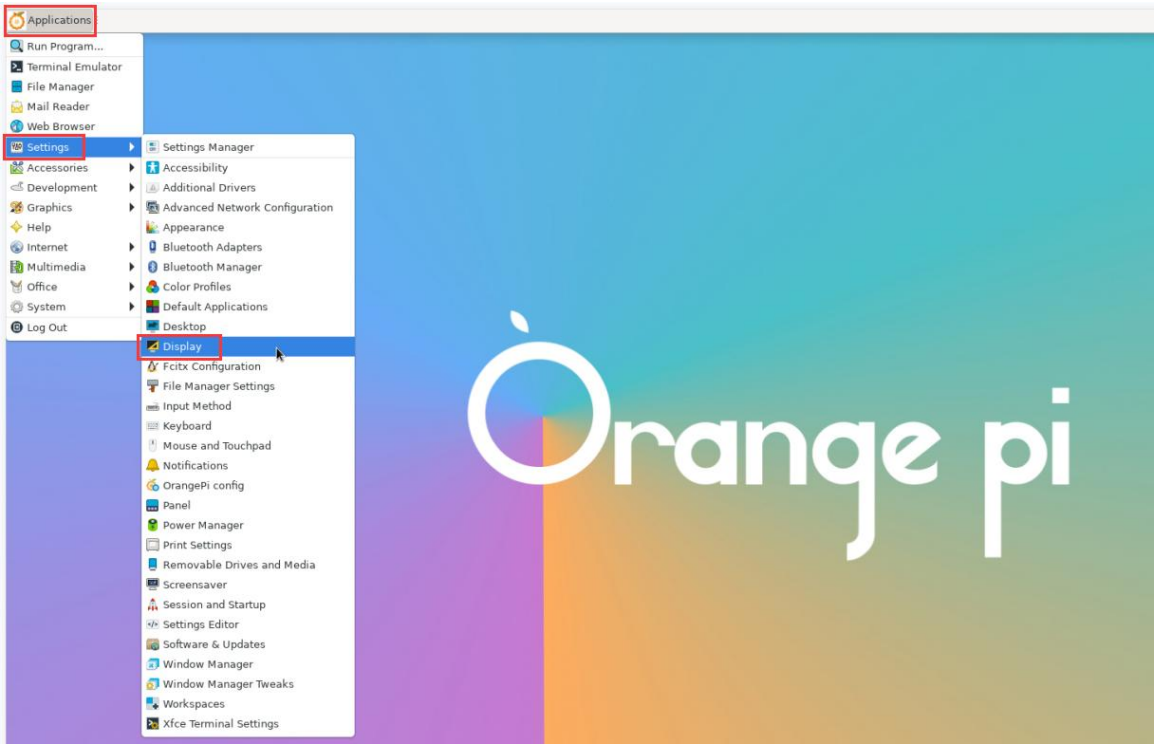
2) HDMI 转 VGA 显示测试如下所示



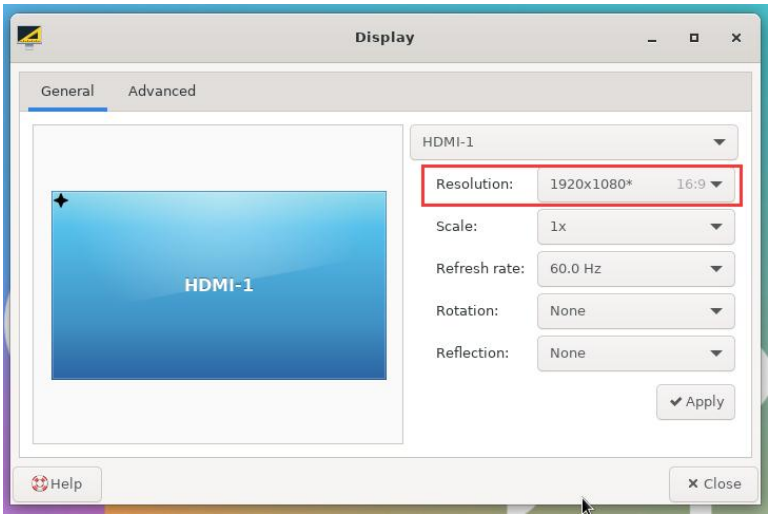
使用 HDMI 转 VGA 显示时，开发板以及开发板的 Linux 系统是不需要做任何设置的，只需要开发板 HDMI 接口能正常显示就可以了。所以如果测试有问题，请检查 HDMI 转 VGA 转换器、VGA 线以及显示器是否有问题。

3.9.3. HDMI 分辨率设置方法

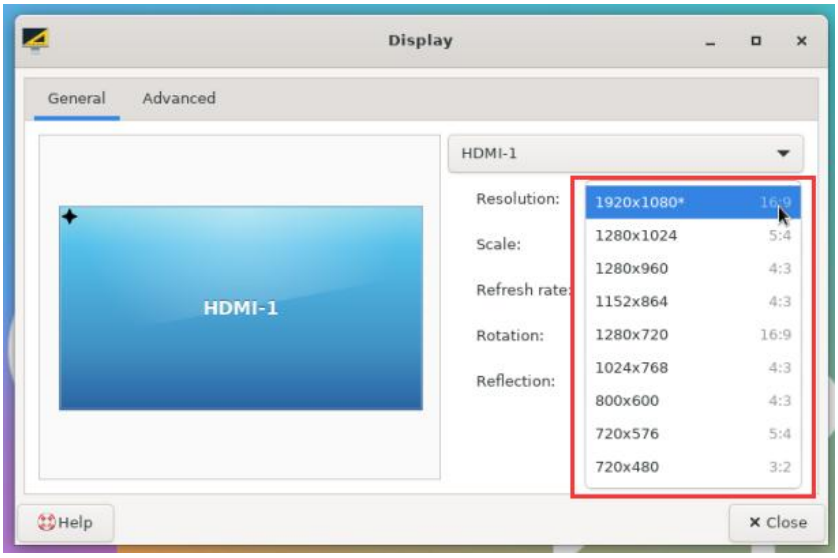
1) 首先在 **Settings** 中打开 **Display**



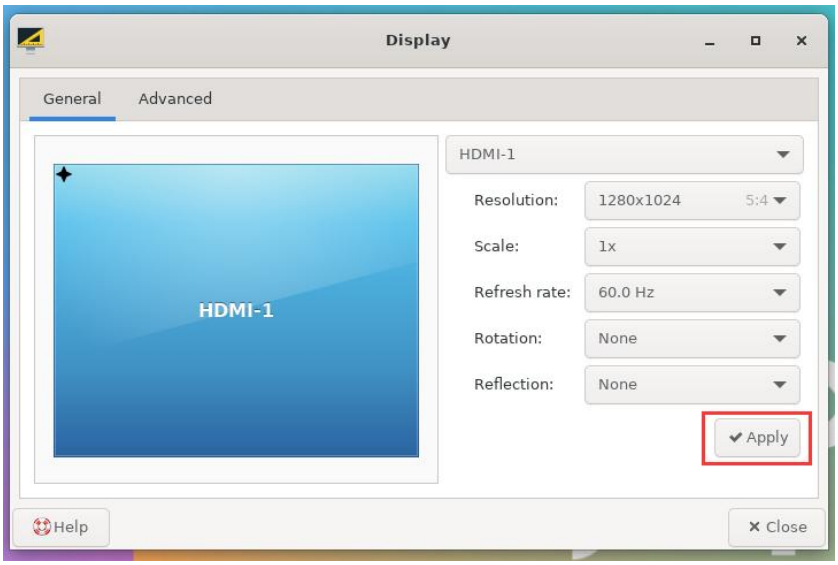
2) 然后就能看到系统当前的分辨率



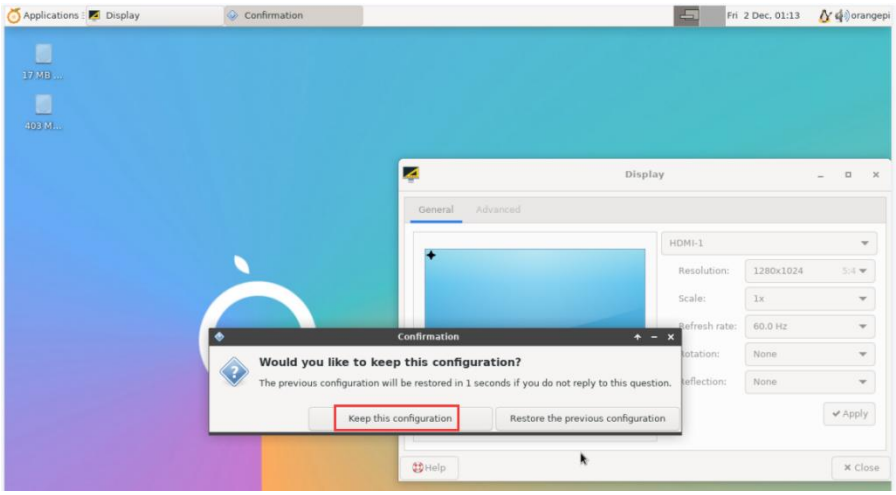
3) 点击 Resolution（分辨率）的下拉框，就可以看到显示器当前支持的所有分辨率



4) 然后选择想要设置的分辨率，再点击 Apply



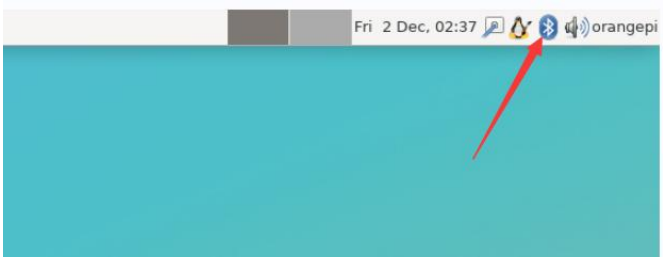
5) 等新的分辨率设置完后再选择 **Keep the configuration** 即可



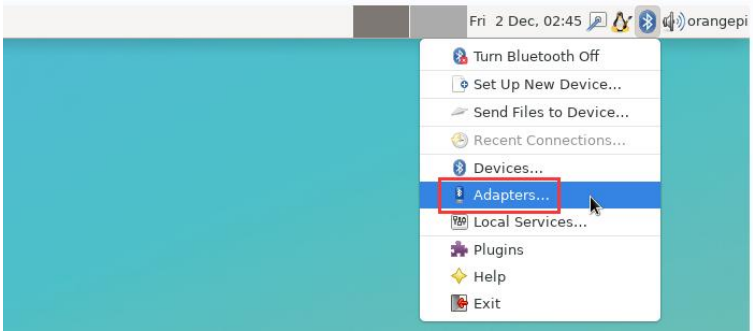
3.10. 蓝牙使用方法

3.10.1. 桌面版镜像的测试方法

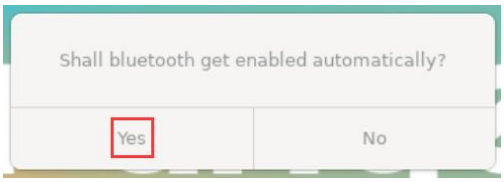
1) 点击桌面右上角的蓝牙图标



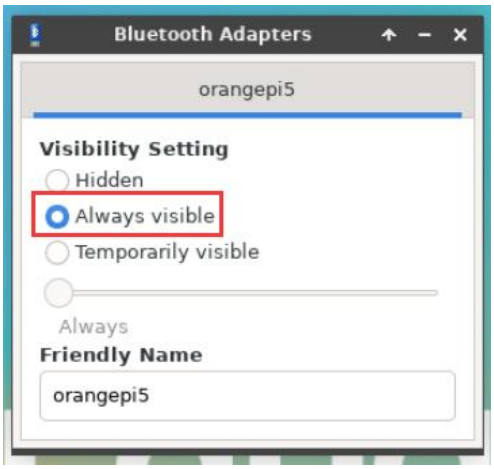
2) 然后选择适配器



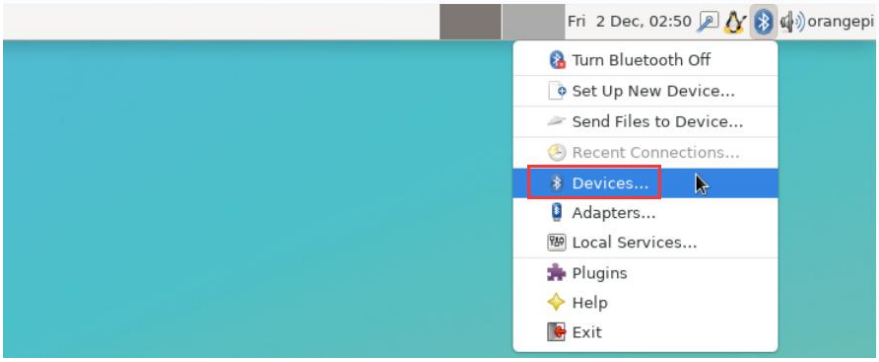
3) 如果有提示下面的界面，请选择 **Yes**



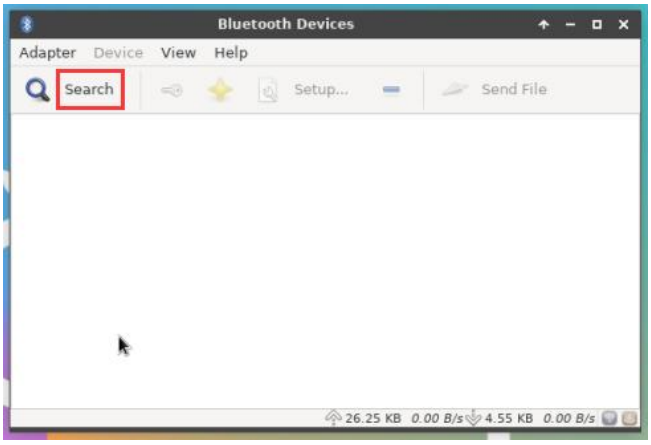
4) 然后在蓝牙的适配器设置界面中设置 **Visibility Setting** 为 **Always visible**，然后关闭即可



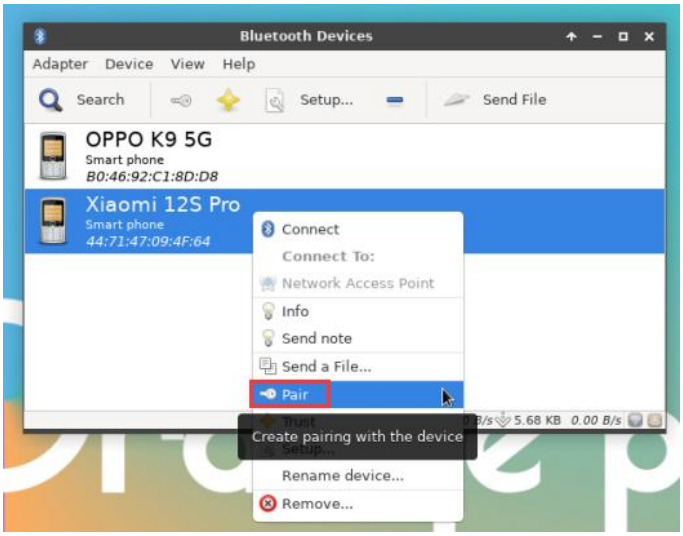
5) 然后打开蓝牙设备的配配置界面



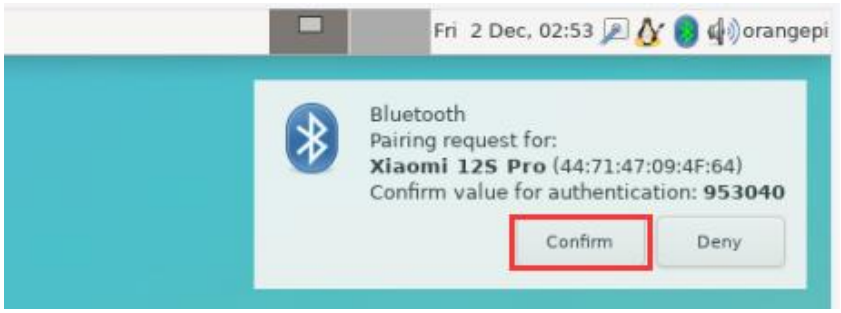
6) 点击 **Search** 即可开始扫描周围的蓝牙设备



7) 然后选择想要连接的蓝牙设备，再点击鼠标右键就会弹出对此蓝牙设备的操作界面，选择 **Pair** 即可开始配对，这里演示的是和 Android 手机配对

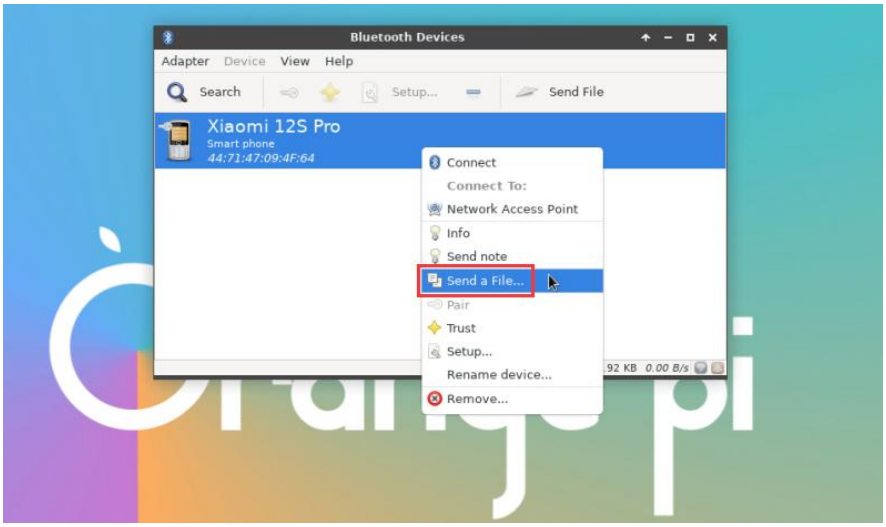


8) 配对时，桌面的右上角会弹出配对确认框，选择 **Confirm** 确认即可，此时手机上也同样需要进行确认

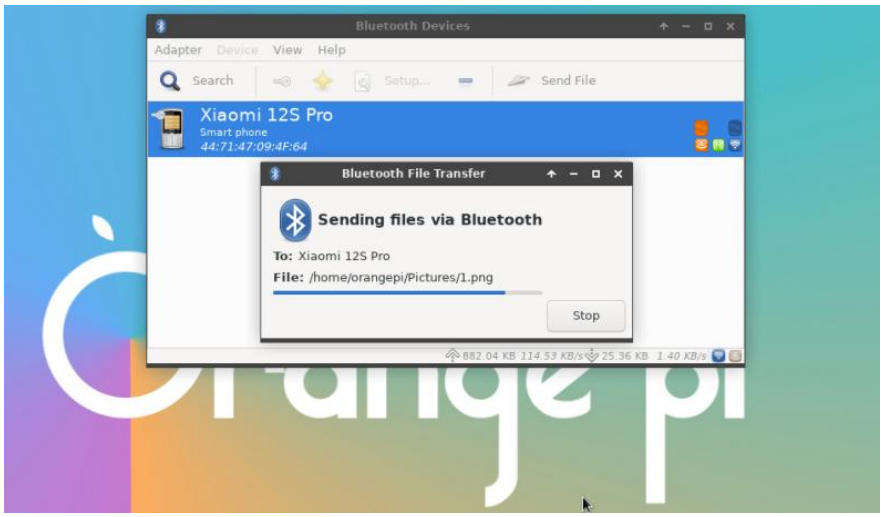


9) 和手机配对完后，可以选择已配对的蓝牙设备，然后右键选择 **Send a File** 即可

开始给手机发送一张图片



10) 发送图片的界面如下所示



3.11. USB 接口测试

USB 接口是可以接 USB hub 来扩展 USB 接口的数量的。

3.11.1. 连接 USB 鼠标或键盘测试

1) 将 USB 接口的键盘插入 Orange Pi 开发板的 USB 接口中

2) 连接 Orange Pi 开发板到 HDMI 显示器

3) 如果鼠标或键盘能正常操作系统说明 USB 接口使用正常（鼠标只有在桌面版的系统中才能使用）

3.11.2. 连接 USB 存储设备测试

1) 首先将 U 盘或者 USB 移动硬盘插入 Orange Pi 开发板的 USB 接口中

2) 执行下面的命令如果能看到 sdX 的输出说明 U 盘识别成功

```
orangepi@orangepi:~$ cat /proc/partitions | grep "sd*"
major minor  #blocks  name
   8        0   30044160 sda
   8        1   30043119 sda1
```

3) 使用 mount 命令可以将 U 盘挂载到**/mnt** 中，然后就能查看 U 盘中的文件了

```
orangepi@orangepi:~$ sudo mount /dev/sda1 /mnt/
orangepi@orangepi:~$ ls /mnt/
test.txt
```

4) 挂载完后通过 **df -h** 命令就能查看 U 盘的容量使用情况和挂载点

```
orangepi@orangepi:~$ df -h | grep "sd"
/dev/sda1              29G  208K   29G   1% /mnt
```

3.11.3. USB 无线网卡测试

目前**测试过的**能用的 USB 无线网卡如下所示，其他型号的 USB 无线网卡请自行测试，如果无法使用就需要移植对应的 USB 无线网卡驱动。

序号	型号	
1	RTL8723BU 支持 2.4G WIFI+BT4.0	
2	RTL8811 支持 2.4G +5G WIFI	

3	RTL8821CU 支持 2.4G +5G WIFI 支持 BT 4.2	

3.11.3.1. RTL8723BU 测试

1) 首先将 RTL8723BU 无线网卡模块插入开发板的 USB 接口中

2) 然后 Linux 系统会自动加载 RTL8723BU 蓝牙和 WIFI 相关的内核模块，通过 lsmod 命令可以看到下面内核模块已自动加载

```
orangepi@orangepi:~$ lsmod
Module                  Size  Used by
rfcomm                 57344  16
rtl8xxxu              106496  0
rtk_btusb              61440  0
```

3) 通过 dmesg 命令可以看到 RTL8723BU 模块的加载信息

```
orangepi@orangepi:~$ dmesg
......
[    83.438901] usb 2-1: new high-speed USB device number 2 using ehci-platform
[    83.588375] usb 2-1: New USB device found, idVendor=0bda, idProduct=b720, bcdDevice= 2.00
[    83.588403] usb 2-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[    83.588422] usb 2-1: Product: 802.11n WLAN Adapter
[    83.588443] usb 2-1: Manufacturer: Realtek
[    83.588460] usb 2-1: SerialNumber: 00e04c000001
[    83.601974] Bluetooth: hci0: RTL: examining hci_ver=06 hci_rev=000b lmp_ver=06 lmp_subver=8723
[    83.603894] Bluetooth: hci0: RTL: rom_version status=0 version=1
[    83.603920] Bluetooth: hci0: RTL: loading rtl_bt/rtl8723b_fw.bin
[    83.610108] Bluetooth: hci0: RTL: loading rtl_bt/rtl8723b_config.bin
[    83.611274] Bluetooth: hci0: RTL: cfg_sz 68, total sz 22564
[    83.658494] rtk_btusb: Realtek Bluetooth USB driver ver 3.1.6d45ddf.20220519-142432
```

```
[    83.658651] usbcore: registered new interface driver rtk_btusb
[    83.667124] usb 2-1: This Realtek USB WiFi dongle (0x0bda:0xb720) is untested!
[    83.667137] usb 2-1: Please report results to Jes.Sorensen@gmail.com
[    83.890140] usb 2-1: Vendor: Realtek
[    83.890153] usb 2-1: Product: 802.11n WLAN Adapter
[    83.890159] usb 2-1: rtl8723bu_parse_efuse: dumping efuse (0x200 bytes):
......
[    83.890412] usb 2-1: RTL8723BU rev E (SMIC) 1T1R, TX queues 3, WiFi=1, BT=1,
GPS=0, HI PA=0
[    83.890417] usb 2-1: RTL8723BU MAC: 00:13:ef:f4:58:ae
[    83.890421] usb 2-1: rtl8xxxu: Loading firmware rtlwifi/rtl8723bu_nic.bin
[    83.895289] usb 2-1: Firmware revision 35.0 (signature 0x5301)
[    84.050893] Bluetooth: hci0: RTL: fw version 0x0e2f9f73
[    84.266905] Bluetooth: RFCOMM TTY layer initialized
[    84.266949] Bluetooth: RFCOMM socket layer initialized
[    84.266999] Bluetooth: RFCOMM ver 1.11
[    84.884270] usbcore: registered new interface driver rtl8xxxu
[    84.912046] rtl8xxxu 2-1:1.2 wlx0013eff458ae: renamed from wlan0
```

4) 然后通过 **sudo ifconfig** 命令可以看到 RTL8723BU WIFI 的设备节点，WIFI 的连接和测试方法请参看 **WIFI 连接测试**一节，这里不再赘述

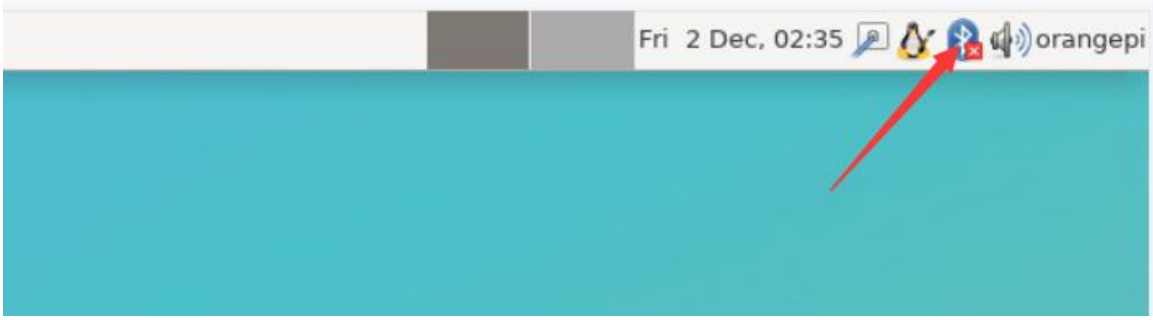
```
orangepi@orangepi:~$ sudo ifconfig wlx0013eff458ae
wlx0013eff458ae: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 00:13:ef:f4:58:ae  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

5) 然后通过 **hciconfig** 命令可以看到 USB 的蓝牙设备

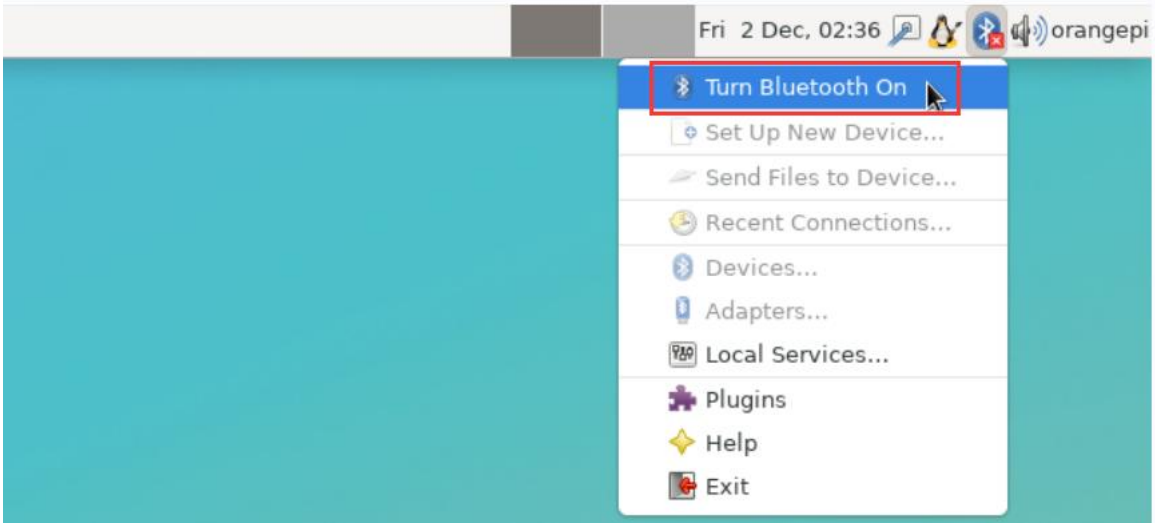
```
orangepi@orangepi:~$ sudo apt update && sudo apt install bluez
orangepi@orangepi:~$ hciconfig
hci0:   Type: Primary  Bus: USB
        BD Address: 00:13:EF:F4:58:AE  ACL MTU: 820:8  SCO MTU: 255:16
        DOWN
```

```
        RX bytes:1252 acl:0 sco:0 events:125 errors:0
        TX bytes:23307 acl:0 sco:0 commands:125 errors:0
```

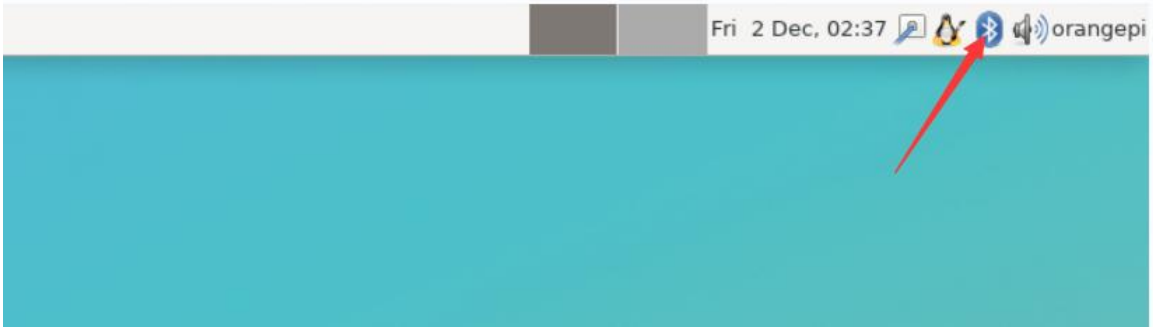
6) 在桌面中也可以看到出现了蓝牙图标，此时蓝牙还未打开，所以会显示一个红色的 x



7) 点击 **Turn Bluetooth On** 可以打开蓝牙



8) 打开蓝牙后的显示如下所示



9) 蓝牙的测试方法请参看蓝牙使用方法一节，这里不再赘述

3.11.3.2. RTL8811 测试

1) 首先将 RTL8811 无线网卡模块插入开发板的 USB 接口中

2) 然后 Linux 系统会自动加载 RTL8811 WIFI 相关的内核模块，通过 lsmod 命令可以看到下面内核模块已自动加载

```
orangepi@orangepi:~$ lsmod
Module                  Size  Used by
8821cu                1839104  0
```

3) 通过 dmesg 命令可以看到 RTL8811 模块的加载信息

```
orangepi@orangepi:~$ dmesg
[   118.618194] usb 2-1: new high-speed USB device number 2 using ehci-platform
[   118.767152] usb 2-1: New USB device found, idVendor=0bda, idProduct=c811, bcdDevice= 2.00
[   118.767181] usb 2-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[   118.767199] usb 2-1: Product: 802.11ac NIC
[   118.767219] usb 2-1: Manufacturer: Realtek
[   118.767235] usb 2-1: SerialNumber: 123456
[   119.500530] usbcore: registered new interface driver rtl8821cu
[   119.525498] rtl8821cu 2-1:1.0 wlx1cbfced9d260: renamed from wlan0
```

4) 然后通过 **sudo ifconfig** 命令可以看到 WIFI 的设备节点，WIFI 的连接和测试方法请参看 **WIFI 连接测试**一节，这里不再赘述

```
orangepi@orangepi:~$ sudo ifconfig wlx1cbfced9d260
wlx1cbfced9d260: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 1c:bf:ce:d9:d2:60  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

3.11.4. USB 摄像头测试

1) 首先需要准备一个下图所示的或者类似的支持 UVC 协议的 USB 摄像头，然后将 USB 摄像头插入到 Orange Pi 开发板的 USB 接口中

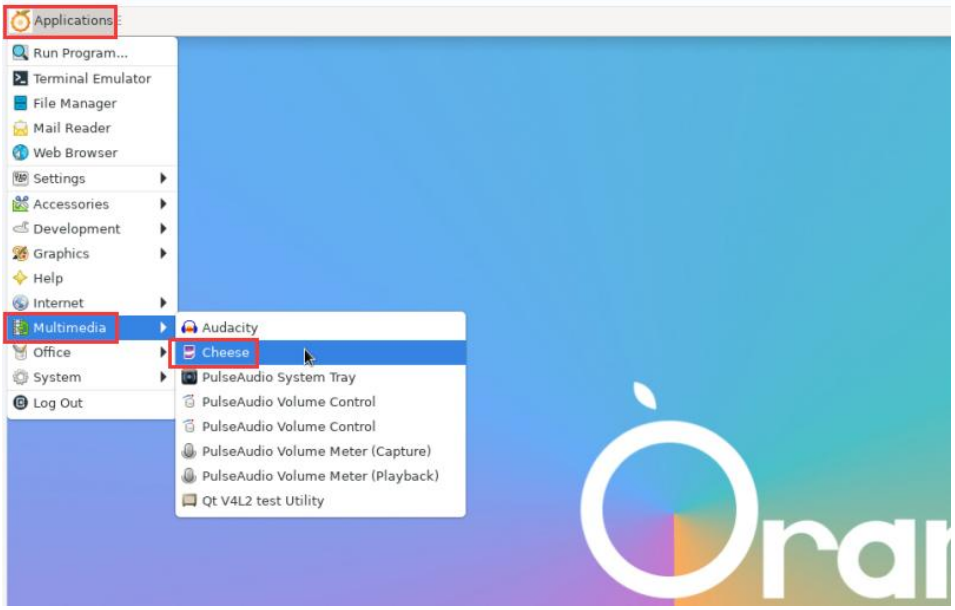
2) 通过 v4l2-ctl 命令可以看到 USB 摄像头的设备节点信息为/dev/video0

```
orangepi@orangepi:~$ v4l2-ctl --list-devices
Q8 HD Webcam: Q8 HD Webcam (usb-fc880000.usb-1):
        /dev/video0
        /dev/video1
        /dev/media0
```

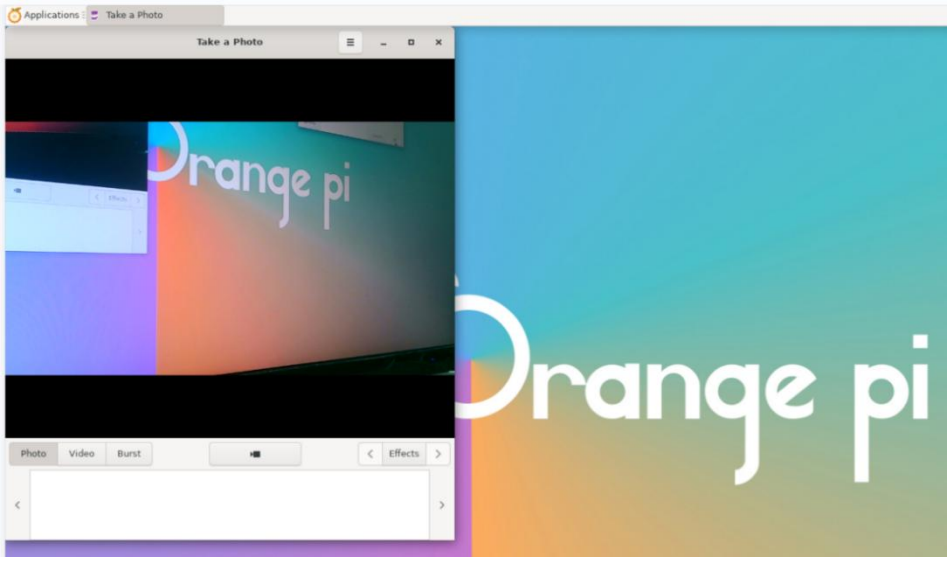
注意 v4l2 中的 l 是小写字母 l，不是数字 1。

另外 video 的序号不一定都是 video0，请以实际看到的为准。

3) 在桌面系统中可以使用 Cheese 直接打开 USB 摄像头，Cheese 打开方法如下图所示：



Cheese 打开 USB 摄像头后的界面如下图所示：



4) 使用 fswebcam 测试 USB 摄像头的方法

a. 安装 fswebcam

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt-get install -y fswebcam
```

b. 安装完 fswebcam 后可以使用下面的命令来拍照

a) -d 选项用于指定 USB 摄像头的设备节点

b) --no-banner 用于去除照片的水印

c) -r 选项用于指定照片的分辨率

d) -S 选项用于设置跳过前面的帧数

e) ./image.jpg 用于设置生成的照片的名字和路径

```
orangepi@orangepi:~$ sudo fswebcam -d /dev/video0  \
--no-banner -r 1280x720 -S 5 ./image.jpg
```

c. 在服务器版的 Linux 系统中，拍完照后可以使用 scp 命令将拍好的图片传到 Ubuntu PC 上镜像观看

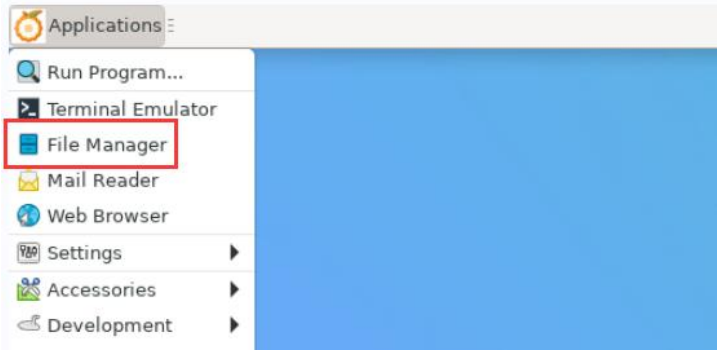
```
orangepi@orangepi:~$ scp image.jpg test@192.168.1.55:/home/test（根据实际情况修改 IP 地址和路径）
```

d. 在桌面版的 Linux 系统中，可以通过 HDMI 显示器直接查看拍摄的图片

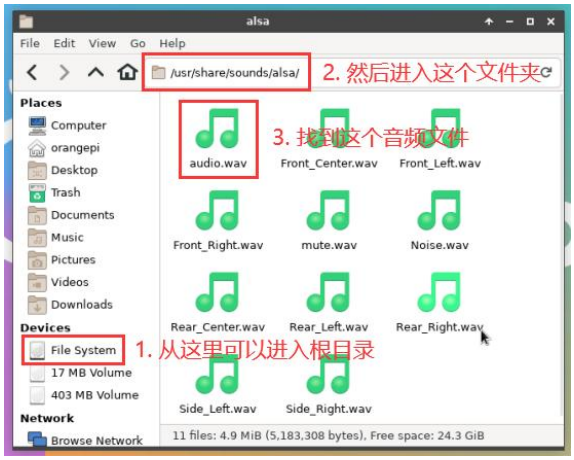
3.12. 音频测试

3.12.1. 在桌面系统中测试音频方法

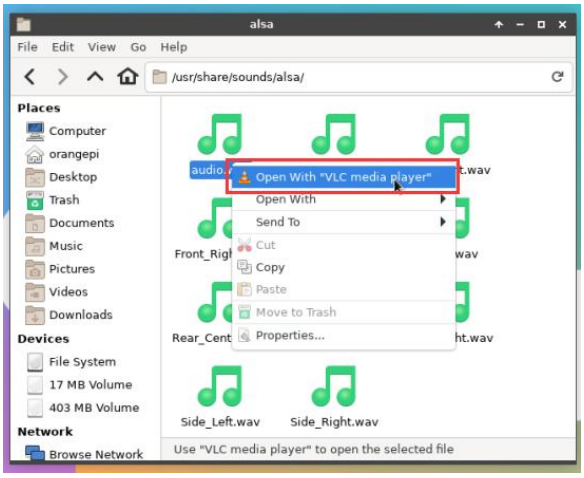
1) 首先打开文件管理器



2) 然后找到下面这个文件（如果系统中没有这个音频文件，可以自己上传一个音频文件到系统中）

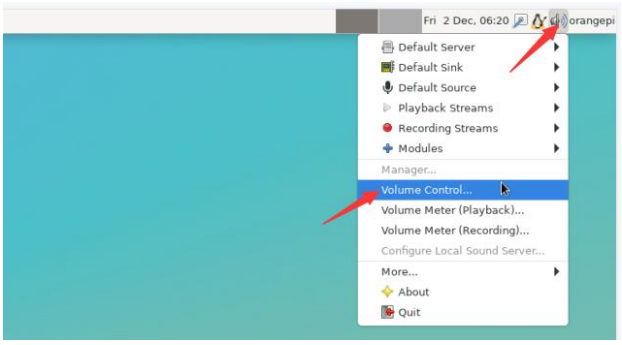


3) 然后选中 audio.wav 文件，右键选择使用 vlc 打开就可以开始播放

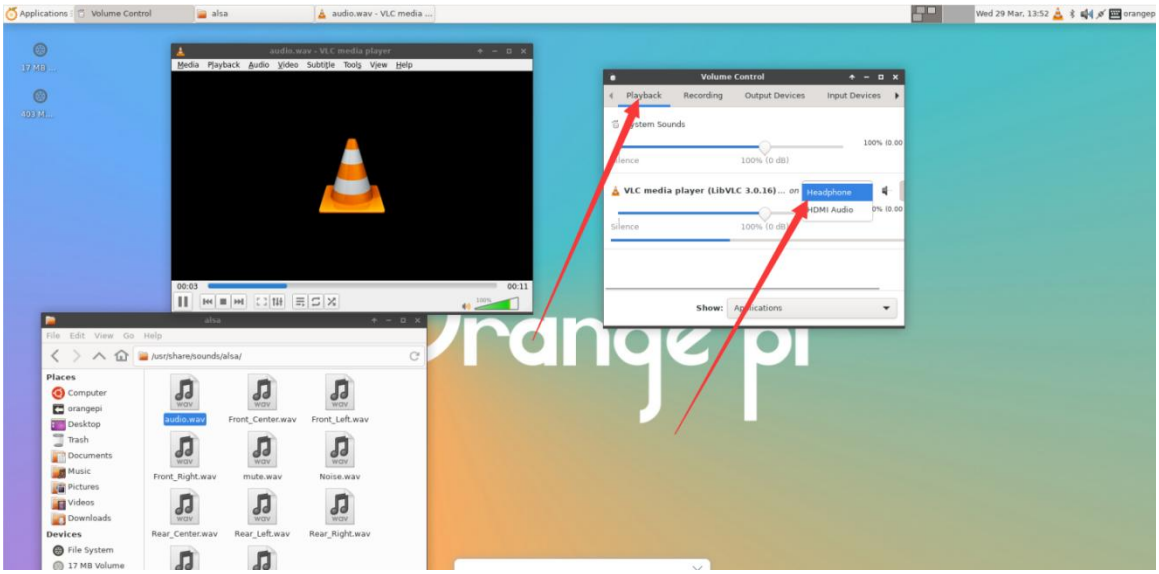


4) 切换 HDMI 播放和耳机播放等不同音频设备的方法

a. 首先打开音量控制界面



b. 播放音频的时候，在 **Playback** 中会显示播放软件可以使用的音频设备选项，如下图所示，在这里可以设置需要播放到哪个音频设备

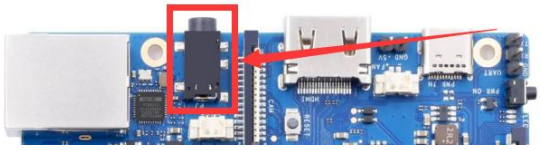


3.12.2. 使用命令播放音频的方法

3.12.2.1. 耳机接口播放音频测试

注意，linux6.6 暂不支持。

1) 首先将耳机插入开发板的耳机孔中



2) 然后可以通过 **aplay -l** 命令可以查看下 Linux 系统支持的声卡设备
   a. linux5.10 系统的输出如下所示，card 0 为 rk809 的声卡设备，也就是耳机的声卡设备

```
orangepi@orangepi:~$ aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: rockchiprk809 [rockchip-rk809], device 0: dailink-multicodecs rk817-hifi-0 [dailink-multicodecs rk817-hifi-0]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
card 1: rockchiphdmi [rockchip,hdmi], device 0: fe400000.i2s-i2s-hifi i2s-hifi-0 [fe400000.i2s-i2s-hifi i2s-hifi-0]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
Subdevice #0: subdevice #0
```

3) 然后使用 **aplay** 命令播放下系统自带的音频文件，如果耳机能听到声音说明硬件能正常使用
   a. linux5.10 系统的播放命令：

```
orangepi@orangepi:~$ aplay -D hw:0,0 /usr/share/sounds/alsa/audio.wav
Playing WAVE 'audio.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
```

3.12.2.2. HDMI 音频播放测试

1) 首先使用 HDMI 转 HDMI 线将 Orange Pi 开发板连接到电视机上（其他的 HDMI 显示器需要确保可以播放音频）

2) 然后查看下 HDMI 的声卡序号，从下面的输出可以知道 HDMI 的声卡为 **card 1**

a. linux5.10 系统的输出如下所示，card 1 为 HDMI 的声卡设备

```
orangepi@orangepi:~$ aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: rockchiprk809 [rockchip-rk809], device 0: dailink-multicodecs rk817-hifi-0 [dailink-multicodecs rk817-hifi-0]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
card 1: rockchiphdmi [rockchip,hdmi], device 0: fe400000.i2s-i2s-hifi i2s-hifi-0 [fe400000.i2s-i2s-hifi i2s-hifi-0]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
  Subdevice #0: subdevice #0
```

b. linux6.6 系统的输出如下所示，card 0 为 HDMI 的声卡设备

```
orangepi@orangepi:~$ aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: HDMI [HDMI], device 0: fe400000.i2s-i2s-hifi i2s-hifi-0 [fe400000.i2s-i2s-hifi i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 2: RK809 [Analog RK809], device 0: fe410000.i2s-rk817-hifi rk817-hifi-0 [fe410000.i2s-rk817-hifi rk817-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

3) 然后使用 **aplay** 命令播放下系统自带的音频文件，如果 HDMI 显示器或者电视能听到声音说明硬件能正常使用

a. linux5.10 系统的播放命令：

```
orangepi@orangepi:~$ aplay -D hw:1,0 /usr/share/sounds/alsa/audio.wav
```

b. linux6.6 系统的播放命令：

```
orangepi@orangepi:~$ aplay -D hw:0,0 /usr/share/sounds/alsa/audio.wav
```

3.12.3. 使用命令测试录音的方法

注意，linux6.6 暂不支持。

1) Orange Pi 3B 开发板没有板载 MIC，只能通过带 MIC 功能的耳机来录制音频。将带 MIC 功能的耳机插入开发板后，运行下面的命令会通过耳机录制一段音频

a. linux5.10 系统的命令：

```
orangepi@orangepi:~$ amixer -c 0 cset name='Capture MIC Path' 'Main Mic'
orangepi@orangepi:~$ arecord -D hw:0,0 -d 5 -f cd -t wav /tmp/test.wav
```

3.13. 温度传感器

1) 查看系统温度传感器的命令为：

```
orangepi@orangepi:~$ sensors
soc_thermal-virtual-0
Adapter: Virtual device
temp1:          +41.9°C  (crit = +115.0°C)

gpu_thermal-virtual-0
Adapter: Virtual device
temp1:          +43.8°C
```

2) 查看 nvme ssd 固态硬盘当前温度的命令为：

```
orangepi@orangepi:~$ sudo smartctl -a /dev/nvme0 | grep "Temperature:"
Temperature:                        40 Celsius
```

3.14. 40 Pin 接口引脚说明

1) Orange Pi 3B 开发板 40 pin 接口引脚的顺序请参考下图

2) Orange Pi 3B 开发板 40 pin 接口引脚的功能如下表所示

a. 下面是 40pin 完整的引脚图

复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

b. 下面的表格是上面完整表格左半边部分的图，能看得清楚点

复用功能	复用功能	GPIO	GPIO序号	引脚序号
		3.3V		1
	I2C2_SDA_M1	GPIO4_B4	140	3
	I2C2_SCL_M1	GPIO4_B5	141	5
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7
		GND		9
		GPIO3_C6	118	11
		GPIO4_A0	128	13
	UART7_TX_M2	GPIO4_A2	130	15
		3.3V		17
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19
	SPI3_MISO_M0	GPIO4_B0	136	21
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23
		GND		25
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27
	UART9_RX_M2	GPIO4_A5	133	29
		GPIO3_D4	124	31
		GPIO3_D7	127	33
		GPIO3_D0	120	35
		GPIO3_D3	123	37
		GND		39

c. 下面的表格是上面完整表格右半边部分的图，能看得清楚点

引脚序号	GPIO序号	GPIO	复用功能	复用功能
2		5V		
4		5V		
6		GND		
8	25	GPIO0_D1	UART2_TX_M0	
10	24	GPIO0_D0	UART2_RX_M0	
12	119	GPIO3_C7		
14		GND		
16	131	GPIO4_A3	UART7_RX_M2	
18	129	GPIO4_A1		
20		GND		
22	132	GPIO4_A4	UART9_TX_M2	
24	134	GPIO4_A6	SPI3_CS0_M0	
26	126	GPIO3_D6		
28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
30		GND		
32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
34		GND		
36	125	GPIO3_D5		
38	122	GPIO3_D2		
40	121	GPIO3_D1		

3) 40pin 接口中总共有 **28** 个 GPIO 口，所有 GPIO 口的电压都是 **3.3v**

3.15. 安装 wiringOP 的方法

注意，Orange Pi 发布的 Linux 镜像中已经预装了 wiringOP，除非 wiringOP 的代码有更新，否则无需重新下载编译安装，直接使用即可。

编译好的 wiringOP 的 deb 包在 orangepi-build 中的存放路径为：
orangepi-build/external/cache/debs/arm64/wiringpi_x.xx.deb

进入系统后可以运行下 gpio readall 命令，如果能看到下面的输出，说明 wiringOP 已经预装并且能正常使用。

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |    PWM15 |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
 |  118 |   5 | GPIO3_C6 |     IN | 0 | 11 || 12 | 0 | IN     | GPIO3_C7 | 6   | 119  |
 |  128 |   7 | GPIO4_A0 |     IN | 0 | 13 || 14 |   |        | GND      |     |      |
 |  130 |   8 |    TXD.7 |     IN | 0 | 15 || 16 | 0 | IN     | RXD.7    | 9   | 131  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | IN     | GPIO4_A1 | 10  | 129  |
 |  138 |  11 | SPI3_TXD |     IN | 0 | 19 || 20 |   |        | GND      |     |      |
 |  136 |  12 | SPI3_RXD |     IN | 0 | 21 || 22 | 0 | IN     | TXD.9    | 13  | 132  |
 |  139 |  14 | SPI3_CLK |     IN | 0 | 23 || 24 | 0 | IN     | SPI3_CS1 | 15  | 134  |
 |      |     |      GND |        |   | 25 || 26 | 0 | IN     | GPIO3_D6 | 16  | 126  |
 |   32 |  17 |    SDA.3 |     IN | 1 | 27 || 28 | 1 | IN     | SCL.3    | 18  | 33   |
 |  133 |  19 |    RXD.9 |     IN | 0 | 29 || 30 |   |        | GND      |     |      |
 |  124 |  20 | GPIO3_D4 |     IN | 0 | 31 || 32 | 0 | IN     | PWM11    | 21  | 144  |
 |  127 |  22 | GPIO3_D7 |     IN | 0 | 33 || 34 |   |        | GND      |     |      |
 |  120 |  23 | GPIO3_D0 |     IN | 0 | 35 || 36 | 0 | IN     | GPIO3_D5 | 24  | 125  |
 |  123 |  25 | GPIO3_D3 |     IN | 0 | 37 || 38 | 0 | IN     | GPIO3_D2 | 26  | 122  |
 |      |     |      GND |        |   | 39 || 40 | 0 | IN     | GPIO3_D1 | 27  | 121  |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
```

1) 下载 wiringOP 的代码

```
orangepi@orangepi:~$ sudo apt update
orangepi@orangepi:~$ sudo apt install -y git
orangepi@orangepi:~$ git clone https://github.com/orangepi-xunlong/wiringOP.git -b next
```

注意，Orange Pi 3B 需要下载 wiringOP next 分支的代码，请别漏了-b next 这个参数。

如果从 GitHub 下载代码有问题，可以直接使用 Linux 镜像中自带的 wiringOP 源码，存放位置为：/usr/src/wiringOP。

2) 编译安装 wiringOP

```
orangepi@orangepi:~$ cd wiringOP
orangepi@orangepi:~/wiringOP$ sudo ./build clean
orangepi@orangepi:~/wiringOP$ sudo ./build
```

3) 测试 gpio readall 命令的输出如下

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |    PWM15 |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
 |  118 |   5 | GPIO3_C6 |     IN | 0 | 11 || 12 | 0 | IN     | GPIO3_C7 | 6   | 119  |
 |  128 |   7 | GPIO4_A0 |     IN | 0 | 13 || 14 |   |        | GND      |     |      |
 |  130 |   8 |    TXD.7 |     IN | 0 | 15 || 16 | 0 | IN     | RXD.7    | 9   | 131  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | IN     | GPIO4_A1 | 10  | 129  |
 |  138 |  11 | SPI3_TXD |     IN | 0 | 19 || 20 |   |        | GND      |     |      |
 |  136 |  12 | SPI3_RXD |     IN | 0 | 21 || 22 | 0 | IN     | TXD.9    | 13  | 132  |
 |  139 |  14 | SPI3_CLK |     IN | 0 | 23 || 24 | 0 | IN     | SPI3_CS1 | 15  | 134  |
 |      |     |      GND |        |   | 25 || 26 | 0 | IN     | GPIO3_D6 | 16  | 126  |
 |   32 |  17 |    SDA.3 |     IN | 1 | 27 || 28 | 1 | IN     | SCL.3    | 18  | 33   |
 |  133 |  19 |    RXD.9 |     IN | 0 | 29 || 30 |   |        | GND      |     |      |
 |  124 |  20 | GPIO3_D4 |     IN | 0 | 31 || 32 | 0 | IN     | PWM11    | 21  | 144  |
 |  127 |  22 | GPIO3_D7 |     IN | 0 | 33 || 34 |   |        | GND      |     |      |
 |  120 |  23 | GPIO3_D0 |     IN | 0 | 35 || 36 | 0 | IN     | GPIO3_D5 | 24  | 125  |
 |  123 |  25 | GPIO3_D3 |     IN | 0 | 37 || 38 | 0 | IN     | GPIO3_D2 | 26  | 122  |
 |      |     |      GND |        |   | 39 || 40 | 0 | IN     | GPIO3_D1 | 27  | 121  |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
```

3.16. 40pin 接口 GPIO、I2C、UART、SPI 和 PWM 测试

注意，如果需要设置 overlays 同时打开多个配置，请像下面这样使用空格隔开写在一行即可。

orangepi@orangepi:~$ **sudo vim /boot/orangepiEnv.txt**

overlays=spi3-m0-cs0-spidev i2c2-m1 i2c3-m0 uart7-m2 uart9-m2 pwm11-m1

3.16.1. 40pin GPIO 口测试

Orange Pi 发布的 Linux 系统中有预装一个 blink_all_gpio 程序，这个程序会设置 40pin 中的所有 28 个 GPIO 口不停的切换高低电平。

运行 blink_all_gpio 程序后，当用万用表去测量 GPIO 口的电平时，会发现 GPIO 引脚会在 0 和 3.3v 之间不停的切换。使用这个程序我们可以来测试 GPIO 口是否能正常工作。

运行 blink_all_gpio 程序的方式如下所示：

orangepi@orangepi3b:~$ **sudo blink_all_gpio** #记得加 sudo 权限

[sudo] password for orangepi: #在这里需要输入密码

1) 开发板 40pin 中总共有 **28** 个 GPIO 口可以使用，下面以 7 号引脚——对应 GPIO 为 GPIO4_C3 ——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的高低电平

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |   PWM15  |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
```

2) 首先设置 GPIO 口为输出模式，其中第三个参数需要输入引脚对应的 wPi 的序号

```
root@orangepi:~/wiringOP# gpio mode 2 out
```

3) 然后设置 GPIO 口输出低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 0v，说明设置低电平成功

```
root@orangepi:~/wiringOP# gpio write 2 0
```

使用 gpio readall 可以看到 7 号引脚的值(V)变为了 0

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |   PWM15  |    OUT | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
```

4) 然后设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功

```
root@orangepi:~/wiringOP# gpio write 2 1
```

使用 gpio readall 可以看到 7 号引脚的值(V)变为了 1

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  |   Name   | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |   PWM15  |    OUT | 1 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
```

5) 其他引脚的设置方法类似，只需修改 wPi 的序号为引脚对应的序号即可

3.16.2. 40pin GPIO 口上下拉电阻的设置方法

注意，Orange Pi 3B 下面 4 个 GPIO 引脚因为外部有 3.3V 上拉，所以设置下拉是无效的，其它的引脚可以正常设置上下拉电阻功能

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |    PWM15 |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
 |  118 |   5 | GPIO3_C6 |     IN | 0 | 11 || 12 | 0 | IN     | GPIO3_C7 | 6   | 119  |
 |  128 |   7 | GPIO4_A0 |     IN | 0 | 13 || 14 |   |        | GND      |     |      |
 |  130 |   8 |    TXD.7 |     IN | 0 | 15 || 16 | 0 | IN     | RXD.7    | 9   | 131  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | IN     | GPIO4_A1 | 10  | 129  |
 |  138 |  11 | SPI3_TXD |     IN | 0 | 19 || 20 |   |        | GND      |     |      |
 |  136 |  12 | SPI3_RXD |     IN | 0 | 21 || 22 | 0 | IN     | TXD.9    | 13  | 132  |
 |  139 |  14 | SPI3_CLK |     IN | 0 | 23 || 24 | 0 | IN     | SPI3_CS1 | 15  | 134  |
 |      |     |      GND |        |   | 25 || 26 | 0 | IN     | GPIO3_D6 | 16  | 126  |
 |   32 |  17 |    SDA.3 |     IN | 1 | 27 || 28 | 1 | IN     | SCL.3    | 18  | 33   |
 |  133 |  19 |    RXD.9 |     IN | 0 | 29 || 30 |   |        | GND      |     |      |
 |  124 |  20 | GPIO3_D4 |     IN | 0 | 31 || 32 | 0 | IN     | PWM11    | 21  | 144  |
 |  127 |  22 | GPIO3_D7 |     IN | 0 | 33 || 34 |   |        | GND      |     |      |
 |  120 |  23 | GPIO3_D0 |     IN | 0 | 35 || 36 | 0 | IN     | GPIO3_D5 | 24  | 125  |
 |  123 |  25 | GPIO3_D3 |     IN | 0 | 37 || 38 | 0 | IN     | GPIO3_D2 | 26  | 122  |
 |      |     |      GND |        |   | 39 || 40 | 0 | IN     | GPIO3_D1 | 27  | 121  |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
```

1) 下面以 11 号引脚——对应 GPIO 为 GPIO3_C6 ——对应 wPi 序号为 5——为例演示如何设置 GPIO 口的上下拉电阻

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |    PWM15 |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
 |  118 |   5 | GPIO3_C6 |     IN | 0 | 11 || 12 | 0 | IN     | GPIO3_C7 | 6   | 119  |
 |  128 |   7 | GPIO4_A0 |     IN | 0 | 13 || 14 |   |        | GND      |     |      |
 |  130 |   8 |    TXD.7 |     IN | 0 | 15 || 16 | 0 | IN     | RXD.7    | 9   | 131  |
```

2) 首先需要设置 GPIO 口为输入模式，其中第三个参数需要输入引脚对应的 wPi 的序号

```
root@orangepi:~/wiringOP# gpio mode 5 in
```

3) 设置为输入模式后，执行下面的命令可以设置 GPIO 口为上拉模式

```
root@orangepi:~/wiringOP# gpio mode 5 up
```

4) 然后输入下面的命令读取 GPIO 口的电平，如果电平为 1，说明上拉模式设置成功

```
root@orangepi:~/wiringOP# gpio read 5
1
```

5) 然后执行下面的命令可以设置 GPIO 口为下拉模式

```
root@orangepi:~/wiringOP# gpio mode 5 down
```

6) 然后输入下面的命令读取 GPIO 口的电平，如果电平为 0，说明下拉模式设置成功

```
root@orangepi:~/wiringOP# gpio read 5
0
```

3.16.3. 40pin SPI 测试

1) 由下表可知，Orange Pi 3B 可用的 spi 为 spi3

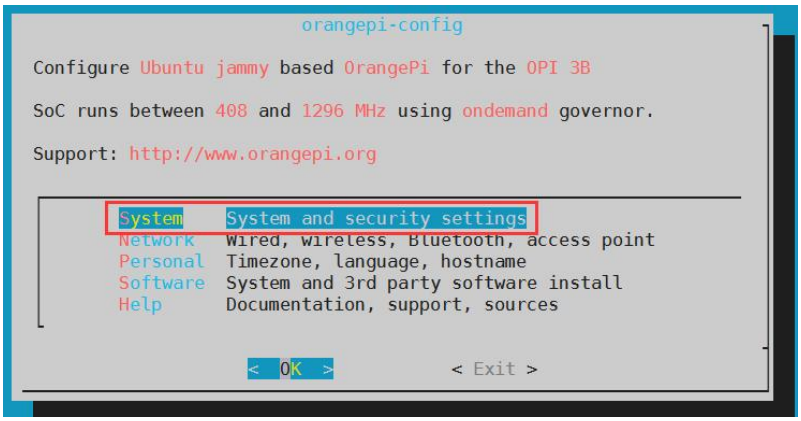
复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

2) 在 linux 系统中，40 pin 中的 SPI 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

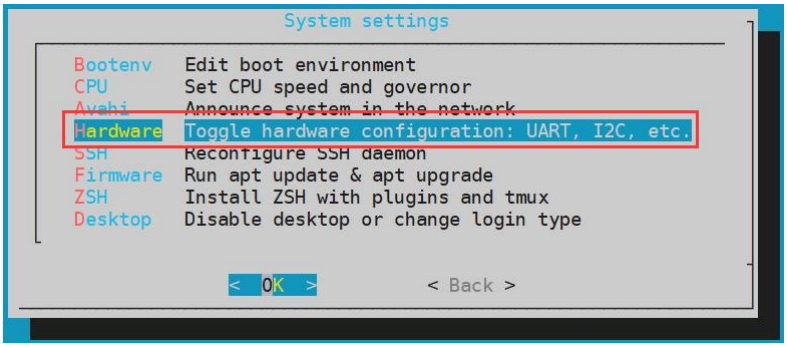
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

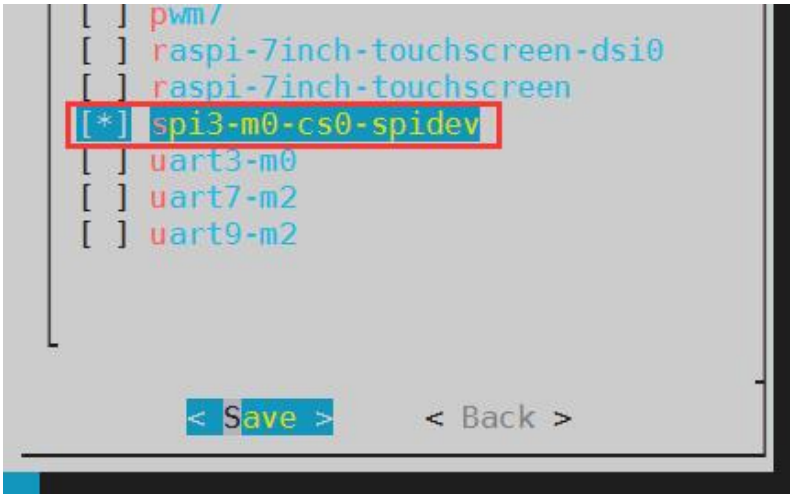
b. 然后选择 **System**



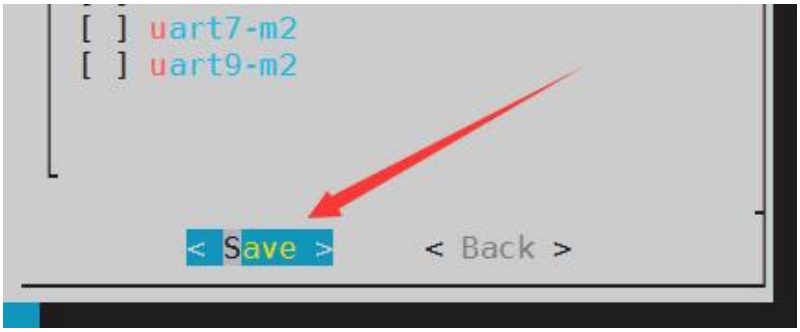
c. 然后选择 **Hardware**



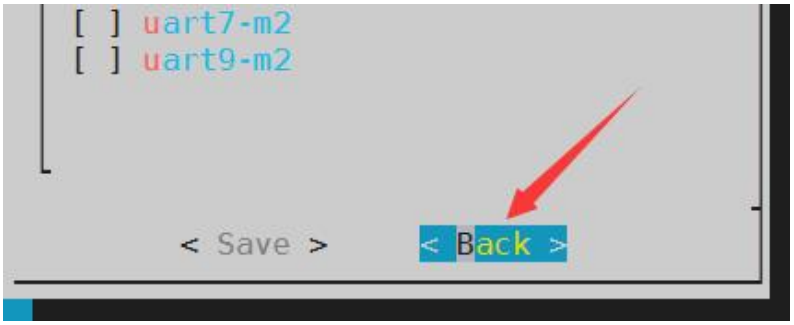
d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中想要打开的 SPI 的配置



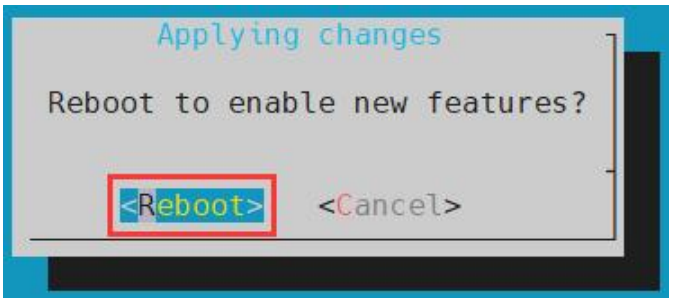
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



3) 重启后进入系统先查看下 Linux 系统中是否存在 spidev3.0 的设备节点，如果存在，说明 SPI3 已经设置好了，可以直接使用

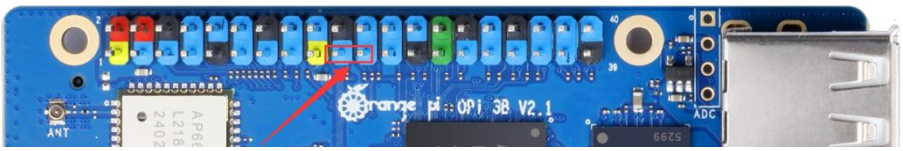
```
orangepi@orangepi:~$ ls /dev/spidev3.0
/dev/spidev3.0
```

4) 先不短接 SPI3 的 mosi 和 miso 两个引脚，运行 spidev_test 的输出结果如下所示，可以看到 TX 和 RX 的数据不一致

```
orangepi@orangepi:~$ sudo spidev_test -v -D /dev/spidev3.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
```

```
FF FF FF FF FF F0 0D   | ......@....▒.................▒.
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF   | ..............................
```

5) 然后短接 SPI3 的 mosi（40pin 接口中的第 19 号引脚）和 miso（40pin 接口中的第 21 号引脚）两个引脚再运行 spidev_test 的输出如下，可以看到发送和接收的数据一样




```
orangepi@orangepi:~$ sudo spidev_test -v -D /dev/spidev3.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D   | ......@....▒.................▒.
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D   | ......@....▒.................▒.
```

3. 16. 4. 40pin I2C 测试

1) 由下表可知，Orange Pi 3B 可用的 i2c 为 i2c2、i2c3 和 i2c4 共三组 i2c 总线

复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

从上表中可以看到，i2c4_m0 和 spi3_m0 是引脚复用的，两者不能同时打开，i2c3_m0 和 uart3_m0 也是引脚复用的，两者不能同时打开

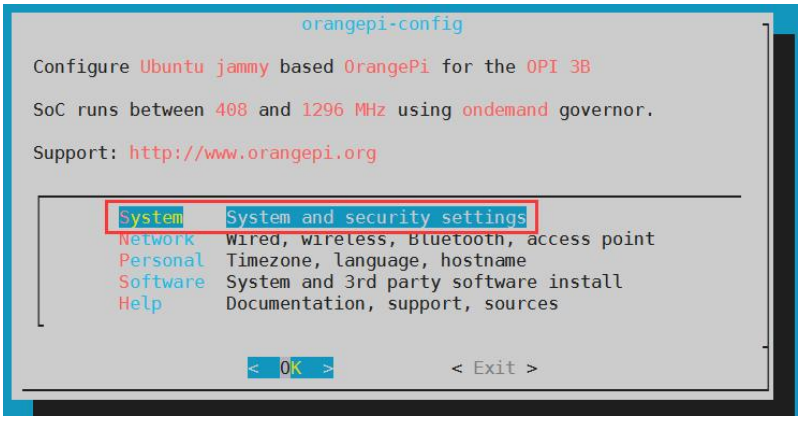
2) 在 linux 系统中，40 pin 中的 I2C 总线默认都是关闭的，需要手动打开才能使用。

详细步骤如下所示：

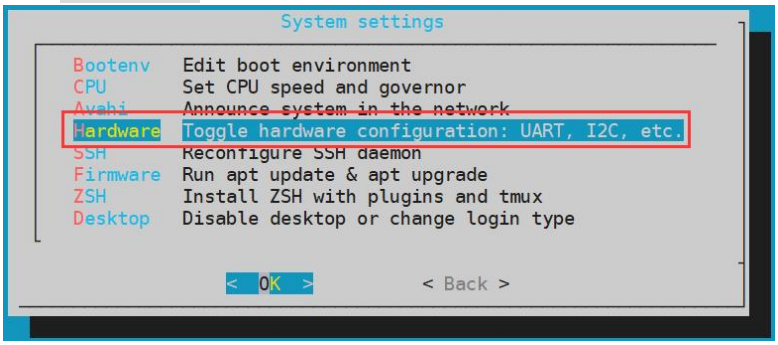
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**

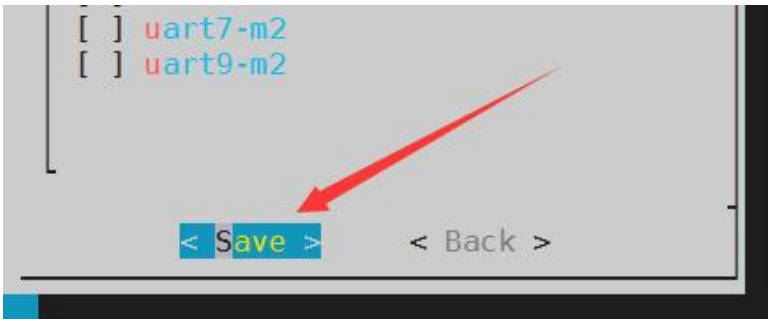


c. 然后选择 **Hardware**

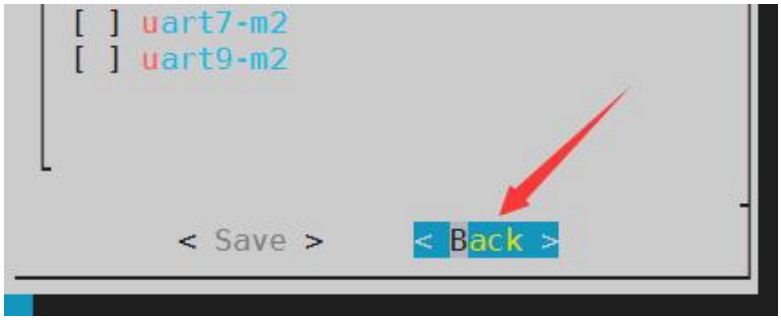


d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中想要打开的 I2C 的配置

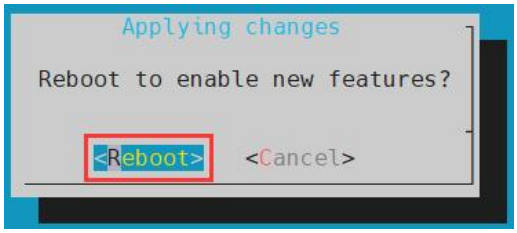
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



3) 启动 Linux 系统后，先确认下/dev 下存在 i2c 的设备节点

```
orangepi@orangepi:~# ls /dev/i2c-*
/dev/i2c-0  /dev/i2c-2  /dev/i2c-3  /dev/i2c-4  /dev/i2c-6
```

4) 然后在 40pin 接头的 i2c 引脚上接一个 i2c 设备

	i2c2-m1	i2c3-m0	i2c4-m0
sda 引脚	对应 3 号引脚	对应 27 号引脚	对应 19 号引脚
sck 引脚	对应 5 号引脚	对应 28 号引脚	对应 23 号引脚
vcc 引脚	对应 1 号引脚	对应 1 号引脚	对应 1 号引脚
gnd 引脚	对应 6 号引脚	对应 6 号引脚	对应 6 号引脚

5) 然后使用 **i2cdetect -y** 命令如果能检测到连接的 i2c 设备的地址，就说明 i2c 能正常使用

```
orangepi@orangepi:~$ sudo i2cdetect -y 2      #i2c2 的命令
orangepi@orangepi:~$ sudo i2cdetect -y 3      #i2c3 的命令
orangepi@orangepi:~$ sudo i2cdetect -y 4      #i2c4 的命令
```

```
root@orangepi3b:~# i2cdetect -y 2
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- 38 -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

3.16.5. 40pin 的 UART 测试

1) 由下表可知，Orange Pi 3B 可用的 uart 为 uart2、uart3、uart7 和 uart9 共四组 uart 总线，其中 uart2 默认是系统的调试串口

复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

从上表中可以看到，i2c3_m0 和 uart3_m0 是引脚复用的，两者不能同时打开

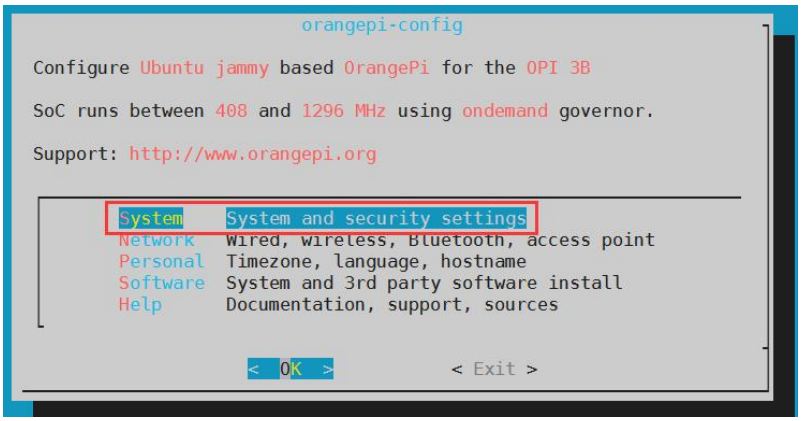
请注意，uart2_m0 默认是作为系统的调试串口，如果打开了 uart2 的配置，调试串口的功能会无法使用

2) 在 linux 系统中，40 pin 中的 UART 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

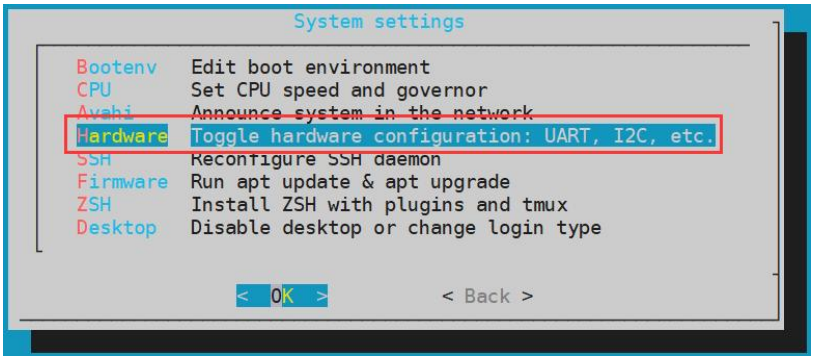
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**

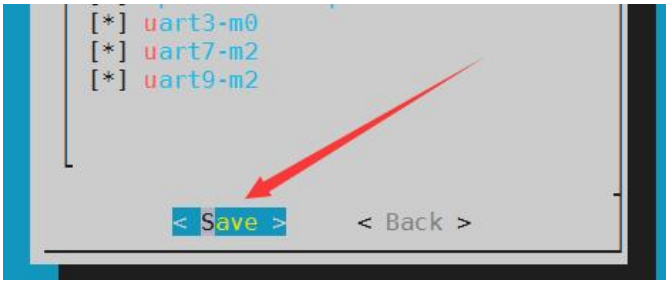


c. 然后选择 **Hardware**

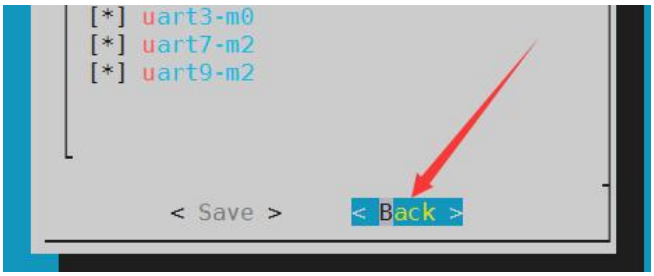


d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中想要打开的 UART 的配置

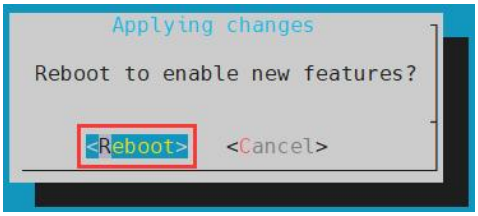
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



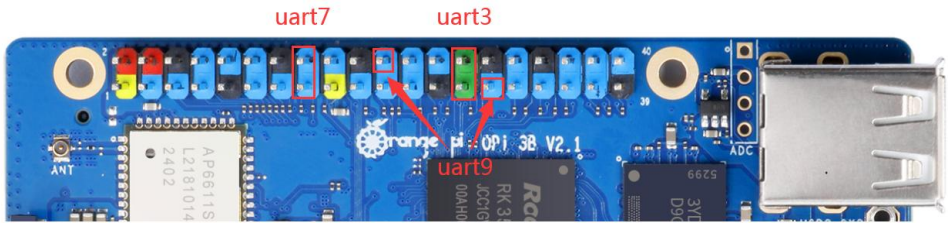
3) 进入 Linux 系统后，先确认下/dev 下是否存在对应 uart 的设备节点

```
orangepi@orangepi:~# ls /dev/ttyS*
/dev/ttyS1   /dev/ttyS3   /dev/ttyS7   /dev/ttyS9
```

注意，linux6.6 中/dev/ttyS0 是 uart9。

4) 然后开始测试 uart 接口，先使用杜邦线短接要测试的 uart 接口的 rx 和 tx

	uart3	uart7	uart9
tx 引脚	对应 28 号引脚	对应 16 号引脚	对应 29 号引脚
rx 引脚	对应 27 号引脚	对应 15 号引脚	对应 22 号引脚



5) 使用 **gpio serial** 命令测试串口的回环功能如下所示，如果能看到下面的打印，说明串口通信正常

a. 测试 UART3

```
orangepi@orangepi:~$ sudo gpio serial /dev/ttyS3
[sudo] password for orangepi:   #在这里输入密码
```

```
Out:    0:   ->    0
Out:    1:   ->    1
Out:    2:   ->    2
Out:    3:   ->    3
Out:    4:   ->    4
Out:    5:   ->    5^C
```

b. 测试 UART7

```
orangepi@orangepi:~$ sudo gpio serial /dev/ttyS7
[sudo] password for orangepi:   #在这里输入密码

Out:    0:   ->    0
Out:    1:   ->    1
Out:    2:   ->    2
Out:    3:   ->    3
Out:    4:   ->    4
Out:    5:   ->    5^C
```

c. 测试 UART9

```
orangepi@orangepi:~$ sudo gpio serial /dev/ttyS9
[sudo] password for orangepi:   #在这里输入密码

Out:    0:   ->    0
Out:    1:   ->    1
Out:    2:   ->    2
Out:    3:   ->    3
Out:    4:   ->    4
Out:    5:   ->    5^C
```

3.16.6. 使用/sys/class/pwm 测试 PWM 的方法

1) 由下表可知，Orange Pi 3B 可用的 pwm11 和 pwm15 共两路 pwm

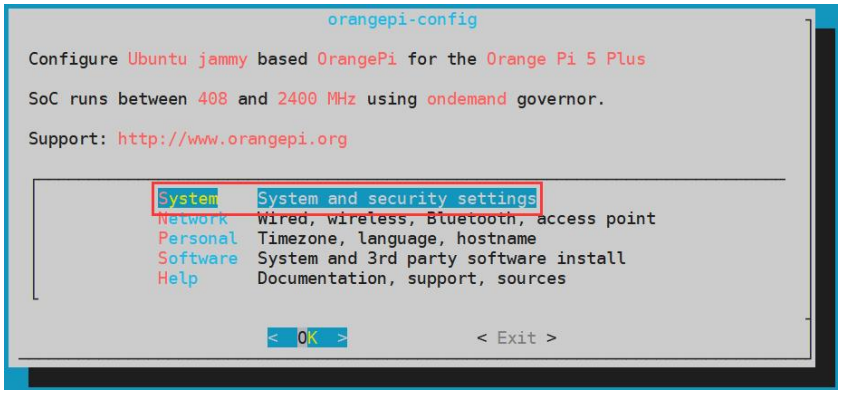
复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

2) 在 linux 系统中，40 pin 中的 PWM 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

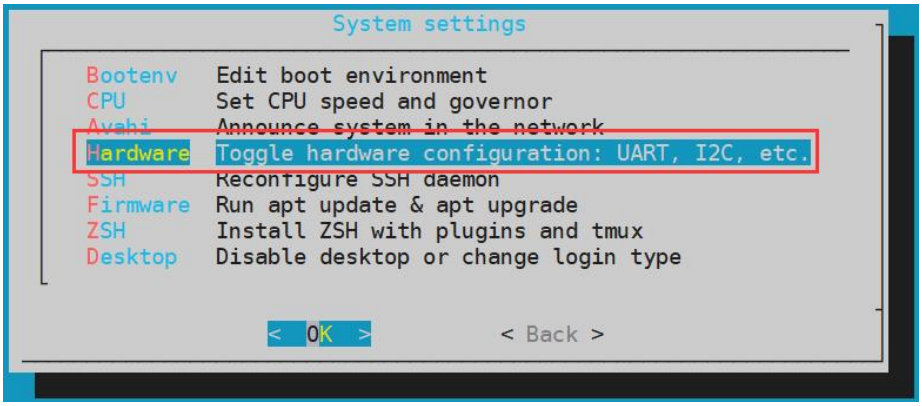
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**

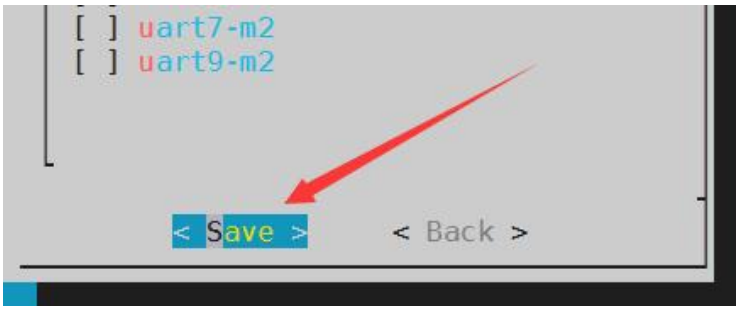


c. 然后选择 **Hardware**

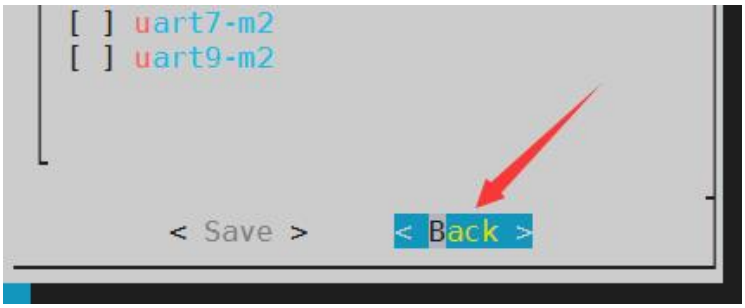


d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中想要打开的 PWM 的配置

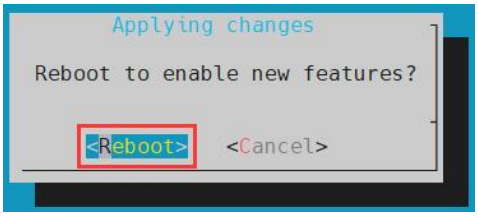
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



3) 当打开一个 pwm 后，在**/sys/class/pwm/**中就会多出一个 pwmchipX（X 为具体的的数字），比如打开 pwm11 后，查看**/sys/class/pwm/**下的 pwmchipX 会由一个变成了两个

```
orangepi@orangepi:~$ ls /sys/class/pwm/
pwmchip0    pwmchip1
```

4) 上面哪个 pwmchip 对应 pwm11 呢，我们先查看下 **ls /sys/class/pwm/ -l** 命令的输出，如下所示：

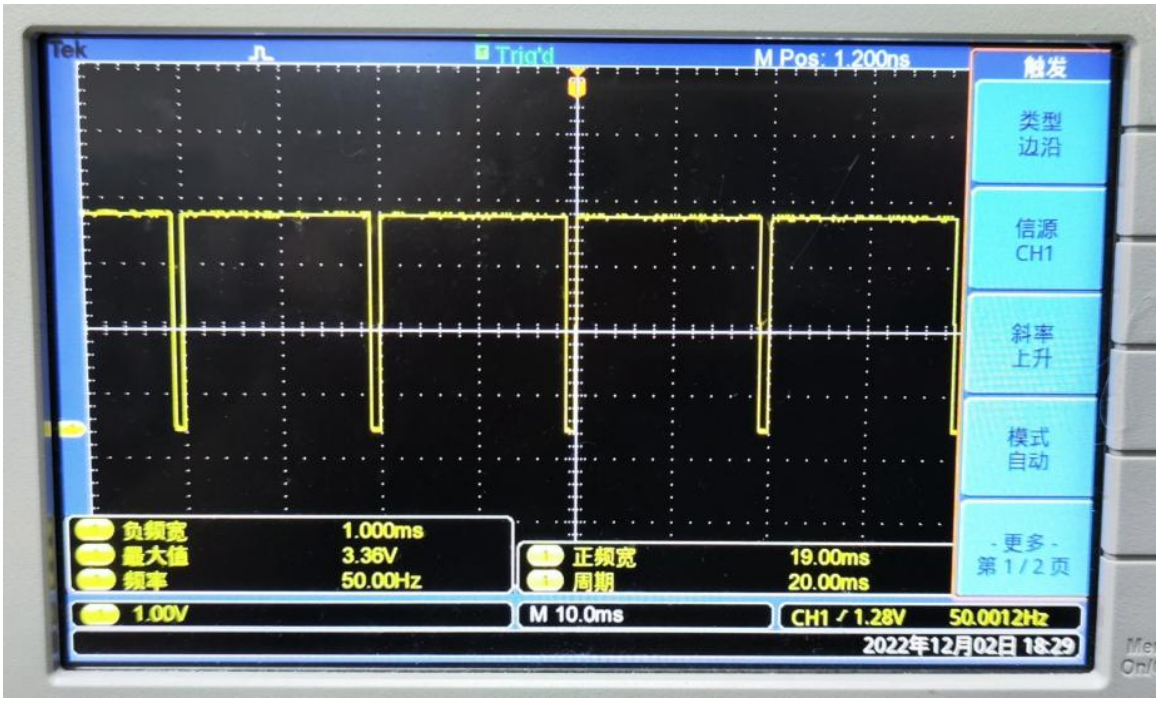
```
orangepi@orangepi3b:~$ ls /sys/class/pwm/ -l
total 0
lrwxrwxrwx 1 root root 0 Jan  1  1970 pwmchip0 -> ../../devices/platform/fe6e0030.pwm/pwm/pwmchip0
lrwxrwxrwx 1 root root 0 Jan  1  1970 pwmchip1 -> ../../devices/platform/fe6f0030.pwm/pwm/pwmchip1
```

5) 然后由下表可知，pwm11 寄存器的基地址为 fe6f0030，再看 **ls /sys/class/pwm/ -l** 命令的输出，可以看到 pwmchip1 中链接到了 fe6f0030.pwm，所以 pwm11 对应 pwmchip 为 pwmchip1

引脚序号	GPIO序号	GPIO	复用功能	复用功能
2		5V		
4		5V		
6		GND		
8	25	GPIO0_D1	UART2_TX_M0	
10	24	GPIO0_D0	UART2_RX_M0	
12	119	GPIO3_C7		
14		GND		
16	131	GPIO4_A3	UART7_RX_M2	
18	129	GPIO4_A1		
20		GND		
22	132	GPIO4_A4	UART9_TX_M2	
24	134	GPIO4_A6	SPI3_CS0_M0	
26	126	GPIO3_D6		
28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
30		GND		
32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
34		GND		
36	125	GPIO3_D5		
38	122	GPIO3_D2		
40	121	GPIO3_D1		

6) 然后使用下面的命令可以让 pwm11 输出一个 50Hz 的方波（请先切换到 root 用户，再执行下面的命令）

```
root@orangepi:~# echo 0 > /sys/class/pwm/pwmchip1/export
root@orangepi:~# echo 20000000 > /sys/class/pwm/pwmchip1/pwm0/period
root@orangepi:~# echo 1000000 > /sys/class/pwm/pwmchip1/pwm0/duty_cycle
root@orangepi:~# echo 1 > /sys/class/pwm/pwmchip1/pwm0/enable
```

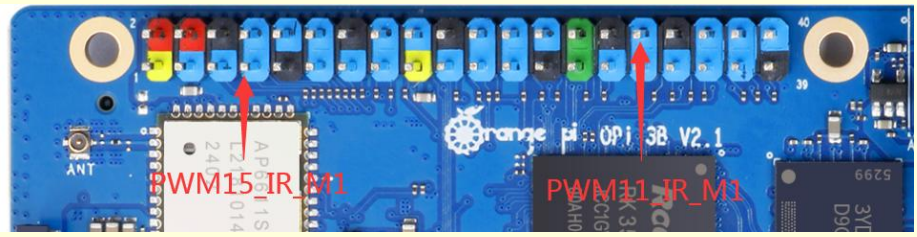


7) 上面演示的 pwm11 的测试方法，其他 pwm 测试方法都是类似的。

3.17. wiringOP 硬件 PWM 的使用方法

下载安装 wiringOP 的方法请参考安装 wiringOP 的方法一小节的说明。

开发板可以使用 PWM15_IR_M1 和 PWM11_IR_M1 共 2 路 PWM，它们所在引脚的位置如下图所示：



3.17.1. 使用 wiringOP 的 gpio 命令设置 PWM 的方法

3.17.1.1. 设置对应引脚为 PWM 模式

1) PWM 引脚与 wPi 序号对应关系如下表所示：

PWM 引脚	wPi 序号
PWM15_IR_M1	2
PWM11_IR_M1	21

2) 设置引脚为 PWM 模式的命令如下，以 PWM15_IR_M1 为例，其中第三个参数需要输入 PWM15_IR_M1 引脚对应的 wPi 的序号

```
orangepi@orangepi:~$ gpio mode 2 pwm
```

3) 引脚设置为 PWM 模式后，默认会输出一个频率为 200Hz，占空比为 50%的方波，此时，我们使用示波器测量对应的 PWM 引脚，就可以看到下面的波形

3.17.1.2. 调节 PWM 占空比的方法

1) PWM 占空比的计算公式如下所示，我们可以通过设置 CCR 和 ARR 的值来调节 PWM 占空比

PWM 占空比 = CCR / ARR

其中：
CCR 的取值范围是 0~65535，默认值是 500。
ARR 的取值范围是 0~65535，默认值是 1000。

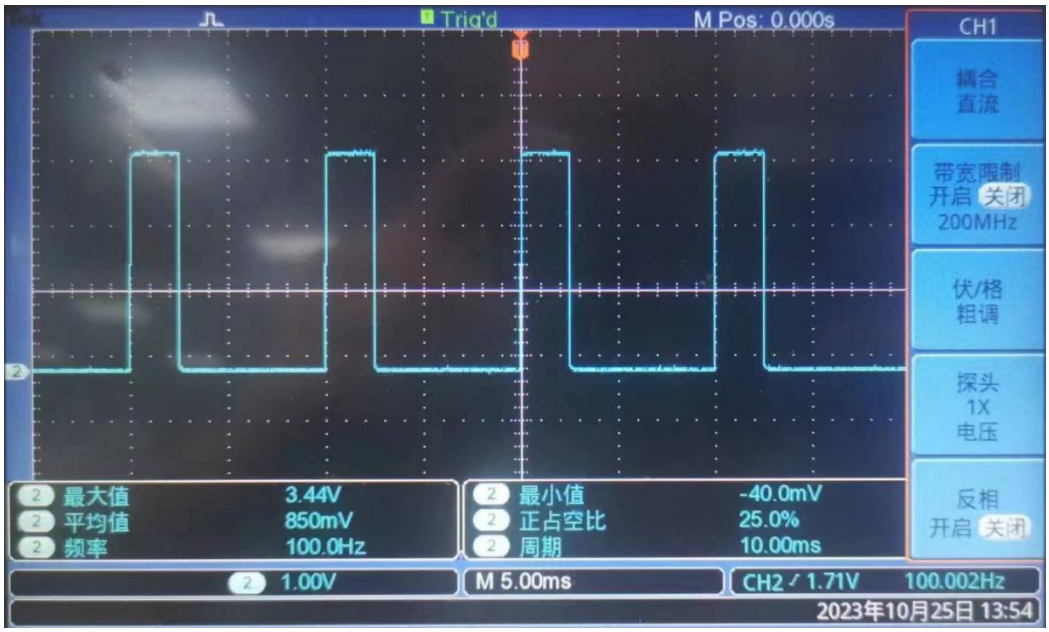
需要注意的是，我们 CCR 值需要小于 ARR 值，因为占空比不能大于 1。
当设置 CCR > ARR 时，会提示如下错误信息：
gpio: CCR should be less than or equal to ARR (XXX)
当设置 ARR < CCR 时，会提示如下错误信息：
gpio: ARR should be greater than or equal to CCR (XXX)

2) 我们可以使用下面的命令设置 PWM15_IR_M1 引脚的 ARR 为 2000

```
orangepi@orangepi:~$ gpio pwmr 2 2000
```

3) 运行上面的命令后，通过示波器可以观察到 PWM 占空比从默认的 50%

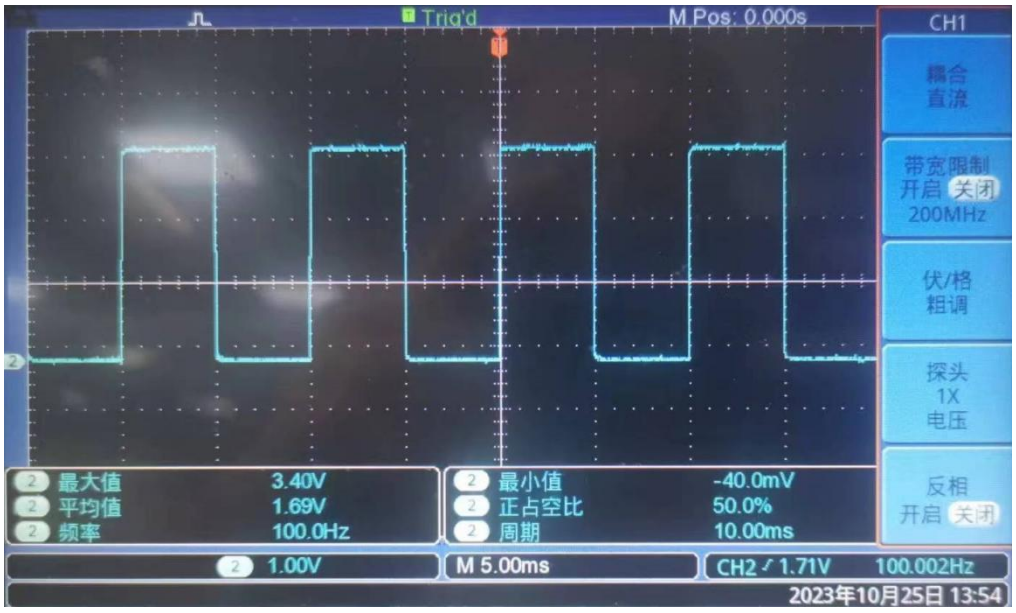
（500/1000）变为 25%（500/2000）



4) 我们可以使用下面的命令设置 PWM15_IR_M1 引脚的 CCR 为 1000

```
orangepi@orangepi:~$ gpio pwm 2 1000
```

5) 运行上面的命令后，通过示波器可以观察到 PWM 占空比从 25%（500/2000）变为 50%（1000/2000）



3.17.1.3. 调节 PWM 频率的方法

PWM 频率的计算公式如下所示：

PWM 频率 = 时钟来源频率 / (分频系数 * ARR)

其中：
时钟来源频率的默认值是 24000000Hz。
分频系数的取值范围是 2 ~ 512，默认值是 120。
ARR 的取值范围是 0 ~ 65535，默认值是 1000。
PWM 频率的默认值是 24000000 / (120 * 1000) = 200Hz。

需要注意的是，如果设置分频系数为奇数，实际的分频系数为设置值减一。

3.17.1.3.1. 通过设置分频系数来调节 PWM 频率的方法

1) 我们可以使用下面的命令设置 PWM15_IR_M1 引脚的分频系数为 4

```
orangepi@orangepi:~$ gpio pwmc 2 4
```

2) 根据上面的公式得到 PWM 频率的计算值为 6000Hz， 通过示波器可以观察到 PWM 频率测量值为 6010Hz，误差可以忽略

3.17.1.3.2. 直接设置 PWM 频率的方法

1) 我们可以使用 **gpio pwmTone** 命令来设置 PWM 引脚的频率，比如使用下面的命令可以设置 PWM15_IR_M1 引脚的 PWM 频率为 500Hz

```
orangepi@orangepi:~$ gpio pwmTone 2 500
```

在设置 PWM 频率时，需要保证：

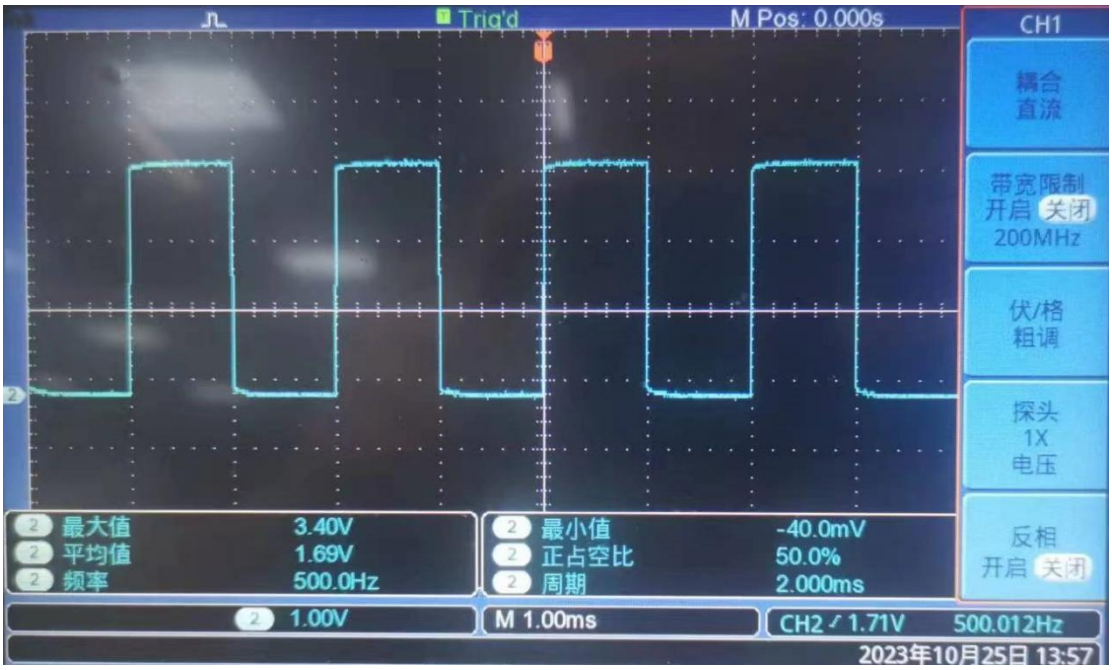
设置的频率值 < 24000000 / (分频系数 * 2)。

比如，默认的分频系数为 120，在没有修改分频系数的情况下，设置的频率值应小于 100000Hz。

如果设置值过大，会出现如下报错：

gpio: The PWM frequency you set is too high to be possible

2) 然后通过示波器可以观察到 PWM 频率变为 500Hz 了



3.17.2. PWM 测试程序的使用方法

1) 在 wiringOP 的 example 目录下，有一个名为 pwm.c 的程序，此程序演示了使用 wiringOP 中 PWM 相关的 API 来操作 PWM 的方法

```
orangepi@orangepi:~$ cd wiringOP/examples/
orangepi@orangepi:~/wiringOP/examples$ ls pwm.c
pwm.c
```

2) 编译 **pwm.c** 为可执行程序的命令如下所示

```
orangepi@orangepi:~/wiringOP/examples$ gcc -o pwm pwm.c -lwiringPi
```

3) 然后就可以执行 PWM 测试程序了，在执行 PWM 测试程序时，需要指定 PWM 引脚，比如可以使用下面的命令对 PWM15_IR_M1 引脚进行测试：

```
orangepi@orangepi:~/wiringOP/examples$ sudo ./pwm 2
```

4) pwm 程序执行后会对以下内容依次进行测试：

a. 通过设置 ARR 调节 PWM 占空比
b. 通过设置 CCR 调节 PWM 占空比
c. 通过设置分频系数调节 PWM 频率
d. 直接设置 PWM 频率

5) 在每完成一项测试后，会保持当前 pwm 波形 5 秒钟，在完成所有测试内容后，会重新开始新一轮测试

6) PWM 测试程序的详细执行过程如下所示：

a. 通过设置 ARR 调节 PWM 占空比：通过示波器可以观察到 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 50%变为 25%，保持 5 秒，然后 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 25%变为 50%，保持 5 秒。
b. 通过设置 CCR 调节 PWM 占空比：通过示波器可以观察到 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 50%变为 100%，保持 5 秒，然后 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 100%变为 50%，保持 5 秒。
c. 通过设置分频系数调节 PWM 频率：通过示波器可以观察到 PWM 频率首先变为 12000Hz，然后 PWM 波形每隔 0.5 秒产生变化，在变化了 9 次后，PWM 频率变为 1200Hz，保持 5 秒，然后 PWM 波形每隔 0.5 秒产生变化，在变化了 9 次后，PWM 频率变为 12000Hz，保持 5 秒。
d. 直接设置 PWM 频率：通过示波器可以观察到 PWM 频率首先变为 2000Hz，然后每隔两秒 PWM 频率增加 2000Hz，在变化了 9 次后，PWM 频率变为

20000Hz，保持 5 秒。

3.18. wiringOP-Python 的安装使用方法

wiringOP-Python 是 wiringOP 的 Python 语言版本的库，用于在 Python 程序中操作开发板的 GPIO、I2C、SPI 和 UART 等硬件资源。

另外请注意下面所有的命令都是在 root 用户下操作的。

3.18.1. wiringOP-Python 的安装方法

1) 首先安装依赖包

```
root@orangepi:~# sudo apt-get update
root@orangepi:~# sudo apt-get -y install git swig python3-dev python3-setuptools
```

2) 然后使用下面的命令下载 wiringOP-Python 的源码

注意，下面的 git clone--recursive 命令会自动下载 wiringOP 的源码，因为 wiringOP-Python 是依赖 wiringOP 的。请确保下载过程没有因为网络问题而报错。

如果从 GitHub 下载代码有问题，可以直接使用 Linux 镜像中自带的 wiringOP-Python 源码，存放位置为：/usr/src/wiringOP-Python。

```
root@orangepi:~# git clone --recursive https://github.com/orangepi-xunlong/wiringOP-Python -b next
root@orangepi:~# cd wiringOP-Python
root@orangepi:~/wiringOP-Python# git submodule update --init --remote
```

3) 然后使用下面的命令编译 wiringOP-Python 并将其安装到开发板的 Linux 系统中

```
root@orangepi:~# cd wiringOP-Python
root@orangepi:~/wiringOP-Python# python3 generate-bindings.py > bindings.i
root@orangepi:~/wiringOP-Python# sudo python3 setup.py install
```

4) 然后输入下面的命令，如果有帮助信息输出，说明 wiringOP-Python 安装成功，按下 **q** 键可以退出帮助信息的界面

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi;  help(wiringpi)"
Help on module wiringpi:
```

```
NAME
    wiringpi

DESCRIPTION
    # This file was automatically generated by SWIG (http://www.swig.org).
    # Version 4.0.2
    #
    # Do not make changes to this file unless you know what you are doing--modify
    # the SWIG interface file instead.
```

5) 在 python 命令行下测试 wiringOP-Python 是否安装成功的步骤如下所示：

a. 首先使用 python3 命令进入 python3 的命令行模式

```
root@orangepi:~# python3
```

b. 然后导入 wiringpi 的 python 模块

```
>>> import wiringpi;
```

c. 最后输入下面的命令可以查看下 wiringOP-Python 的帮助信息，按下 **q** 键可以退出帮助信息的界面

```
>>> help(wiringpi)
Help on module wiringpi:

NAME
    wiringpi

DESCRIPTION
    # This file was automatically generated by SWIG (http://www.swig.org).
    # Version 4.0.2
    #
    # Do not make changes to this file unless you know what you are doing--modify
    # the SWIG interface file instead.

CLASSES
    builtins.object
        GPIO
        I2C
        Serial
```

```
        nes


    class GPIO(builtins.object)
     |  GPIO(pinmode=0)
     |

>>>
```

3.18.2. 40pin GPIO 口测试

wiringOP-Python 跟 wiringOP 一样，也是可以通过指定 wPi 号来确定操作哪一个 GPIO 引脚，因为 wiringOP-Python 中没有查看 wPi 号的命令，所以只能通过 wiringOP 中的 gpio 命令来查看板子 wPi 号与物理引脚的对应关系。

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |    PWM15 |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
 |  118 |   5 | GPIO3_C6 |     IN | 0 | 11 || 12 | 0 | IN     | GPIO3_C7 | 6   | 119  |
 |  128 |   7 | GPIO4_A0 |     IN | 0 | 13 || 14 |   |        | GND      |     |      |
 |  130 |   8 |    TXD.7 |     IN | 0 | 15 || 16 | 0 | IN     | RXD.7    | 9   | 131  |
 |      |     |     3.3V |        |   | 17 || 18 | 0 | IN     | GPIO4_A1 | 10  | 129  |
 |  138 |  11 | SPI3_TXD |     IN | 0 | 19 || 20 |   |        | GND      |     |      |
 |  136 |  12 | SPI3_RXD |     IN | 0 | 21 || 22 | 0 | IN     | TXD.9    | 13  | 132  |
 |  139 |  14 | SPI3_CLK |     IN | 0 | 23 || 24 | 0 | IN     | SPI3_CS1 | 15  | 134  |
 |      |     |      GND |        |   | 25 || 26 | 0 | IN     | GPIO3_D6 | 16  | 126  |
 |   32 |  17 |    SDA.3 |     IN | 1 | 27 || 28 | 1 | IN     | SCL.3    | 18  | 33   |
 |  133 |  19 |    RXD.9 |     IN | 0 | 29 || 30 |   |        | GND      |     |      |
 |  124 |  20 | GPIO3_D4 |     IN | 0 | 31 || 32 | 0 | IN     | PWM11    | 21  | 144  |
 |  127 |  22 | GPIO3_D7 |     IN | 0 | 33 || 34 |   |        | GND      |     |      |
 |  120 |  23 | GPIO3_D0 |     IN | 0 | 35 || 36 | 0 | IN     | GPIO3_D5 | 24  | 125  |
 |  123 |  25 | GPIO3_D3 |     IN | 0 | 37 || 38 | 0 | IN     | GPIO3_D2 | 26  | 122  |
 |      |     |      GND |        |   | 39 || 40 | 0 | IN     | GPIO3_D1 | 27  | 121  |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
```

1) 下面以 7 号引脚——对应 GPIO 为 GPIO4_C3 ——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的高低电平

```
root@orangepi3b:~# gpio readall
 +------+-----+----------+--------+---+   PI3B   +---+--------+----------+-----+------+
 | GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
 +------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
 |      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
 |  140 |   0 |    SDA.2 |     IN | 1 |  3 || 4  |   |        | 5V       |     |      |
 |  141 |   1 |    SCL.2 |     IN | 1 |  5 || 6  |   |        | GND      |     |      |
 |  147 |   2 |    PWM15 |     IN | 0 |  7 || 8  | 1 | ALT1   | RXD.2    | 3   | 25   |
 |      |     |      GND |        |   |  9 || 10 | 1 | ALT1   | TXD.2    | 4   | 24   |
```

2) 直接用命令测试的步骤如下所示：

a. 首先设置 GPIO 口为输出模式，其中 **pinMode** 函数的第一个参数是引脚对应的 wPi 的序号，第二个参数是 GPIO 的模式

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; \
from wiringpi import GPIO; wiringpi.wiringPiSetup() ; \
wiringpi.pinMode(2, GPIO.OUTPUT) ;  "
```

b. 然后设置 GPIO 口输出低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 0v，说明设置低电平成功

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; \
 from wiringpi import GPIO; wiringpi.wiringPiSetup() ;\
wiringpi.digitalWrite(2, GPIO.LOW)"
```

c. 然后设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功

```
root@orangepi:~/wiringOP-Python# python3 -c "import wiringpi; \
 from wiringpi import GPIO; wiringpi.wiringPiSetup() ;\
wiringpi.digitalWrite(2, GPIO.HIGH)"
```

3) 在 python3 的命令行中测试的步骤如下所示：

a. 首先使用 python3 命令进入 python3 的命令行模式

```
root@orangepi:~# python3
```

b. 然后导入 wiringpi 的 python 模块

```
>>> import wiringpi
>>> from wiringpi import GPIO
```

c. 然后设置 GPIO 口为输出模式，其中 **pinMode** 函数的第一个参数是引脚对应的 wPi 的序号，第二个参数是 GPIO 的模式

```
>>> wiringpi.wiringPiSetup()
0
>>> wiringpi.pinMode(2, GPIO.OUTPUT)
```

d. 然后设置 GPIO 口输出低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 0v，说明设置低电平成功

```
>>> wiringpi.digitalWrite(2, GPIO.LOW)
```

e. 然后设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功

```
>>> wiringpi.digitalWrite(2, GPIO.HIGH)
```

4) wiringOP-Python 在 python 代码中设置 GPIO 高低电平的方法可以参考下 examples 中的 **blink.py** 测试程序，**blink.py** 测试程序会设置开发板 40 pin 中所有的 GPIO 口的电压不断的高低变化

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# ls blink.py
blink.py
root@orangepi:~/wiringOP-Python/examples# python3 blink.py
```

3.18.3. 40pin SPI 测试

1) 由下表可知，Orange Pi 3B 可用的 spi 为 spi3

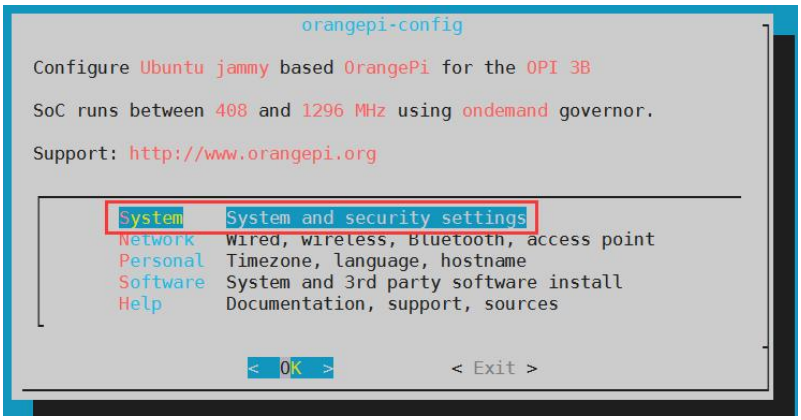
复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

2) 在 linux 系统中，40 pin 中的 SPI 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

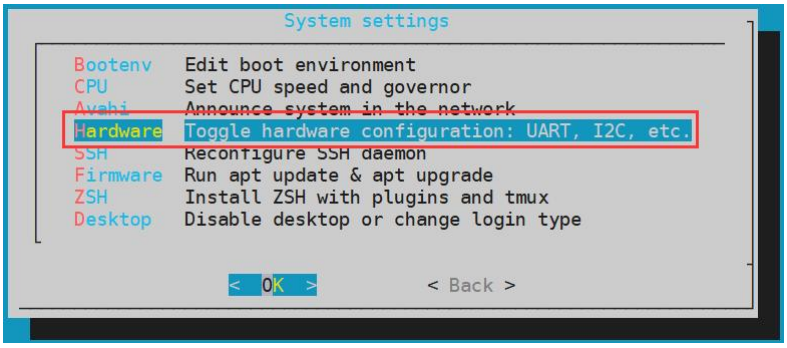
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

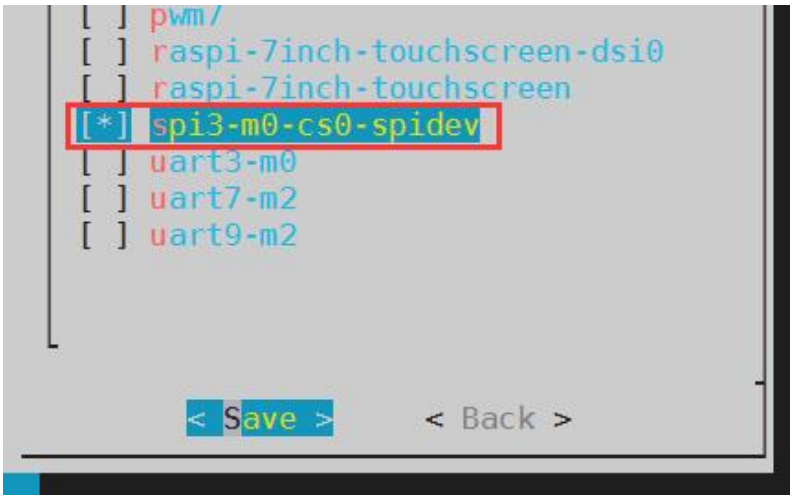
b. 然后选择 System



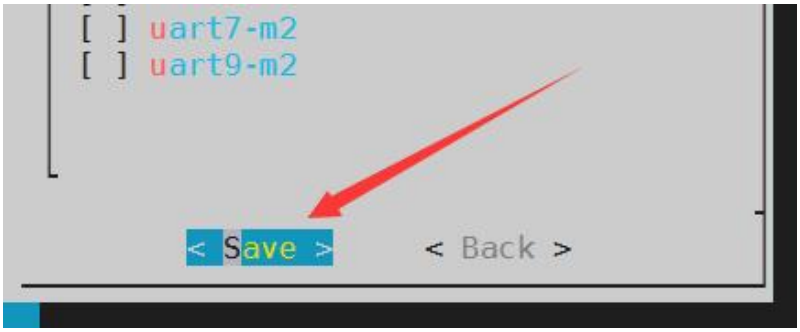
c. 然后选择 Hardware



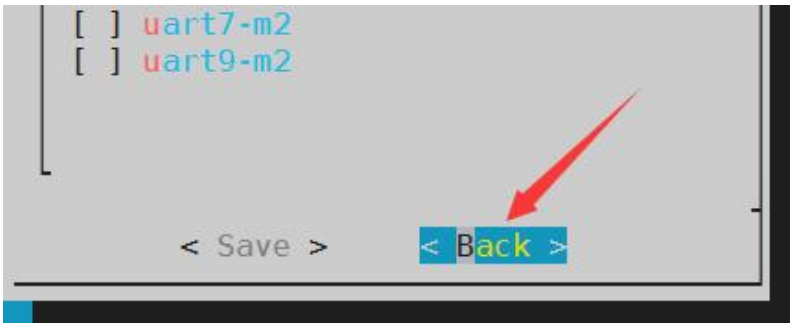
d. 然后使用键盘的方向键定位到下图所示的位置，再使用空格选中想要打开的 SPI 的配置



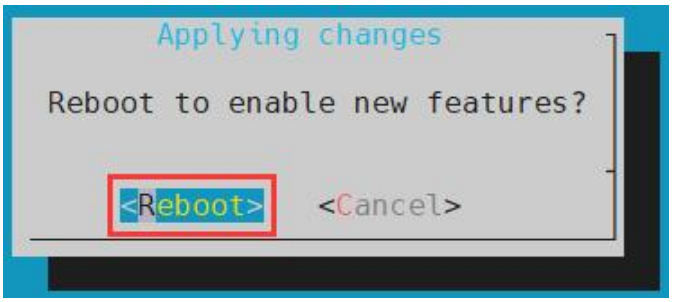
e. 然后选择<Save>保存



f. 然后选择<Back>



g. 然后选择<Reboot>重启系统使配置生效



3) 先查看下 Linux 系统中是否存在 **spidev3.0** 的设备节点，如果存在，说明 SPI3 已经设置好了，可以直接使用

```
orangepi@orangepi:~$ ls /dev/spidev3.0
/dev/spidev3.0
```

4) 然后可以使用 examples 中的 **spidev_test.py** 程序测试下 SPI 的回环功能，**spidev_test.py** 程序需要指定下面的两个参数：

a. **--channel**：指定 SPI 的通道号

b. **--port**：指定 SPI 的端口号

5) 先不短接 SPI3 的 mosi 和 miso 两个引脚，运行 spidev_test.py 的输出结果如下所示，可以看到 TX 和 RX 的数据不一致

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 spidev_test.py \
--channel 3 --port 0
spi mode: 0x0
max speed: 500000 Hz (500 KHz)
Opening device /dev/spidev3.0
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    |......@.....|
```

```
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF    |.....|
```

6) 然后使用杜邦线短接 SPI3 的 txd（40pin 接口中的第 19 号引脚）和 rxd（40pin 接口中的第 21 号引脚）两个引脚再运行 spidev_test.py 的输出如下，可以看到发送和接收的数据一样，说明 SPI3 回环测试正常

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 spidev_test.py \
--channel 3 --port 0
spi mode: 0x0
max speed: 500000 Hz (500 KHz)
Opening device /dev/spidev3.0
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    |......@.....|
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D    |......@.....|
```

3. 18. 4. 40pin I2C 测试

1) 由下表可知，Orange Pi 3B 可用的 i2c 为 i2c2、i2c3 和 i2c4 共三组 i2c 总线

复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

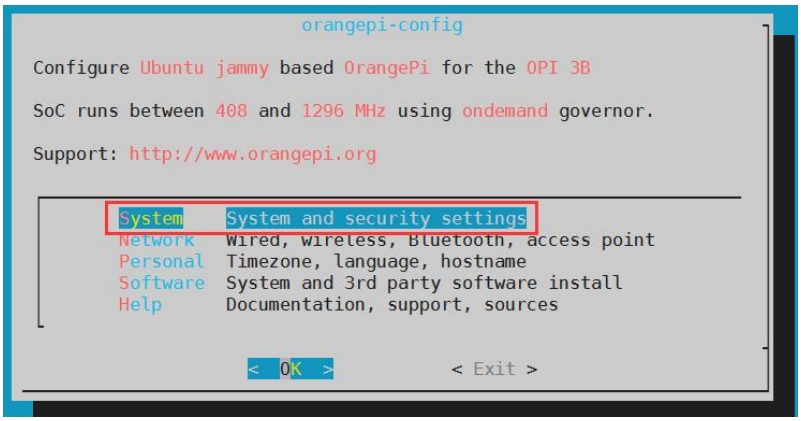
从上表中可以看到，i2c4_m0 和 spi3_m0 是引脚复用的，两者不能同时打开，i2c3_m0 和 uart3_m0 也是引脚复用的，两者不能同时打开

2) 在 linux 系统中，40 pin 中的 I2C 总线默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

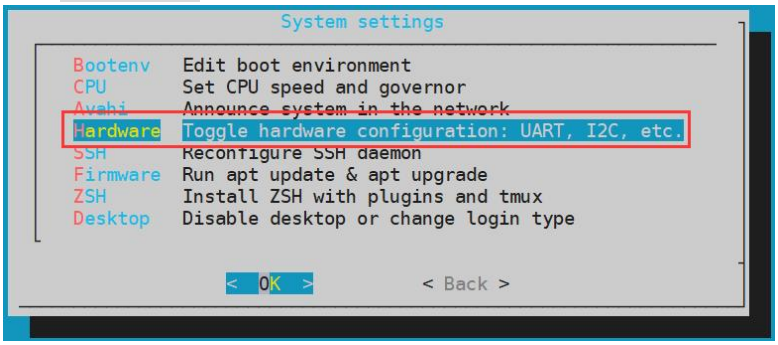
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**

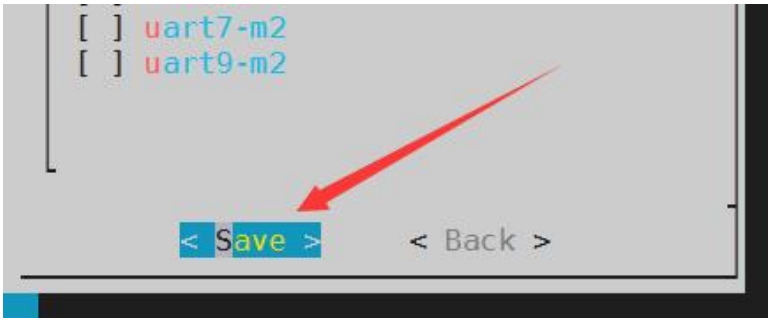


c. 然后选择 **Hardware**

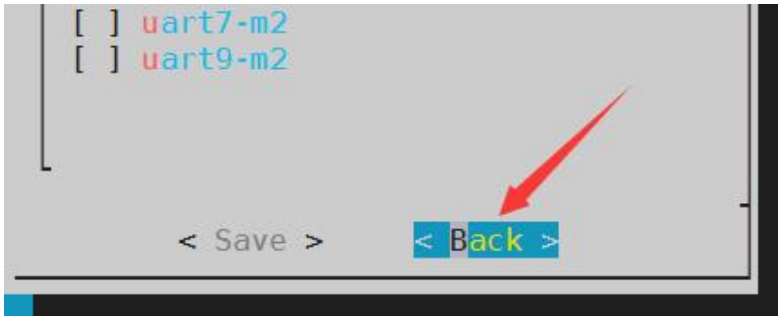


d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中想要打开的 I2C 的配置

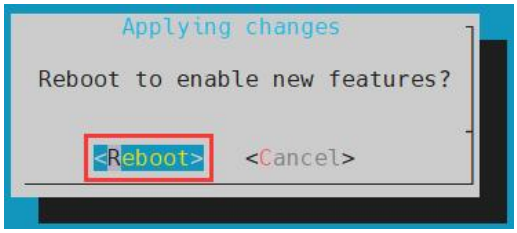
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



3) 启动 Linux 系统后，先确认下/dev 下存在 i2c 的设备节点

```
orangepi@orangepi:~# ls /dev/i2c-*
/dev/i2c-0  /dev/i2c-2  /dev/i2c-3  /dev/i2c-4  /dev/i2c-6
```

4) 然后在 40pin 接头的 i2c 引脚上接一个 i2c 设备，这里以 ds1307 RTC 模块为例

	i2c2-m1	i2c3-m0	i2c4-m0
sda 引脚	对应 3 号引脚	对应 27 号引脚	对应 19 号引脚
sck 引脚	对应 5 号引脚	对应 28 号引脚	对应 23 号引脚
vcc 引脚	对应 1 号引脚	对应 1 号引脚	对应 1 号引脚
gnd 引脚	对应 6 号引脚	对应 6 号引脚	对应 6 号引脚

5) 然后使用 **i2cdetect -y** 命令如果能检测到连接的 i2c 设备的地址，就说明 i2c 能正常使用

```
orangepi@orangepi:~$ sudo i2cdetect -y 2        #i2c2 的命令
```

```
orangepi@orangepi:~$ sudo i2cdetect -y 3      #i2c3 的命令
orangepi@orangepi:~$ sudo i2cdetect -y 4      #i2c4 的命令
```

```
root@orangepi3b:~# i2cdetect -y 4
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

6) 然后可以运行 **examples** 中的 **ds1307.py** 测试程序读取 RTC 的时间

```
root@orangepi:~/wiringOP-Python# cd examples
root@orangepi:~/wiringOP-Python/examples# python3 ds1307.py --device \
"/dev/i2c-4"
Thu 2023-01-05    14:57:55
Thu 2023-01-05    14:57:56
Thu 2023-01-05    14:57:57
^C
exit
```

3.18.5. 40pin 的 UART 测试

1) 由下表可知，Orange Pi 3B 可用的 uart 为 uart2、uart3、uart7 和 uart9 共四组 uart 总线，其中 uart2 默认是系统的调试串口

复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

从上表中可以看到，i2c3_m0 和 uart3_m0 是引脚复用的，两者不能同时打开

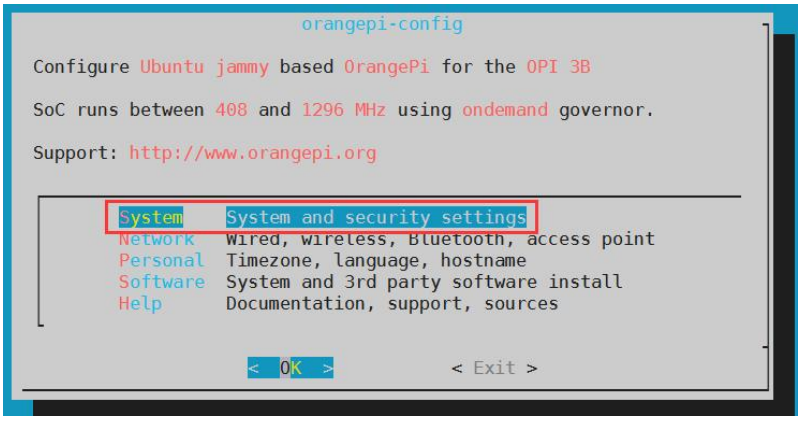
请注意，uart2_m0 默认是作为系统的调试串口，如果打开了 uart2 的配置，调试串口的功能会无法使用

2) 在 linux 系统中，40 pin 中的 UART 默认都是关闭的，需要手动打开才能使用。详细步骤如下所示：

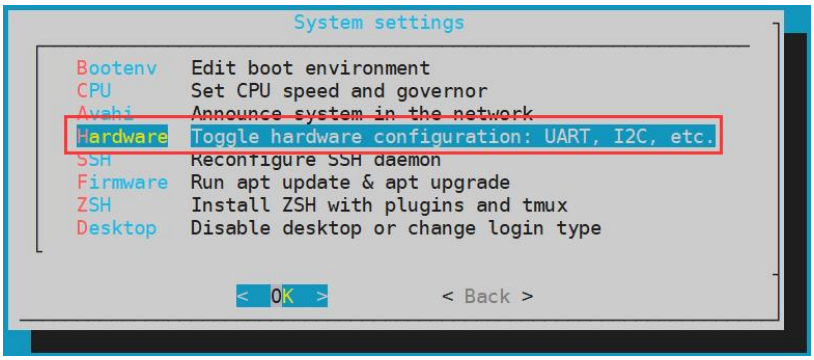
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

b. 然后选择 **System**

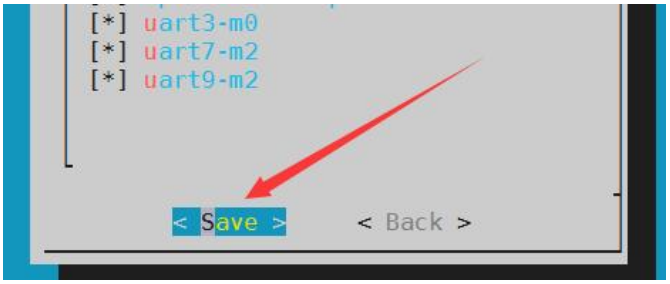


c. 然后选择 **Hardware**

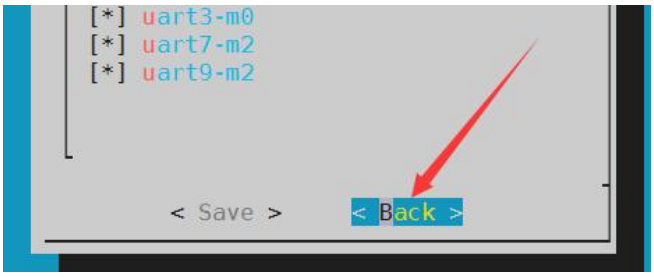


d. 然后使用键盘的方向键定位到下图所示的位置，再使用**空格**选中想要打开的 UART 的配置

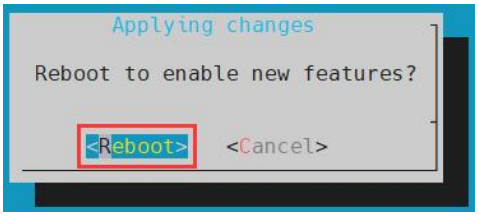
e. 然后选择**<Save>**保存



f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



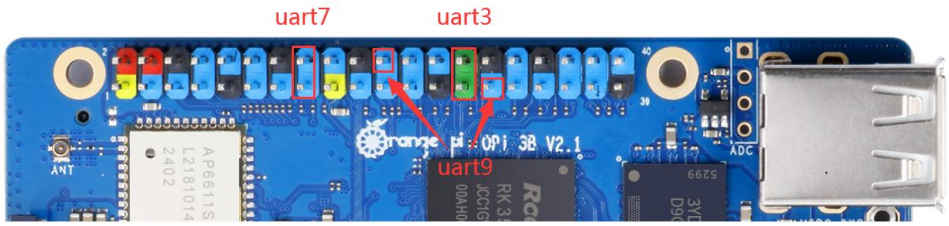
3) 进入 Linux 系统后，先确认下/dev 下是否存在对应 uart 的设备节点

```
orangepi@orangepi:~# ls /dev/ttyS*
/dev/ttyS1   /dev/ttyS3   /dev/ttyS7   /dev/ttyS9
```

注意，linux6.6 中/dev/ttyS0 是 uart9。

4) 然后开始测试 uart 接口，先使用杜邦线短接要测试的 uart 接口的 rx 和 tx

	uart3	uart7	uart9
tx 引脚	对应 28 号引脚	对应 16 号引脚	对应 29 号引脚
rx 引脚	对应 27 号引脚	对应 15 号引脚	对应 22 号引脚



5) 使用 examples 中的 **serialTest.py** 程序测试串口的回环功能如下所示，如果能看到下面的打印，说明串口通信正常

a. 测试 UART3

```
root@orangepi:~/wiringOP-Python/examples# python3 serialTest.py --device \
"/dev/ttyS3"

Out:    0: ->    0
Out:    1: ->    1
Out:    2: ->    2
Out:    3: ->    3
Out:    4:^C
exit
```

b. 测试 UART7

```
root@orangepi:~/wiringOP-Python/examples# python3 serialTest.py --device \
"/dev/ttyS7"

Out:    0: ->    0
Out:    1: ->    1
Out:    2: ->    2
Out:    3: ->    3
Out:    4:^C
exit
```

c. 测试 UART9

```
root@orangepi:~/wiringOP-Python/examples# python3 serialTest.py --device \
"/dev/ttyS9"

Out:    0: ->    0
Out:    1: ->    1
Out:    2: ->    2
Out:    3: ->    3
Out:    4:^C
exit
```

3.19. 硬件看门狗测试

注意，linux6.6 暂不支持。

Orange Pi 发布的 Linux 系统中预装了 watchdog_test 程序，可以直接测试。

运行 watchdog_test 程序的方法如下所示：

a. 第二个参数 10 表示看门狗的计数时间，如果这个时间内没有喂狗，系统会重启

b. 我们可以通过按下键盘上的任意键（ESC 除外）来喂狗，喂狗后，程序会打印一行 keep alive 表示喂狗成功

```
orangepi@orangepi:~$ sudo watchdog_test 10
open success
options is 33152,identity is sunxi-wdt
put_usr return,if 0,success:0
The old reset time is: 16
return ENOTTY,if -1,success:0
return ENOTTY,if -1,success:0
put_user return,if 0,success:0
put_usr return,if 0,success:0
keep alive
keep alive
keep alive
```

3.20. 查看 RK3566 芯片的序列号

查看 RK3566 芯片序列号的命令如下所示，每个芯片的序列号都是不同的，所以可以使用序列号来区分多个开发板。

```
orangepi@orangepi:~$ cat_serial.sh
Serial          : 8fa18eaf489041f0
```

3.21. 下载安装 arm64 版本 balenaEtcher 的方法

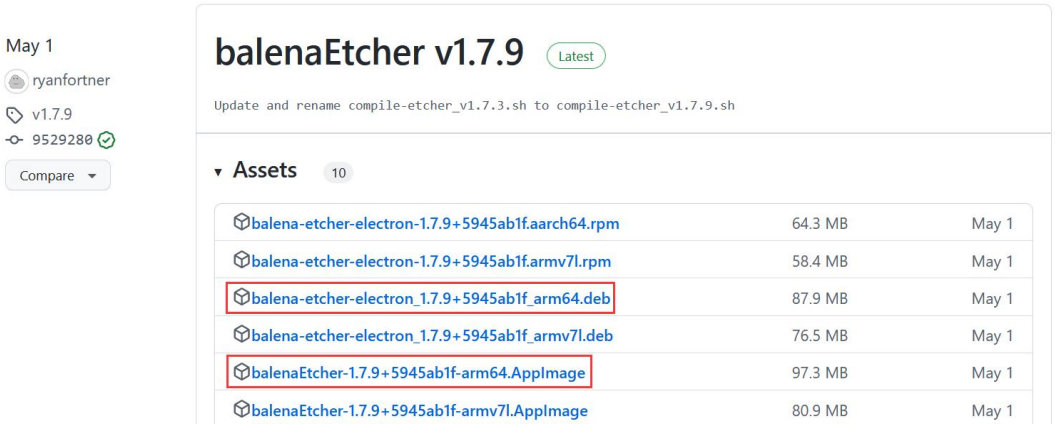
1) balenaEtcher arm64 版本的下载地址为：

a. deb 安装包的下载地址如下所示，需要安装才能使用

https://github.com/Itai-Nelken/BalenaEtcher-arm/releases/download/v1.7.9/balena-etcher-electron_1.7.9+5945ab1f_arm64.deb

b. 无需安装的 AppImage 版本的下载地址如下所示：

https://github.com/Itai-Nelken/BalenaEtcher-arm/releases/download/v1.7.9/balenaEtcher-1.7.9+5945ab1f-arm64.AppImage

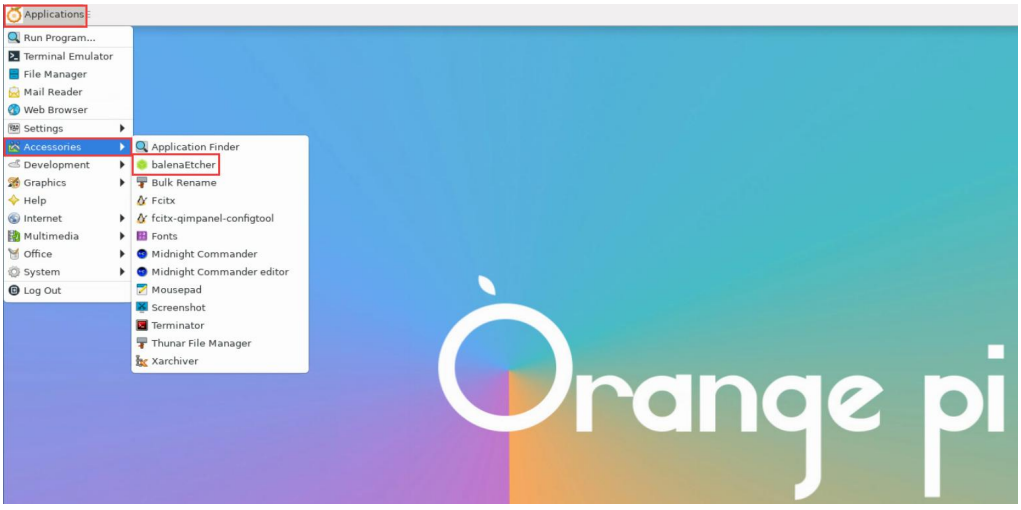


2) deb 版本 balenaEtcher 的安装使用方法：

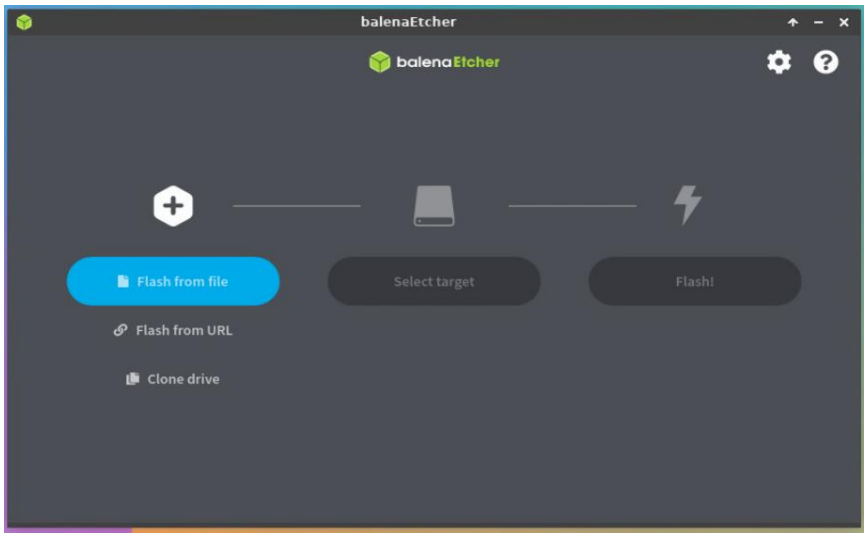
a. deb 版本的 balenaEtcher 安装命令如下所示：

```
orangepi@orangepi:~$ sudo apt install -y  \
--fix-broken ./balena-etcher-electron_1.7.9+5945ab1f_arm64.deb
```

b. deb 版本的 balenaEtcher 安装完成后，在 Application 中就可以打开了



c. balenaEtcher 打开后的界面如下所示：

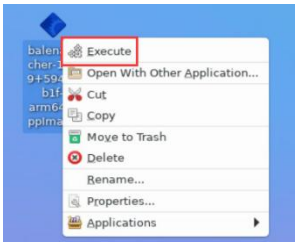


3) AppImage 版本的 balenaEtcher 的使用方法：

a. 首先给 balenaEtcher 添加权限

```
orangepi@orangepi:~/Desktop$ chmod +x balenaEtcher-1.7.9+5945ab1f-arm64.AppImage
```

b. 然后选中 AppImage 版本 balenaEtcher，再点击鼠标右键，再点击 Execute 就可以打开 balenaEtcher 了



3.22. 使用 NPU 的方法

3.22.1. 准备工具

1) 一台装有 Ubuntu22.04 操作系统的 PC。

根据 RKNN-Toolkit2 的官方文档，RKNN-Toolkit2 目前版本支持的操作系统如下所示：

a. Ubuntu18.04 (x64)

b. Ubuntu20.04 (x64)

c. Ubuntu22.04 (x64)

在本文档中我们使用 Ubuntu22.04 (x64)操作系统进行演示，其他版本的操作系统请自行测试。

2) 一块装有 Ubuntu Jammy 系统 5.10 内核的 Orange Pi 3B 开发板，请使用版本不低于 1.0.8 的系统镜像，如：

```
Orangepi3b_1.0.8_ubuntu_jammy_desktop_xfce_linux5.10.160.7z
```

3.22.2. 在 Ubuntu PC 端安装 RKNN-Toolkit2

Toolkit2 是一款在 Ubuntu PC 平台上使用的开发套件,用户使用该工具提供的 Python 接口可以便捷地完成模型转换、推理和性能评估等功能。

1) 在 Ubuntu PC 端，打开一个命令行窗口，然后输入以下命令来安装 python3 和 pip3。

```
test@test:~$ sudo apt-get install python3 python3-dev python3-pip
```

2) 可以使用以下命令来查看已安装的 python3 的版本。

```
test@test:~$ python3 --version
Python 3.10.12
```

3) 然后输入以下命令来安装 RKNN-Toolkit2 的依赖包。

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install libxslt1-dev zlib1g-dev libglib2.0 libsm6 libgl1-mesa-glx \
libprotobuf-dev gcc
```

4) 然后输入以下命令来下载 2.3.0 版本的 RKNN-Toolkit2。

```
test@test:~$ git clone https://github.com/airockchip/rknn-toolkit2 -b v2.3.0
```

5) 然后输入以下命令安装 python3 相应版本的依赖包，这个命令将使用 pip3 安装文件 requirements_cp310-2.3.0.txt 中列出的依赖项。如果依赖安装不全的话就不指定安装源单独安装里面的每个包。

```
test@test:~$ pip3 install -r rknn-toolkit2/rknn-toolkit2/packages/x86_64/requirements_cp310-2.3.0.txt -i https://mirrors.huaweicloud.com/repository/pypi/simple
```

6) 然后输入以下命令使用 pip3 来安装 RKNN-Toolkit2 软件包，安装完成后就可以使用 RKNN-Toolkit2 了。

```
test@test:~$ pip3 install rknn-toolkit2/rknn-toolkit2/packages/x86_64/rknn_toolkit2-2.3.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

3.22.3. 使用 RKNN-Toolkit2 进行模型转换和模型推理

RKNN-Toolkit2 支持将 Caffe、TensorFlow、TensorFlow Lite、ONNX、Dark Net、PyTorch 等模型转为 RKNN 模型，然后在 Ubuntu PC 端通过仿真或使用开发板的 NPU 运行 RKNN 模型来进行推理。

在 RKNN-Toolkit2 的 example 文件夹中提供了相关的示例，以帮助用户更好地理解如何操作，我们以具有 yolov5 功能的 ONNX 模型为例进行说明。

3.22.3.1. 在 Ubuntu PC 端仿真运行模型

RKNN-Toolkit2 搭载了内置模拟器，可以让用户在 Ubuntu PC 端模拟模型在 Rockchip NPU 上的推理过程。

这样模型转换和推理均可以在 Ubuntu PC 端完成，从而帮助用户更快地测试和验证他们的模型。

1) 首先切换到 yolov5 案例目录。

```
test@test:~$ cd rknn-toolkit2/rknn-toolkit2/examples/onnx/yolov5
```

2) 然后运行 test.py 脚本，该脚本首先将 yolov5s_relu.onnx 模型转换为可以在模拟器上运行的 RKNN 模型，然后使用模拟器仿真运行该模型对当前目录下的 bus.jpg 图片进行推理。

```
test@test:~/rknn-toolkit2/rknn-toolkit2/examples/onnx/yolov5$ python3 test.py
```

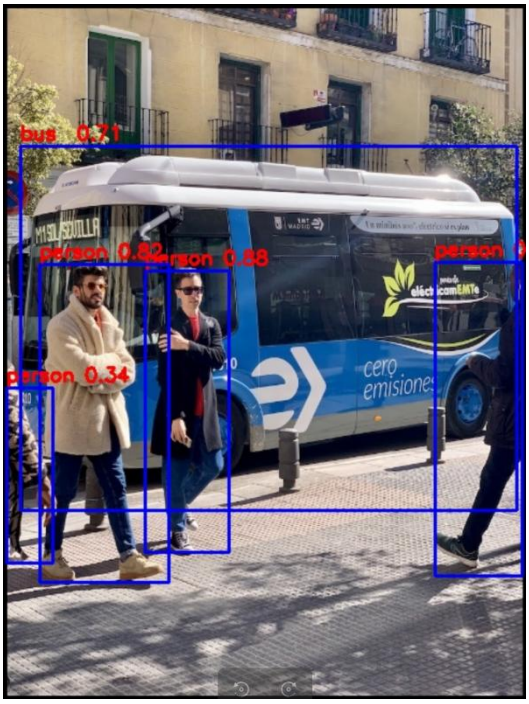
3) test.py 脚本成功运行后将看到如下打印信息,表明该模型在 bus.jpg 图片中成功检测到了四个人和一辆公交车

```
--> Running model
I GraphPreparing : 100%|████████████████████| 153/153 [00:00<00:00, 5884.07it/s]
I SessionPreparing : 100%|████████████████████| 153/153 [00:00<00:00, 1408.98it/s]
done
    class          score          xmin, ymin, xmax, ymax
--------------------------------------------------
    person         0.884          [ 208,   244,   286,   506]
    person         0.868          [ 478,   236,   559,   528]
    person         0.825          [ 110,   238,   230,   533]
```

```
person         0.334    [   79,  353,  122,  517]
bus            0.705    [   92,  128,  554,  467]
Save results to result.jpg!
```

4) 转换得到的模型文件 yolov5s_relu.rknn 和推理的图片结果 result.jpg 保存在当前目录下

5) result.jpg 图片展示了使用 yolov5s_relu.rknn 模型在 bus.jpg 图片中检测到的物体类别和置信率



3.22.3.2. 在 Ubuntu PC 端使用开发板的 NPU 运行模型

RKNN-Toolkit2 为用户提供了通过 adb 使用开发板的 NPU 进行推理的 Python 接口，可以让用户在 Ubuntu PC 端使用开发板的 NPU 来运行模型进行推理。

这样 Ubuntu PC 端可以根据模型在开发板的 NPU 上运行时的实际效果，使用 Python 提供的机器学习库，对模型进行优化和调整。

3.22.3.2.1. 通过网络连接开发板的 adb 服务

1) 系统启动后请先确认下 **adbd** 已经启动了。

```
orangepi@orangepi:~$ ps -ax | grep "adbd"
    808 ?        Sl     0:00 /usr/bin/adbd
   3707 ttyFIQ0  S+     0:00 grep --color=auto adbd
```

2) 然后查看下开发板的 IP 地址，并记下来。

3) 然后在 Ubuntu PC 上安装 adb 工具。

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y adb
```

4) 然后使用下面的命令连接网络 adb。

```
test@test:~$ adb connect 192.168.1.xx:5555          #IP 地址请替换为开发板的 IP 地址
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.1.xx:5555
test@test:~$ adb devices
List of devices attached
192.168.1.xx:5555          device
```

3.22.3.2.2. 更新开发板的 rknn_server 和 librknnrt.so

librknnrt.so 是一个板端的 runtime 库。

rknn_server 是一个运行在开发板上的后台代理服务，用于接收 PC 通过 USB 传输过来的协议，然后执行板端 runtime 库中对应的接口，并返回结果给 PC。

1) 首先在 Ubuntu PC 端输入以下命令下载 2.3.0 版本的 RKNPU2。

```
test@test:~$ git clone https://github.com/airockchip/rknn-toolkit2 -b v2.3.0
```

2) 然后在 Ubuntu PC 端输入以下命令通过 adb 工具更新开发板的 rknn_server。

```
test@test:~$ adb push rknn-toolkit2/rknpu2/runtime/Linux/rknn_server/aarch64/usr/bin/* /usr/bin
```

3) 然后在 Ubuntu PC 端输入以下命令通过 adb 工具更新开发板的 librknnrt.so 库。

```
test@test:~$ adb push rknn-toolkit2/rknpu2/runtime/Linux/librknn_api/aarch64/librknnrt.so /usr/lib
```

4) 通过 adb 工具打开开发板的终端。

```
test@test:~$ adb shell
```

5) 重启开发板的 rknn_server 服务。

```
root@orangepi:/# chmod 777 /usr/bin/rknn_server
root@orangepi:/# chmod +x /usr/bin/restart_rknn.sh
root@orangepi:/# chmod +x /usr/bin/start_rknn.sh
root@orangepi:/# sudo restart_rknn.sh
```

6) 可以使用下面的命令检查下，如果出现 rknn_server 的进程 id，说明 rknn_server 已经打开，这样开发板的运行环境就搭建好了。

```
root@orangepi:/# pgrep rknn_server
3131
```

3.22.3.2.3. 修改示例中的参数

1) 在 Ubuntu PC 端通过下面的命令可以查看到连接到 Ubuntu PC 的开发板的设备 ID，这个 ID 在下面会用到。

```
test@test:~$ adb devices
List of devices attached
192.168.1.xx:5555 device
```

2) 切换到 yolov5 案例目录。

```
test@test:~$ cd rknn-toolkit2/rknn-toolkit2/examples/onnx/yolov5
```

3) 使用 vim 编辑器对 test.py 文件进行修改

```
test@test:~/rknn-toolkit2/rknn-toolkit2/examples/onnx/yolov5$ vim test.py
```

4) 在 test.py 文件中，我们需要对以下内容进行修改：

```
@@ -268,7 +268,7 @@ if __name__ == '__main__':
    # Init runtime environment
    print('--> Init runtime environment')
```

```
-       ret = rknn.init_runtime()
+       ret = rknn.init_runtime(target='rk3566',device_id='192.168.1.xx:5555')
        if ret != 0:
            print('Init runtime environment failed!')
            exit(ret)
```

a. 在预处理配置中，将目标平台修改为 rk3566，这样模型转换后得到的是适用于 RK3566 开发板的 NPU 的 RKNN 模型。

b. 在初始化运行环境中，增加对目标平台和设备 ID 的说明，目标平台是 rk3566，device_id 中的 ip 请换成“adb devices”命令输出的 ip，运行模型进行推理的操作将会在 RK3566 开发板的 NPU 上进行。

3.22.3.2.4. 在 Ubuntu PC 端运行示例

1) 输入以下命令运行 test.py 脚本，该脚本首先转换 yolov5s_relu.onnx 模型为 RKNN 模型，然后加载该模型到开发板的 NPU 上来对当前目录下的 out.jpg 图片进行推理。

```
test@test:~/rknn-toolkit2/rknn-toolkit2/examples/onnx/yolov5$ python3 test.py
```

2) 在打印信息中，我们可以看到 Ubuntu PC 通过 adb 工具使用开发板的 NPU 运行模型进行推理。

a. 如果报错“init_runtime: Get board target failed, ret code: 1. error: device offline”，请依次执行下面的命令重启 rknn_server 服务。

```
test@test:~$ adb shell
root@orangepi:/# sudo restart_rknn.sh
```

b. 在正常运行后，在打印的信息的最后，会显示推理结果。

```
--> Running model
done
    class          score          xmin, ymin, xmax, ymax
-------------------------------------------------
    person         0.884          [ 209,   244,   286,   506]
    person         0.868          [ 478,   238,   559,   526]
    person         0.825          [ 110,   238,   230,   534]
    person         0.339          [  79,   354,   122,   516]
     bus           0.705          [  94,   129,   553,   468]
Save results to result.jpg!
```

3) test.py 脚本成功运行后，转换得到的模型文件 yolov5s_relu.rknn 和推理的图片结果 result.jpg 保存在当前目录下。

4) 运行的结果和在 Ubuntu PC 端仿真运行模型一小节是一样的。

3.22.4. 调用 C 接口部署 RKNN 模型到开发板上运行

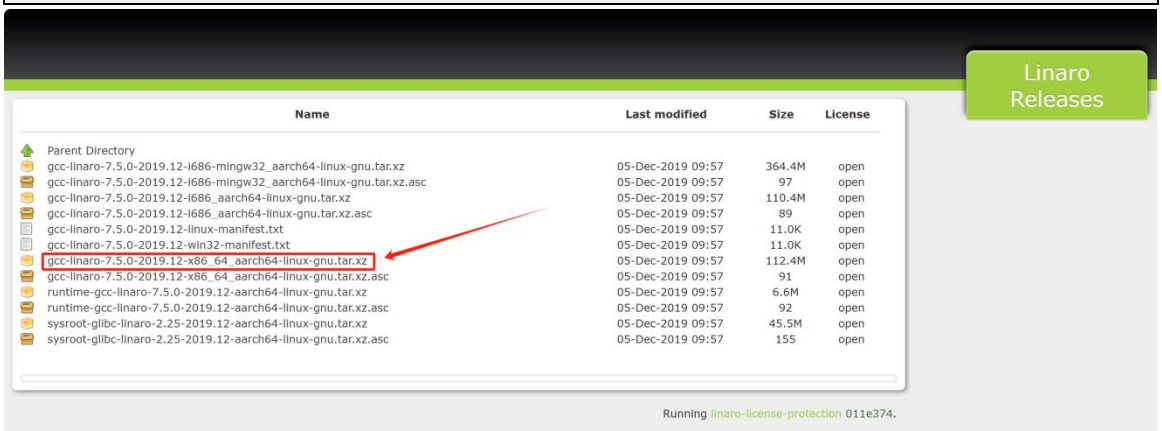
RKNPU2 为带有 Rockchip NPU 的芯片平台提供 C 编程接口，能够帮助用户部署使用 RKNN-Toolkit2 导出的 RKNN 模型，加速 AI 应用的落地。

在 RKNPU2 的 example 文件夹中，提供了将不同功能的 RKNN 模型部署到开发板的示例，我们以部署具有 yolov5 功能的 RKNN 模型到 RK3588 Ubuntu Jammy 平台为例进行说明。

3.22.4.1. 下载交叉编译工具

1) 由于开发板运行的是 Linux 系统，因此需要使用 gcc 交叉编译器来编译。访问下面的链接，下载交叉编译工具链。

https://releases.linaro.org/components/toolchain/binaries/latest-7/aarch64-linux-gnu/



2) 上传并解压到 ubuntu PC 中的/opt 目录。

```
test@test:~$ sudo tar xf gcc-linaro-7.5.0-2019.12-x86_64_aarch64-linux-gnu.tar -C /opt
```

3.22.4.2. 编译 rknn_yolov5_demo

1) 运行 build-linux.sh，该脚本通过交叉编译生成了适用于 RK3566 开发板并能够在其上运行 RKNN 模型进行推理的程序。

```
test@test:~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo$ chmod +x build-linux.sh
test@test:~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo$ export \
GCC_COMPILER=/opt/gcc-linaro-7.5.0-2019.12-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu
test@test:~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo$ ./build-linux.sh -t rk3566 -a aarch64 \
-b Release
```

2) 此处可能在编译时会报错“No rule to make target '../../../3rdparty/mpp/Linux/aarch64/librockchip_mpp.so', needed by 'rknn_yolov5_video_demo'. Stop.”可以按照下面的方法修改，然后重新按照上一步的步骤执行编译。

```
test@test:~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo$ cd ../3rdparty/mpp/Linux/aarch64/
test@test:~/rknn-toolkit2/rknpu2/examples/3rdparty/mpp/Linux/aarch64$ rm librockchip_mpp.so \
librockchip_mpp.so.1
test@test:~/rknn-toolkit2/rknpu2/examples/3rdparty/mpp/Linux/aarch64$ ln -s librockchip_mpp.so.1 \
librockchip_mpp.so
test@test:~/rknn-toolkit2/rknpu2/examples/3rdparty/mpp/Linux/aarch64$ ln -s librockchip_mpp.so.0 \
librockchip_mpp.so.1
```

3) 运行 build-linux.sh 完成后，在当前目录下会多出一个名为 install 的文件夹，该文件夹下的 rknn_yoov5_demo_Linux 文件夹中包含了通过交叉编译生成的程序以及与其相关的文件

```
test@test:~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo$ ls install/rknn_yolov5_demo_Linux/
lib  model  rknn_yolov5_demo  rknn_yolov5_video_demo
```

3.22.4.3. 部署 rknn_yolov5_demo 到开发板

在 Ubuntu PC 端，可以使用以下命令通过 adb 工具将 rknn_yolov5_demo_Linux 文件夹上传至开发板中，从而实现 rknn_yolov5_demo 在开发板上的部署

```
test@test:~/rknn-toolkit2/rknpu2/examples/rknn_yolov5_demo$ adb push \
install/rknn_yolov5_demo_Linux /home/orangepi/rknn_yolov5_demo_Linux
```

3.22.4.4. 在开发板上运行 rknn_yolov5_demo

1) 在 Ubuntu PC 端通过 adb shell 进入开发板的文件系统

```
test@test:~$ adb shell
```

```
root@orangepi:/#
```

2) 切换到 rknn_yolov5_demo_Linux 目录

```
root@orangepi:/# cd /home/orangepi/rknn_yolov5_demo_Linux
root@orangepi:/home/orangepi/rknn_yolov5_demo_Linux# ls
lib  model  rknn_yolov5_demo  rknn_yolov5_video_demo
```

3) 然后运行 rknn_yolov5_demo 程序来进行推理，在下面的命令中该程序使用 yolov5s-640-640.rknn 模型对 bus.jpg 图片进行推理，整个运行过程将在开发板上完成

```
root@orangepi:/home/orangepi/rknn_yolov5_demo_Linux# ./rknn_yolov5_demo \
./model/RK3566_RK3568/yolov5s-640-640.rknn ./model/bus.jpg
```

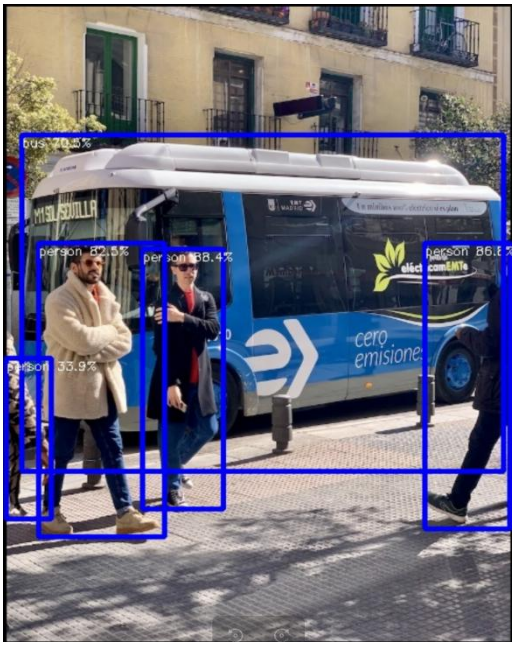
4) 运行完成后，推理的结果 out.jpg 图片保存在当前目录下

```
root@orangepi:/home/orangepi/rknn_yolov5_demo_Linux# ls
lib  model  out.jpg  rknn_yolov5_demo  rknn_yolov5_video_demo
```

5) 在 Ubuntu PC 端，我们可以使用以下命令通过 adb 工具下载 out.jpg 图片，然后使用图像查看器进行查看

```
test@test:~$ adb pull /home/orangepi/rknn_yolov5_demo_Linux/out.jpg ~/Desktop/
```

6) out.jpg 图片展示了使用 yolov5s-640-640.rknn 模型在 bus.jpg 图片中检测到的物体类别和置信率



3.23. 宝塔 Linux 面板的安装方法

宝塔 Linux 面板是提升运维效率的服务器管理软件，支持一键配置 LAMP/LNMP/集群/监控/网站/FTP/数据库/JAVA 等 100 多项服务器管理功能（摘抄自宝塔官网）

1) 首先需要扩展下**/tmp** 空间的大小，设置完后需要**重启下开发板的 linux 系统**，命令如下所示：

```
orangepi@orangepi:~$ sudo sed -i 's/nosuid/&,size=2G/' /etc/fstab
orangepi@orangepi:~$ sudo reboot
```

2) 重启后，可以看到**/tmp** 空间的大小变为 2G 了

```
orangepi@orangepi:~$ df -h | grep "/tmp"
tmpfs                   2.0G     12K  2.0G     1% /tmp
```

3) 然后在 Linux 系统中输入下面的命令就可以开始宝塔的安装

```
orangepi@orangepi:~$ sudo install_bt_panel.sh
```

4) 然后宝塔安装程序会提醒是否安装 **Bt-Panel** 到**/www** 文件夹，此时输入 **y** 即可

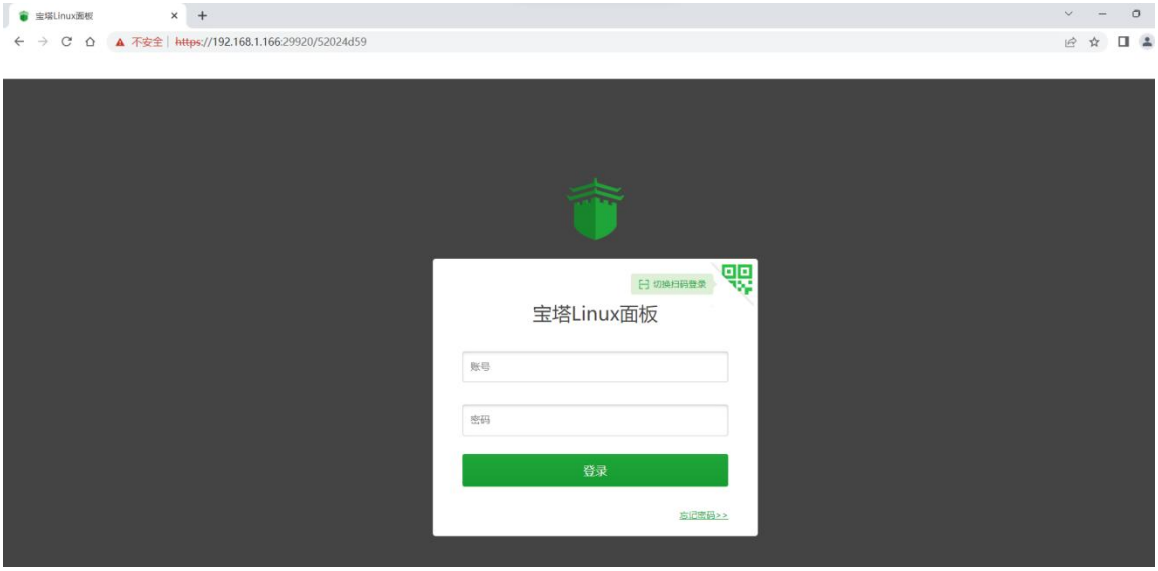
```
+----------------------------------------------------------------------
```

```
| Bt-WebPanel FOR CentOS/Ubuntu/Debian
+-------------------------------------------------------------------
| Copyright © 2015-2099 BT-SOFT(http://www.bt.cn) All rights reserved.
+-------------------------------------------------------------------
| The WebPanel URL will be http://SERVER_IP:8888 when installed.
+-------------------------------------------------------------------

Do you want to install Bt-Panel to the /www directory now?(y/n): y
```

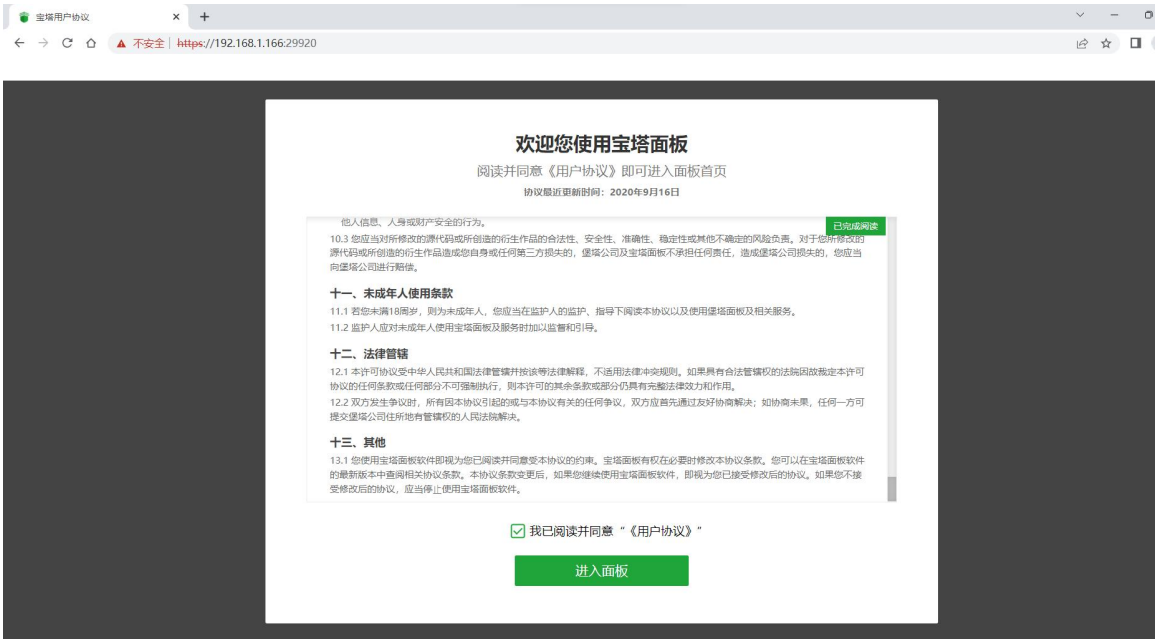
5) 然后要做的就是耐心等待，当看到终端输出下面的打印信息时，说明宝塔已经安装完成，整个安装过程大约耗时 34 分钟，根据网络速度的不同可能会有一些差别

```
==================================================================
Congratulations! Installed successfully!
==================================================================
外网面板地址: https://183.15.204.194:29920/52024d59
内网面板地址: https://192.168.1.166:29920/52024d59
username: 4qhagfrc
password: 27b2d026
If you cannot access the panel,
release the following panel port [29920] in the security group
若无法访问面板，请检查防火墙/安全组是否有放行面板[29920]端口
因已开启面板自签证书，访问面板会提示不匹配证书，请参考以下链接配置证书
https://www.bt.cn/bbs/thread-105443-1-1.html
==================================================================
Time consumed: 34 Minute!
orangepi@orangepi:~$
```

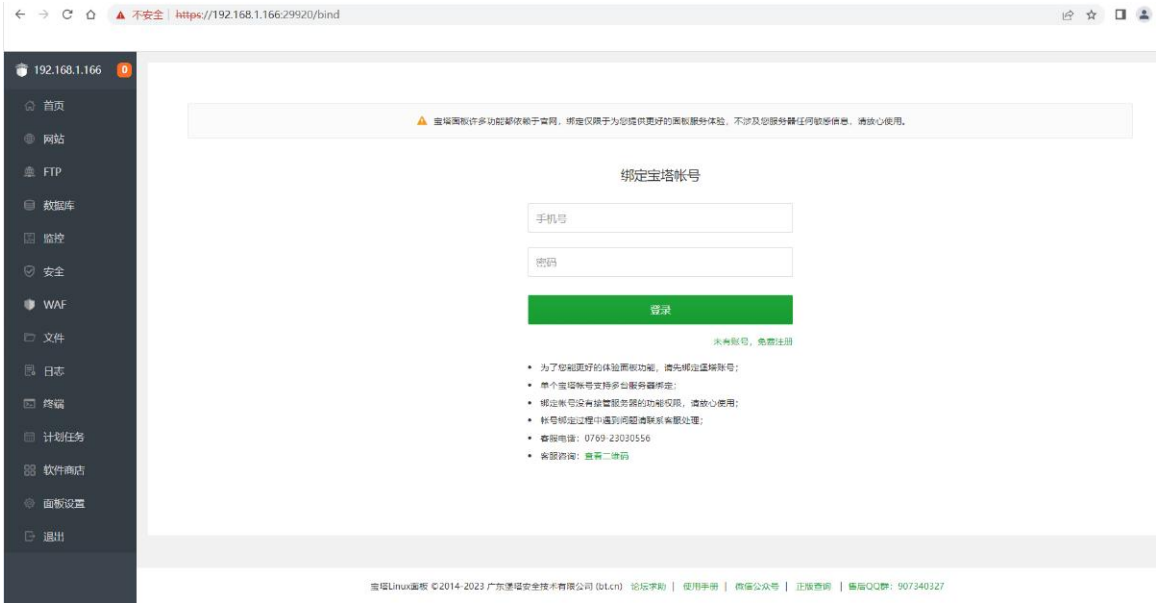
6) 此时在浏览器中输入上面显示的**面板地址**就可以打开宝塔 Linux 面板的登录界面，然后在对应的位置输入上图显示的 **username** 和 **password** 就可以登录进宝塔



7) 成功登录宝塔后的会弹出下面的欢迎界面，首先请将中间的用户须知阅读完拖到最下面，然后就可以选择“我已同意并阅读《用户协议》”，接着点击“进入面板”就可以进入宝塔了

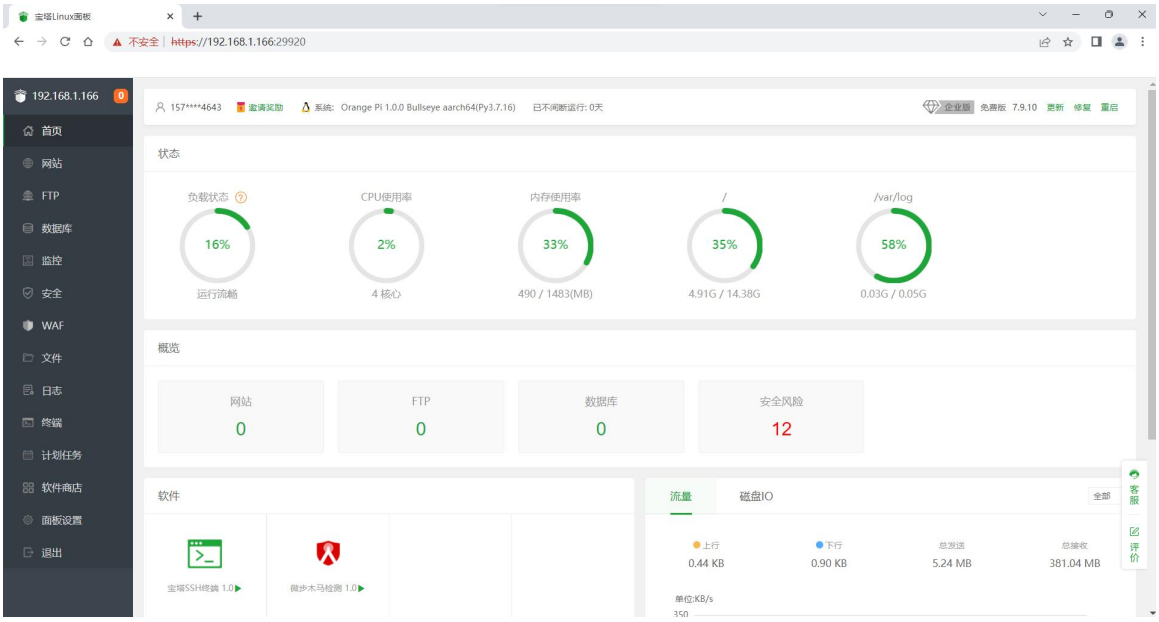


8) 进入宝塔后首先会提示需要绑定宝塔官网的账号，如果没有账号可以去宝塔的官网（**https://www.bt.cn**）注册一个



9) 最终显示的界面如下图所示，可以很直观的看到开发板 Linux 系统的一些状态信

息，比如负载状态、CPU 的使用率、内存使用率和存储空间的使用情况等



10) 宝塔的更多功能可以参考下面资料自行探索

使用手册：**http://docs.bt.cn**
论坛地址：**https://www.bt.cn/bbs**
GitHub 链接：**https://github.com/aaPanel/BaoTa**

3.24. 设置中文环境以及安装中文输入法

注意，安装中文输入法前请确保开发板使用的 Linux 系统为桌面版系统。

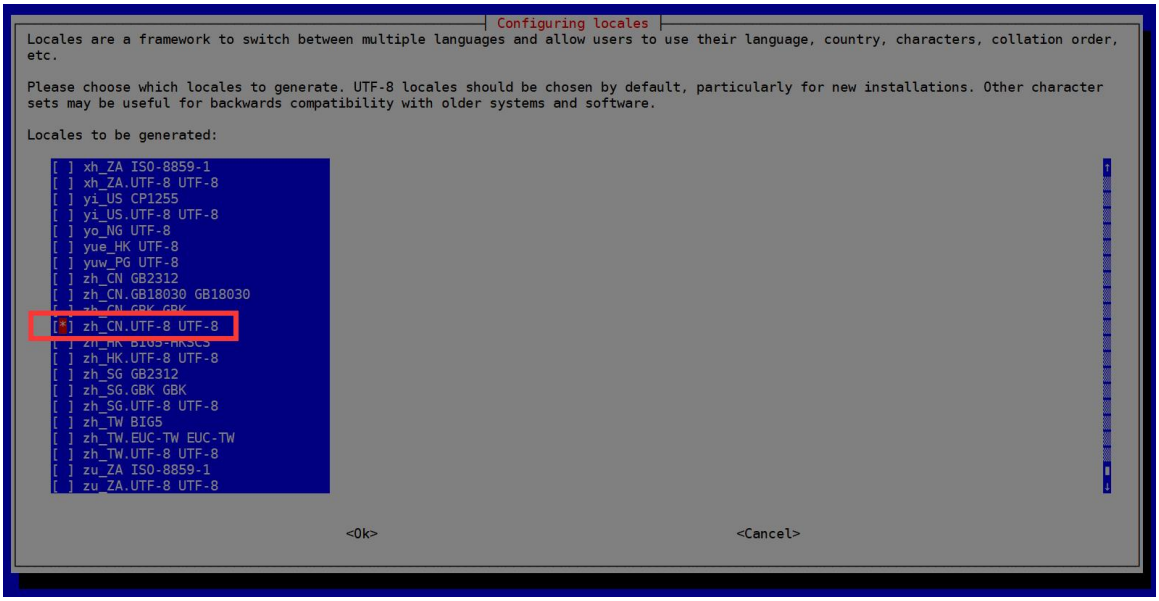
3.24.1. Debian 系统的安装方法

1) 首先设置默认 **locale** 为中文

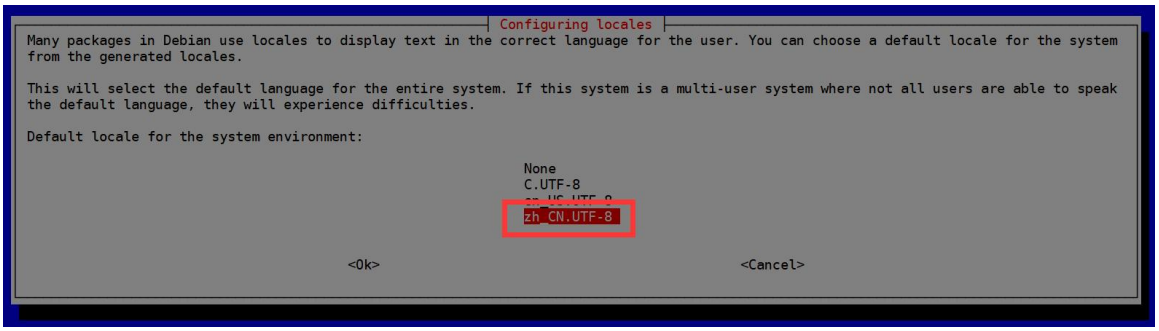
a. 输入下面的命令可以开始配置 **locale**

```
orangepi@orangepi:~$ sudo dpkg-reconfigure locales
```

b. 然后在弹出的界面中选择 **zh_CN.UTF-8 UTF-8**（通过键盘上的上下方向按键来上下移动，通过空格键来选择，最后通过 Tab 键可以将光标移动到 **<OK>**，然后回车即可）



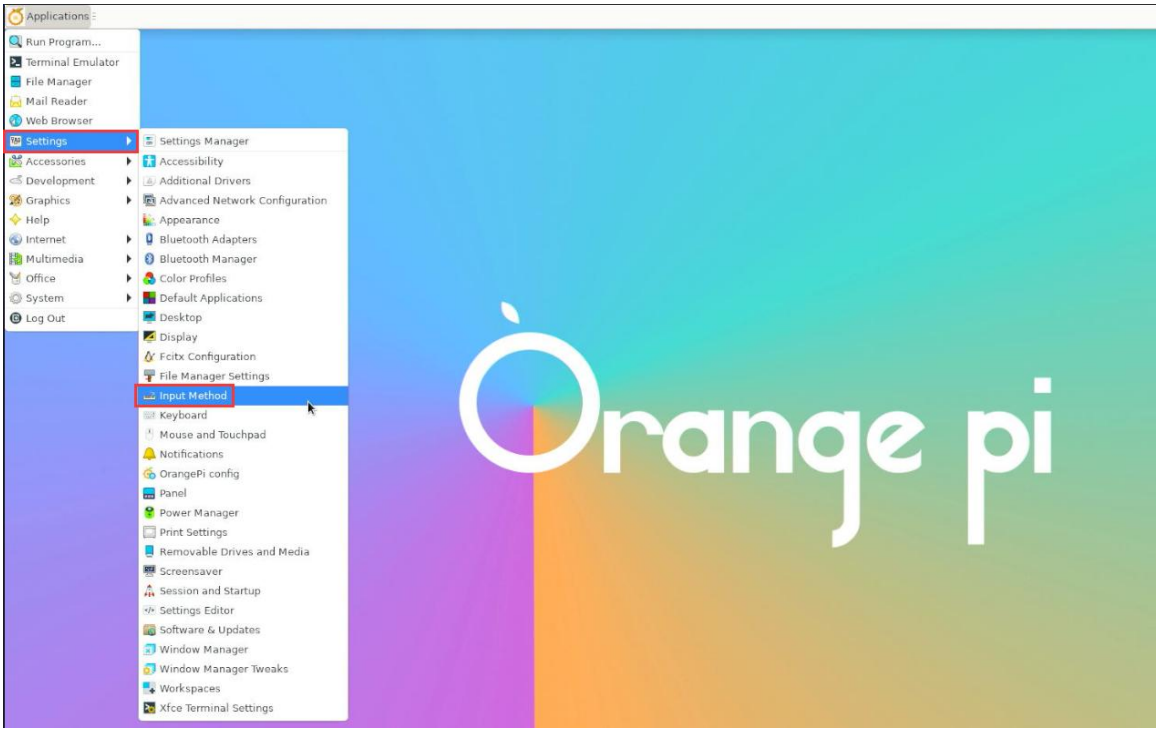
c. 然后设置默认 **locale** 为 **zh_CN.UTF-8**



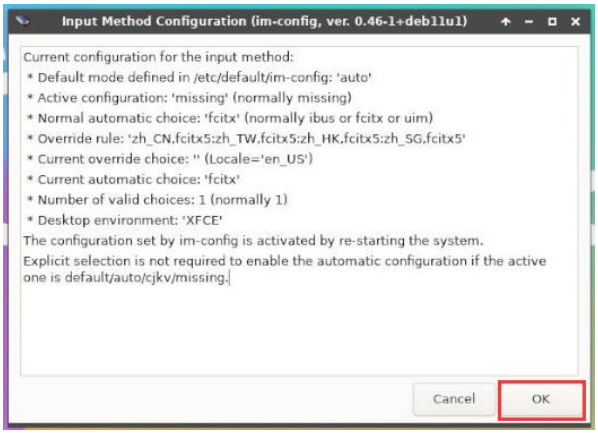
d. 退出界面后就会开始 **locale** 的设置，命令行显示的输出如下所示

```
orangepi@orangepi:~$ sudo dpkg-reconfigure locales
Generating locales (this might take a while)...
  en_US.UTF-8... done
  zh_CN.UTF-8... done
Generation complete.
```

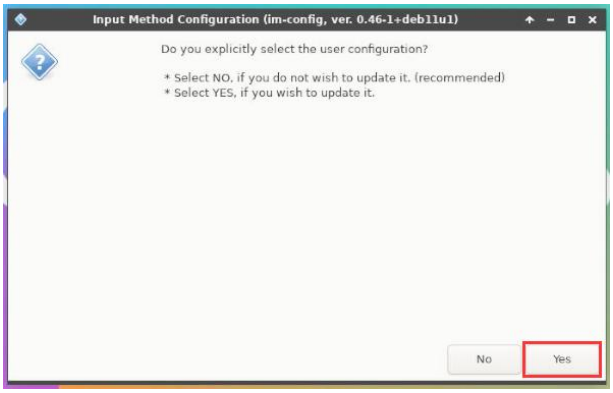
2) 然后打开 **Input Method**



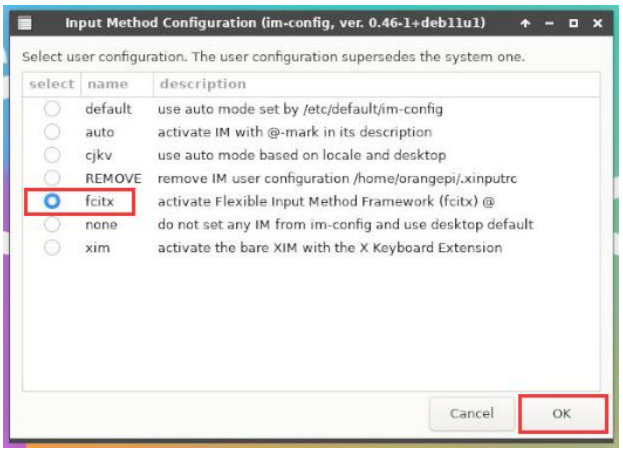
3) 然后选择 **OK**



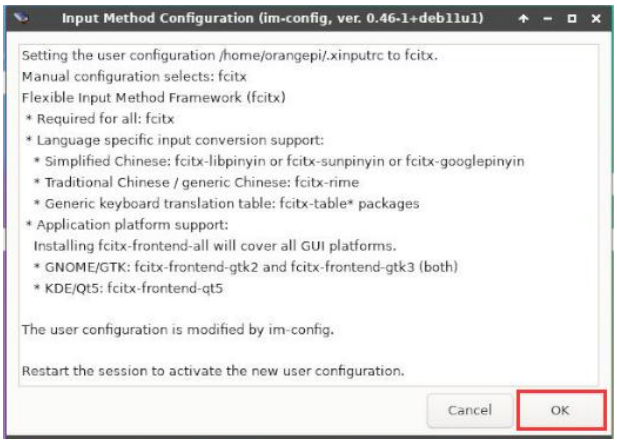
4) 然后选择 **Yes**



5) 然后选择 **fcitx**

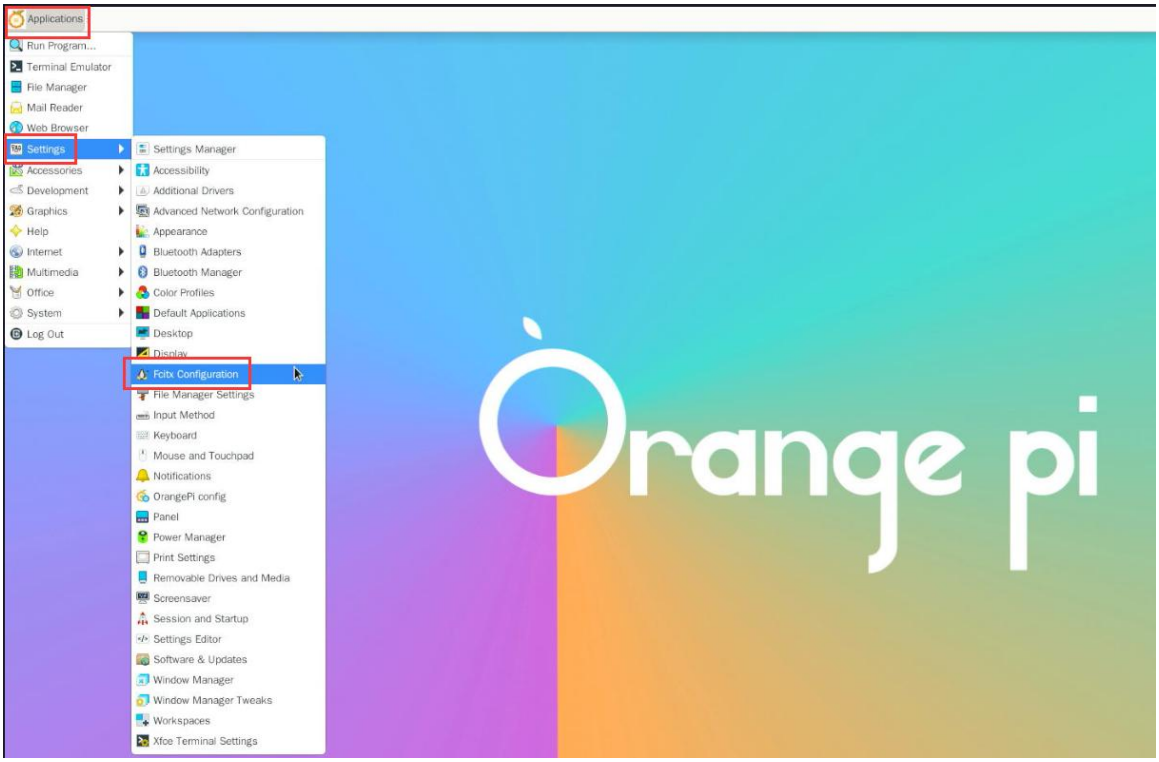


6) 然后选择 **OK**

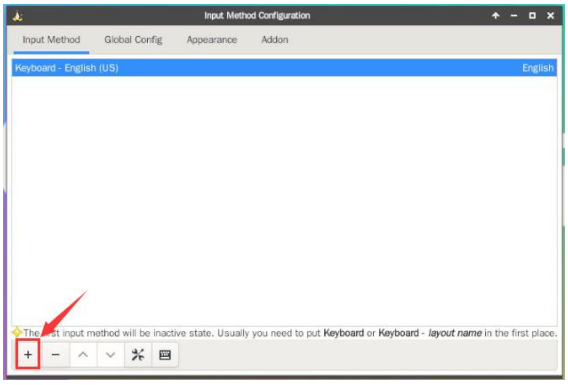


7) **然后重启 Linux 系统才能使配置生效**

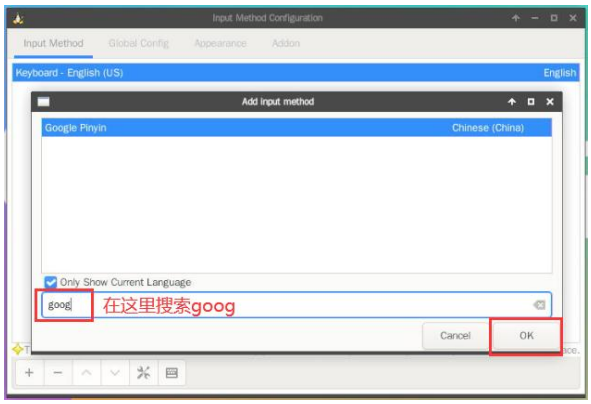
8) 然后打开 **Fcitx configuration**



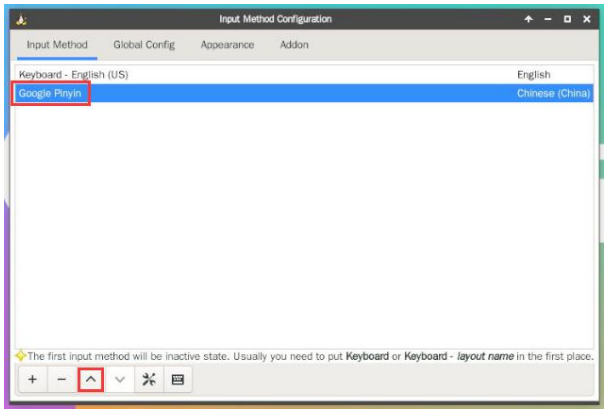
9) 然后点击下图所示位置的+号

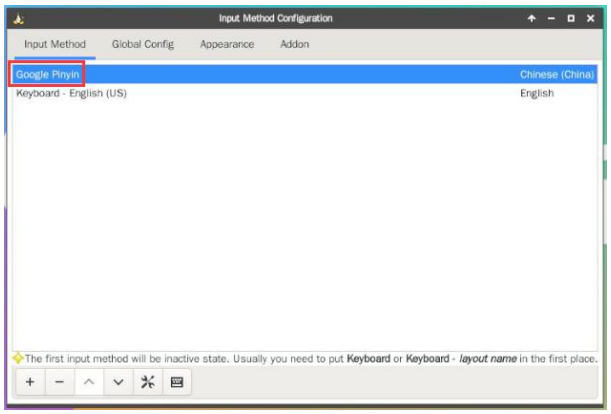


10) 然后搜索 **Google Pinyin** 再点击 **OK**



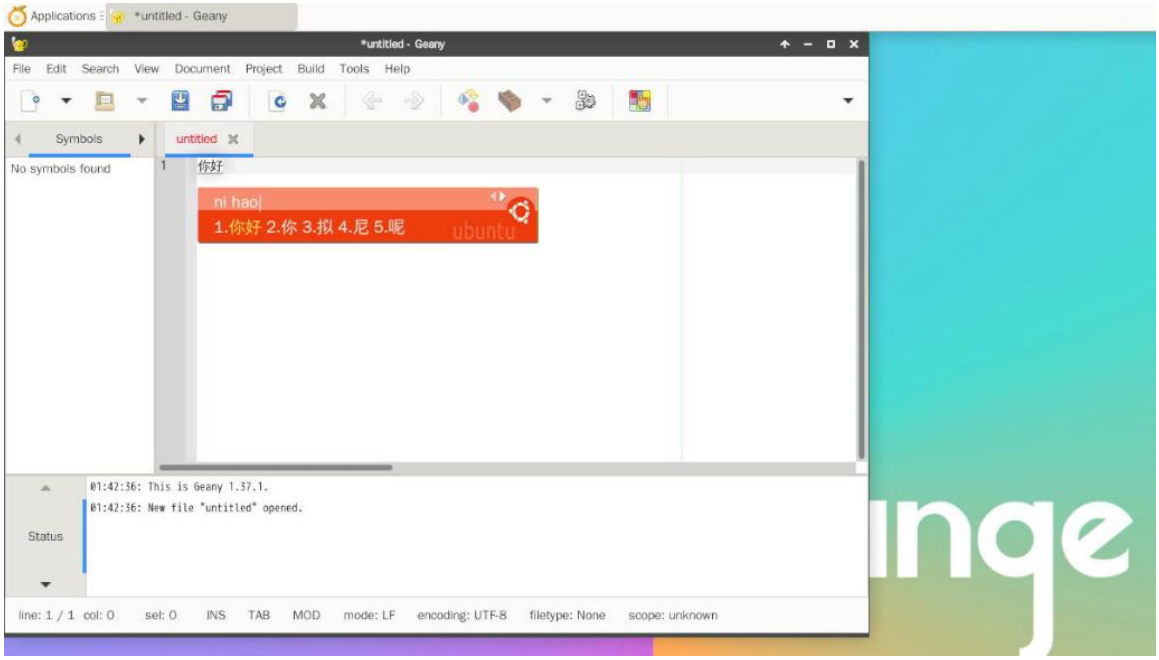
11) 然后将 **Google Pinyin** 放到最前面





12) 然后打开 **Geany** 这个编辑器测试下中文输入法

13) 中文输入法测试如下所示

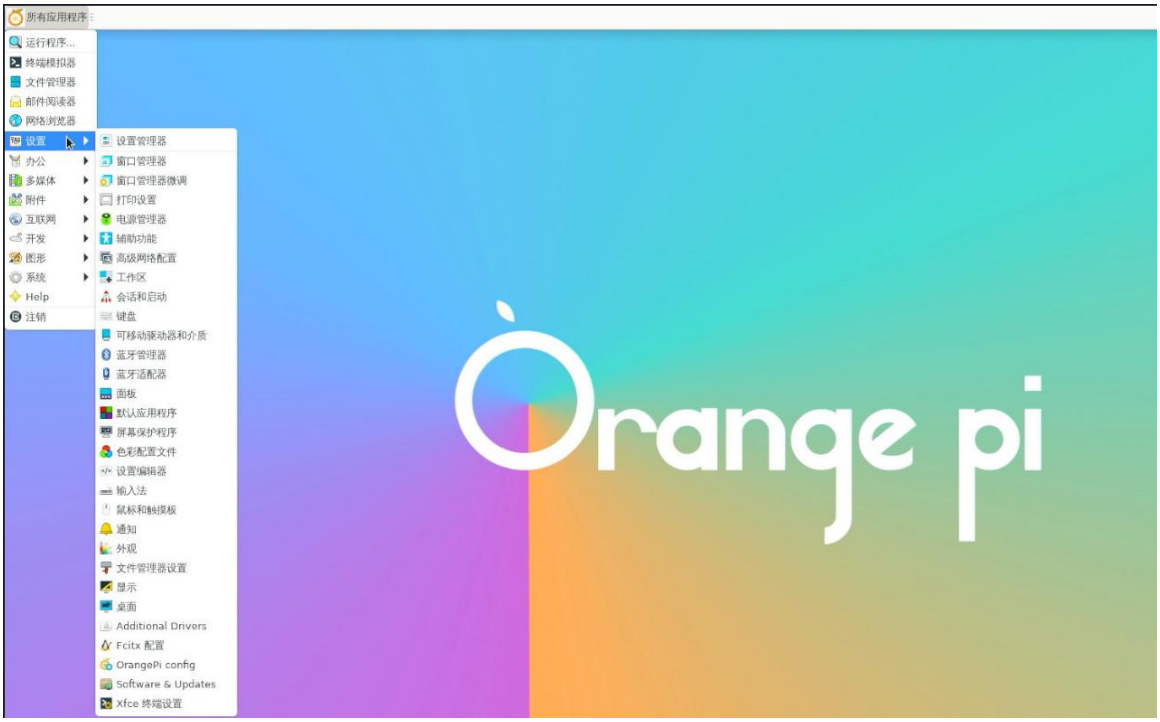


14) 通过 **Ctrl+Space** 快捷键可以切换中英文输入法

15) 如果需要整个系统都显示为中文，可以将**/etc/default/locale** 中的变量都设置为 **zh_CN.UTF-8**

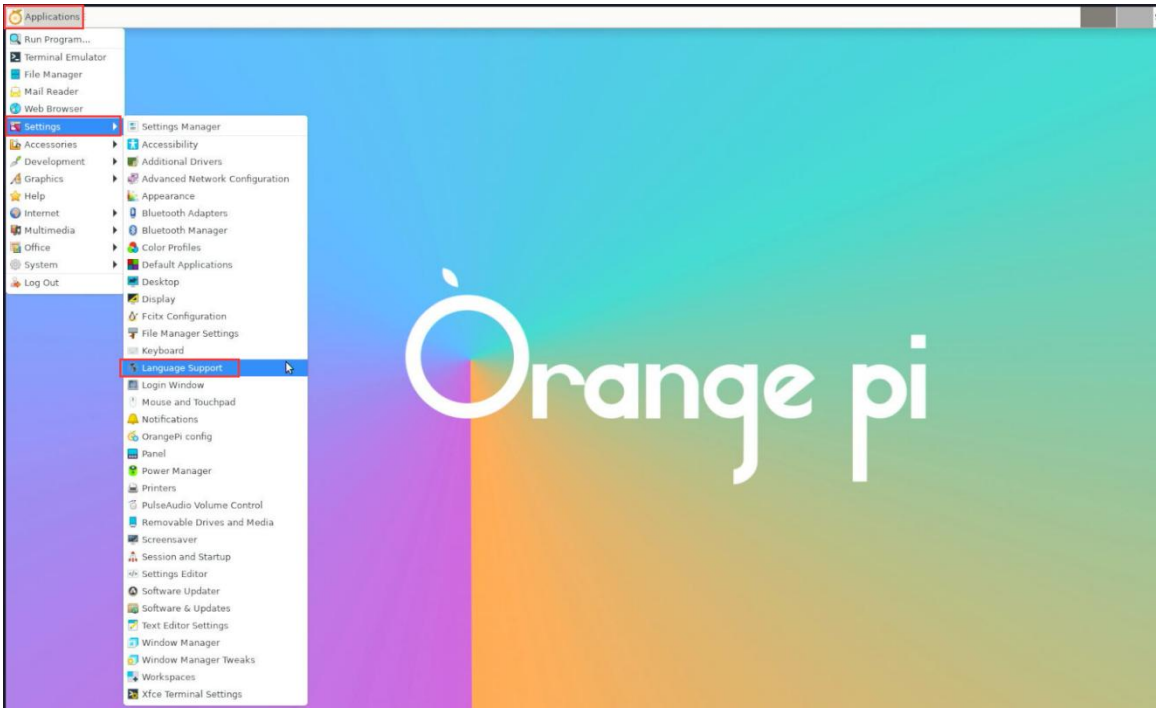
```
orangepi@orangepi:~$ sudo vim /etc/default/locale
#    File generated by update-locale
LC_MESSAGES=zh_CN.UTF-8
LANG=zh_CN.UTF-8
LANGUAGE=zh_CN.UTF-8
```

16) 然后**重启系统**就能看到系统显示为中文了

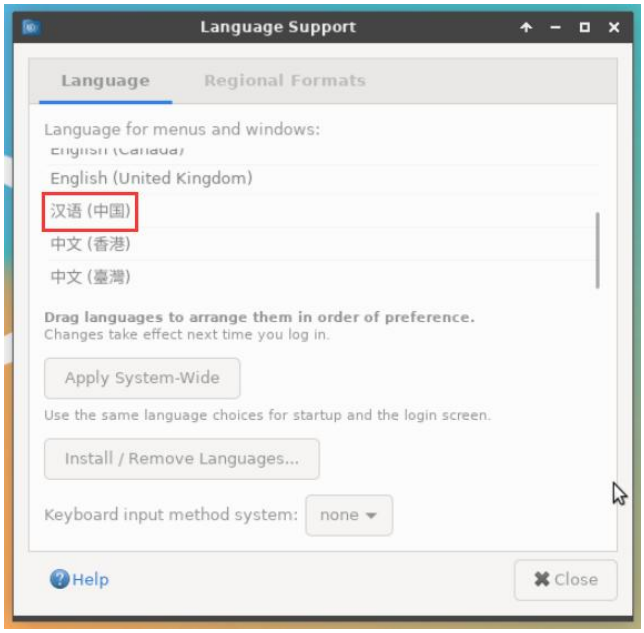


3. 24. 2. Ubuntu 20.04 系统的安装方法

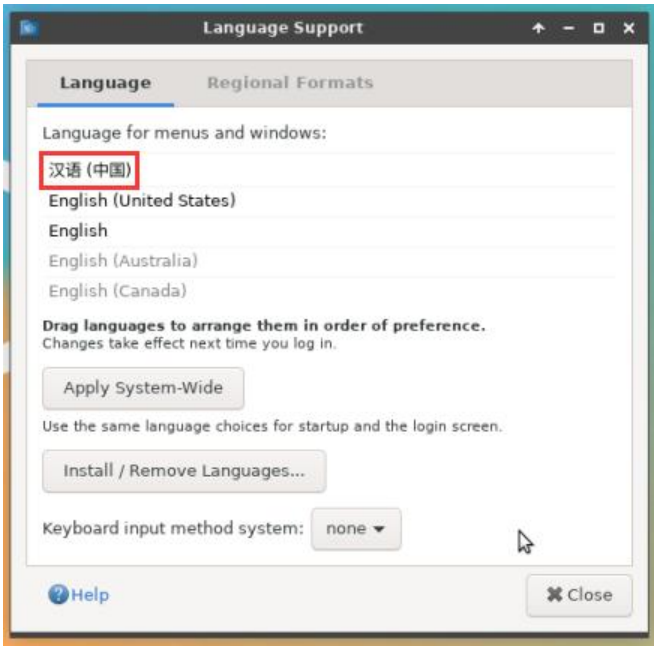
1) 首先打开 **Language Support**



2) 然后找到**汉语（中国）**选项

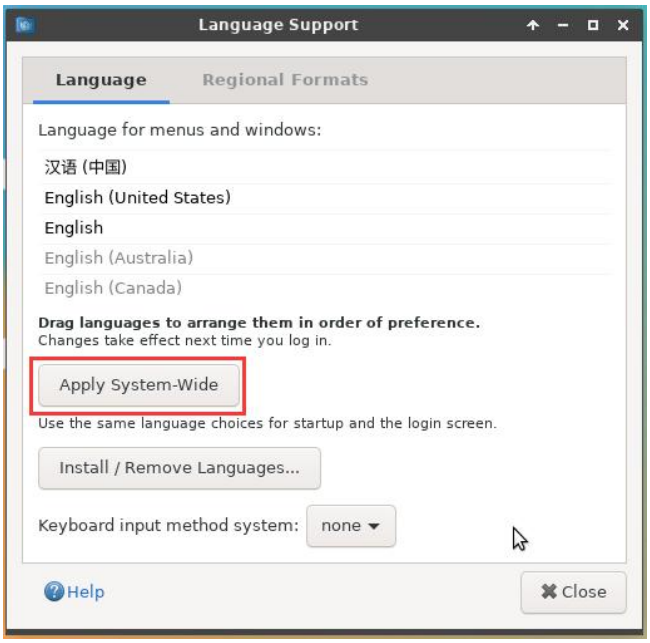


3) 然后请使用鼠标左键选中**汉语（中国）**并按住不动，然后往上将其拖到最开始的位置，拖完后的显示如下图所示：



注意，这一步不是很好拖动的，请耐心多试几次。

4) 然后选择 **Apply System-Wide** 将中文设置应用到整个系统

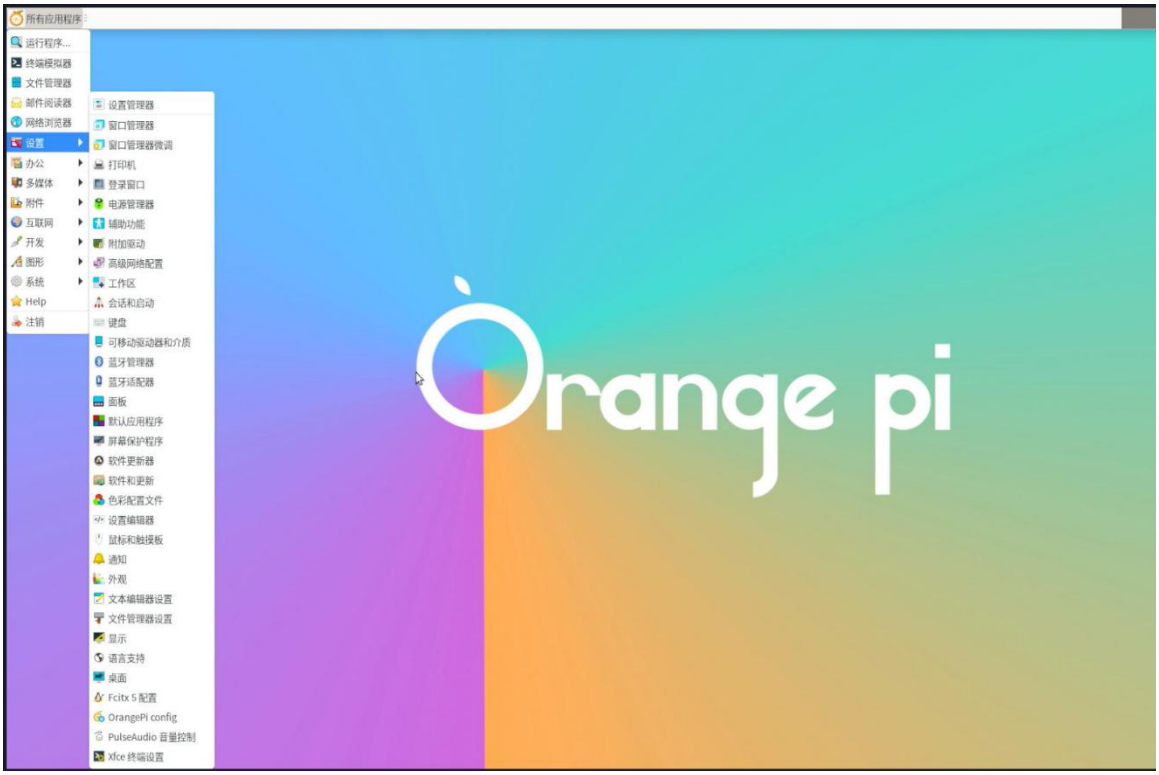


5) 然后设置 **Keyboard input method system** 为 **fcitx**

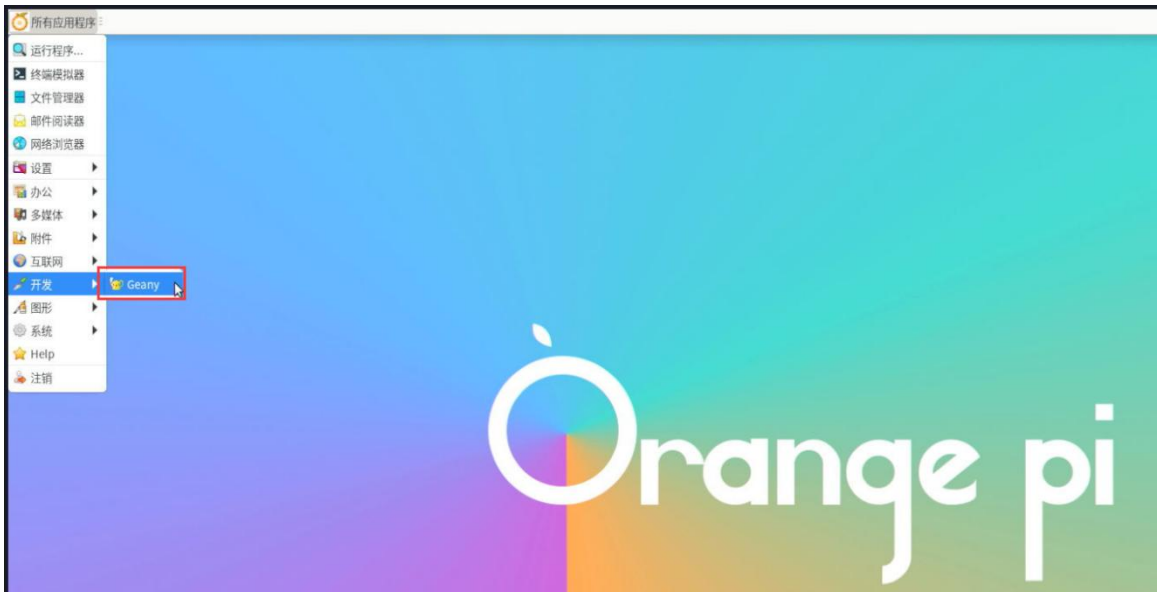
6) **然后重启 Linux 系统使配置生效**

7) 重新进入系统后，在下面的界面请选择**不要再次询问我**，然后请根据自己的喜好决定标准文件夹是否也要更新为中文

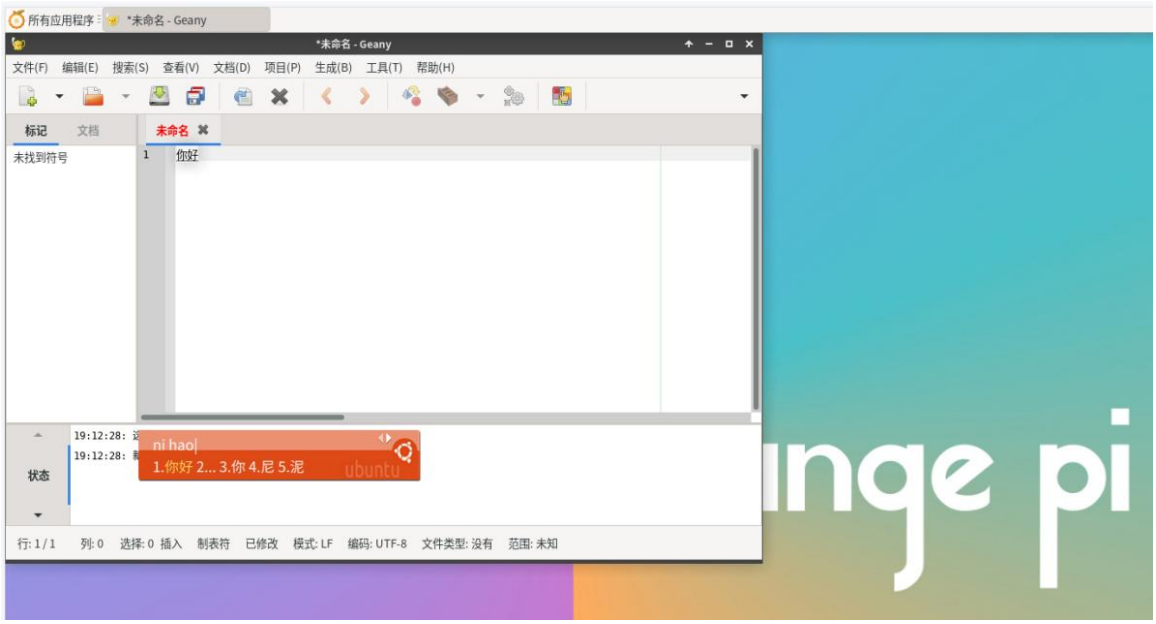
8) 然后可以看到桌面都显示为中文了



9) 然后我们可以打开 **Geany** 测试下中文输入法，打开方式如下图所示

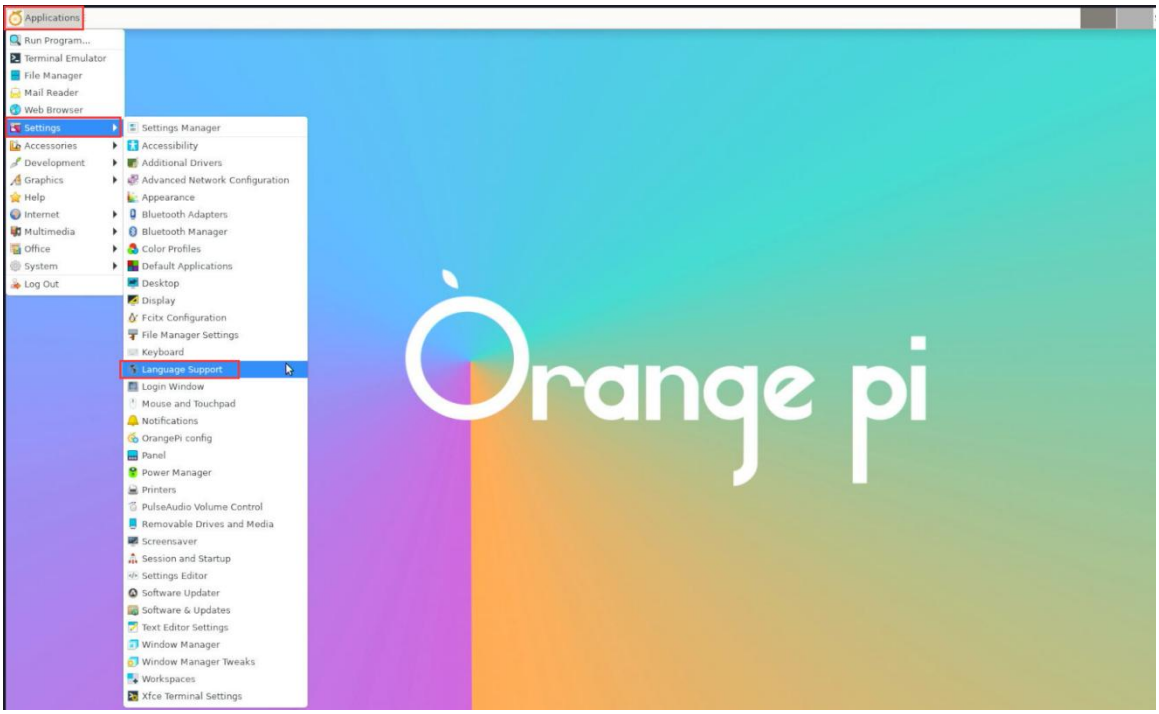


10) 打开 **Geany** 后，默认还是英文输入法，我们可以通过 **Ctrl+Space** 快捷键来切换成中文输入法，然后就能输入中文了

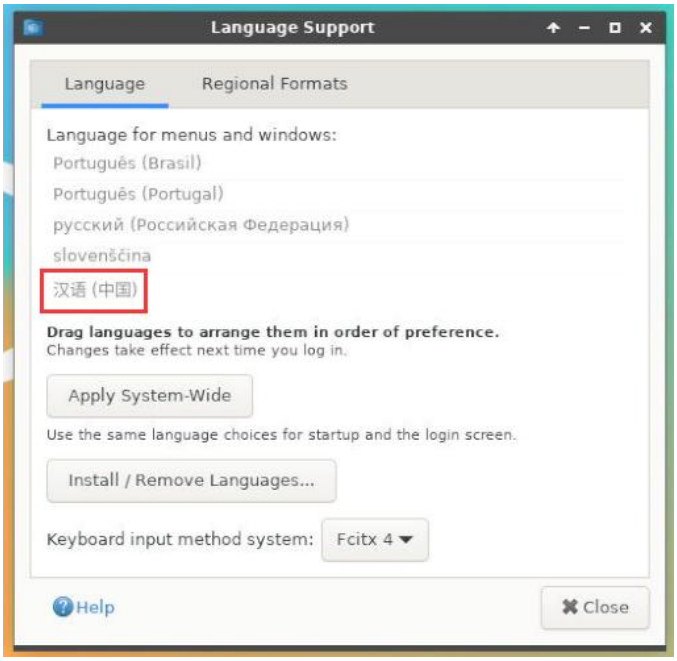


3. 24. 3. Ubuntu 22.04 系统的安装方法

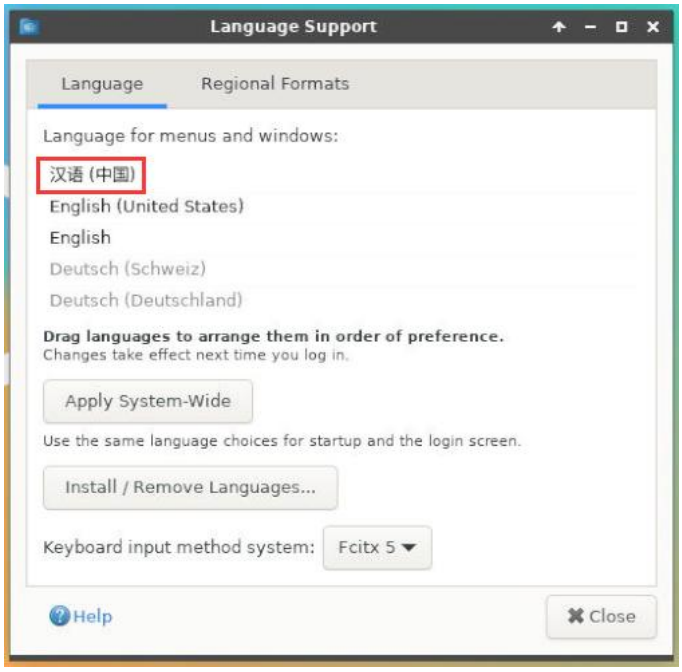
1) 首先打开 **Language Support**



2) 然后找到**汉语（中国）**选项

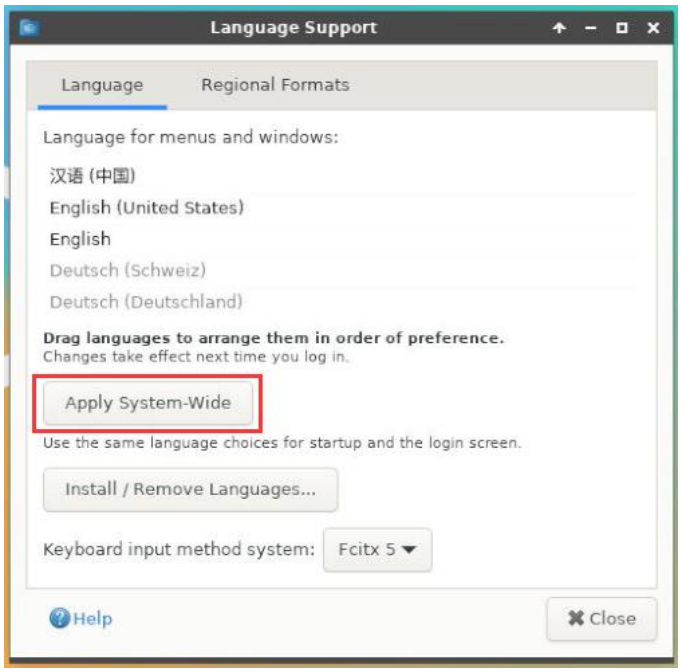


3) 然后请使用鼠标左键选中**汉语（中国）**并按住不动，然后往上将其拖到最开始的位置，拖完后的显示如下图所示：



注意，这一步不是很好拖动的，请耐心多试几次。

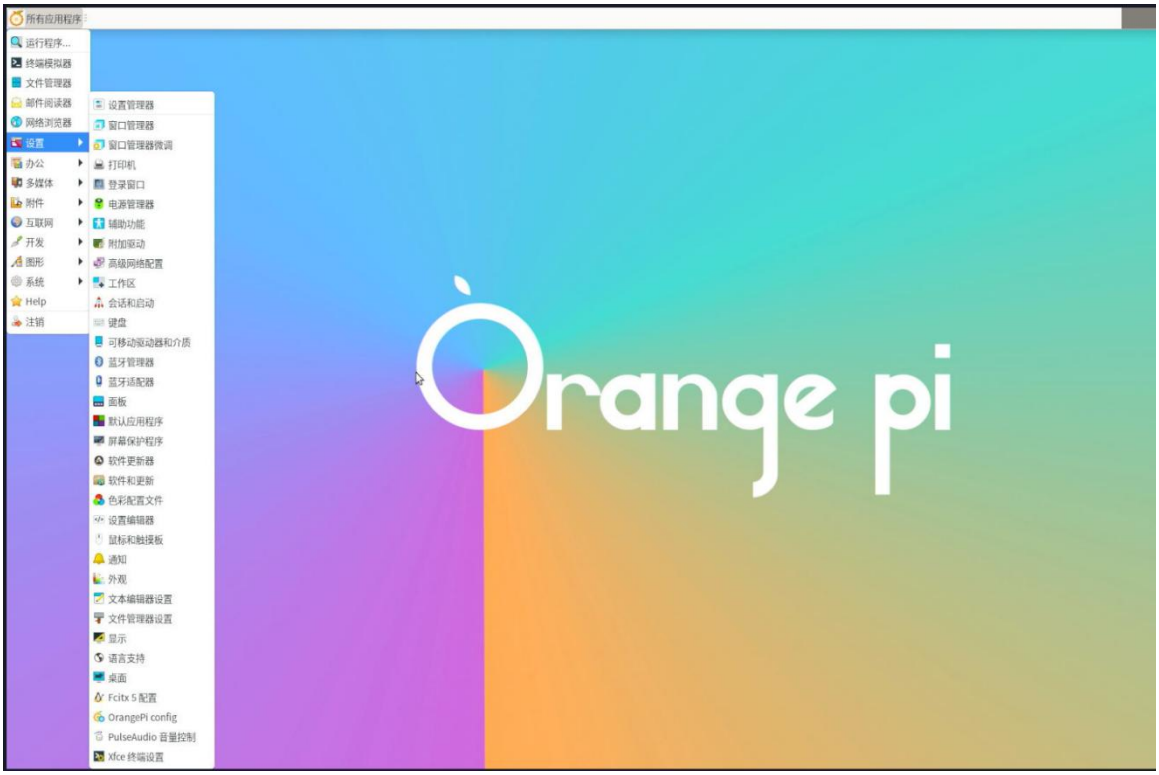
4) 然后选择 **Apply System-Wide** 将中文设置应用到整个系统



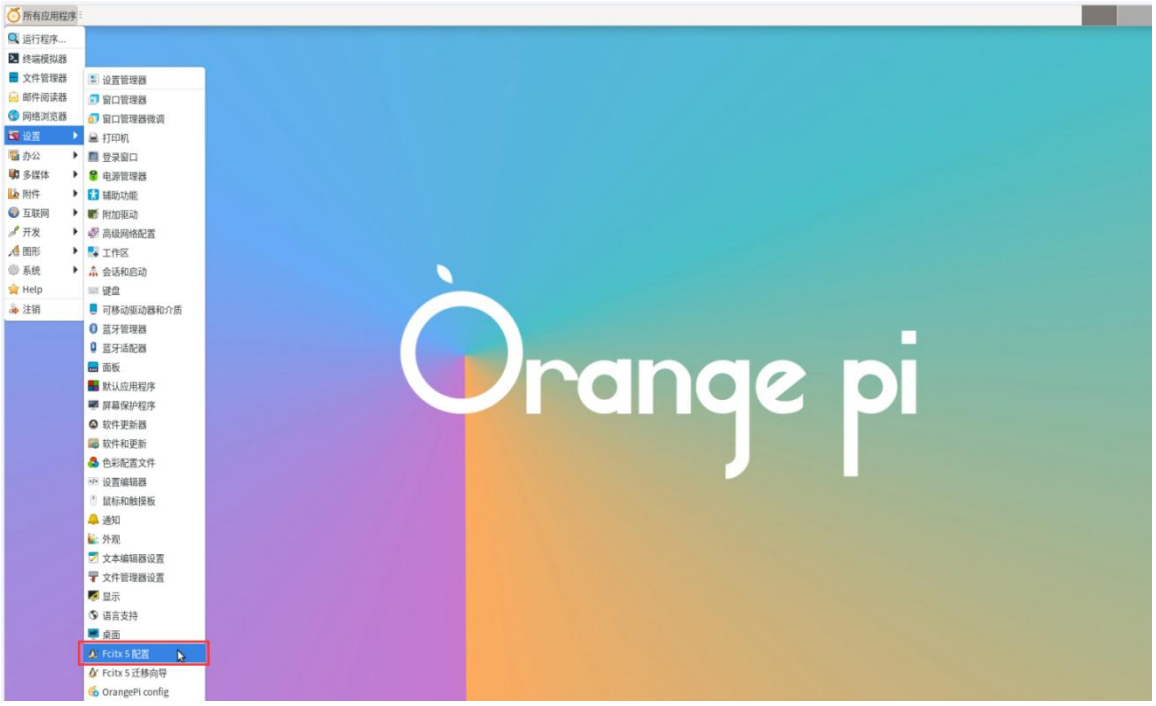
5) **然后重启 Linux 系统使配置生效**

6) 重新进入系统后，在下面的界面请选择**不要再次询问我**，然后请根据自己的喜好决定标准文件夹是否也要更新为中文

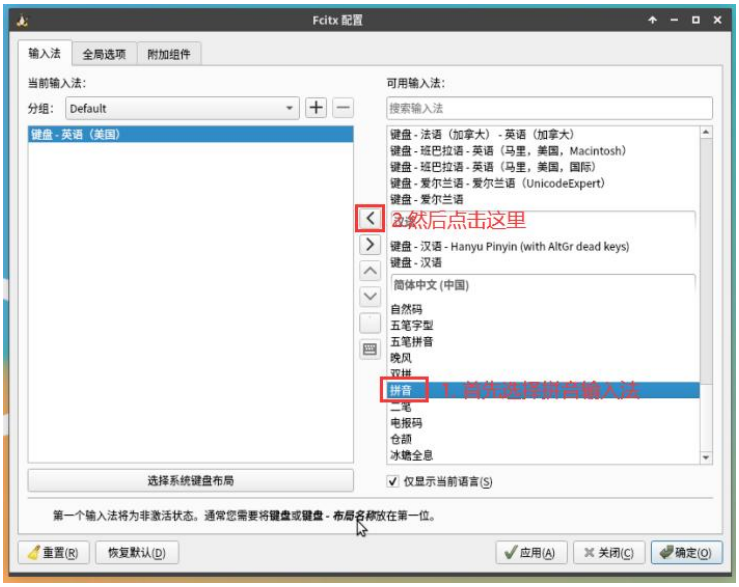
7) 然后可以看到桌面都显示为中文了



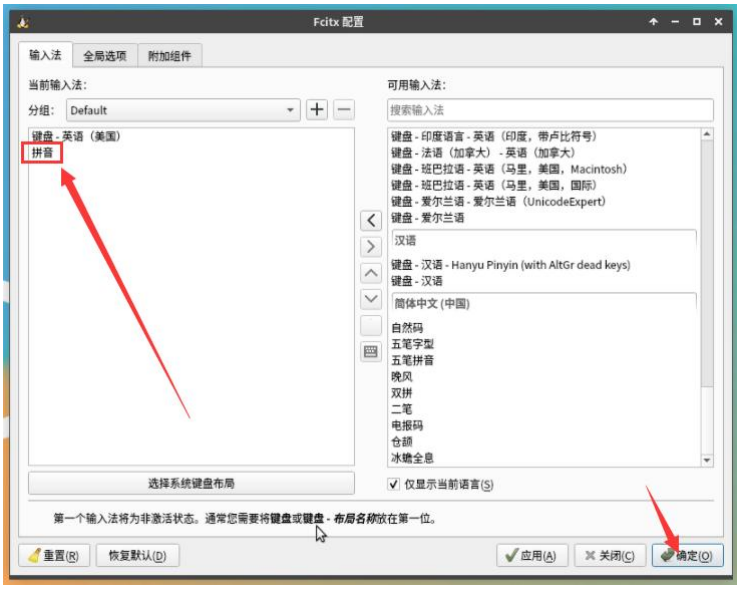
8) 然后打开 Fcitx5 配置程序



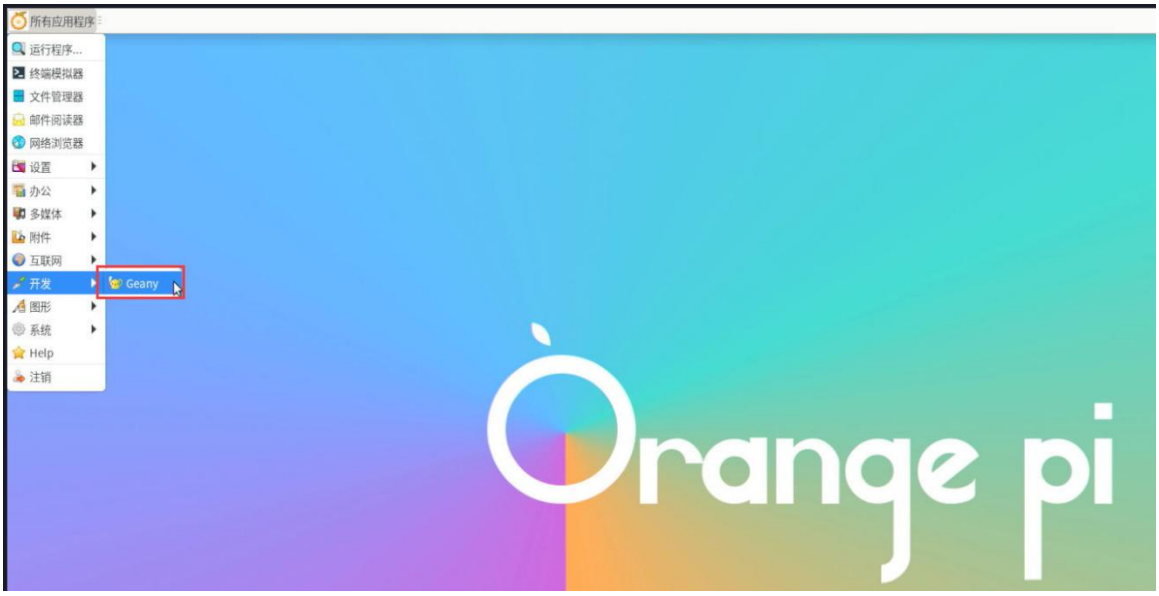
9) 然后选择使用拼音输入法



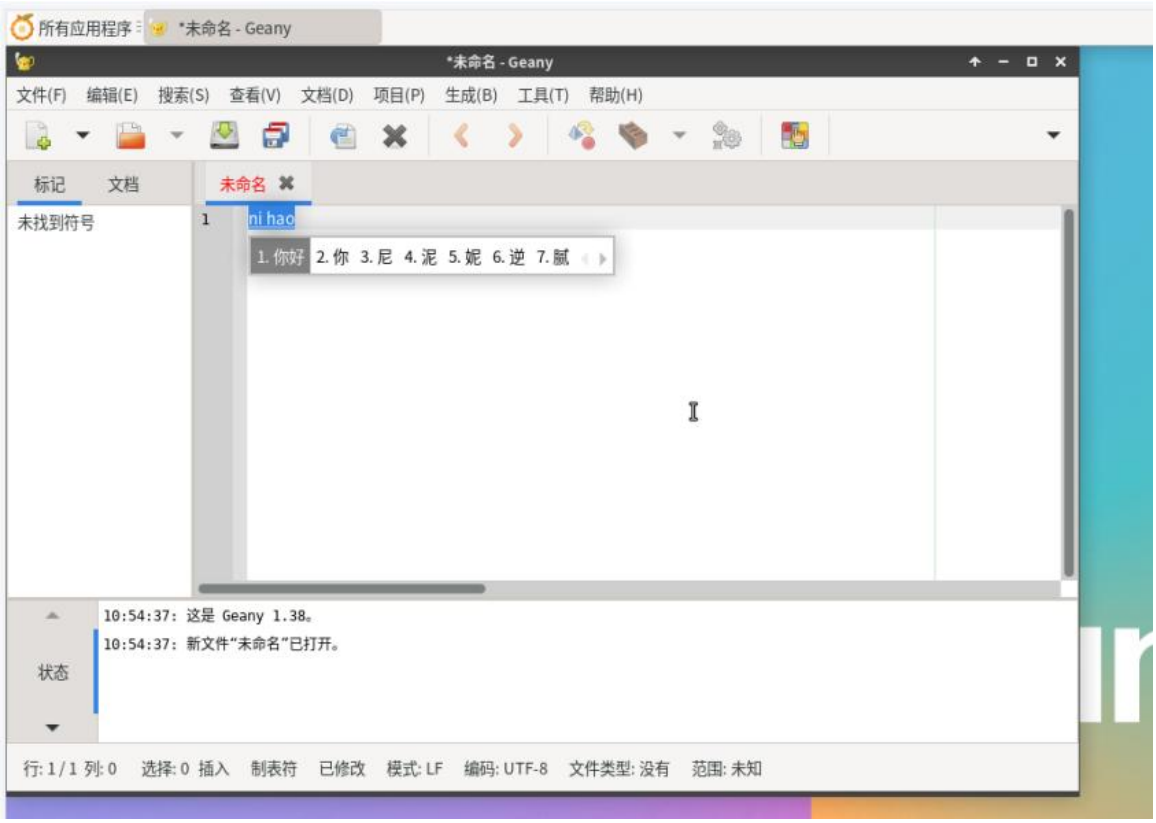
10) 选择后的界面如下所示，再点击确定即可



11) 然后我们可以打开 **Geany** 测试下中文输入法，打开方式如下图所示



12) 打开 **Geany** 后，默认还是英文输入法，我们可以通过 **Ctrl+Space** 快捷键来切换成中文输入法，然后就能输入中文了



3.25. 远程登录 Linux 系统桌面的方法

3.25.1. 使用 NoMachine 远程登录

请确保开发板安装的 Ubuntu 或者 Debian 系统为桌面版本的系统。另外 NoMachine 也提供了详细的使用文档，强烈建议通读此文档来熟悉 NoMachine 的使用，文档链接如下所示：

https://knowledgebase.nomachine.com/DT10R00166

NoMachine 支持 Windows、Mac、Linux、iOS 和安卓平台，所以我们可以在多种设备上通过 NoMachine 来远程登录控制 Orange Pi 开发板。下面演示下在 Windows 中通过 NoMachine 来远程登录 Orange Pi 开发板的 Linux 系统桌面。其他平台的安装方法请参考下 NoMachine 的官方文档。

操作前请先确保 Windwos 电脑和开发板在同一局域网内，并且能正常 ssh 登录

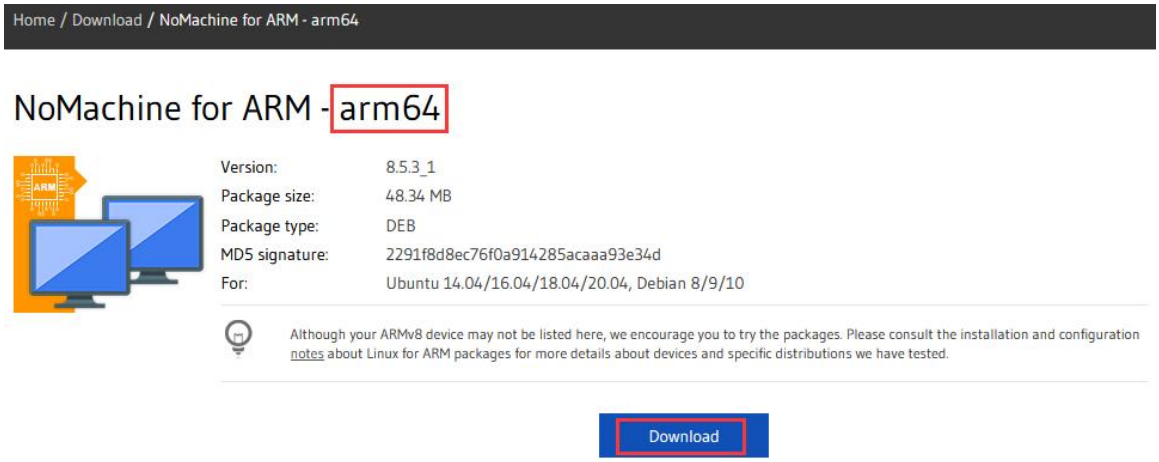
开发板的 Ubuntu 或者 Debian 系统。

1) 首先下载 NoMachine 软件 Linux **arm64** deb 版本的安装包，然后安装到开发板的 Linux 系统中

a. 由于 RK3566 是 ARMv8 架构的 SOC，我们使用的系统为 Ubuntu 或者 Debian，所以这里需要下载 **NoMachine for ARM ARMv8 DEB** 安装包，下载链接如下所示：

注意，这个下载链接可能会变，请认准 Armv8/Arm64 版本的 deb 包。

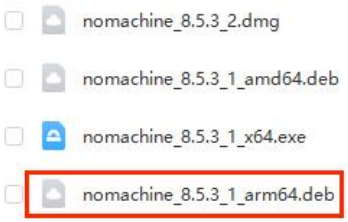
https://downloads.nomachine.com/download/?id=116&distro=ARM



b. 另外在**官方工具**中也可以下载到 **NoMachine** 的安装包

先进入**远程登录软件-NoMachine** 文件夹

然后下载 arm64 版本的 deb 安装包



c. 然后将下载的 **nomachine_x.x.x_x_arm64.deb** 上传到开发板的 Linux 系统中

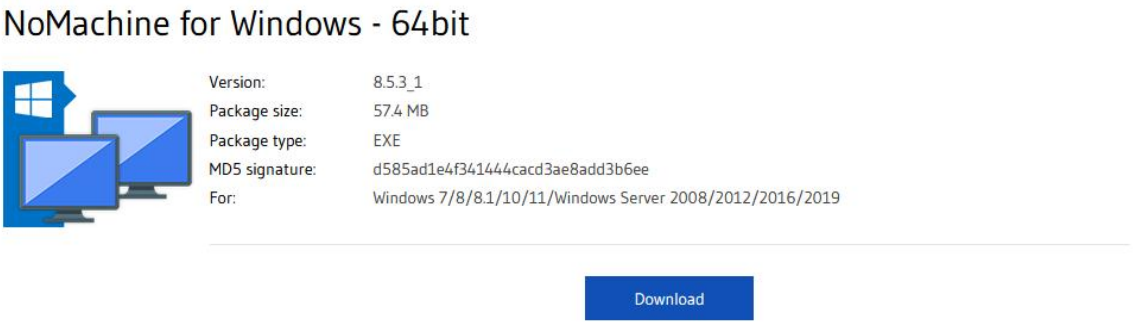
d. 然后使用下面的命令在开发板的 Linux 系统中安装 **NoMachine**

```
orangepi@orangepi:~$ sudo dpkg -i nomachine_x.x.x_x_arm64_arm64.deb
```

2) 然后下载 NoMachine 软件 Windows 版本的安装包，下载地址如下所示

注意，这个下载链接可能会变。

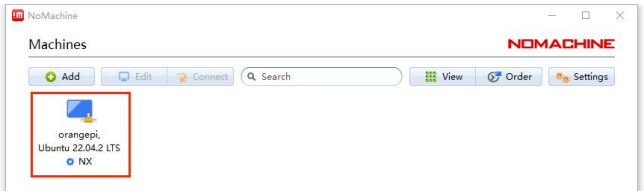
https://downloads.nomachine.com/download/?id=8



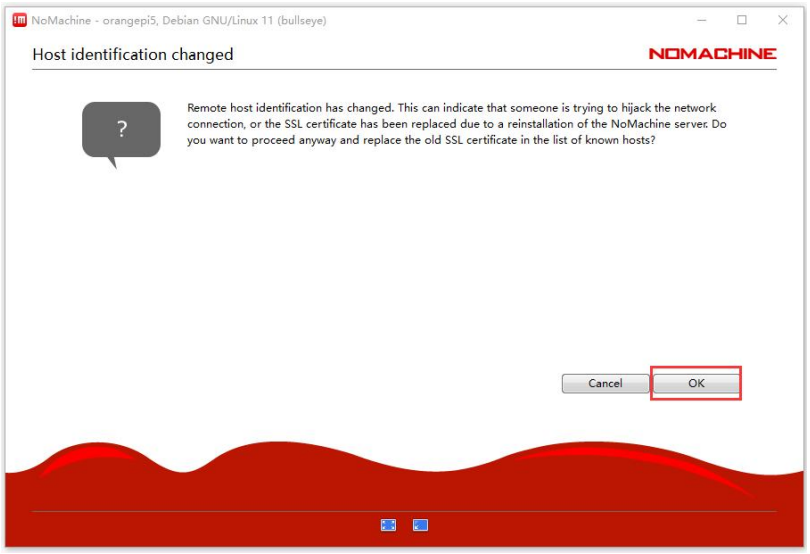
3) 然后在 Windows 中安装 NoMachine，**安装完后请重启下电脑**

4) 然后在 Window 中打开 **NoMachine**

5) NoMachine 启动后会自动扫描局域网内其他安装有 NoMachine 的设备，进入 NoMachine 的主界面后就可以看到开发板已经在可连接的设备列表里了，然后点击下图红色方框所示的位置即可开始登录开发板的 Linux 系统桌面



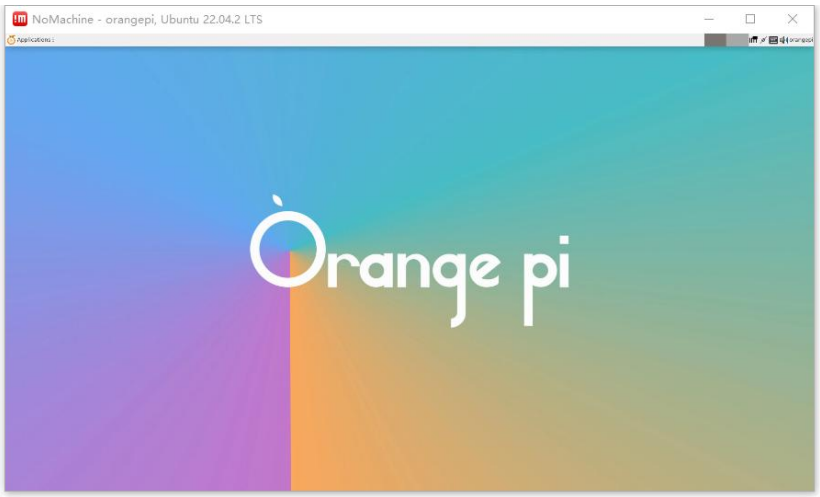
6) 然后点击 **OK**



7) 然后在下图对应的位置输入开发板 Linux 系统的用户名和密码，再点击 **OK** 开始登陆

8) 然后在接下来的界面中都点击 OK

9) 最后就能看到开发板 Linux 系统的桌面了



3.25.2. 使用 VNC 远程登录

操作前请先确保 Windows 电脑和开发板在同一局域网内，并且能正常 ssh 登录开发板的 Ubuntu 或者 Debian 系统。

Ubuntu20.04 测试 VNC 很多问题，请不要使用这种方法。

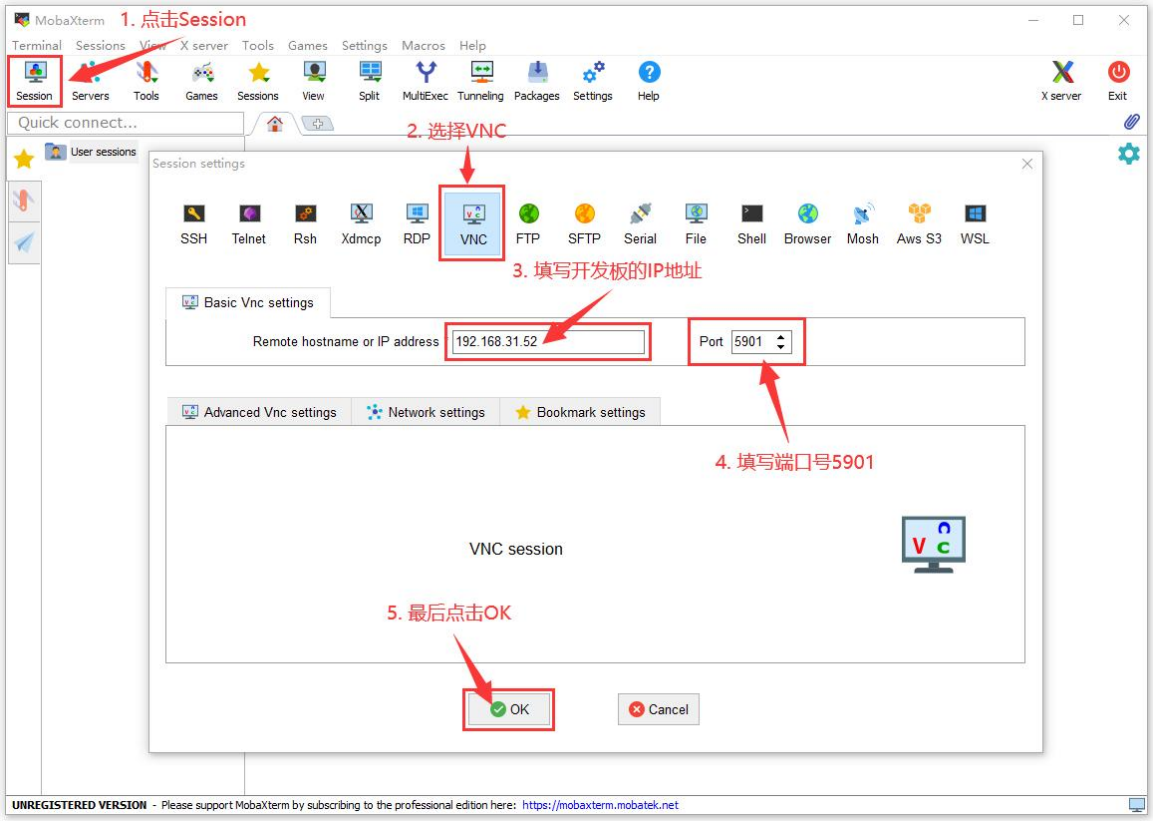
1) 首先运行 **set_vnc.sh** 脚本设置下 vnc，**记得加 sudo 权限**

```
orangepi@orangepi:~$ sudo set_vnc.sh
You will require a password to access your desktops.

Password:          #在这里设置 vnc 的密码，8 位字符
Verify:            #在这里设置 vnc 的密码，8 位字符
Would you like to enter a view-only password (y/n)? n
xauth:    file /root/.Xauthority does not exist

New 'X' desktop is orangepi3b:1

Creating default startup script /root/.vnc/xstartup
Starting applications specified in /root/.vnc/xstartup
Log file is /root/.vnc/orangepi3b:1.log

Killing Xtightvnc process ID 3047

New 'X' desktop is orangepi3b:1
```

```
Starting applications specified in /root/.vnc/xstartup
Log file is /root/.vnc/orangepi3b:1.log
```

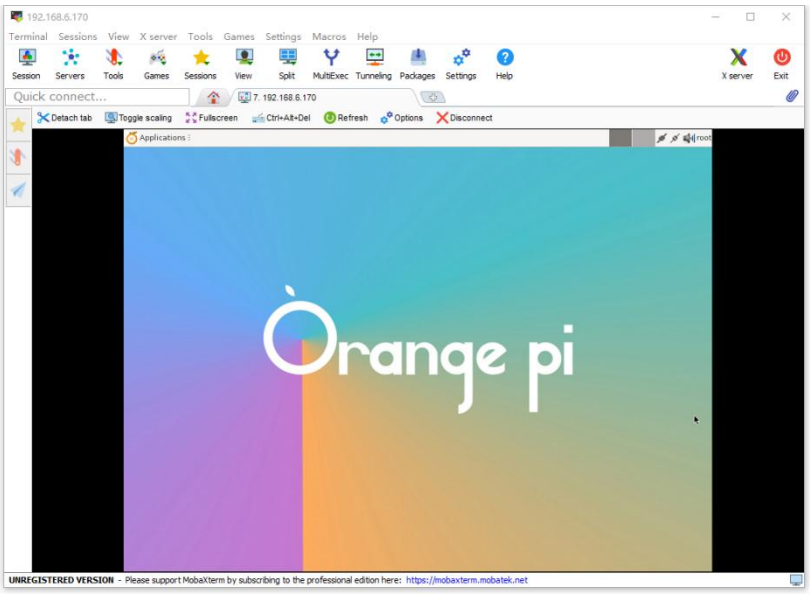
2) 使用 MobaXterm 软件连接开发板 Linux 系统桌面的步骤如下所示：

a. 首先点击 Session，然后选择 VNC，再填写开发板的 IP 地址和端口，最后点击 OK 确认



b. 然后输入前面设置的 VNC 的密码

c. 登录成功后的界面显示如下图所示，然后就可以远程操作开发板 Linux 系统的桌面了



3.26. Linux 系统支持的部分编程语言测试

3.26.1. Debian Bullseye 系统

1) Debian Bullseye 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
gcc (Debian 10.2.1-6) 10.2.1 20210110
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
        printf("Hello World!\n");
```

```
        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Debian Bullseye 默认安装有 Python3

a. Python 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Debian Bullseye 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk，Debian Bullseye 中最新版本为 openjdk-17

```
orangepi@orangepi:~$ sudo apt install -y openjdk-17-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.26.2. Debian Bookworm 系统

1) Debian Bookworm 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
gcc (Debian 12.2.0-14) 12.2.0
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
        printf("Hello World!\n");

        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Debian Bookworm 默认安装有 Python3

a. Python 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.11.2 (main, Mar 13 2023, 12:18:29) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

使用 Ctrl+D 快捷键可退出 python 的交互模式。

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Debian Bookworm 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk，Debian Bookworm 中最新版本为 openjdk-17

```
orangepi@orangepi:~$ sudo apt install -y openjdk-17-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.26.3. Ubuntu Focal 系统

1) Ubuntu Focal 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0
```

```
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
{
        printf("Hello World!\n");

        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Ubuntu Focal 默认安装有 Python3

a. Python3 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.8.10 (default, Nov 14 2022, 12:59:47)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Ubuntu Focal 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk-17

```
orangepi@orangepi:~$ sudo apt install -y openjdk-17-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
openjdk 17.0.2 2022-01-18
OpenJDK Runtime Environment (build 17.0.2+8-Ubuntu-120.04)
OpenJDK 64-Bit Server VM (build 17.0.2+8-Ubuntu-120.04, mixed mode, sharing)
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.26.4. Ubuntu Jammy 系统

1) Ubuntu Jammy 默认安装有 gcc 编译工具链，可以直接在开发板的 Linux 系统中编译 C 语言的程序

a. gcc 的版本如下所示

```
orangepi@orangepi:~$ gcc --version
gcc (Ubuntu 11.2.0-19ubuntu1) 11.2.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

b. 编写 C 语言的 **hello_world.c** 程序

```
orangepi@orangepi:~$ vim hello_world.c
#include <stdio.h>

int main(void)
```

```
{
        printf("Hello World!\n");

        return 0;
}
```

c. 然后编译运行 **hello_world.c**

```
orangepi@orangepi:~$ gcc -o hello_world hello_world.c
orangepi@orangepi:~$ ./hello_world
Hello World!
```

2) Ubuntu Jammy 默认安装有 Python3

a. Python3 具体版本如下所示

```
orangepi@orangepi:~$ python3
Python 3.10.4 (main, Apr  2 2022, 09:04:19) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

b. 编写 Python 语言的 **hello_world.py** 程序

```
orangepi@orangepi:~$ vim hello_world.py
print('Hello World!')
```

c. 运行 **hello_world.py** 的结果如下所示

```
orangepi@orangepi:~$ python3 hello_world.py
Hello World!
```

3) Ubuntu Jammy 默认没有安装 Java 的编译工具和运行环境

a. 可以使用下面的命令安装 openjdk-18

```
orangepi@orangepi:~$ sudo apt install -y openjdk-18-jdk
```

b. 安装完后可以查看下 Java 的版本

```
orangepi@orangepi:~$ java --version
openjdk 18-ea 2022-03-22
OpenJDK Runtime Environment (build 18-ea+36-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 18-ea+36-Ubuntu-1, mixed mode, sharing)
```

c. 编写 Java 版本的 **hello_world.java**

```
orangepi@orangepi:~$ vim hello_world.java
public class hello_world
{
```

```
        public static void main(String[] args)
        {
                System.out.println("Hello World!");
        }
}
```

d. 然后编译运行 **hello_world.java**

```
orangepi@orangepi:~$ javac hello_world.java
orangepi@orangepi:~$ java hello_world
Hello World!
```

3.27. QT 的安装方法

1) 使用下面的脚本可以安装 QT5 和 QT Creator

```
orangepi@orangepi:~$ install_qt.sh
```

2) 安装完后会自动打印 QT 的版本号

a. Ubuntu20.04 自带的 qt 版本为 **5.12.8**

```
orangepi@orangepi:~$ install_qt.sh
......
QMake version 3.1
Using Qt version 5.12.8 in /usr/lib/aarch64-linux-gnu
```

b. Ubuntu22.04 自带的 QT 版本为 **5.15.3**

```
orangepi@orangepi:~$ install_qt.sh
......
QMake version 3.1
Using Qt version 5.15.3 in /usr/lib/aarch64-linux-gnu
```

c. Debian11 自带的 QT 版本为 **5.15.2**

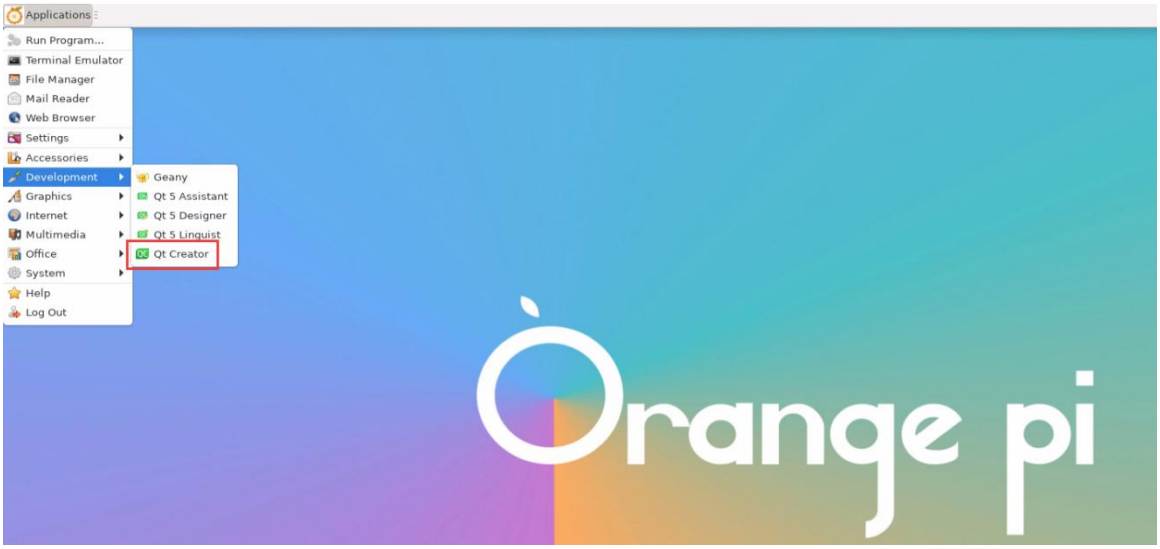
```
orangepi@orangepi:~$ install_qt.sh
......
QMake version 3.1
Using Qt version 5.15.2 in /usr/lib/aarch64-linux-gnu
```

d. Debian12 自带的 QT 版本为 **5.15.8**

```
orangepi@orangepi:~$ install_qt.sh
......
QMake version 3.1
```

```
Using Qt version 5.15.8 in /usr/lib/aarch64-linux-gnu
```

3) 然后在 **Applications** 中就可以看到 QT Creator 的启动图标



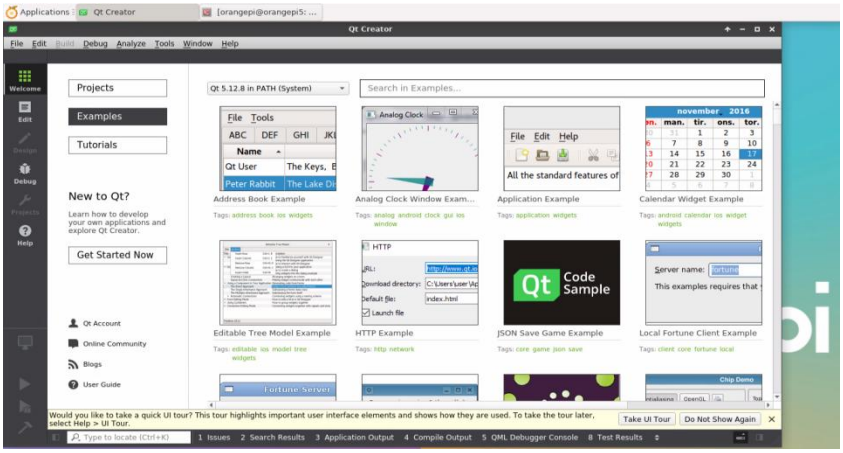
也可以使用下面的命令打开 QT Creator

```
orangepi@orangepi:~$ qtcreator
```

在QT和QT应用的启动过程中，如果提示下面的错误，请直接忽略，此错误对应用的运行不会有影响。

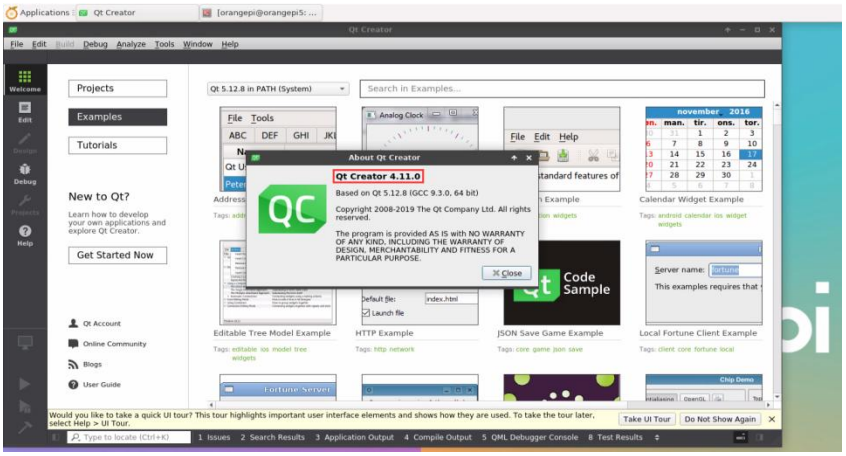
libGL error: failed to create dri screen
libGL error: failed to load driver: rockchip
libGL error: failed to create dri screen
libGL error: failed to load driver: rockchip

4) QT Creator 打开后的界面如下所示

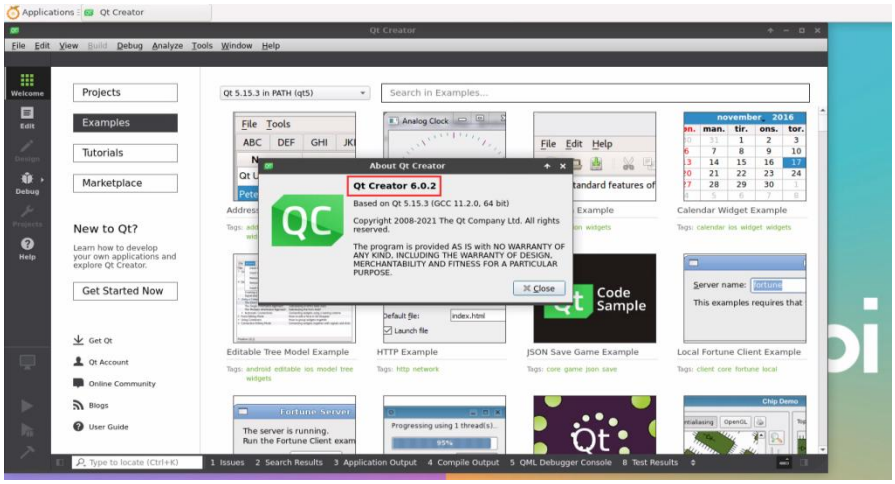


5) QT Creator 的版本如下所示

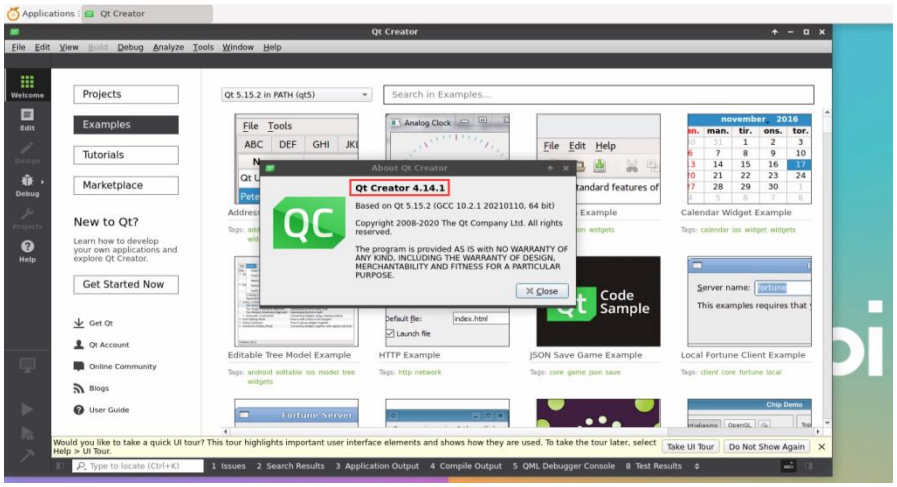
a. QT Creator 在 **Ubuntu20.04** 中的默认版本如下所示



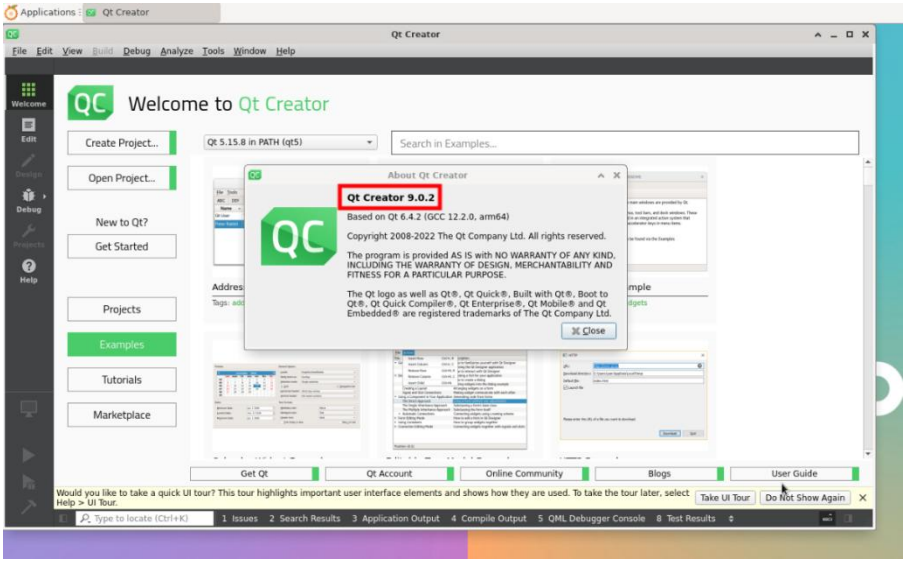
b. QT Creator 在 **Ubuntu22.04** 中的默认版本如下所示



c. QT Creator 在 **Debian11** 中的默认版本如下所示

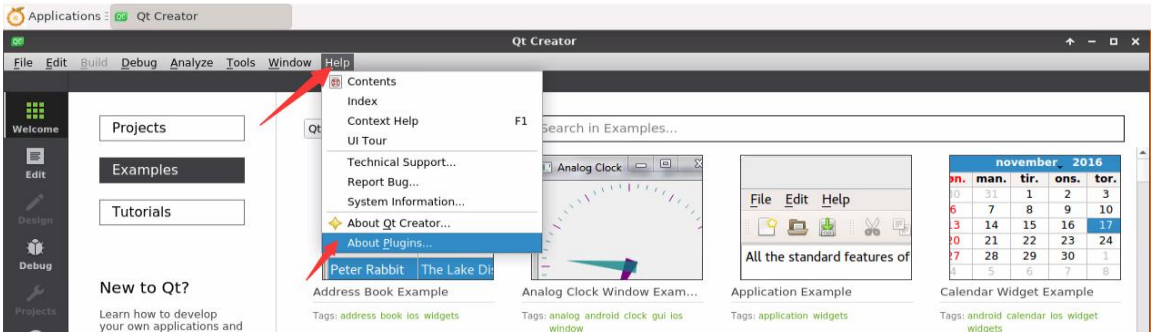


d. QT Creator 在 **Debian12** 中的默认版本如下所示

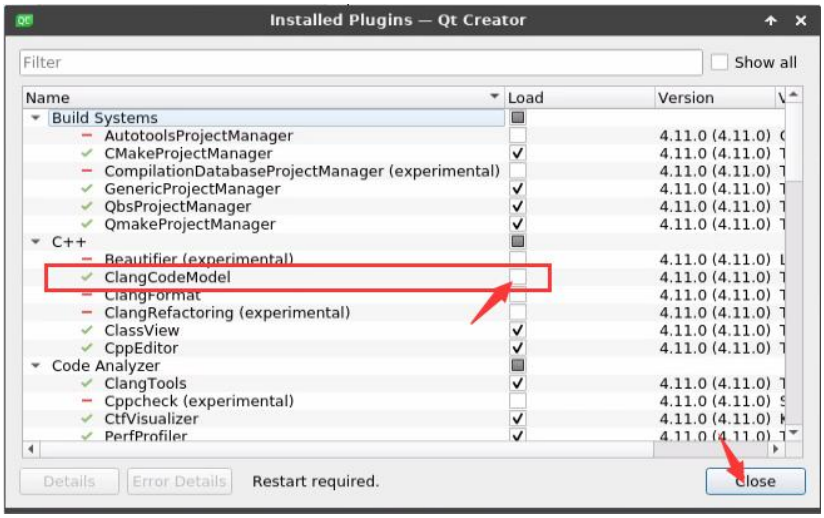


6) 然后设置下 QT

a. 首先打开 **Help->About Plugins...**



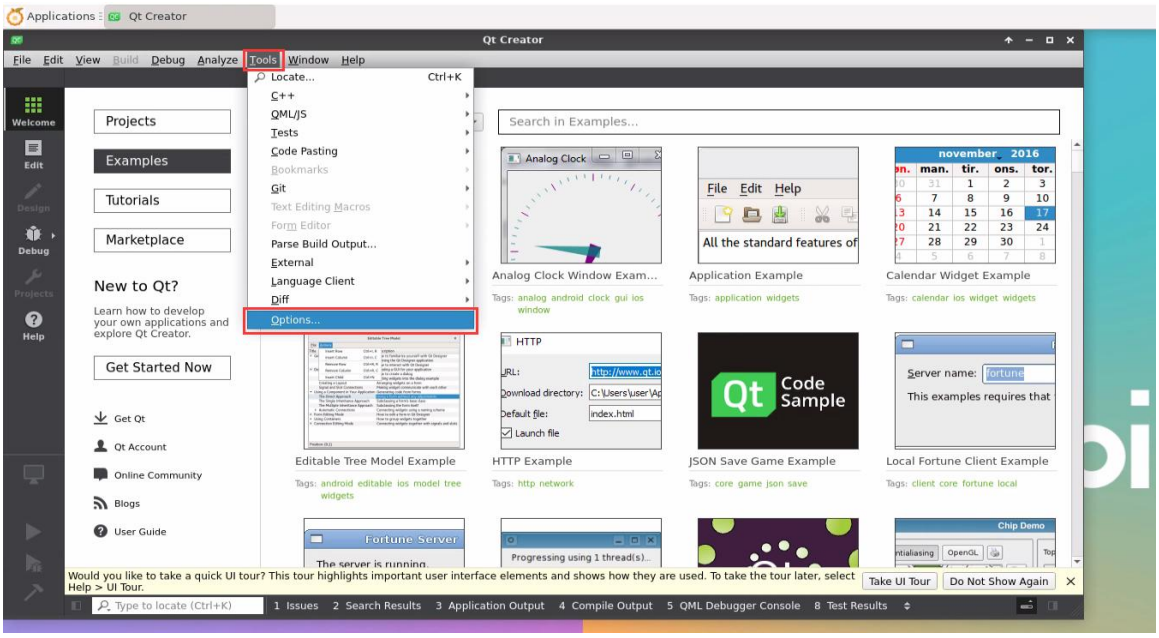
b. 然后去掉 **ClangCodeModel** 的那个勾

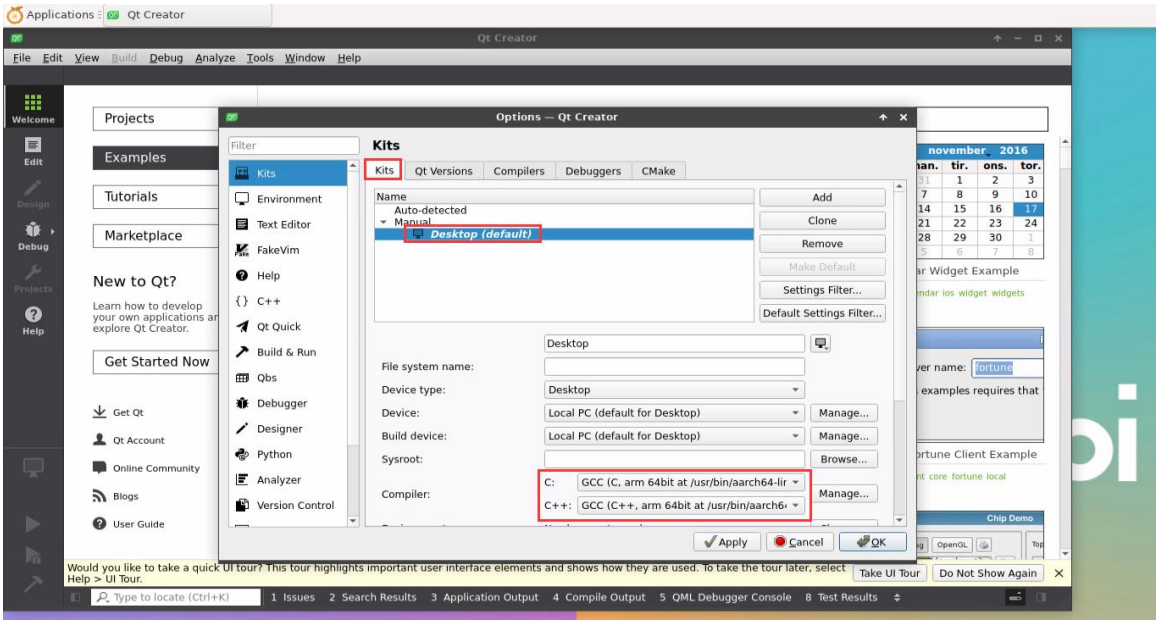


c. **设置完后需要重启下 QT Creator**

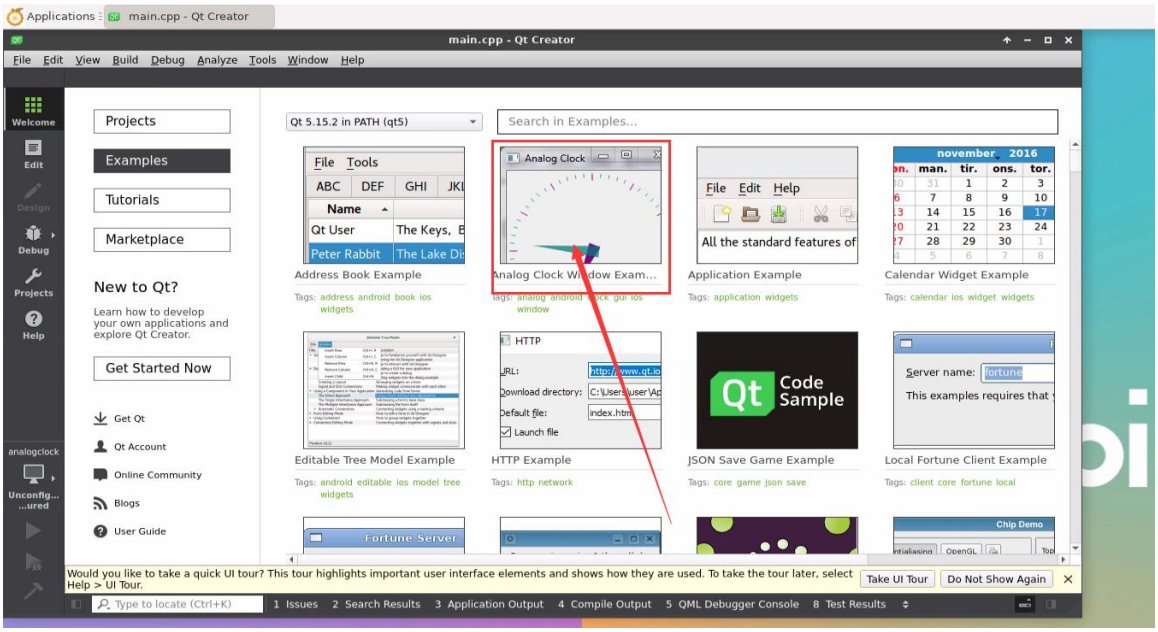
d. 然后确保 QT Creator 使用的 GCC 编译器，如果默认为 Clang，请修改为 GCC

Debian12 请跳过这步。

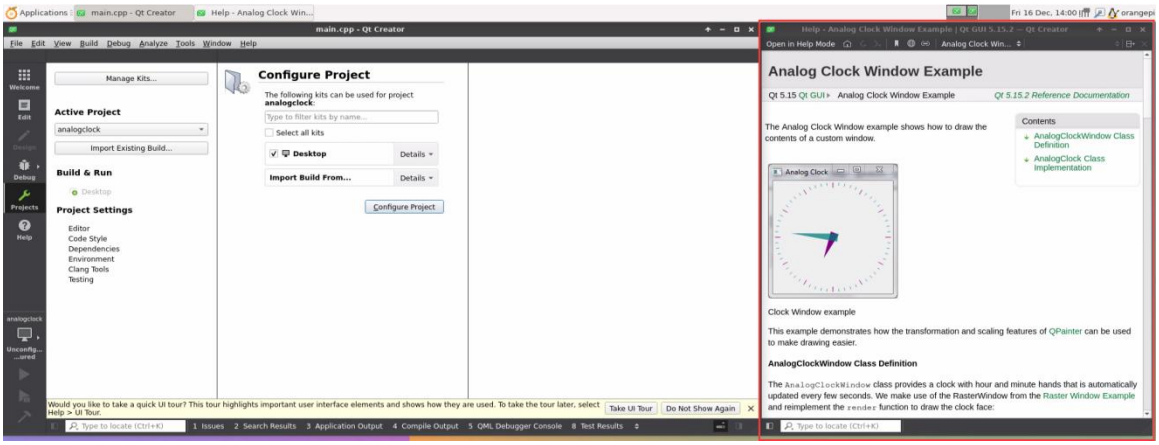




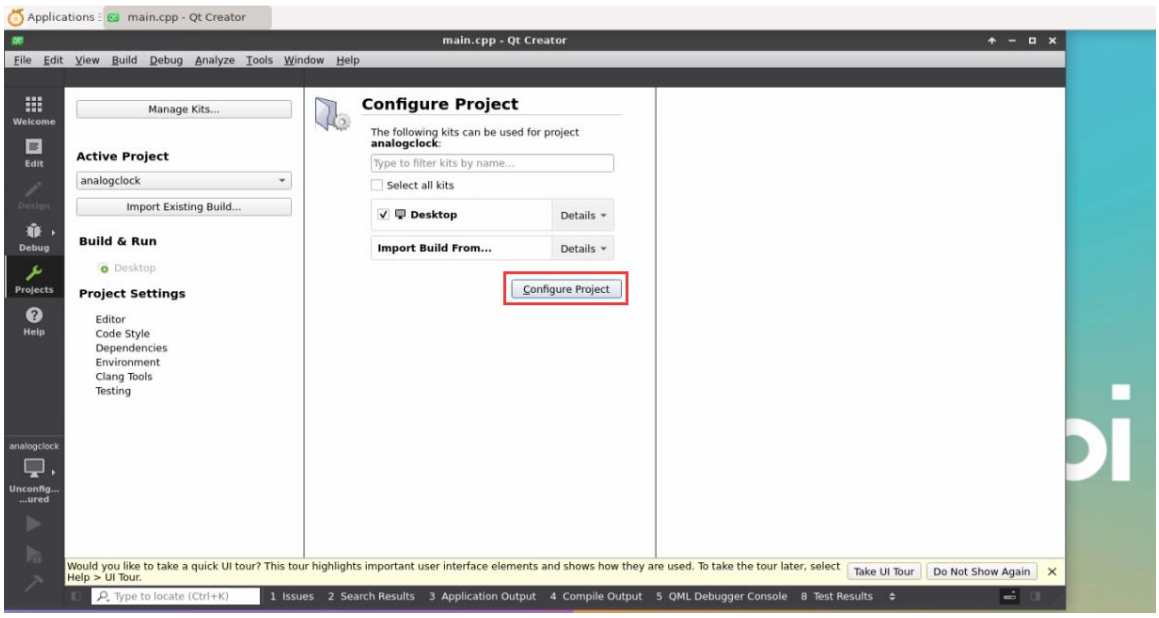
7) 然后就可以打开一个示例代码



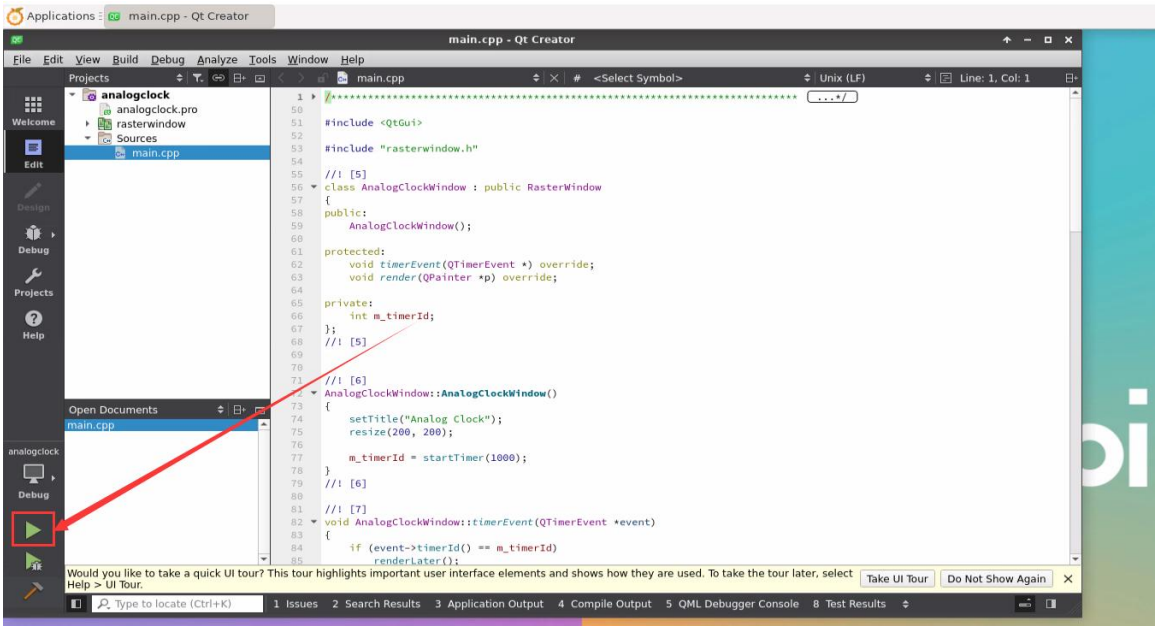
8) 点击示例代码后会自动打开对应的说明文档，可以仔细看下其中的使用说明



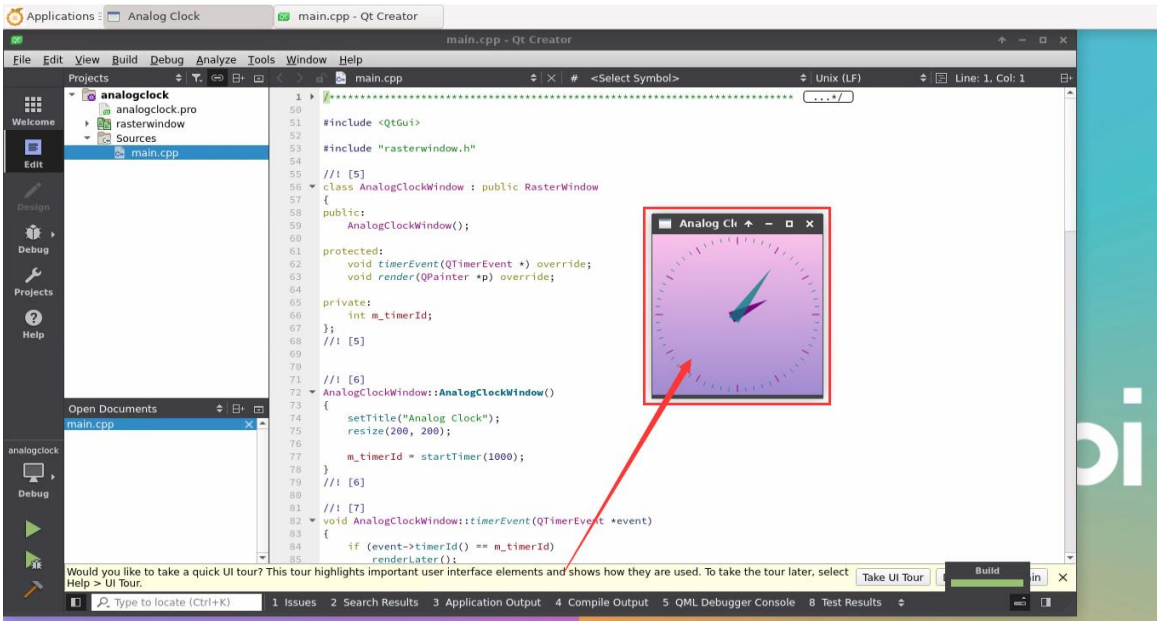
9) 然后点击下 **Configure Project**



10) 然后点击左下角的绿色三角形编译运行下示例代码



11) 等待一段时间后，会弹出下图所示的界面，此时就说明 QT 能正常编译运行



12) 参考资料

https://wiki.qt.io/Install_Qt_5_on_Ubuntu
https://download.qt.io/archive/qtcreator
https://download.qt.io/archive/qt

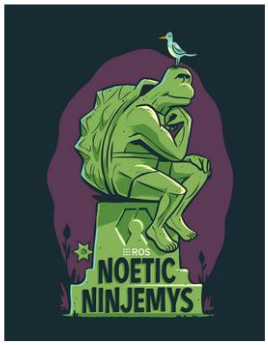
3. 28. ROS 安装方法

3. 28. 1. Ubuntu20.04 安装 ROS 1 Noetic 的方法

1) ROS 1 当前活跃的版本如下所示，推荐版本为 **Noetic Ninjemys**

Active ROS 1 distributions

Recommended



Distro	Release date	Poster	*Tuturtle*, turtle in tutorial	EOL date
ROS Noetic Ninjemys (**Recommended**)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)

http://docs.ros.org
https://wiki.ros.org/Distributions

2) ROS 1 **Noetic Ninjemys** 官方安装文档链接如下所示：

http://wiki.ros.org/noetic/Installation/Ubuntu

3) ROS **Noetic Ninjemys** 官方安装文档中 Ubuntu 推荐使用 Ubuntu20.04，所以请确保开发板使用的系统为 **Ubuntu20.04 桌面版系统**

http://wiki.ros.org/noetic/Installation

4) 然后使用下面的脚本安装 ros1

```
orangepi@orangepi3b:~$ install_ros.sh ros1
```

5) 使用 ROS 工具前，首先需要初始化下 rosdep，然后编译源码时就能快速的安装一些系统依赖和一些 ROS 中的核心组件

注意，运行下面的命令需要确保开发板能正常访问 github，否则会由于网络问题而报错。

install_ros.sh 脚本会尝试修改/etc/hosts 并自动运行下面的命令。但是这种方法无法保证每次都能正常访问 github，如果 install_ros.sh 安装完 ros1 后有提示下面的错误，请自己想其它办法让开发板的 Linux 系统能正常访问 github，然后再手动运行下面的命令。

```
https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
ERROR: error loading sources list:
        The read operation timed out
```

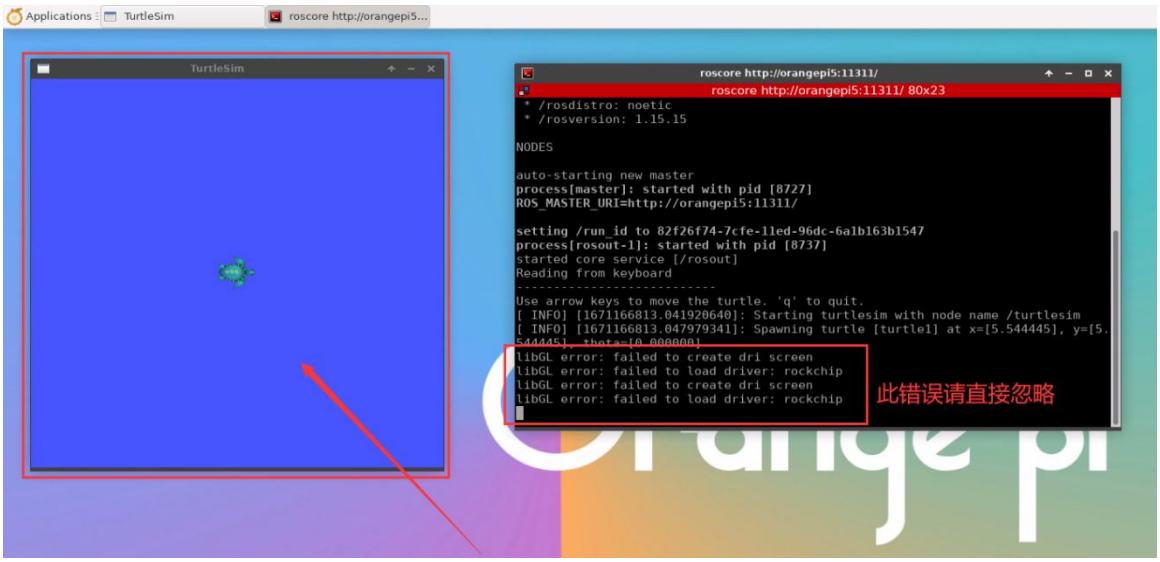
```
orangepi@orangepi:~$ source /opt/ros/noetic/setup.bash
orangepi@orangepi:~$ sudo rosdep init
Wrote /etc/ros/rosdep/sources.list.d/20-default.list
Recommended: please run

        rosdep update
orangepi@orangepi:~$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
```

```
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index
https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Add distro "foxy"
Add distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "jade"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "melodic"
Add distro "noetic"
Add distro "rolling"
updated cache in /home/orangepi/.ros/rosdep/sources.cache
```

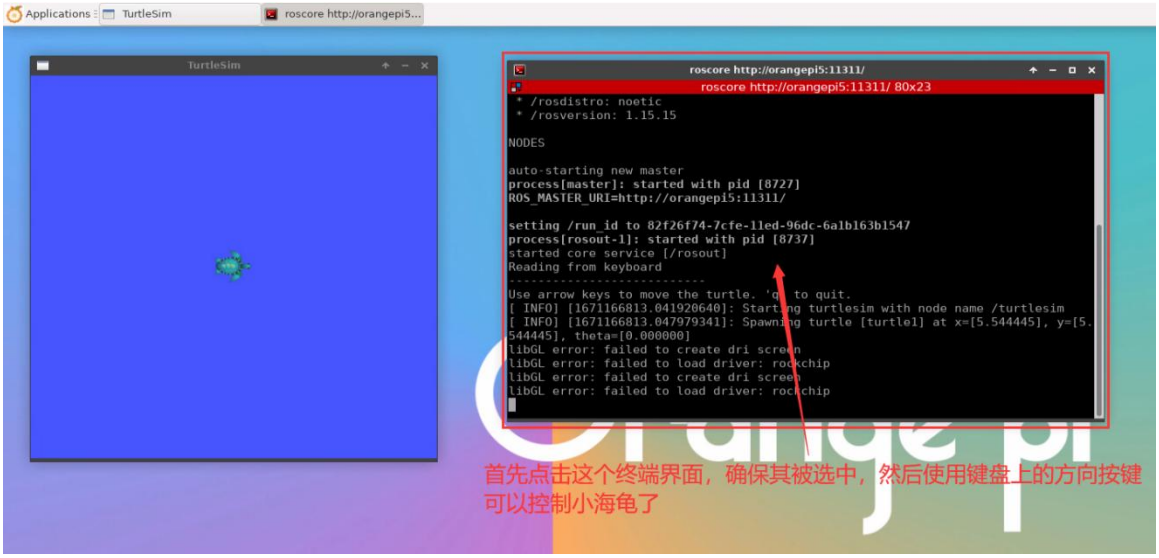
6) 然后在**桌面**中打开一个命令行终端窗口，再使用 **test_ros.sh** 脚本可以启动一个小海龟的例程来测试下 ROS 是否能正常使用

```
orangepi@orangepi:~$ test_ros.sh
```

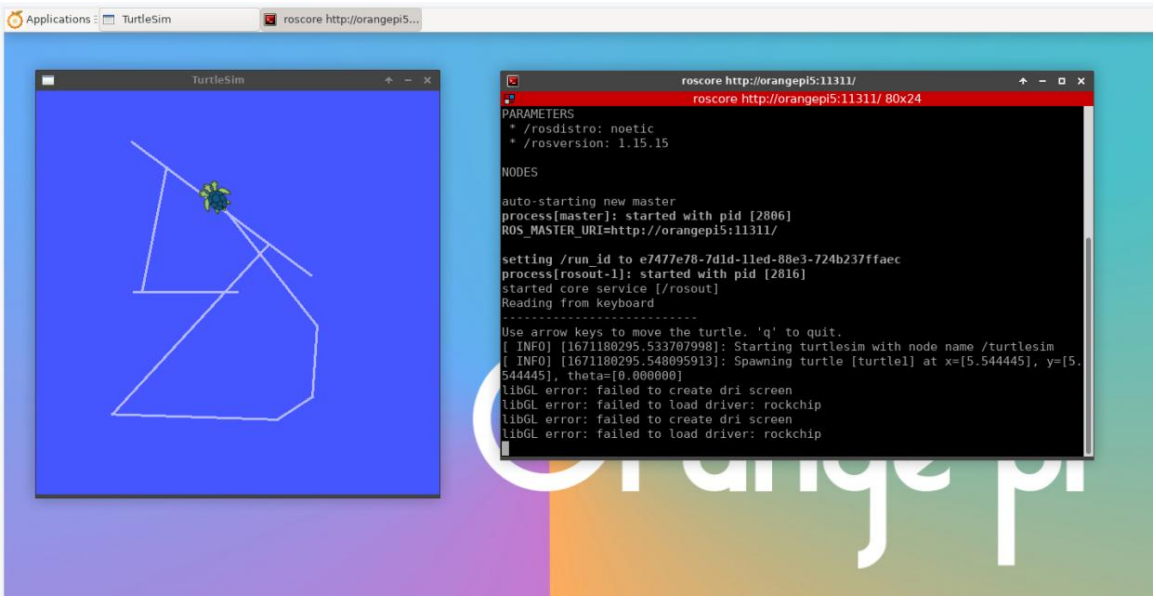
7) 运行完 **test_ros.sh** 脚本后，会弹出下图所示的一个小海龟



8) 然后请保持刚才打开终端窗口在最上面

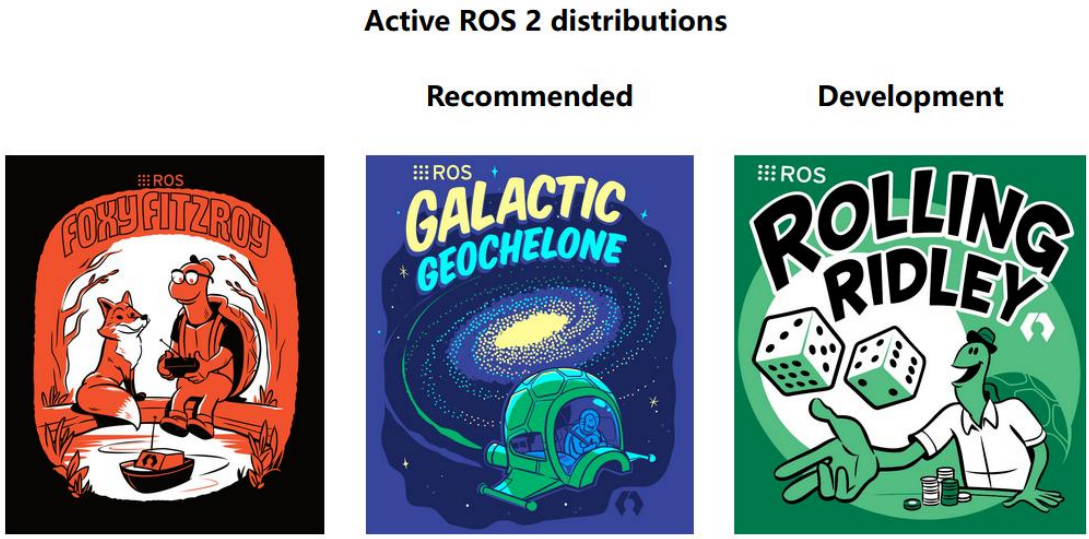


9) 此时按下键盘上的方向按键就可以控制小海龟上下左右移动了



3. 28. 2. Ubuntu20.04 安装 ROS 2 Galactic 的方法

1) ROS 2 当前活跃的版本如下所示，推荐版本为 **Galactic Geochelone**



Distro	Release date	Logo	EOL date
Humble Hawksbill	May 23rd, 2022		May 2027
Galactic Geochelone	May 23rd, 2021		November 2022
Foxy Fitzroy	June 5th, 2020		May 2023

http://docs.ros.org
http://docs.ros.org/en/galactic/Releases.html

2) ROS 2 **Galactic Geochelone** 官方安装文档链接如下所示：

docs.ros.org/en/galactic/Installation.html
http://docs.ros.org/en/galactic/Installation/Ubuntu-Install-Debians.html

3) ROS 2 **Galactic Geochelone** 官方安装文档中 Ubuntu Linux 推荐使用 Ubuntu20.04，所以请确保开发板使用的系统为 **Ubuntu20.04 桌面版系统**。安装 ROS 2 有几种方法，下面演示下通过 **Debian packages** 的方式来安装 ROS 2 **Galactic Geochelone**

4) 使用 **install_ros.sh** 脚本可以安装 ros2

```
orangepi@orangepi:~$ install_ros.sh ros2
```

5) **install_ros.sh** 脚本安装完 ros2 后会自动运行下 **ros2 -h** 命令，如果能看到下面的打印，说明 ros2 安装完成

```
usage: ros2 [-h] Call `ros2 <command> -h` for more detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.

optional arguments:
  -h, --help            show this help message and exit
```

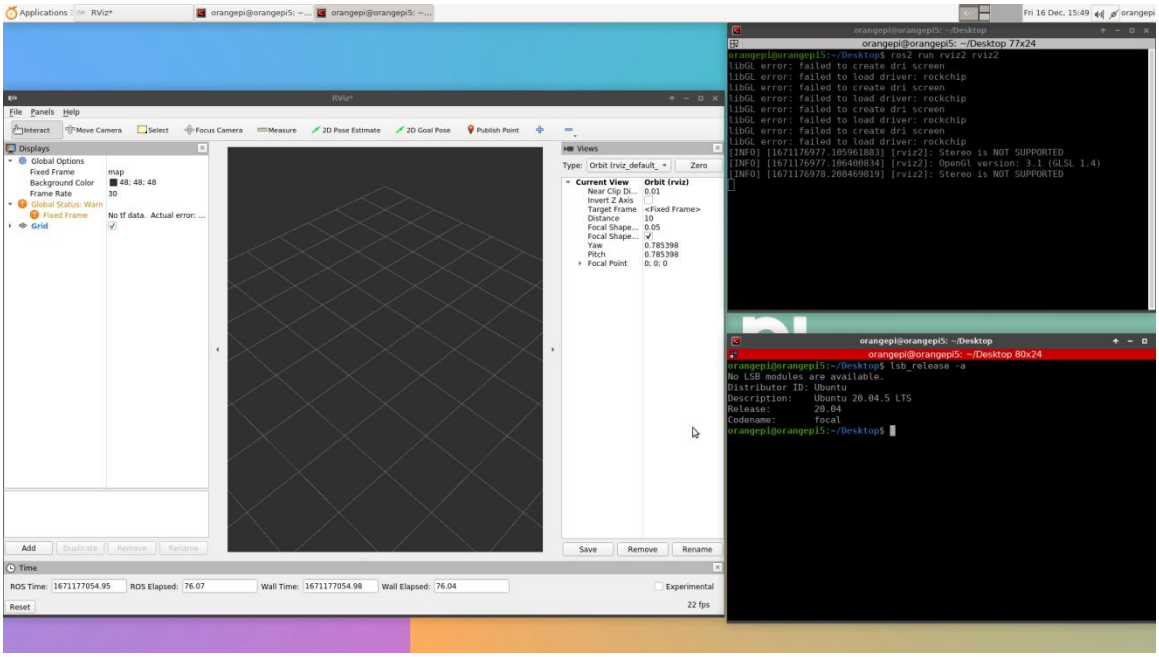
```
Commands:
  action         Various action related sub-commands
  bag            Various rosbag related sub-commands
  component      Various component related sub-commands
  daemon         Various daemon related sub-commands
  doctor         Check ROS setup and other potential issues
  interface      Show information about ROS interfaces
  launch         Run a launch file
  lifecycle      Various lifecycle related sub-commands
  multicast      Various multicast related sub-commands
  node           Various node related sub-commands
  param          Various param related sub-commands
  pkg            Various package related sub-commands
  run            Run a package specific executable
  security       Various security related sub-commands
  service        Various service related sub-commands
  topic          Various topic related sub-commands
  wtf            Use `wtf` as alias to `doctor`

  Call `ros2 <command> -h` for more detailed usage.
```

6) 然后可以使用 **test_ros.sh** 脚本测试下 ROS 2 是否安装成功，如果能看到下面的打印，说明 ROS 2 能正常运行

```
orangepi@orangepi3b:~$ test_ros.sh
[INFO] [1671174101.200091527] [talker]: Publishing: 'Hello World: 1'
[INFO] [1671174101.235661048] [listener]: I heard: [Hello World: 1]
[INFO] [1671174102.199572327] [talker]: Publishing: 'Hello World: 2'
[INFO] [1671174102.204196299] [listener]: I heard: [Hello World: 2]
[INFO] [1671174103.199580322] [talker]: Publishing: 'Hello World: 3'
[INFO] [1671174103.204019965] [listener]: I heard: [Hello World: 3]
```

7) 运行下面的命令可以打开 rviz2

```
orangepi@orangepi:~$ source /opt/ros/galactic/setup.bash
orangepi@orangepi:~$ ros2 run rviz2 rviz2
```



8) ROS 的使用方法请参考下 ROS 2 的文档

http://docs.ros.org/en/galactic/Tutorials.html

3. 28. 3. Ubuntu22.04 安装 ROS 2 Humble 的方法

1) 使用 **install_ros.sh** 脚本可以安装 ros2

```
orangepi@orangepi:~$ install_ros.sh ros2
```

2) **install_ros.sh** 脚本安装完 ros2 后会自动运行下 **ros2 -h** 命令，如果能看到下面的打印，说明 ros2 安装完成

```
usage: ros2 [-h] Call `ros2 <command> -h` for more detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.

optional arguments:
  -h, --help            show this help message and exit

Commands:
  action      Various action related sub-commands
  bag         Various rosbag related sub-commands
  component   Various component related sub-commands
```

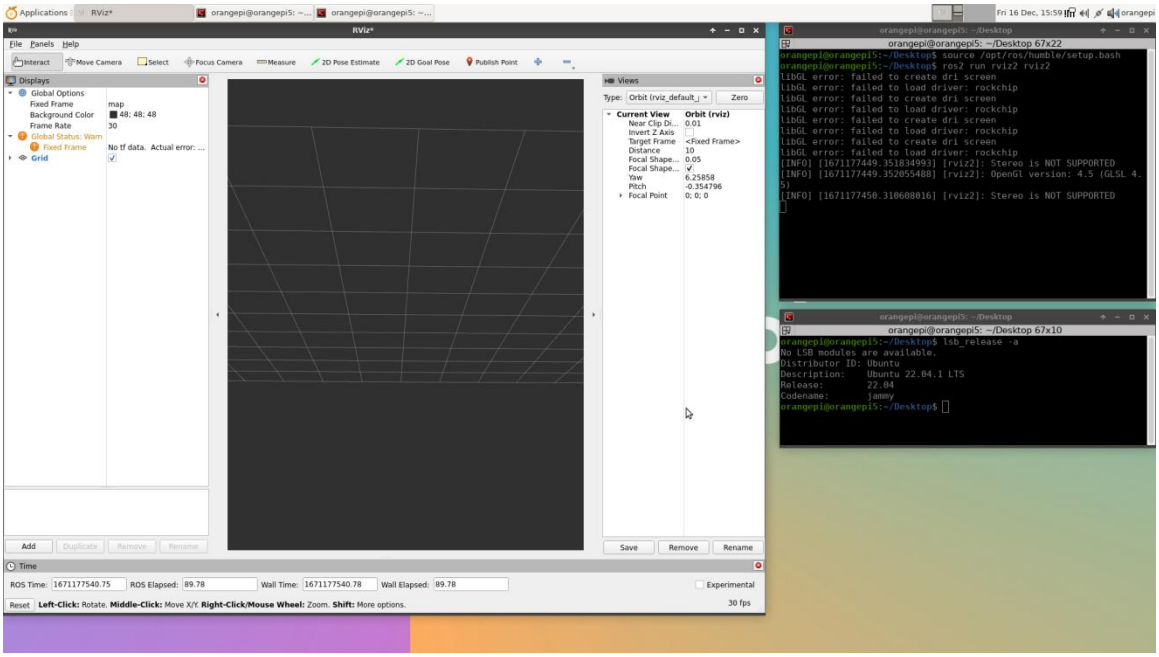
```
daemon      Various daemon related sub-commands
doctor      Check ROS setup and other potential issues
interface   Show information about ROS interfaces
launch      Run a launch file
lifecycle   Various lifecycle related sub-commands
multicast   Various multicast related sub-commands
node        Various node related sub-commands
param       Various param related sub-commands
pkg         Various package related sub-commands
run         Run a package specific executable
security    Various security related sub-commands
service     Various service related sub-commands
topic       Various topic related sub-commands
wtf         Use `wtf` as alias to `doctor`

Call `ros2 <command> -h` for more detailed usage.
```

3) 然后可以使用 **test_ros.sh** 脚本测试下 ROS 2 是否安装成功，如果能看到下面的打印，说明 ROS 2 能正常运行

```
orangepi@orangepi3b:~$ test_ros.sh
[INFO] [1671174101.200091527] [talker]: Publishing: 'Hello World: 1'
[INFO] [1671174101.235661048] [listener]: I heard: [Hello World: 1]
[INFO] [1671174102.199572327] [talker]: Publishing: 'Hello World: 2'
[INFO] [1671174102.204196299] [listener]: I heard: [Hello World: 2]
[INFO] [1671174103.199580322] [talker]: Publishing: 'Hello World: 3'
[INFO] [1671174103.204019965] [listener]: I heard: [Hello World: 3]
```

4) 运行下面的命令可以打开 rviz2

```
orangepi@orangepi:~$ source /opt/ros/humble/setup.bash
orangepi@orangepi:~$ ros2 run rviz2 rviz2
```



5) 参考文档

http://docs.ros.org/en/humble/index.html
http://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html

3.29. 安装内核头文件的方法

1) OPi 发布的 Linux 镜像默认自带了内核头文件的 deb 包，存放的位置为**/opt/**

```
orangepi@orangepi:~$ ls /opt/linux-headers*
/opt/linux-headers-legacy-rockchip-rk356x_x.x.x_arm64.deb
```

2) 使用下面的命令可以安装内核头文件的 deb 包

内核头文件 deb 包的名字需要替换为实际的名字，请不要照抄。

```
orangepi@orangepi:~$ sudo dpkg    -i    /opt/linux-headers-legacy-rockchip-rk356x_1.x.x_arm64.deb
```

3) 安装完后在**/usr/src** 下就能看到内核头文件所在的文件夹

```
orangepi@orangepi:~$ ls /usr/src
linux-headers-5.10.160-rockchip-rk356x
```

4) 然后可以编写一个 hello 内核模块测试下内核头文件

a. 首先编写 hello 内核模块的代码，如下所示：

```
orangepi@orangepi:~$ vim hello.c
#include <linux/init.h>
#include <linux/module.h>

static int hello_init(void)
{
        printk("Hello Orange Pi -- init\n");

        return 0;
}
static void hello_exit(void)
{
        printk("Hello Orange Pi -- exit\n");

        return;
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
```

b. 然后编写编译 hello 内核模块的 Makefile 文件，如下所示：

```
orangepi@orangepi:~$ vim Makefile
ifneq ($(KERNELRELEASE),)
obj-m:=hello.o
else
KDIR :=/lib/modules/$(shell uname -r)/build
PWD  :=$(shell pwd)
all:
    make -C $(KDIR) M=$(PWD) modules
clean:
    rm -f *.ko *.o *.mod.o *.mod *.symvers *.cmd  *.mod.c *.order
endif
```

c. 然后使用 make 命令编译 hello 内核模块，编译过程的输出如下所示：

如果自己复制的代码这里编译如果有问题，请去官方工具中下载源码然后上传

到开发板的 Linux 系统中测试。

```
orangepi@orangepi:~$ make
make -C /lib/modules/5.10.160-rockchip-35xx/build M=/home/orangepi modules
make[1]: Entering directory '/usr/src/linux-headers-5.10.160-rockchip-rk35xx'
  CC [M]  /home/orangepi/hello.o
  MODPOST /home/orangepi/Module.symvers
  CC [M]  /home/orangepi/hello.mod.o
  LD [M]  /home/orangepi/hello.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.10.160-rockchip-rk35xx'
```

d. 编译完后会生成 **hello.ko** 内核模块

```
orangepi@orangepi:~$ ls *.ko
hello.ko
```

e. 使用 **insmod** 命令可以将 **hello.ko** 内核模块插入内核中

```
orangepi@orangepi:~$ sudo insmod hello.ko
```

f. 然后使用 **demsg** 命令可以查看下 **hello.ko** 内核模块的输出，如果能看到下面的输出说明 **hello.ko** 内核模块加载正确

```
orangepi@orangepi:~$ dmesg | grep "Hello"
[ 2871.893988] Hello Orange Pi -- init
```

g. 使用 **rmmod** 命令可以卸载 **hello.ko** 内核模块

```
orangepi@orangepi:~$ sudo rmmod hello
orangepi@orangepi:~$ dmesg | grep "Hello"
[ 2871.893988] Hello Orange Pi -- init
[ 3173.800892] Hello Orange Pi -- exit
```

3.30. 树莓派 5 寸屏幕的使用

3.30.1. 树莓派 5 寸屏幕的组装方法

注意，linux6.6 暂不支持。

1) 首先准备需要的配件

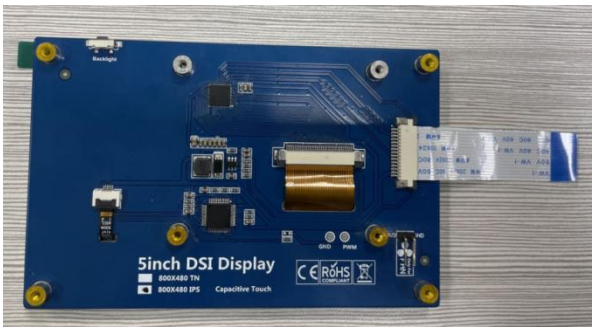
a. 树莓派 5 寸 MIPI LCD 显示屏+触摸屏

再次强调下，只要购买的屏幕和 Orange Pi 测试的屏幕是一样的，那么树莓派 5 寸屏幕的显示和触摸就都是可以使用的。

Orange Pi 测试的屏幕如下图所示，购买的店铺为淘宝的 LCD DIY。其他店铺购买的类似树莓派的屏幕一般也是可以用的，但是由于没有测试过，无法百分百保证。

b. 15pin MIPI 排线

2) 然后将 15pin MIPI 排线按照下图所示的方式连接到树莓派 5 寸屏幕上（注意绝缘面的朝向）



3) 最后连接到 Orange Pi 3B 开发板的 LCD 接口上

3.30.2. 打开树莓派 5 寸屏幕配置的方法

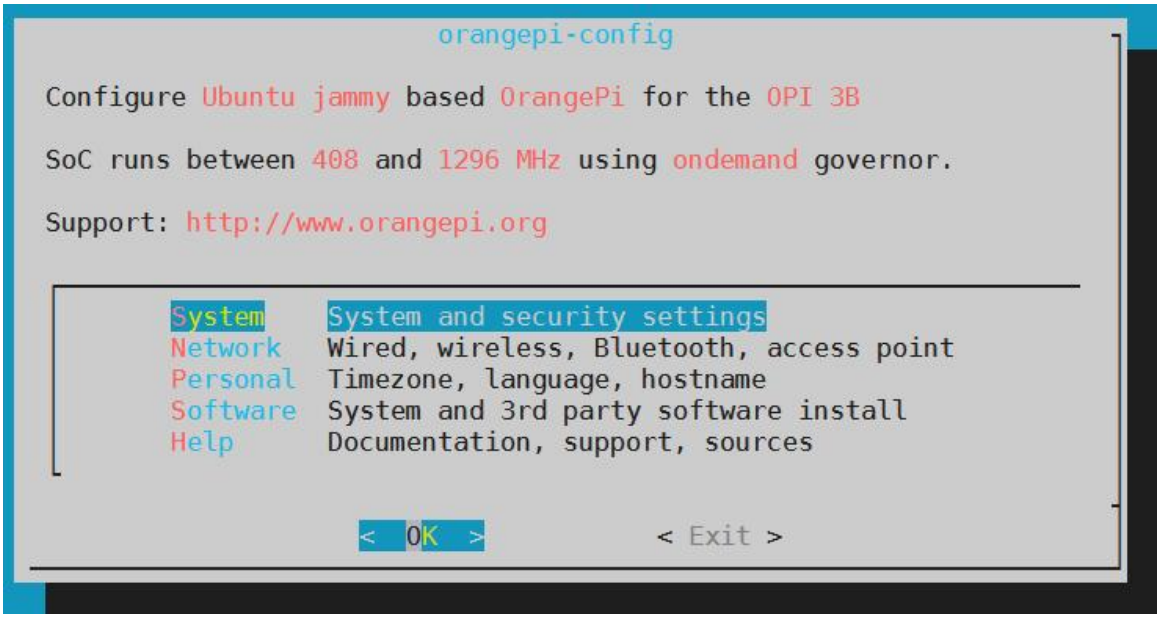
1) Linux 镜像默认是没有打开树莓派 5 寸屏幕的配置的，如果需要使用树莓派 5 寸屏幕，需要手动打开才行。

2) 打开 mipi lcd 配置的步骤如下所示：

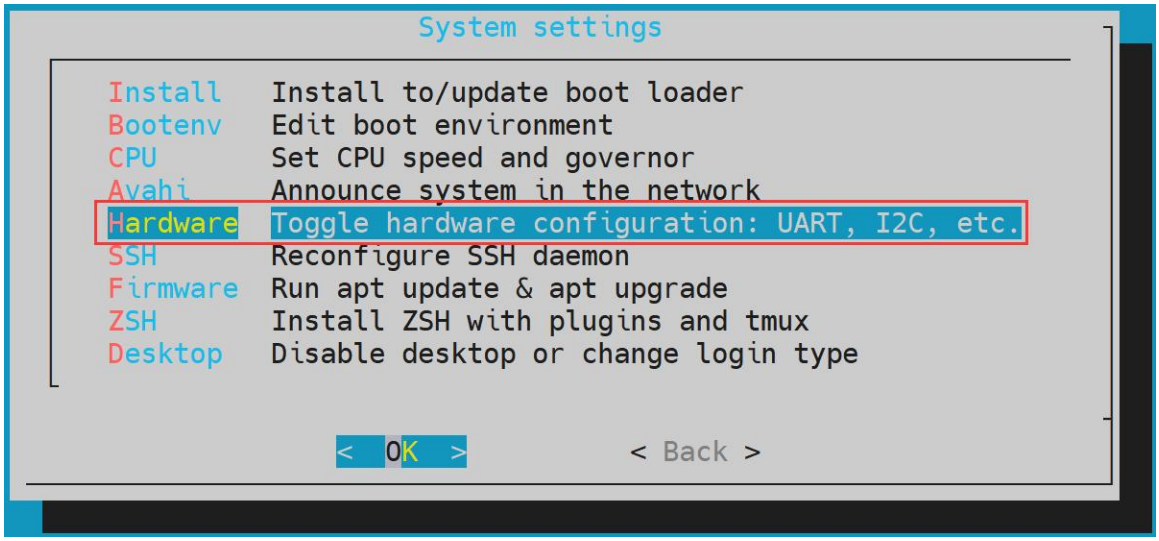
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

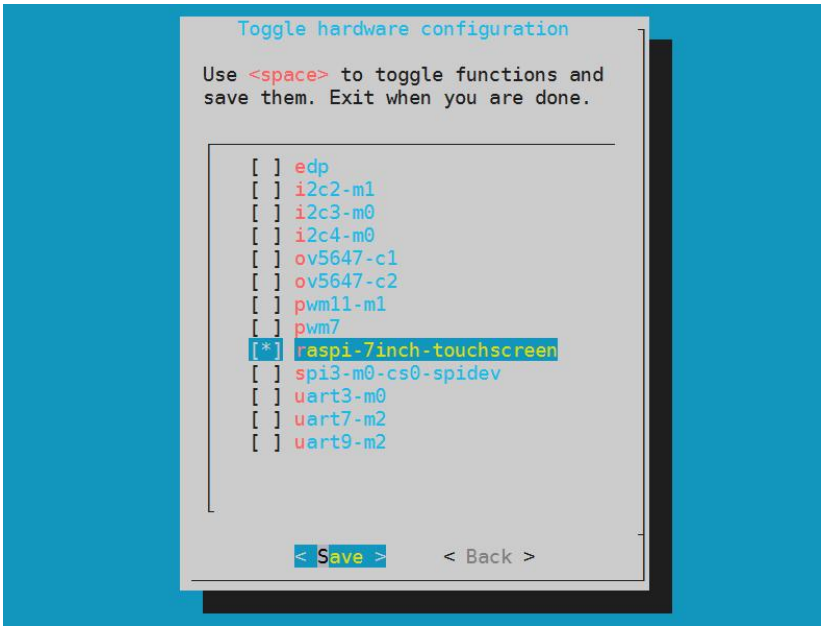
b. 然后选择 **System**



c. 然后选择 **Hardware**



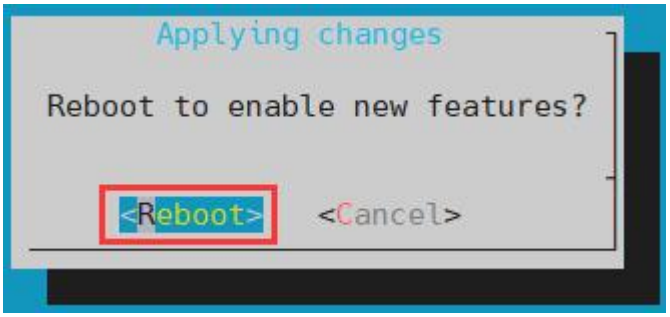
d. 然后使用键盘的方向键定位到 raspi-7inch-touchscreen，再使用**空格**选中



e. 然后选择**<Save>**保存

f. 然后选择**<Back>**

g. 然后选择**<Reboot>**重启系统使配置生效



上面的设置最终会在 **/boot/orangepiEnv.txt** 中加入 **overlays=raspi-7inch-touchscreen** 这句配置。设置完后可以先检查下。如果不存在这句配置，那么设置就是有问题。

如果觉得使用 **orangepi-config** 比较麻烦，也可以使用 **vim** 编辑器打开 **/boot/orangepiEnv.txt**，然后加入 **overlays=raspi-7inch-touchscreen** 这句配置也是可以。

orangepi@orangepi:~$ **cat /boot/orangepiEnv.txt | grep "raspi"**
overlays=raspi-7inch-touchscreen **#示例配置**

3) 启动后可以看到 lcd 屏幕的显示如下所示：

3.30.3. 服务器版镜像旋转显示方向的方法

1) 在**/boot/orangepiEnv.txt** 中加入 **extraargs=fbcon=rotate:要旋转的方向**这行配置就可以设置服务器版本的 Linux 系统显示的方向，其中 **fbcon=rotate:**后面的数字可以设置为：

a. 0: 正常屏（默认为横屏）
b. 1: 顺时钟转 90 度
c. 2: 翻转 180 度
d. 3: 顺时钟转 270 度

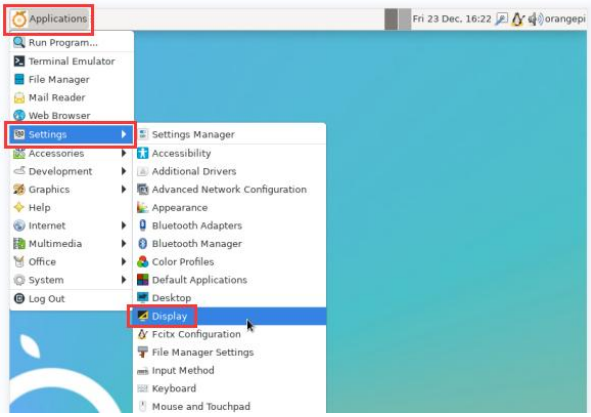
orangepi@orangepi:~$ **sudo vim /boot/orangepiEnv.txt**
overlays=lcd1
extraargs=cma=64M fbcon=rotate:3

注意，/boot/orangepiEnv.txt 中如果默认有 extraargs=cma=64M 这行配置，fbcon=rotate:3 这个配置添加到 extraargs=cma=64M 的后面即可（需要用空格隔开）。

2) 然后重启 Linux 系统就能看到 lcd 屏幕显示的方向已经旋转了

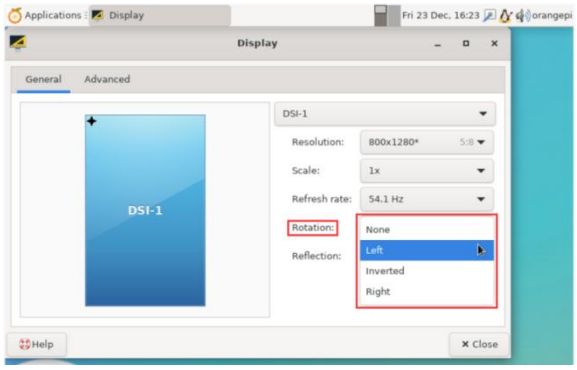
3.30.4. 桌面版镜像旋转显示和触摸方向的方法

1) 首先在 Linux 系统中打开 **Display** 设置

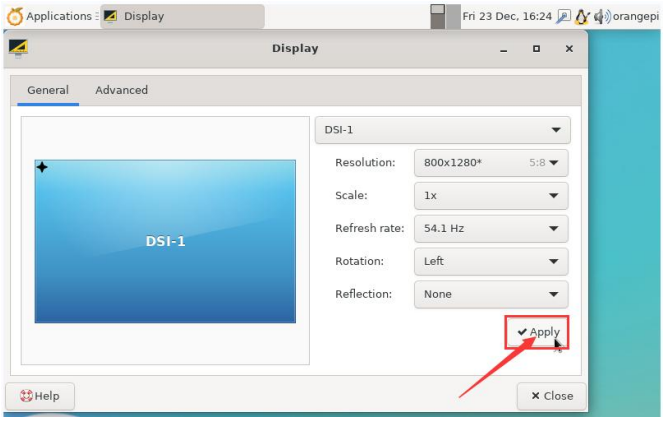


2) 然后在 **Rotation** 中选择想要旋转的方向

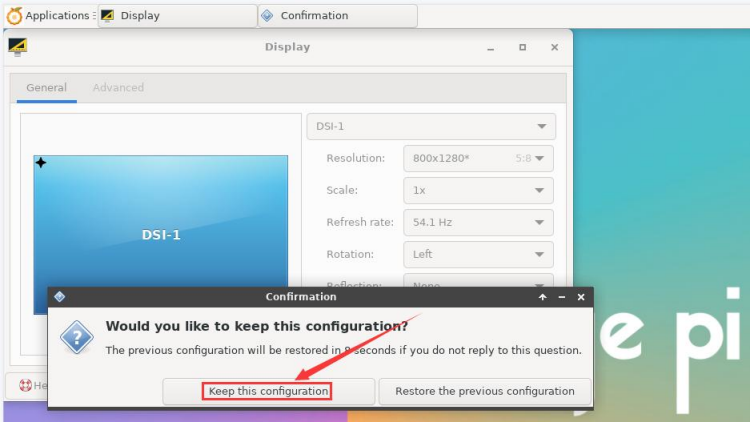
a. **None**: 不旋转
b. **Left**: 向左旋转 90 度
c. **Inverted**: 上下翻转，相当于旋转 180 度
d. **Right**: 向右旋转 90 度



3) 然后点击 **Apply**



4) 然后选择 **Keep this configuration**



5) 此时屏幕显示就已旋转完成，然后关闭掉 **Display** 程序即可

6) 上面的步骤只会选择显示方向，并不会旋转触摸的方向，使用 **set_lcd_rotate.sh** 脚本可以旋转下触摸的方向，此脚本设置完后会自动重启，然后就可以测试触摸是否已经能正常使用了

a. **None**：不旋转

```
orangepi@orangepi:~$ set_lcd_rotate.sh none
```

b. **Left**：向左旋转 90 度

```
orangepi@orangepi:~$ set_lcd_rotate.sh left
```

c. **Inverted**：上下翻转，相当于旋转 180 度

```
orangepi@orangepi:~$ set_lcd_rotate.sh inverted
```

d. **Right**：向右旋转 90 度

```
orangepi@orangepi:~$ set_lcd_rotate.sh right
```

set_lcd_rotate.sh 脚本主要做四件事：

1. **旋转 framebuffer 显示的方向**
2. **旋转触摸的方向**
3. **关闭开机 logo**
4. **重启系统**

旋转触摸的方向是通过在/usr/share/X11/xorg.conf.d/40-libinput.conf 中加入 Option "TransformationMatrix" "x x x x x x x x x"这行配置来实现的。其中"x x x x x x x x x"不同的方向配置不同。

7) 触摸旋转参考资料

https://wiki.ubuntu.com/X/InputCoordinateTransformation

3.31. eDP 屏幕的使用方法

3.31.1. eDP 屏幕的组装方法

注意，linux6.6 暂不支持。

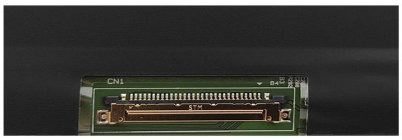
1) 目前仅适配了一款 15.6 寸的 eDP 屏幕，包含的配件如下：

a. 0.5 间距 30pin 单头同向排线

b. 15.6 寸 eDP 显示屏，分辨率为 1920x1080。

正面

背面



30pins EDP接口

2) 将 30pin 单头同向排线的 FPC 端连接到开发板的 eDP 接口，另一端连接到屏幕的 eDP 接口

3.31.2. 打开 eDP 屏幕配置的方法

注意，下面介绍的方法仅适用于已适配过的 eDP 屏幕，如果客户使用的是未经过适配的屏幕，按照下面的方法是无法点亮的。

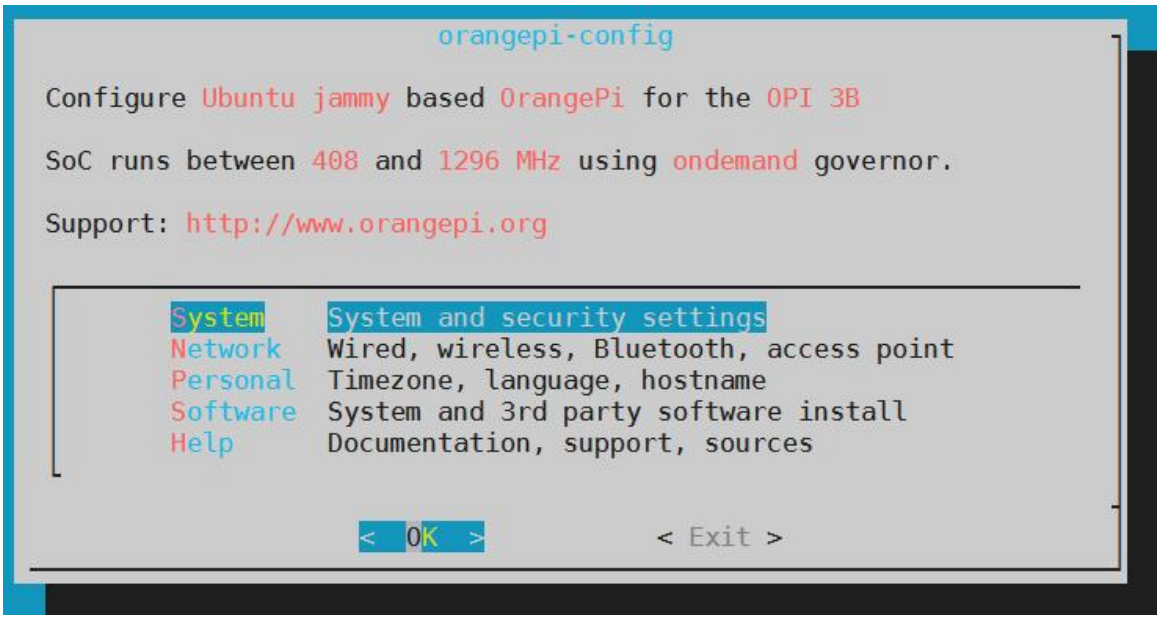
1) Linux 镜像默认是没有打开 eDP 屏幕的配置的，如果需要使用 eDP 屏幕，需要手动打开才行。

2) 打开 eDP 屏配置的步骤如下所示：

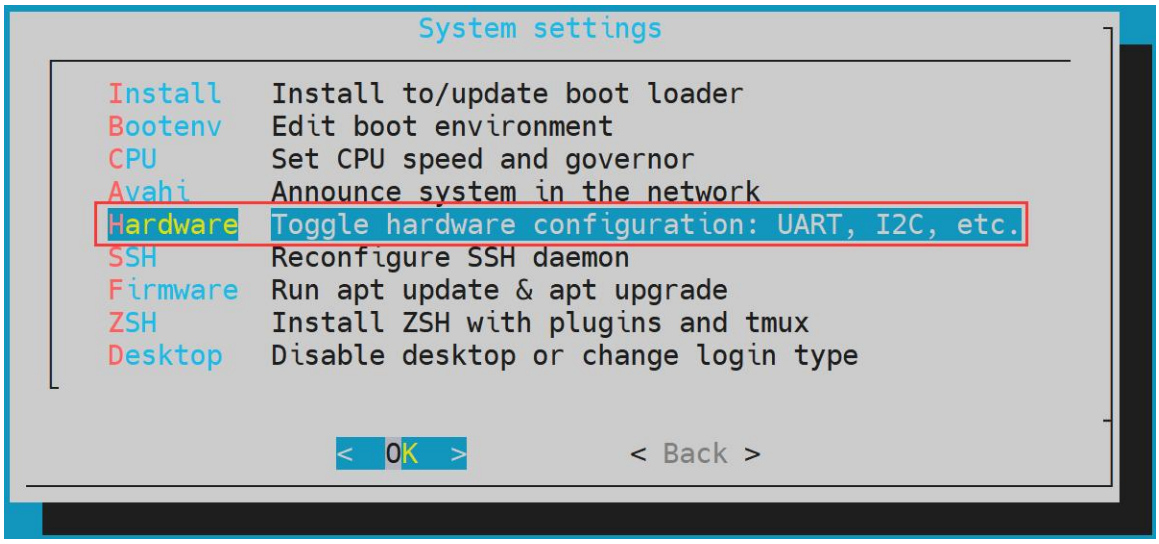
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

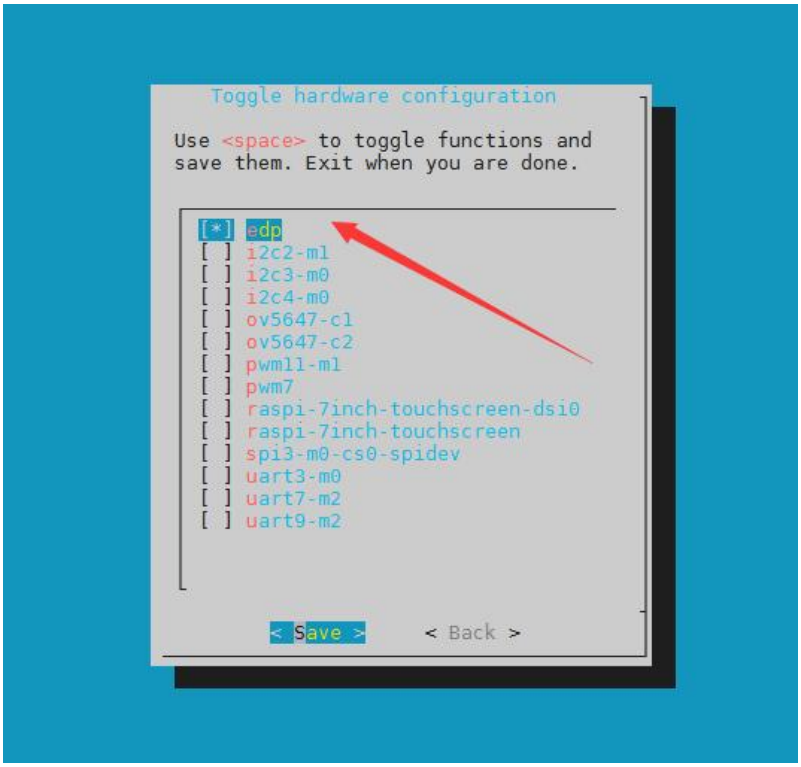
b. 然后选择 **System**



c. 然后选择 **Hardware**

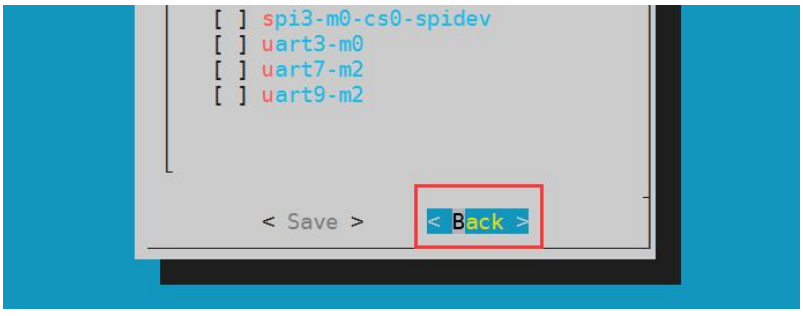


d. 然后使用键盘的方向键定位到 edp，再使用**空格**选中

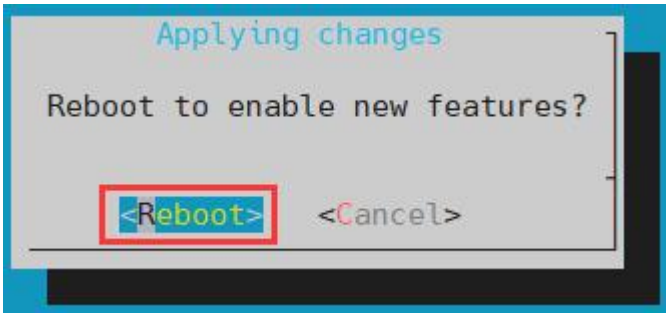


e. 然后选择**<Save>**保存

f. 然后选择**<Back>**



g. 然后选择**<Reboot>**重启系统使配置生效



上面的设置最终会在**/boot/orangepiEnv.txt** 中加入 **overlays=edp** 这句配置。设置完后可以先检查下。如果不存在这句配置，那么设置就是有问题。

如果觉得使用 **orangepi-config** 比较麻烦，也可以使用 **vim** 编辑器打开 **/boot/orangepiEnv.txt**，然后加入 **overlays=edp** 这句配置也是可以。

orangepi@orangepi:~$ **cat /boot/orangepiEnv.txt | grep "edp"**
overlays=edp **#示例配置**

3) 启动后可以看到 eDP 屏幕的显示如下所示：

3.32. 开关机 logo 使用说明

1) 开关机 logo 默认只在桌面版的系统中才会显示

2) 在**/boot/orangepiEnv.txt** 中设置 **bootlogo** 变量为 **false** 可以关闭开关机 logo

orangepi@orangepi:~$ **vim /boot/orangepiEnv.txt**
verbosity=1

```
bootlogo=false
```

3) 在**/boot/orangepiEnv.txt** 中设置 **bootlogo** 变量为 **true** 可以开启开关机 logo

```
orangepi@orangepi:~$ vim /boot/orangepiEnv.txt
verbosity=1
bootlogo=true
```

4) 开机 logo 图片在 Linux 系统中的位置为

```
/usr/share/plymouth/themes/orangepi/watermark.png
```

5) 替换开机 logo 图片后需要运行下命令才能生效

```
orangepi@orangepi:~$ sudo update-initramfs -u
```

3.33. ZFS 文件系统的使用方法

3.33.1. 安装 ZFS 的方法

注意，linux6.6 暂不支持。

安装 zfs 前，请先确保使用的 Linux 镜像为最新版本。另外，如果系统中已经安装了 zfs，就无需重复安装了。

安装 zfs 前首先需要安装内核头文件，安装内核头文件的方法请参考安装内核头文件的方法小节的说明。

在 Ubuntu20.04、Ubuntu22.04 和 Debian11 系统中，zfs 是无法通过 apt 直接安装的，这是因为默认的 apt 源中 zfs 版本低于 2.1.6，存在和 rk Linux5.10 内核不兼容的问题，这个问题在 zfs 的 2.1.6 及以后的版本中得到了修复。

为了解决这个问题，我们提供了能正常安装的 zfs 的 deb 包，可以从开发板的官方工具中下载到。打开**官方工具**，然后进入 **Ubuntu 和 Debian 系统使用的 zfs 相关的 deb 包**文件夹后，可以看到 Ubuntu20.04、Ubuntu22.04 和 Debian11 三种类型的 deb 包，请下载需要的版本。

下载完对应版本的 zfs deb 包后，请将它们上传到开发板的 Linux 系统中。上传方法请参考**上传文件到开发板 Linux 系统中的方法**小节的说明。

上传完成后，再在开发板 Linux 系统的命令行中使用 **cd** 命令进入 deb 包的目录，然后使用下面的命令就可以安装 zfs 的 deb 包。

```
orangepi@orangepi:~$ sudo apt install ./*.deb
```

安装完成后，使用下面的命令可以看到 zfs 相关的内核模块：

```
orangepi@orangepi:~$ ls /lib/modules/5.10.160-rockchip-rk356x/updates/dkms/
icp.ko  spl.ko  zavl.ko  zcommon.ko  zfs.ko  zlua.ko  znvpair.ko  zunicode.ko  zzstd.ko
```

然后重启下 Linux 系统就能看到 zfs 内核模块会自动加载了：

```
orangepi@orangepi:~$ lsmod | grep "zfs"
zfs                  2801664  0
zunicode              327680  1 zfs
zzstd                 471040  1 zfs
zlua                  139264  1 zfs
zcommon                69632  1 zfs
znvpair                61440  2 zfs,zcommon
zavl                   16384  1 zfs
icp                   221184  1 zfs
spl                    77824  6 zfs,icp,zzstd,znvpair,zcommon,zavl
```

在 Debian12 中，zfs 的默认版本为 2.1.11，所以我们可以通过下面的命令直接安装 zfs，再次提醒下，安装前需要确保系统已安装内核头文件的 deb 包。

```
orangepi@orangepi:~$ sudo apt install -y zfsutils-linux zfs-dkms
```

3.33.2. 创建 ZFS 池的方法

ZFS 是基于存储池的，我们可以将多个物理存储设备添加到池中，然后从这个池中分配存储空间。

下面的内容是基于开发板接了一个 NVMe SSD 和一个 U 盘来演示的。

1) 首先我们可以通过 **lsblk** 命令查看下开发板所有的存储设备，当前开发板接了一个 NVMe SSD 以及一个 U 盘，输出如下所示：

```
orangepi@orangepi:~$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda           8:0    1  28.8G  0 disk
├─sda1        8:1    1  28.8G  0 part
└─sda9        8:9    1     8M  0 part
mtdblock0    31:0    0    16M  0 disk
mmcblk0     179:0    0  29.7G  0 disk
├─mmcblk0p1 179:1    0     1G  0 part /boot
└─mmcblk0p2 179:2    0  28.4G  0 part /var/log.hdd
                                      /
zram0       254:0    0   7.7G  0 disk [SWAP]
zram1       254:1    0   200M  0 disk /var/log
nvme0n1     259:0    0 476.9G  0 disk
├─nvme0n1p1 259:3    0 476.9G  0 part
└─nvme0n1p9 259:4    0     8M  0 part
orangepi@orangepi:~$
```

2) 然后输入下面的命令可以创建一个 ZFS 池，包含 NVMe SSD 和 U 盘两个存储设备

```
orangepi@orangepi:~$ sudo zpool create -f pool1 /dev/nvme0n1 /dev/sda
```

3) 然后使用 **zpool list** 命令可以看到系统已经创建了一个名为 **pool1** 的 ZFS 池，并且 ZFS 池 pool1 的大小是 NVME SSD 的大小加上 U 盘的大小

```
orangepi@orangepi:~$ zpool list
NAME    SIZE  ALLOC   FREE  CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP    HEALTH  ALTROOT
pool1   504G   114K   504G        -         -     0%     0%  1.00x    ONLINE  -
```

4) 然后执行 **df -h** 可以看到 **pool1** 被挂载到了**/pool1** 目录

```
orangepi@orangepi:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G   18M  1.6G   2% /run
/dev/mmcblk0p2   29G  6.0G   22G  22% /
tmpfs           7.7G   46M  7.7G   1% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
```

```
tmpfs           7.7G  944K  7.7G   1% /tmp
/dev/mmcblk0p1 1022M  115M  908M  12% /boot
/dev/zram1      188M  4.5M  169M   3% /var/log
tmpfs           1.6G   80K  1.6G   1% /run/user/1000
pool1           489G  9.3M  489G   1% /pool1
```

5) 使用下面的命令可以看到 pool1 的文件系统类型为 zfs

```
orangepi@orangepi:~$ mount | grep pool1
pool1 on /pool1 type zfs (rw,xattr,noacl)
```

6) 然后我们可以测试下拷贝一个文件到 ZFS 池中

```
orangepi@orangepi:~$ sudo cp -v /usr/local/test.mp4 /pool1/
'/usr/local/test.mp4' -> '/pool1/test.mp4'
```

3.33.3. 测试 ZFS 的数据去重功能

1) ZFS 的数据去重功能默认是关闭的，我们需要执行下面的命令打开

```
orangepi@orangepi:~$ sudo zfs set dedup=on pool1
```

2) 然后做一个简单的测试，首先进入 pool1 中，再执行下面的命令生成 1 个 1G 大小的随机文件

```
orangepi@orangepi:~$ cd /pool1/
root@orangepi:/pool1$ sudo dd if=/dev/urandom of=test.1g bs=1M count=1024
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 5.04367 s, 213 MB/s
```

3) 然后使用下面的命令将 1G 大小的随机文件拷贝 1000 份

```
root@orangepi:/pool1$ for ((i=0; i<1000; i++)); do sudo cp test.1g $i.test.1g; done
```

4) 然后用 **du -lh** 可以看到目前池中总共有 1002G 的数据，但实际上 ZFS 池的大小只有 **504GB**（SSD+U 盘的总容量），是装不下那么大的数据的

```
root@orangepi:/pool1$ du -lh
1002G
```

5) 然后用 **zpool list** 命令可以看到实际只占用了 1.01G，因为这 1001 个文件都是重

复的，说明数据去重功能有效。

```
orangepi@orangepi:/pool1$ zpool list
NAME    SIZE  ALLOC   FREE  CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP    HEALTH  ALTROOT
pool1   504G  1.01G   503G        -         -     0%     0%  6.00x    ONLINE  -
```

3.33.4. 测试 ZFS 的数据压缩功能

1) 因为存储的数据不同，压缩节省的磁盘空间也会有所不同，所以我们选择压缩比较大的纯文本文件来进行压缩测试，执行下面的命令将**/var/log/**和**/etc/**目录打包成 tar 包

```
orangepi@orangepi:~$ cd /pool1/
root@orangepi:/pool1$ sudo tar -cf text.tar /var/log/ /etc/
```

2) 然后通过 **ls -lh** 命令可以看到的文件大小以及在 ZFS 池中占用的空间都是 **27M**

```
orangepi@orangepi:/pool1$ ls -lh
total 27M
-rw-r--r-- 1 root root 27M Jun  1 14:46 text.tar
orangepi@orangepi:/pool1$ zpool list
NAME    SIZE  ALLOC   FREE  CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP    HEALTH  ALTROOT
pool1   504G  26.7M   504G        -         -     0%     0%  1.00x    ONLINE  -
orangepi@orangepi:/pool1$
```

3) 然后我们在 ZFS 池 pool1 中启用压缩功能

```
root@orangepi:/pool1$ sudo zfs set compression=lz4 pool1
```

4) 然后再次执行下面的命令将**/var/log/**和**/etc/**目录打包成 tar 包

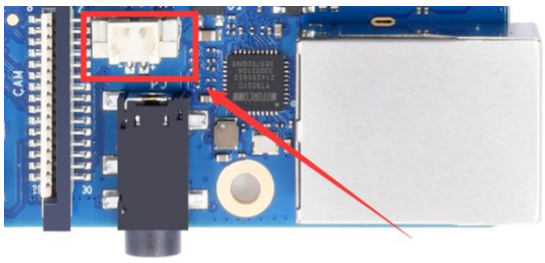
```
root@orangepi:/pool1$ sudo tar -cf text.tar /var/log/ /etc/
```

5) 这时可以看到 **text.tar** 文件大小还是 27M，但是在 ZFS 池中只占用 9.47M 的空间，说明文件被压缩了

```
orangepi@orangepi:/pool1$ ls -lh
total 9.2M
-rw-r--r-- 1 root root 27M Jun  1 14:54 text.tar
orangepi@orangepi:/pool1$ zpool list
NAME    SIZE  ALLOC   FREE  CKPOINT  EXPANDSZ   FRAG    CAP  DEDUP    HEALTH  ALTROOT
pool1   504G  9.47M   504G        -         -     0%     0%  1.00x    ONLINE  -
```

3.34. RTC 的使用方法

1) 开发板上预留了一个 RTC 电池的接口，位置如下所示：



2) 需要购买的 RTC 电池如下所示，接口为 2pin，1.25mm 间距

3) 开发板上使用的 RTC 芯片为 HYM8563TS，此芯片的特点有：
   a. 宽工作电压范围：1.0~5.5v
   b. 低休眠电流:典型值为 0.25μA(VDD =3.0V, TA =25°C)

4) 在开发板上接好 RTC 电池后，使用下面的方法测试下 RTC 是否已正常工作：
   a. 首先开机进入系统，然后记录下系统的当前时间
   b. 然后使用 poweroff 命令正常关闭系统
   c. 然后拔掉电源，确保开发板没有接入网线和无线 WIFI，再等待几分钟
   d. 然后再次开机启动系统，如果进入系统后看到时间向前走了几分钟，就说明 RTC 模块和电池正常工作了

5) 通过 Linux 系统的 procfs 接口查看 RTC 信息的命令为

```
orangepi@orangepi:~$ cat /proc/driver/rtc
rtc_time             : 10:10:36
rtc_date             : 2023-10-19
alrm_time            : 00:00:00
alrm_date            : 1999-12-16
alarm_IRQ            : no
alrm_pending         : no
update IRQ enabled          : no
periodic IRQ enabled        : no
periodic IRQ frequency      : 1
max user IRQ frequency      : 64
```

```
24hr                    : yes
```

3. 35. Linux6.6 系统 GPU 的测试方法

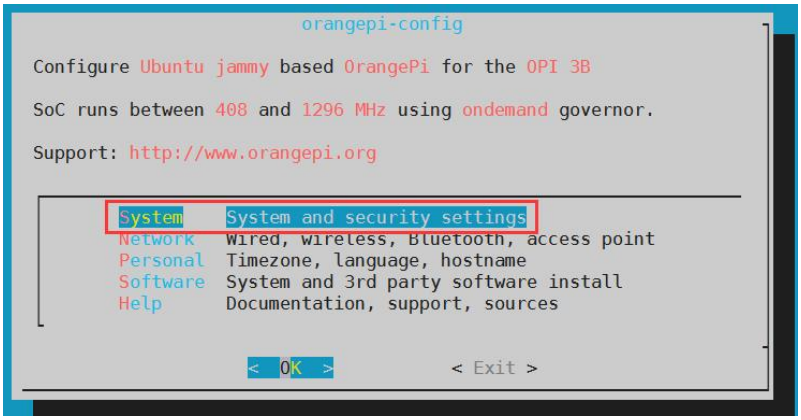
注意，GPU 打开后桌面壁纸有可能会显示黑屏，所以 GUP 默认是关闭的。
GPU 仅在 Ubuntu22.04 和 Debian12 中可以使用。

1) 在 linux6.6 系统中，GPU 默认是关闭的，需要手动打开才能使用。详细步骤如下所示：

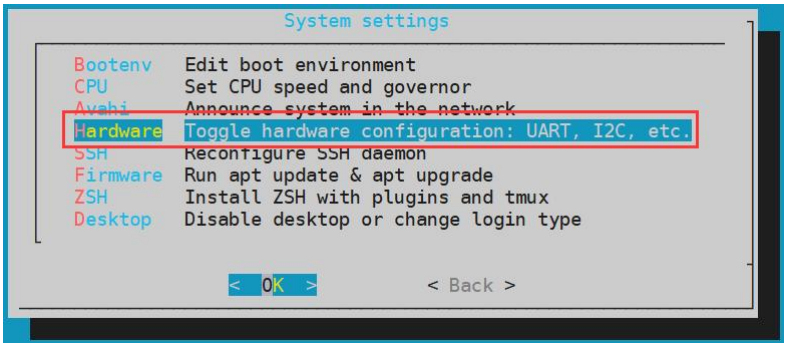
a. 首先运行下 **orangepi-config**，普通用户记得加 **sudo** 权限

```
orangepi@orangepi:~$ sudo orangepi-config
```

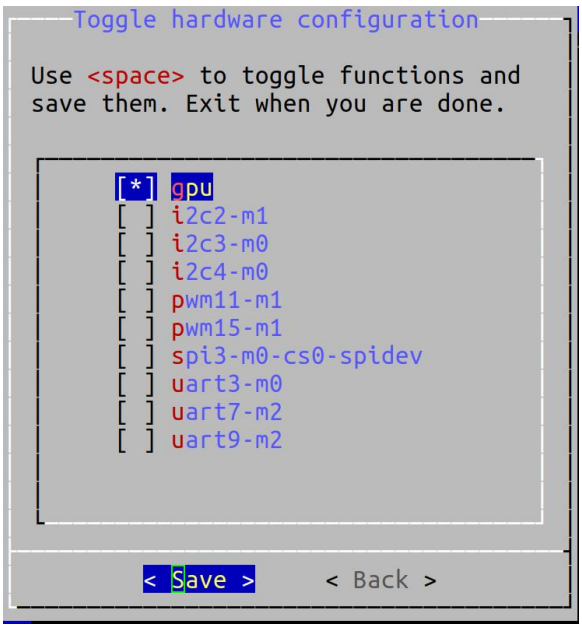
b. 然后选择 System



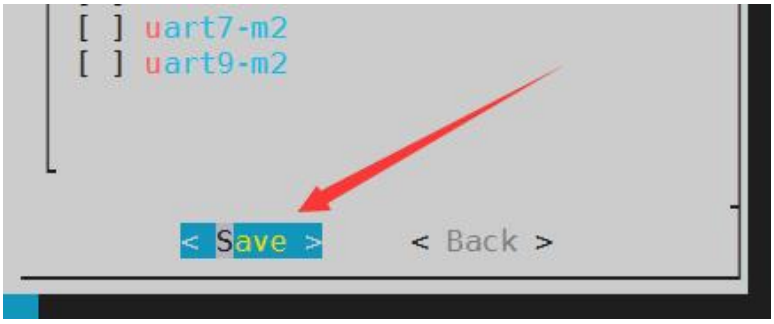
c. 然后选择 Hardware



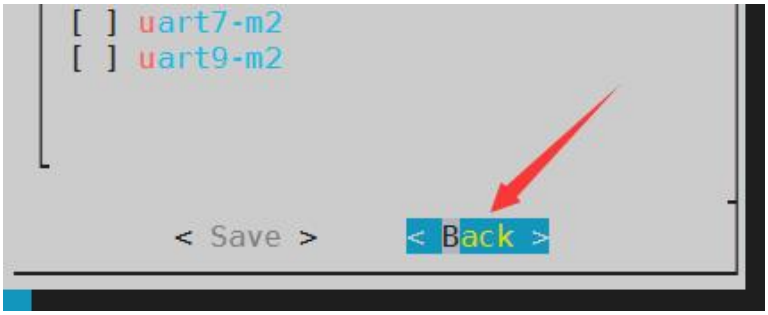
d. 然后使用键盘的方向键定位到下图所示的位置，再使用空格选中 gpu 选项



e. 然后选择<Save>保存



f. 然后选择<Back>



g. 然后选择<Reboot>重启系统使配置生效

2) 在桌面中打开一个终端，然后输入 glmark2 命令，如果能看到 GL_VENDOR 后面显示为 **Panfrost** 说明有使用到 GPU

```
orangepi@orangepi:~$ glmark2
=======================================================
    glmark2 2023.01
```

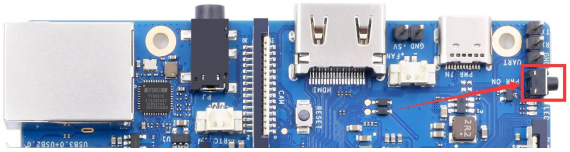
```
==========================================================
    OpenGL Information
    GL_VENDOR:          Panfrost
    GL_RENDERER:        Mali-G52 r1 (Panfrost)
    GL_VERSION:         3.1 Mesa 22.3.6
    Surface Config: buf=32 r=8 g=8 b=8 a=8 depth=24 stencil=0 samples=0
    Surface Size:       800x600 windowed
==========================================================
```

3.36. 关机和重启开发板的方法

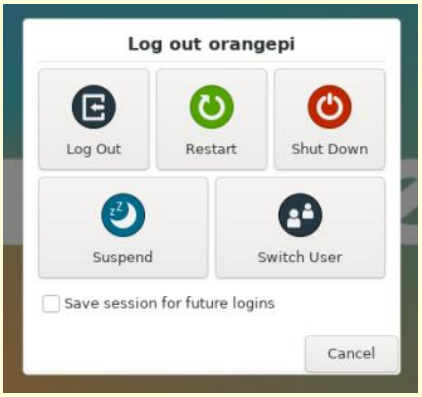
1) 在 Linux 系统运行的过程中，如果直接拔掉 Type-C 电源断电，可能会导致文件系统丢失某些数据或者损坏，所以在断电前请先使用 **poweroff** 命令关闭开发板的 Linux 系统，然后再拔掉电源。

```
orangepi@orangepi:~$ sudo poweroff
```

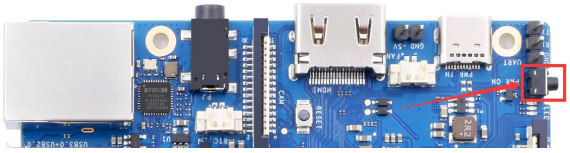
2) 另外开发板配有开关机按键，还可以**短按**开发板上的开关机按键来关机。



注意，Linux 桌面版系统按下开关机按键后会弹出下图所示的确认框，需要点击 Shut Down 选项后才会关机。



3) 关机后短按开发板上的开关机按键即可开机。



4) 重启 Linux 系统的命令为

```
orangepi@orangepi:~$ sudo reboot
```

4. Linux SDK——orangepi-build 使用说明

4.1. 编译系统需求

我们可以在 x64 的电脑中交叉编译开发板的 Linux 镜像，也可以在开发板的 Ubuntu22.04 系统中来编译开发板的 Linux 镜像，请根据自己的喜好二选一。

如果是在开发板的 Ubuntu22.04 系统中使用 orangepi-build 来编译 Linux 镜像，请做好散热（尤其是 SSD 启动时）。如果散热没做好，容易出现文件系统跑飞的错误。

4.1.1. 使用 x64 的 Ubuntu22.04 电脑编译

1) Linux SDK，即 **orangepi-build**，支持在安装有 **Ubuntu 22.04** 的电脑上运行，所以下载 orangepi-build 前，请首先确保自己电脑已安装的 Ubuntu 版本是 Ubuntu 22.04。查看电脑已安装的 Ubuntu 版本的命令如下所示，如果 Release 字段显示的不是 **22.04**，说明当前使用的 Ubuntu 版本不符合要求，请更换系统后再进行下面的操作。

```
test@test:~$ lsb_release -a
No LSB modules are available.
Distributor ID:	Ubuntu
Description:	Ubuntu 22.04 LTS
Release:	22.04
Codename:	jammy
```

2) 如果电脑安装的是 Windows 系统，没有安装有 Ubuntu 22.04 的电脑，可以考虑使用 **VirtualBox** 或者 **VMware** 来在 Windows 系统中安装一个 Ubuntu 22.04 虚拟机。但是请注意，不要在 WSL 虚拟机上编译 orangepi-build，因为 orangepi-build 没有在 WSL 虚拟机中测试过，所以无法确保能正常在 WSL 中使用 orangepi-build。

3) Ubuntu 22.04 **amd64** 版本的安装镜像下载地址为：

https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/22.04/ubuntu-22.04-desktop-amd64.iso
或者
https://repo.huaweicloud.com/ubuntu-releases/22.04/ubuntu-22.04.1-desktop-amd64.iso

4) 在电脑中或者虚拟机中安装完 Ubuntu 22.04 后，请先设置 Ubuntu 22.04 的软件源为清华源，不然后面安装软件的时候很容易由于网络原因而出错

a. 替换清华源的方法参考这个网页的说明即可

https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/

b. 注意 Ubuntu 版本需要切换到 22.04

c. 需要替换的**/etc/apt/sources.list** 文件的内容为

```
test@test:~$ sudo mv /etc/apt/sources.list /etc/apt/sources.list.bak
test@test:~$ sudo vim /etc/apt/sources.list
# 默认注释了源码镜像以提高 apt update 速度，如有需要可自行取消注释
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse


# 预发布软件源，不建议启用
# deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
```

d. 替换完后需要更新下包信息，并确保没有报错

```
test@test:~$ sudo apt update
```

e. 另外，由于内核和 U-boot 等源码都是存放在 GitHub 上的，所以编译镜像

的时候请确保电脑能正常从 GitHub 下载代码，这点是非常重要的。

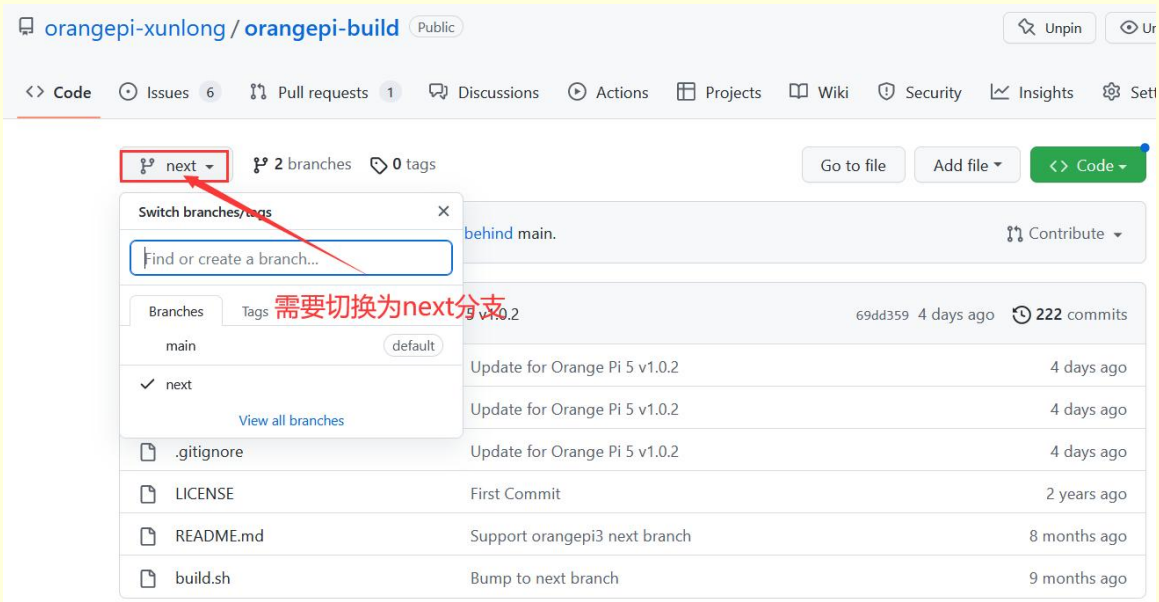
4.2. 获取 Linux sdk 的源码

4.2.1. 从 github 下载 orangepi-build

1) Linux sdk 其实指的就是 orangepi-build 这套代码，orangepi-build 是基于 armbian build 编译系统修改而来的，使用 orangepi-build 可以编译出多个版本的 Linux 镜像。首先下载 orangepi-build 的代码，命令如下所示：

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y git
test@test:~$ git clone https://github.com/orangepi-xunlong/orangepi-build.git -b next
```

注意，Orange Pi 3B 开发板是需要下载 orangepi-build 的 next 分支源码的，上面的 git clone 命令需要指定 orangepi-build 源码的分支为 next。



通过 git clone 命令下载 orangepi-build 的代码是不需要输入 github 账号的用户名和密码的（下载本手册中的其他代码也是一样的），如果如输入 git clone 命令后 Ubuntu PC 提示需要输入 github 账号的用户名和密码，一般都是 git clone 后面的 orangepi-build 仓库的地址输入错误了，请仔细检查命令拼写是否有错误，而不是以为我们这里忘了提供 github 账号的用户名和密码。

2) 开发板当前使用的 u-boot 和 Linux 内核版本如下所示

分支	u-boot 版本	Linux 内核版本
legacy	**u-boot 2017.09**	**Linux5.10**
current	**u-boot 2017.09**	**Linux6.6**

这里所说的分支和 orangepi-build 源代码的分支不是同一个东西，请不要搞混了。此分支主要是用来区分不同内核源码版本的。

目前 RK 提供的 Linux5.10 bsp 内核我们定义为 legacy 分支。最新的 Linux6.6 内核我们定义为 current 分支。

3) orangepi-build 下载完后会包含下面的文件和文件夹

a. **build.sh**: 编译启动脚本
b. **external**: 包含编译镜像需要用的配置文件、特定的脚本以及部分程序的源码等
c. **LICENSE**: GPL 2 许可证文件
d. **README.md**: orangepi-build 说明文件
e. **scripts**: 编译 Linux 镜像的通用脚本

```
test@test:~/orangepi-build$ ls
build.sh    external    LICENSE    README.md    scripts
```

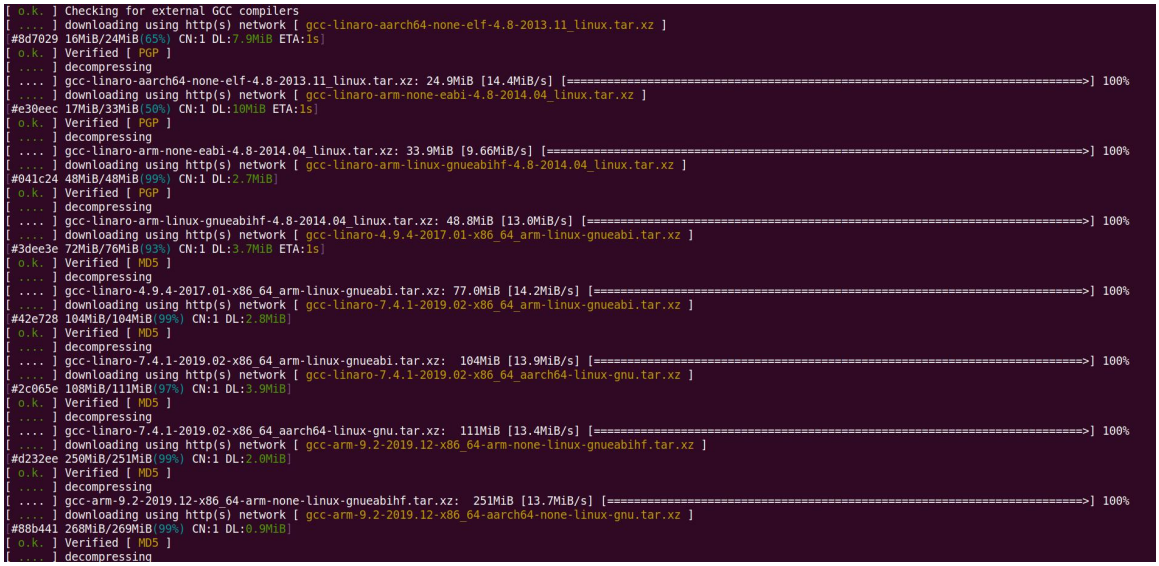
如果是从 github 下载的 orangepi-build 的代码，下载完后你可能会发现 orangepi-build 中并没有包含 u-boot 和 Linux 内核的源码，也没有编译 u-boot 和 Linux 内核需要用到交叉编译工具链，这是正常的，因为这些东西都存放在其它单独的 github 仓库或者某些服务器上了（下文会详述其地址）。orangepi-build 在脚本和配置文件中会指定 u-boot、Linux 内核和交叉编译工具链的地址，运行 orangepi-build 时，当其发现本地没有这些东西，会自动去相应的地方下载的。

4.2.2. 下载交叉编译工具链

只有在 x64 的电脑中使用 orangepi-build 编译镜像才会下载交叉编译工具链。在开发板的 Ubuntu22.04 中编译开发板的 Linux 镜像是不会下载交叉编译工具链的，此时 orangepi-build/toolchains 会是一个空文件夹。

1) orangepi-build 第一次运行的时候会自动下载交叉编译工具链放在 **toolchains** 文件夹中，每次运行 orangepi-build 的 build.sh 脚本后，都会检查 **toolchains** 中的交叉编译工具链是否都存在，如果不存在则会重新开始下载，如果存在则直接使用，不会

重复下载。



2) 交叉编译工具链在中国境内的镜像网址为清华大学的开源软件镜像站

https://mirrors.tuna.tsinghua.edu.cn/armbian-releases/_toolchain/

3) **toolchains** 下载完后会包含多个版本的交叉编译工具链，开发板只会使用其中的两个

```
test@test:~/orangepi-build$ ls toolchains/
gcc-arm-11.2-2022.02-x86_64-aarch64-none-linux-gnu
gcc-arm-11.2-2022.02-x86_64-arm-none-linux-gnueabihf
gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu
gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabihf
gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi
gcc-linaro-5.5.0-2017.10-x86_64_arm-linux-gnueabihf
gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu
gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi
gcc-linaro-aarch64-none-elf-4.8-2013.11_linux
gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux
gcc-linaro-arm-none-eabi-4.8-2014.04_linux
```

4) 编译 Linux 内核源码使用的交叉编译工具链为

a. Linux5.10

gcc-arm-11.2-2022.02-x86_64-aarch64-none-linux-gnu

5) 编译 u-boot 源码使用的交叉编译工具链为

a. v2017.09

gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu

4.2.3. orangepi-build 完整目录结构说明

1) orangepi-build 仓库下载完后并不包含 Linux 内核、u-boot 的源码以及交叉编译工具链，Linux 内核和 u-boot 的源码存放在独立的 git 仓库中

a. Linux 内核源码存放的 git 仓库如下，注意切换 linux-orangepi 仓库的分支为

a) Linux5.10

https://github.com/orangepi-xunlong/linux-orangepi/tree/orange-pi-5.10-rk35xx

b) Linux6.6

https://github.com/orangepi-xunlong/linux-orangepi/tree/orange-pi-6.6-rk35xx

b. u-boot 源码存放的 git 仓库如下所示：

https://github.com/orangepi-xunlong/u-boot-orangepi/tree/v2017.09-rk3588

2) orangepi-build 第一次运行的时候会去下载交叉编译工具链、u-boot 和 Linux 内核源码，成功编译完一次 Linux 镜像后在 orangepi-build 中可以看到的文件和文件夹有

a. **build.sh**: 编译启动脚本
b. **external**: 包含编译镜像需要用的配置文件、特定功能的脚本以及部分程序的源码，编译镜像过程中缓存的 rootfs 压缩包也存放在 external 中
c. **kernel**: 存放 Linux 内核的源码，里面名为 **orange-pi-5.10-rk35xx** 的文件夹存放的就是 RK3588/RK3588S/RK3566 系列开发板 legacy 分支的内核源码，名为 orange-pi-6.6-rk35xx 的文件夹存放的是 RK3566 系列开发板 current 分支的内核源码，内核源码的文件夹的名字请不要手动修改，如果修改了，编译系统运行时会重新下载内核源码
d. **LICENSE**: GPL 2 许可证文件
e. **README.md**: orangepi-build 说明文件
f. **output**: 存放编译生成的 u-boot、Linux 等 deb 包、编译日志以及编译生成的镜像等文件
g. **scripts**: 编译 Linux 镜像的通用脚本
h. **toolchains**: 存放交叉编译工具链
i. **u-boot**: 存放 u-boot 的源码，里面名为 **v2017.09-rk3588** 的文件夹存放的就是 RK3588/RK3588S/RK3566 系列开发板 legacy 分支的 u-boot 源码，u-boot 源码的文件夹的名字请不要手动修改，如果修改了，编译系统运行时会重新

下载 u-boot 源码

j. **userpatches**：存放编译脚本需要用到的配置文件

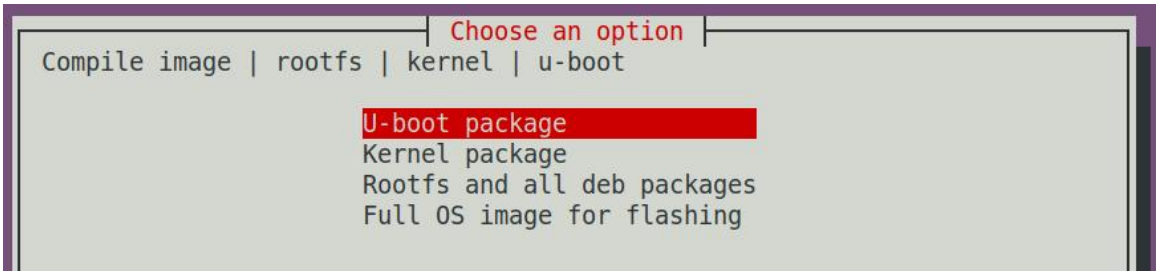
```
test@test:~/orangepi-build$ ls
build.sh  external  kernel  LICENSE  output  README.md  scripts
toolchains  u-boot  userpatches
```

4.3. 编译 u-boot

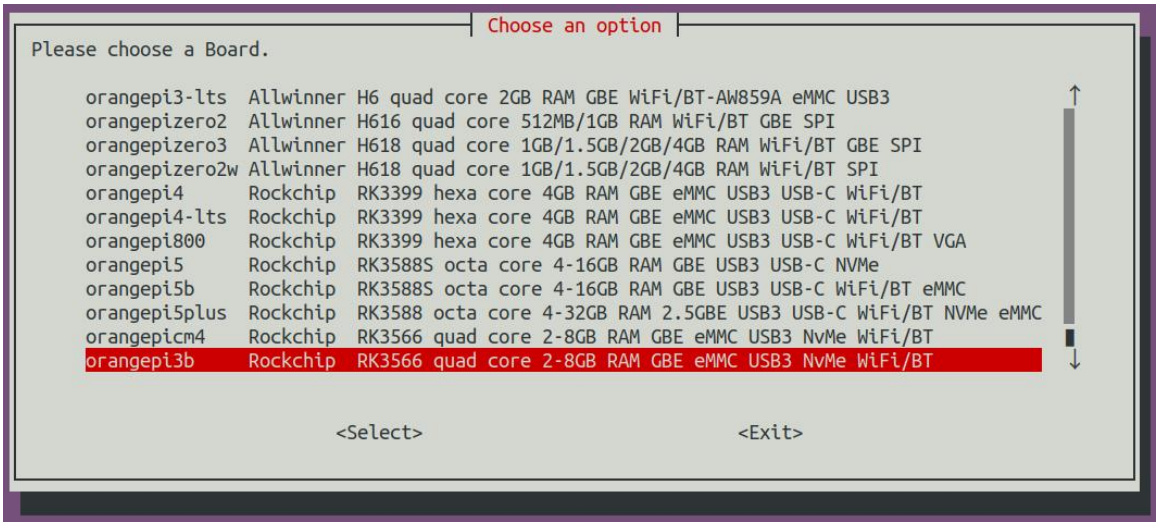
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

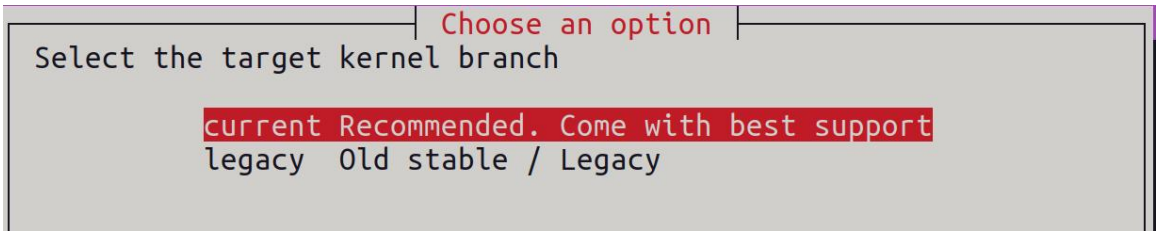
2) 选择 **U-boot package**，然后回车



3) 接着选择开发板的型号



4) 然后选择 u-boot 选择分支类型，current 分支和 legacy 分支都会编译会编译需要使用的 u-boot v2021.07 版本的代码。



5) 然后就会开始编译 u-boot，编译时提示的部分信息说明如下

a. u-boot 源码的版本

[o.k.] Compiling u-boot [**v2017.09**]

b. 交叉编译工具链的版本

[o.k.] Compiler version [**aarch64-linux-gnu-gcc 7.4.1**]

c. 编译生成的 u-boot deb 包的路径

[o.k.] Target directory [**orangepi-build/output/debs/u-boot**]

d. 编译生成的 u-boot deb 包的包名

[o.k.] File name [**linux-u-boot-legacy-orangepi3b_1.0.0_arm64.deb**]

e. 编译使用的时间

[o.k.] Runtime [**1 min**]

f. 重复编译 u-boot 的命令，使用下面的命令无需通过图形界面选择，可以直接开始编译 u-boot

[o.k.] Repeat Build Options [**sudo ./build.sh BOARD=orangepi3b BRANCH=legacy BUILD_OPT=u-boot KERNEL_CONFIGURE=no**]

6) 查看编译生成的 u-boot deb 包

```
test@test:~/orangepi-build$ ls output/debs/u-boot/
linux-u-boot-legacy-orangepi3b_1.0.0_arm64.deb
```

7) 生成的 u-boot 的 deb 包包含的文件如下所示

a. 使用下面的命令可以解压 deb 包

```
test@test:~/orangepi-build$ cd output/debs/u-boot
test@test:~/orangepi_build/output/debs/u-boot$ $ dpkg -x \
linux-u-boot-legacy-orangepi3b_1.0.0_arm64.deb .    (注意命令最后有个“.”)
test@test:~/orangepi_build/output/debs/u-boot$ ls
linux-u-boot-legacy-orangepi3b_1.0.0_arm64.deb    usr
```

b. 解压后的文件如下所示

```
test@test:~/orangepi-build/output/debs/u-boot$ tree usr
```

```
usr
└── lib
    ├── linux-u-boot-legacy-orangepi3b_1.0.0_arm64
    │   ├── idbloader.img
    │   ├── rkspi_loader.img
    │   └── u-boot.itb
    └── u-boot
        ├── LICENSE
        ├── orangepi-3b-rk3566_defconfig
        └── platform_install.sh

3 directories, 6 files
```

8) orangepi-bulid 编译系统编译 u-boot 源码时首先会将 u-boot 的源码和 github 服务器的 u-boot 源码进行同步，所以如果想修改 u-boot 的源码，首先需要关闭源码的下载更新功能（**需要完整编译过一次 u-boot 后才能关闭这个功能，否则会提示找不到 u-boot 的源码，如果是从百度云盘下载的源码压缩包，则没有这个问题，因为 u-boot 的源码都已缓存好了**），否则所作的修改都会被还原，方法如下：

设置 userpatches/config-default.conf 中的 IGNORE_UPDATES 变量为“yes”

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
IGNORE_UPDATES="yes"
```

9) 调试 u-boot 代码时，可以使用下面的方法来更新 Linux 镜像中的 u-boot 进行测试

a. 将编译好的 u-boot 的 deb 包上传到开发板的 Linux 系统中

```
test@test:~/orangepi-build$ cd output/debs/u-boot
test@test:~/orangepi_build/output/debs/u-boot$ scp \
linux-u-boot-legacy-orangepi3b_1.0.0_arm64.deb root@192.168.1.xxx:/root
```

b. 然后登录到开发板，卸载已安装的 u-boot 的 deb 包

```
root@orangepi:~# apt purge -y linux-u-boot-orangepi3b-legacy
```

c. 再安装刚才上传的新的 u-boot 的 deb 包

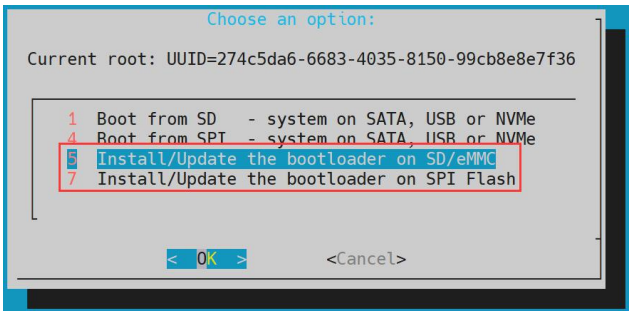
```
root@orangepi:~# dpkg -i linux-u-boot-legacy-orangepi3b_1.0.0_arm64.deb
```

d. 然后运行 nand-sata-install 脚本

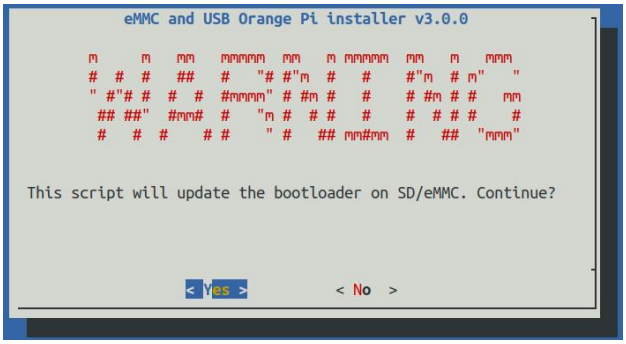
```
root@orangepi:~# nand-sata-install
```

e. 然后选择 **5 Install/Update the bootloader on SD/eMMC** 来更新 TF 卡中的 u-boot 或者 **7 Install/Update the bootloader on SPI Flash** 来更新 SPI Flash 中

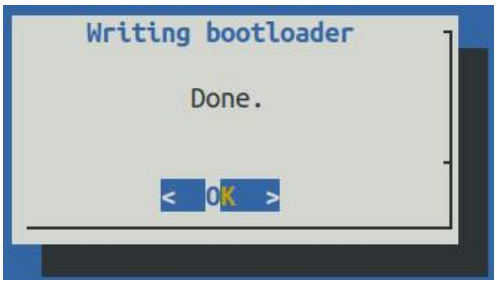
的 u-boot



f. 按下回车键后首先会弹出一个 Warning



g. 再按下回车键就会开始更新 u-boot，更新完后会显示下面的信息



h. 然后就可以重启开发板来测试 u-boot 的修改是否生效了

10) 其它有用的信息

a. u-boot 2017.09 源码中，开发板使用的 defconfig 配置文件为

orangepi-build/u-boot/v2017.09-rk3588/configs/orangepi-3b-rk3566_defconfig

b. u-boot 2017.09 源码中，开发板使用 dts 文件为

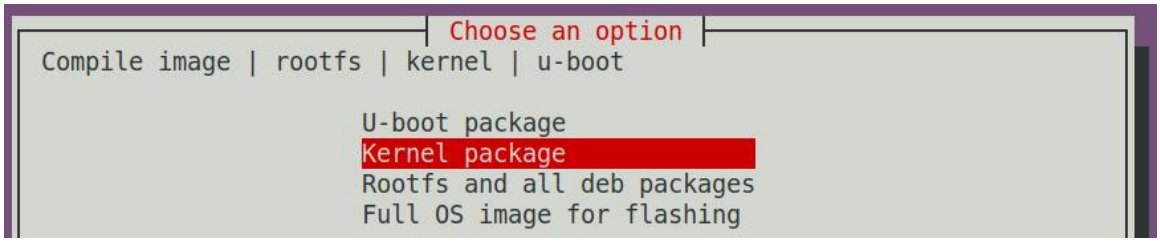
orangepi-build/u-boot/v2017.09-rk3588/arch/arm/dts/rk3566-orangepi-3b.dts

4. 4. 编译 Linux 内核

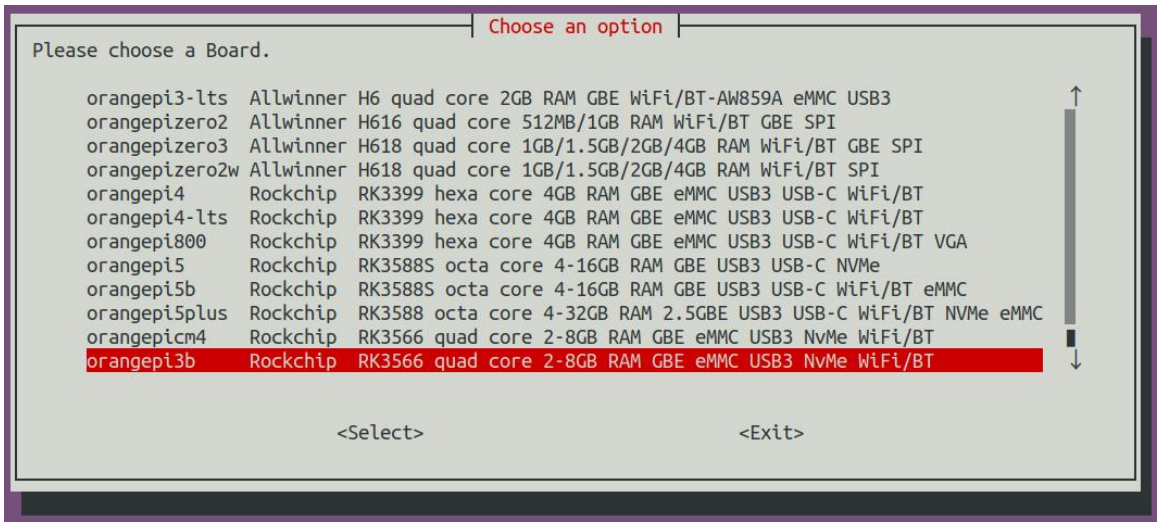
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

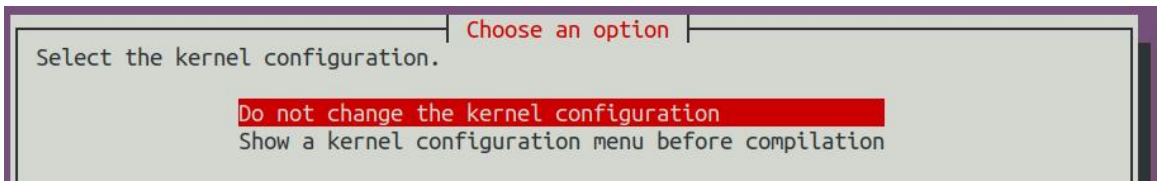
2) 选择 **Kernel package**，然后回车



3) 接着选择开发板的型号



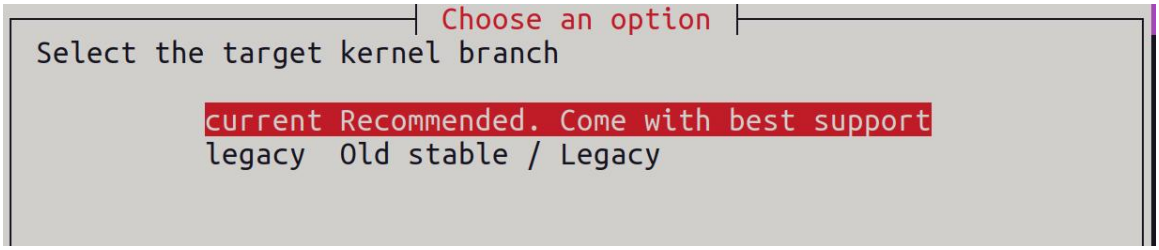
4) 然后会提示是否需要显示内核配置界面，如果不需要修改内核配置，则选择第一个即可，如果需要修改内核配置，则选择第二个



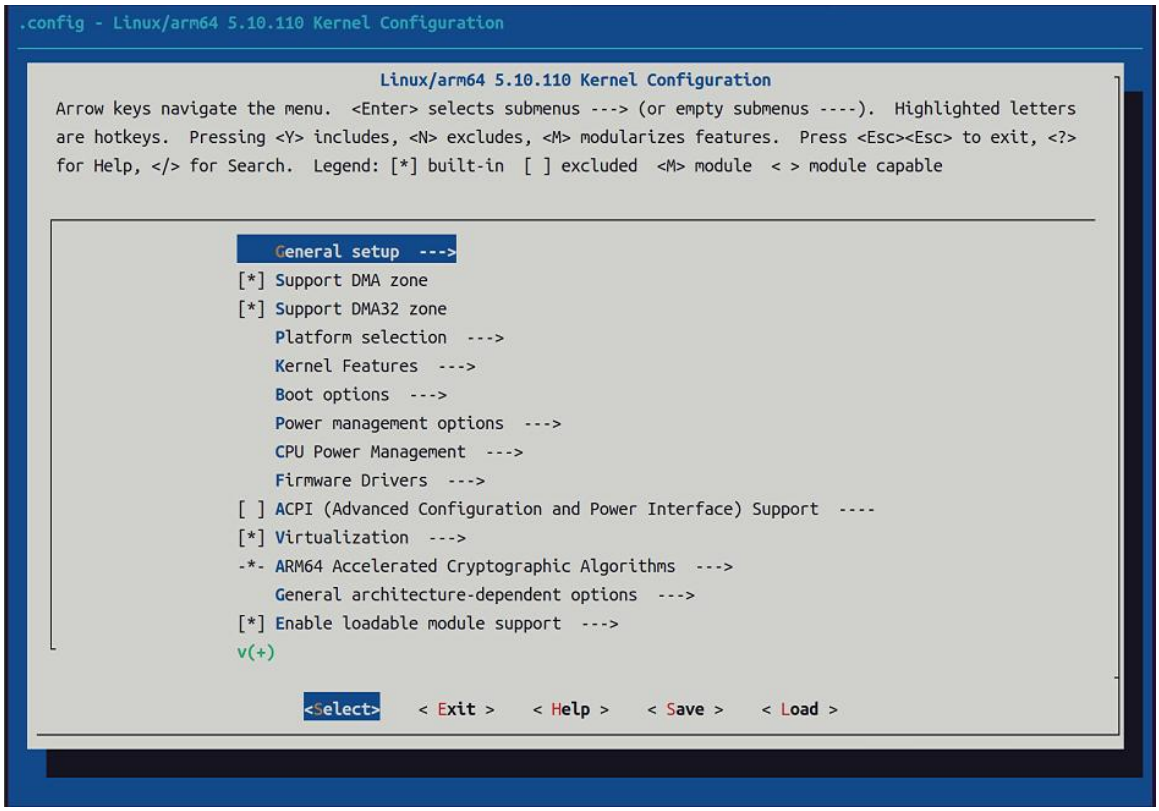
5) 然后选择内核源码的分支类型

a. legacy 分支会编译 linux5.10 内核源码

b. current 分支会编译 linux6.6 内核源码



6) 如果第 4) 步选择了需要显示内核配置菜单（第二个选项），则会弹出通过 **make menuconfig** 打开的内核配置的界面，此时可以直接修改内核的配置，修改完后再保存退出即可，退出后会开始编译内核源码



a. 如果不需要修改内核的配置选项，在运行 build.sh 脚本时，传入 **KERNEL_CONFIGURE=no** 就可临时屏蔽弹出内核的配置界面了

```
test@test:~/orangepi-build$ sudo ./build.sh KERNEL_CONFIGURE=no
```

b. 也可以设置 **orangepi-build/userpatches/config-default.conf** 配置文件中的 **KERNEL_CONFIGURE=no**，这样可以永久禁用这个功能

c. 编译内核的时候如果提示下面的错误，这是由于 Ubuntu PC 的终端界面太小，导致 **make menuconfig** 的界面无法显示，请把 Ubuntu PC 的终端调到最大，然后重新运行 build.sh 脚本

```
  HOSTCC  scripts/kconfig/mconf.o
  HOSTCC  scripts/kconfig/lxdialog/checklist.o
  HOSTCC  scripts/kconfig/lxdialog/util.o
  HOSTCC  scripts/kconfig/lxdialog/inputbox.o
  HOSTCC  scripts/kconfig/lxdialog/textbox.o
  HOSTCC  scripts/kconfig/lxdialog/yesno.o
  HOSTCC  scripts/kconfig/lxdialog/menubox.o
  HOSTLD  scripts/kconfig/mconf
scripts/kconfig/mconf  Kconfig
Your display is too small to run Menuconfig!
It must be at least 19 lines by 80 columns.
scripts/kconfig/Makefile:28: recipe for target 'menuconfig' failed
make[1]: *** [menuconfig] Error 1
Makefile:560: recipe for target 'menuconfig' failed
make: *** [menuconfig] Error 2
[ error ] ERROR in function compile_kernel [ compilation.sh:376 ]
[ error ] Error kernel menuconfig failed
[ o.k. ] Process terminated
```

7) 编译 legacy 分支内核源码时提示的部分信息说明如下

a. Linux 内核源码的版本

[o.k.] Compiling current kernel [**5.10.160**]

b. 使用的交叉编译工具链的版本

[o.k.] Compiler version [**aarch64-none-linux-gnu-gcc 11.2.1**]

c. 内核默认使用的配置文件以及它存放的路径

[o.k.] Using kernel config file [**config/kernel/linux-rockchip-rk356x-legacy.config**]

d. 编译生成的内核相关的 deb 包的路径

[o.k.] Target directory [**orangepi-build/output/debs/**]

e. 编译生成的内核镜像 deb 包的包名

[o.k.] File name [**linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb**]

f. 编译使用的时间

[o.k.] Runtime [**5 min**]

g. 最后会显示重复编译上一次选择的内核的编译命令，使用下面的命令无需通过图形界面选择，可以直接开始编译内核源码

[o.k.] Repeat Build Options [**sudo ./build.sh BOARD=orangepi3b BRANCH=legacy BUILD_OPT=kernel KERNEL_CONFIGURE=no**]

8) 查看编译生成的内核相关的 deb 包

a. **linux-dtb-legacy-rockchip-rk356x_1.0.0_arm64.deb** 包含内核使用的 dtb 文件

b. **linux-headers-legacy-rockchip-rk356x_1.0.0_arm64.deb** 包含内核头文件

c. **linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb** 包含内核镜像和内核模块

```
test@test:~/orangepi-build$ ls output/debs/linux-*
output/debs/linux-dtb-legacy-rockchip-rk356x_1.0.0_arm64.deb
```

```
output/debs/linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb
output/debs/linux-headers-legacy-rockchip-rk356x_1.0.0_arm64.deb
```

9) 生成的 linux-image 的 deb 包包含的文件如下所示

a. 使用下面的命令可以解压 deb 包

```
test@test:~/orangepi-build$ cd output/debs
test@test:~/orangepi_build/output/debs$ mkdir test
test@test:~/orangepi_build/output/debs$ cp \
linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb test/
test@test:~/orangepi_build/output/debs$ cd test
test@test:~/orangepi_build/output/debs/test$ dpkg -x \
linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb .
test@test:~/orangepi_build/output/debs/test$ ls
boot  etc  lib  linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb  usr
```

b. 解压后的文件如下所示

```
test@test:~/orangepi-build/output/debs/test$ tree -L 2
.
├── boot
│   ├── config-5.10.160-rockchip-rk356x
│   ├── System.map-5.10.160-rockchip-rk356x
│   └── vmlinuz-5.10.160-rockchip-rk356x
├── etc
│   └── kernel
├── lib
│   └── modules
├── linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb
└── usr
    ├── lib
    └── share
```

10) orangepi-bulid 编译系统编译 Linux 内核源码时首先会将 Linux 内核源码和 github 服务器的 Linux 内核源码进行同步，所以如果想修改 Linux 内核的源码，首先需要关闭源码的更新功能（**需要完整编译过一次 Linux 内核源码后才能关闭这个功能，否则会提示找不到 Linux 内核的源码，如果是从百度云盘下载的源码压缩包，则没有这个问题，因为 Linux 的源码都已缓存好了**），否则所作的修改都会被还原，方

法如下：

设置 **userpatches/config-default.conf** 中的 IGNORE_UPDATES 变量为“yes”

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
IGNORE_UPDATES="yes"
```

11) 如果对内核做了修改，可以使用下面的方法来更新开发板 Linux 系统的内核和内核模块

a. 将编译好的 Linux 内核的 deb 包上传到开发板的 Linux 系统中

```
test@test:~/orangepi-build$ cd output/debs
test@test:~/orangepi-build/output/debs$ scp \
linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb root@192.168.1.xxx:/root
```

b. 然后登录到开发板，卸载已安装的 Linux 内核的 deb 包

```
root@orangepi:~# apt purge -y linux-image-legacy-rockchip-rk356x
```

c. 再安装刚才上传的新的 Linux 内核的 deb 包

```
root@orangepi:~# dpkg -i linux-image-legacy-rockchip-rk356x_1.0.0_arm64.deb
```

d. 然后重启开发板，再查看内核相关的修改是否已生效

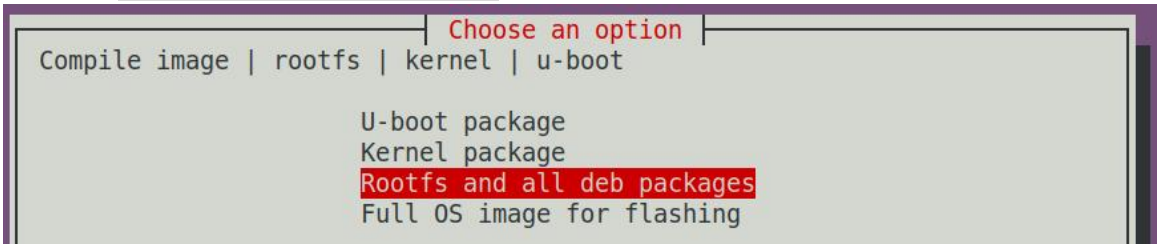
```
root@orangepi:~# reboot
```

4.5. 编译 rootfs

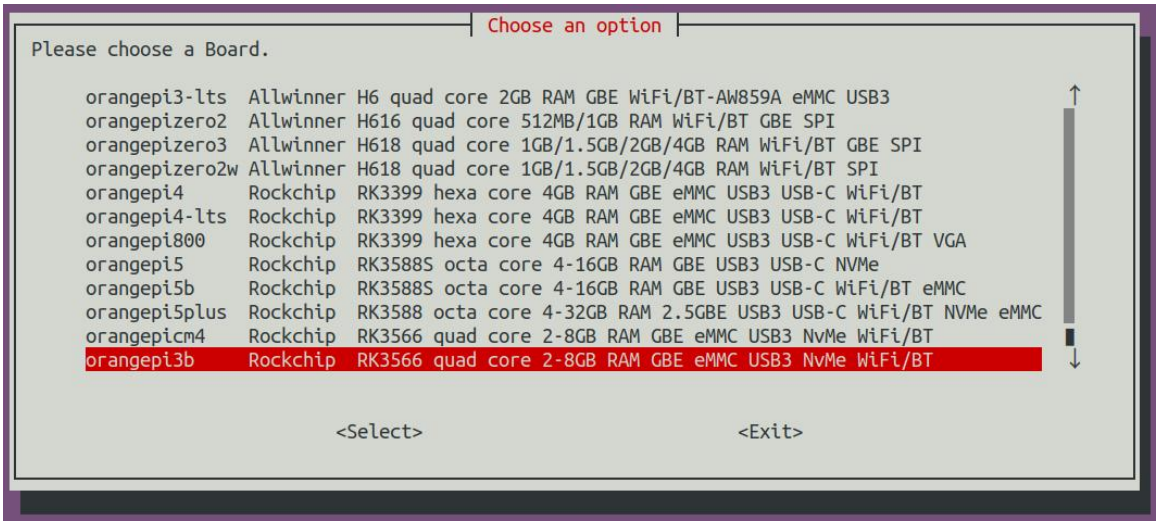
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

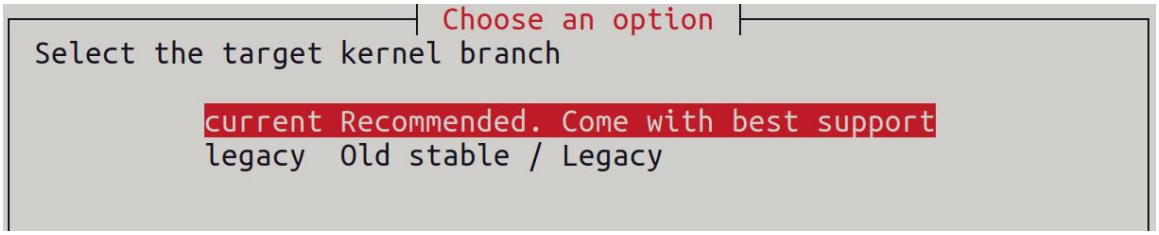
2) 选择 **Rootfs and all deb packages**，然后回车



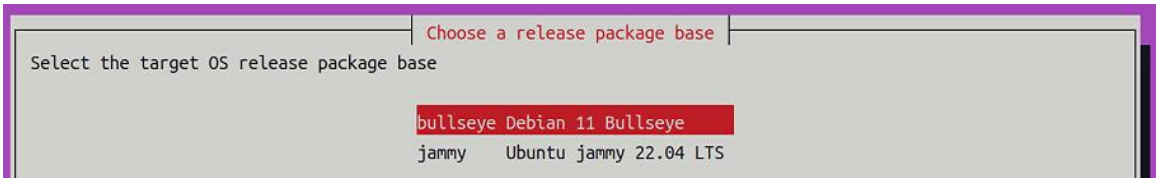
3) 接着选择开发板的型号



4) 然后选择内核源码的分支类型，目前内核源码维护的 rootfs 使用同一套

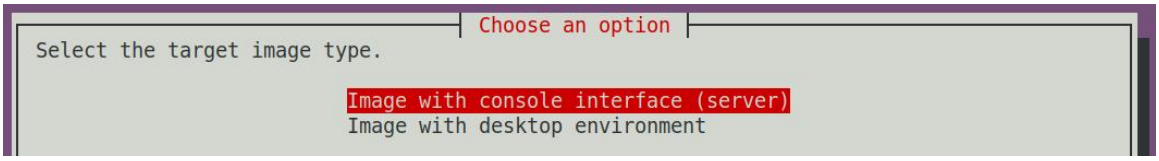


5) 然后选择 rootfs 的类型

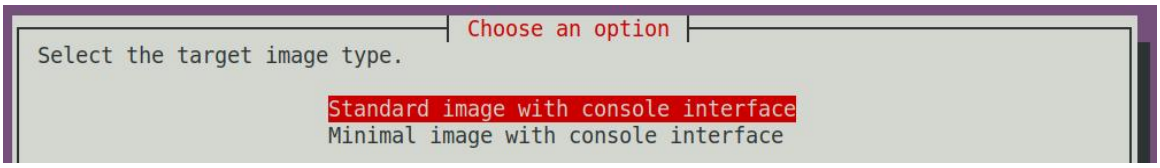


6) 然后选择镜像的类型

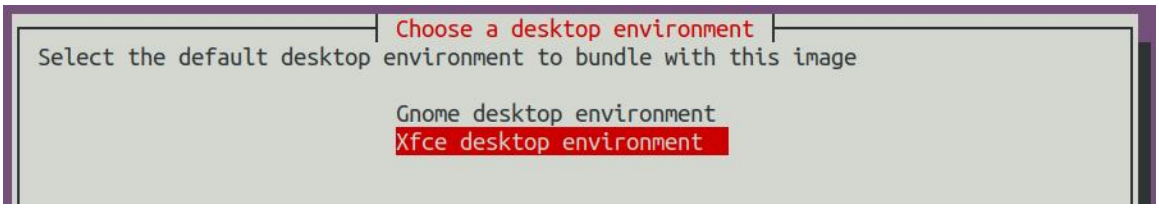
a. **Image with console interface (server)**表示服务器版的镜像，体积比较小
b. **Image with desktop environment** 表示带桌面的镜像，体积比较大

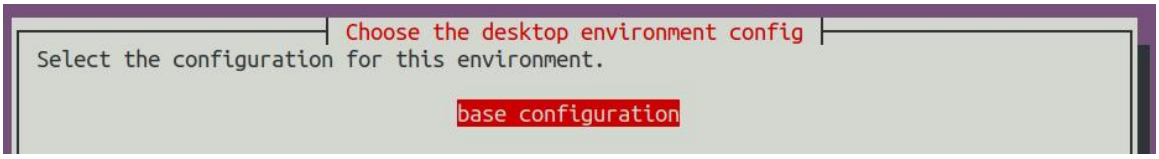


7) 如果是编译服务器版的镜像，还可以选择编译 Standard 版本或者 Minimal 版本，Minimal 版本预装的软件会比 Standard 版本少很多（**没特殊需求请不要选择 Minimal 版本，因为很多东西默认没有预装，部分功能可能用不了**）

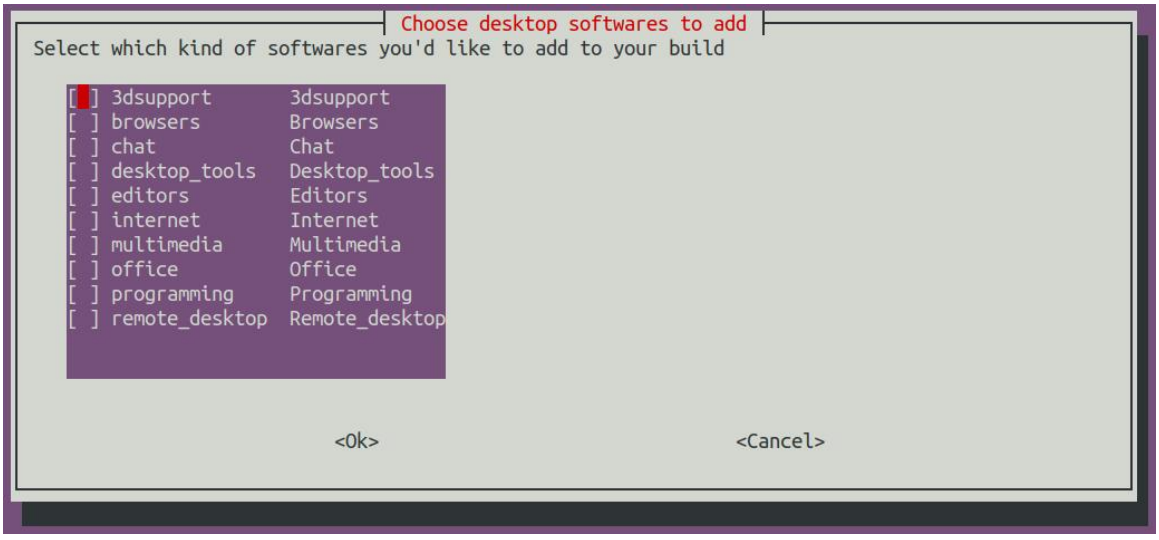


8) 如果是编译桌面版本的镜像还需要选择桌面环境的类型，目前 Ubuntu Jammy 主要维护 XFCE 和 Gnome 两种桌面，Ubuntu Focal 只维护 XFCE 桌面，Debian Bullseye 主要维护 XFCE 和 KDE 桌面





然后可以选择需要安装的额外的软件包。这里请按下回车键直接跳过。



9) 然后就会开始编译 rootfs，编译时提示的部分信息说明如下所示

a. rootfs 的类型

```
[ o.k. ] local not found [ Creating new rootfs cache for jammy]
```

b. 编译生成的 rootfs 压缩包的存放路径

```
[ o.k. ] Target directory [ external/cache/rootfs ]
```

c. 编译生成的 rootfs 压缩包的名字

```
[ o.k. ] File name [ jammy-xfce-arm64.f930ff6ebbac1a72108a2e100762b18f.tar.lz4 ]
```

d. 编译使用的时间

```
[ o.k. ] Runtime [ 13 min ]
```

10) 查看编译生成的 rootfs 压缩包

a. **jammy-xfce-arm64.f930ff6ebbac1a72108a2e100762b18f.tar.lz4** 是 rootfs 的压缩包，名字各字段的含义为

a) **jammy** 表示 rootfs 的 Linux 发行版的类型

b) **xfce** 表示 rootfs 为桌面版的类型，如果为 **cli** 则表示服务器版类型

c) **arm64** 表示 rootfs 的架构类型

d) **f930ff6ebbac1a72108a2e100762b18f** 是由 rootfs 安装的所有软件包的包名生成的 MD5 哈希值，只要没有修改 rootfs 安装的软件包的列表，那么这个值就不会变，编译脚本会通过这个 MD5 哈希值来判断是否需要重新编译 rootfs

b. **jammy-xfce-arm64.f930ff6ebbac1a72108a2e100762b18f.tar.lz4.list** 列出了 rootfs 安装的所有软件包的包名

```
test@test:~/orangepi-build$ ls external/cache/rootfs/
jammy-xfce-arm64.f930ff6ebbac1a72108a2e100762b18f.tar.lz4
jammy-xfce-arm64.f930ff6ebbac1a72108a2e100762b18f.tar.lz4.current
jammy-xfce-arm64.f930ff6ebbac1a72108a2e100762b18f.tar.lz4.list
```

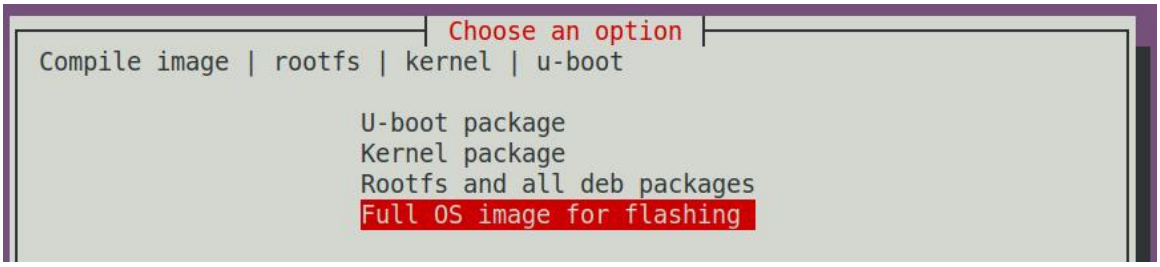
11) 如果需要的 rootfs 在 **external/cache/rootfs** 下已经存在，那么再次编译 rootfs 就会直接跳过编译过程，不会重新开始编译，编译镜像的时候也会去 **external/cache/rootfs** 下查找是否已经有缓存可用的 rootfs，如果有就直接使用，这样可以节省大量的下载编译时间。

4.6. 编译 Linux 镜像

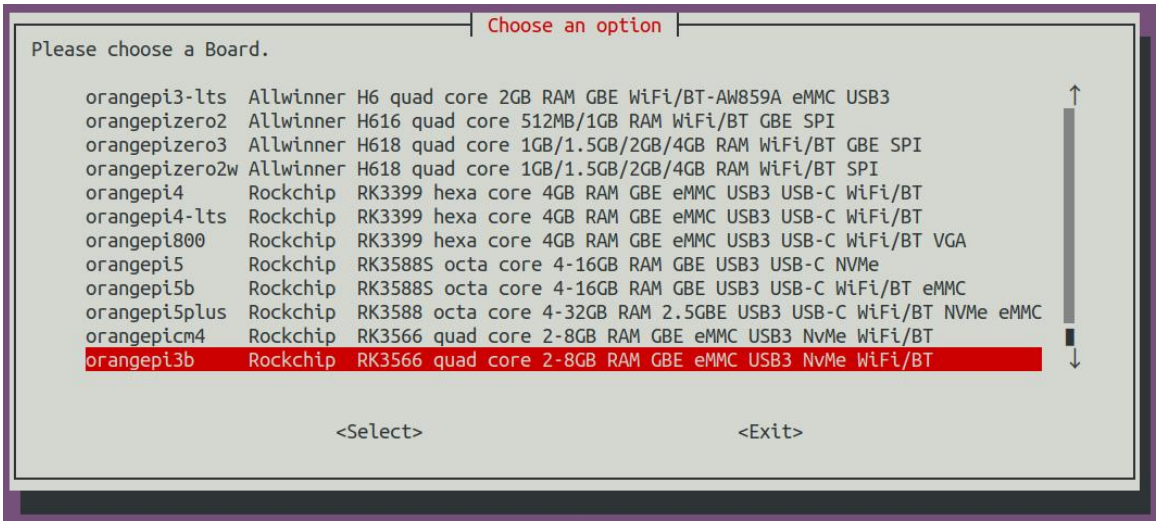
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

2) 选择 **Full OS image for flashing**，然后回车



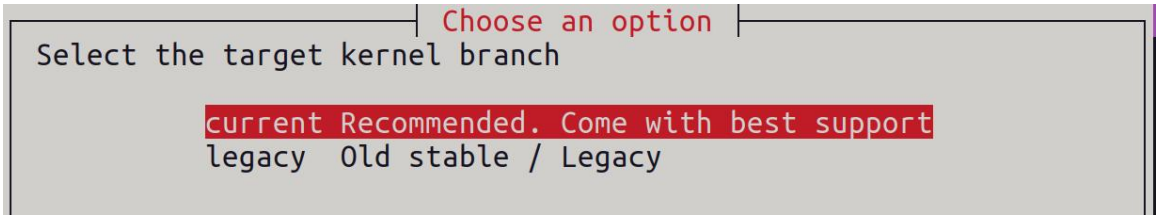
3) 然后选择开发板的型号



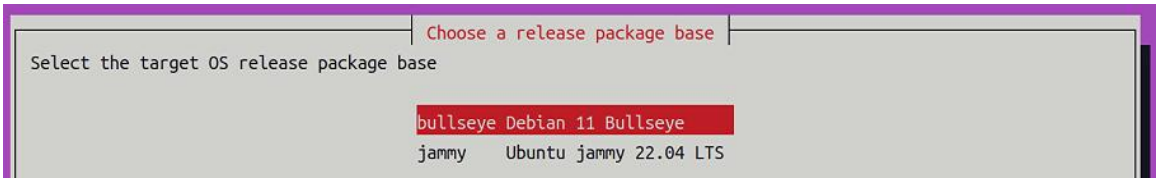
4) 然后选择内核源码的分支类型

a. legacy 分支会编译 linux5.10 内核源码

b. current 分支会编译 linux6.6 内核源码

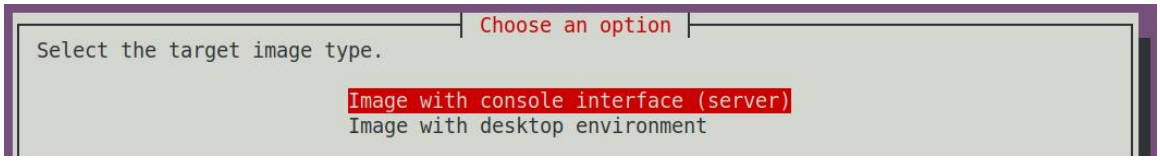


5) 然后选择 rootfs 的类型

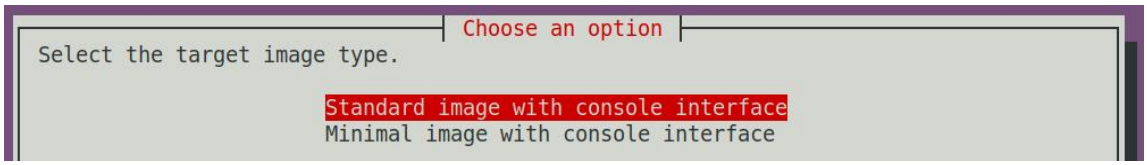


6) 然后选择镜像的类型

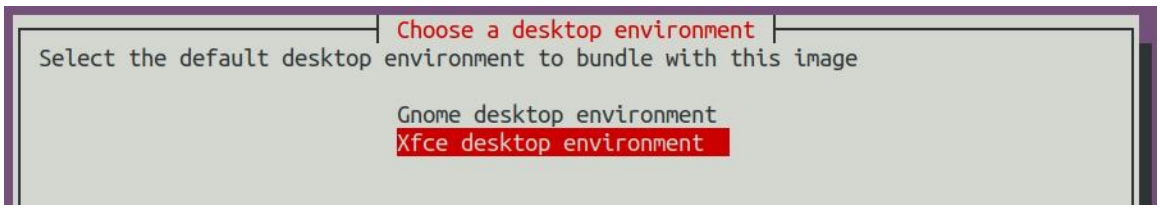
a. **Image with console interface (server)**表示服务器版的镜像，体积比较小
b. **Image with desktop environment** 表示带桌面的镜像，体积比较大

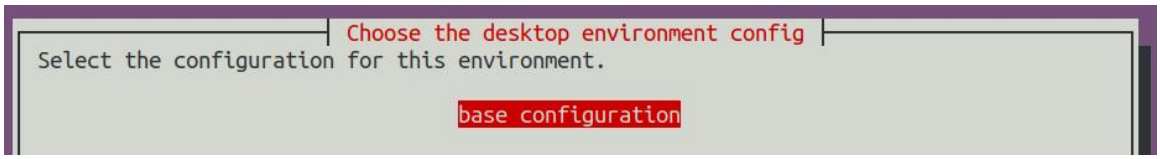


7) 如果是编译服务器版的镜像，还可以选择编译 Standard 版本或者 Minimal 版本，Minimal 版本预装的软件会比 Standard 版本少很多（**没特殊需求请不要选择 Minimal 版本，因为很多东西默认没有预装，部分功能可能用不了**）

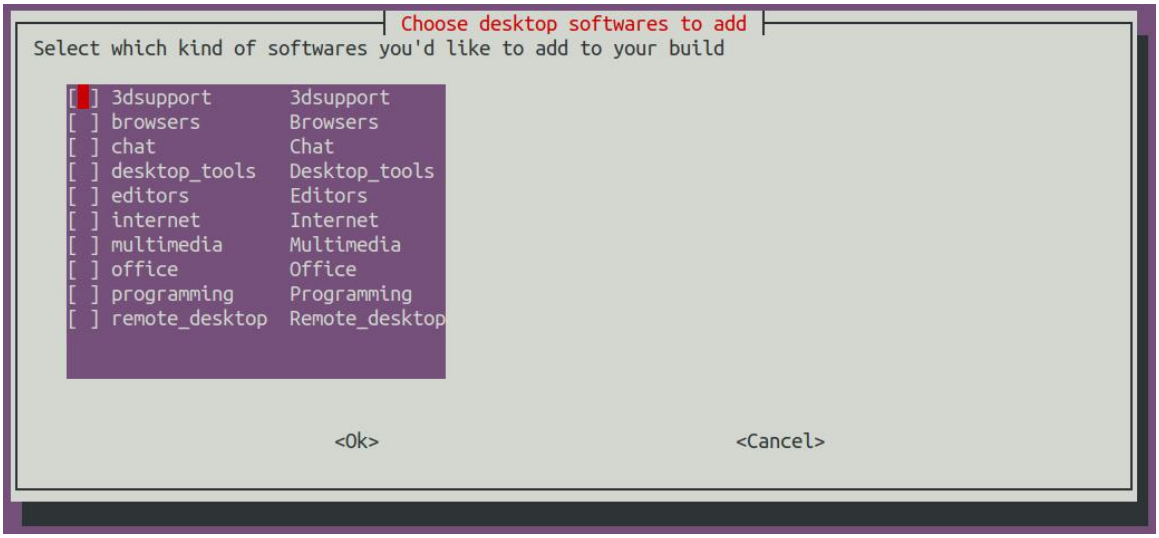


8) 如果是编译桌面版本的镜像还需要选择桌面环境的类型，目前 Ubuntu Jammy 主要维护 XFCE 和 Gnome 两种桌面，Ubuntu Focal 只维护 XFCE 桌面，Debian Bullseye 主要维护 XFCE 和 KDE 桌面





然后可以选择需要安装的额外的软件包。这里请按下回车键直接跳过。



9) 然后就会开始编译 Linux 镜像，编译的大致流程如下

a. 初始化 Ubuntu PC 的编译环境，安装编译过程需要的软件包
b. 下载 u-boot 和 Linux 内核的源码（如果已经缓存，则只更新代码）
c. 编译 u-boot 源码，生成 u-boot 的 deb 包
d. 编译 Linux 源码，生成 Linux 相关的 deb 包
e. 制作 Linux firmware 的 deb 包
f. 制作 orangepi-config 工具的 deb 包
g. 制作板级支持的 deb 包
h. 如果是编译 desktop 版镜像，还会制作 desktop 相关的 deb 包
i. 检查 rootfs 是否已经缓存，如果没有缓存，则重新制作 rootfs，如果已经缓存，则直接解压使用
j. 安装前面生成的 deb 包到 rootfs 中
k. 对不同的开发板和不同类型镜像做一些特定的设置，如预装额外的软件包，修改系统配置等
l. 然后制作镜像文件，并格式化分区，默认类型为 ext4
m. 再将配置好的 rootfs 拷贝到镜像的分区中
n. 然后更新 initramfs
o. 最后将 u-boot 的 bin 文件通过 dd 命令写入到镜像中

10) 编译完镜像后会提示下面的信息

a. 编译生成的镜像的存放路径

```
[ o.k. ] Done building
[ output/images/Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160/Or
```

angepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.img]

b. 编译使用的时间

[o.k.] Runtime [19 min]

c. 重复编译镜像的命令，使用下面的命令无需通过图形界面选择，可以直接开始编译镜像

[o.k.] Repeat Build Options [**sudo ./build.sh BOARD=orangepi3b BRANCH=legacy BUILD_OPT=image RELEASE=bullseye BUILD_MINIMAL=no BUILD_DESKTOP=no KERNEL_CONFIGURE=yes**]

5. Orange Pi OS Arch 系统的使用说明

5.1. Orange Pi OS Arch 系统功能适配情况

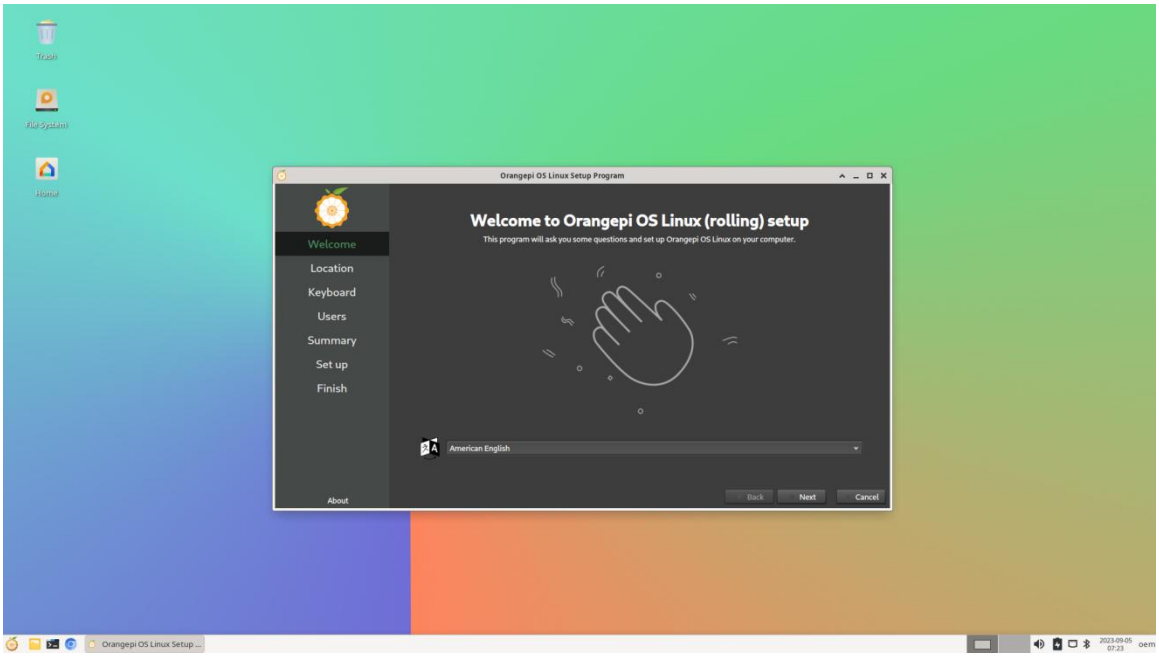
功能	OPi OS Arch
USB2.0x3	OK
USB3.0x1	OK
SPIFlash+M.2 NVMe SSD 启动	OK
WIFI	OK
蓝牙	OK
GPIO（40pin）	OK
UART（40pin）	OK
SPI（40pin）	OK
I2C（40pin）	OK
PWM（40pin）	OK
3pin 调试串口	OK
eMMC 启动	OK
TF 卡启动	OK
HDMI 视频	OK
HDMI 音频	OK
树莓派 5 寸屏幕显示	OK
树莓派 5 寸屏幕触摸功能	OK
eDP 显示	OK
OV5647 摄像头	内核驱动 OK，3A 未调
千兆网口	OK
网口状态灯	OK
耳机播放	OK
耳机录音	OK
LED 灯	OK
GPU	NO
NPU	NO
VPU	NO

5.2. Orange Pi OS Arch 系统用户向导使用说明

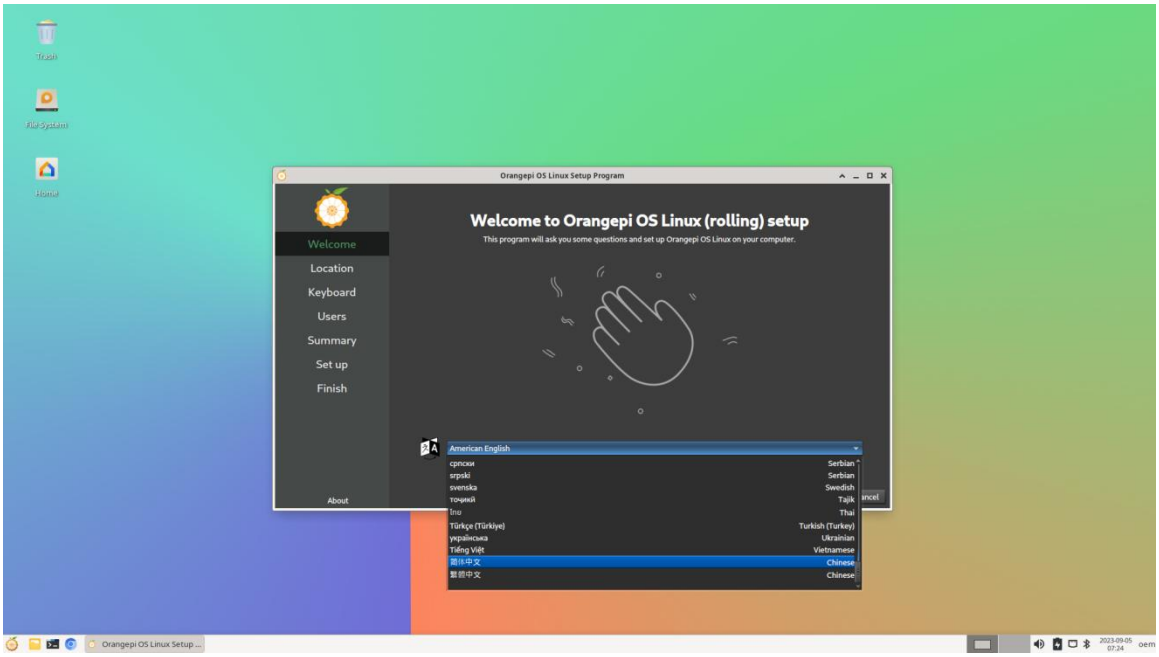
首先请注意，OPi OS Arch 系统是没有设置默认的 orangepi 用户和密码的，所以烧录完系统启动后是无法直接通过串口和 ssh 远程登录的（root 用户也不行）。这一点和 Ubuntu、Debian 系统是有区别的。

OPi OS Arch 系统第一次启动时需要接上 HDMI 显示器，然后通过用户向导来初始化系统设置（其中包括新建用户名和设置密码）。用户向导的设置步骤如下所示：

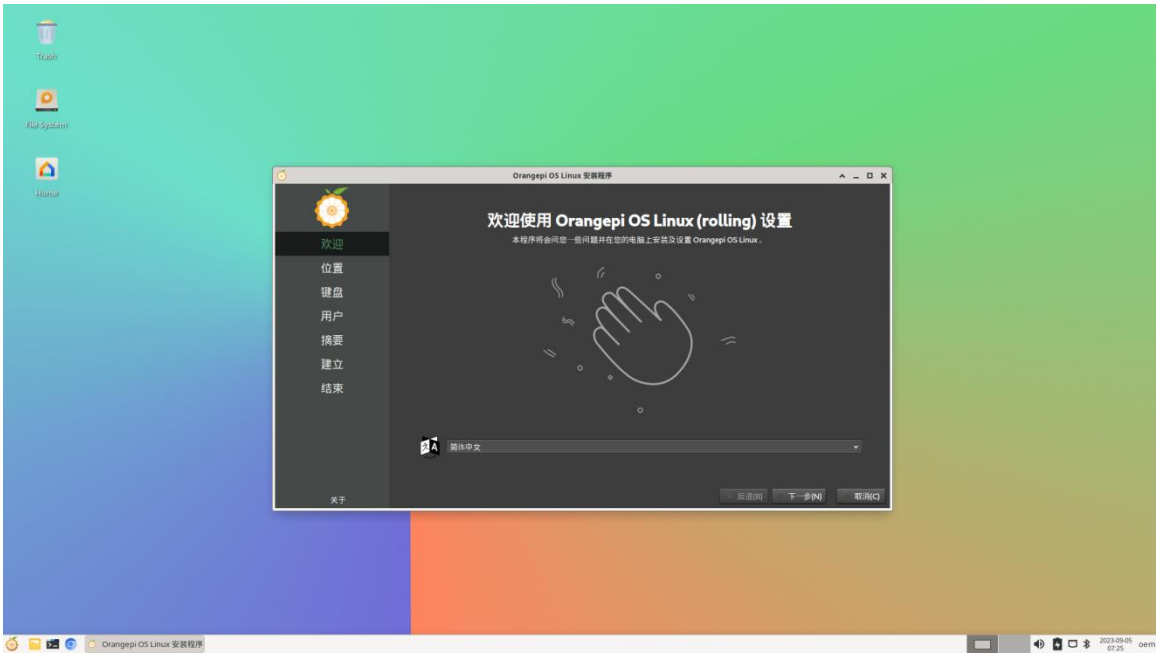
a) 烧录完系统第一次启动进入桌面后会看到下图所示的用户向导程序



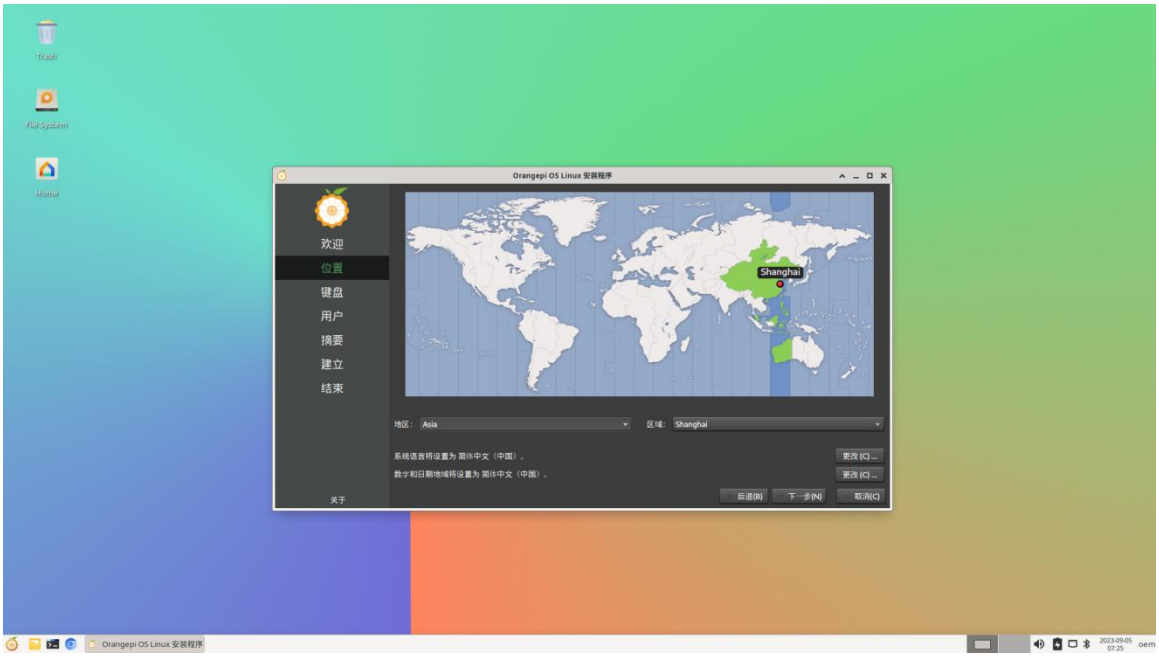
b) 首先需要选择想要语言



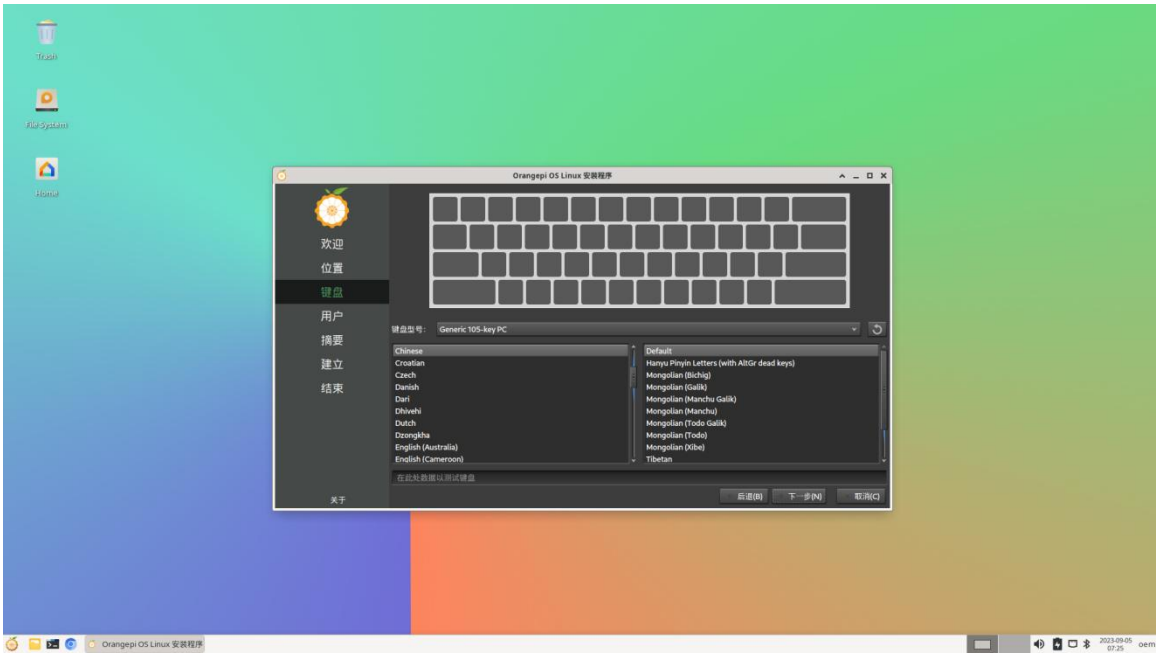
c) 在选择完语言后，用户向导会立即切换为对应的语言界面，如中文显示如下所示



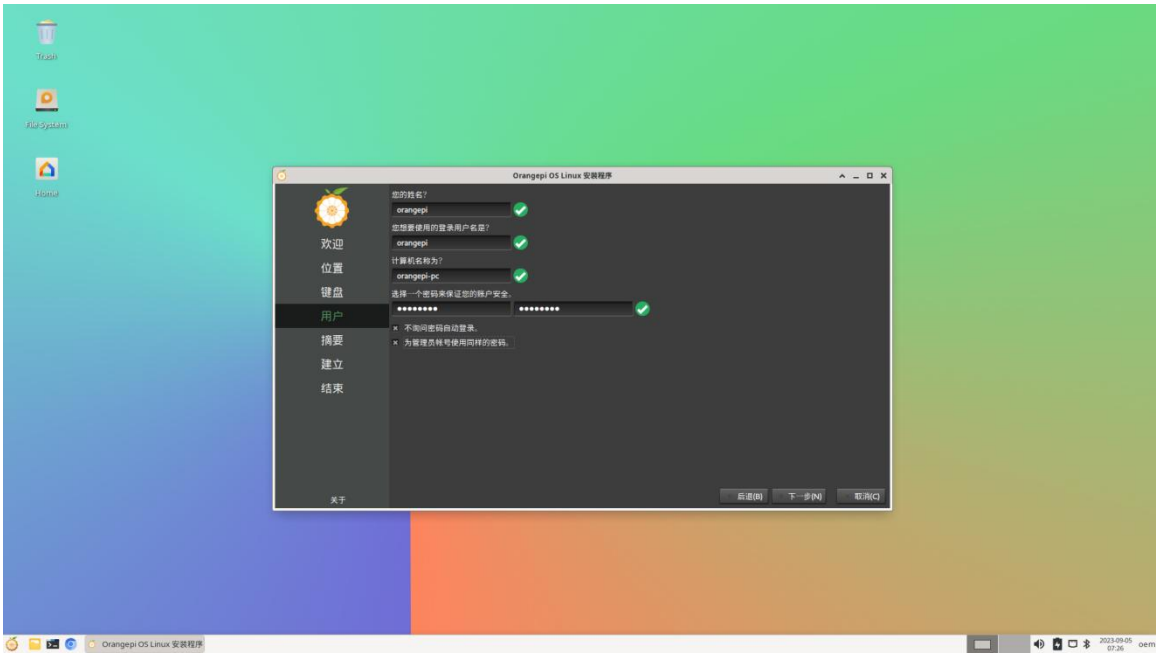
d) 然后选择区域



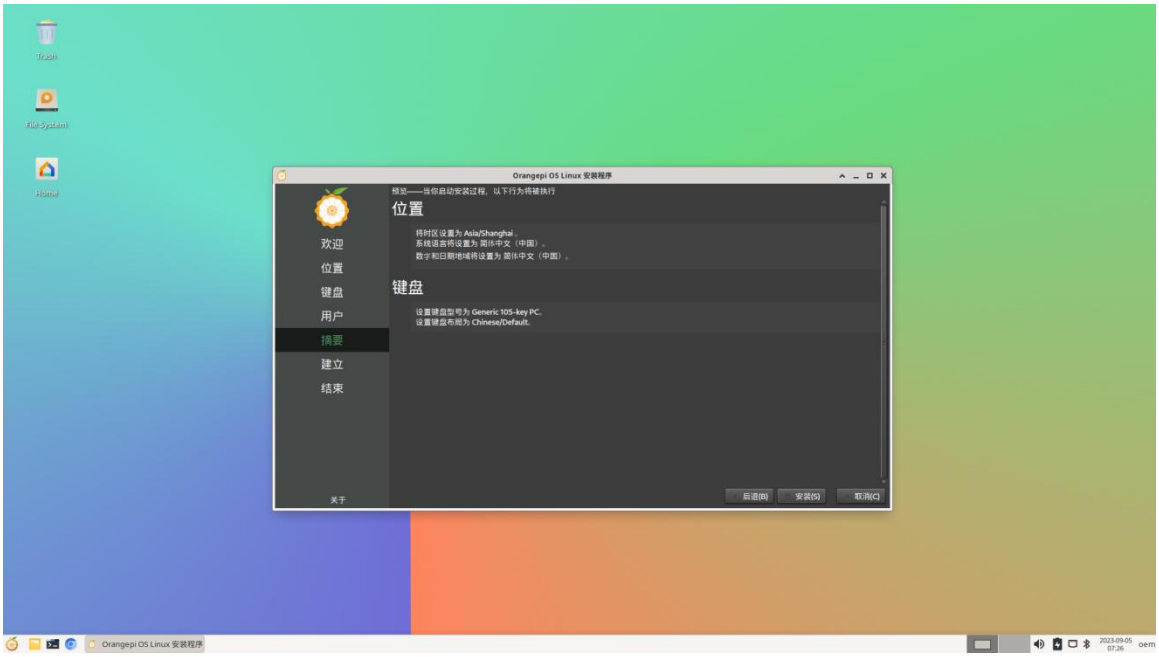
e) 然后选择键盘型号



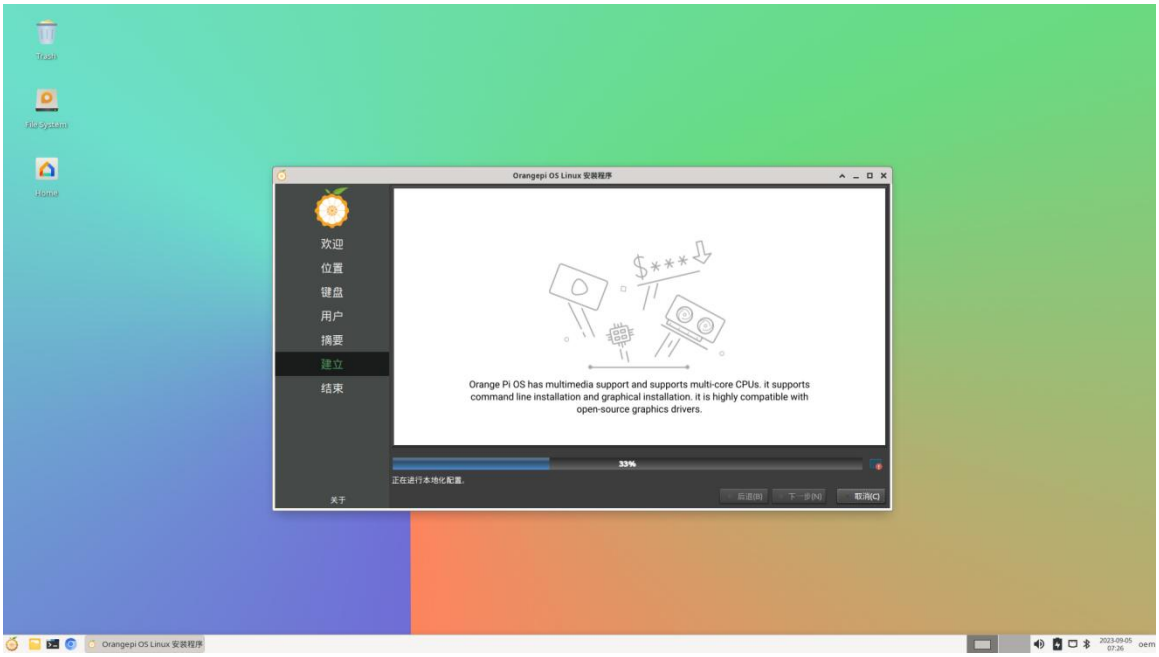
f) 然后新建用户名和设置密码



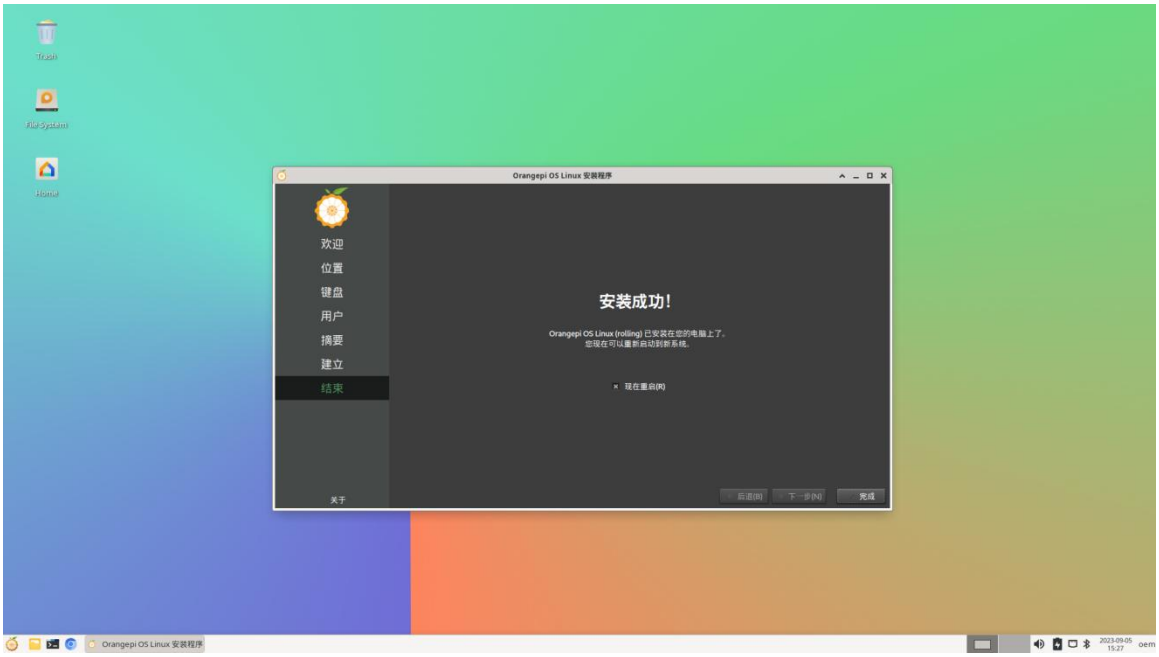
g) 然后确保选择没问题后，再点击安装按钮



h) 然后等待安装完成



i) 安装完成需要点击**完成**按钮重启系统



j) 重启后会自动启动 Orange Pi Hello 程序，此时需要去掉右下角**开机时启动**的勾选状态，不然每次启动都需要手动关闭 Orange Pi Hello 程序

此时就可以使用刚才新建的用户名和密码通过串口或者 ssh 登录 OPi OS 系统了。

5.3. 设置 DT overlays 的方法

LCD MIPI 屏幕、eDP 屏幕和 40pin 中的 I2C/SPI/UART/PWM 等复用功能默认在内核的 dts 中都是关闭的，需要手动打开对应的 DT overlays 才能使用。

在 OPi OS Arch 系统中打开 DT overlays 的方法如下所示：

1) 首先打开**/boot/extlinux/extlinux.conf** 配置文件

```
[orangepi@orangepi-pc ~]$ sudo vim /boot/extlinux/extlinux.conf
```

2) 然后在**/boot/extlinux/extlinux.conf** 中通过添加 **FDTOVERLAYS /dtbs/rockchip/overlay/xxx.dtbo** 来打开对应的配置

注意 FDTOVERLAYS /dtbs/rockchip/overlay/xxx.dtbo 中的 xxx.dtbo 需要替换为具体的 dtbo 配置，请不要照抄。

```
[orangepi@orangepi-pc ~]$ sudo vim /boot/extlinux/extlinux.conf
LABEL OPIOS ARM
LINUX /Image
```

```
FDT /dtbs/rockchip/rk3566-orangepi-3b.dtb
FDTOVERLAYS /dtbs/rockchip/overlay/xxx.dtbo      #需要添加的配置
```

3) xxx.dtbo 在 OPi OS Arch 镜像中的存放路径如下所示，请注意，此路径下面不是所有的 dtbo 都可以使用的。

```
/boot/dtbs/rockchip/overlay/
```

4) 开发板可以使用的 DT overlays 配置如下所示

开发板上的功能	对应的 DT overlays 配置
EDP 屏幕	rk356x-edp.dtbo
树莓派 5 寸屏幕	rk356x-raspi-7inch-touchscreen.dtbo
40pin 扩展接口 - I2C2	rk356x-i2c2-m1.dtbo
40pin 扩展接口 - I2C3	rk356x-i2c3-m0.dtbo
40pin 扩展接口 - I2C4	rk356x-i2c4-m0.dtbo
40pin 扩展接口 - PWM11	rk356x-pwm11-m1.dtbo
40pin 扩展接口 - PWM15	rk356x-pwm15-m1.dtbo
40pin 扩展接口 - UART3	rk356x-uart3-m0.dtbo
40pin 扩展接口 - UART7	rk356x-uart7-m2.dtbo
40pin 扩展接口 - UART9	rk356x-uart9-m2.dtbo
40pin 扩展接口 - SPI3	rk356x-spi3-m0-cs0-spidev.dtbo

5) 如果需要同时打开多个配置，直接在 **FDTOVERLAYS** 后面将多个配置的路径加上即可，比如同时打开 i2c2 和 pwm11 的配置如下所示

```
[orangepi@orangepi-pc ~]$ sudo vim /boot/extlinux/extlinux.conf
LABEL OPIOS ARM
LINUX /Image
FDT /dtbs/rockchip/rk3566-orangepi-3b.dtb
FDTOVERLAYS /dtbs/rockchip/overlay/rk356x-i2c2-m1.dtbo  /dtbs/rockchip/overlay/rk356x-pwm11-m1.dtbo
```

6) 设置好后需要重启系统才能让配置生效

```
[orangepi@orangepi-pc ~]$ sudo reboot
```

5.4. 树莓派 5 寸屏幕的使用

5.4.1. 树莓派 5 寸屏幕的组装方法

请参考**树莓派 5 寸屏幕的组装方法**（**点击蓝色部分的文字即可跳转到对应位置**）。

5.4.2. 打开树莓派 5 寸屏幕配置的方法

OPi OS Arch 镜像默认是没有打开树莓派 5 寸屏幕的配置的，如果需要使用树莓派 5 寸屏幕，需要手动打开才行。打开配置的方法如下所示：

a. 首先在**/boot/extlinux/extlinux.conf** 中加入下面的配置

```
[orangepi@orangepi-pc ~]$ sudo vim /boot/extlinux/extlinux.conf
LABEL OPIOS ARM
LINUX /Image
FDT /dtbs/rockchip/rk3566-orangepi-3b.dtb
FDTOVERLAYS /dtbs/rockchip/overlay/rk356x-raspi-7inch-touchscreen.dtbo          #需要添加的配置
```

b. 然后重启系统

```
[orangepi@orangepi-pc ~]$ sudo reboot
```

重启后可以看到 LCD 屏幕的显示如下所示：

5.5. eDP 屏幕的使用方法

5.5.1. eDP 屏幕的组装方法

请参考 **eDP 屏幕的使用方法**（**点击蓝色部分的文字即可跳转到对应位置**）。

5.5.2. 打开 eDP 屏幕配置的方法

OPi OS Arch 镜像默认是没有打开 eDP 屏幕配置的，如果想使用 eDP 屏幕，需要手动打开才行。打开配置的方法如下所示：

a. 首先在**/boot/extlinux/extlinux.conf** 中加入下面的配置

```
[orangepi@orangepi-pc ~]$ sudo vim /boot/extlinux/extlinux.conf
LABEL OPIOS ARM
LINUX /Image
FDT /dtbs/rockchip/rk3566-orangepi-3b.dtb
FDTOVERLAYS /dtbs/rockchip/overlay/rk356x-edp.dtbo        #需要添加的配置
```

b. 然后重启系统

```
[orangepi@orangepi-pc ~]$ sudo reboot
```

重启后可以看到 eDP 屏幕的显示如下所示：

5.6. 安装软件的方法

使用 pacman 包管理工具可以安装 OPi OS 中没有的软件，比如安装 vim 编辑器的命令如下所示，如果想安装其他软件，只需要把 vim 替换想要安装的软件的包名即可。

```
[orangepi@orangepi-pc ~]$ sudo pacman -Syy vim
```

6. Orange Pi OS OH 系统的使用说明

6.1. Orange Pi OS OH 系统功能适配情况

功能	OPi OS OH
USB2.0x3	OK
USB3.0x1	OK
SPIFlash+M.2 NVMe SSD 启动	NO
WIFI	OK
蓝牙	NO
3pin 调试串口	OK
eMMC 启动	OK
TF 卡启动	OK
HDMI 视频	OK
HDMI 音频	NO
树莓派 5 寸屏幕显示	NO
树莓派 5 寸屏幕触摸功能	NO
eDP 显示	NO
OV5647 摄像头	NO
千兆网口	OK
网口状态灯	OK
耳机播放	OK
耳机录音	NO
LED 灯	OK
GPU	OK
NPU	NO
VPU	NO

目前 8GB 内存版本的开发板在 OPi OS OH 系统中只能使用 4GB 大小的内存，这点请特别注意。

6.2. 千兆网口的使用方法

1) 首先使用网线连接开发板和路由器

2) 然后在调试串口中通过 ifconfig 命令就能看到路由器给开发板网口分配的 IP 地址

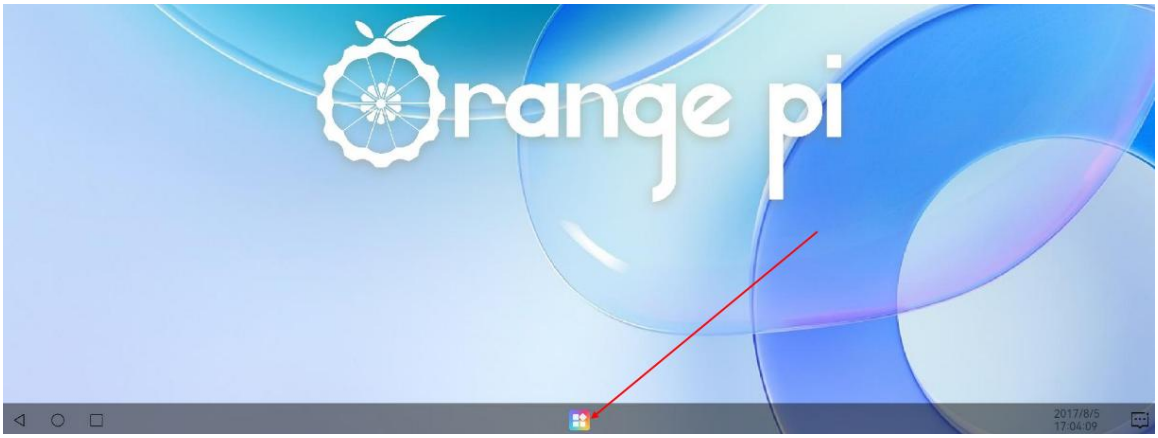
```
# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 4e:fc:9d:f3:67:26  Driver rk_gmac-dwmac
          inet addr:192.168.1.189  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::4cfc:9dff:fef3:6726/64 Scope: Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:378 errors:0 dropped:0 overruns:0 frame:0
          TX packets:24 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:30663 TX bytes:2904
          Interrupt:45

#
```

3) 然后使用 ping 命令可以测试网络是否能正常使用

```
# ping www.orangepi.cn
Ping www.orangepi.cn (123.57.147.237): 56(84) bytes.
64 bytes from 123.57.147.237: icmp_seq=1 ttl=0 time=42 ms
64 bytes from 123.57.147.237: icmp_seq=2 ttl=0 time=43 ms
64 bytes from 123.57.147.237: icmp_seq=3 ttl=0 time=44 ms

--- 123.57.147.237 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
round-trip min/avg/max = 0/0/43 ms
#
```

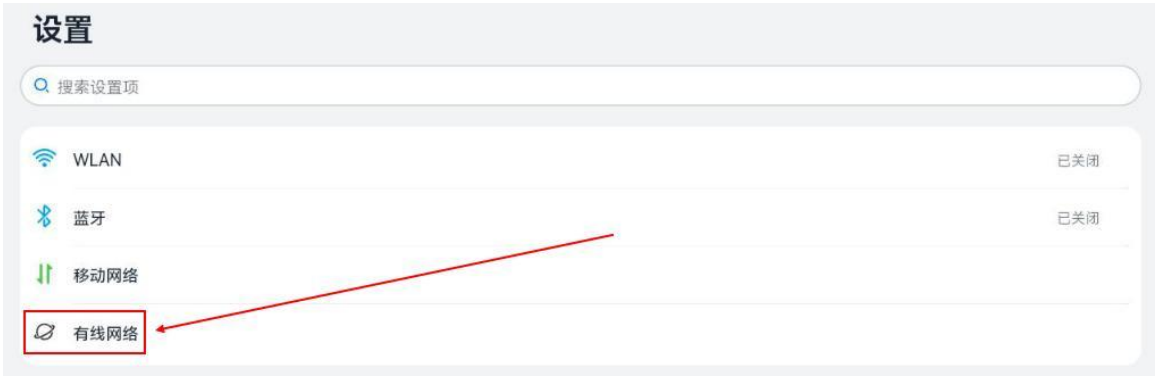
4) 除了在命令行中查看网口的 IP 地址外，还可以在 OH 的设置中查看网口的 IP 地址，方法如下所示

   a. 首先点击**应用列表**



b. 然后打开**设置**

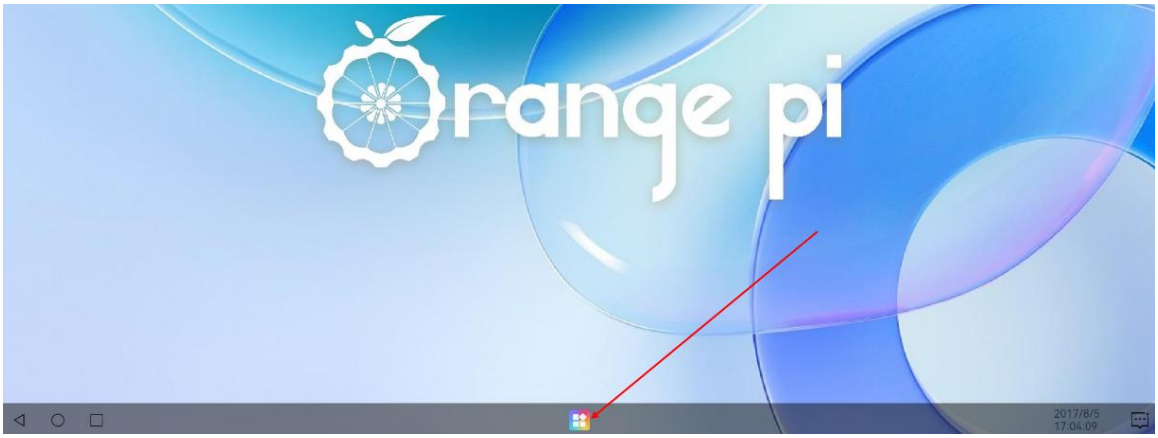
c. 然后选择**有线网络**



d. 然后就可以看到网口的 IP 地址等信息了

6.3. WIFI 的使用方法

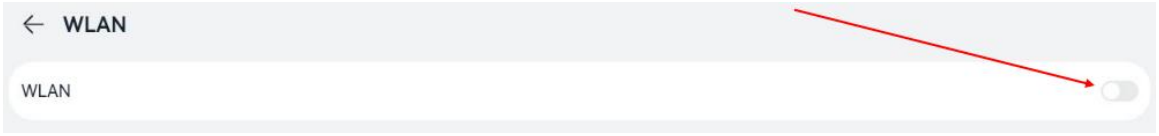
1) 首先点击**应用列表**



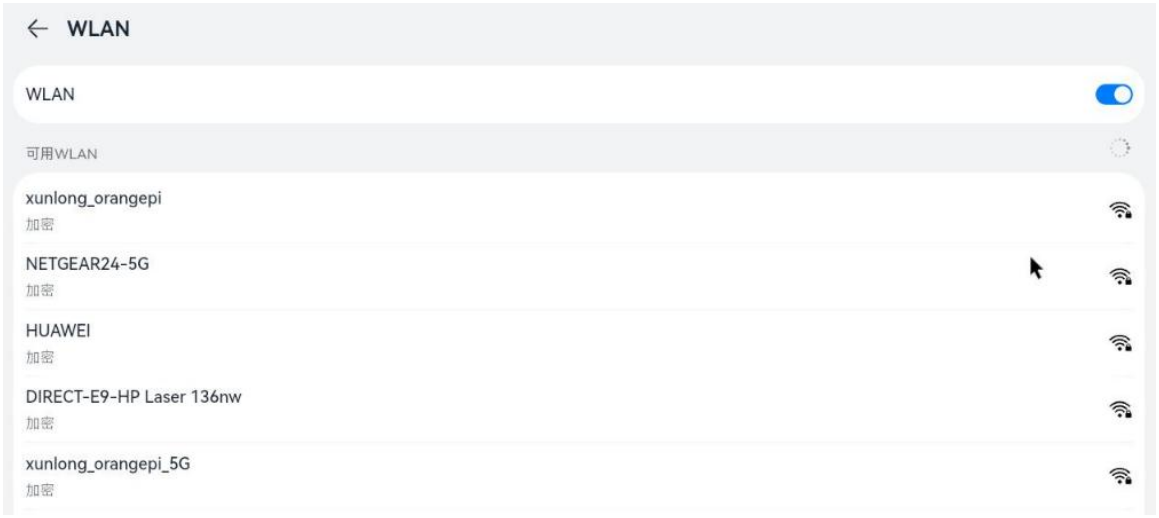
2) 然后打开**设置**

3) 然后选择 **WLAN**

4) 然后打开 **WLAN**



5) 然后连接搜索到的 WIFI 热点即可



7. Android 11 系统的使用说明

7.1. 已支持的 Android 版本

Android 版本	内核版本
Android 11	Linux4.19

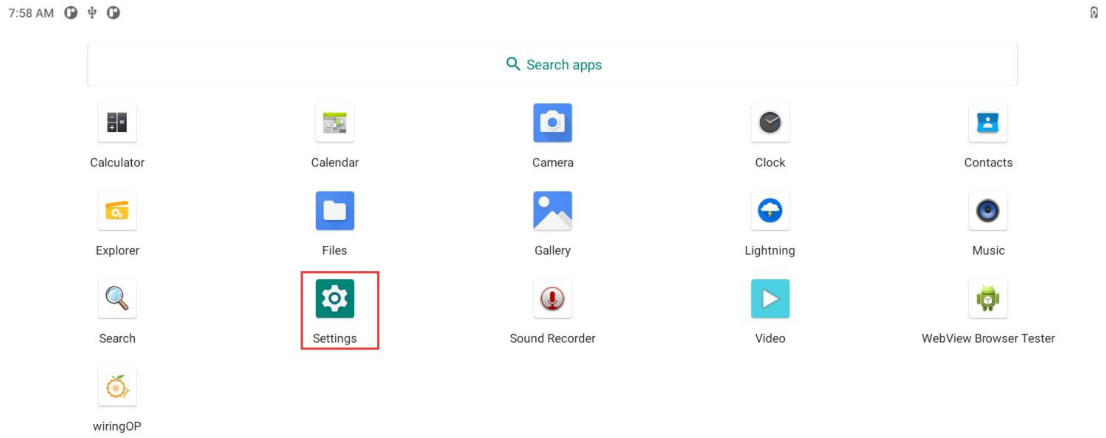
7.2. Android 功能适配情况

功能	Android 11
USB2.0x3	OK
USB3.0x1	OK
M.2 NVMe SSD 启动	OK
WIFI	OK
蓝牙	OK
GPIO（40pin）	OK
UART（40pin）	OK
SPI（40pin）	OK
I2C（40pin）	OK
PWM（40pin）	OK
PWM 风扇接口	OK
3pin 调试串口	OK
EMMC	OK
TF 卡启动	OK
HDMI 视频	OK
HDMI 音频	OK
LCD	OK
eDP 显示	OK
OV5647 摄像头	内核驱动 OK，3A 未调
千兆网口	OK
网口状态灯	OK
耳机播放	OK

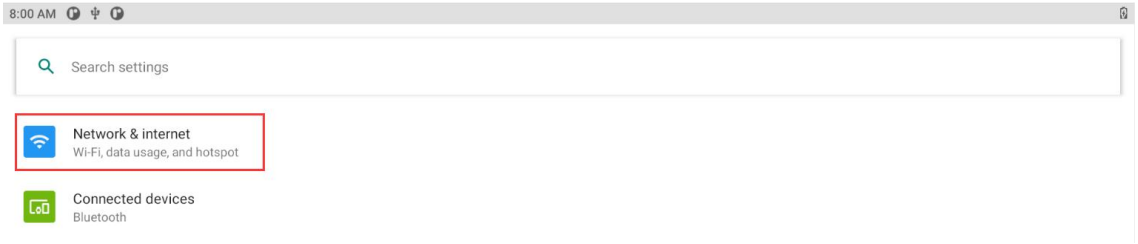
耳机录音	OK
LED 灯	OK
GPU	OK
NPU	OK
VPU	OK
RTC	OK

7.3. WIFI 的连接测试方法

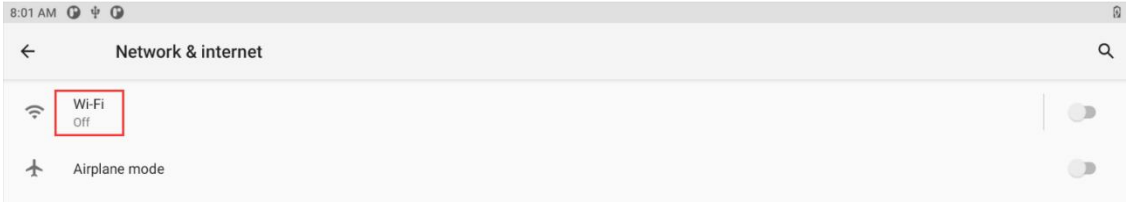
1) 首先点击进入 **Setting**



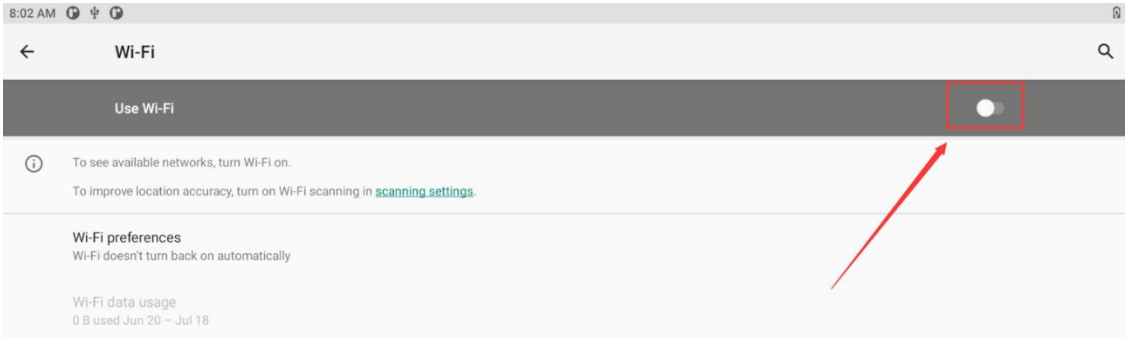
2) 然后选择 **Network & internet**



3) 然后选择 **Wi-Fi**

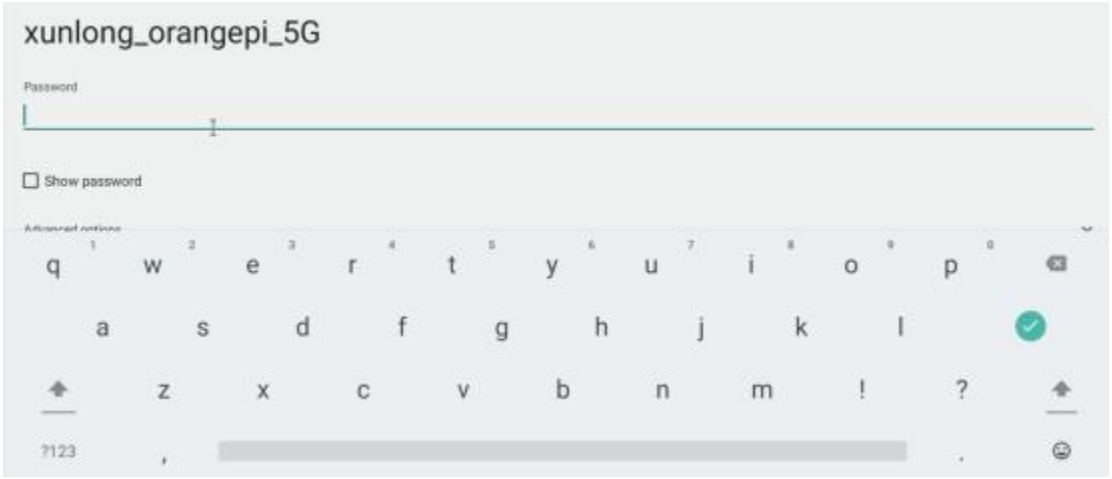


4) 然后打开 **Wi-Fi** 开关

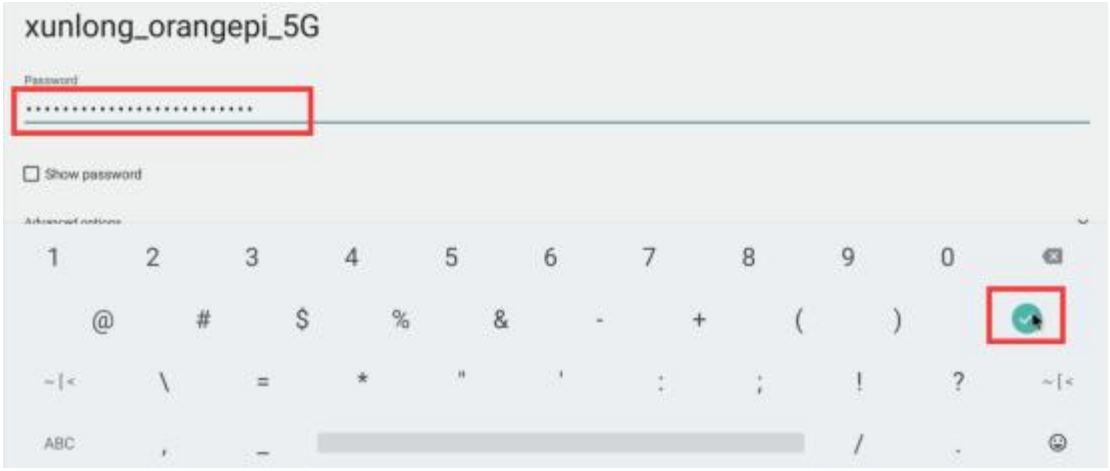


5) 打开 **Wi-Fi** 后如果一切正常，就可以扫描到附近的 Wi-Fi 热点了

6) 然后选择想连接的 Wi-Fi 后会弹出下图所示的密码输入界面



7) 然后使用键盘输入 Wi-Fi 对应的密码，再使用鼠标点击虚拟键盘中的回车按钮就会开始连接 Wi-Fi 了

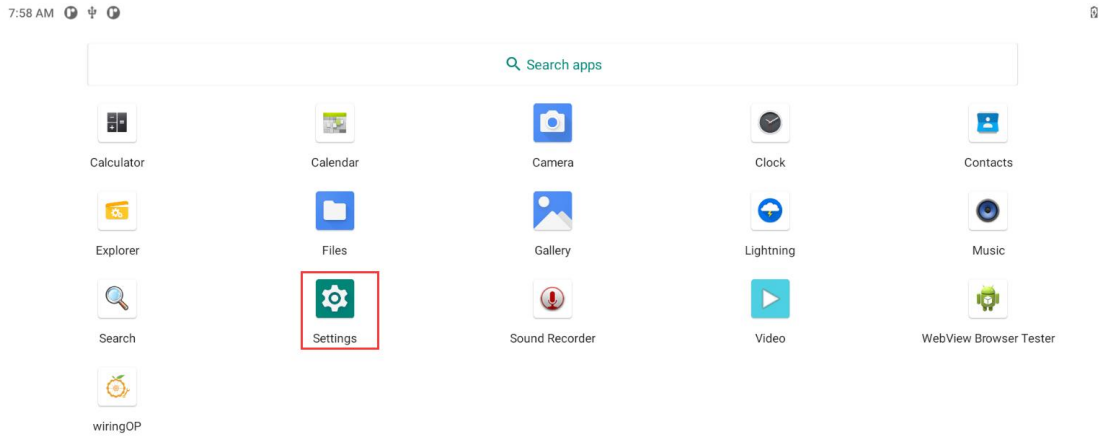


8) Wi-Fi 连接成功后的显示如下图所示：

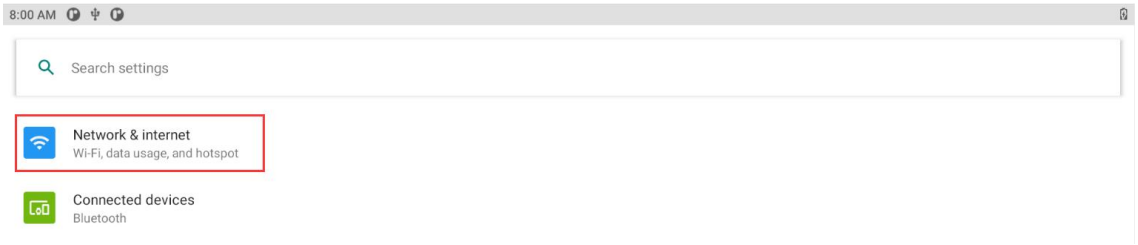
7. 4. Wi-Fi hotspot 的使用方法

1) 首先请确保以太网口已连接网线，并且能正常上网

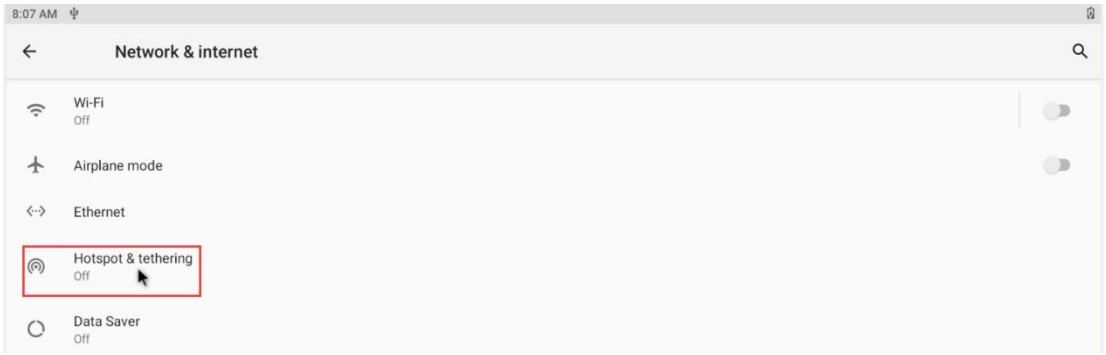
2) 然后选择 **Settings**



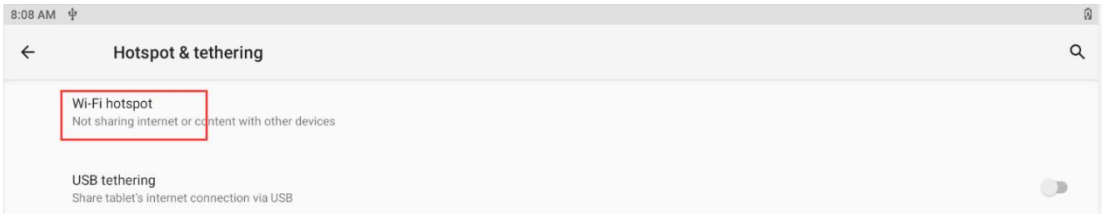
3) 然后选择 **Network & internet**



4) 然后选择 **Hotspot & tethering**



5) 然后选择 **Wi-Fi hotspot**



6) 然后打开 **Wi-Fi hotspot**，下图中还可以看到生成的热点的名字和密码，记住它们，在连接热点的时候要用到（如果需要修改热点的名字和密码，需要先关闭 **Wi-Fi hotspot**，然后才能修改）

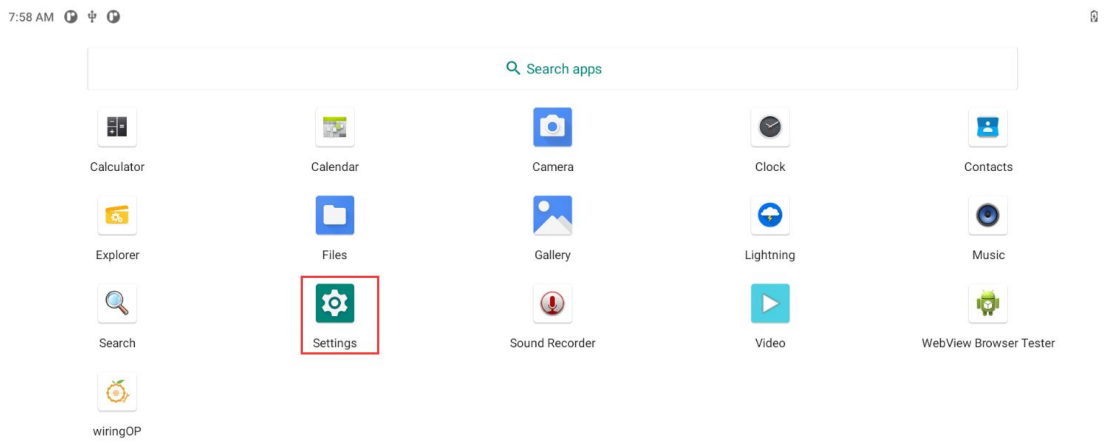
7) 此时可以拿出你的手机，如果一切正常，在手机搜索到的 WI-FI 列表中就能找到

上图 **Hotspot name** 下面显示的同名（**这里为 AndroidAP_6953**）的 WIFI 热点了。然后可以点击 **AndroidAP_6953** 连接热点，密码在上图的 **Hotspot password** 下面可以看到

8) 连接成功后显示如下图所示（不同手机界面会有区别，具体界面以你手机显示的为准）。此时就可以在手机上打开一个网页看下能否上网了，如果能正常打开网页，说明开发板的 **WI-FI Hotspot** 能正常使用

7.5. 蓝牙的测试方法

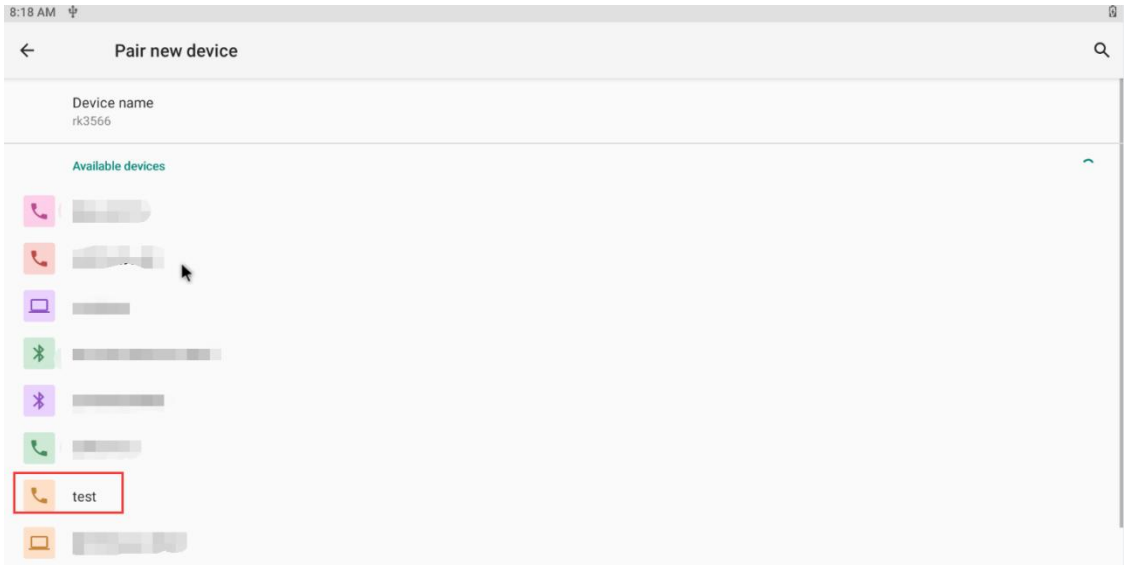
1) 首先点击进入 **Setting**



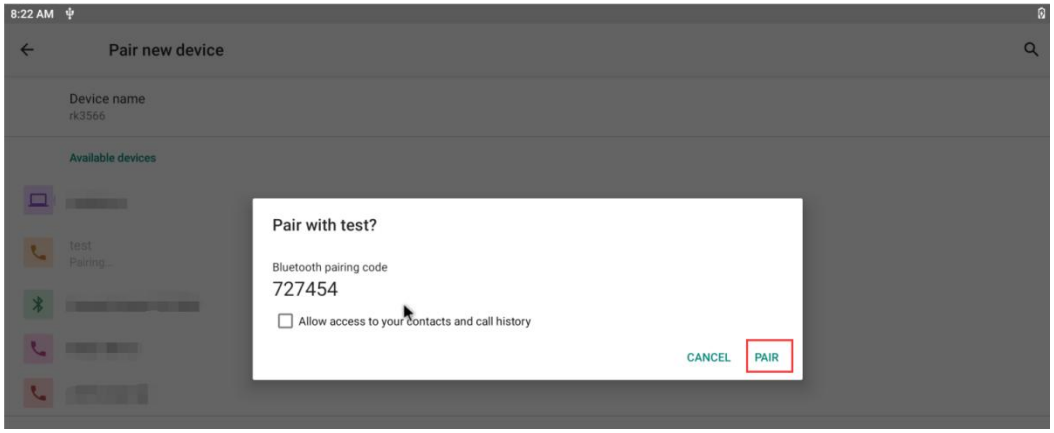
2) 然后选择 **Connected devices**

3) 然后点击 **Pair new device** 打开蓝牙并开始扫描周围的蓝牙设备

4) 搜索到的蓝牙设备会在 **Available devices** 下面显示出来

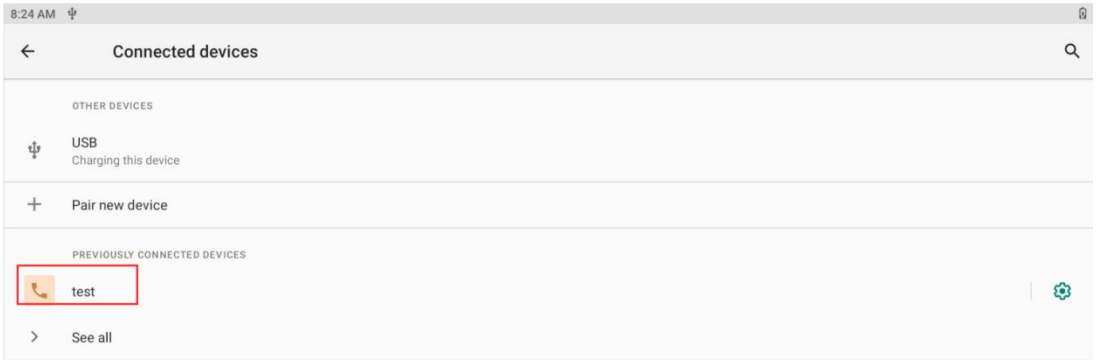


5) 然后点击想要连接的蓝牙设备就可以开始配对了，当弹出下面的界面时，请使用鼠标选择 **Pair** 选项

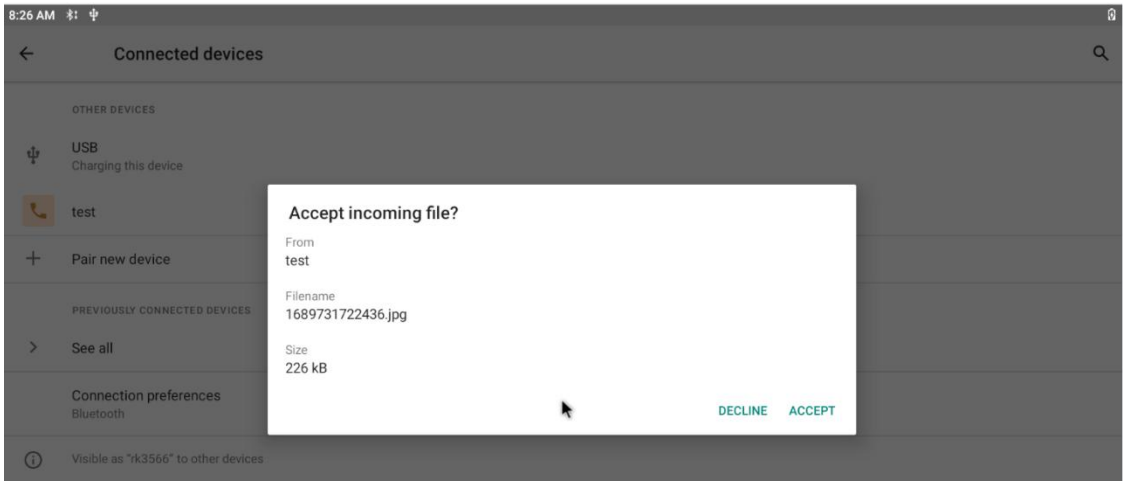


6) 这里测试的是开发板和安卓手机蓝牙的配置过程，此时在手机上会弹出下面的确认界面，在手机上也点击配对按钮后就会开始配对过程

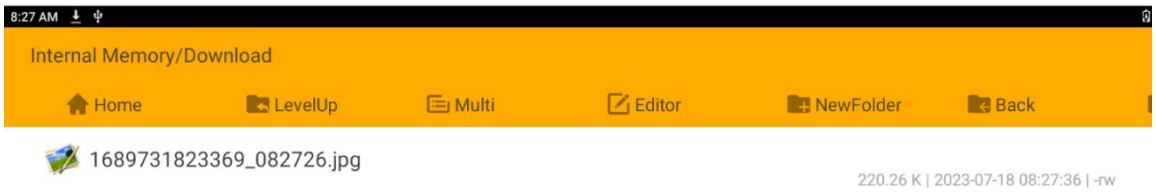
7) 配对完成后，可以看到如下图所示的已配对的蓝牙设备



8) 此时可以使用手机蓝牙给开发板发送一张图片，发送后，在开发板的安卓系统中可以看到下面的确认界面，然后点击 **Accept** 就可以开始接收手机发过来的图片了



9) 开发板 Android 系统蓝牙接收到的图片可以在文件管理器中打开 **Download** 目录查看



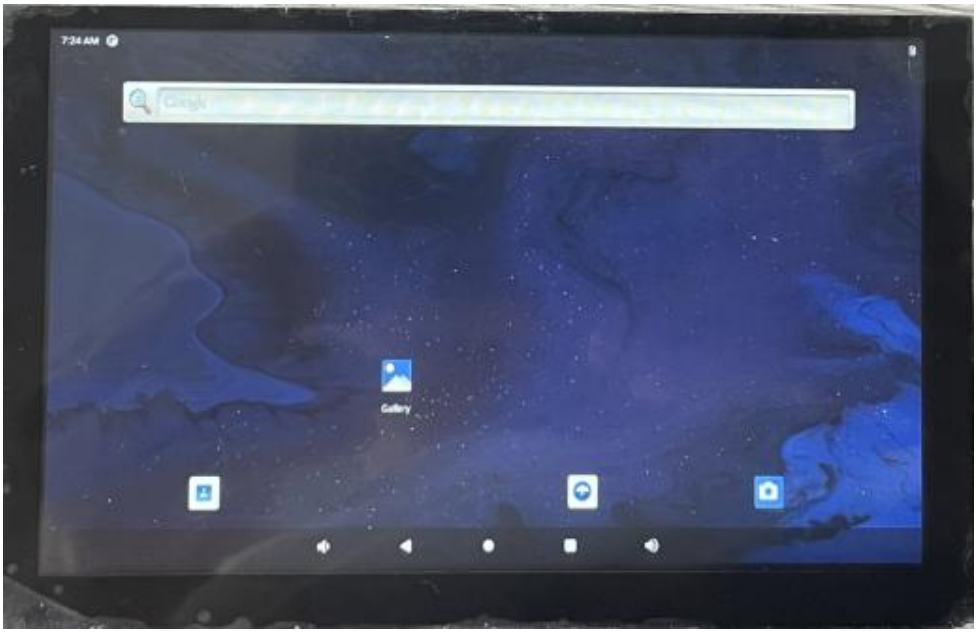
7.6. 树莓派 5 寸屏幕的使用方法

请确保使用的镜像为下面的两个版本的镜像：
OrangePi3B_RK3566_Android11_lcd_v1.x.x.img
OrangePi3B_RK3566_Android11_spi-nvme_lcd_v1.x.x.img

1) 首先需要组装好屏幕，请参考**树莓派 5 寸屏幕的组装方法**

2) 给板子接通 Type-C 电源，并上电，系统启动后，就可以看到屏幕显示如下图所示

树莓派 5 寸屏幕的显示和触摸都是可以用的，如果你的屏幕测试有问题，请先确保下购买的屏幕和 Orange Pi 适配的屏幕是一模一样的。
Orange Pi 适配的屏幕在树莓派 5 寸屏幕的组装方法中有说明。



7.7. eDP 屏幕的使用方法

请确保使用的镜像为下面的两个版本的镜像：
OrangePi3B_RK3566_Android11_lcd_v1.x.x.img
OrangePi3B_RK3566_Android11_spi-nvme_lcd_v1.x.x.img

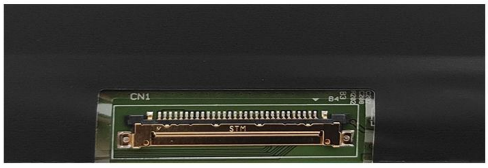
eDP 屏幕是没有触摸功能的。

1) 目前仅适配了一款 15.6 寸的 eDP 屏幕，包含的配件如下：
   a. 0.5 间距 30pin 单头同向排线

   b. 15.6 寸 eDP 显示屏，分辨率为 1920x1080

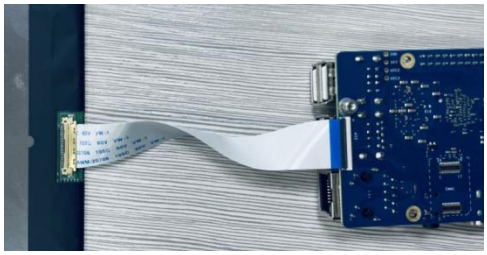
正面

背面



30pins EDP接口

2) 将 30pin 单头同向排线的 FPC 端连接到开发板的 eDP 接口，另一端连接到屏幕的 eDP 接口

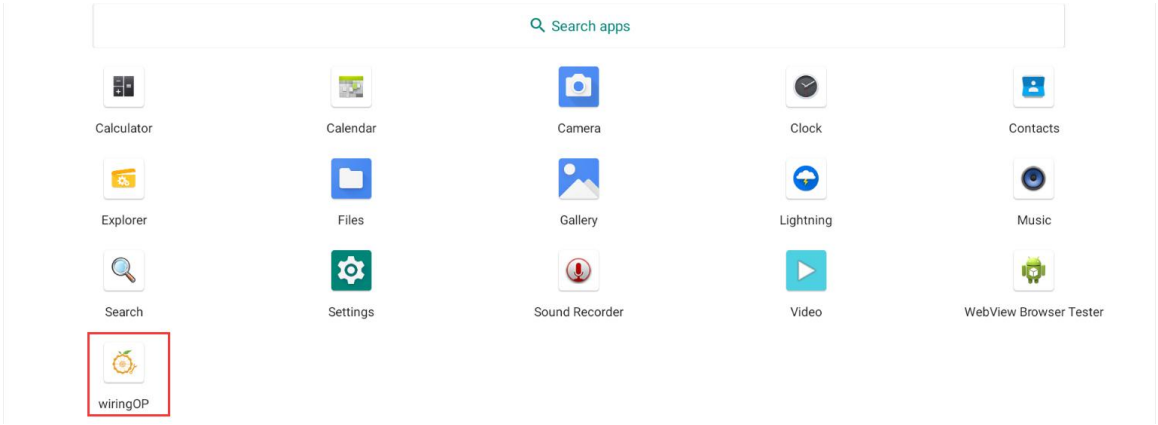


3) 再给板子接通 Type-C 电源，并上电，系统启动后，就可以看到屏幕显示如下图所示

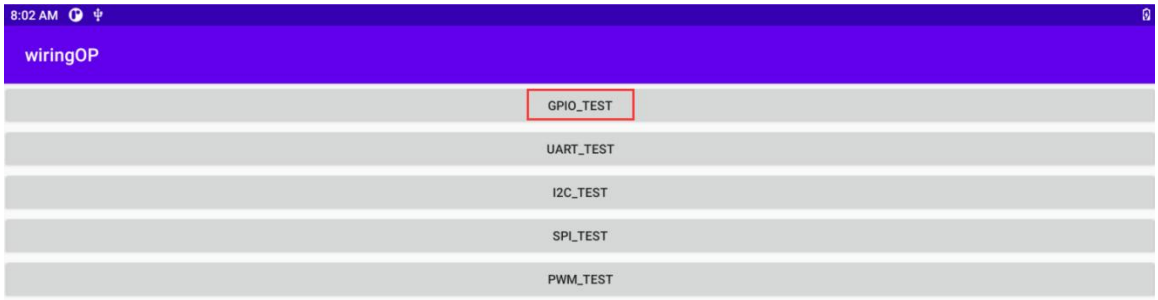
7.8. 40pin 接口 GPIO、UART、SPI 和 PWM 测试

7.8.1. 40pin GPIO 口测试

1) 首先点击 wiringOP 图标打开 wiringOP APP

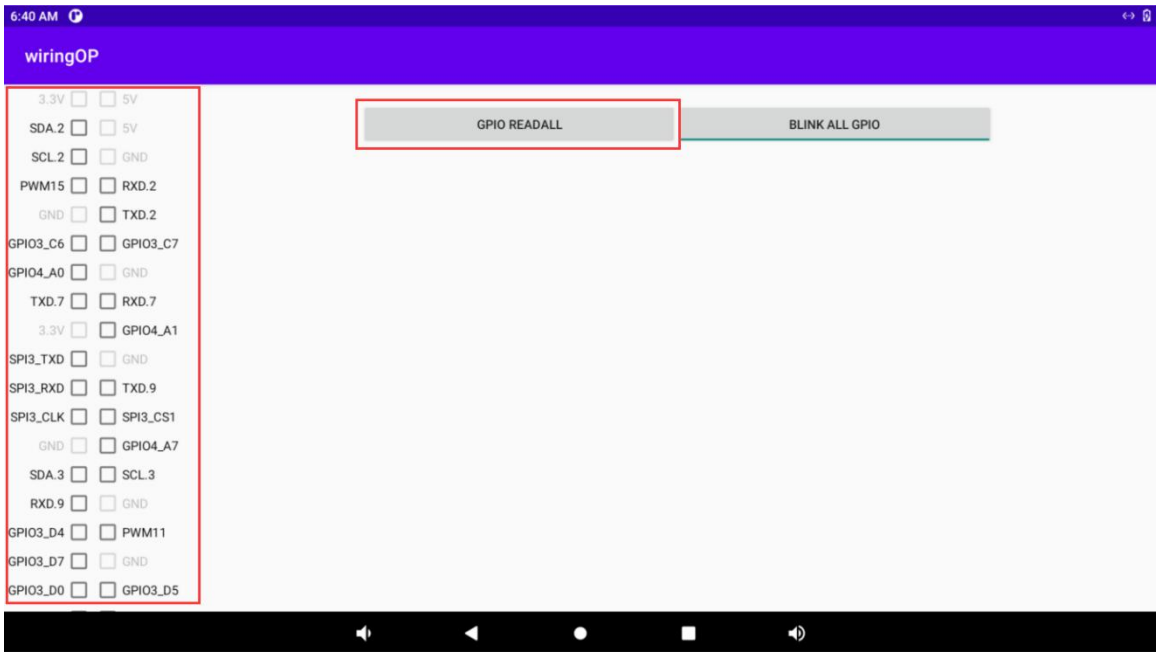


2) wiringOP APP 的主界面显示如下图所示，然后点击 **GPIO_TEST** 按钮打开 GPIO 测试界面

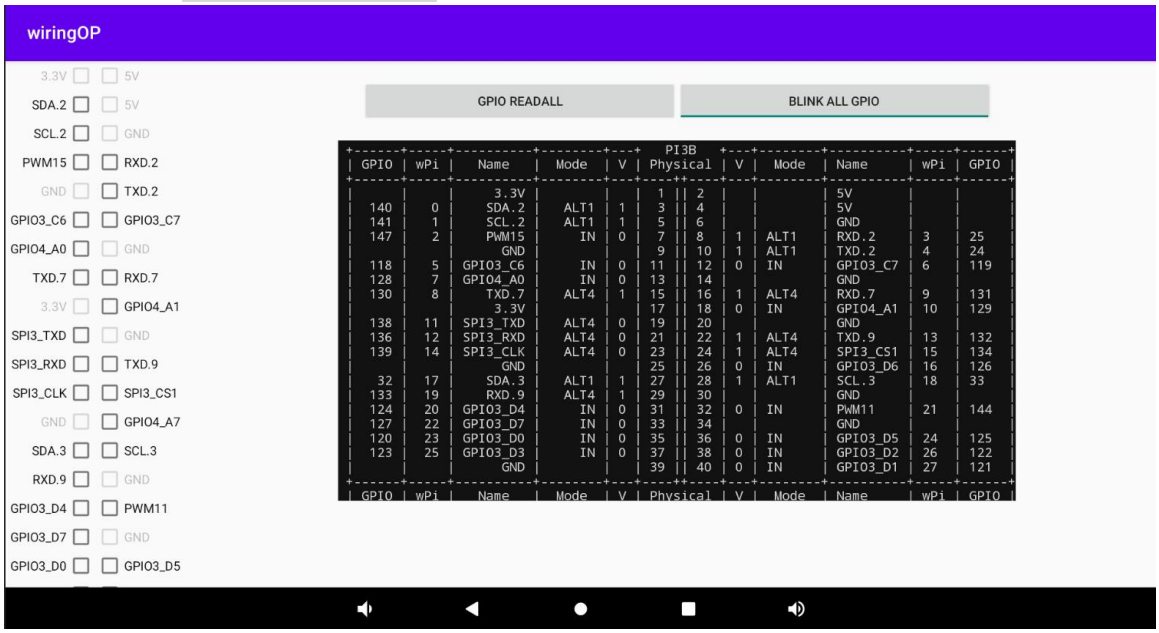


3) GPIO 测试界面如下图所示，左边的两排 **CheckBox** 按钮跟 40pin 引脚是一一对应的关系。当勾选 **CheckBox** 按钮时，对应的 GPIO 引脚会被设置为 **OUT** 模式，引脚电平设置为高电平；当取消勾选时，GPIO 引脚电平设置为低电平；当点击右边

的 **GPIO READALL** 按钮时，可以获取到 wPi 号、GPIO 模式、引脚电平等信息；当点击 **BLINK ALL GPIO** 按钮时，程序会控制 28 个 GPIO 口不停的切换高低电平



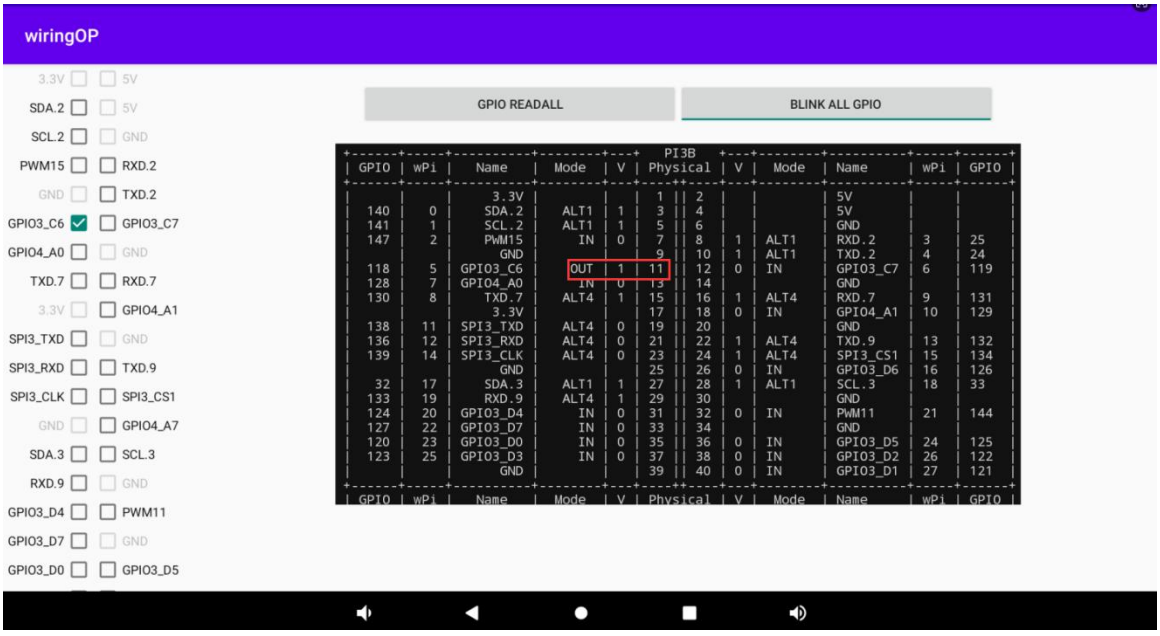
4) 然后点击 **GPIO READALL** 按钮，输出信息如下图所示：



5) 开发板 40pin 中总共有 28 个 GPIO 口可以使用，下面以 11 号引脚——对应 GPIO 为 GPIO3_C6 ——对应 wPi 序号为 5——为例演示如何设置 GPIO 口的高低电平。首先点击 11 号引脚对应的 **CheckBox** 按钮，当按钮为选中状态时，11 号引脚会设置为高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 **3.3v**，说明

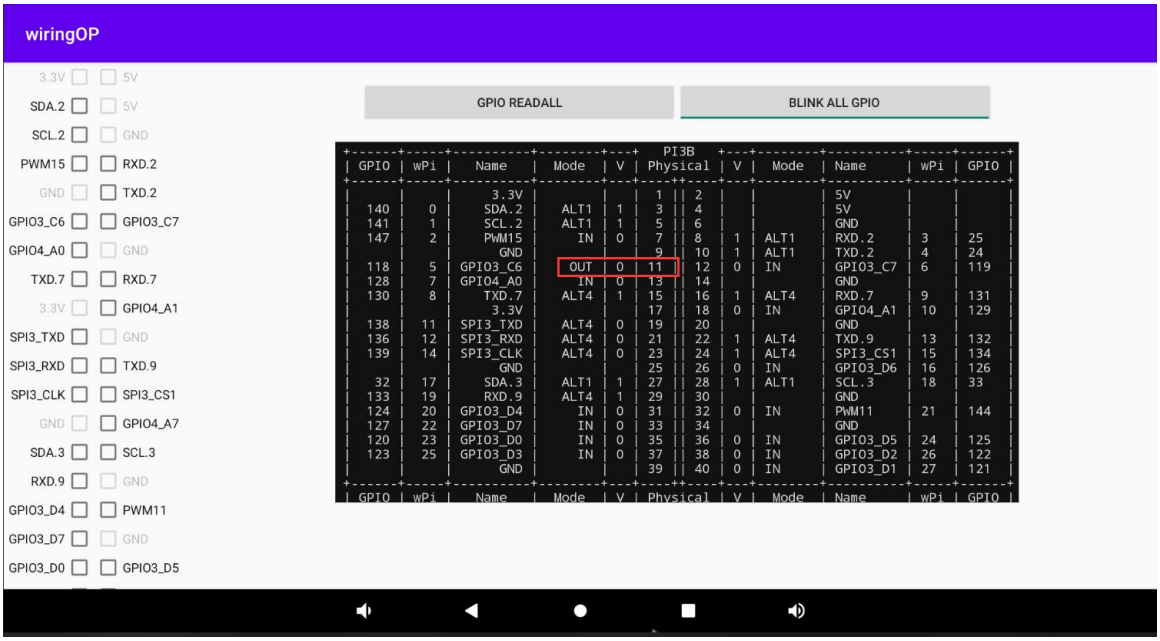
设置高电平成功

6) 然后点击 **GPIO READALL** 按钮，可以看到当前的 11 号引脚模式为 **OUT**，引脚电平为高电平



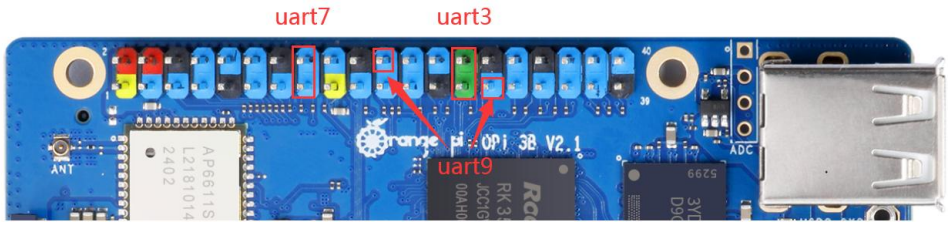
7) 再次点击下图的 **CheckBox** 按钮取消勾选状态，11 号引脚会设置为低电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 **0v**，说明设置低电平成功

8) 然后点击 **GPIO READALL** 按钮，可以看到当前的 7 号引脚模式为 OUT，引脚电平为低电平

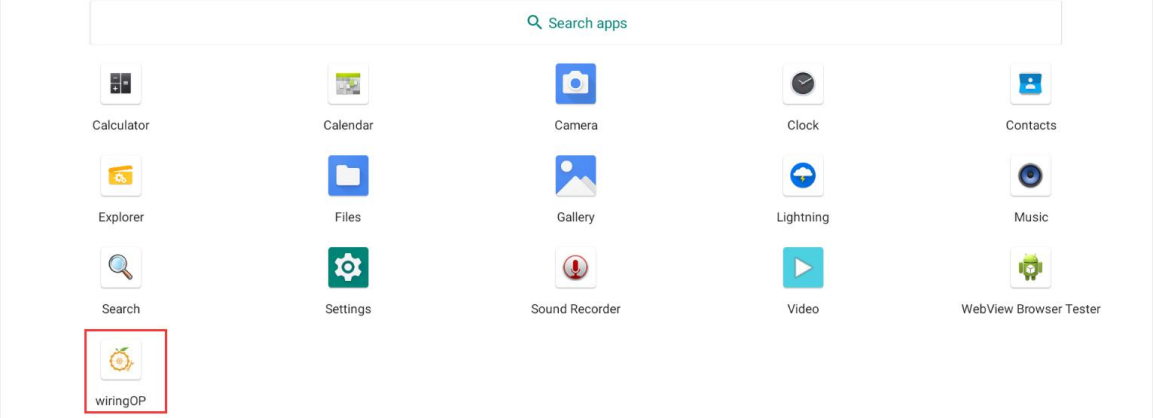


7.8.2. 40pin 的 UART 测试

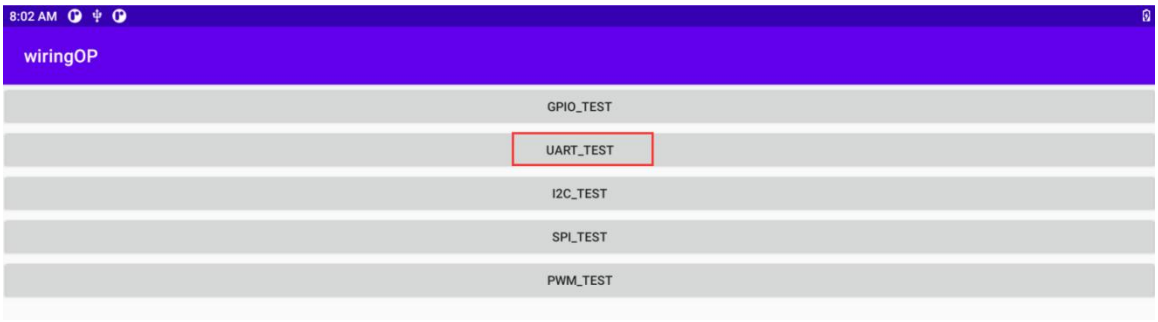
1) Android 中默认打开了 UART7 和 UART9，在 40pin 的位置如下图所示，对应的设备节点分别是**/dev/ttyS7** 和**/dev/ttyS9**



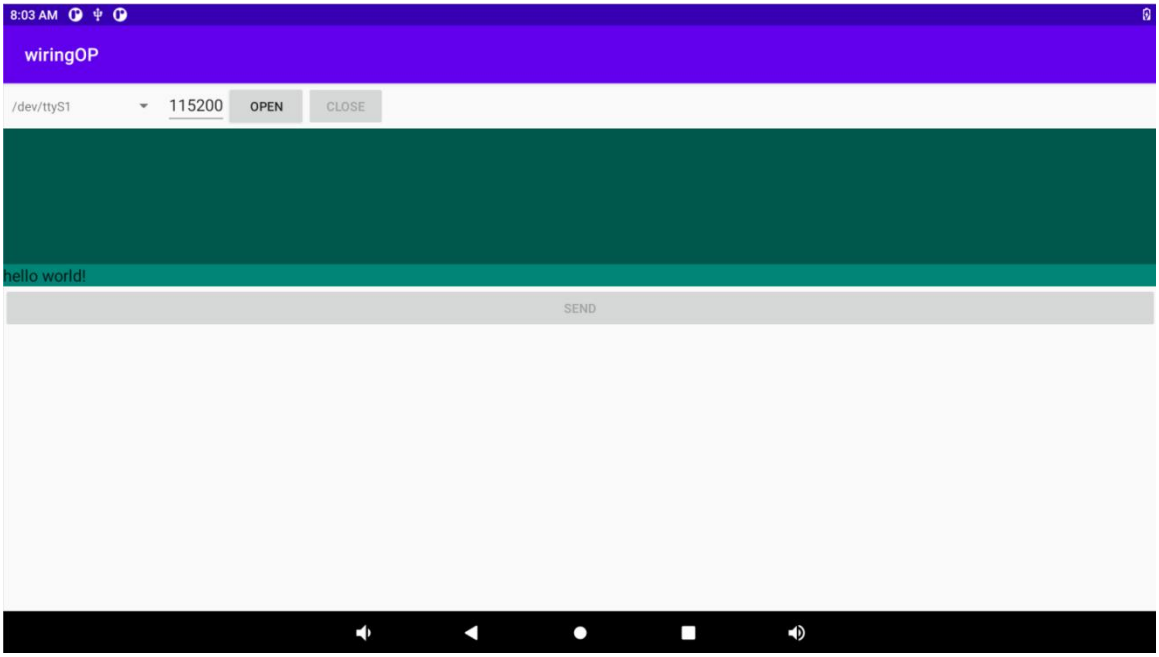
2) 首先点击 wiringOP 图标打开 wiringOP APP



3) wiringOP APP 的主界面显示如下图所示，然后点击 **UART_TEST** 按钮打开 UART 测试界面



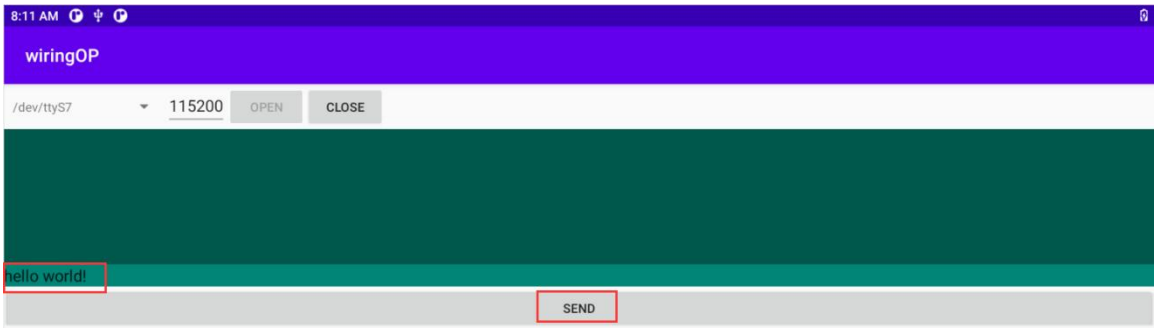
4) APP 的串口测试界面如下图所示



5) 下面以测试 **UART7** 为例，在选择框中选择**/dev/ttyS7** 节点，在编辑框中输入想要设置的波特率，然后点击 **OPEN** 按钮打开**/dev/ttyS7** 节点，打开成功后，**OPEN** 按钮变为不可选中状态，**CLOSE** 按钮和 **SEND** 按钮变为可选中状态

6) 然后使用杜邦线短接 uart7 的 RXD 和 TXD 引脚

7) 然后可以在下面的发送编辑框中输入一段字符，点击 **SEND** 按钮开始发送



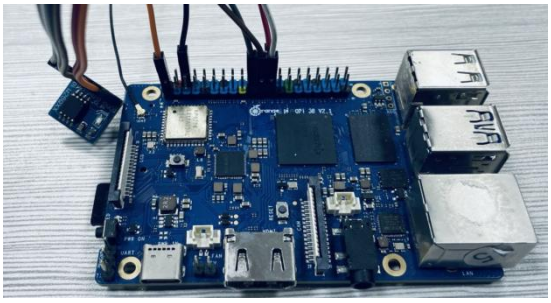
8) 如果一切正常，接收框内会显示已接收到的字符串

7.8.3. 40pin 的 SPI 测试

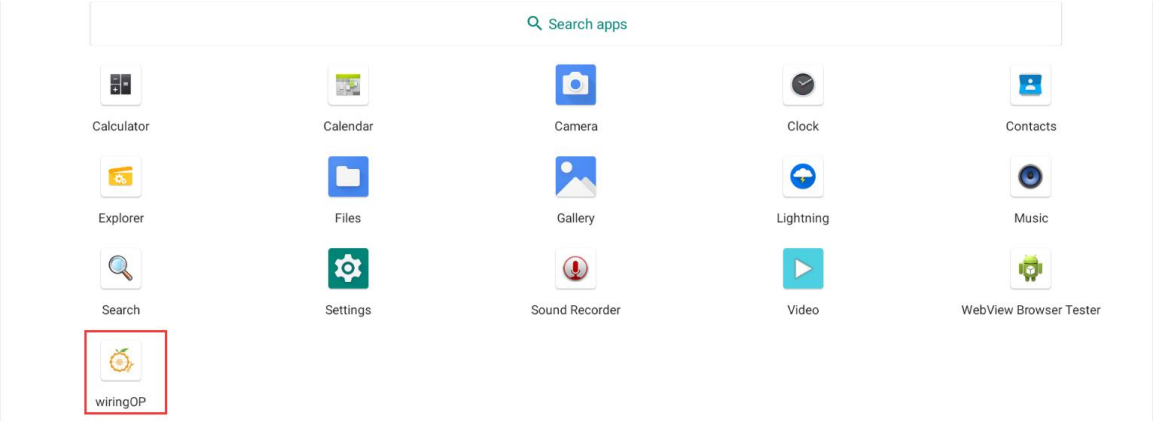
1) 由下表可知，Orange Pi 3B 可用的 spi 为 spi3

复用功能	复用功能	GPIO	GPIO序号	引脚序号	引脚序号	GPIO序号	GPIO	复用功能	复用功能
		3.3V		1	2		5V		
	I2C2_SDA_M1	GPIO4_B4	140	3	4		5V		
	I2C2_SCL_M1	GPIO4_B5	141	5	6		GND		
	PWM15_IR_M1(fe700030)	GPIO4_C3	147	7	8	25	GPIO0_D1	UART2_TX_M0	
		GND		9	10	24	GPIO0_D0	UART2_RX_M0	
		GPIO3_C6	118	11	12	119	GPIO3_C7		
		GPIO4_A0	128	13	14		GND		
	UART7_TX_M2	GPIO4_A2	130	15	16	131	GPIO4_A3	UART7_RX_M2	
		3.3V		17	18	129	GPIO4_A1		
I2C4_SDA_M0	SPI3_MOSI_M0	GPIO4_B2	138	19	20		GND		
	SPI3_MISO_M0	GPIO4_B0	136	21	22	132	GPIO4_A4	UART9_TX_M2	
I2C4_SCL_M0	SPI3_CLK_M0	GPIO4_B3	139	23	24	134	GPIO4_A6	SPI3_CS0_M0	
		GND		25	26	126	GPIO3_D6		
UART3_RX_M0	I2C3_SDA_M0	GPIO1_A0	32	27	28	33	GPIO1_A1	I2C3_SCL_M0	UART3_TX_M0
	UART9_RX_M2	GPIO4_A5	133	29	30		GND		
		GPIO3_D4	124	31	32	144	GPIO4_C0	PWM11_IR_M1(fe6f0030)	
		GPIO3_D7	127	33	34		GND		
		GPIO3_D0	120	35	36	125	GPIO3_D5		
		GPIO3_D3	123	37	38	122	GPIO3_D2		
		GND		39	40	121	GPIO3_D1		

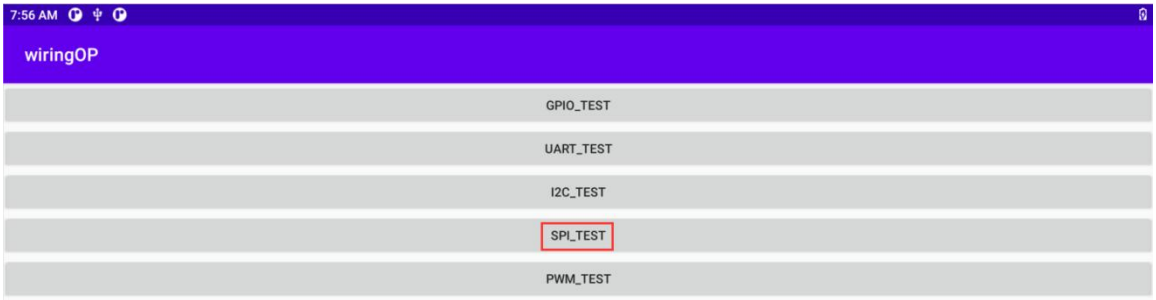
2) 这里通过 w25q64 模块来测试 SPI 接口，首先在 SPI3 接口接入 w25q64 设备



3) 然后点击 wiringOP 图标打开 wiringOP APP



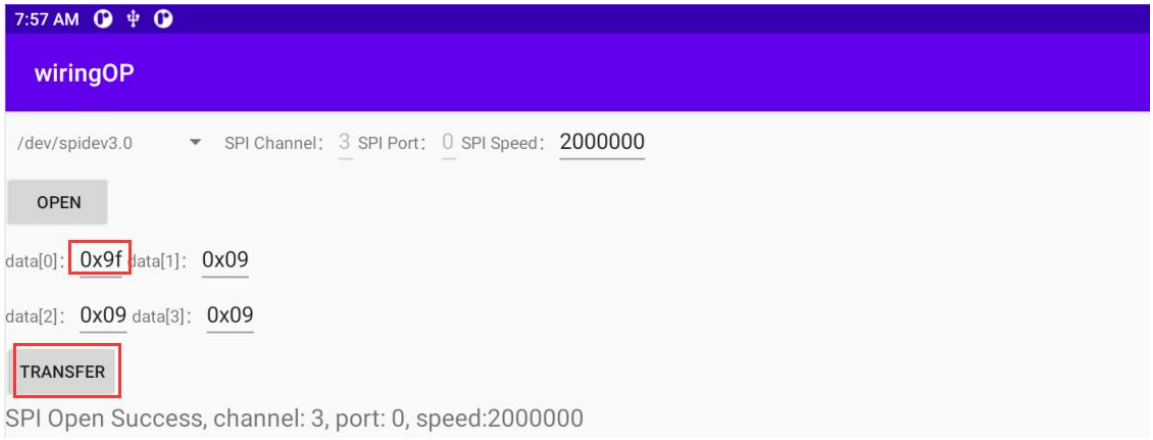
4) wiringOP APP 的主界面显示如下图所示，点击 SPI_TEST 按钮打开 SPI 的测试界面



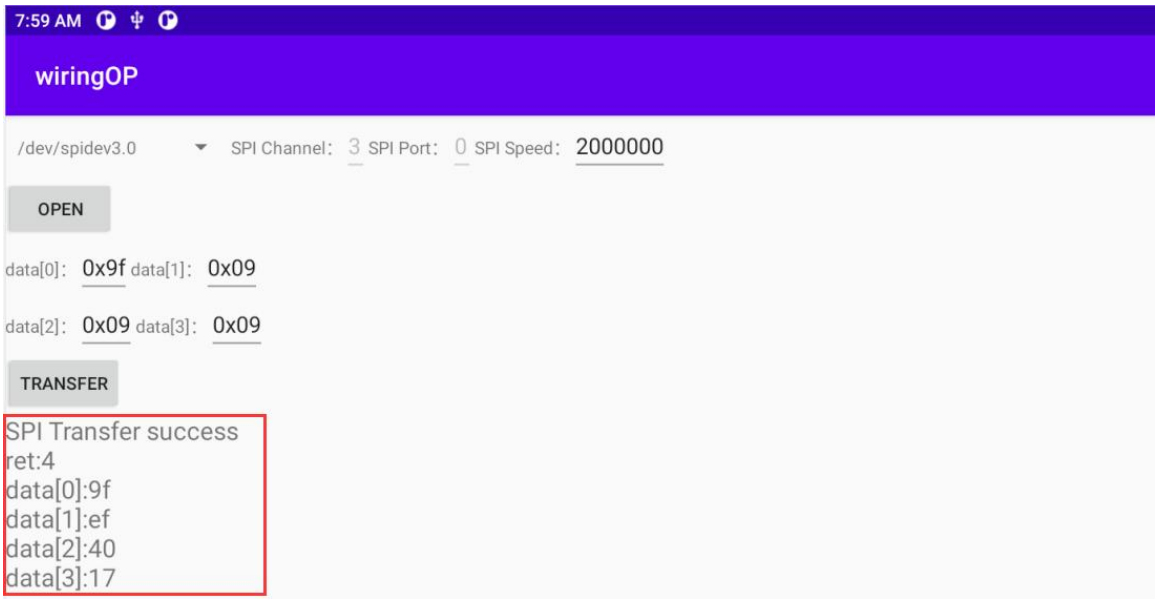
5) 然后点击 **OPEN** 按钮初始化 SPI

6) 然后填充需要发送的字节，比如读取 w25q64 的 ID 信息，在 data[0]中填入地址

0x9f，然后点击 **TRANSFER** 按钮



7) 最后 APP 会显示读取到的 ID 信息

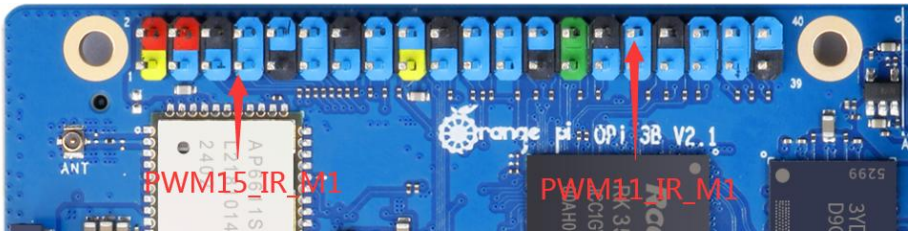


8) w25q64 模块的 MANUFACTURER ID 为 EFh，Device ID 为 4017h，跟上面读取到的值是对应的（h 代表是 16 进制）

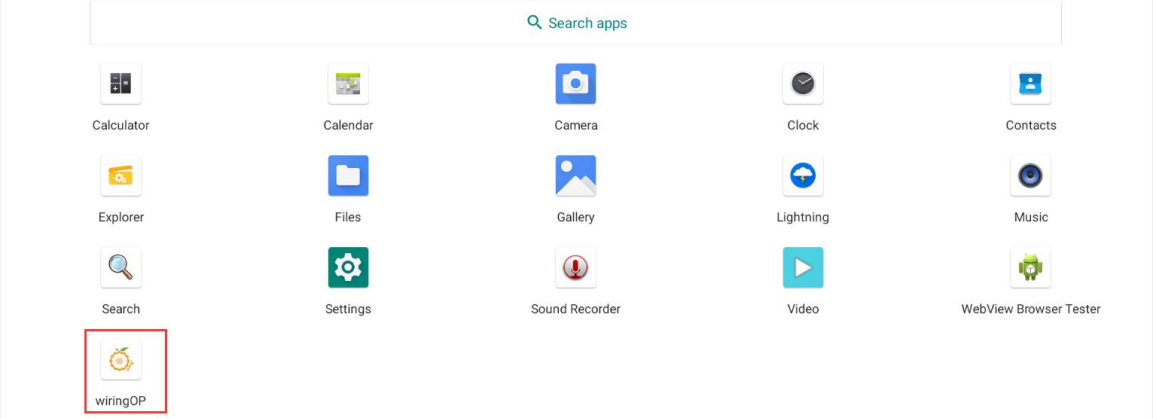
MANUFACTURER ID	**(MF7 - MF0)**	
Winbond Serial Flash	EFh	
Device ID	**(ID7 - ID0)**	**(ID15 - ID0)**
Instruction	**ABh, 90h, 92h, 94h**	**9Fh**
W25Q64FV (SPI)	16h	4017h
W25Q64FV (QPI)	16h	6017h

7.8.4. 40pin 的 PWM 测试

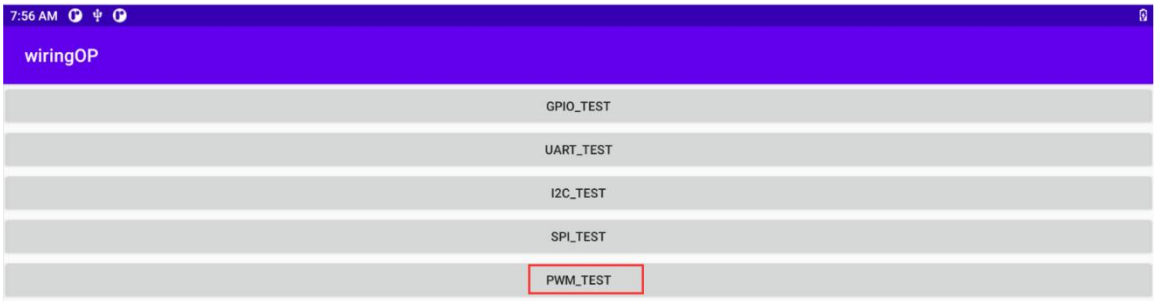
1) Android 默认开启了 **PWM11** 和 **PWM15**，对应的引脚在 40pin 的所在位置如下图所示



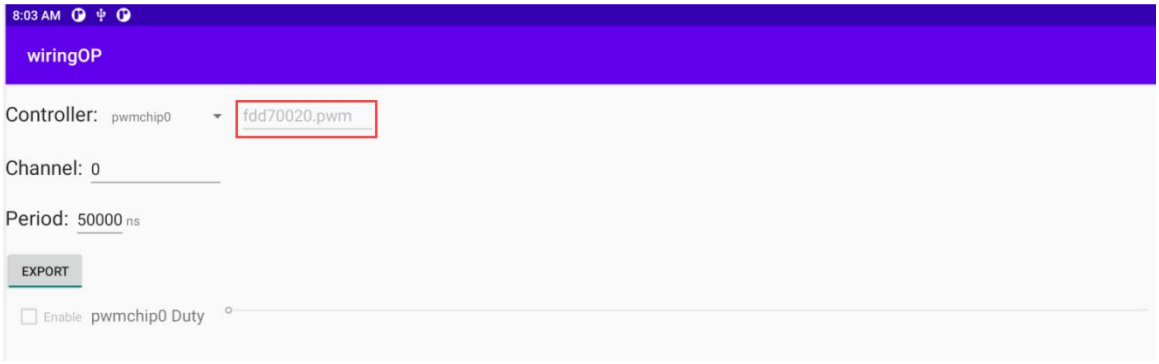
2) 首先点击 wiringOP 图标打开 wiringOP APP



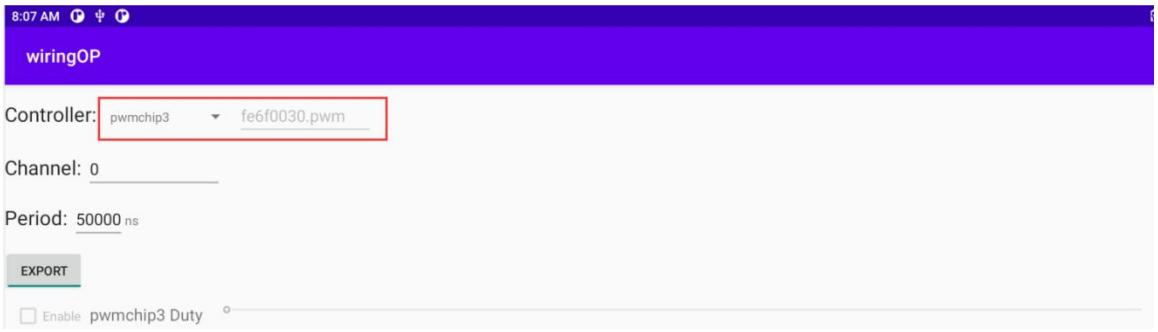
3) 然后在 wiringOP 的主界面点击 **PWM_TEST** 按钮进入 PWM 的测试界面



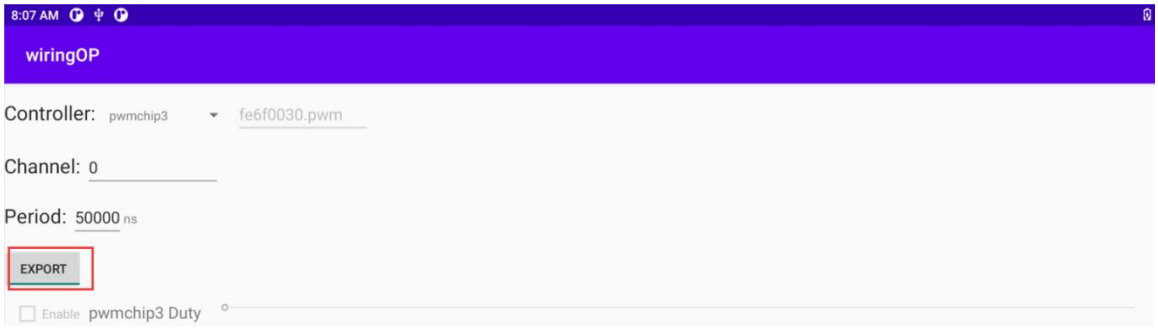
4) PWM11 对应的基地址是 **fe6f0030**，PWM15 对应的基地址是 **fe700030**，这里 pwmchip0 右边显示是 **fdd70020.pwm**， 这时需要点击下拉选项选择其它的 pwmchip，直到右边显示 **fe6f0030.pwm** 或者 **fe700030.pwm** 为止



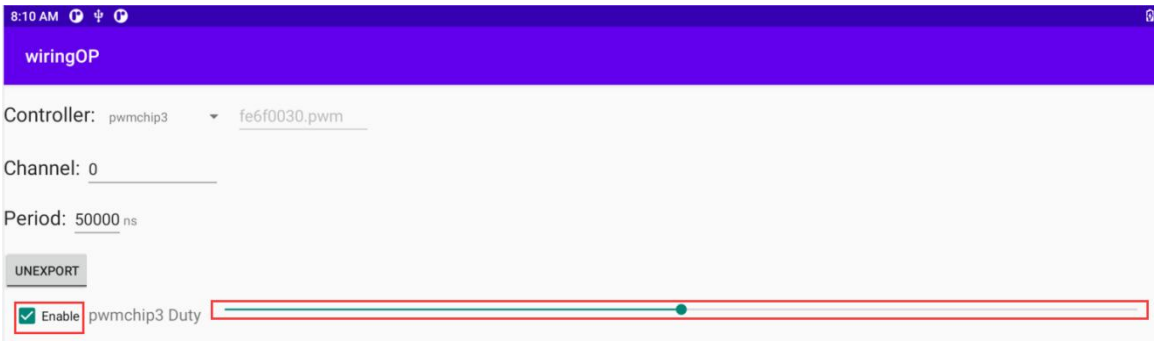
5) 当下拉选项选择 **pwmchip3** 时，右边对应的就是 PWM11 对应的基地址 **fe6f0030**



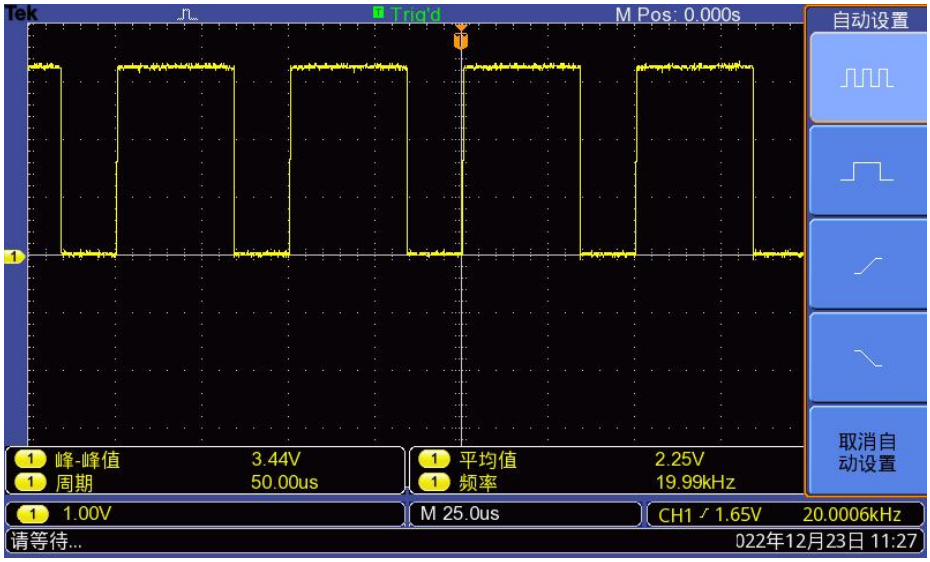
6) 然后确认 PWM 的通道，默认是 0 通道，并确认 PWM 的周期，默认的配置是 **50000ns**，转换为 PWM 频率是 **20KHz**，可自行修改，点击 **EXPORT** 按钮导出 **PWM11**



7) 然后拖动下面的拖动条，就可以改变 PWM 的占空比，然后勾选 Enable 就可以输出 PWM 波形了



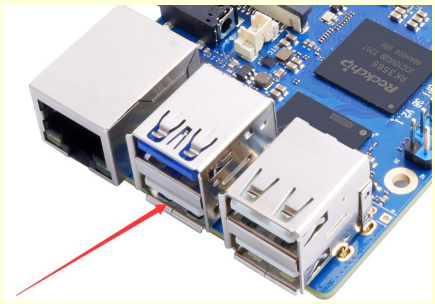
8) 然后使用示波器测量开发板 40pin 中的第 32 号引脚就可以看到下面的波形了



7.9. ADB 的使用方法

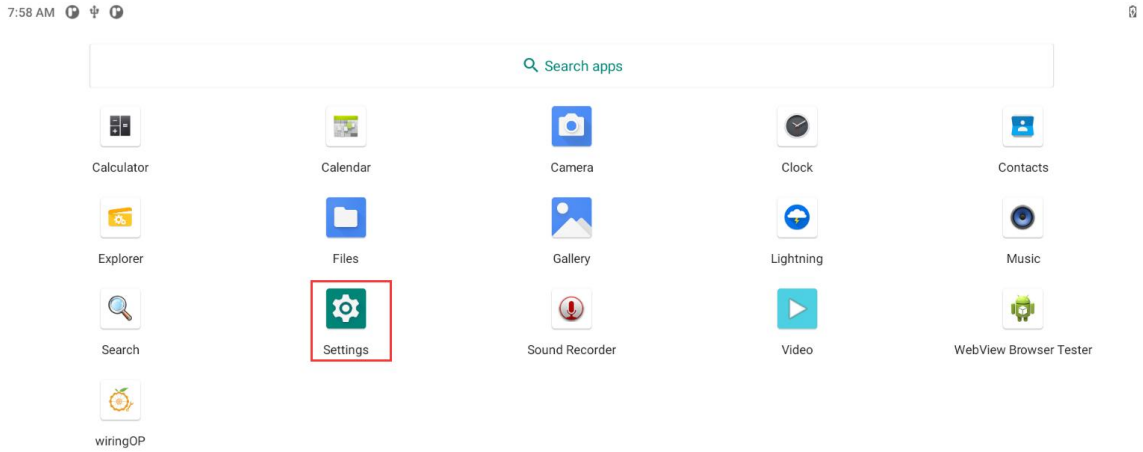
7.9.1. USB OTG 模式切换的方法

开发板有 4 个 USB 接口，其中下图红框标注的 USB 接口既可以支持 Host 模式，又可以支持 Device 模式，其它 3 个 USB 接口只支持 Host 模式。

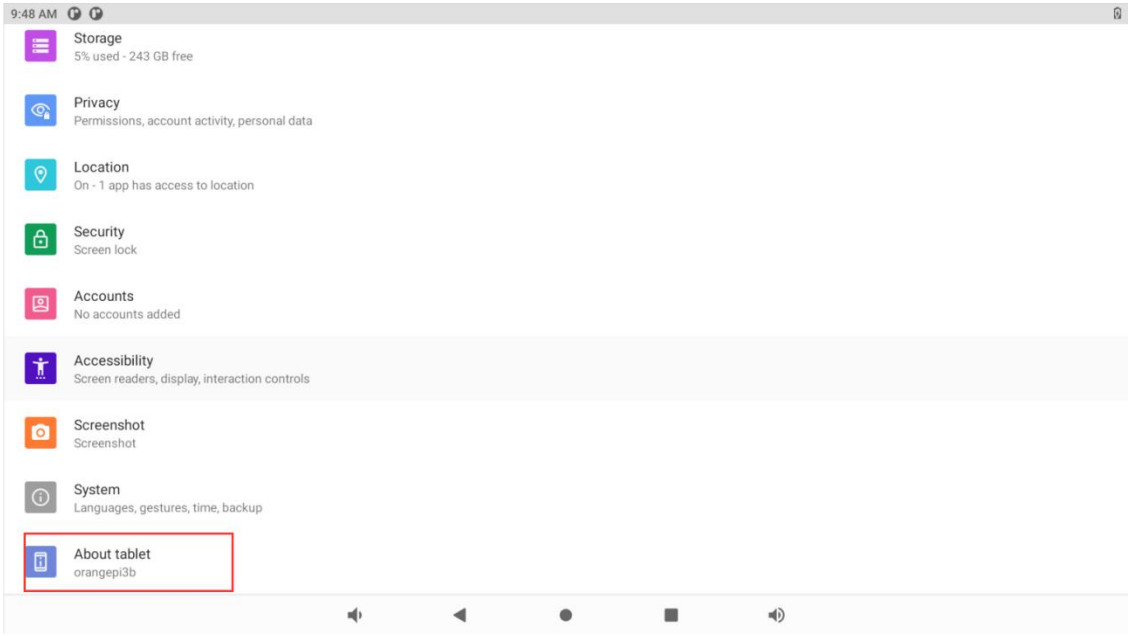


USB OTG 接口默认为 Host 模式，可以用来接鼠标键盘等 USB 设备，如果要使用 ADB 需要手动切换为 Device 模式。

1) 首先打开 Settings

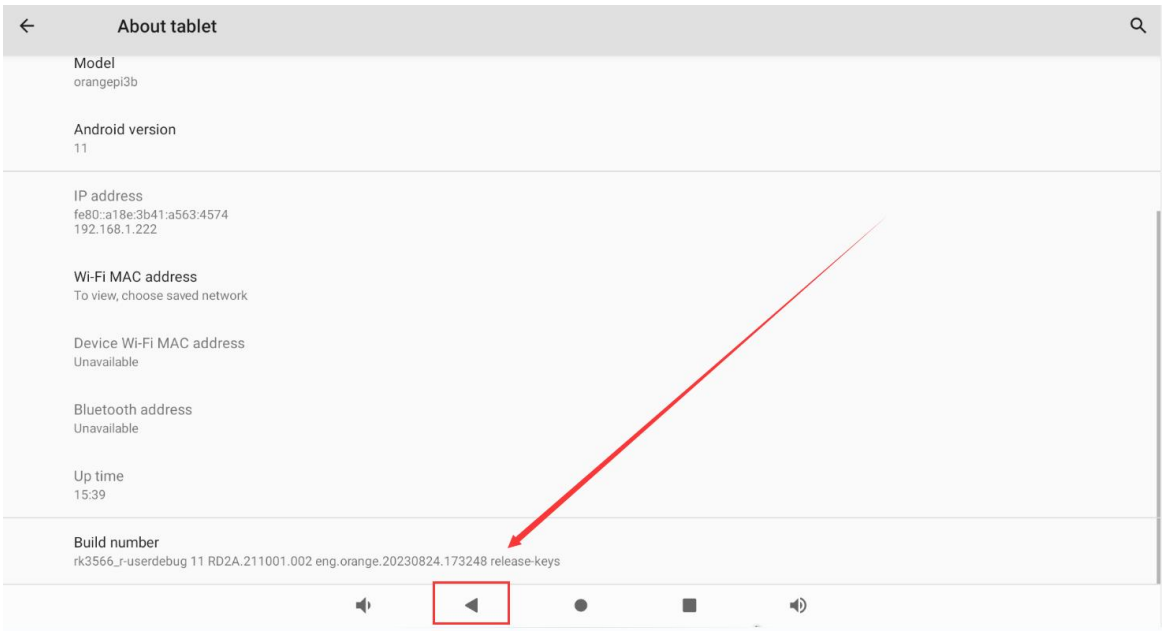


2) 然后选择 **About tablet**

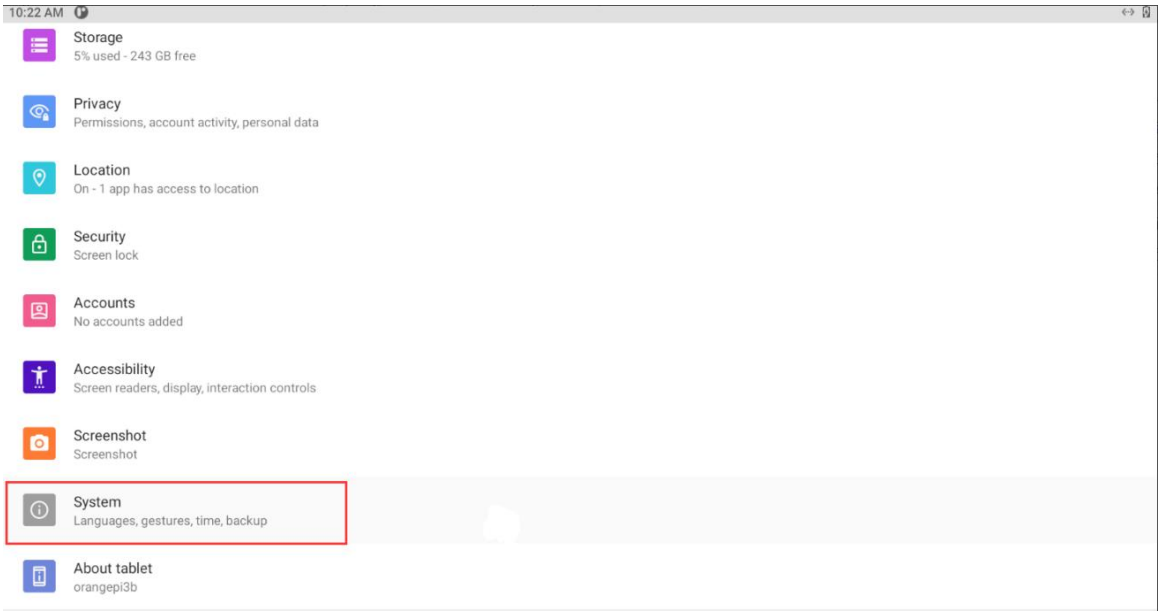


3) 然后用鼠标多次点击 **Build number** 菜单栏，直到出现 **You are now a developer!** 的提示

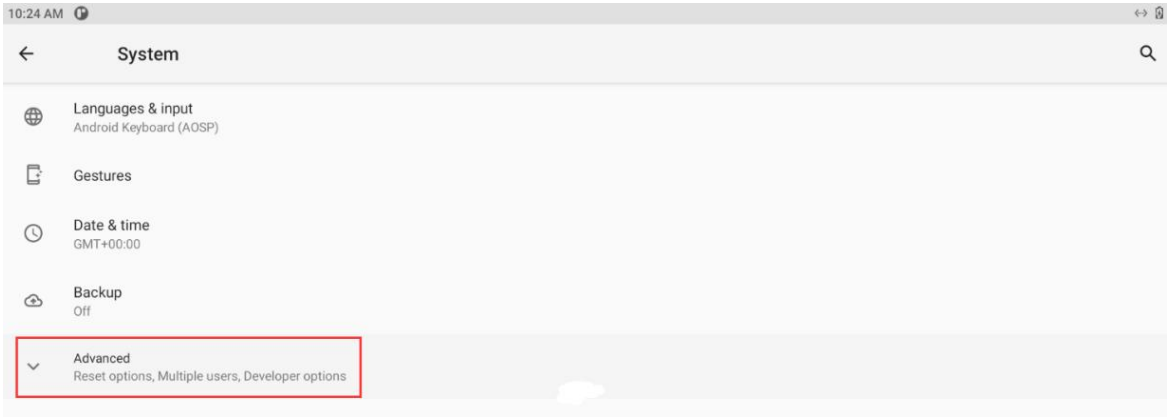
4) 然后点击返回上一级菜单



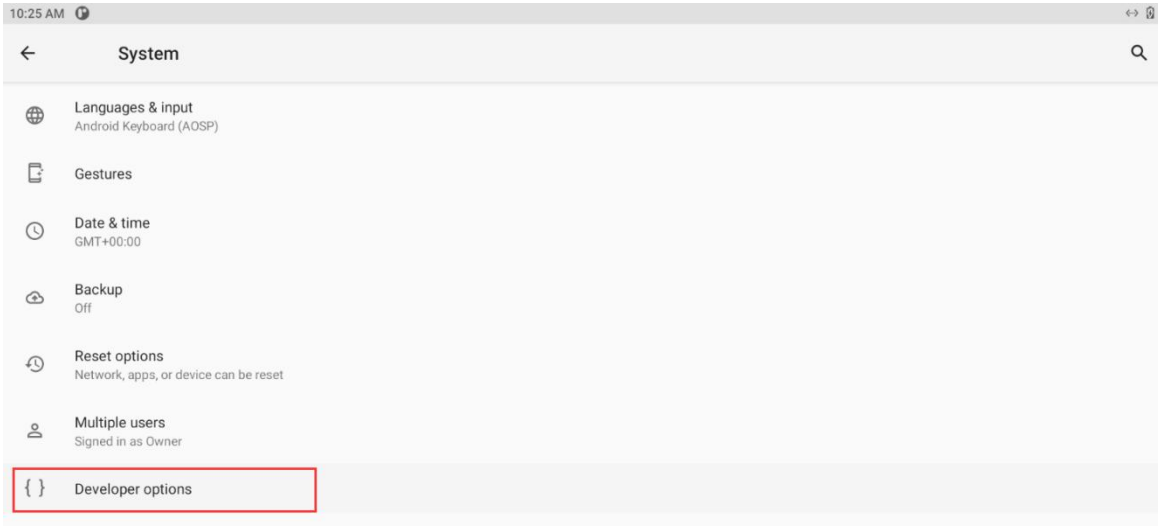
5) 然后选择 **System**



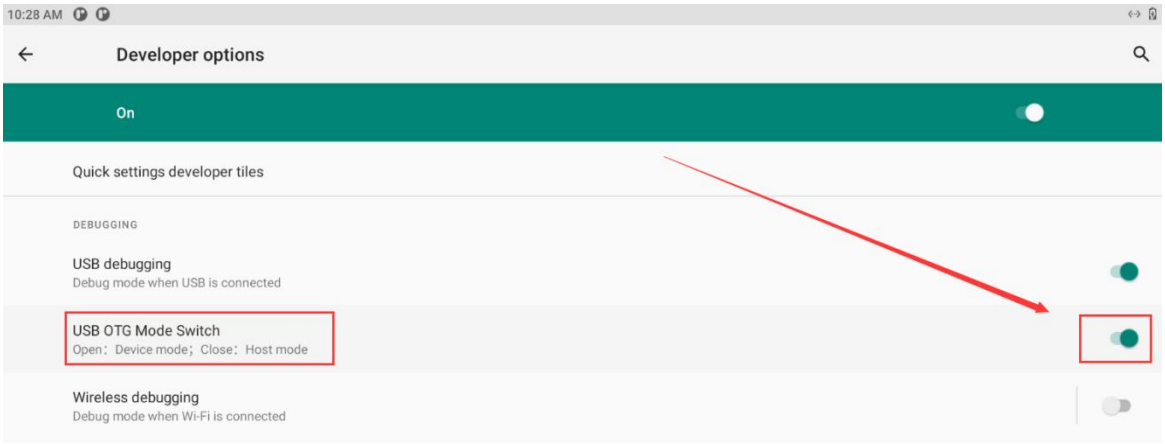
6) 然后选择 **Advanced**



7) 然后在展开的栏目中选择 **Developer options**



8) 最后找到 **USB OTG Mode Switch** 开关，**打开开关切换为 Device 模式，关闭开关切换为 Host 模式**

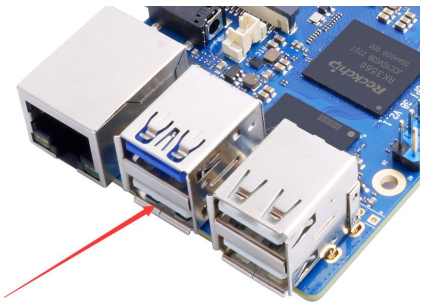


7.9.2. 使用数据线连接 adb 调试

1) 首先准备一根品质良好的 USB2.0 公对公数据线

2) 然后参考 **USB OTG 模式切换的方法**将 USB OTG 切换为 Device 模式

3) 然后通过 USB2.0 公对公数据线连接好开发板与 Ubuntu PC，开发板 USB OTG 接口的位置如下图所示：



4) 然后在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt update
test@test:~$ sudo apt -y install adb
```

5) 通过下面的命令可以查看识别到的 ADB 设备

```
test@test:~$ adb devices
List of devices attached
S63QCF54CJ      device
test@test:~$ lsusb
Bus 003 Device 006: ID 2207:0006
```

6) 然后在 Ubuntu PC 上通过 adb shell 就可以登录到 android 系统了

```
test@test:~$ adb shell
console:/ $
```

7）执行下面的命令可以重新挂载 Android 系统

```
test@test:~$ adb root
test@test:~$ adb remount
```

8）然后就可以传输文件到 Android 系统了

```
test@test:~$ adb push example.txt /system/
```

7.9.3. 使用网络连接 adb 调试

使用网络 adb 无需数据线来连接电脑和开发板，而是通过网络来通信，所以首先请确保开发板的有线或者无线网络已经连接好了，然后获取开发板的 IP 地址，

后面要用到。

1) 确保 Android 系统的 **service.adb.tcp.port** 设置为 5555 端口号

```
console:/ # getprop | grep "adb.tcp"
[service.adb.tcp.port]: [5555]
```

2) 如果 **service.adb.tcp.port** 没有设置，可以使用下面的命令设置网络 adb 的端口号

```
console:/ # setprop service.adb.tcp.port 5555
console:/ # stop adbd
console:/ # start adbd
```

3) 在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt update
test@test:~$ sudo apt install -y adb
```

4) 然后在 Ubuntu PC 上连接网络 adb

```
test@test:~$ adb connect 192.168.1.xxx     (IP 地址需要修改为开发板的 IP 地址)
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.1.xxx:5555

test@test:~$ adb devices
List of devices attached
192.168.1.xxx:5555          device
```

5) 然后在 Ubuntu PC 上通过 adb shell 就可以登录到 android 系统

```
test@test:~$ adb shell
console:/ #
```

8. OpenWRT 系统的使用说明

8.1. OpenWRT 版本

OpenWRT 版本	内核版本
24.10.1	Linux6.6

8.2. OpenWRT 适配情况

功能	OpenWRT
USB2.0x3	OK
USB3.0x1	OK
3pin 调试串口	OK
TF 卡启动	OK
UWE5622 WIFI	NO
M.2 NVMe SSD 启动	OK
网口	OK
网口状态灯	OK
RTL8811 USB 网卡	OK
USB 转有线网卡	OK
LED 灯	OK
FAN 风扇接口	OK
eMMC 扩展接口	OK

8.3. 第一次启动扩容 rootfs

1) 第一次启动 OpenWRT 系统时会执行 **resize-rootfs.sh** 脚本进行 rootfs 的扩容，并且扩容完成后会自动重启

2) 登录系统后可以通过 **df -h** 命令来查看 rootfs 的大小，如果和存储设备（TF 卡、eMMC 或者 NVME SSD）实际容量一致，说明自动扩容运行正确

```
root@OpenWrt:~# df -h
Filesystem                Size      Used Available Use% Mounted on
```

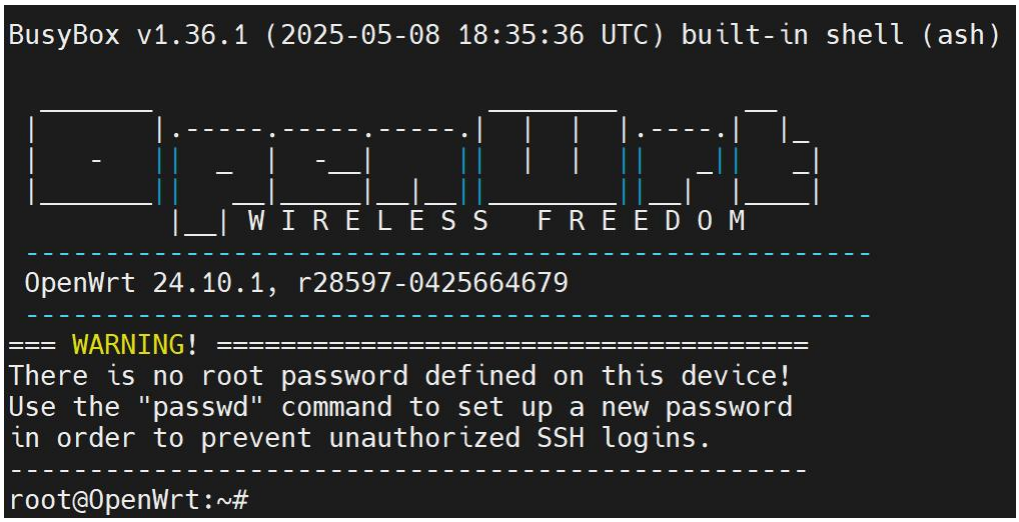
/dev/root	**14.8G**	**14.7G**	**91.6M**	**99% /**
tmpfs	495.5M	6.1M	489.4M	1% /tmp
tmpfs	512.0K	0	512.0K	0% /dev
/dev/root	14.8G	14.7G	91.6M	99% /opt/docker

8.4. 登录系统的方法

8.4.1. 通过串口登录

1) 首先调试串口的使用可以参考调试串口的使用方法章节

2) OpenWrt 系统默认会以 **root** 用户自动登录，显示界面如下



8.4.2. 通过 SSH 登录系统

请注意，在 Orange Pi 3B 的 OpenWrt 系统中，默认将板载网口配置为 LAN 口功能

1) 首先用网线连接板子的 LAN 口到电脑的网口，以便电脑的网口能通过 DHCP 获取到 IP 地址

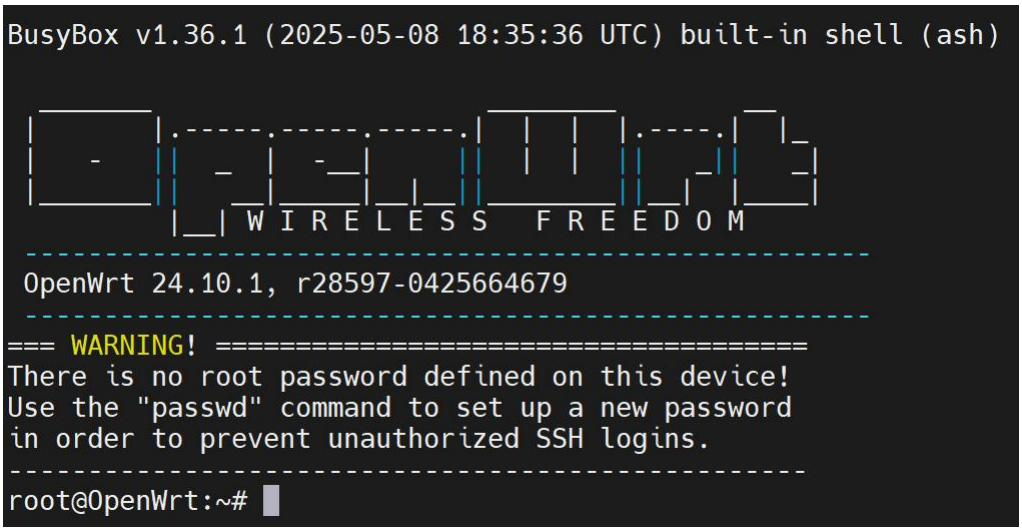
2) 默认板子的 LAN 口 IP 是设置为 **192.168.2.1** 的，所以此时电脑可以获取到以 **192.168.2** 开头的 IP 地址

3) 如果电脑安装的 Ubuntu 系统，可以执行下列命令通过 SSH 登录系统，默认是不

需要密码直接就可以登录的

```
test@ubuntu:~$ ssh root@192.168.2.1
```

4) 成功登录系统后的显示如下图所示



5) 如果电脑装的 Windows 系统，可以参考 Linux 系统使用说明中的 **Windows 下 SSH 远程登录开发板**的方法进行登录

8.4.3. 登录 LuCI 管理界面

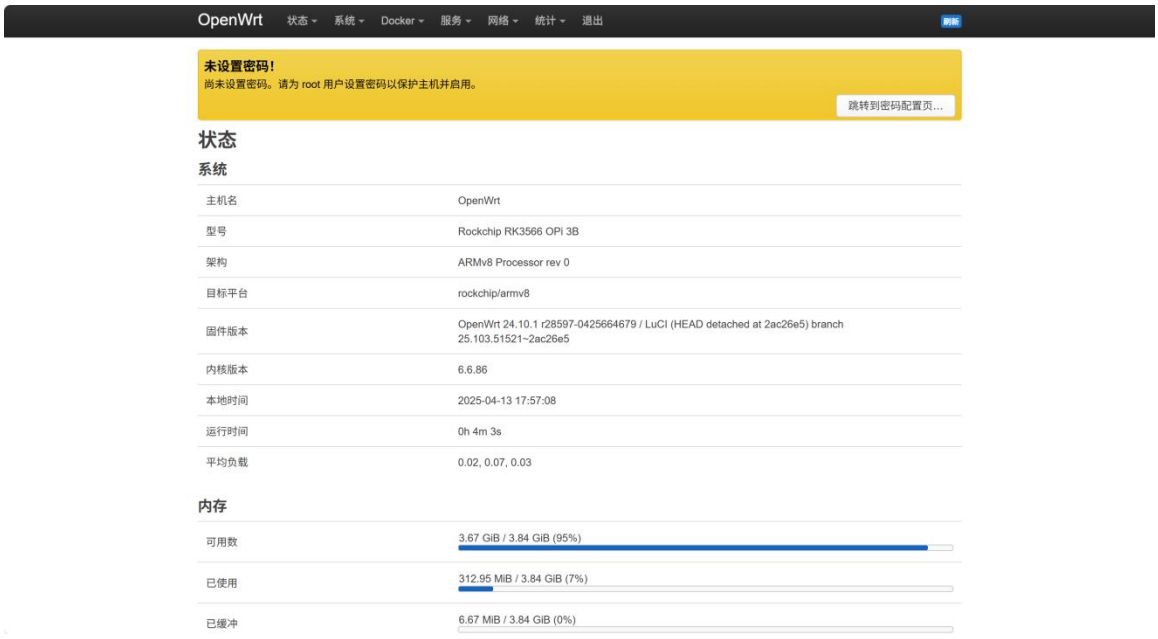
请注意，在 Orange Pi 3B 的 OpenWrt 系统中，默认将板载网口配置为 LAN 口功能

1) 首先用网线连接板子的 LAN 口到电脑的网口，以便电脑的网口能通过 DHCP 获取到 IP 地址

2) 默认板子的 LAN 口 IP 是设置为 **192.168.2.1** 的，所以此时电脑可以获取到以 **192.168.2** 开头的 IP 地址

3) 在电脑上的浏览器中输入 IP 地址 **192.168.2.1** 就可以登录 LuCI 界面了

4) OpenWrt 系统默认是没有设置密码的，所以直接点击登录按钮即可，登录成功后界面显示如下图所示



8. 4. 4. 通过 LuCI 管理界面登录终端

请注意，在 Orange Pi 3B 的 OpenWrt 系统中，默认将板载网口配置为 LAN 口功能

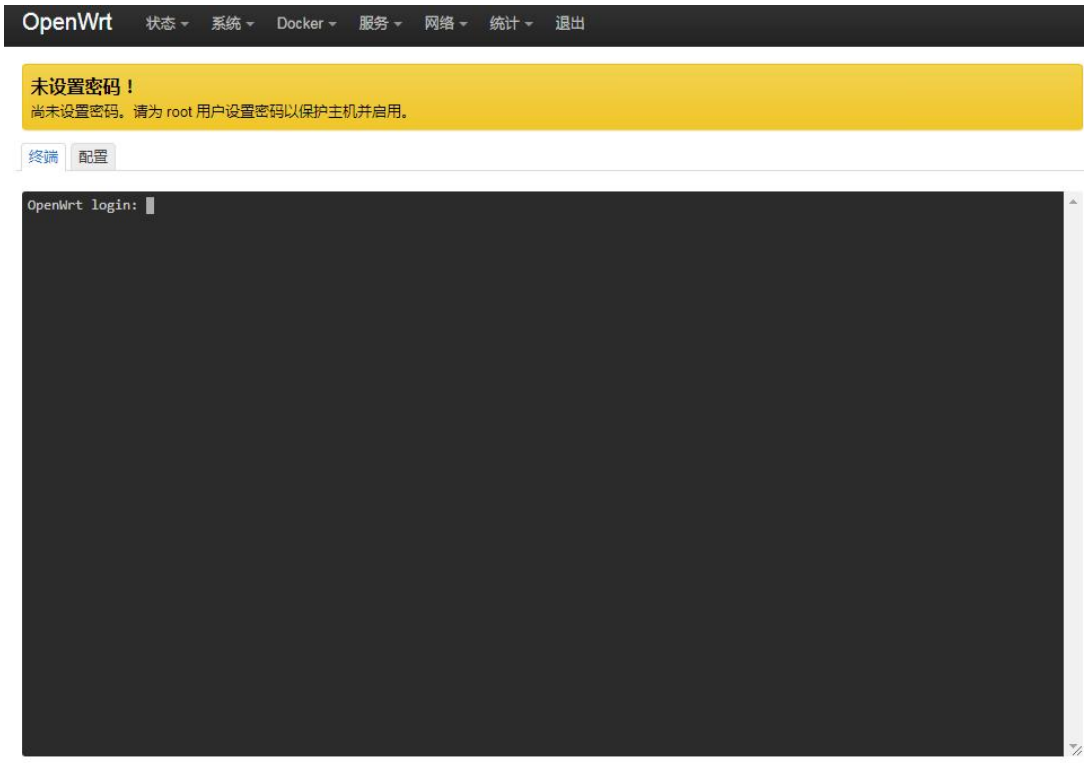
1) 首先用网线连接板子的 LAN 口到电脑的网口，以便电脑的网口能通过 DHCP 获取到 IP 地址

2) 默认板子的 LAN 口 IP 是设置为 **192.168.2.1** 的，所以此时电脑可以获取到以 **192.168.2** 开头的 IP 地址

3) 在电脑上的浏览器中输入 IP 地址 **192.168.2.1** 就可以登录 LuCI 界面了

4) 在导航栏的**“服务”**一栏中选择**“终端”**并点击进入

5) 此时终端显示界面如下图所示



6) 输入用户名 **root** 即可登录

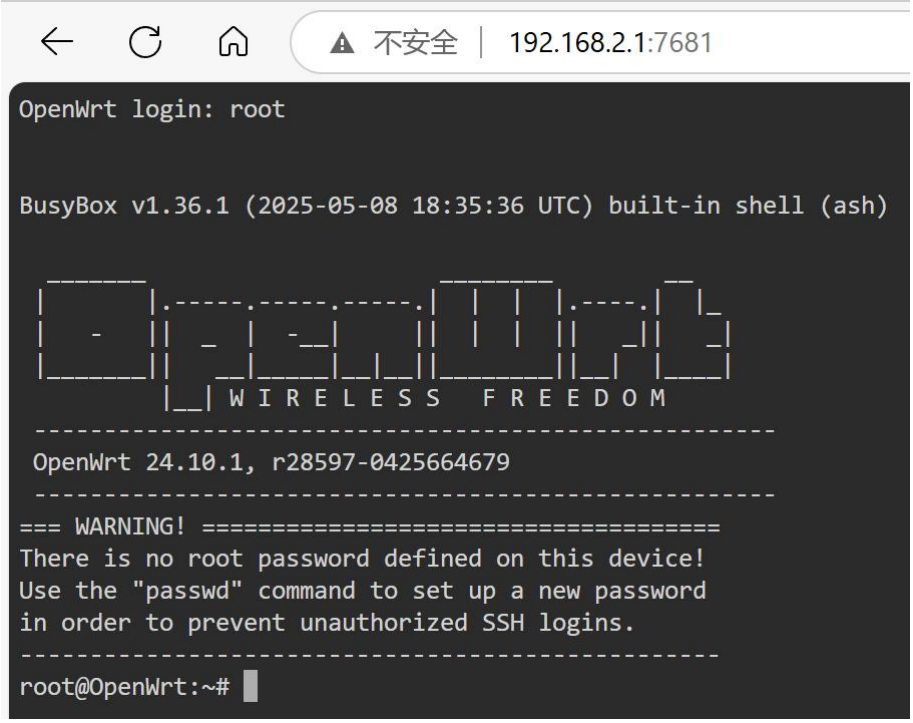
8.4.5. 使用 IP 地址+端口号方式登录终端

请注意，在 Orange Pi 3B 的 OpenWrt 系统中，默认将板载网口配置为 LAN 口功能

1) 首先用网线连接板子的 LAN 口到电脑的网口，以便电脑的网口能通过 DHCP 获取到 IP 地址

2) 默认板子的 LAN 口 IP 是设置为 **192.168.2.1** 的，所以此时电脑可以获取到以 **192.168.2** 开头的 IP 地址

3) 然后在浏览器输入 **192.168.2.1:7681** 即可登录 OpenWRT 的终端



8.5. 通过命令行修改 LAN 口 IP 地址的方法

1) 在 OpenWrt 系统中，提供了一个命令行工具 uci，它可以很方便地对配置文件中的内容进行修改、添加、删除和读取，详细说明请参考官方文档

2) 首先使用以下命令获取网络配置，对应的配置文件是**/etc/config/network**，可以看到 **network.lan.ipaddr** 这一项的值是 **192.168.2.1**

```
root@OpenWrt:~# uci show network
...
network.lan=interface
network.lan.device='br-lan'
network.lan.proto='static'
network.lan.ipaddr='192.168.2.1'
network.lan.netmask='255.255.255.0'
network.lan.ip6assign='60'
....
```

3) 然后输入以下命令修改 **network.lan.ipaddr** 这一项

```
root@OpenWrt:~# uci set network.lan.ipaddr='192.168.100.1'
```

4) 然后输入以下命令完成提交，即写入到配置文件

```
root@OpenWrt:~# uci commit
```

如果红色字体的 IP 地址与要设置的一致，说明修改成功

```
root@OpenWrt:~# cat /etc/config/network
...
config interface 'lan'
        option device 'br-lan'
        option proto 'static'
        option netmask '255.255.255.0'
        option ip6assign '60'
        option ipaddr '192.168.100.1'
...
```

5) 通过 ubus 重启网络，ubus 的使用说明请参考官方文档

```
root@OpenWrt:~# ubus call network restart
```

6) 此时，输入命令可以看到 LAN 口的 IP 已经是 **192.168.100.1**

```
root@OpenWrt:~# ifconfig br-lan
br-lan    Link encap:Ethernet  HWaddr FE:55:13:A3:EF:E7
          inet addr:192.168.100.1  Bcast:192.168.100.255  Mask:255.255.255.0
          inet6 addr: fd60:c4cd:1033::1/60 Scope:Global
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:370 (370.0 B)
```

8.6. 修改 root 密码的方法

8.6.1. 通过命令行修改

1) 首先在系统的命令行输入 passwd root，会出现以下提示信息，此时可以输入想要设置的密码，按下回车键确认

```
root@OpenWrt:/# passwd root
Enter new UNIX password:
```

2) 接着会提示重新输入密码，此时再次输入密码确认并回车即可

```
Retype password:
```

3) 修改成功的显示如下

```
passwd: password for root changed by root
```

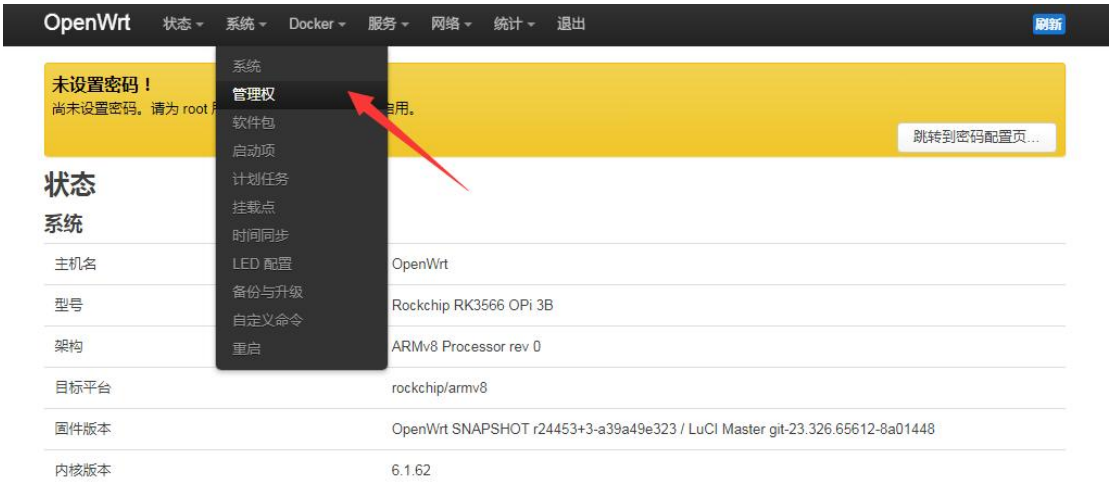
8.6.2. 通过 LuCI 管理界面修改

1) 首先参考**登录 LuCI 管理界面**进入 OpenWRT 的管理界面

2) 然后按照下面的步骤修改密码

a. 在导航栏中找到“**系统**”选项并点击

b. 在系统下方的竖栏选项中，选择“**管理权**”并点击



c. 在 Tab 页中选择“**路由器密码**”选项

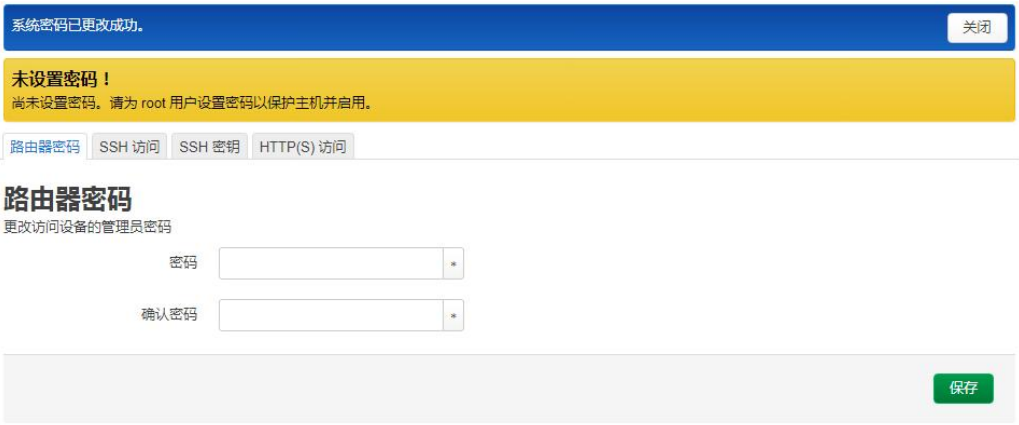
3) 修改并保存路由器密码

a. 在“**密码**”和“**确认密码**”的对话框中输入自己设定的密码（如果不确定密码是否输入正确，可点击对话框后面的“*”图标以显示输入字符）

b. 点击“**保存**”即可保存新修改的密码

注：在“密码”和“确认密码”的对话框中，需要两次输入的密码保持一致。

4) 密码修改成功之后，会弹出“**系统密码已更改成功**”的弹框，此时登录 OpenWRT 就需要密码才能登录



8.7. USB 接口测试

8.7.1. 在命令行下挂载 USB 存储设备

1) 首先将 U 盘插入 Orange Pi 开发板的 USB 接口中

2) 执行下面的命令如果能看到 sdX 的输出说明 U 盘识别成功

```
root@OpenWrt:~# cat /proc/partitions | grep "sd*"
major minor  #blocks  name
   8        0   15126528 sda
```

3) 使用 mount 命令可以将 U 盘挂载到/mnt 中，然后就能查看 U 盘中的文件了

```
root@OpenWrt:~# mount /dev/sda /mnt/
root@OpenWrt:~# ls /mnt/
test.txt
```

4) 挂载完后通过 df -h 命令就能查看 U 盘的容量使用情况和挂载点

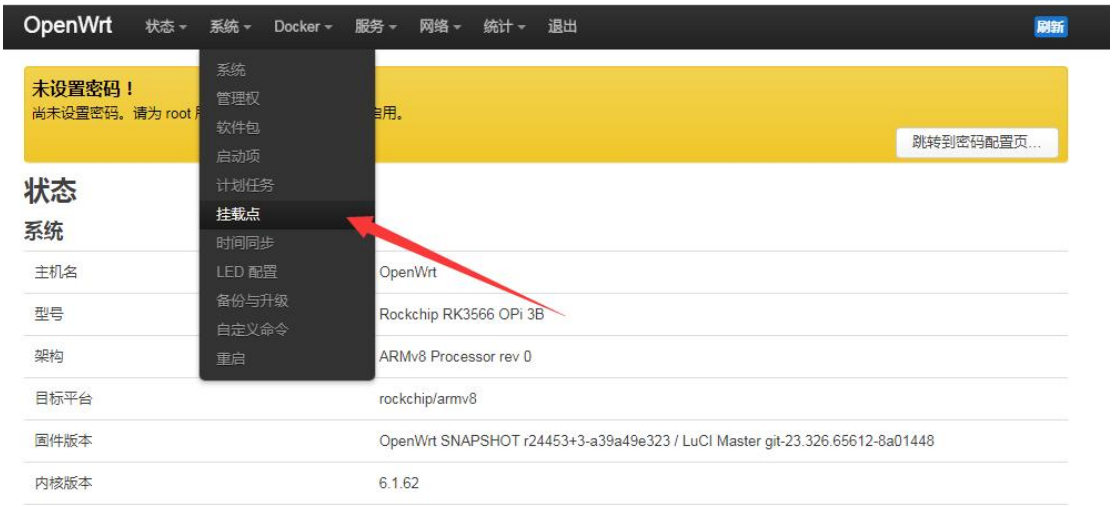
```
root@OpenWrt:~# df -h | grep "sd"
/dev/sda                 14.4G     187.2M     14.2G   1% /mnt
```

8.7.2. 在 LuCI 管理界面挂载 USB 存储设备

1) 首先将 U 盘（或者其它存储设备）通过 USB2.0 连接到开发板

2) 然后按照**登录 LuCI 管理界面**进入 LuCI 管理界面

3) 然后在 LuCI 管理界面中，点击“系统->挂载点”进入挂载点的配置界面

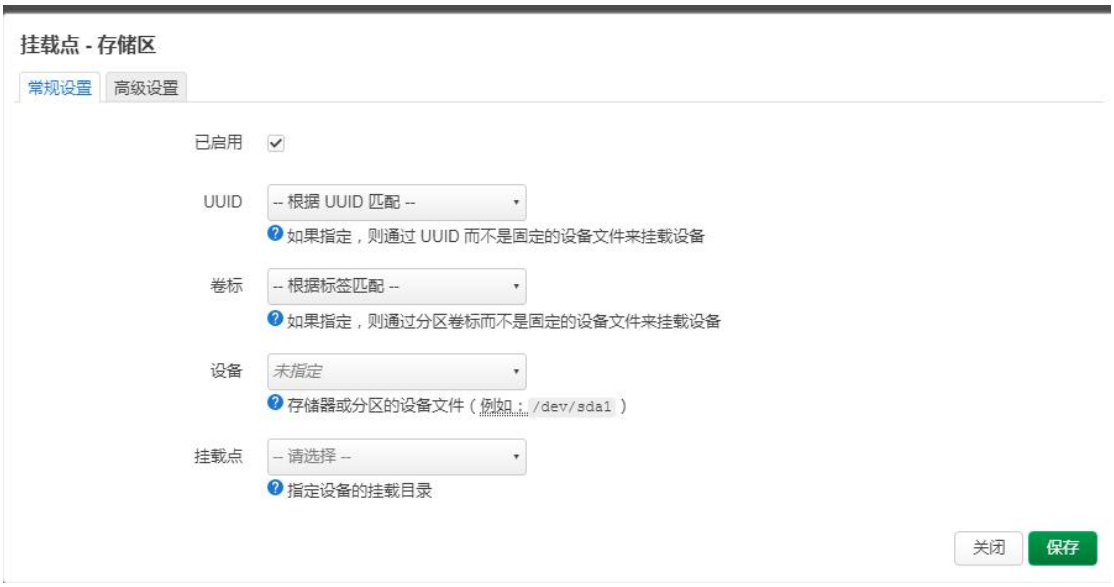


4) 然后按照下面的步骤新增一个挂载点

a. 在挂载点**全局设置界面**下方找到“**挂载点**”

b. 在**挂载点**下方，选择“添加”按钮并点击进入

c. 接着会弹出下面的弹窗界面



d. 然后就可以开始挂载存储设备

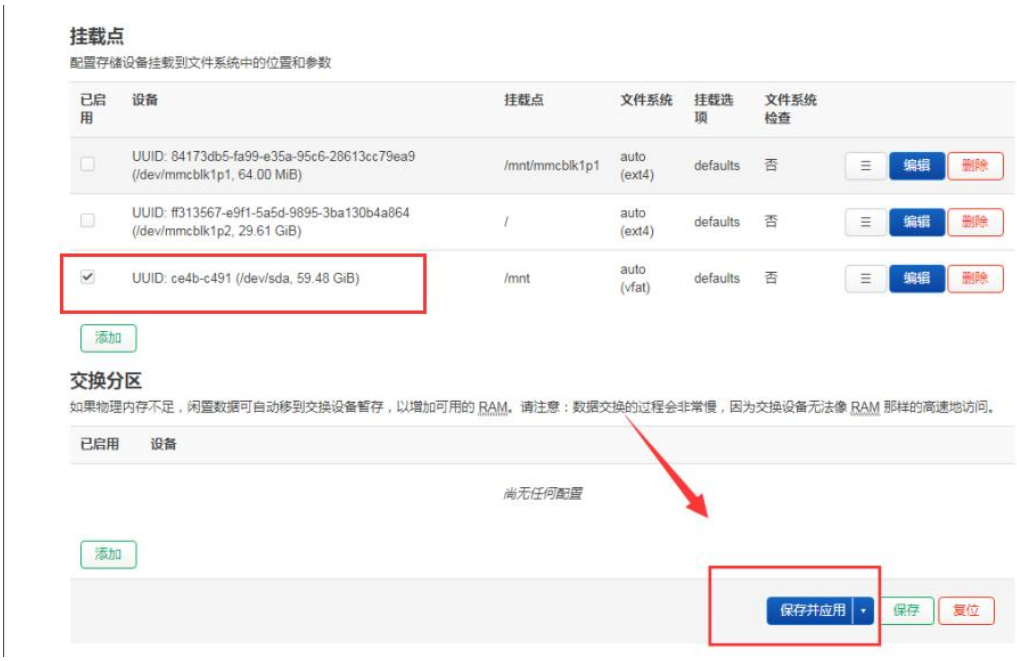
a) 勾选“**已启用**”

b) 在常规设置 UUID 一栏中选择实际接入的设备/dev/sda（根据自己的设备选择）

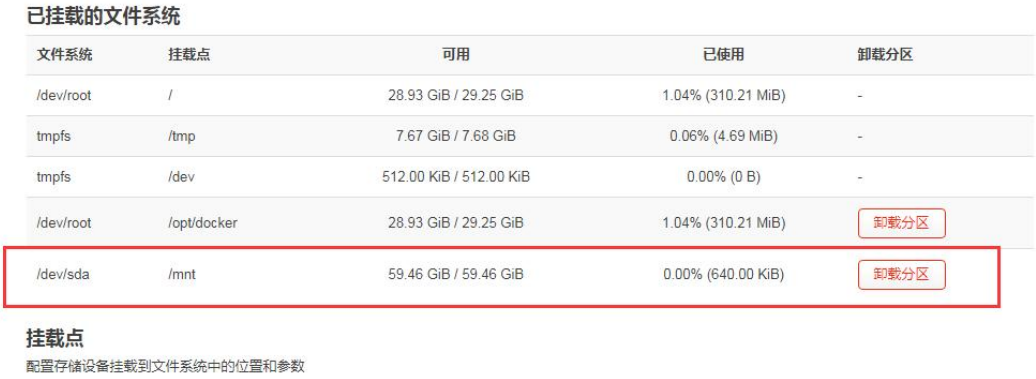
c) 在挂载点一栏中选择“**自定义**”，并填入要挂载到的目标目录，这里以**/mnt**目录为例，填好后**回车**确认

d) 然后点击右下角的“**保存**”按钮

5) 然后会回到挂载点全局设置页面，在页面的左下角点击“**保存并应用**”使挂载点生效



6) 保存完后可以看到“**已挂载的文件系统**”中，该存储设备已经挂载成功



8.8. USB 无线网卡测试

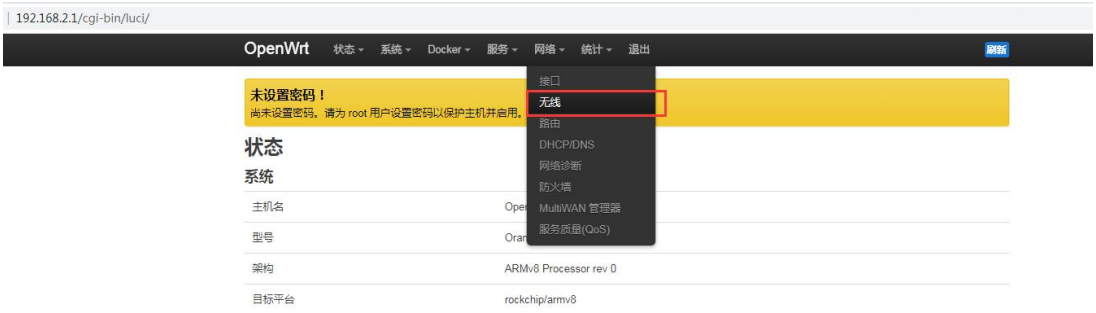
目前**测试过的**能用的 USB 无线网卡如下所示，其他型号的 USB 无线网卡请自行测试，如果无法使用就需要移植对应的 USB 无线网卡驱动

序号	型号	
1	RTL8811 支持 2.4G +5G WIFI	

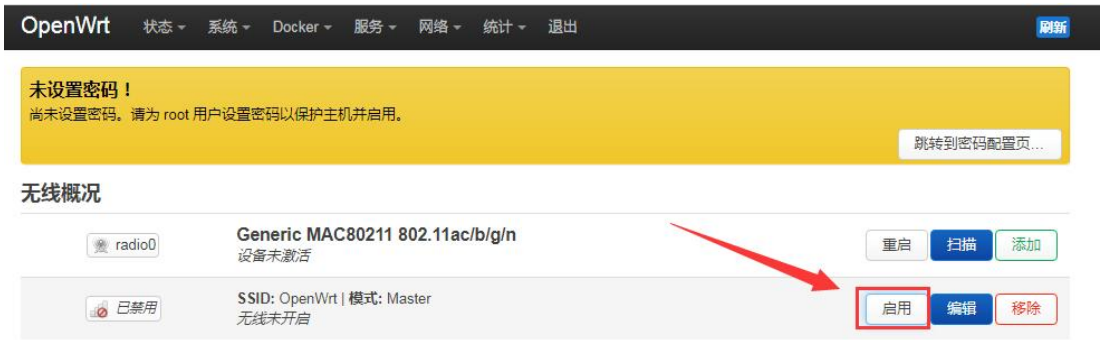
8.8.1. 使用 USB 无线网卡创建 WIFI 热点的方法

1）将 USB 无线网卡插入开发板的 USB 端口，然后接上电源给开发板上电

2）系统启动完成后，点击 **网络 -> 无线** 进入无线 WiFi 配置界面，如果没有**无线**这个选项，说明 USB 无线网卡型号不被系统支持

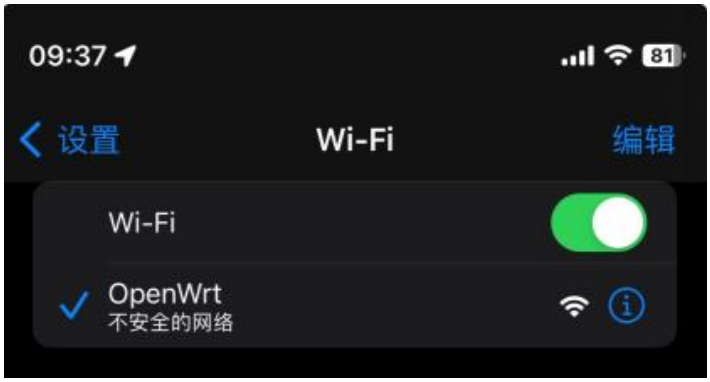


3）OpenWRT 系统默认的无线配置就是 **Master** 模式，我们直接点击**启用**，就可以创建一个开放的 WIFI 热点



4）成功创建 WIFI 热点的显示界面如下图所示

5）此时就可以通过手机连接名称为 **OpenWrt** 的 WIFI 热点了

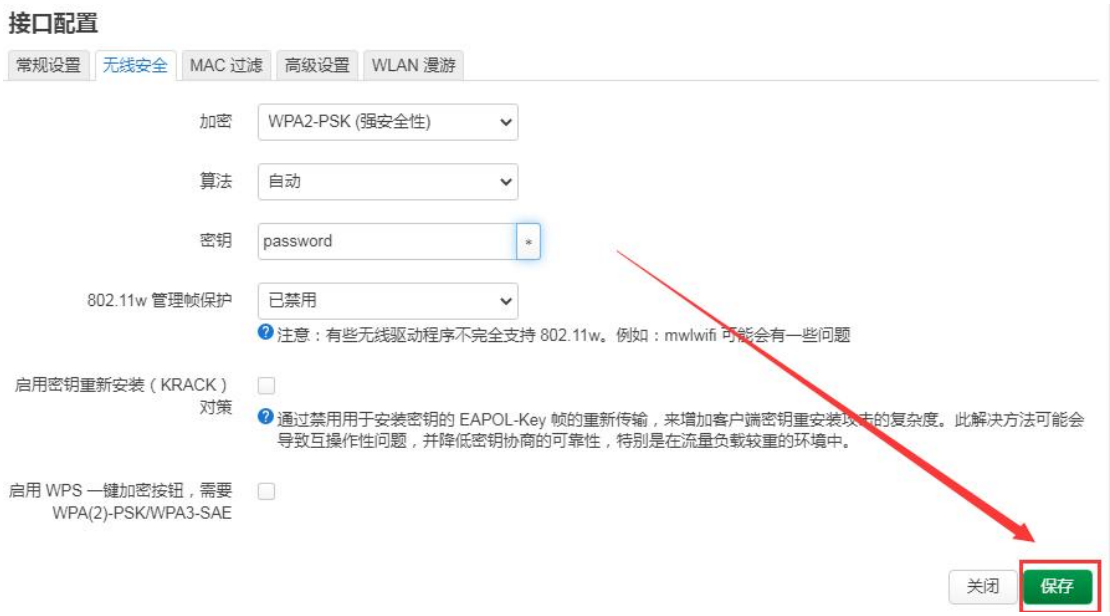


6）如果想要创建一个有密码的 WIFI 热点，点击**编辑**选项

7）在弹出标签页中，我们点击**无线安全**一栏。

8）然后在 **接口配置 -> 无线安全** 中，加密算法选择 **WPA2-PSK**；密钥(无线密码)设置为 **password**

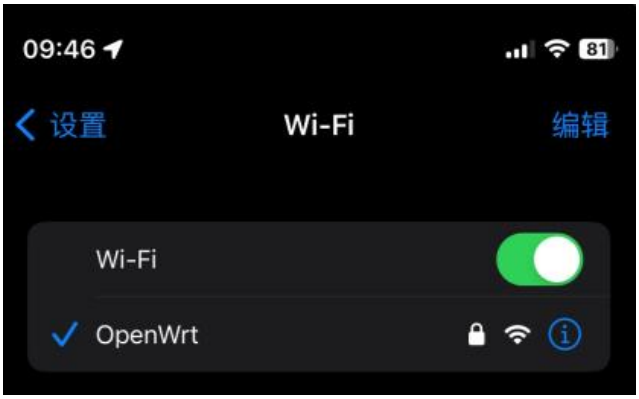
9）以上设置完成后，点击页面右下角 **保存**，之后会退出标签页



10）然后点击页面右下角 **保存并应用** 等待配置被应用完成。

11）成功创建热点的显示界面如下图所示，此时可以看到 WIFI 热点已加密

12）然后使用手机或电脑搜索对应 SSID 的 WiFi 进行连接，连接成功后如下图所示



8. 8. 2. 使用 USB 无线网卡连接 WIFI 热点的方法

1）将 USB 无线网卡插入开发板的 USB 端口，然后接上电源给开发板上电

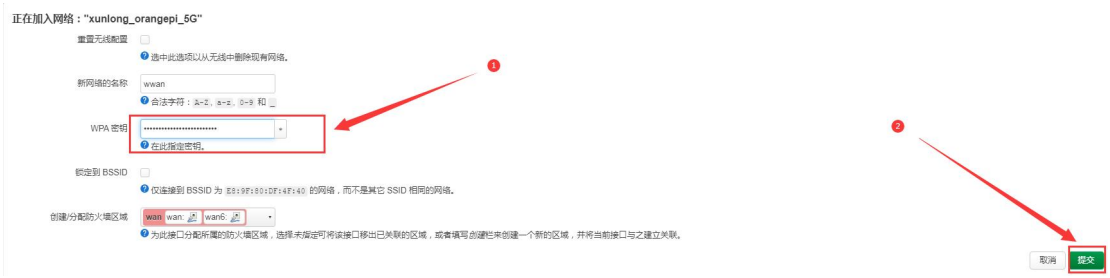
2）系统启动完成，点击**“网络>无线”**进入配置无线 WiFi 界面

3）首先需要将默认的无线配置移除，点击下图所示的**“移除”**按钮

4）然后点击**“扫描”**按钮扫描周围的 WiFi 热点

5）然后会弹出下面的窗口显示可用的 WiFi 热点，点击想要连接的 WiFi 热点右边的**“加入网络”**按钮

6）然后在下图所示的位置输入密码，再点击**“提交”**



7）然后会弹出下面的界面，点击保存即可

8）最后会回到无线配置的主界面，点击**“保存并应用”**

9）成功连接到 WiFi 热点后，界面显示如下图所示

8.9. 通过命令行安装软件包

8.9.1. 在终端通过 opkg 安装

1) 更新可获取的软件包列表

```
root@OpenWrt:/# opkg update
```

2) 获取软件列表

```
root@OpenWrt:/# opkg list
```

3) 安装指定的软件包

```
root@OpenWrt:/# opkg install <包名称>
```

4) 查看已经安装的软件

```
root@OpenWrt:/# opkg list-installed
```

5) 卸载软件

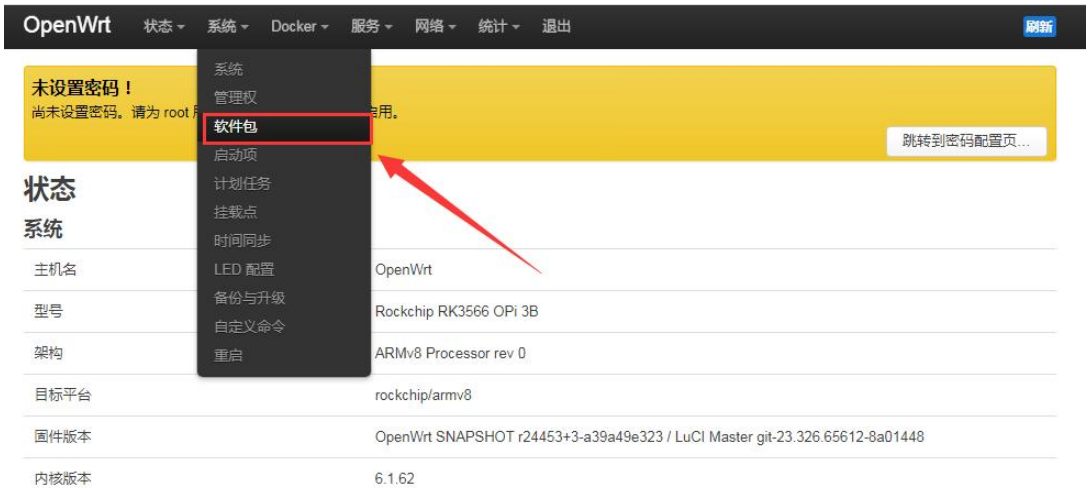
```
root@OpenWrt:/# opkg remove <包名称>
```

8.10. OpenWRT 管理界面安装软件包

若需要新增软件包，可通过 OpenWRT 的管理界面进行安装

8.10.1. 查看系统可用软件包列表

1) 首先进入软件包管理页面
   a. 在导航栏中找到“**系统**”选项并点击进入
   b. 在系统下方的竖栏选项中，选择“**软件包**”并点击进入



2) 然后会出现软件包的主页面，如下图所示，获取可用的软件列表
   a. 在软件包的“**操作**”选项中，点击“**更新列表**”，获取可用的软件包列表
   b. 在 Tab 页中，点击“**可用**”，查看当前可用软件包
   c. 查看当前可用软件包的数量

8.10.2. 安装软件包示例

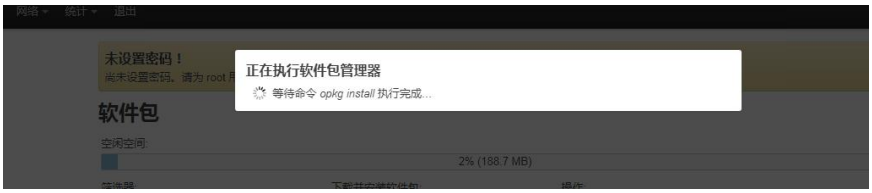
1) 以安装软件包“**luci-app-acl**”为例

a. 在 OpenWRT 的软件包管理界面下，点击筛选器的对话框并输入“**luci-app-acl**”

b. 在软件包的列表中可以看到“**luci-app-acl**”软件包的版本、软件包大小以及描述信息，然后点击“**安装**”按钮

c. 然后会出现以下弹窗，点击“**安装**”即可

d. 然后等待安装完成



e. 安装完成的显示如下

2) 查看软件包是否安装成功

a. 在 OpenWRT 的软件包管理界面下，点击筛选器的对话框并输入“**luci-app-acl**”

b. 在 Tab 页选择点击“**可用**”

c. 在软件包列表中会显示“**luci-app-acl**”软件包，并更新状态为“**已安装**”

8.10.3. 移除软件包示例

1) 以移除软件包“**luci-app-acl**”为例

a. 在 OpenWRT 的软件包管理界面下，点击筛选器的对话框并输入

“**luci-app-acl**”

b. 在 Tab 页中选择“**已安装**”，显示已安装的软件包列表

c. 点击右边的“**移除**”，即可移除相应的软件包

a. 然后会显示下面弹窗，点击“**移除**”即可

2) 查看软件包是否移除成功

a. 在 OpenWRT 的软件包管理界面下，点击筛选器的对话框并输入“**luci-app-acl**”

b. 在 Tab 页选择点击“**已安装**”

c. 在软件包列表中不会显示“**luci-app-acl**”软件包，此时“**luci-app-acl**”软件包已移除成功

8. 11. 使用 Samba 网络共享

OpenWRT局域网文件共享实现主要有两个软件选择，Samba和NFS。Samba系统兼容性较好，NFS则性能表现占优。对于需要使用Windows设备的用户来说，建议选择Samba。

1) 进入 Samba 网络共享的管理页面
   a. 在导航栏中找到“**服务**”选项并点击进入
   b. 在服务下方的竖栏选项中，选择“**网络共享**”并点击进入

2) 选择 Samba 服务需要监听的接口
   a. 在网络共享的导航栏中选择“**常规设置**”并点击进入

b. 接口根据实际需求指定，如果想通过“wan 口”访问，则设置为“**wan**”

3) 设置网络共享的共享目录

a. 在网络共享的“**常规设置**”的“**共享目录**”总点击“**新增**”共享目录地址

b. 在名称下面输入共享文件夹的名称为“**mmt**”

c. 在共享目录的路径下，选择设置共享的目录位置“**/mnt**”

d. 勾选“**可浏览**”和“**运行匿名用户**”

e. 点击“**保存并应用**”，保存配置

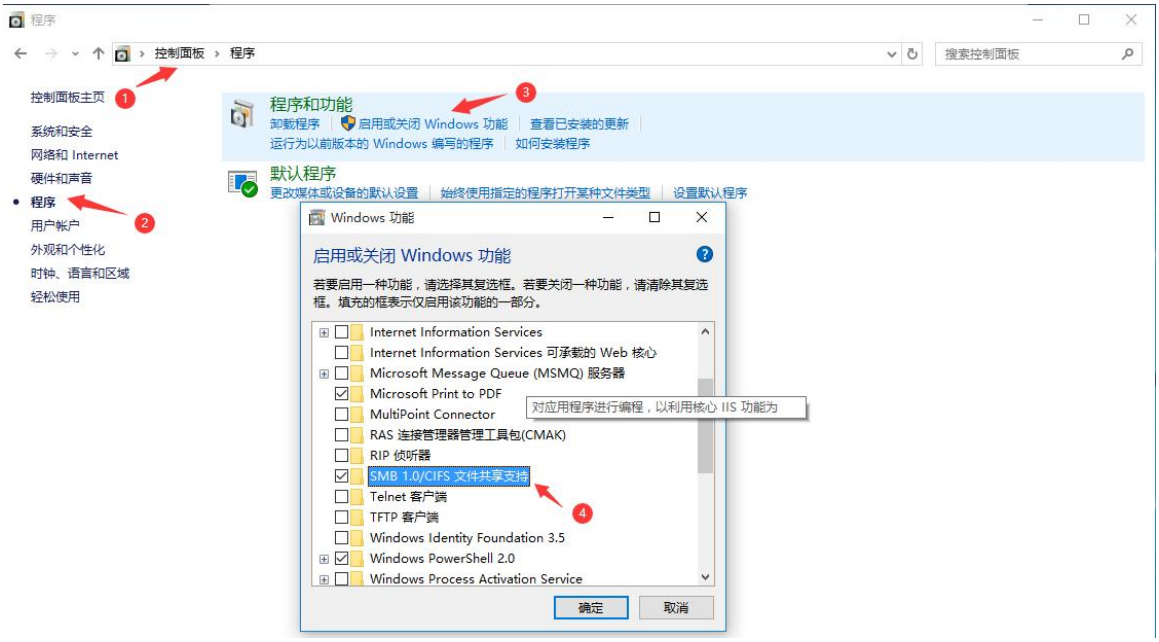
4) window10 启动网络发现与共享

注：在windows10 系统下访问Samba，共享需要先确认window10 是否已经启动网络发现与共享，如未启动，先进行如下设置。

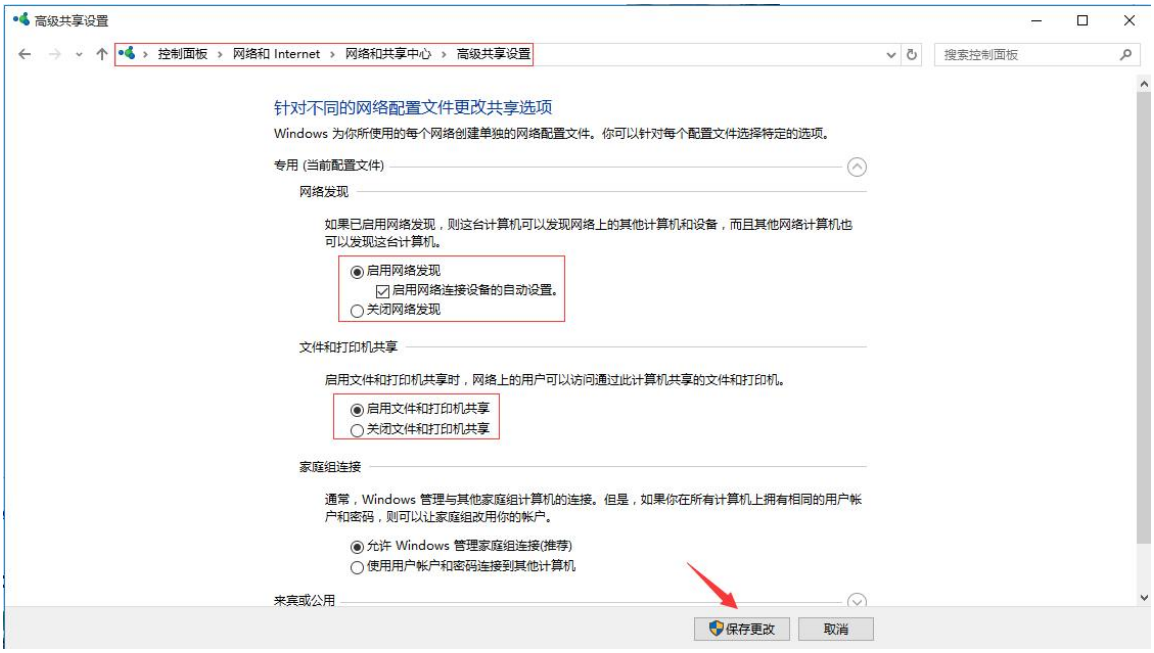
a. 启用 Samba v1/v2 的访问

a) 进入 windows10 的“控制面板”

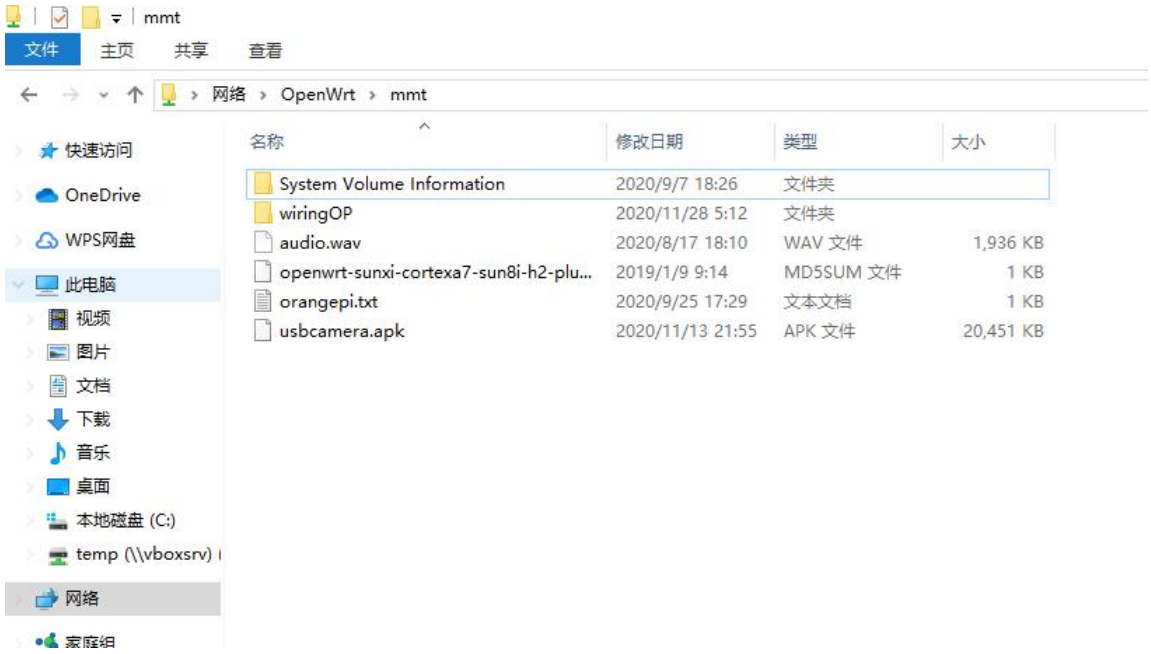
b) 在控制面板的左侧导航栏点击“程序”
c) 在程序和功能中选择“启用或关闭 Windows 功能”
d) 在启用或关闭 Windows 功能的弹框中勾选“SMB 1.0/CIFS 文件共享支持”
e) 点击“确定”，配置应用



b. 打开 Windows10 的网络发现
a) 进入 windows10 的“控制面板”
b) 在控制面板中选择“网络和 Internet”
c) 然后打开“网络和共享中心”
d) 点击|“高级共享设置”
e) 打开“**启用网络发现**”和“**启用文件和打印机共享**”
f) 点击“保存更改”，保存 Windows10 的网络发现配置



5) 设置完成后，在资源管理器的地址栏输入\\OpenWrt 即可访问共享目录,用户名为root,密码为开发板主机设定的密码

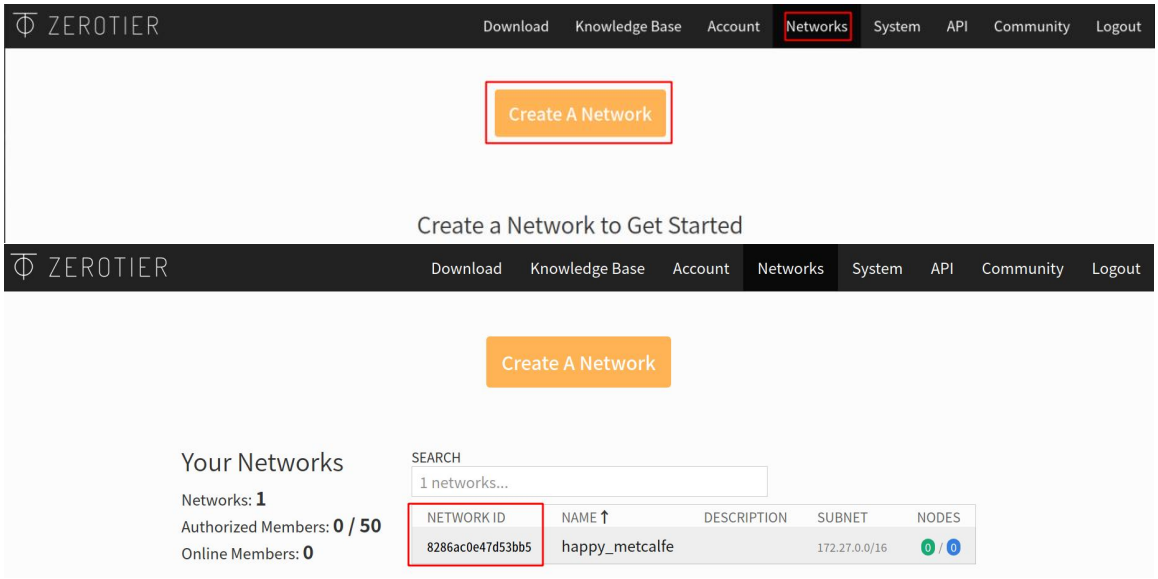


8.12. zerotier 使用说明

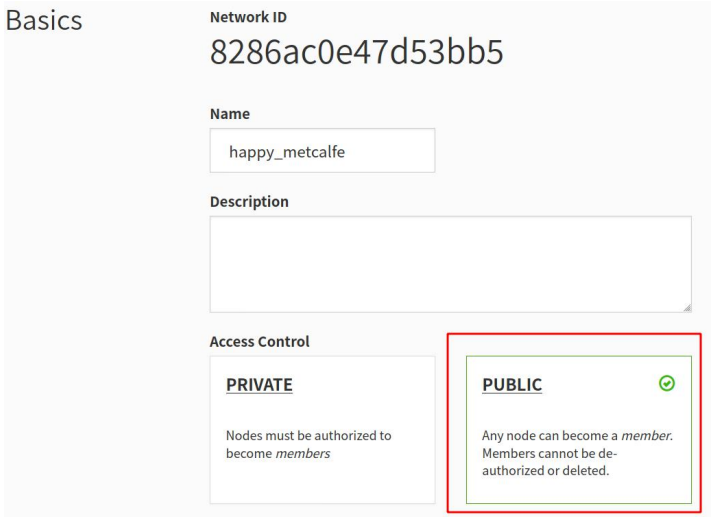
OpenWRT系统已经预装了zerotier客户端，在zerotier官网创建虚拟局域网后，

在客户端可以通过Network ID直接加入其中，具体操作如下文所示。

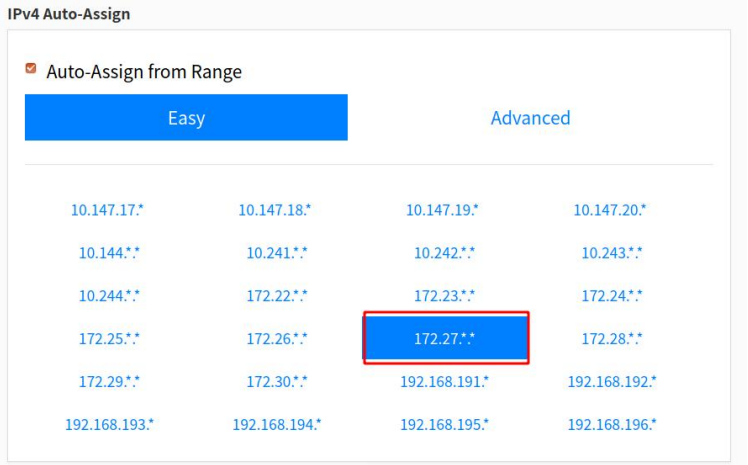
1) 登录 zerotier 官网 **https://my.zerotier.com/network**，注册登录后点击 Network->Create A Network，创建一个虚拟局域网



2) 点击进入网络控制台页面，可以把隐私选项设置为公共，这样加入的网络节点就不需要验证



3) 下面自动分配地址这里可以自己选择网段，此处选择的是 172.27.*.*



4) 在 OpenWRT 终端输入下面命令就可以加入到上面创建的虚拟局域网，**其中 8286ac0e47d53bb5 是上面创建的虚拟局域网的 Network ID**

```
root@OpenWrt:/# zerotier-one -d                    #启动 zerotier 客户端
root@OpenWrt:/# zerotier-cli join 8286ac0e47d53bb5  #加入网络
```

5) 在终端输入 ifconfig 可以看到已经有个新增的 **ztks54inm2** 设备，IP 地址为 **172.27.214.213**

```
root@OpenWrt:/# ifconfig
ztks54inm2 Link encap:Ethernet  HWaddr F6:4E:DE:BF:D8:52
          inet addr:172.27.214.213  Bcast:172.27.255.255  Mask:255.255.0.0
          inet6 addr: fe80::e82f:d0ff:fe5a:867e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:2800  Metric:1
          RX packets:18 errors:0 dropped:0 overruns:0 frame:0
          TX packets:48 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1720 (1.6 KiB)  TX byte81 (8.2 KiB)
```

6) 在另一台设备上安装 zerotier 客户端（此处以 Ubuntu18.04 为例），执行下面命令进行安装，安装完成后需要重启电脑

```
test@ubuntu:~$ curl -s https://install.zerotier.com | sudo bash
```

7) 重启后根据 Network ID 加入虚拟局域网，也可以看到已经获取到 zerotier 分配的 ip 地址，此时该 Ubuntu PC 和开发板处于同一局域网中，两者可以自由通信

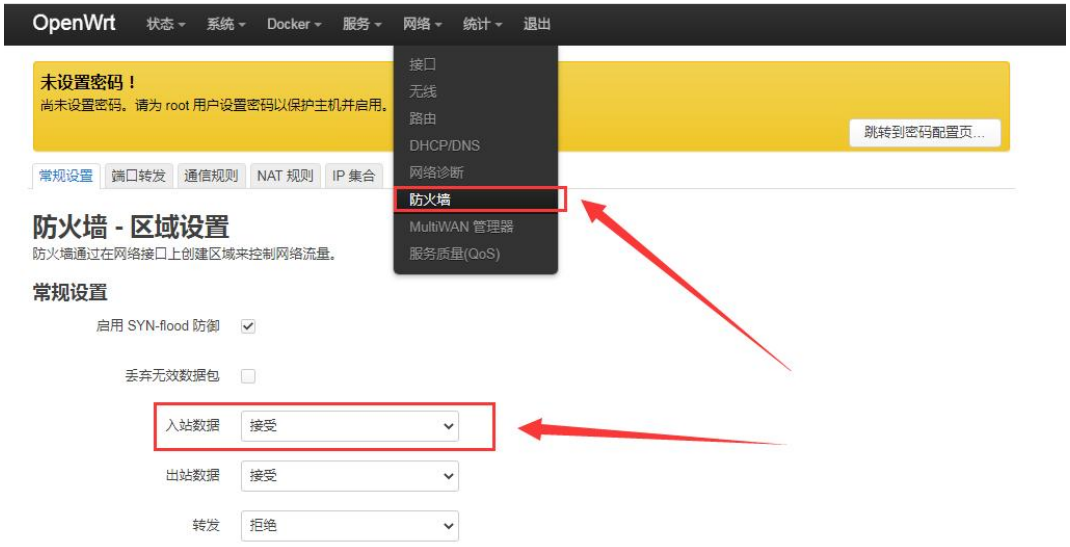
```
test@ubuntu:~$ sudo zerotier-cli join 8286ac0e47d53bb5
```

```
test@ubuntu:~$ ifconfig
ztks54inm2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 2800
        inet 172.27.47.214  netmask 255.255.0.0  broadcast 172.27.255.255
        inet6 fe80::5ce1:85ff:fe2b:6918  prefixlen 64  scopeid 0x20<link>
        ether f6:fd:87:68:12:cf  txqueuelen 1000  (以太网)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 46  bytes 10006 (10.0 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

8) 测试两个终端是否可以通信

```
root@OpenWrt:/# ping 172.27.47.214 -I ztks54inm2
PING 172.27.47.214 (172.27.47.214): 56 data bytes
64 bytes from 172.27.47.214: seq=0 ttl=64 time=1.209 ms
64 bytes from 172.27.47.214: seq=1 ttl=64 time=1.136 ms
64 bytes from 172.27.47.214: seq=2 ttl=64 time=1.203 ms
64 bytes from 172.27.47.214: seq=3 ttl=64 time=1.235 ms
^C
--- 172.27.47.214 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 1.136/1.195/1.235 ms
```

9) 如果电脑端无法 ping 通开发板，请在登录 LuCI 界面，然后把防火墙配置的入站数据改为接受



10) zerotier 其他常用命令

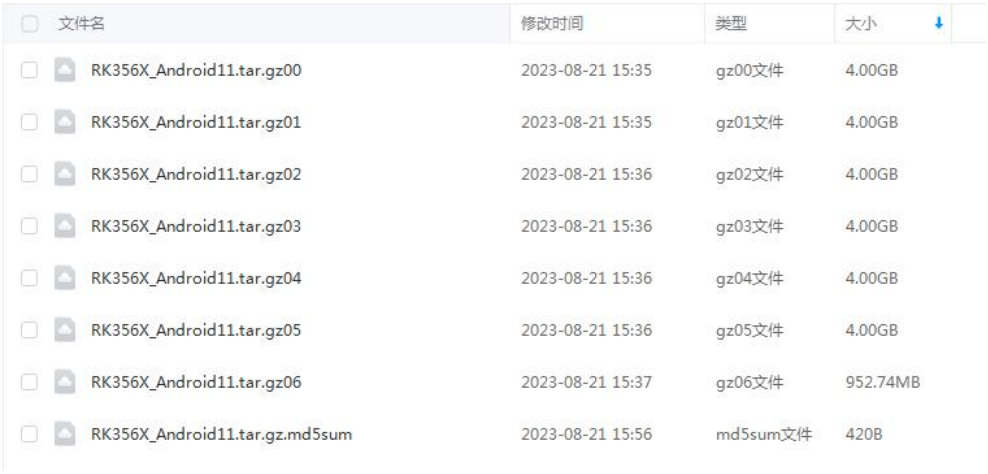
```
root@OpenWrt:/# zerotier-one -d                          #启动 zerotier 客户端
root@OpenWrt:/# zerotier-cli status                      #获取地址和服务状态
root@OpenWrt:/# zerotier-cli join # Network ID           #加入网络
root@OpenWrt:/# zerotier-cli leave # Network ID          #离开网络
root@OpenWrt:/# zerotier-cli listnetworks                #列出网络
OPENWRT_DEVICE_REVISION="v0"
OPENWRT_RELEASE="OpenWrt 22.03.4 r20123-38ccc47687"
```

9. Android11 源码的编译方法

9. 1. 下载 Android 11 的源码

1) 首先从百度云盘或者谷歌网盘下载 Android 11 源码的分卷压缩包

a. 百度云盘



文件名	修改时间	类型	大小
RK356X_Android11.tar.gz00	2023-08-21 15:35	gz00文件	4.00GB
RK356X_Android11.tar.gz01	2023-08-21 15:35	gz01文件	4.00GB
RK356X_Android11.tar.gz02	2023-08-21 15:36	gz02文件	4.00GB
RK356X_Android11.tar.gz03	2023-08-21 15:36	gz03文件	4.00GB
RK356X_Android11.tar.gz04	2023-08-21 15:36	gz04文件	4.00GB
RK356X_Android11.tar.gz05	2023-08-21 15:36	gz05文件	4.00GB
RK356X_Android11.tar.gz06	2023-08-21 15:37	gz06文件	952.74MB
RK356X_Android11.tar.gz.md5sum	2023-08-21 15:56	md5sum文件	420B

b. 谷歌网盘

名称	所有者	上次修改日期	文件大小
RK356X_Android11.tar.gz06	OrangePi	19:55 OrangePi	962.1 MB
RK356X_Android11.tar.gz05	OrangePi	19:31 OrangePi	4 GB
RK356X_Android11.tar.gz04	OrangePi	19:31 OrangePi	4 GB
RK356X_Android11.tar.gz03	OrangePi	19:32 OrangePi	4 GB
RK356X_Android11.tar.gz02	OrangePi	16:37 OrangePi	4 GB
RK356X_Android11.tar.gz01	OrangePi	16:37 OrangePi	4 GB
RK356X_Android11.tar.gz00	OrangePi	16:37 OrangePi	4 GB
RK356X_Android11.tar.gz.md5sum	OrangePi	16:37 OrangePi	420 个字节

2) Android 11 源码的分卷压缩包下载完后，请先检查下 MD5 校验和是否正确，如果不正确，请重新下载源码

```
test@test:~$ md5sum -c RK356X_Android11.tar.gz.md5sum
RK356X_Android11.tar.gz00: OK
RK356X_Android11.tar.gz01: OK
RK356X_Android11.tar.gz02: OK
RK356X_Android11.tar.gz03: OK
RK356X_Android11.tar.gz04: OK
```

```
RK356X_Android11.tar.gz05: OK
RK356X_Android11.tar.gz06: OK
```

3) 然后需要将多个压缩文件合并进行解压

```
test@test:~$ cat RK356X_Android11.tar.gz0* | tar -xvzf -
```

9.2. 编译 Android 11 的源码

1) 首先安装编译 Android11 源码需要的软件包

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y git gnupg flex bison gperf build-essential \
zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 libncurses5 \
lib32ncurses5-dev x11proto-core-dev libx11-dev lib32z1-dev ccache \
libgl1-mesa-dev libxml2-utils xsltproc unzip liblz4-tool
```

2) 源码中有 build.sh 编译脚本，编译参数如下

c. **-U**：编译 uboot
d. **-K**：编译 kernel
e. **-A**：编译 android
f. **-u**：打包生成 update.img 与 update_spi_nvme.img
g. **-o**：编译 OTA 包
h. **-d**：指定 kernel dts

3) 编译 uboot、kernel、android 并打包成 update.img

a. 编译支持 HDMI 4K 显示镜像（默认关闭 LCD）的命令如下所示：

```
test@test:~$ cd RK356X_Android11
test@test:~/ RK356X_Android11$ export BOARD=orangepi3b
test@test:~/ RK356X_Android11$ source build/envsetup.sh
test@test:~/ RK356X_Android11$ lunch rk3566_r-userdebug
test@test:~/ RK356X_Android11$ ./build.sh -AUKu
```

b. 编译支持 LCD 显示镜像（默认关闭 HDMI）的命令如下所示：

```
test@test:~$ cd RK356X_Android11
test@test:~/ RK356X_Android11$ export BOARD=orangepi3b
test@test:~/ RK356X_Android11$ export DUAL_LCD=true
```

```
test@test:~/ RK356X_Android11$ source build/envsetup.sh
test@test:~/ RK356X_Android11$ lunch rk3566_r-userdebug
test@test:~/ RK356X_Android11$ ./build.sh -AUKu
```

4) 编译完成后会打印下面的信息

```
********rkImageMaker ver 2.1********
Generating new image, please wait...
storage is spinor
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
********rkImageMaker ver 2.1********
Merging storage firmware, please wait...
storage count = 2
adding spinor_update.img...ok
adding pcie_update.img...ok
Merging firmware success.
Making update_spi_nvme.img OK.
Make update image ok!
/wspace3/RK3566/RK356X_Android11
```

5) 最终生成的镜像文件会放在 **rockdev/Image-rk3566_r/**目录下。其中 **update.img** 是支持 TF 卡和 eMMC 启动镜像，**update_spi_nvme.img** 是 NVME SSD 启动镜像

```
test@test:~/RK356X_Android11$ cd rockdev/Image-rk3566_r
test@test:~/RK356X_Android11/rockdev/Image-rk3566_r $ ls update*
update.img update_spi_nvme.img
```

10. OpenWRT 源码的编译方法

10.1. 下载 OpenWRT 源码

1) 首先执行下面的命令下载源码

```
test@test:~$ sudo apt update
test@test:~$ sudo apt install -y git
test@test:~$ git clone https://github.com/orangepi-xunlong/openwrt.git -b openwrt-24.10.1
```

2) OpenWRT 代码下载完后会包含下面的文件和文件夹

```
test@test:~/openwrt$ ls
BSDmakefile  Config.in  include  Makefile  README.md  scripts  toolchain
Config  feeds.conf.default  LICENSE  package  rules.mk  target  tools
```

10.2. 编译 OpenWRT 源码

1) 首先安装下面这些依赖软件（目前仅测试了在 Ubuntu22.04 上编译需要安装下列软件，如果在其他版本的系统上编译，请根据出错信息自行安装依赖软件）

```
test@test:~/openwrt$ sudo apt update
test@test:~/openwrt$ sudo apt install -y ack antlr3 asciidoc autoconf   \
automake autopoint binutils bison build-essential   \
bzip2 ccache cmake cpio curl device-tree-compiler fastjar \
flex gawk gettext gcc-multilib g++-multilib git gperf haveged \
help2man intltool libc6-dev-i386 libelf-dev libglib2.0-dev \
libgmp3-dev libltdl-dev libmpc-dev libmpfr-dev \
libncurses5-dev \libncursesw5-dev libreadline-dev libssl-dev \
libtool lrzsz mkisofs msmtp nano ninja-build p7zip p7zip-full \
patch pkgconf python2.7 python3 python3-pyelftools \
libpython3-dev qemu-utils rsync scons squashfs-tools \
subversion swig texinfo uglifyjs upx-ucl unzip \
vim wget xmlto xxd zlib1g-dev
```

2) 然后执行**./scripts/feeds update -a** 和**./scripts/feeds install -a** 下载依赖包

```
test@test:~/openwrt$ ./scripts/feeds update -a
test@test:~/openwrt$ ./scripts/feeds install -a
```

3) 然后选择使用 OrangePi 3B 的配置文件

```
test@test:~/openwrt$ cp configs/orangepi-3b-rk3566_defconfig .config
```

4) 然后执行下面的命令使配置生效

```
test@test:~/openwrt$ make defconfig
```

5) 执行下面的命令开始编译 openwrt 源码

```
test@test:~/openwrt$ make V=s
```

6) 编译完成后，镜像生成所在的路径为：

```
test@test:~/openwrt$ tree -L 1 bin/targets/rockchip/armv8/
bin/targets/rockchip/armv8/
├── config.buildinfo
├── feeds.buildinfo
├── openwrt-rockchip-armv8-xunlong_orangepi-3b-ext4-sysupgrade.img.gz
├── openwrt-rockchip-armv8-xunlong_orangepi-3b.manifest
├── openwrt-rockchip-armv8-xunlong_orangepi-3b-squashfs-sysupgrade.img.gz
├── packages
├── profiles.json
├── sha256sums
└── version.buildinfo

1 directory, 9 files
```

11. 附录

11.1. 用户手册更新历史

版本	日期	更新说明
v1.0	2023-08-17	初始版本
v1.1	2023-08-24	1. Android11 源码的编译方法 2. Android11：USB OTG 模式切换的方法 3. Android11：使用数据线连接 adb 调试的方法
v1.2	2023-08-25	1. 添加购买 PCIe NVMe SSD 的注意说明
v1.3	2023-09-05	1. Orange Pi OS Arch 系统的使用说明
v1.4	2023-09-21	1. Linux：通过 create_ap 创建 WIFI 热点的方法
v1.5	2023-11-08	1. 烧录 Orange Pi OS（OH）镜像到 TF 卡中的方法 2. 烧录 Orange Pi OS（OH）镜像到 eMMC 中的方法 3. Orange Pi OS OH 系统的使用说明 4. 添加 Linux6.6 系统的使用说明
v1.6	2023-11-24	1. wiringOP 硬件 PWM 的使用方法 2. OpenWRT 系统的使用说明 3. OpenWRT 源码的编译方法
v1.7	2024-09-24	1. v2.1 硬件版本手册图片更新
v1.8	2025-05-13	1. 更新 OpenWRT 版本为 24.10.1
V1.9	2025-06-16	1. 增加 NPU 的使用方法

11.2. 镜像更新历史

日期	更新说明
2023-08-17	Orangepi3b_1.0.0_ubuntu_focal_server_linux5.10.160.7z Orangepi3b_1.0.0_ubuntu_jammy_server_linux5.10.160.7z Orangepi3b_1.0.0_debian_bullseye_server_linux5.10.160.7z Orangepi3b_1.0.0_debian_bookworm_server_linux5.10.160.7z Orangepi3b_1.0.0_ubuntu_focal_desktop_xfce_linux5.10.160.7z Orangepi3b_1.0.0_ubuntu_jammy_desktop_xfce_linux5.10.160.7z Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux5.10.160.7z

	Orangepi3b_1.0.0_debian_bookworm_desktop_xfce_linux5.10.160.7z OrangePi3B_RK3566_Android11_v1.0.0.tar.gz OrangePi3B_RK3566_Android11_lcd_v1.0.0.tar.gz OrangePi3B_RK3566_Android11_spi-nvme_v1.0.0.tar.gz OrangePi3B_RK3566_Android11_lcd_spi-nvme_v1.0.0.tar.gz * 初始版本
2023-08-23	Opios-arch-aarch64-xfce-opi3b-23.08-linux5.10.160.img.xz * 初始版本
2023-08-24	OrangePi3B_RK3566_Android11_v1.0.1.tar.gz OrangePi3B_RK3566_Android11_lcd_v1.0.1.tar.gz OrangePi3B_RK3566_Android11_spi-nvme_v1.0.1.tar.gz OrangePi3B_RK3566_Android11_lcd_spi-nvme_v1.0.1.tar.gz * 支持 USB OTG 模式切换功能
2023-08-25	Opios-arch-aarch64-xfce-opi3b-23.08.1-linux5.10.160.img.xz * 解决应用商店无法使用的问题
2023-09-21	Orangepi3b_1.0.2_ubuntu_focal_server_linux5.10.160.7z Orangepi3b_1.0.2_ubuntu_jammy_server_linux5.10.160.7z Orangepi3b_1.0.2_debian_bullseye_server_linux5.10.160.7z Orangepi3b_1.0.2_debian_bookworm_server_linux5.10.160.7z Orangepi3b_1.0.2_ubuntu_focal_desktop_xfce_linux5.10.160.7z Orangepi3b_1.0.2_ubuntu_jammy_desktop_xfce_linux5.10.160.7z Orangepi3b_1.0.2_debian_bullseye_desktop_xfce_linux5.10.160.7z Orangepi3b_1.0.2_debian_bookworm_desktop_xfce_linux5.10.160.7z * 解决 CPU 频率被限制在 1.2GHz 问题 * 添加 rk356x-uart2-m0.dtbo
2023-11-06	Opios-openharmony-4.0-beta1-aarch64-opi3b-23.11-linux5.10.img.tar.gz * 初始版本
2023-11-08	Orangepi3b_1.0.0_ubuntu_jammy_desktop_xfce_linux6.6.0-rc50.7z

	Orangepi3b_1.0.0_debian_bullseye_desktop_xfce_linux6.6.0-rc5.7z Orangepi3b_1.0.0_debian_bookworm_desktop_xfce_linux6.6.0-rc5.7z * 初始版本
2023-11-08	Opios-oh-4.0-beta1-aarch64-opi3b-23.11.1-linux5.10.img.tar.gz * 支持 TF 卡启动
2023-11-24	openwrt-aarch64-opi3b-23.05-linux6.1.62-ext4.img.gz * 初始版本
2025-05-13	openwrt-aarch64-opi3b-24.10.1-linux6.6.86-ext4.img.gz * 初始版本